

# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

## PDF VERSION OF THE NOTES

The PDF version of these notes are autogenerated from the HTML version. There are a few conversion/formatting artifacts that are easy to fix (please feel free to point them out). But there are also interactive elements in the HTML version are not easy to put into the PDF. When possible, I try to provide a link. But I consider the [online HTML version](#) to be the main version.

## TABLE OF CONTENTS

- [Preface](#)
- [Chapter 1: Fully-actuated vs Underactuated Systems](#)
  - [Motivation](#)
    - Honda's ASIMO vs. passive dynamic walkers
    - Birds vs. modern aircraft
    - Manipulation
    - The common theme
  - [Definitions](#)
  - [Feedback Equivalence](#)
  - [Input and State Constraints](#)
    - Nonholonomic constraints
  - [Underactuated robotics](#)
  - [Goals for the course](#)
  - [Exercises](#)
- **[MODEL SYSTEMS](#)**
- [Chapter 2: The Simple Pendulum](#)
  - [Introduction](#)
  - [Nonlinear dynamics with a constant torque](#)
    - The overdamped pendulum
    - The undamped pendulum with zero torque
      - Orbit calculations
    - The undamped pendulum with a constant torque
  - [The torque-limited simple pendulum](#)
    - Energy-shaping control
  - [Exercises](#)
- [Chapter 3: Acrobats, Cart-Poles, and Quadrotors](#)
  - [The Acrobot](#)
    - Equations of motion
  - [The Cart-Pole system](#)
    - Equations of motion
  - [Quadrotors](#)
    - The Planar Quadrotor

- The Full 3D Quadrotor
  - [Balancing](#)
    - Linearizing the manipulator equations
    - Controllability of linear systems
      - The special case of non-repeated eigenvalues
      - A general solution
      - Controllability vs. underactuated
      - Stabilizability of a linear system
    - LQR feedback
  - [Partial feedback linearization](#)
    - PFL for the Cart-Pole System
      - Collocated
      - Non-collocated
    - General form
      - Collocated linearization
      - Non-collocated linearization
      - Task-space partial feedback linearization
  - [Swing-up control](#)
    - Energy shaping
    - Cart-Pole
    - Acrobot
    - Discussion
  - [Other model systems](#)
  - [Exercises](#)
- [Chapter 4: Simple Models of Walking and Running](#)
  - [Limit Cycles](#)
    - Poincaré Maps
  - [Simple Models of Walking](#)
    - The Rimless Wheel
      - Stance Dynamics
      - Foot Collision
      - Forward simulation
      - Poincaré Map
      - Fixed Points and Stability
      - Stability of standing still
    - The Compass Gait
    - The Knead Walker
    - Curved feet
    - And beyond...
  - [Simple Models of Running](#)
    - The Spring-Loaded Inverted Pendulum (SLIP)
      - Analysis on the apex-to-apex map
      - SLIP Control
      - SLIP extensions
    - Hopping robots from the MIT Leg Laboratory
      - The Planar Monopod Hopper
      - Running on four legs as though they were one
    - Towards human-like running
  - [A simple model that can walk and run](#)
  - [Exercises](#)
- [Chapter 5: Highly-articulated Legged Robots](#)
  - [A thought experiment](#)
    - A spacecraft model
    - Robots with (massless) legs
  - [Centroidal dynamics](#)
    - Impact dynamics
    - The special case of flat terrain
      - An aside: the zero-moment point derivation
  - [ZMP-based planning](#)
    - Heuristic footstep planning
    - Planning trajectories for the center of mass
      - The ZMP "Stability" Metric
    - From a CoM plan to a whole-body plan
  - [Whole-Body Control](#)

- [Footstep planning and push recovery](#)
- [Beyond ZMP planning](#)
- [Exercises](#)
- [Chapter 6: Model Systems with Stochasticity](#)
  - [The Master Equation](#)
  - [Stationary Distributions](#)
  - [Extended Example: The Rimless Wheel on Rough Terrain](#)
  - [Noise models for real robots/systems.](#)

## **[NONLINEAR PLANNING AND CONTROL](#)**

- [Chapter 7: Dynamic Programming](#)
  - [Formulating control design as an optimization](#)
    - Additive cost
  - [Optimal control as graph search](#)
  - [Continuous dynamic programming](#)
    - The Hamilton-Jacobi-Bellman Equation
    - Solving for the minimizing control
    - Numerical solutions for  $J^*$ 
      - Value iteration with function approximation
      - Linear function approximators
      - Value iteration on a mesh
      - Neural fitted value iteration
      - Continuous-time systems
  - [Extensions](#)
    - Discounted and average cost formulations
    - Stochastic control for finite MDPs
      - Stochastic interpretation of deterministic, continuous-state value iteration
    - Linear Programming Dynamic Programming
    - Sums-of-Squares Dynamic Programming
  - [Exercises](#)
- [Chapter 8: Linear Quadratic Regulators](#)
  - [Basic Derivation](#)
    - Local stabilization of nonlinear systems
  - [Finite-horizon formulations](#)
    - Finite-horizon LQR
    - Time-varying LQR
    - Local trajectory stabilization for nonlinear systems
    - Linear Quadratic Optimal Tracking
    - Linear Final Boundary Value Problems
  - [Variations and extensions](#)
    - Discrete-time Riccati Equations
    - LQR with input and state constraints
    - LQR on a manifold
    - LQR for linear systems in implicit form
    - LQR as a convex optimization
    - Finite-horizon LQR via least squares
    - Minimum-time LQR
    - Parameterized Riccati Equations
  - [Exercises](#)
  - [Notes](#)
    - Finite-horizon LQR derivation (general form)
- [Chapter 9: Lyapunov Analysis](#)
  - [Lyapunov Functions](#)
    - Global Stability
    - LaSalle's Invariance Principle
    - Relationship to the Hamilton-Jacobi-Bellman equations
    - Lyapunov functions for estimating regions of attraction
    - Robustness analysis using "common Lyapunov functions"
    - Barrier functions
  - [Lyapunov analysis with convex optimization](#)
    - Linear systems
    - Global analysis for polynomial systems
    - Region of attraction estimation for polynomial systems

- The S-procedure
  - Basic region of attraction formulation
  - The equality-constrained formulation
  - Searching for  $V(\mathbf{x})$
  - Convex outer approximations
  - Regions of attraction codes in Drake
  - Robustness analysis using the S-procedure
  - Piecewise-polynomial systems
  - Rigid-body dynamics are (rational) polynomial
    - Linear feedback and quadratic forms
    - Alternatives for obtaining polynomial equations
  - Verifying dynamics in implicit form
- [Finite-time Reachability](#)
  - Time-varying dynamics and Lyapunov functions
  - Finite-time reachability
  - Reachability via Lyapunov functions
- [Control design](#)
  - Control design via alternations
    - Global stability
    - Maximizing the region of attraction
  - State feedback for linear systems
  - Control-Lyapunov Functions
  - Approximate dynamic programming with SOS
    - Upper and lower bounds on cost-to-go
    - Linear Programming Dynamic Programming
    - Sums-of-Squares Dynamic Programming
- [Alternative computational approaches](#)
  - Sampling Quotient-Ring Sum-of-Squares
  - "Satisfiability modulo theories" (SMT)
  - Mixed-integer programming (MIP) formulations
  - Continuation methods
- [Neural Lyapunov functions](#)
- [Contraction metrics](#)
- [Other variations and extensions](#)
- [Exercises](#)
- [Chapter 10: Trajectory Optimization](#)
  - [Problem Formulation](#)
  - [Convex Formulations for Linear Systems](#)
    - Direct Transcription
    - Direct Shooting
    - Computational Considerations
    - Continuous Time
  - [Nonconvex Trajectory Optimization](#)
    - Direct Transcription and Direct Shooting
    - Direct Collocation
    - Pseudo-spectral Methods
      - Dynamic constraints in implicit form
  - [Solution techniques](#)
    - Efficiently computing gradients
    - The special case of direct shooting without state constraints
    - Penalty methods and the Augmented Lagrangian
    - Zero-order optimization
    - Getting good solutions... in practice.
  - [Local Trajectory Feedback Design](#)
    - Finite-horizon LQR
    - Model-Predictive Control
      - Receding-horizon MPC
      - Recursive feasibility
      - MPC and Lyapunov functions
  - [Case Study: A glider that can land on a perch like a bird](#)
    - The Flat-Plate Glider Model
    - Trajectory optimization
    - Trajectory stabilization

- Trajectory funnels
    - Beyond a single trajectory
  - [Pontryagin's Minimum Principle](#)
    - Lagrange multiplier derivation of the adjoint equations
    - Necessary conditions for optimality in continuous time
  - [Variations and Extensions](#)
    - Differential Flatness
    - Iterative LQR and Differential Dynamic Programming
    - Mixed-integer convex optimization for non-convex constraints
    - Explicit model-predictive control
  - [Exercises](#)
- [Chapter 11: Policy Search](#)
  - [Problem formulation](#)
  - [Linear Quadratic Regulator](#)
    - Policy Evaluation
    - A nonconvex objective in  $\mathbf{K}$
    - No local minima
    - True gradient descent
  - [More convergence results and counter-examples](#)
  - [Trajectory-based policy search](#)
    - Infinite-horizon objectives
    - Search strategies for global optimization
  - [Policy Iteration](#)
- [Chapter 12: Motion Planning as Search](#)
  - [Artificial Intelligence as Search](#)
  - [Sampling-based motion planning](#)
    - Rapidly-Exploring Random Trees (RRTs)
    - RRTs for robots with dynamics
    - Variations and extensions
    - Discussion
  - [Decomposition methods](#)
  - [Planning as Combinatorial + Continuous Optimization](#)
    - Motion Planning on Graphs of Convex Sets (GCS)
  - [Exercises](#)
- [Chapter 13: Feedback Motion Planning](#)
- [Chapter 14: Robust and Stochastic Control](#)
  - [Stochastic models](#)
  - [Costs and constraints for stochastic systems](#)
  - [Finite Markov Decision Processes](#)
  - [Linear optimal control](#)
    - Stochastic LQR
    - $L_2$  gain
      - Dissipation inequalities
      - Small-gain theorem
      - Model uncertainty as a special case.
    - Robust LQR as  $\mathcal{H}_\infty$
    - Linear Exponential-Quadratic Gaussian (LEQG)
    - Adaptive control
    - Structured uncertainty
    - Linear parameter-varying (LPV) control
  - [Trajectory optimization](#)
    - Monte-carlo trajectory optimization
    - Iterative  $\mathcal{H}_2$ /iLQG
    - Finite-time (reachability) analysis
  - [Nonlinear analysis and control](#)
  - [Domain randomization](#)
  - [Extensions](#)
    - Alternative risk/robustness metrics
- [Chapter 15: Output Feedback \(aka Pixels-to-Torques\)](#)
  - [Background](#)
    - The classical perspective
    - From pixels to torques
  - [Static Output Feedback](#)

- A hardness result
  - Via policy search
- Observer-based Feedback
  - Luenberger Observer
  - Linear Quadratic Regulator w/ Gaussian Noise (LQG)
  - Partially-observable Markov Decision Processes
  - Trajectory optimization with Iterative LQG
- Disturbance-based feedback
  - System-Level Synthesis
- Optimizing dynamic policies
  - Convex reparameterizations of  $H_2$ ,  $H_\infty$ , and LQG
  - Policy gradient for LQG
  - Sums-of-squares alternations
  - Teacher-student learning
- Feedback from pixels
  - Visuomotor policies
- Chapter 16: Algorithms for Limit Cycles
  - Trajectory optimization
  - Lyapunov analysis
    - Transverse coordinates
    - Transverse linearization
    - Region of attraction estimation using sums-of-squares
  - Feedback design
    - For underactuation degree one.
    - Transverse LQR
    - Orbital stabilization for non-periodic trajectories
- Chapter 17: Planning and Control through Contact
  - (Autonomous) Hybrid Systems
    - Hybrid trajectory optimization
      - Given a fixed mode sequence
      - Direct shooting
    - Deriving hybrid models: minimal vs floating-base coordinates
    - Discrete control (between events)
    - Hybrid LQR
    - Hybrid Lyapunov analysis
  - Contact-implicit trajectory optimization
  - Exercises

## ESTIMATION AND LEARNING

- Chapter 18: System Identification
  - Problem formulation
    - Equation error vs simulation error
    - Online optimization
    - Learning models for control
  - Parameter Identification for Mechanical Systems
    - Kinematic parameters and calibration
    - Estimating inertial parameters (and friction)
    - Simultaneous kinematic and inertial identification via lumped parameters.
    - Identification using energy instead of inverse dynamics.
    - Residual physics models with linear function approximators
    - Experiment design as a trajectory optimization
    - Online estimation and adaptive control
    - Identification with contact
  - Identifying (time-domain) linear dynamical systems
    - From state observations
      - Model-based Iterative Learning Control (ILC)
      - Compression using the dominant eigenmodes
      - Linear dynamics in a nonlinear basis
    - From input-output data (the state-realization problem)
    - Adding stability constraints
    - Autoregressive models

- Statistical analysis of learning linear models
- [Identification of finite \(PO\)MDPs](#)
  - From state observations
  - Identifying Hidden Markov Models (HMMs)
- [Neural network models](#)
  - Generating training data
  - From state observations
  - State-space models from input-output data (recurrent networks)
  - Input-output (autoregressive) models
  - Particle-based models
  - Object-centric models
  - Modeling stochasticity
  - Control design for neural network models
- [Alternatives for nonlinear system identification](#)
- [Identification of hybrid systems](#)
- [Task-relevant models](#)
- [Exercises](#)
- [Chapter 19: State Estimation](#)
  - [Observers and the Kalman Filter](#)
  - [Recursive Bayesian Filters](#)
  - [Smoothing](#)
- [Chapter 20: Model-Free Policy Search](#)
  - [Policy Gradient Methods](#)
    - The Likelihood Ratio Method (aka REINFORCE)
    - Sample efficiency
    - Stochastic Gradient Descent
    - The Weight Perturbation Algorithm
    - Weight Perturbation with an Estimated Baseline
    - REINFORCE w/ additive Gaussian noise
    - Summary
  - [Sample performance via the signal-to-noise ratio.](#)
    - Performance of Weight Perturbation

## [APPENDIX](#)

- [Appendix A: Drake](#)
  - [Pydrake](#)
  - [Online Jupyter Notebooks](#)
    - Running on Deepnote
    - Running on Google Colab
    - Enabling licensed solvers
  - [Running on your own machine](#)
  - [Getting help](#)
- [Appendix B: Multi-Body Dynamics](#)
  - [Deriving the equations of motion](#)
  - [The Manipulator Equations](#)
    - Recursive Dynamics Algorithms
    - Bilateral Position Constraints
    - Bilateral Velocity Constraints
    - Hybrid models via constraint forces
  - [The Dynamics of Contact](#)
    - Compliant Contact Models
    - Rigid Contact with Event Detection
      - Impulsive Collisions
      - Putting it all together
    - Time-stepping Approximations for Rigid Contact
      - Complementarity formulations
      - Anitescu's convex formulation
      - Todorov's regularization
      - The Semi-Analytic Primal (SAP) solver
      - Beyond Point Contact
  - [Variational mechanics](#)
    - Virtual work
    - D'Alembert's principle and the force of inertia

- Principle of Stationary Action
  - Hamiltonian Mechanics
- [Exercises](#)
- [Appendix C: Optimization and Mathematical Programming](#)
  - [Optimization software](#)
  - [General concepts](#)
    - Convex vs nonconvex optimization
    - Constrained optimization with Lagrange multipliers
  - [Convex optimization](#)
    - Linear Programs/Quadratic Programs/Second-Order Cones
    - Semidefinite Programming and Linear Matrix Inequalities
      - Semidefinite programming relaxation of general quadratic optimization
    - Sums-of-squares optimization
      - Sums of squares on a Semi-Algebraic Set
      - Sums of squares optimization on an Algebraic Variety
      - DSOS and SDSOS
    - Solution techniques
  - [Nonlinear programming](#)
    - Second-order methods (SQP / Interior-Point)
    - First-order methods (SGD / ADMM)
      - Penalty methods
      - Projected Gradient Descent
    - Zero-order methods (CMA)
    - Example: Inverse Kinematics
  - [Combinatorial optimization](#)
    - Search, SAT, First order logic, SMT solvers, LP interpretation
    - Mixed-integer convex optimization
  - ["Black-box" optimization](#)
- [Appendix D: An Optimization Playbook](#)
  - [Matrices](#)
  - [Ellipsoids](#)
  - [Polytopes](#)
  - [\(Mixed-\)Integer Programming](#)
  - [Bilinear Matrix Inequalities \(BMIs\)](#)
  - [Geometry \(SE\(3\), Penetration, and Contact\)](#)
- [Appendix E: Miscellaneous](#)
  - [How to cite these notes](#)
  - [Annotation tool etiquette](#)
  - [Some great final projects](#)
  - [Please give me feedback!](#)

You can find documentation for the source code supporting these notes [here](#).

## PREFACE

This book is about nonlinear dynamics and control, with a focus on mechanical systems. I've spent my career thinking about how to make robots move robustly, but also with speed, efficiency, and grace. I believe that this is best achieved through a tight coupling between mechanical design, passive dynamics, and nonlinear control synthesis. These notes contain selected material from dynamical systems theory, as well as linear and nonlinear control. But the dynamics of our robots quickly get too complex for us to handle with a pencil-and-paper approach. As a result, the primary focus of these notes is on computational approaches to control design, especially using optimization and machine learning.

When I started teaching this class, and writing these notes, the computational approach to control was far from mainstream in robotics. I had just finished my Ph.D. focused on reinforcement learning (applied to a bipedal robot), and was working on optimization-based motion planning. I remember sitting at a robotics



conference dinner as a young faculty, surrounded by people I admired, talking about optimization. One of the senior faculty said "Russ: the people that talk like you aren't the people that get real robots to work." Wow, have things changed. Now almost every advanced robot is using optimization or learning in the planning/control system.

Today, the conversations about reinforcement learning (RL) are loud and passionate enough to drown out almost every other conversation in the room. Ironically, now I am the older professor and I find myself still believing in RL, but not with the complete faith of my youth. There is so much one can understand about the structure of the equations that govern our mechanical systems; algorithms which don't make use of that structure are missing obvious opportunities for data efficiency and robustness. The dream is to make the learning algorithms discover this structure on their own; but even then it pays for you (the designer) to understand the optimization landscape the learning systems are operating on. That's why my goal for this course is to help *you* discover this structure, and to learn how to use this structure to develop stronger algorithms and to guide your scientific endeavors into learning-based control.

I'll go even further. I'm willing to bet that our views of intelligence in 10-20 years will look less like feedforward networks with a training mode and a test mode, and more like a *system* with dynamics that ebb and flow in a beautiful dance with streams of incoming data and the ever-changing dynamics of the environment. These systems will move more flexibly between perception, forward prediction / sequential decision making, storing and retrieving long-term memories, and taking action. Dynamical systems theory offers us a way to understand and harness the complexity of these systems that we are building.

Although the material in the book comes from many sources, the presentation is targeted very specifically at a handful of robotics problems. Concepts are introduced only when and if they can help progress the capabilities we are trying to develop. Many of the disciplines that I am drawing from are traditionally very rigorous, to the point where the basic ideas can be hard to penetrate for someone that is new to the field. I've made a conscious effort in these notes to keep a very informal, conversational tone even when introducing these rigorous topics, and to reference the most powerful theorems but only to prove them when that proof would add particular insights without distracting from the mainstream presentation. I hope that the result is a broad but reasonably self-contained and readable manuscript that will be of use to any enthusiastic roboticist.

## ORGANIZATION

The material in these notes is organized into a few main parts. "Model Systems" introduces a series of increasingly complex dynamical systems and overviews some of the relevant results from the literature for each system. "Nonlinear Planning and Control" introduces quite general computational algorithms for reasoning about those dynamical systems, with optimization theory playing a central role. Many of these algorithms treat the dynamical system as known and deterministic until the last chapters in this part which introduce stochasticity and robustness. "Estimation and Learning" follows this up with techniques from statistics and machine learning which capitalize on this viewpoint to introduce additional algorithms which can operate with less assumptions on knowing the model or having perfect sensors. The book closes with an "Appendix" that provides slightly more background (and references) for the main topics used in the course.

The order of the chapters was chosen to make the book valuable as a reference. When teaching the course, however, I take a spiral trajectory through the material, introducing robot dynamics and control problems one at a time, and introducing only the techniques that are required to solve that particular problem.

## SOFTWARE

All of the examples and algorithms in this book, plus many more, are now available as a part of our open-source software project: [DRAKE](#). [DRAKE](#) is a C++ project, but in this text we will use Drake's [Python bindings](#). I encourage super-users or readers who want to dig deeper to explore the C++ code as well (and to contribute back).

Please see the [appendix](#) for specific instructions for using [DRAKE](#) along with these notes.

[First chapter](#)

# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 1

# Fully-actuated VS Underactuated Systems

Many robots today move far too conservatively, and accomplish only a fraction of the tasks and achieve a fraction of the performance that they are mechanically capable of. In some cases, we are still fundamentally limited by control technology which matured on rigid robotic arms in structured factory environments, where large actuators could be used to "shape" the dynamics of the machine to achieve precision and repeatability. The study of underactuated robotics focuses on building control systems which instead exploit the natural dynamics of the machines in an attempt to achieve extraordinary performance in terms of speed, efficiency, or robustness.

In the last few years, controllers designed using machine learning have showcased the potential power of optimization-based control, but these methods do not yet enjoy the sample-efficiency nor algorithmic reliability that we've come to expect from more mature control technologies. The study of underactuated robotics asks us to look closely at the optimization landscapes that occur when we are optimizing mechanical systems, to understand and exploit that structure in our optimization and learning algorithms.

### 1.1 MOTIVATION

Let's start with some examples, and some videos.

#### 1.1.1 Honda's ASIMO vs. passive dynamic walkers

The world of robotics changed when, in late 1996, Honda Motor Co. announced that they had been working for nearly 15 years (behind closed doors) on walking robot technology. Their designs have continued to evolve, resulting in a humanoid robot they call ASIMO (Advanced Step in Innovative MObility). For nearly 20 years, Honda's robots were widely considered to represent the state of the art in walking robots, although there are now many robots with designs and performance very similar to ASIMO's. We will dedicate effort to understanding a few of the details of ASIMO when we discuss algorithms for walking... for now I just want you to become

familiar with the look and feel of ASIMO's movements [watch the asimo video below now].

Figure 1.1 - Honda's ASIMO (from <http://world.honda.com/ASIMO/video/>)

I hope that your first reaction is to be incredibly impressed with the quality and versatility of ASIMO's movements. Now take a second look. Although the motions are very smooth, there is something a little unnatural about ASIMO's gait. It feels a little like an astronaut encumbered by a heavy space suit. In fact this is a reasonable analogy... ASIMO is walking a bit like somebody that is unfamiliar with his/her dynamics. Its control system is using high-gain feedback, and therefore considerable joint torque, to cancel out the natural dynamics of the machine and strictly follow a desired trajectory. This control approach comes with a stiff penalty. ASIMO uses roughly 20 times the energy (scaled) that a human uses to walk on the flat (measured by cost of transport)[1]. Also, control stabilization in this approach only works in a relatively small portion of the state space (when the stance foot is flat on the ground), so ASIMO can't move nearly as quickly as a human, and cannot walk on unmodelled or uneven terrain.

Figure 1.2 - A 3D passive dynamic walker by Steve Collins, Martijn Wisse, and Andy Ruina[2].

For contrast, let's now consider a very different type of walking robot, called a passive dynamic walker (PDW). This "robot" has no motors, no controllers, no computer, but is still capable of walking stably down a small ramp, powered only by gravity [watch videos above now]. Most people will agree that the passive gait of this machine is more natural than ASIMO's; it is certainly more efficient. Passive walking machines have a long history - there are patents for passively walking toys dating back to the mid 1800's. We will discuss, in detail, what people know about the dynamics of these machines and what has been accomplished experimentally. This most impressive passive dynamic walker to date was built by Steve Collins in Andy Ruina's lab at Cornell[2].

Passive walkers demonstrate that the high-gain, dynamics-cancelling feedback approach taken on ASIMO is not a necessary one. In fact, the dynamics of walking is beautiful, and should be exploited - not cancelled out.

The world is just starting to see what this vision could look like. This video from Boston Dynamics is one of my favorites of all time:

Figure 1.3 - Boston Dynamics' Atlas robot does a backflip. Make sure you've seen the [dancing video](#), too.

This result is a marvel of engineering (the mechanical design alone is amazing...). In this class, we'll teach you the computational tools required to make robots perform this way. We'll also try to reason about how robust these types of maneuvers are and can be. Don't worry: if you do not have a super lightweight, super capable, and super durable humanoid, then a simulation will be provided for you.

### 1.1.2 Birds vs. modern aircraft

The story is surprisingly similar in a very different type of machine. Modern airplanes are extremely effective for steady-level flight in still air. Propellers produce thrust very efficiently, and today's cambered airfoils are highly optimized for speed and/or efficiency. It would be easy to convince yourself that we have nothing left to learn from birds. But, like ASIMO, these machines are mostly confined to a very conservative, low angle-of-attack flight regime where the aerodynamics on the wing are well understood. Birds routinely execute maneuvers outside of this flight envelope (for instance, when they are landing on a perch), and are considerably more effective than our best aircraft at exploiting energy (eg, wind) in the air.

As a consequence, birds are extremely efficient flying machines; some are capable of migrating thousands of kilometers with incredibly small fuel supplies. The wandering albatross can fly for hours, or even days, without flapping its wings - these birds exploit the shear layer formed by the wind over the ocean surface in a technique called dynamic soaring. Remarkably, the metabolic cost of flying for these birds is indistinguishable from the baseline metabolic cost[3], suggesting that they can travel incredible distances (upwind or downwind) powered almost completely by gradients in the wind. Other birds achieve efficiency through similarly rich interactions with the air - including formation flying, thermal soaring, and ridge soaring. Small birds and large insects, such as butterflies and locusts, use "gust soaring" to migrate hundreds or even thousands of kilometers carried primarily by the wind.

Birds are also incredibly maneuverable. The roll rate of a highly acrobatic aircraft (e.g, the A-4 Skyhawk) is approximately 720 deg/sec[4]; a barn swallow has a roll rate in excess of 5000 deg/sec[4]. Bats can be flying at full-speed in one direction, and completely reverse direction while maintaining forward speed, all in just over 2 wing-beats and in a distance less than half the wingspan[5]. Although quantitative flow visualization data from maneuvering flight is scarce, a dominant theory is that

the ability of these animals to produce sudden, large forces for maneuverability can be attributed to unsteady aerodynamics, e.g., the animal creates a large suction vortex to rapidly change direction[6]. These astonishing capabilities are called upon routinely in maneuvers like flared perching, prey-catching, and high speed flying through forests and caves. Even at high speeds and high turn rates, these animals are capable of incredible agility - bats sometimes capture prey on their wings, Peregrine falcons can pull 25 G's out of a 240 mph dive to catch a sparrow in mid-flight[7], and even the small birds outside our building can be seen diving through a chain-link fence to grab a bite of food. In the last years, we are starting to see our unmanned aerial vehicles capitalize on their potential. Quadrotors, though not efficient, have been leading the way in terms of acrobatics, and a learning-based control system recently reached champion-level in drone racing [8].

Although many impressive statistics about avian flight have been recorded, our understanding is partially limited by experimental accessibility - it is quite difficult to carefully measure birds (and the surrounding airflow) during their most impressive maneuvers without disturbing them. The dynamics of a swimming fish are closely related, and can be more convenient to study. Dolphins have been known to swim gracefully through the waves alongside ships moving at 20 knots[6]. Smaller fish, such as the bluegill sunfish, are known to possess an escape response in which they propel themselves to full speed from rest in less than a body length; flow visualizations indeed confirm that this is accomplished by creating a large suction vortex along the side of the body[9] - similar to how bats change direction in less than a body length. There are even observations of a dead fish swimming upstream by pulling energy out of the wake of a cylinder; this passive propulsion is presumably part of the technique used by rainbow trout to swim upstream at mating season[10].

### 1.1.3 Manipulation

Despite a long history of success in industrial applications, and the huge potential for consumer applications, we still don't have robot arms that can perform any meaningful tasks in the home. Admittedly, the perception problem (using sensors to detect/localize objects and understand the scene) for home robotics is incredibly difficult. But even if we were given a perfect perception system, our robots are still a long way from performing basic object manipulation tasks with the dexterity and versatility of a human.

Most robots that perform object manipulation today use a stereotypical pipeline. First, we enumerate a handful of contact locations on the hand (these points, and only these points, are allowed to contact the world). Then, given a localized object in the environment, we plan a collision-free trajectory for the arm that will move the hand into a "pre-grasp" location. At this point the robot closes it's eyes (figuratively) and closes the hand, hoping that the pre-grasp location was good enough that the object will be successfully grasped using e.g. only current feedback in the fingers to know when to stop closing. "Underactuated hands" make this approach more successful, but the entire approach really only works well for enveloping grasps.

The enveloping grasps approach may actually be sufficient for a number of simple pick-and-place tasks, but it is a very poor representation of how humans do manipulation. When humans manipulate objects, the contact interactions with the object and the world are very rich -- we often use pieces of the environment as fixtures to reduce uncertainty, we commonly *exploit* slipping behaviors (e.g. for picking things up, or reorienting it in the hand), and our brains don't throw NaNs if we use the entire surface of our arms to e.g. manipulate a large object.

In the last few years, the massive progress in computer vision has completely opened up this space. I've begun to focus my own research to problems in the manipulation domain. In manipulation, the interaction between dynamics and perception is incredibly rich. As a result, I've started an entirely separate set of notes (and a second course) on manipulation. That field, in particular, feels like it is on the precipice: it seems clear that very soon we will have some form of "foundation model" [11] for manipulation; these likely won't solve the entire problem but will bring an entirely new "common sense" capability to the problem of open-world

manipulation.

By the way, in most cases, if the robots fail to make contact at the anticipated contact times/locations, bad things can happen. The results are hilarious and depressing at the same time. (Let's fix that!)

### 1.1.4 The common theme

Classical control techniques for robotics are based on the idea that feedback control can be used to override the dynamics of our machines. In contrast, the examples I've given above suggest that to achieve outstanding dynamic performance (efficiency, agility, and robustness) from our robots, we need to understand how to design control systems which take advantage of the dynamics, not cancel them out. That is the topic of this course.

Surprisingly, many formal control ideas that developed in robotics do not support the idea of "exploiting" the dynamics. Optimal control formulations (which we will study in depth) allow it in principle, but optimal control of nonlinear systems remains a challenging problem. Back when I started these notes, I used to joke that in order to convince a robotics control researcher to consider the dynamics, you have to do something drastic like taking away her control authority - remove a motor, or enforce a torque-limit. Systems that are interesting in this way are called the "underactuated" systems. It is in this field of "underactuated robotics" where research on the type of control I am advocating for began.

## 1.2 DEFINITIONS

According to Newton, the dynamics of mechanical systems are second order ( $F = ma$ ). Their state is given by a vector of positions,  $\mathbf{q}$  (also known as the configuration vector), and a vector of velocities,  $\dot{\mathbf{q}}$ , and (possibly) time. The general form for a second-order control dynamical system is:

$$\ddot{\mathbf{q}} = \mathbf{f}(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{u}, t),$$

where  $\mathbf{u}$  is the control vector.

**Definition 1.1 (Underactuated Control Differential Equations)** A second-order control differential equation described by the equations

$$\ddot{\mathbf{q}} = \mathbf{f}(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{u}, t) \quad (1)$$

is *fully actuated* in state  $\mathbf{x} = (\mathbf{q}, \dot{\mathbf{q}})$  and time  $t$  if the resulting map  $\mathbf{f}$  is surjective: for every  $\ddot{\mathbf{q}}$  there exists a  $\mathbf{u}$  which produces the desired response. Otherwise it is *underactuated* (at state  $\mathbf{x}$  at time  $t$ ).

This definition can also be extended to discrete-time systems and/or differential inclusions.

As we will see, the dynamics for many of the robots that we care about turn out to be affine in commanded torque (if  $\mathbf{q}$ ,  $\dot{\mathbf{q}}$ , and  $t$  are fixed, then the dynamics are a linear function of  $\mathbf{u}$  plus a constant), so let's consider a slightly constrained form:

$$\ddot{\mathbf{q}} = \mathbf{f}_1(\mathbf{q}, \dot{\mathbf{q}}, t) + \mathbf{f}_2(\mathbf{q}, \dot{\mathbf{q}}, t)\mathbf{u}. \quad (2)$$

For a control dynamical system described by equation (2), if we have

$$\text{rank} [\mathbf{f}_2(\mathbf{q}, \dot{\mathbf{q}}, t)] < \dim [\mathbf{q}], \quad (3)$$

then the system is underactuated at  $(\mathbf{q}, \dot{\mathbf{q}}, t)$ . The implication is only in one direction, though -- sometimes we will write equations that look like (2) and have a full rank  $\mathbf{f}_2$  but additional constraints like  $|\mathbf{u}| \leq 1$  can also make a system underactuated.

Note also that we are using the word *system* here to describe the mathematical



model (potentially of a physical robot). When we say that the *system* is underactuated, we are talking about the mathematical model. Imagine a two-link robot with two actuators, a typical model with rigid links could be fully actuated, but if we add extra degrees of freedom to model small amounts of flexibility in the links then that system model could be underactuated. These two models describe the same robot, but at different levels of fidelity. Two actuators might be enough to completely control the joint angles, but not the joint angles *and* the flexing modes of the link.

Notice that whether or not a control system is underactuated may depend on the state of the system or even on time, although for most systems (including all of the systems in this book) underactuation is a global property of the model. We will refer to a model as underactuated if it is underactuated in *all* states and times. In practice, we often refer informally to systems as fully actuated as long as they are fully actuated in *most* states (e.g., a "fully-actuated" system might still have joint limits or lose rank at a kinematic singularity). Admittedly, this permits the existence of a gray area, where it might feel awkward to describe the *model* as either fully- or underactuated (we should instead only describe its states); even powerful robot arms on factory floors do have actuator limits, but we can typically design controllers for them as though they were fully actuated. The primary interest of this text is in models for which the underactuation is useful/necessary for developing a control strategy.

### Example 1.1 (Robot Manipulators)

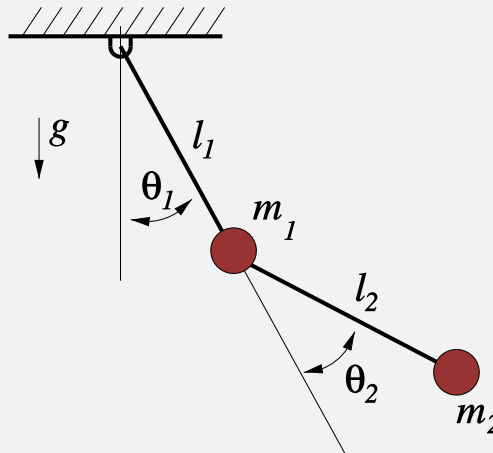


Figure 1.4 - Simple double pendulum. [Click here for an animation.](#)

Consider the simple robot manipulator illustrated above. As described in the [Appendix](#), the equations of motion for this system are quite simple to derive, and take the form of the standard "manipulator equations":

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u}.$$

It is well known that the inertia matrix,  $\mathbf{M}(\mathbf{q})$  is (always) uniformly symmetric and positive definite, and is therefore invertible. Putting the system into the form of equation 2 yields:

$$\ddot{\mathbf{q}} = \mathbf{M}^{-1}(\mathbf{q}) [\tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}].$$

Because  $\mathbf{M}^{-1}(\mathbf{q})$  is always full rank, we find that a system described by the manipulator equations is fully actuated if and only if  $\mathbf{B}$  is full row rank. For this particular example,  $\mathbf{q} = [\theta_1, \theta_2]^T$  and  $\mathbf{u} = [\tau_1, \tau_2]^T$  (motor torques at the joints), and  $\mathbf{B} = \mathbf{I}_{2 \times 2}$ . The system is fully actuated.

#### Python Example

I personally learn best when I can experiment and get some physical intuition.

Most chapters in these notes have an associated Jupyter notebook that can run on Deepnote; this chapter's notebook makes it easy for you to see this system in action.

Try it out! You'll see how to simulate the double pendulum, and even how to inspect the dynamics symbolically.

Note: You can also run the code on your own machines (see the [Appendix](#) for details).

While the basic double pendulum is fully actuated, imagine the somewhat bizarre case that we have a motor to provide torque at the elbow, but no motor at the shoulder. In this case, we have  $\mathbf{u} = \tau_2$ , and  $\mathbf{B}(\mathbf{q}) = [0, 1]^T$ . This system is clearly underactuated. While it may sound like a contrived example, it turns out that it is almost exactly the dynamics we will use to study as our simplest model of walking later in the class.

The matrix  $\mathbf{f}_2$  in equation 2 always has  $\dim[\mathbf{q}]$  rows, and  $\dim[\mathbf{u}]$  columns. Therefore, as in the example, one of the most common cases for underactuation, which trivially implies that  $\mathbf{f}_2$  is not full row rank, is  $\dim[\mathbf{u}] < \dim[\mathbf{q}]$ . This is the case when a robot has joints with no motors. But this is not the only case. The human body, for instance, has an incredible number of actuators (muscles), and in many cases has multiple muscles per joint; despite having more actuators than position variables, when I jump through the air, there is no combination of muscle inputs that can change the ballistic trajectory of my center of mass (barring aerodynamic effects). My control system is underactuated.

A quick note about notation. When describing the dynamics of rigid-body systems in this class, I will use  $\mathbf{q}$  for configurations (positions),  $\dot{\mathbf{q}}$  for velocities, and use  $\mathbf{x}$  for the full state ( $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$ ). There is an important limitation to this convention (3D angular velocity should not be represented as the derivative of 3D pose) described in the Appendix, but it will keep the notes cleaner. Unless otherwise noted, vectors are always treated as column vectors. Vectors and matrices are bold (scalars are not).

## 1.3 FEEDBACK EQUIVALENCE

Fully-actuated systems are dramatically easier to control than underactuated systems. The key observation is that, for fully-actuated systems with known dynamics (e.g.,  $\mathbf{f}_1$  and  $\mathbf{f}_2$  are known for a second-order control-affine system), it is possible to use feedback to effectively change an arbitrary control problem into the problem of controlling a trivial linear system.

When  $\mathbf{f}_2$  is full row rank, it has a right inverse. Consider the potentially nonlinear feedback control:

$$\mathbf{u} = \pi(\mathbf{q}, \dot{\mathbf{q}}, t) = \mathbf{f}_2^{-1}(\mathbf{q}, \dot{\mathbf{q}}, t) [\mathbf{u}' - \mathbf{f}_1(\mathbf{q}, \dot{\mathbf{q}}, t)],$$

where  $\mathbf{u}'$  is the new control input (an input to your controller). Applying this feedback controller to equation 2 results in the linear, decoupled, second-order system:

$$\ddot{\mathbf{q}} = \mathbf{u}'.$$

In other words, if  $\mathbf{f}_1$  and  $\mathbf{f}_2$  are known and  $\mathbf{f}_2$  is invertible, then we say that the system is "feedback equivalent" to  $\ddot{\mathbf{q}} = \mathbf{u}'$ . There are a number of strong results which generalize this idea to the case where  $\mathbf{f}_1$  and  $\mathbf{f}_2$  are estimated, rather than known (e.g, [12]).



square, for instance you have multiple actuators per joint, then this inverse may not be unique.

## Example 1.2 (Feedback Cancellation on the Double Pendulum)

Let's say that we would like our simple double pendulum to act like a simple single pendulum (with damping), whose dynamics are given by:

$$\begin{aligned}\ddot{\theta}_1 &= -\frac{g}{l}\sin\theta_1 - b\dot{\theta}_1 \\ \ddot{\theta}_2 &= 0.\end{aligned}$$

This is easily achieved using

$$\mathbf{u} = \mathbf{B}^{-1} \left[ \mathbf{C}\dot{\mathbf{q}} - \boldsymbol{\tau}_g + \mathbf{M} \begin{bmatrix} -\frac{g}{l}s_1 - b\dot{q}_1 \\ 0 \end{bmatrix} \right].$$

Since we are embedding a nonlinear dynamics (not a linear one), we refer to this as "feedback cancellation", or "dynamic inversion". This idea reveals why I say control is easy -- for the special case of a fully-actuated deterministic system with known dynamics. For example, it would have been just as easy for me to invert gravity. Observe that the control derivations here would not have been any more difficult if the robot had 100 joints.

You can run these examples in the notebook:

As always, I highly recommend that you take a few minutes to read through the source code.

Note that our chosen dynamics do not actually stabilize  $\theta_2$  - this detail was left out for clarity, but would be necessary for any real implementation.

Fully-actuated systems are feedback equivalent to  $\ddot{\mathbf{q}} = \mathbf{u}$ , whereas *underactuated systems are not feedback equivalent to  $\ddot{\mathbf{q}} = \mathbf{u}$* . Therefore, unlike fully-actuated systems, the control designer has no choice but to reason about the more complex dynamics of the plant in the control design. When these dynamics are nonlinear, this can dramatically complicate feedback controller design.

A related concept is feedback linearization. The feedback-cancellation controller in the example above is an example of feedback linearization -- using feedback to convert a nonlinear system into a controllable linear system. Asking whether or not a system is "feedback linearizable" is not the same as asking whether it is underactuated; even a controllable linear system can be underactuated, as we will discuss soon.

Perhaps you are coming with a background in optimization or machine learning, rather than linear controls? To you, I could say that for fully-actuated systems, there is a straight-forward change of variables that can make the optimization landscape (for many control performance objectives) convex. Optimization/learning for these systems is relatively easy. For underactuated systems, we can still aim to leverage the mechanics to improve the optimization landscape but it requires more insights. Developing those insights is one of the themes in these notes, and continues to be an active area of research.

## 1.4 INPUT AND STATE CONSTRAINTS

Although the dynamic constraints due to missing actuators certainly embody the spirit of this course, many of the systems we care about could be subject to other dynamic constraints as well. For example, the actuators on our machines may only be mechanically capable of producing some limited amount of torque, or there may be a physical obstacle in the free space with which we cannot permit our robot to come into contact with.

**Definition 1.2 (Input and State Constraints)** A dynamical

system described by  $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, t)$  may be subject to one or more constraints described by  $\phi(\mathbf{x}, \mathbf{u}, t) \geq 0$ .

In practice it can be useful to separate out constraints which depend only on the input, e.g.  $\phi(\mathbf{u}) \geq 0$ , such as actuator limits, as they can often be easier to handle than state constraints. An obstacle in the environment might manifest itself as one or more constraints that depend only on position, e.g.  $\phi(\mathbf{q}) \geq 0$ .

By our generalized definition of underactuation, we can see that input constraints can certainly cause a system to be underactuated. Position equality constraints are more subtle -- in general these actually reduce the dimensionality of the state space, therefore requiring less dimensions of actuation to achieve "full" control, but we only reap the benefits if we are able to perform the control design in the "minimal coordinates" (which is often difficult).

### Example 1.3 (Input limits)

Consider the constrained second-order linear system

$$\ddot{x} = u, \quad |u| \leq 1.$$

By our definition, this system is underactuated. For example, there is no  $u$  which can produce the acceleration  $\ddot{x} = 2$ .

Input and state constraints can complicate control design in similar ways to having an insufficient number of actuators, (i.e., further limiting the set of the feasible trajectories), and often require similar tools to find a control solution.

## 1.4.1 Nonholonomic constraints

You might have heard of the term "nonholonomic system" (see e.g. [13]), and be thinking about how nonholonomy relates to underactuation. Briefly, a nonholonomic constraint is a constraint of the form  $\phi(\mathbf{q}, \dot{\mathbf{q}}, t) = 0$ , which cannot be integrated into a constraint of the form  $\phi(\mathbf{q}, t) = 0$  (a holonomic constraint). A nonholonomic constraint does not restrain the possible configurations of the system, but rather the manner in which those configurations can be reached. While a holonomic constraint reduces the number of degrees of freedom of a system by one, a nonholonomic constraint does not. An automobile or traditional wheeled robot provides a canonical example:

### Example 1.4 (Wheeled robot)

Consider a simple model of a wheeled robot whose configuration is described by its Cartesian position  $x, y$  and its orientation,  $\theta$ , so  $\mathbf{q} = [x, y, \theta]^T$ . The system is subject to a differential constraint that prevents side-slip,

$$\begin{aligned}\dot{x} &= v \cos \theta \\ \dot{y} &= v \sin \theta \\ v &= \sqrt{\dot{x}^2 + \dot{y}^2}\end{aligned}$$

or equivalently,

$$\dot{y} \cos \theta - \dot{x} \sin \theta = 0.$$

This constraint cannot be integrated into a constraint on configuration—the car can get to any configuration  $(x, y, \theta)$ , it just can't move directly sideways—so this is a nonholonomic constraint.

Contrast the wheeled robot example with a robot on train tracks. The train tracks correspond to a holonomic constraint: the track constraint can be written directly in terms of the configuration  $\mathbf{q}$  of the system, without using the velocity  $\dot{\mathbf{q}}$ . Even

though the track constraint could also be written as a differential constraint on the velocity, it would be possible to integrate this constraint to obtain a constraint on configuration. The track restrains the possible configurations of the system.

A nonholonomic constraint like the no-side-slip constraint on the wheeled vehicle certainly results in an underactuated system. The converse is not necessarily true—the double pendulum system which is missing an actuator is underactuated but would not typically be called a nonholonomic system. Note that the Lagrangian equations of motion are a constraint of the form

$$\phi(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}, \mathbf{u}, t) = 0,$$

so do not qualify as a nonholonomic constraint.

## 1.5 UNDERACTUATED ROBOTICS

Today, control design for underactuated systems relies heavily on optimization and optimal control, with fast progress but still many open questions in both model-based optimization and machine learning for control. It's a great time to be studying this material! Now that computer vision has started to work, and we can ask large language models for high-level instructions what our robot should be doing (e.g. "chat GPT, give me detailed instructions on how to make a pizza"), we still have many interesting problems in robotics that are still hard because they are underactuated:

- Legged robots are underactuated. Consider a legged machine with  $N$  internal joints and  $N$  actuators. If the robot is not bolted to the ground, then the degrees of freedom of the system include both the internal joints and the six degrees of freedom which define the position and orientation of the robot in space. Since  $\mathbf{u} \in \mathbb{R}^N$  and  $\mathbf{q} \in \mathbb{R}^{N+6}$ , equation (3) is satisfied.
- (Most) Swimming and flying robots are underactuated. The story is the same here as for legged machines. Each control surface adds one actuator and one DOF. And this is already a simplification, as the true state of the system should really include the (infinite-dimensional) state of the flow.
- Robot manipulation is (often) underactuated. Consider a fully-actuated robotic arm. When this arm is manipulating an object with degrees of freedom (even a brick has six), it can become underactuated. If force closure is achieved, and maintained, then we can think of the system as fully-actuated, because the degrees of freedom of the object are constrained to match the degrees of freedom of the hand. That is, of course, unless the manipulated object has extra DOFs (for example, any object that is deformable).

Being underactuated changes the approach we take for planning and control. For instance, if you look at the chapter on trajectory optimization [in these notes](#), and compare them to the chapter on trajectory optimization [in my manipulation notes](#), you will see that they formulate the problem quite differently. For a fully-actuated robot arm, we can focus on planning kinematic trajectories without immediately worrying about the dynamics. For underactuated systems, one *must* worry about the dynamics.

Even control systems for fully-actuated and otherwise unconstrained systems can be improved using the lessons from underactuated systems, particularly if there is a need to increase the efficiency of their motions or reduce the complexity of their designs.

## 1.6 GOALS FOR THE COURSE

This course studies the rapidly advancing computational tools from optimization theory, control theory, motion planning, and machine learning which can be used to design feedback control for underactuated systems. The goal of this class is to develop these tools in order to design robots that are more dynamic and more agile

than the current state-of-the-art.

The target audience for the class includes both computer science and mechanical/aero students pursuing research in robotics. Although I assume a comfort with linear algebra, ODEs, and Python, the course notes aim to provide most of the material and references required for the course.

I have a confession: I actually think that the material we'll cover in these notes is valuable far beyond robotics. I think that systems theory provides a powerful language for organizing computation in exceedingly complex systems -- especially when one is trying to program and/or analyze systems with continuous variables in a feedback loop (which happens throughout computer science and engineering, by the way). I hope you find these tools to be broadly useful, even if you don't have a humanoid robot capable of performing a backflip at your immediate disposal.

## 1.7 EXERCISES

### Exercise 1.1 (Atlas Backflip)

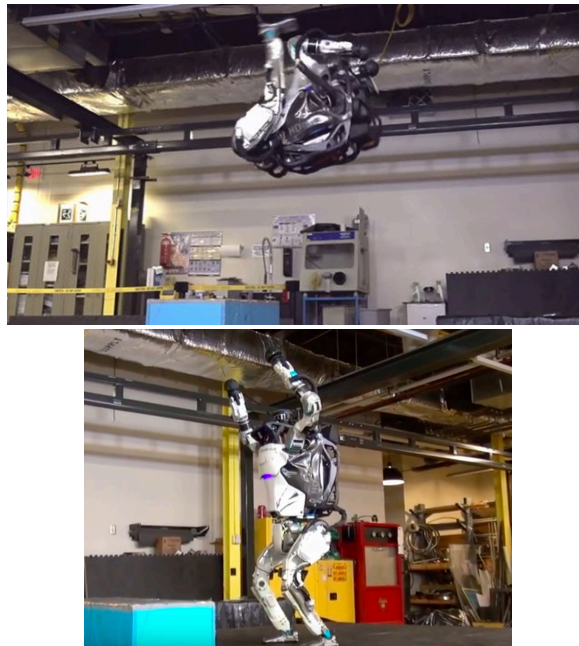


Figure 1.5 - Atlas doing a backflip and Atlas standing.

At the beginning of this chapter you have seen the Atlas humanoid doing a backflip. Now consider the robot in the two states captured in the figure above. Assuming that Atlas' actuators can produce unbounded torques  $\mathbf{u}$ , establish whether or not each of the following statements is true. Briefly justify your answer.

- The state of the humanoid can be represented by the angles and the angular velocities of all its joints.
- While doing the backflip (state in the left figure), the humanoid is fully actuated.
- While standing (state in the right figure), the humanoid is fully actuated.

### Exercise 1.2 (Trajectory Tracking in State Space)

Take a robot whose dynamics is governed by equation 2, and assume it to be fully actuated in all states  $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$  at all times  $t$ .

- For any twice-differentiable desired trajectory  $\mathbf{q}_{\text{des}}(t)$ , is it always possible to find a control signal  $\mathbf{u}(t)$  such that  $\mathbf{q}(t) = \mathbf{q}_{\text{des}}(t)$  for all  $t \geq 0$ , provided that  $\mathbf{q}(0) = \mathbf{q}_{\text{des}}(0)$  and  $\dot{\mathbf{q}}(0) = \dot{\mathbf{q}}_{\text{des}}(0)$ ?
- Now consider the simplest fully-actuated robot: the double integrator. The dynamics of this system reads  $m\ddot{q} = u$ , and you can think of it as a cart of mass  $m$  moving on a straight rail, controlled by a force  $u$ . The figure below depicts its **phase portrait** when  $u = 0$ . Is it possible to find a control signal  $u(t)$  that drives the double integrator from the initial state  $\mathbf{x}(0) = [2, 0.5]^T$  to the origin along a straight line (blue trajectory)? Does the answer change if we set  $\mathbf{x}(0)$  to be  $[2, -0.5]^T$  (red trajectory)?

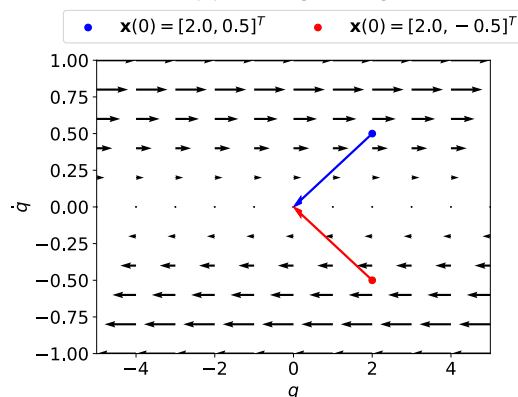


Figure 1.6 - Phase portrait of the double integrator.

- The dynamics **2** are  $n = \dim[\mathbf{q}]$  **second-order** differential equations. However, it is always possible (and we'll frequently do it) to describe these equations in terms of  $2n$  **first-order** differential equations  $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, t)$ . To this end, we simply define

$$\mathbf{f}(\mathbf{x}, t) = \begin{bmatrix} \dot{\mathbf{q}} \\ \mathbf{f}_1(\mathbf{q}, \dot{\mathbf{q}}, t) + \mathbf{f}_2(\mathbf{q}, \dot{\mathbf{q}}, t)\mathbf{u} \end{bmatrix}.$$

For any twice-differentiable trajectory  $\mathbf{x}_{\text{des}}(t)$ , is it always possible to find a control  $\mathbf{u}(t)$  such that  $\mathbf{x}(t) = \mathbf{x}_{\text{des}}(t)$  for all  $t \geq 0$ , provided that  $\mathbf{x}(0) = \mathbf{x}_{\text{des}}(0)$ ?

### Exercise 1.3 (Task-Space Control of the Double Pendulum)

In [the example above](#), we have seen that the double pendulum with one motor per joint is a fully-actuated system. Here we consider a variation of it: instead of controlling the robot with the actuators at the shoulder and the elbow, we directly apply a force on the mass  $m_2$  (tip of the second link). Let  $\mathbf{u} = [u_1, u_2]^T$  be this force, with  $u_1$  horizontal component (positive to the right) and  $u_2$  vertical component (positive upwards). This modification leaves the equations of motion derived in the [appendix example](#) almost unchanged; the only difference is that the matrix  $\mathbf{B}$  is now a function of  $\mathbf{q}$ . Namely, using the notation from the appendix,

$$\mathbf{B}(\mathbf{q}) = \begin{bmatrix} l_1 c_1 + l_2 c_{1+2} & l_1 s_1 + l_2 s_{1+2} \\ l_2 c_{1+2} & l_2 s_{1+2} \end{bmatrix}.$$

Is the double pendulum with the new control input still fully-actuated in all states? If not, identify the states in which it is underactuated.



## Exercise 1.4 (Underactuation of the Planar Quadrotor)

Later in the course we will study the dynamics of a quadrotor quite in depth, for the moment just look at the structure of the resulting equations of motion from the [planar quadrotor section](#). The quadrotor is constrained to move in the vertical plane, with the gravity pointing downwards. The configuration vector  $\mathbf{q} = [x, y, \theta]^T$  collects the position of the center of mass and the pitch angle. The control input is the thrust  $\mathbf{u} = [u_1, u_2]^T$  produced by the two rotors. The input  $\mathbf{u}$  can assume both signs and has no bounds.

- Identify the set of states  $\mathbf{x} = [\mathbf{q}^T, \dot{\mathbf{q}}^T]^T$  in which the system is underactuated.
- For *all* the states in which the system is underactuated, identify an acceleration  $\ddot{\mathbf{q}}(\mathbf{q}, \dot{\mathbf{q}})$  that cannot be instantaneously achieved. Provide a rigorous proof of your claim by using the equations of motion: plug the candidate accelerations in the dynamics, and try to come up with contradictions such as  $mg = 0$ .

## Exercise 1.5 (Drake Systems)

The course software, Drake, provides a powerful modeling language for dynamical systems. The exercise in will help you write your first Drake System.

## REFERENCES

- Steven H. Collins and Andy Ruina and Russ Tedrake and Martijn Wisse, "Efficient bipedal robots based on passive-dynamic walkers", *Science*, vol. 307, pp. 1082-1085, February 18, 2005. [ [link](#) ]
- Steven H. Collins and Martijn Wisse and Andy Ruina, "A Three-Dimensional Passive-Dynamic Walking Robot with Two Legs and Knees", *International Journal of Robotics Research*, vol. 20, no. 7, pp. 607-615, July, 2001.
- J.P.Y. Arnould and D.R. Briggs and J.P. Croxall and P.A. Prince and A.G. Wood, "The foraging behaviour and energetics of wandering albatrosses brooding chicks", *Antarctic Science*, vol. 8, no. 3, pp. 229-236, 1996.
- Wei Shyy and Yongsheng Lian and Jian Teng and Dragos Viieru and Hao Liu, "Aerodynamics of Low Reynolds Number Flyers", Cambridge University Press, 2008.
- Xiaodong Tian and Jose Iriarte-Diaz and Kevin Middleton and Ricardo Galvao and Emily Israeli and Abigail Roemer and Allyce Sullivan and Arnold Song and Sharon Swartz and Kenneth Breuer, "Direct measurements of the kinematics and dynamics of bat flight", *Bioinspiration & Biomimetics*, vol. 1, pp. S10-S18, 2006.
- Michael S. Triantafyllou and George S. Triantafyllou, "An efficient swimming machine", *Scientific American*, vol. 272, no. 3, pp. 64, March, 1995.
- Vance A. Tucker, "Gliding Flight: Speed and Acceleration of Ideal Falcons During Diving and Pull Out", *Journal of Experimental Biology*, vol. 201, pp. 403-414, Nov, 1998.
- Elia Kaufmann and Leonard Bauersfeld and Antonio Loquercio and Matthias Müller and Vladlen Koltun and Davide Scaramuzza, "Champion-level drone racing using deep reinforcement learning", *Nature*, vol. 620, no. 7976, pp. 982-987, 2023.



9. Eric D. Tytell and George V. Lauder, "Hydrodynamics of the escape response in bluegill sunfish, *Lepomis macrochirus*", *The Journal of Experimental Biology*, vol. 211, pp. 3359-3369, 2008.
10. D.N. Beal and F.S. Hover and M.S. Triantafyllou and J.C. Liao and G. V. Lauder, "Passive propulsion in vortex wakes", *Journal of Fluid Mechanics*, vol. 549, pp. 385-402, 2006.
11. Rishi Bommasani and Drew A Hudson and Ehsan Adeli and Russ Altman and Simran Arora and Sydney von Arx and Michael S Bernstein and Jeannette Bohg and Antoine Bosselut and Emma Brunskill and others, "On the opportunities and risks of foundation models", *arXiv preprint arXiv:2108.07258*, 2021.
12. Jean-Jacques E. Slotine and Weiping Li, "Applied Nonlinear Control", Prentice Hall , October, 1990.
13. Anthony Bloch and P. Crouch and J. Baillieul and J. Marsden, "Nonholonomic Mechanics and Control", Springer , April 8, 2003.

---

[Table of contents](#)

[Next Chapter](#)

[Accessibility](#)

© Russ Tedrake, 2024

# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 2

# The Simple Pendulum

### 2.1 INTRODUCTION

Our goals for this chapter are modest: we'd like to understand the dynamics of a pendulum.

Why a pendulum? In part, because the dynamics of a majority of our multi-link robotics manipulators are simply the dynamics of a large number of coupled pendula. Also, the dynamics of a single pendulum are rich enough to introduce most of the concepts from nonlinear dynamics that we will use in this text, but tractable enough for us to (mostly) understand in the next few pages.

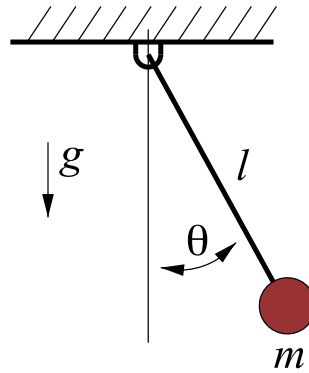


Figure 2.1 - The simple pendulum

The Lagrangian derivation of the equations of motion (as described in the appendix) of the simple pendulum yields:

$$ml^2\ddot{\theta}(t) + mgl \sin \theta(t) = Q.$$

We'll consider the case where the generalized force,  $Q$ , models a damping torque (from friction) plus a control torque input,  $u(t)$ :

$$Q = -b\dot{\theta}(t) + u(t).$$

## 2.2 NONLINEAR DYNAMICS WITH A CONSTANT TORQUE

Let us first consider the dynamics of the pendulum if it is driven in a particular simple way: a torque which does not vary with time:

$$ml^2\ddot{\theta} + b\dot{\theta} + mgl \sin \theta = u_0. \quad (1)$$

### Example 2.1 (Simple Pendulum in Python)

You can experiment with this system in [DRAKE](#) using

These are relatively simple differential equations, so if I give you  $\theta(0)$  and  $\dot{\theta}(0)$ , then you should be able to integrate them to obtain  $\theta(t)$ ... right? Although it is possible, integrating even the simplest case ( $b = u = 0$ ) involves elliptic integrals of the first kind; there is relatively little intuition to be gained here.

This is in stark contrast to the case of linear systems, where much of our understanding comes from being able to explicitly integrate the equations. For instance, for a simple linear system we have

$$\dot{q} = aq \quad \rightarrow \quad q(t) = q(0)e^{at},$$

and we can immediately understand that the long-term behavior of the system is a (stable) decaying exponential if  $a < 0$ , an (unstable) growing exponential if  $a > 0$ , and that the system does nothing if  $a = 0$ . Here we are with certainly one of the simplest nonlinear systems we can imagine, and we can't even solve this system?

All is not lost. If what we care about is the long-term behavior of the system, then there are a number of techniques we can apply. In this chapter, we will start by investigating graphical solution methods. These methods are described beautifully in a book by Steve Strogatz[1].

### 2.2.1 The overdamped pendulum

Let's start by studying a special case -- intuitively when  $b\dot{\theta} \gg ml^2\ddot{\theta}$  -- which via [dimensional analysis](#) (using the natural frequency  $\sqrt{\frac{g}{l}}$  to match units) occurs when  $b\sqrt{\frac{l}{g}} \gg ml^2$ . This is the case of heavy damping, for instance if the pendulum was moving in molasses. In this case, the damping term dominates the acceleration term, and we have:

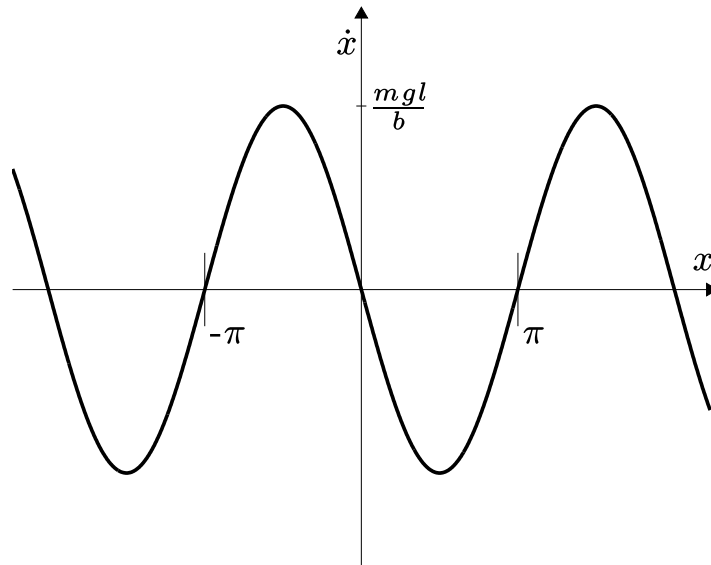
$$ml^2\ddot{\theta} + b\dot{\theta} \approx b\dot{\theta} = u_0 - mgl \sin \theta.$$

In other words, in the case of heavy damping, the system looks approximately first order. This is a general property of heavily-damped systems, such as fluids at very low Reynolds number.

I'd like to ignore one detail for a moment: the fact that  $\theta$  wraps around on itself every  $2\pi$ . To be clear, let's write the system without the wrap-around as:

$$b\dot{x} = u_0 - mgl \sin x. \quad (2)$$

Our goal is to understand the long-term behavior of this system: to find  $x(\infty)$  given  $x(0)$ . Let's start by plotting  $\dot{x}$  vs  $x$  for the case when  $u_0 = 0$ :



The first thing to notice is that the system has a number of **fixed points** or **steady states**, which occur whenever  $\dot{x} = 0$ . In this simple example, the zero-crossings are  $x^* = \{\dots, -\pi, 0, \pi, 2\pi, \dots\}$ . When the system is in one of these states, it will never leave that state. If the initial conditions are at a fixed point, we know that  $x(\infty)$  will be at the same fixed point.

Next let's investigate the behavior of the system in the local vicinity of the fixed points. Examining the fixed point at  $x^* = \pi$ , if the system starts just to the right of the fixed point, then  $\dot{x}$  is positive, so the system will move away from the fixed point. If it starts to the left, then  $\dot{x}$  is negative, and the system will move away in the opposite direction. We'll call fixed-points which have this property **unstable**. If we look at the fixed point at  $x^* = 0$ , then the story is different: trajectories starting to the right or to the left will move back towards the fixed point. We will call this fixed point **locally stable**. More specifically, we'll distinguish between multiple types of stability (where  $\epsilon$  is used to denote an arbitrary **small** scalar quantity):

- Locally **stable** in the sense of Lyapunov (i.s.L.). A fixed point,  $x^*$  is locally stable i.s.L. if for every (small)  $\epsilon > 0$ ,  $\exists \delta > 0$  such that if  $\|x(0) - x^*\| < \delta$  then  $\forall t \ \|x(t) - x^*\| < \epsilon$ . In words, this means that for any ball of size  $\epsilon$  around the fixed point, I can create a ball of size  $\delta$  which guarantees that if the system is started inside the  $\delta$  ball then it will remain inside the  $\epsilon$  ball for all of time.
- Locally **attractive**. A fixed point is locally attractive if for every (small)  $\epsilon$  we have  $x(0) = x^* + \epsilon$  implies that  $\lim_{t \rightarrow \infty} x(t) = x^*$ .
- Locally **asymptotically stable**. A fixed point is locally asymptotically stable if it is locally stable i.s.L. and locally attractive.
- Locally **exponentially stable**. A fixed point is locally exponentially stable if for every (small)  $\epsilon$ , we have  $x(0) = x^* + \epsilon$  implies that  $\|x(t) - x^*\| < Ce^{-\alpha t}$ , for some positive constants  $C$  and  $\alpha$ .
- **Unstable**. A fixed point is unstable if it is not locally stable i.s.L.

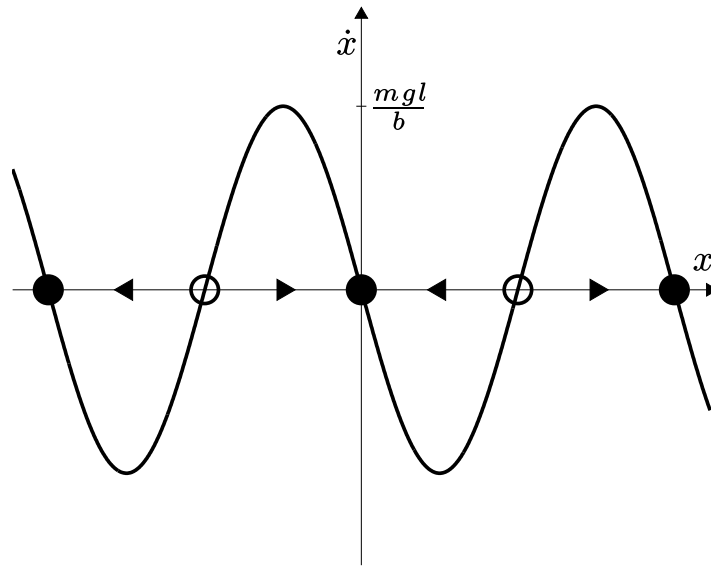
An initial condition near a fixed point that is stable in the sense of Lyapunov may never reach the fixed point (but it won't diverge), near an asymptotically stable fixed point will reach the fixed point as  $t \rightarrow \infty$ , and near an exponentially stable fixed point will reach the fixed point with a bounded rate. An exponentially stable fixed point is also an asymptotically stable fixed point, but the converse is not true. Attractivity does not actually imply Lyapunov stability<sup>†</sup>, which is why we require i.s.L. specifically for the definition of asymptotic stability. Systems which are stable i.s.L. but not asymptotically stable are easy to construct (e.g.  $\dot{x} = 0$ ). Interestingly, it is also possible to have nonlinear systems that converge (or diverge) in finite-time; a so-called **finite-time stability**; we will see examples of this later in the book, but it is a difficult topic to penetrate with graphical analysis. Rigorous nonlinear system analysis is rich with subtleties and surprises. Moreover, these differences actually

<sup>†</sup> we can't see that in one dimension so will have to hold that example for a momen

matter -- the code that we will write to stabilize the systems will be subtly different depending on what type of stability we want, and it can make or break the success of our methods.

Our graph of  $\dot{x}$  vs.  $x$  can be used to convince ourselves of i.s.L. and asymptotic stability by visually inspecting  $\dot{x}$  in the vicinity of a fixed point. Even exponential stability can be inferred if we can find a negatively-sloped line passing through the equilibrium point which separates the graph of  $f(x)$  from the horizontal axis. The existence of this separating line implies that the nonlinear system will converge at least as fast as the linear system represented by the straight line. I will graphically illustrate unstable fixed points with open circles and stable fixed points (i.s.L.) with filled circles.

Next, we need to consider what happens to initial conditions which begin farther from the fixed points. If we think of the dynamics of the system as a flow on the  $x$ -axis, then we know that anytime  $\dot{x} > 0$ , the flow is moving to the right, and  $\dot{x} < 0$ , the flow is moving to the left. If we further annotate our graph with arrows indicating the direction of the flow, then the entire (long-term) system behavior becomes clear:



For instance, we can see that any initial condition  $x(0) \in (-\pi, \pi)$  will result in  $\lim_{t \rightarrow \infty} x(t) = 0$ . This region is called the **region of attraction** (or **basin of attraction**) of the fixed point at  $x^* = 0$ . Basins of attraction of two fixed points cannot overlap, and the manifold separating two basins of attraction is called the **separatrix**. Here the unstable fixed points, at  $x^* = \{\dots, -\pi, \pi, 3\pi, \dots\}$  form the separatrix between the basins of attraction of the stable fixed points. In general, regions of attraction are always open, connected, invariant sets, with boundaries formed by trajectories.

As these plots demonstrate, the behavior of a first-order one dimensional system on a line is relatively constrained. The system will either monotonically approach a fixed-point or monotonically move toward  $\pm\infty$ . There are no other possibilities. Oscillations, for example, are impossible. Graphical analysis is a fantastic analysis tool for many first-order nonlinear systems (not just pendula); as illustrated by the following example:

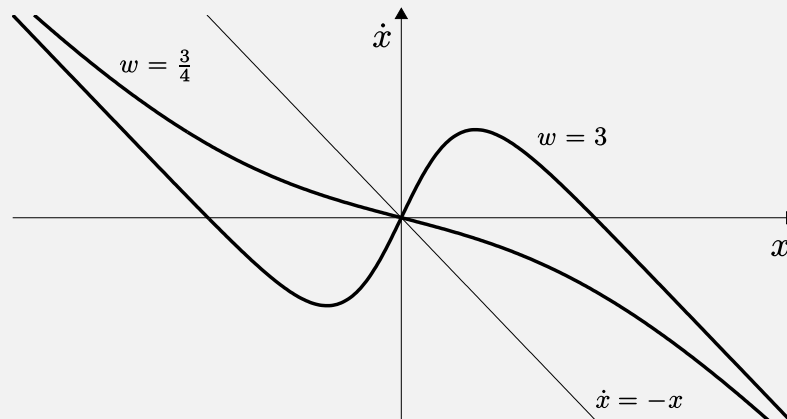
### Example 2.2 (Nonlinear autapse)

Consider the following system:

$$\dot{x} + x = \tanh(wx), \quad (3)$$

which is plotted below for two values of  $w$ . It's convenient to note that  $\tanh(z) \approx z$

for small  $z$ . For  $w \leq 1$  the system has only a single fixed point. For  $w > 1$  the system has three fixed points : two stable and one unstable.



These equations are not arbitrary - they are actually a model for one of the simplest neural networks, and one of the simplest models of persistent memory[2]. In the equation  $x$  models the firing rate of a single neuron, which has a feedback connection to itself.  $\tanh$  is the activation (sigmoidal) function of the neuron, and  $w$  is the weight of the synaptic feedback.

Experiment with it for yourself:

As I've also provided the equations of an **LSTM** unit that you can also experiment with. See if you can figure it out!

Architectures like the **Transformer** for sequence modeling can also be understood through the lens of dynamical systems theory (as autoregressive models), but we will defer that discussion until later in the notes.

One last piece of terminology. In the neuron example, and in many dynamical systems, the dynamics were parameterized; in this case by a single parameter,  $w$ . As we varied  $w$ , the fixed points of the system moved around. In fact, if we increase  $w$  through  $w = 1$ , something dramatic happens - the system goes from having one fixed point to having three fixed points. This is called a **bifurcation**. This particular bifurcation is called a pitchfork bifurcation. We often draw bifurcation diagrams which plot the fixed points of the system as a function of the parameters, with solid lines indicating stable fixed points and dashed lines indicating unstable fixed points, as seen in the figure:

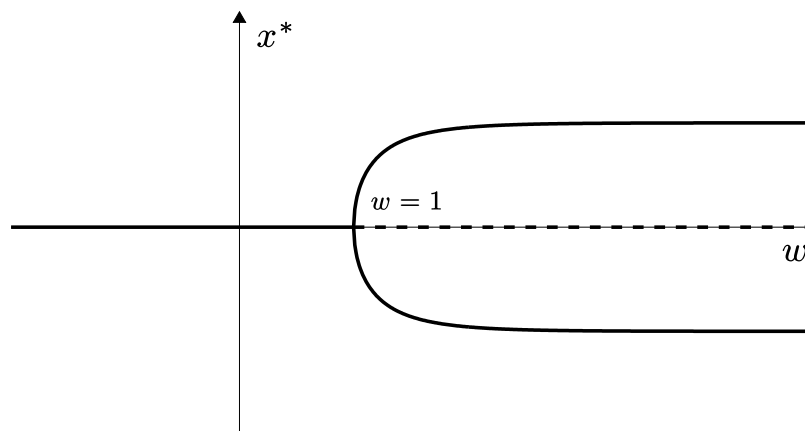


Figure 2.5 - Bifurcation diagram of the nonlinear autapse.

Our pendulum equations also have a (saddle-node) bifurcation when we change the

constant torque input,  $u_0$ . Finally, let's return to the original equations in  $\theta$ , instead of in  $x$ . Only one point to make: because of the wrap-around, this system will *appear* to have oscillations. In fact, the graphical analysis reveals that the pendulum will turn forever whenever  $|u_0| > mgl$ , but now you understand that this is not an oscillation, but an instability with  $\theta \rightarrow \pm\infty$ .

You can find another beautiful example of these concepts (fixed points, basins of attraction, bifurcations) from recurrent neural networks in the exercise below on [Hopfield networks](#).

### 2.2.2 The undamped pendulum with zero torque

Consider again the system

$$ml^2\ddot{\theta} = u_0 - mgl \sin \theta - b\dot{\theta},$$

this time with  $b = 0$ . This time the system dynamics are truly second-order. We can always think of any second-order system,

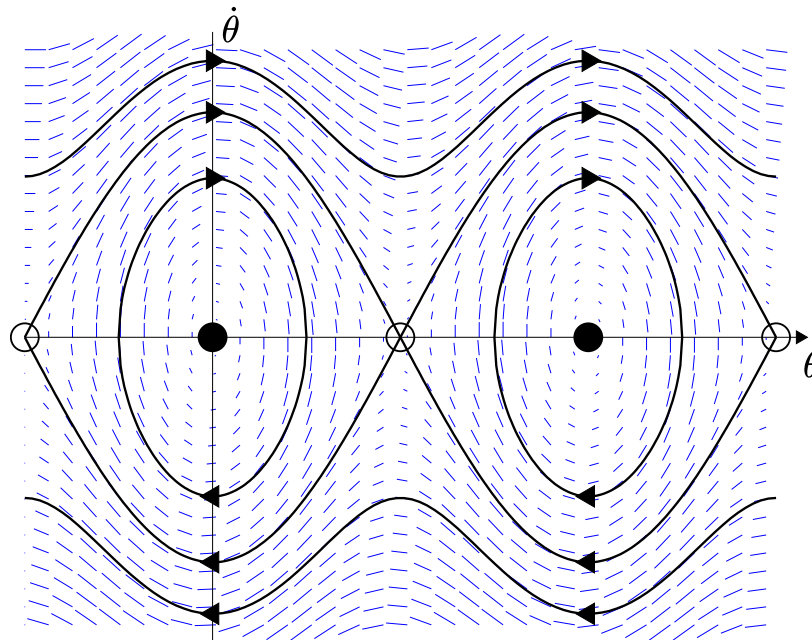
$$\ddot{\theta} = f(\theta, \dot{\theta}),$$

as (coupled) first-order system with twice as many variables. This system is equivalent to the two-dimensional first-order system

$$\begin{aligned}\dot{x}_1 &= x_2 \\ \dot{x}_2 &= f(x_1, x_2),\end{aligned}$$

where  $x_1 = \theta$  and  $x_2 = \dot{\theta}$ . Therefore, the graphical depiction of this system is not a line, but a vector field where the vectors  $[\dot{x}_1, \dot{x}_2]^T$  are plotted over the domain  $(x_1, x_2)$ . This vector field is known as the *phase portrait* of the system.

In this section we restrict ourselves to the simplest case when  $u_0 = 0$ . Let's sketch the phase portrait. First sketch along the  $\theta$ -axis. The  $x$ -component of the vector field here is zero, the  $y$ -component is  $-\frac{g}{l}\sin \theta$ . As expected, we have fixed points at  $\pm\pi, \dots$ . Now sketch the rest of the vector field. Can you tell me which fixed points are stable? Some of them are stable i.s.L., none are asymptotically stable.



### Orbit calculations

You might wonder how we drew the black contour lines in the figure above. We could have obtained them by simulating the system numerically, but those lines can be easily obtained in closed-form. Directly integrating the equations of motion is difficult, but at least for the case when  $u_0 = 0$ , we have some additional physical insight for this problem that we can take advantage of. The kinetic energy,  $T$ , and potential energy,  $U$ , of the pendulum are given by

$$T = \frac{1}{2}I\dot{\theta}^2, \quad U = -mgl \cos(\theta),$$

where  $I = ml^2$  and the total energy is  $E(\theta, \dot{\theta}) = T(\dot{\theta}) + U(\theta)$ . The undamped pendulum is a conservative system: total energy is a constant over system trajectories. Using conservation of energy, we have:

$$\begin{aligned} E(\theta(t), \dot{\theta}(t)) &= E(\theta(0), \dot{\theta}(0)) = E_0 \\ \frac{1}{2}I\dot{\theta}^2(t) - mgl \cos(\theta(t)) &= E_0 \\ \dot{\theta}(t) &= \pm \sqrt{\frac{2}{I}[E_0 + mgl \cos(\theta(t))]} \end{aligned}$$

Using this, if you tell me  $\theta$  I can determine one of two possible values for  $\dot{\theta}$ , and the solution has all of the richness of the black contour lines from the plot. This equation has a real solution when  $\cos(\theta) > \cos(\theta_{max})$ , where

$$\theta_{max} = \begin{cases} \cos^{-1}\left(-\frac{E_0}{mgl}\right), & E_0 < mgl \\ \pi, & \text{otherwise.} \end{cases}$$

Of course this is just the intuitive notion that the pendulum will not swing above the height where the total energy equals the potential energy. As an exercise, you can verify that differentiating this equation with respect to time indeed results in the equations of motion.

The particular orbit defined by  $E = mgl$  is special -- this is the orbit that visits the (unstable) equilibrium at the upright. This is the [homoclinic orbit](#) of the pendulum.

### 2.2.3 The undamped pendulum with a constant torque

Now what happens if we add a constant torque? If you visualize the bifurcation diagram, you'll see that the fixed points come together, towards  $q = \frac{\pi}{2}, \frac{5\pi}{2}, \dots$ , until they disappear. One fixed-point is unstable, and one is stable.

Before we continue, let me now give you the promised example of a system that is not stable i.s.L., but which attracts all trajectories as time goes to infinity. We can accomplish this with a very pendulum-like example (written here in polar coordinates):

#### **Example 2.3 (Unstable equilibrium point that attracts all trajectories)**

Consider the system

$$\begin{aligned} \dot{r} &= r(1 - r), \\ \dot{\theta} &= \sin^2\left(\frac{\theta}{2}\right). \end{aligned}$$

This system has two equilibrium points, one at  $r^* = 0, \theta^* = 0$ , and the other at  $r^* = 1, \theta^* = 0$ . The fixed point at zero is clearly unstable. The fixed point with  $r^* = 1$  attracts all other trajectories, but it is not stable by any of our definitions.



Although the equations are simpler in polar coordinates, it's useful to draw the vector field in Cartesian coordinates:

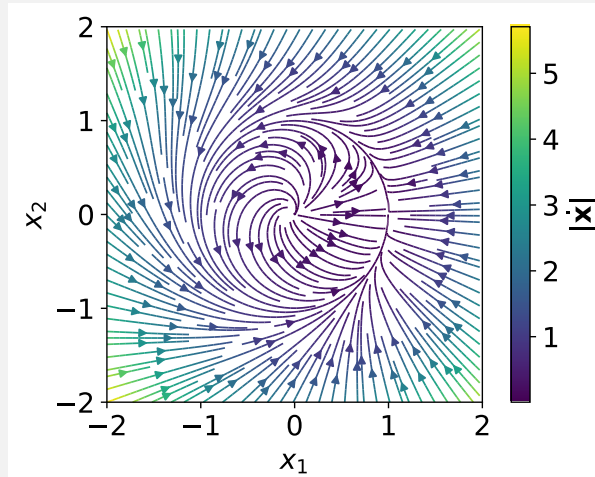


Figure 2.7 - Phase diagram of a two-dimensional system.

Take a minute to examine the vector field of this to make sure you understand. This example may seem contrived, but we will soon see this same phenomenon happen in practice when we introduce the energy-shaping swing-up controller for the pendulum in the next chapter.

## 2.2.4 The damped pendulum

Now let's add damping back. You can still add torque to move the fixed points (in the same way).

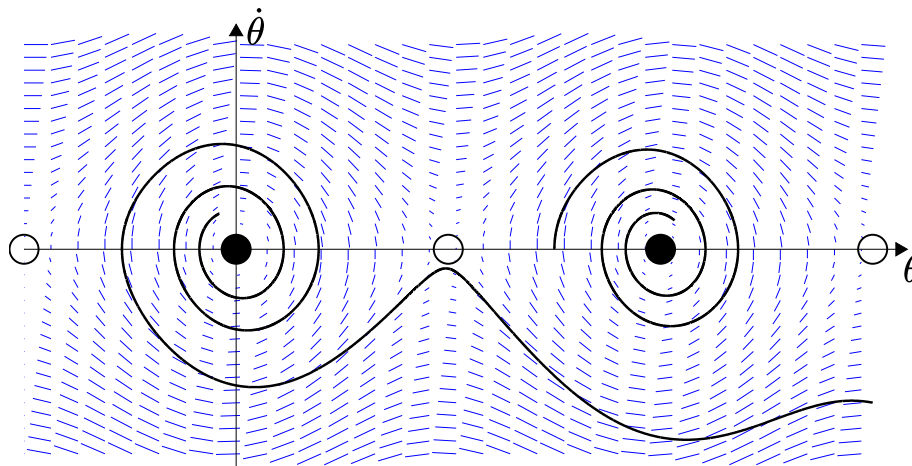


Figure 2.8 - Phase diagram for the damped pendulum

With damping, the downright fixed point of the pendulum now becomes asymptotically stable (in addition to stable i.s.L.). Is it also exponentially stable? How can we tell? One technique is to linearize the system at the fixed point. A smooth, time-invariant, nonlinear system that is locally exponentially stable *must* have a stable linearization; we'll discuss linearization more in the next chapter.

Here's a thought exercise. If  $u$  is no longer a constant, but a function  $\pi(\theta, \dot{\theta})$ , then how would you choose  $\pi$  to stabilize the vertical position. Feedback cancellation is the trivial solution, for example:

$$u = \pi(\theta, \dot{\theta}) = 2mgl \sin \theta.$$

But these plots we've been making tell a different story. How would you shape the natural dynamics - at each point pick a  $u$  from the stack of phase plots - to stabilize the vertical fixed point *with minimal torque effort*? This is exactly the way that I would like you to think about control system design. And in the [dynamic programming chapter](#), we'll start to develop our computational solution techniques.

## 2.3 THE TORQUE-LIMITED SIMPLE PENDULUM

The simple pendulum is fully actuated. Given enough torque, we can produce any number of control solutions to stabilize the originally unstable fixed point at the top (such as designing a feedback controller to effectively invert gravity).

The problem begins to get interesting (a.k.a. becomes underactuated) if we impose a torque-limit constraint,  $|u| \leq u_{max}$ . Looking at the phase portraits again, you can now visualize the control problem. Via feedback, you are allowed to change the direction of the vector field at each point, but only by a fixed amount. Clearly, if the maximum torque is small (smaller than  $mgl$ ), then there are some states which cannot be driven directly to the goal, but must pump up energy to reach the goal. Furthermore, if the torque-limit is too severe and the system has damping, then it may be impossible to swing up to the top. The existence of a solution, and number of pumps required to reach the top, is a non-trivial function of the initial conditions and the torque-limits.

Although this system is very simple, obtaining a solution with general-purpose algorithms requires much of the same reasoning necessary for controlling much more complex underactuated systems; this problem will be a work-horse for us as we introduce new algorithms throughout this book.

### 2.3.1 Energy-shaping control

In the specific case of the pendulum, we can give a satisfying hand-designed nonlinear controller based on our intuition of pumping energy into the system. We have already observed that the total energy of the pendulum is given by

$$E = \frac{1}{2}ml^2\dot{\theta}^2 - mgl \cos \theta.$$

To understand how to control the energy, let us examine how that energy changes with time:

$$\begin{aligned}\dot{E} &= ml^2\dot{\theta}\ddot{\theta} + \dot{\theta}mgl \sin \theta \\ &= \dot{\theta} [u - mgl \sin \theta] + \dot{\theta}mgl \sin \theta \\ &= u\dot{\theta}.\end{aligned}$$

In words, adding energy to the system is simple - apply torque in the same direction as  $\dot{\theta}$ . To remove energy, apply torque in the opposite direction (e.g., damping).

To swing up the pendulum, even with torque limits, let us use this observation to drive the system to its homoclinic orbit, and then let the dynamics of the pendulum carry us to the upright equilibrium. Recall that the homoclinic orbit has energy  $mgl$  -- let's call this our *desired* energy:

$$E^d = mgl.$$

Furthermore, let's define the difference between our current energy and this desired energy as  $\tilde{E} = E - E^d$ , and note that we still have

$$\dot{\tilde{E}} = \dot{E} = u\dot{\theta}.$$

Now consider the feedback controller of the form

$$u = -k\dot{\theta}\tilde{E}, \quad k > 0.$$

I admit that this looks a bit strange; it was chosen for the simple reason that it turns the resulting "error dynamics" into something simple:

$$\dot{\tilde{E}} = -k\dot{\theta}^2\tilde{E}.$$

Think about the graphical analysis of this system if you were to draw  $\dot{\tilde{E}}$  vs.  $\tilde{E}$  for any fixed  $\dot{\theta}$  -- it's a straight line separated from the horizontal axis which would imply an exponential convergence:  $\tilde{E} \rightarrow 0$ . This is true for any  $\dot{\theta}$ , except for  $\dot{\theta} = 0$  (so it will not actually swing us up from the downright fixed point... but if you nudge the system just a bit, then it will start pumping energy and will swing all of the way up). The essential property is that when  $E > E^d$ , we should remove energy from the system (damping) and when  $E < E^d$ , we should add energy (negative damping). Even if the control actions are bounded, the convergence is easily preserved.

This is a nonlinear controller that will push all system trajectories to the unstable equilibrium. But does it make the unstable equilibrium locally stable? With only this controller, the fixed point is *attractive, but is not stable* -- just like our example above. Small perturbations may cause the system to drive all of the way around the circle in order to once again return to the unstable equilibrium. For this reason, to actually balance the system, we'll have to switch to a different controller once we get near the fixed point (an idea that we'll discuss more in the next chapter).

### Example 2.4 (Energy Shaping for the Pendulum)

Take a minute to play around with the energy-shaping controller for swinging up the pendulum

Make sure that you take a minute to look at the code which runs during these examples. Note the somewhat arbitrary threshold for switching to the balancing controller. We'll give a much more satisfying answer for that in the [chapter on Lyapunov methods](#).

There are a few things that are extremely nice about this controller. Not only is it simple, but it is actually incredibly robust to errors we might have in our estimate of the model parameters,  $m$ ,  $l$ , and  $g$ . In fact, the only place that the model enters our control equation is in the calculation of  $\tilde{E} = \frac{1}{2}ml^2\dot{\theta}^2 - mgl(1 + \cos\theta)$ , and the only property of this estimate that impacts stability is the location of the orbit when  $\tilde{E} = 0$ , which is  $\frac{1}{2}l\dot{\theta}^2 = g(\cos\theta + 1)$ . Therefore, we could have used  $\frac{E-E_d}{m} = \frac{1}{2}l^2\dot{\theta}^2 - gl(1 + \cos\theta)$  in our controller and obtained the same result. This doesn't depend at all on our estimate of the mass, and only linearly on our estimates of the length and gravity (and if one cannot measure the length of the pendulum accurately, then a proper measuring device would make an excellent investment). This is somewhat amazing; we will develop many optimization-based algorithms capable of swinging up the pendulum, but it will take a lot of work to find one that is as insensitive to the model parameters.

If there is damping in the original system, of course we can cancel this out, too, using  $u = -k\dot{\theta}\tilde{E} + b\dot{\theta}$ . And once again, the controller is quite robust if we have errors in the estimated damping ratio.

## 2.4 EXERCISES

### Exercise 2.1 (Graphical Analysis)

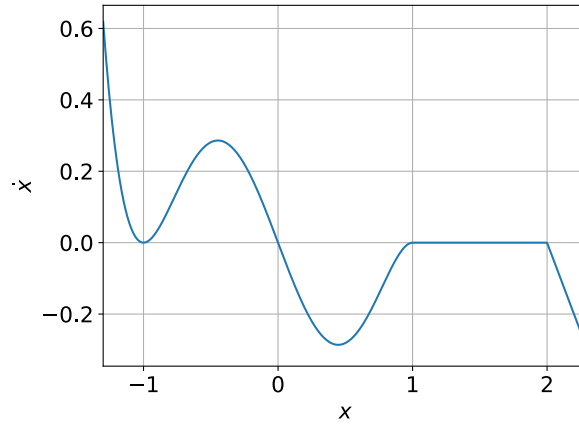


Figure 2.9 - First-order systems with multiple equilibria. Consider the first-order system

$$\dot{x} = \begin{cases} -x^5 + 2x^3 - x & \text{if } x \leq 1, \\ 0 & \text{if } 1 < x \leq 2, \\ -x + 2 & \text{if } x > 2, \end{cases}$$

whose dynamics is represented in the figure above. Notice that, together with  $x^* = -1$  and  $x^* = 0$ , all the points in the closed interval  $[1, 2]$  are equilibria for this system. For each equilibrium point, determine whether it is unstable, stable i.s.L., asymptotically stable, or exponentially stable.

### Exercise 2.2 (Basin of Attraction and Bifurcation Diagram)

Consider the first-order system with polynomial dynamics

$$\dot{x} = f(x) = -x^3 + 4x^2 + 11x - 30.$$

- Sketch the graph of the function  $f(x)$ , and identify all the equilibrium points. (Feel free to plot this with a tool of your choice to check your work.)
- For each equilibrium point, determine whether it is stable (i.s.L.) or unstable. For each one of the stable equilibria, identify the basin of attraction.
- Consider an additive term  $w$  in the system dynamics:  $\dot{x} = f(x) + w$ . As  $w$  ranges from 0 to  $\infty$ , determine how many stable and unstable equilibrium points the system has. Support your answer with a sketch of the bifurcation diagram for nonnegative values of  $w$ .

### Exercise 2.3 (Enumerate Unstable Equilibria)

We are given a dynamical system  $\dot{x} = f(x)$  with a scalar state  $x$ . The dynamics  $f(x)$  is unknown, but we are given two pieces of information:

- the function  $f(x)$  is continuous;
- the system has exactly three stable (i.s.L.) equilibrium points.

Identify the minimum  $n_{\min}$  and the maximum  $n_{\max}$  number of unstable equilibria that the system can have. Support your answer with the sketch of two functions

$f_{\min}(x)$  and  $f_{\max}(x)$  that verify the requirements above and have, respectively,  $n_{\min}$  and  $n_{\max}$  unstable equilibria.

## Exercise 2.4 (Attractivity vs Stability)

Consider the dynamical system given by

$$\begin{aligned} \dot{x}_1 &= \begin{cases} x_1(1 - |\mathbf{x}|) + x_2 \frac{x_1 - |\mathbf{x}|}{2|\mathbf{x}|}, & \text{if } |\mathbf{x}| \neq 0, \\ 0, & \text{if } |\mathbf{x}| = 0, \end{cases} \\ \dot{x}_2 &= \begin{cases} x_2(1 - |\mathbf{x}|) - x_1 \frac{x_1 - |\mathbf{x}|}{2|\mathbf{x}|}, & \text{if } |\mathbf{x}| \neq 0, \\ 0, & \text{if } |\mathbf{x}| = 0, \end{cases} \end{aligned}$$

where  $|\mathbf{x}| = \sqrt{x_1^2 + x_2^2}$ . To help you with the analysis of this system, we set up . Take your time to understand the code in it, and then answer the following questions.

- Find all the equilibrium points of this system (no need to look outside the portion of state space depicted above). Use the notebook to double check your answer: simulate the evolution of the system setting the equilibrium points you identified as initial conditions.
- Determine whether the equilibria you identified at the previous point are attractive and/or stable i.s.L. Explain briefly your reasoning, and feel free to include some plots generated with the notebook in your written answer.
- This dynamical system is a (very) close relative of one of the systems we analyzed in this chapter. Can you guess which one is it? What is the connection between the two? Extra points: support your claim with a mathematical derivation.

## Exercise 2.5 (Pendulum with Vibrating Base)

- Identify some dynamics function  $\dot{q} = f(q)$  whose unique equilibrium point is stable and is located in  $q = 1$ .
- Consider an actuated pendulum whose base (pivot of the rod) is forced to oscillate horizontally according to the harmonic law  $h \sin(\omega t)$ , where  $h$  denotes the amplitude of the oscillation and  $\omega$  the frequency. The equation of motion for this system is

$$ml^2 \ddot{\theta} + mgl \sin \theta = ml\omega^2 h \sin(\omega t) \cos \theta + u.$$

(If this equation seems obscure to you, try to derive it using the [Lagrangian approach](#); but be careful, the kinetic energy  $T(\theta, \dot{\theta}, t)$  of this system depends explicitly on time.) Our goal is to design a time-dependent control law  $u = \pi(\theta, \dot{\theta}, t)$  that makes the pendulum spin at constant velocity  $\dot{\theta} = 1$ .

Use feedback cancellation to design a control law  $u = \pi(\theta, \dot{\theta}, t)$  that results in the closed-loop dynamics  $\ddot{\theta} = f(\dot{\theta})$ , where  $f(\cdot)$  is the function you found in part (a).

- , we set up a simulation environment to test your control law. Try to understand the structure of the code: this workflow is quite general, and it could be a good starting point for your future Drake projects. Implement your control law in the dedicated cell, and check your work using the animation and the plots at the end of the notebook.

## Exercise 2.6 (Hopfield Networks)

Consider the following dynamical system

$$\dot{x} = A^T \text{softmax}(\beta A x) - x \quad (4)$$

where  $A \in \mathbb{R}^{m \times n}$  and  $\beta \in \mathbb{R}$  are a constant matrix and a scalar, respectively, and the softmax function is given by  $\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_k e^{z_k}}$ .

- a. Draw the phase portrait of the system with constants  $\beta = 5$  and

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ -1 & -1 \end{bmatrix}.$$

- b. Identify (approximately by looking at the phase portrait) equilibriums of the above system. What are the stable and unstable equilibriums?
- c. How are the stable equilibriums related to the matrix  $A$ ?
- d. Using , implement an image recovery algorithm using Hopfield network. Modify the dynamical system above to memorize a subset of MNIST data set. Using a corrupted image as the initial condition, simulate the dynamical system to restore images.

## REFERENCES

1. Steven H. Strogatz, "Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering", Perseus Books , 1994.
2. H. Sebastian Seung and Daniel D. Lee and Ben Y. Reis and David W. Tank, "The autapse: a simple illustration of short-term analog memory storage by tuned synaptic feedback", *Journal of Computational Neuroscience*, vol. 9, pp. 171-85, 2000.

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

[Accessibility](#)

© Russ Tedrake, 2024

# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 3

# Acrobots, Cart-Poles, and Quadrotors

A great deal of work in the control of underactuated systems has been done in the context of low-dimensional model systems. These model systems capture the essence of the problem without introducing all of the complexity that is often involved in more real-world examples. In this chapter we will focus on two of the most well-known and well-studied model systems--the Acrobot and the Cart-Pole. After we have developed some tools, we will see that they can be applied directly to other model systems; we will give a number of examples using Quadrotors. All of these systems are trivially underactuated, having less actuators than degrees of freedom.

### 3.1 THE ACROBOT

The Acrobot is a planar two-link robotic arm in the vertical plane (working against gravity), with an actuator at the elbow, but no actuator at the shoulder. It was first described in detail in [1]. The companion system, with an actuator at the shoulder but not at the elbow, is known as the Pendubot[2]. The Acrobot is so named because of its resemblance to a gymnast (or acrobat) on a parallel bar, who controls his/her motion predominantly by effort at the waist (and not effort at the wrist). The most common control task studied for the Acrobot is the swing-up task, in which the system must use the elbow (or waist) torque to move the system into a vertical configuration then balance.

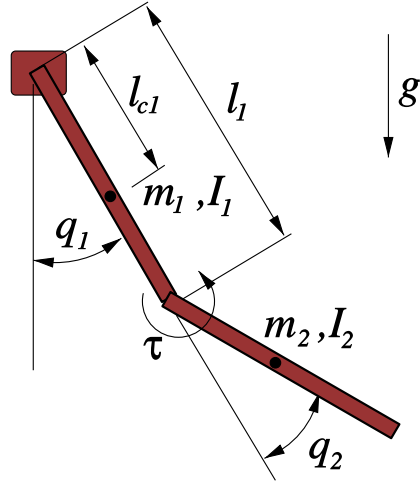


Figure 3.1 - The Acrobot. [Click here to see a physical Acrobot swing up and balance.](#)

The Acrobot is representative of the primary challenge in underactuated robots. In order to swing up and balance the entire system, the controller must reason about and exploit the state-dependent coupling between the actuated degree of freedom and the unactuated degree of freedom. It is also an important system because, as we will see, it closely resembles one of the simplest models of a walking robot.

### 3.1.1 Equations of motion

Figure 3.1 illustrates the model parameters used in our analysis.  $\theta_1$  is the shoulder joint angle,  $\theta_2$  is the elbow (relative) joint angle, and we will use  $\mathbf{q} = [\theta_1, \theta_2]^T$ ,  $\dot{\mathbf{x}} = [\mathbf{q}, \dot{\mathbf{q}}]^T$ . The zero configuration is with both links pointed directly down. The moments of inertia,  $I_1, I_2$  are taken about the pivots. The task is to stabilize the unstable fixed point  $\mathbf{x} = [\pi, 0, 0, 0]^T$ .

We will derive the equations of motion for the Acrobot using the method of Lagrange. The locations of the center of mass of each link,  $\mathbf{p}_{c1}, \mathbf{p}_{c2}$ , are given by the kinematics:

$$\mathbf{p}_{c1} = \begin{bmatrix} l_{c1}s_1 \\ -l_{c1}c_1 \end{bmatrix}, \quad \mathbf{p}_{c2} = \begin{bmatrix} l_1s_1 + l_{c2}s_{1+2} \\ -l_1c_1 - l_{c2}c_{1+2} \end{bmatrix}, \quad (1)$$

where  $s_1$  is shorthand for  $\sin(\theta_1)$ ,  $c_{1+2}$  is shorthand for  $\cos(\theta_1 + \theta_2)$ , etc. The energy is given by:

$$T = T_1 + T_2, \quad T_1 = \frac{1}{2}I_1\dot{q}_1^2 \quad (2)$$

$$T_2 = \frac{1}{2}(m_2l_1^2 + I_2 + 2m_2l_1l_{c2}c_2)\dot{q}_1^2 + \frac{1}{2}I_2\dot{q}_2^2 + (I_2 + m_2l_1l_{c2}c_2)\dot{q}_1\dot{q}_2 \quad (3)$$

$$U = -m_1gl_{c1}c_1 - m_2g(l_1c_1 + l_{c2}c_{1+2}) \quad (4)$$

Entering these quantities into the Lagrangian yields the equations of motion:

$$(I_1 + I_2 + m_2l_1^2 + 2m_2l_1l_{c2}c_2)\ddot{q}_1 + (I_2 + m_2l_1l_{c2}c_2)\ddot{q}_2 - 2m_2l_1l_{c2}s_2\dot{q}_1\dot{q}_2 \quad (5)$$

$$-m_2l_1l_{c2}s_2\dot{q}_2^2 + m_1gl_{c1}s_1 + m_2g(l_1s_1 + l_{c2}s_{1+2}) = 0 \quad (6)$$

$$(I_2 + m_2l_1l_{c2}c_2)\ddot{q}_1 + I_2\ddot{q}_2 + m_2l_1l_{c2}s_2\dot{q}_1^2 + m_2gl_{c2}s_{1+2} = \tau \quad (7)$$

In standard, manipulator equation form:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u},$$

using  $\mathbf{q} = [\theta_1, \theta_2]^T$ ,  $\mathbf{u} = \tau$  we have:



$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} I_1 + I_2 + m_2 l_1^2 + 2m_2 l_1 l_{c2} c_2 & I_2 + m_2 l_1 l_{c2} c_2 \\ I_2 + m_2 l_1 l_{c2} c_2 & I_2 \end{bmatrix}, \quad (8)$$

$$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) = \begin{bmatrix} -2m_2 l_1 l_{c2} s_2 \dot{q}_2 & -m_2 l_1 l_{c2} s_2 \dot{q}_2 \\ m_2 l_1 l_{c2} s_2 \dot{q}_1 & 0 \end{bmatrix}, \quad (9)$$

$$\tau_g(\mathbf{q}) = \begin{bmatrix} -m_1 g l_{c1} s_1 - m_2 g (l_1 s_1 + l_{c2} s_{1+2}) \\ -m_2 g l_{c2} s_{1+2} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}. \quad (10)$$

### Example 3.1 (Dynamics of the Acrobot)

You can experiment with the Acrobot dynamics in **DRAKE** using, e.g.

## 3.2 THE CART-POLE SYSTEM

The other model system that we will investigate here is the cart-pole system, in which the task is to balance a simple pendulum around its unstable equilibrium, using only horizontal forces on the cart. Balancing the cart-pole system is used in many introductory courses in control, including 6.003 at MIT, because it can be accomplished with simple linear control (e.g. pole placement) techniques. In this chapter we will consider the full swing-up and balance control problem, which requires a full nonlinear control treatment.

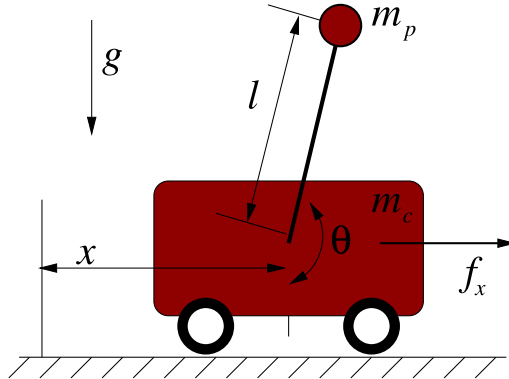


Figure 3.2 - The Cart-Pole system. Click [here to see a real robot](#).

The figure shows our parameterization of the system.  $x$  is the horizontal position of the cart,  $\theta$  is the counter-clockwise angle of the pendulum (zero is hanging straight down). We will use  $\mathbf{q} = [x, \theta]^T$ , and  $\mathbf{x} = [\mathbf{q}, \dot{\mathbf{q}}]^T$ . The task is to stabilize the unstable fixed point at  $\mathbf{x} = [0, \pi, 0, 0]^T$ .

### 3.2.1 Equations of motion

The kinematics of the system are given by

$$\mathbf{x}_1 = \begin{bmatrix} x \\ 0 \end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix} x + l \sin \theta \\ -l \cos \theta \end{bmatrix}. \quad (11)$$

The energy is given by

$$T = \frac{1}{2} (m_c + m_p) \dot{x}^2 + m_p \dot{x} \dot{\theta} l \cos \theta + \frac{1}{2} m_p l^2 \dot{\theta}^2 \quad (12)$$

$$U = -m_p g l \cos \theta. \quad (13)$$

The Lagrangian yields the equations of motion:

$$(m_c + m_p)\ddot{x} + m_p l \ddot{\theta} \cos \theta - m_p l \dot{\theta}^2 \sin \theta = f_x \quad (14)$$

$$m_p l \ddot{x} \cos \theta + m_p l^2 \ddot{\theta} + m_p g l \sin \theta = 0 \quad (15)$$

In standard, manipulator equation form:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u},$$

using  $\mathbf{q} = [x, \theta]^T$ ,  $\mathbf{u} = f_x$ , we have:

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} m_c + m_p & m_p l \cos \theta \\ m_p l \cos \theta & m_p l^2 \end{bmatrix}, \quad \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) = \begin{bmatrix} 0 & -m_p l \dot{\theta} \sin \theta \\ 0 & 0 \end{bmatrix},$$

$$\tau_g(\mathbf{q}) = \begin{bmatrix} 0 \\ -m_p g l \sin \theta \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

In this case, it is particularly easy to solve directly for the accelerations:

$$\ddot{x} = \frac{1}{m_c + m_p \sin^2 \theta} \left[ f_x + m_p \sin \theta (l \dot{\theta}^2 + g \cos \theta) \right] \quad (16)$$

$$\ddot{\theta} = \frac{1}{l(m_c + m_p \sin^2 \theta)} \left[ -f_x \cos \theta - m_p l \dot{\theta}^2 \cos \theta \sin \theta - (m_c + m_p) g \sin \theta \right] \quad (17)$$

In some of the analysis that follows, we will study the form of the equations of motion, ignoring the details, by arbitrarily setting all constants to 1:

$$2\ddot{x} + \ddot{\theta} \cos \theta - \dot{\theta}^2 \sin \theta = f_x \quad (18)$$

$$\ddot{x} \cos \theta + \ddot{\theta} + \sin \theta = 0. \quad (19)$$

### Example 3.2 (Dynamics of the Cart-Pole System)

You can experiment with the Cart-Pole dynamics in [DRAKE](#) using, e.g.

## 3.3 QUADROTORS

Quadrotors have become immensely popular over the last few years -- advances in outrunner motors from the hobby community made them powerful, light-weight, and inexpensive! They are strong enough to carry an interesting payload (e.g. of sensors for mapping / photographing the environment), but dynamic enough to move relatively quickly. The most interesting dynamics start showing up at higher speeds, when the propellers start achieving lift from the airflow across them due to horizontal motion, or when they are close enough to the ground to experience significant ground-effect, but we won't try to capture those effects (yet) here.

When the quadrotor revolution started to happen, I predicted that it would be followed quickly by a realization that fixed-wing vehicles are better for most applications. Propellers are almost optimally efficient for producing thrust -- making quadrotors very efficient for hovering -- but to be efficient in forward flight you probably want to have an airfoil. Wings are a really good idea! But I was wrong -- quadrotors have completely dominated fixed-wings for commercial UAVs. Perhaps it's only because they are easier to control? I suspect that as the field matures and achieving core functionality is not the primary obstacle, then people will eventually start worrying again about efficiency (a bit like we think about efficiency for automobiles).

### 3.3.1 The Planar Quadrotor

We can get started with an extremely simple model of a quadrotor that is restricted to live in the plane. In that case, it actually only needs two propellers, but calling it a

"birotor" doesn't have the same ring to it. The equations of motion are almost trivial, since it is only a single rigid body, and certainly fit into our standard manipulator equations:

$$m\ddot{x} = -(u_1 + u_2) \sin \theta, \quad (20)$$

$$m\ddot{y} = (u_1 + u_2) \cos \theta - mg, \quad (21)$$

$$I\ddot{\theta} = r(u_1 - u_2) \quad (22)$$

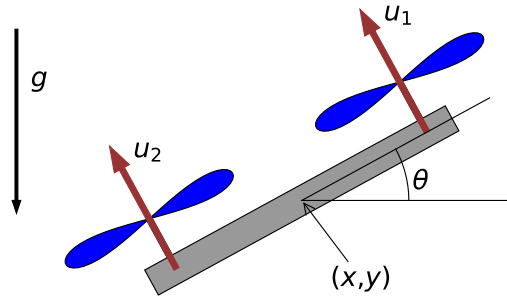


Figure 3.3 - The Planar Quadrotor System (which we also refer to in the code as "Quadrotor2D", to keep it next to "Quadrotor" in the directory listing). The model parameters are mass,  $m$ , moment of inertia,  $I$ , and the distance from the center to the base of the propeller,  $r$ .

### 3.3.2 The Full 3D Quadrotor

The dynamics of the 3D Quadrotor follow similarly. The only complexity comes from handling the 3D rotations. The code implementing the dynamics of the `QuadrotorPlant` example in Drake is almost as readable as a LaTeX derivation. The most interesting feature of this model that we include the *moment* produced by the rotating propellers. Interestingly, without these moments, the system linearized about the hovering configuration is actually not controllable.

The `QuadrotorPlant` example implements the simple model. But often you may want to add extra features to the quadrotor model. For instance, you might wish to add collision dynamics to land on the ground or to bounce off obstacles, or you might want to add a pendulum to the quadrotor to balance, or a suspended payload hanging off the bottom. In these cases, rather than implement the equations by hand, it is much more convenient to use Drake's main physics engine (in `MultibodyPlant`). We just need to manually wire the `Propeller` forces into the `Diagram` (because `Propeller` is not a concept supported in URDF nor SDF just yet). I've implemented both versions side-by-side in the notebook.

Note that by default, the orientation of each floating body in `MultibodyPlant` is represented by a unit quaternion. If you were to try to linearize the model using the quaternion representation of state, without accounting for unit norm constraint, the linearization of this model is also uncontrollable. To avoid this for now, I have manually added a "roll-pitch-yaw" floating base to the model. This keeps linearization simple but does introduce a singularity at "gimble lock". I've tried to mostly avoid dealing with 3D rotations throughout these notes, but my [notes on manipulation cover 3D rotations](#) in a bit more detail.

## 3.4 BALANCING

For both the Acrobot and the Cart-Pole systems, we will begin by designing a linear controller which can balance the system when it begins in the vicinity of the unstable fixed point. To accomplish this, we will linearize the nonlinear equations about the fixed point, examine the controllability of this linear system, then using [linear](#)

quadratic regulator (LQR) theory to design our feedback controller.

### 3.4.1 Linearizing the manipulator equations

Although the equations of motion of both of these model systems are relatively tractable, the forward dynamics still involve quite a few nonlinear terms that must be considered in any linearization. Let's consider the general problem of linearizing a system described by the manipulator equations.

We can perform linearization around a fixed point,  $(\mathbf{x}^*, \mathbf{u}^*)$ , using a Taylor expansion:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}) \approx \mathbf{f}(\mathbf{x}^*, \mathbf{u}^*) + \left[ \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right]_{\mathbf{x}=\mathbf{x}^*, \mathbf{u}=\mathbf{u}^*} (\mathbf{x} - \mathbf{x}^*) + \left[ \frac{\partial \mathbf{f}}{\partial \mathbf{u}} \right]_{\mathbf{x}=\mathbf{x}^*, \mathbf{u}=\mathbf{u}^*} (\mathbf{u} - \mathbf{u}^*) \quad (23)$$

Let us consider the specific problem of linearizing the manipulator equations around a (stable or unstable) fixed point. In this case,  $\mathbf{f}(\mathbf{x}^*, \mathbf{u}^*)$  is zero, and we are left with the standard linear state-space form:

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{\mathbf{q}} \\ \mathbf{M}^{-1}(\mathbf{q}) [\tau_g(\mathbf{q}) + \mathbf{B}(\mathbf{q})\mathbf{u} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}] \end{bmatrix}, \quad (24)$$

$$\approx \mathbf{A}_{lin}(\mathbf{x} - \mathbf{x}^*) + \mathbf{B}_{lin}(\mathbf{u} - \mathbf{u}^*), \quad (25)$$

where  $\mathbf{A}_{lin}$ , and  $\mathbf{B}_{lin}$  are constant matrices. Let us define  $\bar{\mathbf{x}} = \mathbf{x} - \mathbf{x}^*$ ,  $\bar{\mathbf{u}} = \mathbf{u} - \mathbf{u}^*$ , and write

$$\dot{\bar{\mathbf{x}}} = \mathbf{A}_{lin}\bar{\mathbf{x}} + \mathbf{B}_{lin}\bar{\mathbf{u}}.$$

Evaluation of the Taylor expansion around a fixed point yields the following, very simple equations, given in block form by:

$$\mathbf{A}_{lin} = \begin{bmatrix} \mathbf{0} & \mathbf{I} \\ \mathbf{M}^{-1} \frac{\partial \tau_g}{\partial \mathbf{q}} + \sum_j \mathbf{M}^{-1} \frac{\partial \mathbf{B}_j}{\partial \mathbf{q}} u_j & \mathbf{0} \end{bmatrix}_{\mathbf{x}=\mathbf{x}^*, \mathbf{u}=\mathbf{u}^*} \quad (26)$$

$$\mathbf{B}_{lin} = \begin{bmatrix} \mathbf{0} \\ \mathbf{M}^{-1} \mathbf{B} \end{bmatrix}_{\mathbf{x}=\mathbf{x}^*, \mathbf{u}=\mathbf{u}^*} \quad (27)$$

where  $\mathbf{B}_j$  is the  $j$ th column of  $\mathbf{B}$ . Note that the term involving  $\frac{\partial \mathbf{M}^{-1}}{\partial q_i}$  disappears because  $\tau_g + \mathbf{B}\mathbf{u} - \mathbf{C}\dot{\mathbf{q}}$  must be zero at the fixed point. All of the  $\mathbf{C}\dot{\mathbf{q}}$  derivatives drop out, too, because  $\dot{\mathbf{q}}^* = 0$ , and any terms with  $\mathbf{C}$  drop out as well, since centripetal and centrifugal forces are zero when velocity is zero. In many cases, including both the Acrobot and Cart-Pole systems (but not the Quadrotors), the matrix  $\mathbf{B}(\mathbf{q})$  is a constant, so the  $\frac{\partial \mathbf{B}}{\partial \mathbf{q}}$  terms also drop out.

#### Example 3.3 (Linearization of the Acrobot)

Linearizing around the (unstable) upright point, we have:

$$\left[ \frac{\partial \tau_g}{\partial \mathbf{q}} \right]_{\mathbf{x}=\mathbf{x}^*} = \begin{bmatrix} g(m_1 l_{c1} + m_2 l_1 + m_2 l_{c2}) & m_2 g l_{c2} \\ m_2 g l_{c2} & m_2 g l_{c2} \end{bmatrix} \quad (28)$$

The linear dynamics follow directly from these equations and the manipulator form of the Acrobot equations.

#### Example 3.4 (Linearization of the Cart-Pole System)

Linearizing around the unstable fixed point in this system, we have:

$$\left[ \frac{\partial \tau_g}{\partial \mathbf{q}} \right]_{\mathbf{x}=\mathbf{x}^*} = \begin{bmatrix} 0 & 0 \\ 0 & m_p g l \end{bmatrix} \quad (29)$$

Again, the linear dynamics follow simply.

Studying the properties of the linearized system can tell us some things about the (local) properties of the nonlinear system. For instance, having a strictly stable linearization implies local exponential stability of the nonlinear system [3] (Theorem 4.15). It's worth noting that having an unstable linearization also implies that the system is locally unstable, but if the linearization is marginally stable then one cannot conclude whether the nonlinear system is asymptotically stable, stable i.s.L., or unstable (only that it is not exponentially stable)[4].

### 3.4.2 Controllability of linear systems

**Definition 3.1 (Controllability)** A control system of the form

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$$

is called *controllable* if it is possible to construct an unconstrained input signal,  $\mathbf{u}(t)$ ,  $t \in [0, t_f]$ , which will move the system from any initial state to any final state in a finite interval of time,  $0 < t < t_f$  [5].

For the linear system

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u},$$

we can integrate this linear system in closed form, so it is possible to derive the exact conditions of controllability. In particular, for linear systems it is sufficient to demonstrate that there exists a control input which drives any initial condition to the origin.

### The special case of non-repeated eigenvalues

Let us first examine a special case, which falls short as a general tool but may be more useful for understanding the intuition of controllability. Let's perform an eigenvalue analysis of the system matrix  $\mathbf{A}$ , so that:

$$\mathbf{A}\mathbf{v}_i = \lambda_i \mathbf{v}_i,$$

where  $\lambda_i$  is the  $i$ th eigenvalue, and  $\mathbf{v}_i$  is the corresponding (right) eigenvector. There will be  $n$  eigenvalues for the  $n \times n$  matrix  $\mathbf{A}$ . Collecting the (column) eigenvectors into the matrix  $\mathbf{V}$  and the eigenvalues into a diagonal matrix  $\mathbf{\Lambda}$ , we have

$$\mathbf{A}\mathbf{V} = \mathbf{V}\mathbf{\Lambda}.$$

Here comes our primary assumption: let us assume that each of these  $n$  eigenvalues takes on a distinct value (no repeats). With this assumption, it can be shown that the eigenvectors  $\mathbf{v}_i$  form a linearly independent basis set, and therefore  $\mathbf{V}^{-1}$  is well-defined.

We can continue our eigenmodal analysis of the linear system by defining the modal coordinates,  $\mathbf{r}$ , with:

$$\mathbf{x} = \mathbf{V}\mathbf{r}, \quad \text{or} \quad \mathbf{r} = \mathbf{V}^{-1}\mathbf{x}.$$

In modal coordinates, the dynamics of the linear system are given by

$$\dot{\mathbf{r}} = \mathbf{V}^{-1}\mathbf{A}\mathbf{V}\mathbf{r} + \mathbf{V}^{-1}\mathbf{B}\mathbf{u} = \mathbf{\Lambda}\mathbf{r} + \mathbf{V}^{-1}\mathbf{B}\mathbf{u}.$$

This illustrates the power of modal analysis; in modal coordinates, the dynamics diagonalize yielding independent linear equations:

$$\dot{r}_i = \lambda_i r_i + \sum_j \beta_{ij} u_j, \quad \beta = \mathbf{V}^{-1} \mathbf{B}.$$

Now the concept of controllability becomes clear. Input  $j$  can influence the dynamics in modal coordinate  $i$  if and only if  $\beta_{ij} \neq 0$ . In the special case of non-repeated eigenvalues, having control over each individual eigenmode is sufficient to (in finite time) regulate all of the eigenmodes[5]. Therefore, we say that the system is controllable if and only if

$$\forall i, \exists j \text{ such that } \beta_{ij} \neq 0.$$

**A general solution** Included only for completeness. Click to expand the details.

A more general solution to the controllability issue, which removes our assumption about the eigenvalues, can be obtained by examining the time-domain solution of the linear equations. The solution of this system is

$$\mathbf{x}(t) = e^{\mathbf{A}t} \mathbf{x}(0) + \int_0^t e^{\mathbf{A}(t-\tau)} \mathbf{B} \mathbf{u}(\tau) d\tau.$$

Without loss of generality, let's consider the that the final state of the system is zero. Then we have:

$$\mathbf{x}(0) = - \int_0^{t_f} e^{-\mathbf{A}\tau} \mathbf{B} \mathbf{u}(\tau) d\tau.$$

You might be wondering what we mean by  $e^{\mathbf{A}t}$ ; a scalar raised to the power of a matrix..? Recall that  $e^z$  is actually defined by a convergent infinite sum:

$$e^z = 1 + z + \frac{1}{2}z^2 + \frac{1}{6}z^3 + \dots$$

The notation  $e^{\mathbf{A}t}$  uses the same definition:

$$e^{\mathbf{A}t} = \mathbf{I} + \mathbf{A}t + \frac{1}{2}(\mathbf{A}t)^2 + \frac{1}{6}(\mathbf{A}t)^3 + \dots$$

Not surprisingly, this has many special forms. For instance, if  $\mathbf{A}$  is diagonalizable,  $e^{\mathbf{A}t} = \mathbf{V} e^{\mathbf{\Lambda}t} \mathbf{V}^{-1}$ , where  $\mathbf{A} = \mathbf{V} \mathbf{\Lambda} \mathbf{V}^{-1}$  is the eigenvalue decomposition of  $\mathbf{A}$  [6]. The particular form we will use here is

$$e^{-\mathbf{A}\tau} = \sum_{k=0}^{n-1} \alpha_k(\tau) \mathbf{A}^k.$$

This is a particularly surprising form, because the infinite sum above is represented by this finite sum; the derivation uses Sylvester's Theorem [5], [7]. Then we have,

$$\begin{aligned} \mathbf{x}(0) &= - \sum_{k=0}^{n-1} \mathbf{A}^k \mathbf{B} \int_0^{t_f} \alpha_k(\tau) \mathbf{u}(\tau) d\tau \\ &= - \sum_{k=0}^{n-1} \mathbf{A}^k \mathbf{B} \mathbf{w}_k, \text{ where } \mathbf{w}_k = \int_0^{t_f} \alpha_k(\tau) \mathbf{u}(\tau) d\tau \\ &= - [\mathbf{B} \quad \mathbf{A}\mathbf{B} \quad \mathbf{A}^2\mathbf{B} \quad \dots \quad \mathbf{A}^{n-1}\mathbf{B}]_{n \times (nm)} \begin{bmatrix} \mathbf{w}_0 \\ \mathbf{w}_1 \\ \mathbf{w}_2 \\ \vdots \\ \mathbf{w}_{n-1} \end{bmatrix} \end{aligned}$$

The matrix containing the vectors  $\mathbf{B}$ ,  $\mathbf{AB}$ , ...  $\mathbf{A}^{n-1}\mathbf{B}$  is called the controllability matrix. In order for the system to be controllable, for every initial condition  $\mathbf{x}(0)$  we must be able to find the corresponding vector  $\mathbf{w}$ . This is only possible when the rows of the controllability matrix are linearly independent. Therefore, the condition of controllability is that this controllability matrix is full rank. In **DRAKE** you can obtain the controllability matrix for a linear system using `ControllabilityMatrix`, or check the rank condition directly using `IsControllable`.

Note that a linear feedback to change the eigenvalues of the eigenmodes is not sufficient to accomplish our goal of getting to the goal in finite time. In fact, the open-loop control to reach the goal is easily obtained with a final-value LQR problem, and (for  $\mathbf{R} = \mathbf{I}$ ) is actually a simple function of the controllability Gramian[7].

## Controllability vs. underactuated

Analysis of the controllability of both the Acrobot and Cart-Pole systems reveals that the linearized dynamics about the upright are, in fact, controllable. This implies that the linearized system, if started away from the zero state, can be returned to the zero state in finite time. This is potentially surprising - after all the systems are underactuated. For example, it is interesting and surprising that the Acrobot can balance itself in the upright position without having a shoulder motor.

The controllability of these model systems demonstrates an extremely important point: *An underactuated system is not necessarily an uncontrollable system.* Underactuated systems cannot follow arbitrary trajectories, but that does not imply that they cannot arrive at arbitrary points in state space. However, the trajectory required to place the system into a particular state may be arbitrarily complex.

The controllability analysis presented here is for linear time-invariant (LTI) systems. A comparable analysis exists for linear time-varying (LTV) systems. We will even see extensions to nonlinear systems; although it will often be referred to by the synonym of "reachability" analysis.

## Stabilizability of a linear system

Closely related to controllability is the notion of *stabilizability*. For linear systems, stabilizability is a strictly weaker condition than controllability (a system can be stabilizable but not controllable, but not the other way around).

**Definition 3.2 (Stabilizability of a linear system)** A control system of the form

$$\dot{\mathbf{x}} = \mathbf{Ax} + \mathbf{Bu}$$

is called *stabilizable* if it is possible to construct an unconstrained input signal,  $\mathbf{u}(t)$ ,  $t \in [0, \infty]$ , which results in

$$\lim_{t \rightarrow \infty} \mathbf{x}(t) = 0.$$

Controllability requires that we arrive at the origin in a finite time, stabilizability allows for asymptotic convergence. Essentially, a system can still be stabilizable if the uncontrollable subspace is naturally stable.

Interestingly, for nonlinear systems the relationship between stabilizability and controllability is much more subtle. In a famous (sometimes misquoted) result from Roger Brockett states that nonlinear controllability does not necessarily imply stabilizability by differentiable control policies[8].

### 3.4.3 LQR feedback

Controllability and stabilizability tell us that a trajectory to the fixed point exists, but

does not tell us which one we should take or what control inputs cause it to occur. Why not? There are potentially infinitely many solutions. We have to pick one.

The tools for controller design in linear systems are very advanced. In particular, as we describe in the [linear optimal control chapter](#), one can easily design an optimal feedback controller for a regulation task like balancing, so long as we are willing to linearize the system around the operating point and define optimality in terms of a quadratic cost function:

$$J(\mathbf{x}_0) = \int_0^\infty [\mathbf{x}^T(t)\mathbf{Q}\mathbf{x}(t) + \mathbf{u}^T(t)\mathbf{R}\mathbf{u}(t)] dt, \quad \mathbf{x}(0) = \mathbf{x}_0, \mathbf{Q} = \mathbf{Q}^T > 0, \mathbf{R} = \mathbf{R}^T > 0.$$

The linear feedback matrix  $\mathbf{K}$  used as

$$\mathbf{u}(t) = -\mathbf{K}\mathbf{x}(t),$$

is the so-called optimal linear quadratic regulator (LQR). Even without understanding the detailed derivation, we can quickly become practitioners of LQR. `DRAKE` provides a function,

```
K = LinearQuadraticRegulator(A, B, Q, R)
```

for linear systems. It also provides a version

```
controller = LinearQuadraticRegulator(system, context, Q, R)
```

that will linearize the system for you around an equilibrium and return the controller (in the original coordinates). Therefore, to use LQR, one simply needs to define the symmetric positive-definite cost matrices,  $\mathbf{Q}$  and  $\mathbf{R}$ . In their most common form,  $\mathbf{Q}$  and  $\mathbf{R}$  are positive diagonal matrices, where the entries  $Q_{ii}$  penalize the relative errors in state variable  $x_i$  compared to the other state variables, and the entries  $R_{ii}$  penalize actions in  $u_i$ .

Take a moment to appreciate this. If the linearized system is stabilizable, then for any (positive semi-definite)  $\mathbf{Q}$  and (positive definite)  $\mathbf{R}$  matrices, LQR will give us a stabilizing controller. If the system is not stabilizable, then LQR will tell us that no linear controller exists. Pretty amazing!

### Example 3.5 (LQR for the Acrobot and Cart-Pole)

Take a minute to play around with the LQR controller for the Acrobot and the Cart-Pole

Make sure that you take a minute to look at the code which runs during these examples. Can you set the  $\mathbf{Q}$  and  $\mathbf{R}$  matrices differently, to improve the performance?

Simulation of the closed-loop response with LQR feedback shows that the task is indeed completed - and in an impressive manner. Oftentimes the state of the system has to move violently away from the origin in order to ultimately reach the origin. Further inspection reveals the (linearized) closed-loop dynamics are in fact non-minimum phase (acrobot has 3 right-half zeros, cart-pole has 1).

### Example 3.6 (LQR for Quadrotors)

LQR works essentially out of the box for Quadrotors, if linearized around a nominal fixed point (where the non-zero thrust from the propellers is balancing gravity).

or [Click here for the animation](#).



Note that LQR, although it is optimal for the linearized system, is not necessarily the best linear control solution for maximizing basin of attraction of the fixed-point. The theory of *robust control* (e.g., [9]), which explicitly takes into account the differences between the linearized model and the nonlinear model, will produce controllers which outperform our LQR solution in this regard.

## 3.5 PARTIAL FEEDBACK LINEARIZATION



In the introductory chapters, we made the point that the underactuated systems are not feedback equivalent to  $\ddot{q} = u$ . Although we cannot always simplify the full dynamics of the system, it is still possible to linearize a portion of the system dynamics. The technique is called *partial feedback linearization*.

Consider the cart-pole example. The dynamics of the cart are affected by the motions of the pendulum. If we know the model, then it seems quite reasonable to think that we could create a feedback controller which would push the cart in exactly the way necessary to counter-act the dynamic contributions from the pendulum - thereby linearizing the cart dynamics. What we will see, which is potentially more surprising, is that we can also use a feedback controller for the cart to feedback linearize the dynamics of the passive pendulum joint.

We'll use the term *collocated* partial feedback linearization to describe a controller which linearizes the dynamics of the actuated joints. What's more surprising is that it is often possible to achieve *non-collocated* partial feedback linearization - a controller which linearizes the dynamics of the unactuated joints. The treatment presented here follows from [10].

### 3.5.1 PFL for the Cart-Pole System

#### Collocated

Starting from the equations 18 and 19, we have

$$\begin{aligned}\ddot{\theta} &= -\ddot{x}_c - s \\ \ddot{x}(2 - c^2) - sc - \dot{\theta}^2 s &= f_x\end{aligned}$$

Therefore, applying the feedback control

$$f_x = (2 - c^2)\ddot{x}^d - sc - \dot{\theta}^2 s \quad (30)$$

results in

$$\begin{aligned}\ddot{x} &= \ddot{x}^d \\ \ddot{\theta} &= -\ddot{x}^d c - s,\end{aligned}$$

which are valid globally.

Look carefully at the resulting equations. Of course, it says that we can impose whatever accelerations we like on the cart. But even the resulting equations of the pole happen to take a nice form here: they have been reduced to the equations of the simple pendulum (without a cart), where the torque input is now given instead by  $\ddot{x}_c$ . It's as if we have a simple pendulum with torque control, except our command is modulated by a  $\cos\theta$  term, and this  $\cos\theta$  term is fundamental -- it's true that our control authority goes to zero when the pole is horizontal, because no amount of force on the cart in that configuration can act like a torque on the pole.

#### Non-collocated

Starting again from equations 18 and 19, we have



$$\ddot{x} = -\frac{\ddot{\theta} + s}{c}$$

$$\ddot{\theta}(c - \frac{2}{c}) - 2 \tan \theta - \dot{\theta}^2 s = f_x$$

Applying the feedback control

$$f_x = (c - \frac{2}{c})\ddot{\theta}^d - 2 \tan \theta - \dot{\theta}^2 s \quad (31)$$

results in

$$\ddot{\theta} = \ddot{\theta}^d$$

$$\ddot{x} = -\frac{1}{c}\ddot{\theta}^d - \tan \theta.$$

Note that this expression is only valid when  $\cos \theta \neq 0$ . Once again, we know that the force cannot create a torque when the pole is perfectly horizontal. In fact, the controller we have written will "blow-up" -- requesting infinite force at  $\cos \theta = 0$ ; so make sure you saturate the command before you implement it on hardware (or even in simulation). Although it may come as a small consolation, at least we have that  $(c - \frac{2}{c})$  never goes to zero; in fact you can check for yourself that  $|c - \frac{2}{c}| \geq 1$ .

### 3.5.2 General form

For systems that are trivially underactuated (torques on some joints, no torques on other joints), we can, without loss of generality, reorganize the joint coordinates in any underactuated system described by the manipulator equations into the form:

$$\mathbf{M}_{11}\ddot{\mathbf{q}}_1 + \mathbf{M}_{12}\ddot{\mathbf{q}}_2 = \boldsymbol{\tau}_1, \quad (32)$$

$$\mathbf{M}_{21}\ddot{\mathbf{q}}_1 + \mathbf{M}_{22}\ddot{\mathbf{q}}_2 = \boldsymbol{\tau}_2 + \mathbf{u}, \quad (33)$$

with  $\mathbf{q} \in \mathbb{R}^n$ ,  $\mathbf{q}_1 \in \mathbb{R}^l$ ,  $\mathbf{q}_2 \in \mathbb{R}^m$ ,  $l = n - m$ .  $\mathbf{q}_1$  represents all of the passive joints, and  $\mathbf{q}_2$  represents all of the actuated joints, and the  $\boldsymbol{\tau} = \boldsymbol{\tau}_g - \mathbf{C}\dot{\mathbf{q}}$  terms capture all of the Coriolis and gravitational terms, and

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} \mathbf{M}_{11} & \mathbf{M}_{12} \\ \mathbf{M}_{21} & \mathbf{M}_{22} \end{bmatrix}.$$

Fortunately, because  $\mathbf{M}$  is uniformly (e.g. for all  $\mathbf{q}$ ) positive definite,  $\mathbf{M}_{11}$  and  $\mathbf{M}_{22}$  are also positive definite, by the [Schur complement condition for positive definiteness](#).

### Collocated linearization

Performing the same substitutions into the full manipulator equations, we get:

$$\ddot{\mathbf{q}}_1 = \mathbf{M}_{11}^{-1} [\boldsymbol{\tau}_1 - \mathbf{M}_{12}\ddot{\mathbf{q}}_2] \quad (34)$$

$$(\mathbf{M}_{22} - \mathbf{M}_{21}\mathbf{M}_{11}^{-1}\mathbf{M}_{12})\ddot{\mathbf{q}}_2 - \boldsymbol{\tau}_2 + \mathbf{M}_{21}\mathbf{M}_{11}^{-1}\boldsymbol{\tau}_1 = \mathbf{u} \quad (35)$$

It can be easily shown that the matrix  $(\mathbf{M}_{22} - \mathbf{M}_{21}\mathbf{M}_{11}^{-1}\mathbf{M}_{12})$  is invertible[10]; we can see from inspection that it is symmetric. PFL follows naturally, and is valid globally.

### Non-collocated linearization

$$\ddot{\mathbf{q}}_2 = \mathbf{M}_{12}^+ [\boldsymbol{\tau}_1 - \mathbf{M}_{11}\ddot{\mathbf{q}}_1] \quad (36)$$

$$(\mathbf{M}_{21} - \mathbf{M}_{22}\mathbf{M}_{12}^+\mathbf{M}_{11})\ddot{\mathbf{q}}_1 - \boldsymbol{\tau}_2 + \mathbf{M}_{22}\mathbf{M}_{12}^+\boldsymbol{\tau}_1 = \mathbf{u} \quad (37)$$

where  $\mathbf{M}_{12}^+$  is a Moore-Penrose pseudo-inverse. This inverse provides a unique solution when the rank of  $\mathbf{M}_{12}$  equals  $l$ , the number of passive degrees of freedom in the system (it cannot be more, since the matrix only has  $l$  rows). The rank condition in this context is sometimes called the property of "Strong Inertial Coupling". It is

state dependent; in the cart-pole example above  $\mathbf{M}_{12} = \cos \theta$  and drops rank exactly when  $\cos \theta = 0$ . A system has Global Strong Inertial Coupling if it exhibits Strong Inertial Coupling in every state.

## Task-space partial feedback linearization

In general, we can define some combination of active and passive joints that we would like to control. This combination is sometimes called a "task space". Consider an output function of the form,

$$\mathbf{y} = \mathbf{h}(\mathbf{q}),$$

with  $\mathbf{y} \in \mathbb{R}^p$ , which defines the task space. Define  $\mathbf{H}_1 = \frac{\partial \mathbf{h}}{\partial \mathbf{q}_1}$ ,  $\mathbf{H}_2 = \frac{\partial \mathbf{h}}{\partial \mathbf{q}_2}$ ,  $\mathbf{H} = [\mathbf{H}_1, \mathbf{H}_2]$ .

### Theorem 3.1 - Task Space PFL

If the actuated joints are commanded so that

$$\ddot{\mathbf{q}}_2 = \bar{\mathbf{H}}^+ [\ddot{\mathbf{y}}^d - \dot{\mathbf{H}}\dot{\mathbf{q}} - \mathbf{H}_1 \mathbf{M}_{11}^{-1} \tau_1], \quad (38)$$

where  $\bar{\mathbf{H}} = \mathbf{H}_2 - \mathbf{H}_1 \mathbf{M}_{11}^{-1} \mathbf{M}_{12}$ . and  $\bar{\mathbf{H}}^+$  is the right Moore-Penrose pseudo-inverse,

$$\bar{\mathbf{H}}^+ = \bar{\mathbf{H}}^T (\bar{\mathbf{H}} \bar{\mathbf{H}}^T)^{-1},$$

then we have

$$\ddot{\mathbf{y}} = \ddot{\mathbf{y}}^d. \quad (39)$$

subject to

$$\text{rank}(\bar{\mathbf{H}}) = p, \quad (40)$$

**Proof Sketch.** Differentiating the output function we have

$$\begin{aligned} \dot{\mathbf{y}} &= \mathbf{H}\dot{\mathbf{q}} \\ \ddot{\mathbf{y}} &= \dot{\mathbf{H}}\dot{\mathbf{q}} + \mathbf{H}_1 \ddot{\mathbf{q}}_1 + \mathbf{H}_2 \ddot{\mathbf{q}}_2. \end{aligned}$$

Solving 32 for the dynamics of the unactuated joints we have:

$$\ddot{\mathbf{q}}_1 = \mathbf{M}_{11}^{-1}(\tau_1 - \mathbf{M}_{12}\ddot{\mathbf{q}}_2) \quad (41)$$

Substituting, we have

$$\ddot{\mathbf{y}} = \dot{\mathbf{H}}\dot{\mathbf{q}} + \mathbf{H}_1 \mathbf{M}_{11}^{-1}(\tau_1 - \mathbf{M}_{12}\ddot{\mathbf{q}}_2) + \mathbf{H}_2 \ddot{\mathbf{q}}_2 \quad (42)$$

$$= \dot{\mathbf{H}}\dot{\mathbf{q}} + \bar{\mathbf{H}}\ddot{\mathbf{q}}_2 + \mathbf{H}_1 \mathbf{M}_{11}^{-1} \tau_1 \quad (43)$$

$$= \ddot{\mathbf{y}}^d \quad (44)$$

Note that the last line required the rank condition (40) on  $\bar{\mathbf{H}}$  to ensure that the rows of  $\bar{\mathbf{H}}$  are linearly independent, allowing  $\bar{\mathbf{H}}\bar{\mathbf{H}}^+ = \mathbf{I}$ .

In order to execute a task space trajectory one could command

$$\ddot{\mathbf{y}}^d = \mathbf{K}_d(\dot{\mathbf{y}}^d - \dot{\mathbf{y}}) + \mathbf{K}_p(\mathbf{y}^d - \mathbf{y}).$$

Assuming the internal dynamics are stable, this yields converging error dynamics when  $\mathbf{K}_p, \mathbf{K}_d > 0$  [4], which implies  $\mathbf{y}(t) \rightarrow \mathbf{y}^d(t)$ . For a position control robot, the acceleration command of (38) suffices. Alternatively, a torque command follows by substituting (38) and (41) into (33).

### Example 3.7 (End-point trajectory following with the Cart-Pole system)

Consider the task of trying to track a desired kinematic trajectory with the endpoint of pendulum in the cart-pole system. With one actuator and kinematic constraints, we might be hard-pressed to track a trajectory in both the horizontal and vertical coordinates. But we can at least try to track a trajectory in the vertical position of the end-effector.

Using the task-space PFL derivation, we have:

$$\begin{aligned} y &= h(\mathbf{q}) = -l \cos \theta \\ \dot{y} &= l \dot{\theta} \sin \theta \end{aligned}$$

If we define a desired trajectory:

$$y^d(t) = \frac{l}{2} + \frac{l}{4} \sin(t),$$

then the task-space controller is easily implemented using the derivation above.

The task space derivation above provides a convenient generalization of the partial feedback linearization (PFL) [10], which encompasses both the collocated and non-collocated results. If we choose  $\mathbf{y} = \mathbf{q}_2$  (collocated), then we have

$$\mathbf{H}_1 = 0, \mathbf{H}_2 = \mathbf{I}, \dot{\mathbf{H}} = 0, \bar{\mathbf{H}} = \mathbf{I}, \bar{\mathbf{H}}^+ = \mathbf{I}.$$

From this, the command in (38) reduces to  $\ddot{\mathbf{q}}_2 = \ddot{\mathbf{q}}_2^d$ . The actuator command is then

$$\mathbf{u} = \mathbf{M}_{21} \mathbf{M}_{11}^{-1} (\tau_1 - \mathbf{M}_{12} \ddot{\mathbf{q}}_2^d) + \mathbf{M}_{22} \ddot{\mathbf{q}}_2^d - \tau_2,$$

and the rank condition (40) is always met.

If we choose  $\mathbf{y} = \mathbf{q}_1$  (non-collocated), we have

$$\mathbf{H}_1 = \mathbf{I}, \mathbf{H}_2 = 0, \dot{\mathbf{H}} = 0, \bar{\mathbf{H}} = -\mathbf{M}_{11}^{-1} \mathbf{M}_{12}.$$

The rank condition (40) requires that  $\text{rank}(\mathbf{M}_{12}) = l$ , in which case we can write  $\bar{\mathbf{H}}^+ = -\mathbf{M}_{12}^+ \mathbf{M}_{11}$ , reducing the rank condition to precisely the "Strong Inertial Coupling" condition described in [10]. Now the command in (38) reduces to

$$\ddot{\mathbf{q}}_2 = \mathbf{M}_{12}^+ [\tau_1 - \mathbf{M}_{11} \ddot{\mathbf{q}}_1^d] \quad (45)$$

The actuator command is found by substituting (45) into (33), yielding the same results as in [10].

## 3.6 SWING-UP CONTROL

### 3.6.1 Energy shaping

Recall in the last chapter, [we used energy shaping to swing up the simple pendulum](#). This idea turns out to be a bit more general than just for the simple pendulum. As we will see, we can use similar concepts of "energy shaping" to produce swing-up controllers for the acrobot and cart-pole systems. It's important to note that it only takes one actuator to change the total energy of a system.

Although a large variety of swing-up controllers have been proposed for these model systems (c.f. [11], [12], [13], [14], [15], [16], [1], [17]), the energy shaping controllers tend to be the most natural to derive and perhaps the most well-known.

### 3.6.2 Cart-Pole

Let's try to apply the energy-shaping ideas to the cart-pole system. The basic idea, from [18], is to use collocated PFL to simplify the dynamics, use energy shaping to regulate the pendulum to its homoclinic orbit, then to add a few terms to make sure that the cart stays near the origin. This is a bit surprising... if we want to control the pendulum, shouldn't we use the non-collocated version? Actually, we ultimately want to control both the cart and the pole, and we will build on the collocated version both because it avoids inverting the  $\cos \theta$  term that can go to zero and because (when all parameters are set to 1) we were left with a particularly simple form of the equations:

$$\ddot{x} = u \quad (46)$$

$$\ddot{\theta} = -uc - s. \quad (47)$$

The first equation is clearly simple, but even the second equation is exactly the equations of a decoupled pendulum, just with a slightly odd control input that is modulated by  $\cos \theta$ .

Let us regulate the energy of this decoupled pendulum using energy shaping. The energy of the pendulum (a unit mass, unit length, simple pendulum in unit gravity) is given by:

$$E(\mathbf{x}) = \frac{1}{2}\dot{\theta}^2 - \cos \theta.$$

The desired energy, equivalent to the energy at the desired fixed-point, is

$$E^d = 1.$$

Again defining  $\tilde{E}(\mathbf{x}) = E(\mathbf{x}) - E^d$ , we now observe that

$$\begin{aligned} \dot{\tilde{E}}(\mathbf{x}) &= \dot{E}(\mathbf{x}) = \dot{\theta}\ddot{\theta} + \dot{\theta}s \\ &= \dot{\theta}[-uc - s] + \dot{\theta}s \\ &= -u\dot{\theta}\cos \theta. \end{aligned}$$

Therefore, if we design a controller of the form

$$u = k\dot{\theta}\cos \theta\tilde{E}, \quad k > 0$$

the result is

$$\dot{\tilde{E}} = -k\dot{\theta}^2 \cos^2 \theta \tilde{E}.$$

This guarantees that  $|\tilde{E}|$  is non-increasing, but isn't quite enough to guarantee that it will go to zero. For example, if  $\theta = \dot{\theta} = 0$ , the system will never move. However, if we have that

$$\int_0^t \dot{\theta}^2(t') \cos^2 \theta(t') dt' \rightarrow \infty, \quad \text{as } t \rightarrow \infty,$$

then we have  $\tilde{E}(t) \rightarrow 0$ . This condition, a version of the LaSalle's theorem that we will develop in our notes on Lyapunov methods, is satisfied for all but the trivial constant trajectories at fixed points.

Now we return to the full system dynamics (which includes the cart). [18] and [19] use the simple pendulum energy controller with an addition PD controller designed to regulate the cart:

$$\ddot{x}^d = k_E \dot{\theta} \cos \theta \tilde{E} - k_p x - k_d \dot{x}.$$

[18] provides a proof of convergence for this controller with some nominal

parameters. [11] provides a slightly different controller derived directly from a Lyapunov argument.

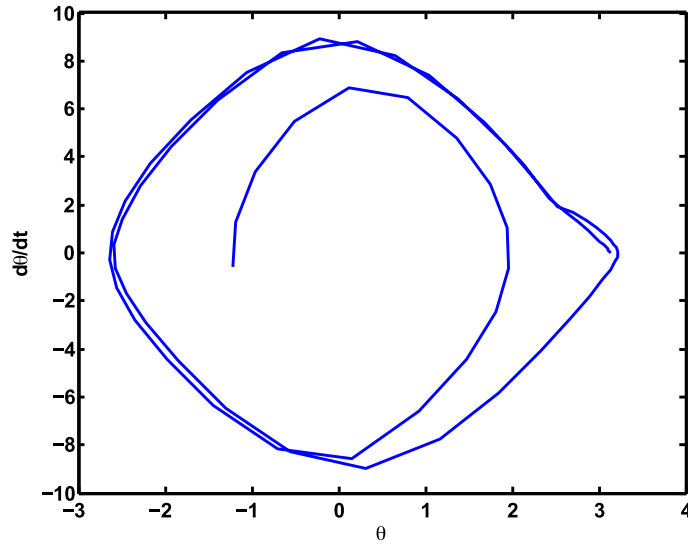


Figure 3.4 - Cart-Pole Swingup: Example phase plot of the pendulum subsystem using energy shaping control. The controller drives the system to the homoclinic orbit, then switches to an LQR balancing controller near the top.

### 3.6.3 Acrobot

Swing-up control for the acrobot can be accomplished in much the same way. [14] - pump energy. Clean and simple. No proof. Slightly modified version (uses arctan instead of sat) in [20]. Clearest presentation in [19].

Use collocated PFL. ( $\ddot{q}_2 = \ddot{q}_2^d$ ).

$$E(\mathbf{x}) = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{M} \dot{\mathbf{q}} + U(\mathbf{x}).$$

$$E_d = U(\mathbf{x}^*).$$

$$\bar{u} = \dot{q}_1 \tilde{E}.$$

$$\ddot{q}_2^d = -k_1 q_2 - k_2 \dot{q}_2 + k_3 \bar{u},$$

Extra PD terms prevent proof of asymptotic convergence to homoclinic orbit. Proof of another energy-based controller in [13].

### 3.6.4 Discussion

The energy shaping controller for swing-up presented here is a pretty faithful representative from the field of nonlinear underactuated control. Typically these control derivations require some clever tricks for simplifying or canceling out terms in the nonlinear equations, then some clever Lyapunov function to prove stability. In these cases, PFL was used to simplify the equations, and therefore the controller design.

We will see another nice example of changing coordinates in the nonlinear equations to make the problem easier when we study "differential flatness" for trajectory optimization. This approach is perhaps most famous these days for very dynamic trajectory planning with quadrotors.

These controllers are important, representative, and relevant. But clever tricks

with nonlinear equations seem to be fundamentally limited. Most of the rest of the material presented in this book will emphasize more general computational approaches to formulating and solving these and other control problems.

### 3.7 OTHER MODEL SYSTEMS

The acrobot and cart-pole systems are just two of the model systems used heavily in underactuated control research. Other examples include:

- Pendubot
- Inertia wheel pendulum
- Furuta pendulum (horizontal rotation and vertical pendulum)
- Hovercraft

### 3.8 EXERCISES

#### Exercise 3.1 (Cart-Pole: Linearization and Balancing)

For this exercise you will work exclusively in . You will be asked to complete the following steps.

- Derive the state-space dynamics  $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$  of [the cart-pole system](#).
- Linearize the dynamics from point (a) around the unstable equilibrium point.
- Analyze the linearization error for different values of the state  $\mathbf{x}$  and the control  $\mathbf{u}$ .
- Among the states from point (c), identify which ones are stabilized to the origin by the LQR controller.

#### Exercise 3.2 (Cart-Poles: Writing URDFs and balancing with LQR)

For this exercise you will work in . You will complete the following steps.

- Construct the cart-pole pendulum URDF structure for the [single pendulum cart-pole](#).
- Extend the single pendulum cart-pole to a double pendulum cart-pole by modifying the URDF model of the robot and testing LQR's ability to control it.

#### Exercise 3.3 (Controllability of Discrete and Continuous LTI Systems)

- (2-D grid with discrete double integrator) Consider a discretized grid world where

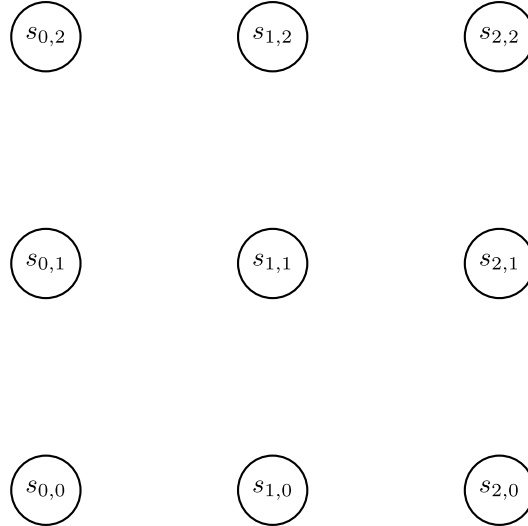
$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \text{ and } x_1[n], x_2[n] \in \mathbb{Z}.$$

The dynamics of  $x[n]$  is a discrete-time version of double integrator,

$$\mathbf{x}[n+1] = \begin{bmatrix} x_1[n] + x_2[n] \\ x_2[n] + u[n] \end{bmatrix},$$

where the control input  $u[n] \in \mathbb{Z}$  is also an integer.

- Suppose we represent the discretized double integrator dynamics on the following graph, where the nodes  $s_{i,j}$  represent state vector values ( $\mathbf{x} = s_{i,j}$  means  $x_1 = i, x_2 = j$ ) and edges are state transitions defined by a control law. In the following truncated graph, draw the edges when  $u = -1$ ,  $u = 0$ , and  $u = 1$ . (Please draw edges for all the nine nodes in the truncated graph. If a node transitions to another node outside the graph, please draw the destination node as well.)



- Suppose we start from  $s_{0,0}$ , construct a control input sequence ( $u[n]$  could be a function of time step and can take any arbitrary integer values) to reach an arbitrary state  $s_{i,j}$  that satisfies  $j > i > 0$ . What would be the minimal number of time steps needed to reach any  $s_{i,j}$  that satisfies  $j > i > 0$ ? Show that the number you've chosen is indeed minimum.
  - Now consider a more general case, from an arbitrary initial state  $s_{i_1,j_1}$ , can you find a control input sequence  $u[n]$  with the same number of time steps as in (2) that allows the state to reach any final state  $s_{i_2,j_2}$ ? Also show that the number you've chosen is minimum.
- b.
- Considering linear systems

$$\dot{\mathbf{x}} = \mathbf{Ax} + \mathbf{Bu},$$

are the following linear systems controllable?

$$\mathbf{A}_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \mathbf{B}_1 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}; \quad \mathbf{A}_2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \mathbf{B}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix};$$

$$\mathbf{A}_3 = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \mathbf{B}_3 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}; \quad \mathbf{A}_4 = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 2 & 0 & 0 \end{bmatrix}, \mathbf{B}_4 = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}.$$

You should only need the conditions described in [the definition of controllability](#) to explain your conclusion. You can also use the more general tools found in [this collapsable section](#); these will prove especially for the  $(\mathbf{A}_4, \mathbf{B}_4)$  matrix pair.



2. Consider the two systems,  $(\mathbf{A}_3, \mathbf{B}_3)$  and  $(\mathbf{A}_4, \mathbf{B}_4)$ . Can you make a conclusion whether these two systems are underactuated? (Note that the definition of underactuation requires the system to be interpreted as a second-order system, i.e.,  $\mathbf{x} = [\mathbf{q}, \dot{\mathbf{q}}]^T$ )

### Exercise 3.4 (Comparison between stability of nonlinear system and linearization)

In this problem, we use phase portraits to examine how linearization technique helps with local stability analysis for nonlinear systems and its limitations using the nonlinear pendulum with zero torque as an example. The pendulum parameters are chosen as  $m = 1$ ,  $l = 1$ ,  $g = 9.81$ . Follow the exercises below and complete the graphical analysis for equilibrium points  $\mathbf{x}^* = [0, 0]^T$ ,  $\mathbf{x}^* = [\pi, 0]^T$ , which will guide you to studying the systems. When you are done, populate this table:

|                                                           | sign<br>( $Re(\lambda_1)$ ) | sign<br>( $Re(\lambda_2)$ ) | stable<br>i.s.L. | Asymp.<br>Stable | Exp.<br>Stable |
|-----------------------------------------------------------|-----------------------------|-----------------------------|------------------|------------------|----------------|
| Nonlinear<br>$b = 0$ ,<br>$\mathbf{x}^* = [0, 0]^T$       | N/A                         | N/A                         |                  |                  |                |
| Nonlinear<br>$b = 0$ ,<br>$\mathbf{x}^* = [\pi, 0]^T$     | N/A                         | N/A                         |                  |                  |                |
| Linearization<br>$b = 0$ ,<br>$\mathbf{x}^* = [0, 0]^T$   |                             |                             |                  |                  |                |
| Linearization<br>$b = 0$ ,<br>$\mathbf{x}^* = [\pi, 0]^T$ |                             |                             |                  |                  |                |
| Nonlinear<br>$b = 1$ ,<br>$\mathbf{x}^* = [0, 0]^T$       | N/A                         | N/A                         |                  |                  |                |
| Nonlinear<br>$b = 1$ ,<br>$\mathbf{x}^* = [\pi, 0]^T$     | N/A                         | N/A                         |                  |                  |                |
| Linearization<br>$b = 1$ ,<br>$\mathbf{x}^* = [0, 0]^T$   |                             |                             |                  |                  |                |
| Linearization<br>$b = 1$ ,<br>$\mathbf{x}^* = [\pi, 0]^T$ |                             |                             |                  |                  |                |

- a. Let's first look at an undamped pendulum  $b = 0$ .
  1. Using as an example, draw the phase portrait using `plot_2d_phase_portrait` function around equilibrium  $\mathbf{x}^*$ . Is the nonlinear undamped pendulum stable i.s.L.? asymptotically stable? exponentially stable?
  2. Derive the linearization of the undamped nonlinear pendulum around equilibrium  $\mathbf{x}^*$ . Compute the eigenvalues of the Jacobian  $\left[\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right]_{\mathbf{x}=\mathbf{x}^*}$ , what are the signs of the real parts of these two eigenvalues,  $\text{sign}(Re(\lambda_1))$ ,  $\text{sign}(Re(\lambda_2))$ ?
  3. Using phase portrait, is the linearized system  $\dot{\mathbf{x}} = \left[\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right]_{\mathbf{x}=\mathbf{x}^*}(\mathbf{x} - \mathbf{x}^*)$  stable i.s.L.? exponentially stable?

- b. Now let's consider a pendulum with damping coefficient  $b = 1$ .
  1. Using as an example, draw the phase portrait using `plot_2d_phase_portrait` function around equilibrium  $\mathbf{x}^*$ . Is the nonlinear damped pendulum stable i.s.L.? asymptotically stable? exponentially stable?
  2. Derive the linearization of the damped nonlinear pendulum around equilibrium  $\mathbf{x}^*$ . Compute the eigenvalues of the Jacobian  $\left[\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right]_{\mathbf{x}=\mathbf{x}^*}$ , what are the signs of the real parts of these two eigenvalues,  $\text{sign}(\text{Re}(\lambda_1))$ ,  $\text{sign}(\text{Re}(\lambda_2))$ ?
  3. Using phase portrait, is the linearized system  $\dot{\mathbf{x}} = \left[\frac{\partial \mathbf{f}}{\partial \mathbf{x}}\right]_{\mathbf{x}=\mathbf{x}^*}(\mathbf{x} - \mathbf{x}^*)$  stable i.s.L.? exponentially stable?

## REFERENCES

1. Richard M. Murray and John Hauser, "A case Study in Approximate Linearization: The Acrobot Example", *Memorandum No. UCB/ERL (unknown)*, April, 1991.
2. M. W. Spong, "Underactuated Mechanical Systems", *Control Problems in Robotics and Automation*, 1997.
3. Hassan K. Khalil, "Nonlinear Systems", Prentice Hall, December, 2001.
4. Jean-Jacques E. Slotine and Weiping Li, "Applied Nonlinear Control", Prentice Hall, October, 1990.
5. Katsuhiko Ogata, "Modern Control Engineering", Prentice Hall Incorporated, August, 1996.
6. Gilbert Strang, "Introduction to Linear Algebra", Wellesley-Cambridge Press, October, 1998.
7. Chi-Tsong Chen, "Linear System Theory and Design", Oxford University Press, Sept 10, 1998.
8. R.W. Brockett, "Asymptotic stability and feedback stabilization", *Differential Geometric Control Theory*, pp. 181-191, 1983.
9. Kemin Zhou and John C. Doyle, "Essentials of Robust Control", Prentice Hall, 1997.
10. Mark Spong, "Partial feedback linearization of underactuated mechanical systems", *Proceedings of the IEEE International Conference on Intelligent Robots and Systems*, vol. 1, pp. 314-321, September, 1994.
11. Isabelle Fantoni and Rogelio Lozano, "Non-linear Control for Underactuated Mechanical Systems", Springer-Verlag, 2002.
12. Araki and N.; Okada and M.; Konishi and Y.; Ishigaki and H., "Parameter identification and swing-up control of an Acrobot system", *International Conference on International Technology*, 2005.
13. Xin Xin and M. Kaneda, "New analytical results of the energy based swinging up control of the Acrobot", *Proceedings of the 43rd IEEE Conference on Decision and Control (CDC)*, vol. 1, pp. 704 - 709, Dec, 2004.
14. Mark W. Spong, "Swing up control of the Acrobot", *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp.

2356-2361, 1994.

15. Arun D. Mahindrakar and Ravi N. Banavar, "A swing-up of the acrobot based on a simple pendulum strategy", *International Journal of Control*, vol. 78, no. 6, pp. 424-429, April, 2005.
16. M.D. Berkemeier and R.S. Fearing, "Tracking fast inverted trajectories of the underactuated Acrobot", *Robotics and Automation, IEEE Transactions on*, vol. 15, no. 4, pp. 740-750, Aug, 1999.
17. Xu-Zhi Lai and Jin-Hua She and Simon X. Yang and Min Wu, "Stability Analysis and Control Law Design for Acrobots", *Proceedings of the 2006 IEEE International Conference on Robotics and Automation*, May, 2006.
18. Chung Choo Chung and John Hauser, "Nonlinear Control of a Swinging Pendulum", *Automatica*, vol. 31, no. 6, pp. 851-862, June, 1995.
19. Mark W. Spong, "Energy Based Control of a Class of Underactuated Mechanical Systems", *Proceedings of the 1996 IFAC World Congress*, 1996.
20. Mark Spong, "The Swingup Control Problem for the Acrobot", *IEEE Control Systems Magazine*, vol. 15, no. 1, pp. 49-55, February, 1995.

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

[Accessibility](#)

© Russ Tedrake, 2024

# UNDERACTUATED ROBOTICS

## *Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 4

# Simple Models of Walking and Running

Practical legged locomotion is one of the fundamental problems in robotics; we've seen amazing progress over the last few years, but there are still some major unsolved problems. Much of the recent progress is due to improvements in hardware -- a legged robot must carry all of its sensors, actuators and power and traditionally this meant underpowered motors that act as far-from-ideal force/torque sources -- Boston Dynamics and other companies have made incredible progress here. The control systems implemented on these systems, though, are still more heuristic than you might think. They also require dramatically higher bandwidth and lower latency than the human motor control system and still perform worse in challenging environments.

The control of walking robots is definitely underactuated: assuming we cannot pull on the ground (and barring any aerodynamic effects!), then no matter how powerful my actuators are, there is nothing that we can do to accelerate the center of mass of the robot towards the ground faster than gravity. But beyond the underactuated systems we have studied so far, the control of walking robots faces an additional complexity: **controlling through contact**. The fact that we can get non-zero contact forces if and only if two bodies are in contact introduces a fundamentally discrete/combinatorial element into the problem<sup>†</sup>.

Contact is essential for many aspects of robotics (manipulation is my other favorite example); it's sad to see so many robots going through life avoiding contact at any cost. Controlling contact means that your robot is capable of performing physical work on the environment; isn't that the point of robotics, afterall?

I like to start our discussion of contact with walking robots for a few important reasons. As you'll see in this chapter, there are a number of elegant "simple models" of walking that capture much of the essence of the problem with minimal complexity. But there is another, more subtle, reason: I want to continue to use the language of stability. At this point we've seen both finite-horizon and infinite-horizon problem formulations -- when we can say something about the behavior of a system in the limit as time goes to infinity, it's almost always cleaner than a finite-time analysis (because time drops away). Walking provides a very clean way to use the language of stability in studying behavior of systems that are sometimes in contact but

<sup>†</sup>I've had some interesting arguments with folks on this point because it's possible to write contact models that smooth the discontinuity, and/or to model systems that have negligible collision events. But replacing discontinuity in the vector field with a stiff but smooth transition does not remove the need to decide whether or not your robot should

sometimes out of contact in the limiting case; we just need to extend our notions from stability of a fixed point to the stability of a (periodic) cycle.

## 4.1 LIMIT CYCLES

A limit cycle is an periodic orbit that is a **limit set** of the dynamical system; they can be stable (the limit set of neighboring trajectories as time goes to infinity) or unstable (the limit set of neighboring trajectories as time goes to negative infinity). One of the simplest models of limit cycle behavior is the Van der Pol oscillator. Let's examine that first...

### Example 4.1 (Van der Pol Oscillator)

The Van der Pol oscillator is described by a second-order differential equation in one variable:

$$\ddot{q} + \mu(q^2 - 1)\dot{q} + q = 0, \quad \mu > 0$$

One can think of this system as almost a simple **spring-mass-damper** system, except that it has nonlinear damping. In particular, the velocity term dissipates energy when  $|q| > 1$ , and adds energy when  $|q| < 1$ . Therefore, it is not terribly surprising to see that the system settles into a stable oscillation from almost any initial conditions (the exception is the state  $q = 0, \dot{q} = 0$ ). This can be seen nicely in the phase portrait below.

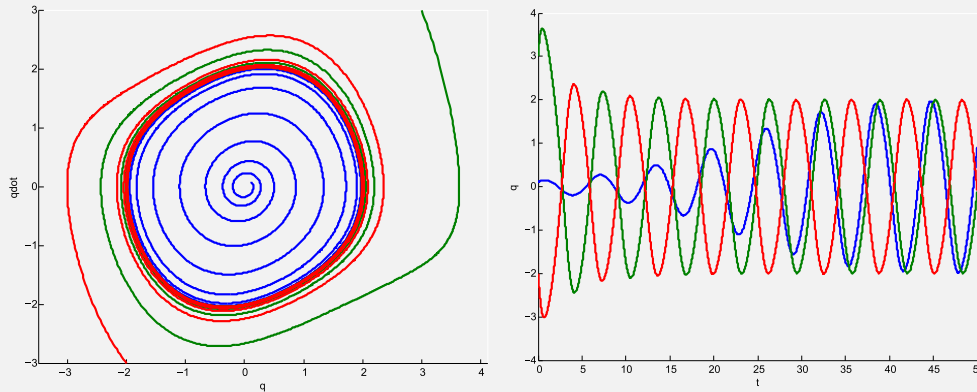


Figure 4.1 - System trajectories of the Van der Pol oscillator with  $\mu = .2$ . (Left) phase portrait. (Right) time domain.

However, if we examine system trajectories in the time domain (see the right panel of the figure above), then we can see that our traditional notion for stability of a trajectory,  $\lim_{t \rightarrow \infty} |\mathbf{x}(t) - \mathbf{x}^*(t)| = 0$ , is not satisfied here for any  $\mathbf{x}^*(t)$ . Although the phase portrait clearly reveals that all trajectories converge to the same orbit, the time domain plot reveals that these trajectories do not necessarily synchronize in time.

This example shows that stability of a trajectory (in time) is not the definition we want. Instead we will define the stability of an orbit,  $\mathbf{x}^*(t)$ , using

$$\min_{\tau} \|\mathbf{x}(t) - \mathbf{x}^*(\tau)\|.$$

Here the quantity of interest is the distance between our trajectory and the **closest point** on the orbit. Depending on exactly how this distance evolves with time, we can define **orbital stability**: for every small  $\epsilon > 0$ , we can find a  $\delta > 0$  such that  $\min_{\tau} \|\mathbf{x}(t_0) - \mathbf{x}^*(\tau)\| < \delta$  implies  $\forall t, \min_{\tau} \|\mathbf{x}(t) - \mathbf{x}^*(\tau)\| < \epsilon$ . The definitions for asymptotic orbital stability, exponential orbital stability, finite-time orbital stability

follow naturally, as does the definition of an unstable orbit. In the case of limit cycles, this  $\mathbf{x}^*(t)$  is a periodic solution with  $\mathbf{x}^*(t + t_{\text{period}}) = \mathbf{x}^*(t)$ .

As we begin to extend our notions of stability, allow me to make a few quick connections to the discussion of stability in the [chapter on Lyapunov analysis](#). It's interesting to know that if we can find an [invariant set](#) in state space which does not contain any fixed points, then this set must contain a closed orbit (this is the Poincaré-Bendixson Theorem) [1]. It's also interesting to note that gradient potential fields (and Lyapunov functions with  $\dot{V}(\mathbf{x}) < 0$ ) cannot have a closed orbit [1], and consequently we will also have to slightly modify the Lyapunov analysis we have introduced so far to handle limit cycles. We'll discuss this in some details in an [upcoming chapter](#).

### 4.1.1 Poincaré Maps

The first tool one should know for analyzing the stability of a limit cycle, and one that will serve us quite well for the examples in this chapter, is the method of Poincaré. Let's consider a dynamical system,  $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ , with  $\mathbf{x} \in \mathbb{R}^n$ . Define an  $n - 1$  dimensional [surface of section](#),  $S$ . We will also require that  $S$  is transverse to the flow (i.e., all trajectories starting on  $S$  flow through  $S$ , not parallel to it). [The Poincaré map](#) (or return map) is a mapping from  $S$  to itself:

$$\mathbf{x}_p[n + 1] = \mathbf{P}(\mathbf{x}_p[n]),$$

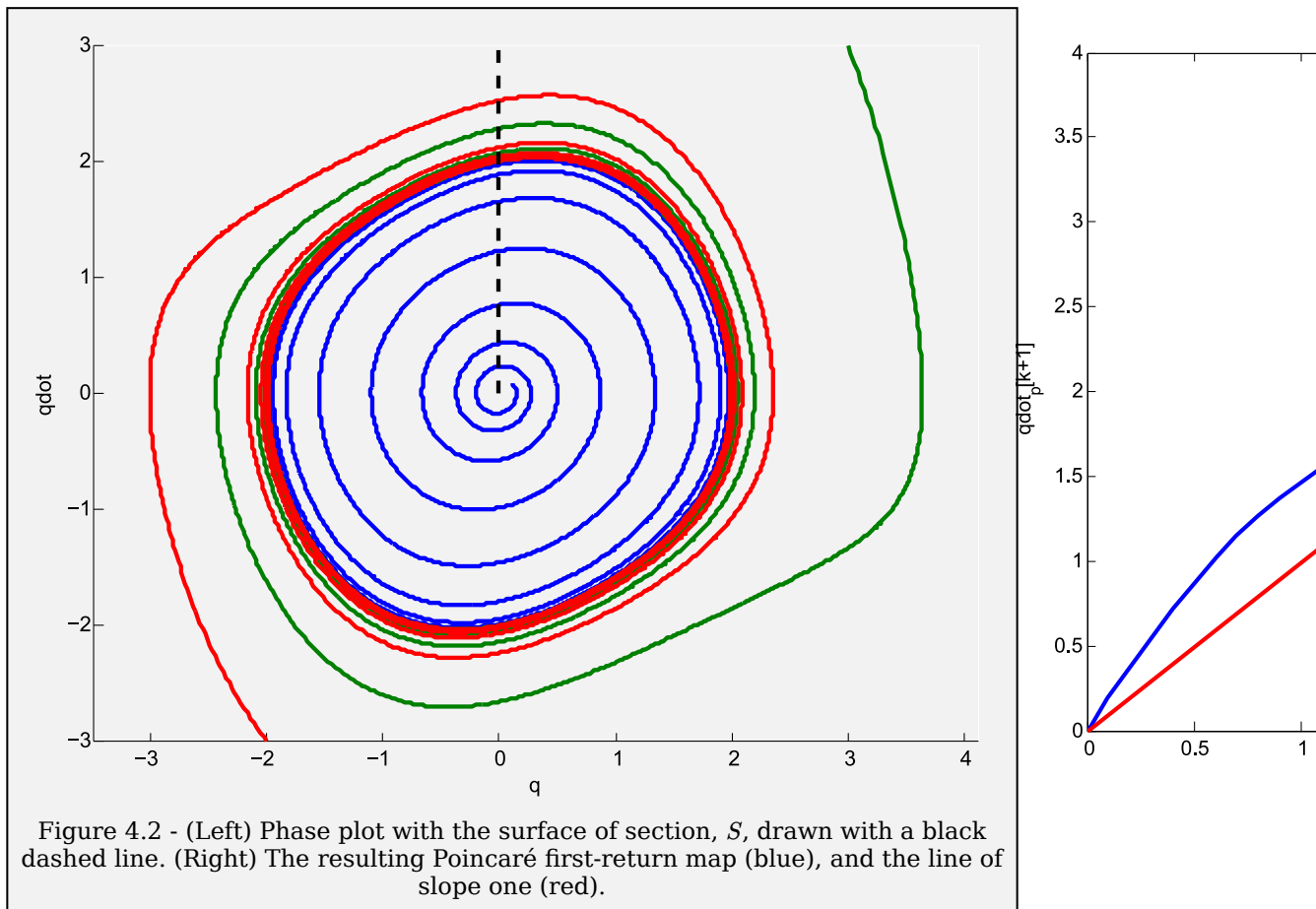
where  $\mathbf{x}_p[n]$  is the state of the system at the  $n$ th crossing of the surface of section. Note that we will use the notation  $\mathbf{x}_p$  to distinguish the state of the discrete-time system from the continuous-time system; they are related by  $\mathbf{x}_p[n] = \mathbf{x}(t_c[n])$ , where  $t_c[n]$  is the time of the  $n$ th crossing of  $S$ .

For our purposes in this chapter, it will be useful for us to allow the Poincaré maps to also include fixed points of the original continuous-time system, and to define the return time to be zero. These points do not satisfy the transversality condition, and require some care numerically, but they do not compromise our analysis.

#### Example 4.2 (Return map for the Van der Pol Oscillator)

Since the state space of this system is two-dimensional, a return map for the system must have dimension one. For instance, we can take  $S$  to be the line segment where  $q = 0$ ,  $\dot{q} \geq 0$ . It is easy to see that all trajectories that start on  $S$  are transverse to it.

One dimensional maps, like one dimensional flows, are [amenable](#) to graphical analysis.



If we can demonstrate that  $\mathbf{P}(\mathbf{x}_p)$  exists for all  $\mathbf{x}_p \in S$ , (all trajectories that leave  $S$  eventually return to  $S$ ), then something remarkable happens: we can reduce the stability analysis for a limit cycle in the original coordinates into the stability analysis of a fixed point on the discrete map. If  $\mathbf{x}_p^*$  is a stable (or unstable) fixed point of the map  $\mathbf{P}$ , then there exists a unique periodic orbit  $\mathbf{x}^*(t)$  which passes through  $\mathbf{x}_p^*$  that is a (stable, or unstable) limit cycle. If we are able to upper-bound the time it takes for trajectories leaving  $S$  to return, then it is also possible to infer asymptotic orbital or even exponential orbital stability.

In practice it is often difficult or impossible to find  $\mathbf{P}$  analytically, but it can be sampled quite reasonably numerically. Once a fixed point of  $\mathbf{P}$  is obtained (by finding a solution to  $\mathbf{x}_p^* = \mathbf{P}(\mathbf{x}_p^*)$ ), we can infer local limit cycle stability with an eigenvalue analysis. If this eigen-analysis is performed in the original coordinates (as opposed to the  $n - 1$  dimensional coordinates of  $S$ ), then there will always be a at least one zero eigenvalue - corresponding to perturbations along the limit cycle which do not change the state of first return. The limit cycle is considered locally exponentially stable if all remaining eigenvalues,  $\lambda_i$ , have magnitude less than one,  $|\lambda_i| < 1$ .

It is sometimes possible to infer more global stability properties of the return map by examining  $\mathbf{P}$ . We will study computational methods in the [later chapter](#). [2] describes some less computational stability properties known for *unimodal* maps which are useful for the simple systems we study in this chapter.

A particularly graphical method for understanding the dynamics of a one-dimensional iterated map is the "staircase method": Sketch the Poincaré map -- a plot of  $x_p[n]$  vs  $x_p[n+1]$  -- and also the line of slope one. In order to "iterate" the map, take an initial condition on the plot as a point  $(x_p[k], x_p[k])$ , and move vertically on the plot to the point  $(x_p[k], x_p[k+1] = f(x_p[k]))$ . Now move horizontally to the next point  $(x_p[k+1], x_p[k+1])$  and repeat.

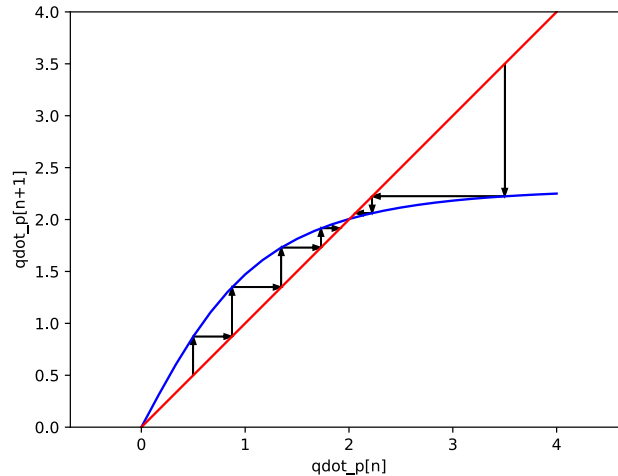


Figure 4.3 - The "staircase" on the return map of the Van der Pol Oscillator with  $\mu = 0.2$ , starting from two initial conditions ( $\dot{q}_p[0] = .5$  and  $\dot{q}_p[0] = 3.5$ ).

We can quickly recognize the fixed points as the points where the return map crosses this line. We can infer local exponential stability if the absolute value of the slope of return map at the fixed point is less than one ( $|\lambda| < 1$ ). The figure above illustrates the stability of the Van der Pol limit cycle; it also illustrates the (one-sided) instability of the fixed point at the origin. Understanding the regions of attraction of fixed points on an iterated map (or any discrete-time system) graphically requires a little more care, though, than the continuous-time flows we examined before. The discrete maps can jump a finite distance on each step, and will in fact oscillate back and forth from either side of the fixed point when  $\lambda < 0$ .

## 4.2 SIMPLE MODELS OF WALKING

Much of the fundamental work in the dynamics of legged robots was done originally in the context of studying *passive-dynamic walkers* (often just "passive walker"). We've already seen my favorite example of a passive walker, in the first chapter, but here it is again:



Figure 4.4 - A 3D passive dynamic walker by Steve Collins and Andy Ruina[3].

This amazing robot has no motors and no controllers... it walks down a tiny ramp and is powered entirely by gravity. We can build up our understanding of this robot in a series of steps. As we will see, what is remarkable is that even though the system is passive, the periodic gait you see in the video is actually a stable limit cycle!

Well before the first passive walkers, one of the earliest models of walking was proposed by McMahon[4], who argued that humans use a mostly ballistic (passive) gait. He observed that muscle activity (recorded via [EMG](#)) in the "stance leg" is relatively high, but muscle activity in the "swing leg" is very low, except for at the very beginning and end of swing. He proposed a "ballistic walker" model -- a three-link pendulum in the [sagittal plane](#), with one link from the ankle to the hip representing a straight "stance leg", the second link from the hip back down to the "swing leg" knee, and the final link from the knee down to the swing foot -- and argued that this model could largely reproduce the kinematics of [gait](#). This model of "walking by vaulting" is considered overly simplistic today, as we now understand much better the role of compliance in the stance leg during walking. His model was also restricted to the continuous portion of gait, not the actual dynamics of taking a step. But it was the start. Nearly a decade later, McGeer[5] followed up with a series of similarly inspired walking machines, which he coined "passive-dynamic walkers".

### 4.2.1 The Rimless Wheel

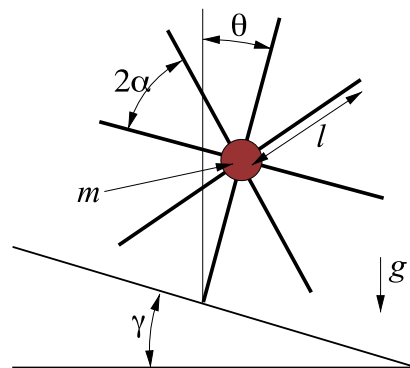


Figure 4.5 - The rimless wheel. The orientation of the stance leg,  $\theta$ , is measured clockwise from the vertical axis (the camera is looking in the positive  $y$  direction in vehicle coordinates).

Perhaps the simplest possible model of a legged robot, introduced as the conceptual framework for passive-dynamic walking by McGeer[5], is the rimless wheel. This system has rigid legs and only a point mass at the hip as illustrated in the figure above. To further simplify the analysis, we make the following modeling assumptions:

- Collisions with ground are inelastic and impulsive (only [angular momentum](#) is conserved around the point of collision).
- The stance foot acts as a pin joint and does not slip.
- The transfer of support at the time of contact is instantaneous (no double support phase).
- $0 \leq \gamma < \frac{\pi}{2}$ ,  $0 < \alpha < \frac{\pi}{2}$ ,  $l > 0$ .

Note that the coordinate system used here is slightly different than for the simple pendulum ( $\theta = 0$  is at the top, and the sign of  $\theta$  has changed).

The most comprehensive analysis of the rimless wheel was done by [6].

### Stance Dynamics

The dynamics of the system when one leg is on the ground are given by

$$\ddot{\theta} = \frac{g}{l} \sin(\theta).$$

If we assume that the system is started in a configuration directly after a transfer of support ( $\theta(0^+) = \gamma - \alpha$ ), then forward walking occurs when the system has an initial velocity,  $\dot{\theta}(0^+) > \omega_1$ , where

$$\omega_1 = \sqrt{2 \frac{g}{l} [1 - \cos(\gamma - \alpha)]}.$$

$\omega_1$  is the threshold at which the system has enough kinetic energy to vault the mass over the stance leg and take a step. This threshold is zero for  $\gamma = \alpha$  and does not exist for  $\gamma > \alpha$  (because when  $\gamma > \alpha$  the mass is always ahead of the stance foot, and the standing fixed point disappears). The next foot touches down when  $\theta(t) = \gamma + \alpha$ , at which point the conversion of potential energy into kinetic energy yields the velocity

$$\dot{\theta}(t^-) = \sqrt{\dot{\theta}^2(0^+) + 4 \frac{g}{l} \sin \alpha \sin \gamma}.$$

$t^-$  denotes the time immediately before the collision.

## Foot Collision

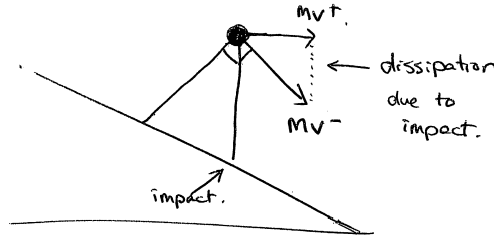


Figure 4.6 - Angular momentum is conserved around the point of impact

The angular momentum around the point of collision at time  $t$  just before the next foot collides with the ground is

$$L(t^-) = -ml^2 \dot{\theta}(t^-) \cos(2\alpha).$$

The angular momentum at the same point immediately after the collision is

$$L(t^+) = -ml^2 \dot{\theta}(t^+).$$

Assuming angular momentum is conserved, this collision causes an instantaneous loss of velocity:

$$\dot{\theta}(t^+) = \dot{\theta}(t^-) \cos(2\alpha).$$

## Forward simulation

Given the stance dynamics, the collision detection logic ( $\theta = \gamma \pm \alpha$ ), and the collision update, we can numerically simulate this simple model. Doing so reveals something remarkable... the wheel starts rolling, but then one of two things happens: it either runs out of energy and stands still, or it quickly falls into a stable periodic solution. Convergence to this stable periodic solution appears to happen from a huge range of initial conditions. Try it yourself.

### Example 4.3 (Numerical simulation of the rimless wheel)

I've setup the notebook to make it easy for you to try a handful of interesting initial conditions. And even made an interactive widget for you to watch the simulation

phase portrait as you change those initial conditions. Give it a try!

The [rimless wheel model](#) actually uses a number of the more nuanced features of [DRAKE](#) in order to simulate the hybrid system accurately (as do many of the examples in this chapter). It actually registers the collision events of the system -- the simulator uses zero-crossing detection to ensure that the time/state of collision is obtained accurately, and then applies the reset map.

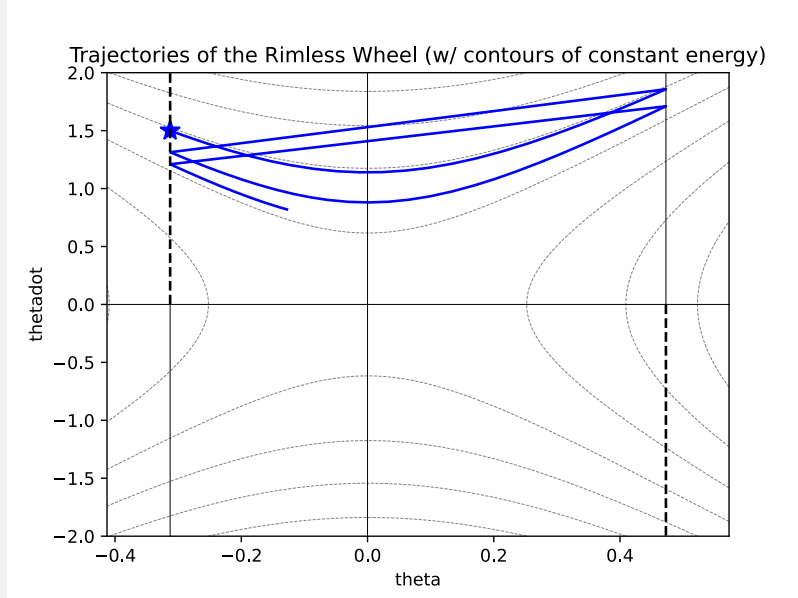


Figure 4.7 - Phase portrait of the rimless wheel ( $m = 1$ ,  $l = 1$ ,  $g = 9.8$ ,  $\alpha = \pi/8$ ,  $\gamma = 0.08$ ).

One of the fantastic things about the rimless wheel is that the dynamics are exactly the undamped simple pendulum that we've thought so much about. So you will recognize the phase portraits of the system -- they are centered around the unstable fixed point of the simple pendulum.

## Poincaré Map

We can now derive the angular velocity at the beginning of each stance phase as a function of the angular velocity of the previous stance phase. First, we will handle the case where  $\gamma \leq \alpha$  and  $\dot{\theta}_n^+ > \omega_1$ . The "step-to-step return map", factoring losses from a single collision, the resulting map is:

$$\dot{\theta}_{n+1}^+ = \cos(2\alpha) \sqrt{(\dot{\theta}_n^+)^2 + 4 \frac{g}{l} \sin \alpha \sin \gamma}.$$

where the  $\dot{\theta}^+$  indicates the velocity just *after* the energy loss at impact has occurred.

Using the same analysis for the remaining cases, we can complete the return map. The threshold for taking a step in the opposite direction is

$$\omega_2 = -\sqrt{2 \frac{g}{l} [1 - \cos(\alpha + \gamma)]}.$$

For  $\omega_2 < \dot{\theta}_n^+ < \omega_1$ , we have

$$\dot{\theta}_{n+1}^+ = -\dot{\theta}_n^+ \cos(2\alpha).$$

Finally, for  $\dot{\theta}_n^+ < \omega_2$ , we have

$$\dot{\theta}_{n+1}^+ = -\cos(2\alpha)\sqrt{(\dot{\theta}_n^+)^2 - 4\frac{g}{l}\sin\alpha\sin\gamma}.$$

Altogether, we have (for  $\gamma \leq \alpha$ )

$$\dot{\theta}_{n+1}^+ = \begin{cases} \cos(2\alpha)\sqrt{(\dot{\theta}_n^+)^2 + 4\frac{g}{l}\sin\alpha\sin\gamma}, & \omega_1 < \dot{\theta}_n^+ \\ -\dot{\theta}_n^+ \cos(2\alpha), & \omega_2 < \dot{\theta}_n^+ < \omega_1 \\ -\cos(2\alpha)\sqrt{(\dot{\theta}_n^+)^2 - 4\frac{g}{l}\sin\alpha\sin\gamma}, & \dot{\theta}_n^+ < \omega_2 \end{cases}$$

Notice that the return map is undefined for  $\dot{\theta}_n = \{\omega_1, \omega_2\}$ , because from these configurations, the wheel will end up in the (unstable) equilibrium point where  $\theta = 0$  and  $\dot{\theta} = 0$ , and will therefore never return to the map.

This return map blends smoothly into the case where  $\gamma > \alpha$ . In this regime,

$$\dot{\theta}_{n+1}^+ = \begin{cases} \cos(2\alpha)\sqrt{(\dot{\theta}_n^+)^2 + 4\frac{g}{l}\sin\alpha\sin\gamma}, & 0 \leq \dot{\theta}_n^+ \\ -\dot{\theta}_n^+ \cos(2\alpha), & \omega_2 < \dot{\theta}_n^+ < 0 \\ -\cos(2\alpha)\sqrt{(\dot{\theta}_n^+)^2 - 4\frac{g}{l}\sin\alpha\sin\gamma}, & \dot{\theta}_n^+ \leq \omega_2 \end{cases}$$

Notice that the formerly undefined points at  $\{\omega_1, \omega_2\}$  are now well-defined transitions with  $\omega_1 = 0$ , because it is kinematically impossible to have the wheel statically balancing on a single leg.

## Fixed Points and Stability

For a fixed point, we require that  $\dot{\theta}_{n+1}^+ = \dot{\theta}_n^+ = \omega^*$ . Our system has two possible fixed points, depending on the parameters:

$$\omega_{stand}^* = 0, \quad \omega_{roll}^* = \cot(2\alpha)\sqrt{4\frac{g}{l}\sin\alpha\sin\gamma}.$$

The limit cycle plotted above illustrates a state-space trajectory in the vicinity of the rolling fixed point.  $\omega_{stand}^*$  is a fixed point whenever  $\gamma < \alpha$ .  $\omega_{roll}^*$  is a fixed point whenever  $\omega_{roll}^* > \omega_1$ . It is interesting to view these bifurcations in terms of  $\gamma$ . For small  $\gamma$ ,  $\omega_{stand}^*$  is the only fixed point, because energy lost from collisions with the ground is not compensated for by gravity. As we increase  $\gamma$ , we obtain a stable rolling solution, where the collisions with the ground exactly balance the conversion of gravitational potential to kinetic energy. As we increase  $\gamma$  further to  $\gamma > \alpha$ , it becomes impossible for the center of mass of the wheel to be inside the support polygon, making standing an unstable configuration.

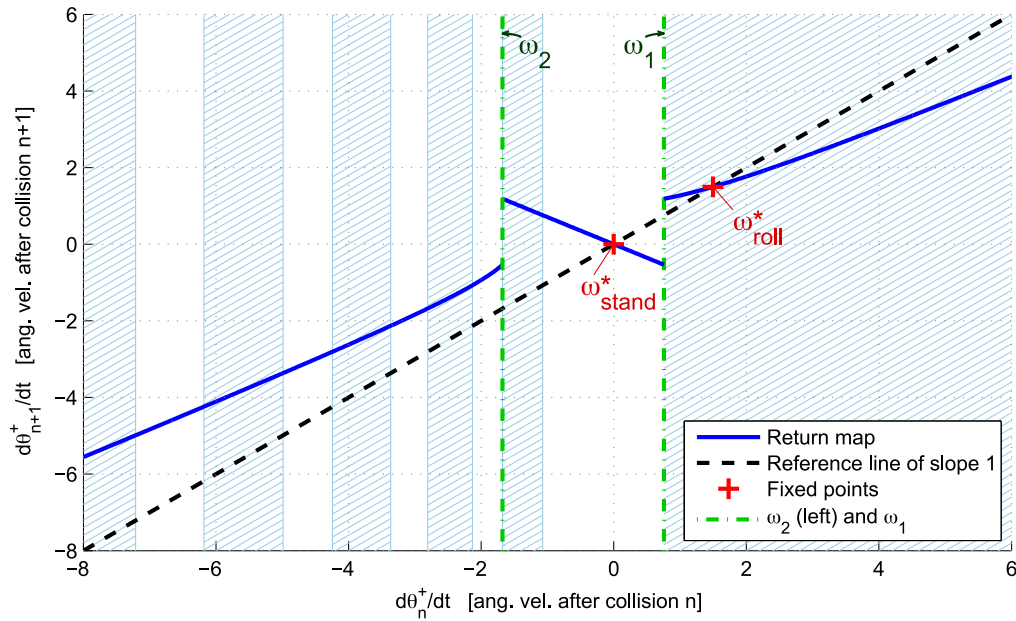


Figure 4.8 - Limit cycle trajectory of the rimless wheel ( $m = 1, l = 1, g = 9.8, \alpha = \pi/8, \gamma = 0.15$ ). All hatched regions converge to the rolling fixed point,  $\omega_{roll}^*$ ; the white regions converge to zero velocity ( $\omega_{stand}^*$ ).

### Stability of standing still

I opened this chapter with the idea that the natural notion of stability for a walking system is periodic stability, and I'll stand by it. But we can also examine the stability of a fixed-point (of the original coordinates; no Poincaré this time) for a system that has contact mechanics. For a legged robot, a fixed point means standing still. This can actually be a little subtle in our hybrid models: in the rimless wheel, standing still is the limiting case where we have infinitely frequent collisions. [Details coming soon.]

### 4.2.2 The Compass Gait

The rimless wheel models only the dynamics of the stance leg, and simply assumes that there will always be a swing leg in position at the time of collision. To remove this assumption, we take away all but two of the spokes, and place a pin joint at the hip. To model the dynamics of swing, we add point masses to each of the legs. I've derived the equations of motion for the system assuming that we have a torque actuator at the hip - resulting in swing dynamics equivalent to an Acrobot (although in a different coordinate frame) - but let's examine the passive dynamics of the system first ( $\mathbf{u} = 0$ ).

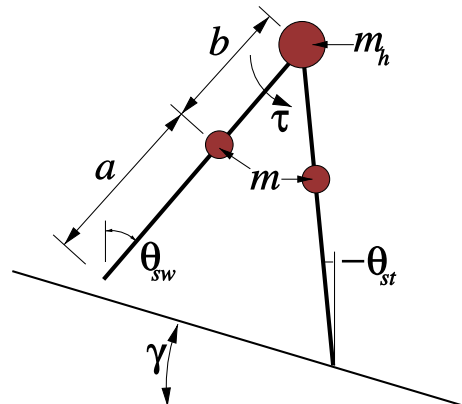


Figure 4.9 - The compass gait

In addition to the modeling assumptions used for the rimless wheel, we also assume that the swing leg retracts in order to clear the ground without disturbing the position of the mass of that leg. This model, known as the compass gait, is well studied in the literature using numerical methods [7, 8, 9], but relatively little is known about it analytically.

The state of this robot can be described by 4 variables:  $\theta_{st}, \theta_{sw}, \dot{\theta}_{st}$ , and  $\dot{\theta}_{sw}$ . The abbreviation *st* is shorthand for the stance leg and *sw* for the swing leg. Using  $\mathbf{q} = [\theta_{st}, \theta_{sw}]^T$  and  $\mathbf{u} = \tau$ , we can write the dynamics as

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u},$$

with

$$\begin{aligned} \mathbf{M} &= \begin{bmatrix} (m_h + m)l^2 + ma^2 & -mlb \cos(\theta_{st} - \theta_{sw}) \\ -mlb \cos(\theta_{st} - \theta_{sw}) & mb^2 \end{bmatrix} \\ \mathbf{C} &= \begin{bmatrix} 0 & -mlb \sin(\theta_{st} - \theta_{sw})\dot{\theta}_{sw} \\ mlb \sin(\theta_{st} - \theta_{sw})\dot{\theta}_{st} & 0 \end{bmatrix} \\ \tau_g(\mathbf{q}) &= \begin{bmatrix} (m_h l + ma + ml)g \sin(\theta_{st}) \\ -mbg \sin(\theta_{sw}) \end{bmatrix}, \\ \mathbf{B} &= \begin{bmatrix} -1 \\ 1 \end{bmatrix} \end{aligned}$$

and  $l = a + b$ .

The foot collision is an instantaneous change of velocity caused by a impulsive force at the foot that brings the foot to rest. The update to the velocities can be calculated using the derivation for impulsive collisions [derived in the appendix](#). To use it, we proceed with the following steps:

- Add a "floating base" to the system by adding a free (x,y) joint at the pre-impact stance toe,  $\mathbf{q}_{fb} = [x, y, \theta_{st}, \theta_{sw}]^T$ .
- Calculate the mass matrix for this new system, call it  $\mathbf{M}_{fb}$ .
- Write the position of the (pre-impact) swing toe as a function  $\phi(\mathbf{q}_{fb})$ . Define the Jacobian of this function:  $\mathbf{J} = \frac{\partial \phi}{\partial \mathbf{q}_{fb}}$ .
- The post-impact velocity that ensures that  $\dot{\phi} = 0$  after the collision is given by

$$\dot{\mathbf{q}}^+ = [\mathbf{I} - \mathbf{M}_{fb}^{-1} \mathbf{J}^T (\mathbf{J} \mathbf{M}_{fb}^{-1} \mathbf{J}^T)^{-1} \mathbf{J}] \dot{\mathbf{q}}^-,$$

noting that  $\dot{x}^- = \dot{y}^- = 0$ , and that you need only read out the last two elements of  $\dot{\mathbf{q}}^+$ . The velocity of the post-impact stance foot will be zero by definition, and the new velocity of the pre-impact stance foot can be completely determined from the minimal coordinates.

- Switch the stance and swing leg positions and velocities.

#### Example 4.4 (Numerical simulation of the compass gait)

You can simulate the passive compass gait using:

Try playing with the initial conditions. You'll find this system is *much* more sensitive to initial conditions than the rimless wheel. It actually took some work to find the initial conditions that make it walk stably.

Numerical integration of these equations reveals a stable limit cycle, plotted below. Notice that when the left leg is in stance (top part of the plot), the trajectory quite resembles the familiar pendulum dynamics of the rimless wheel. But this time, when

the leg impacts, it takes a long arc around as the swing leg before returning to stance. The impacts are also clearly visible as discontinuous changes (decreases) in velocity. The dependence of this limit cycle on the system parameters has been studied extensively in [7].

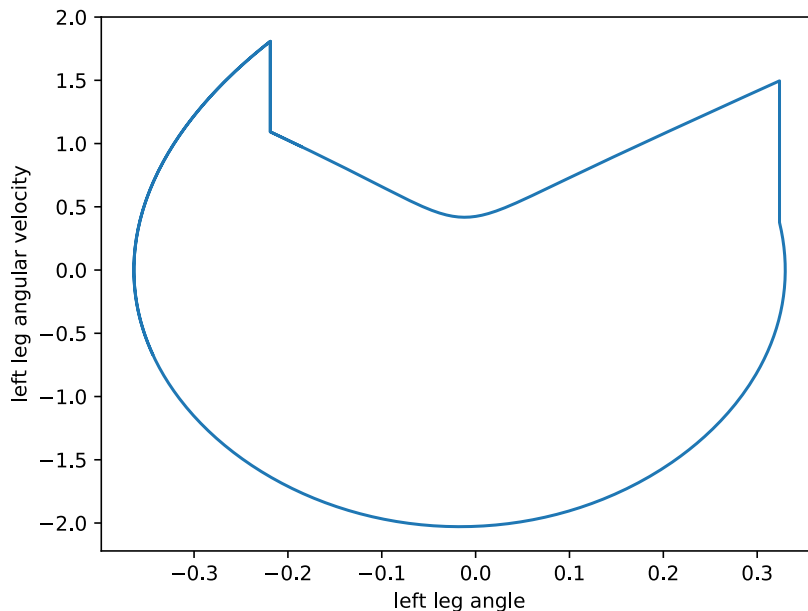


Figure 4.10 - Passive limit cycle trajectory of the compass gait. ( $m = 5\text{kg}$ ,  $m_h = 10\text{kg}$ ,  $a = b = 0.5\text{m}$ ,  $\phi = 0.0525\text{rad}$ .  $\mathbf{x}(0) = [0, 0, 0.4, -2.0]^T$ ). Drawn is the phase portrait of the left leg angle, which is recovered from  $\theta_{st}$  and  $\theta_{sw}$  in the simulation with some simple book-keeping.

The basin of attraction of the stable limit cycle is a narrow band of states surrounding the steady state trajectory. Although the simplicity of this model makes it attractive for conceptual explorations, this lack of stability makes it difficult to implement on a physical device.

### 4.2.3 The Knead Walker

To achieve a more anthropomorphic gait, as well as to acquire better foot clearance and ability to walk on rough terrain, we want to model a walker that includes a knee[10]. The mathematical modeling would be simpler if we used a single link for the stance leg, but we'll go ahead and use two links for each leg (and each with a point mass) because this robot can actually be built!

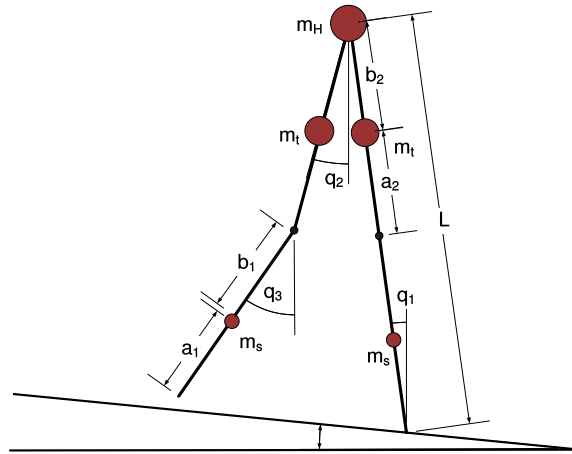


Figure 4.11 - The Kneed Walker

At the beginning of each step, the system is modeled as a three-link pendulum, like the ballistic walker[4, 11, 9]. The stance leg is the one in front, and it is the first link of a pendulum, with two point masses. The swing leg has two links, with the joint between them unconstrained until knee-strike. Given appropriate mass distributions and initial conditions, the swing leg bends the knee and swings forward. When the swing leg straightens out (the lower and upper length are aligned), knee-strike occurs. The knee-strike is modeled as a discrete inelastic collision, conserving angular momentum and changing velocities instantaneously.

After this collision, the knee is locked and we switch to the compass gait model with a different mass distribution. In other words, the system becomes a two-link pendulum. Again, the heel-strike is modeled as an inelastic collision. After the collision, the legs switch instantaneously. After heel-strike then, we switch back to the ballistic walker's three-link pendulum dynamics. This describes a full step cycle of the kneed walker, which is shown above.

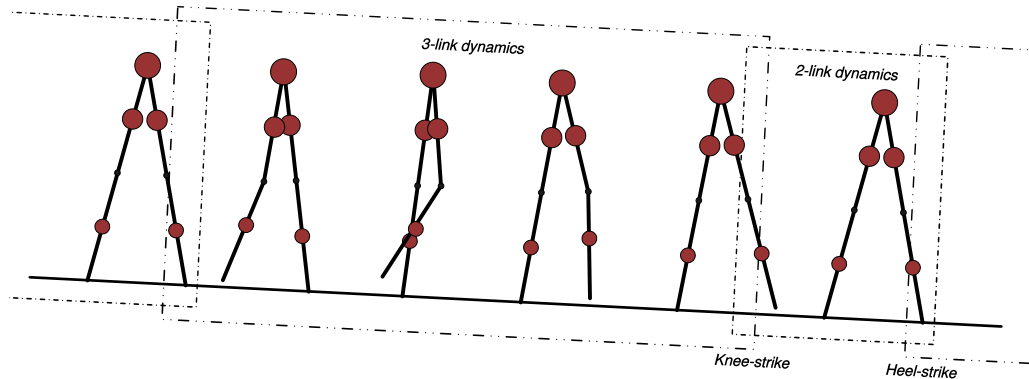


Figure 4.12 - Limit cycle trajectory for kneed walker

By switching between the dynamics of the continuous three-link and two-link pendulums with the two discrete collision events, we characterize a full point-feed kneed walker walking cycle. After finding appropriate parameters for masses and link lengths, a stable gait is found. As with the compass gait's limit cycle, there is a swing phase (top) and a stance phase (bottom). In addition to the two heel-strikes, there are two more instantaneous velocity changes from the knee-strikes as marked in the figure. This limit cycle is traversed clockwise.



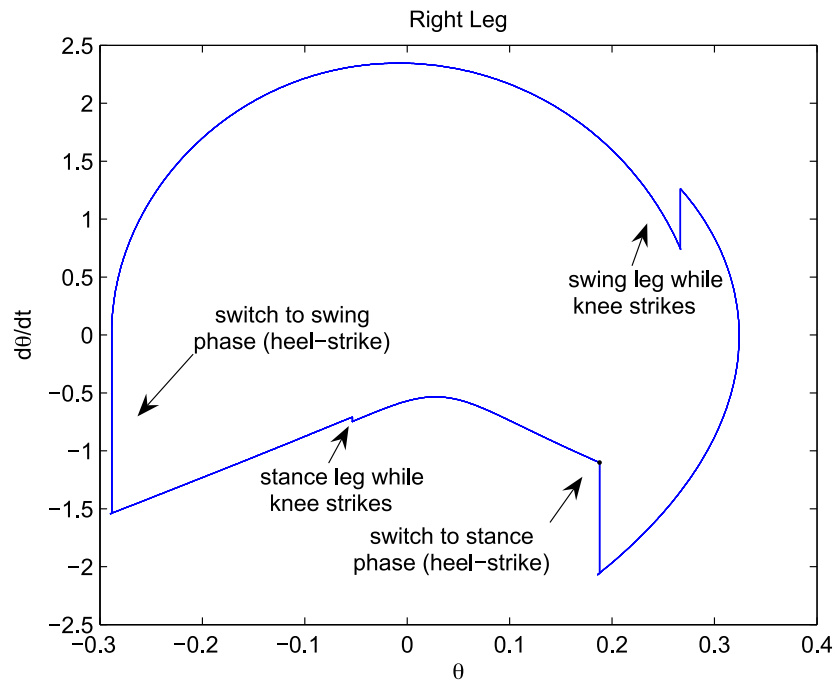


Figure 4.13 - Limit cycle (phase portrait) of the kneed walker

#### 4.2.4 Curved feet

The region of attraction of the kneed-walking limit cycle can be improved considerably by the addition of curved feet...

Figure 4.14 - Tad McGeer's kneed walker. [Here is](#) Matt Haberland's guide to launching it successfully. It's nontrivial!

In [12], we studied the benefits of adding a small amount of energy to these passive dynamic walkers in order to allow them to walk on flat terrain -- sometimes very efficiently! You can find the videos of the Cornell biped [here](#), the TU Delft biped [here](#), and the MIT learning biped (aka "Toddler") [here](#). Toddler can sometimes still be found in the [MIT Museum](#).

### 4.2.5 And beyond...

The world has seen all sorts of great variations on the passive-dynamic walker theme. Almost certainly the most impressive are the creations of Dutch artist Theo Jansen -- he likes to say that he is creating "new forms of life" which he calls the Strandbeest. There are many variations of these amazing machines -- including his beach walkers which are powered only by the wind (I've actually been able to visit Theo's workshop once; it is *very* windy there).

These results are very real. Robin Deits (an exceptionally talented student who felt particularly inspired once on a long weekend) once reproduced one of Theo's earliest creations in DRAKE.

Figure 4.15 - Drake simulation of the Strandbeest.

## 4.3 SIMPLE MODELS OF RUNNING

Just like walking, our understanding of the dynamics and control of running has evolved by a nice interplay between the field of biomechanics and the field of robotics. But in running, I think it's fair to say that the roboticists got off the starting blocks first. And I think it's fair to say that it started with a series of hopping machines built by Marc Raibert and the Leg Laboratory (which was first at CMU, but moved to MIT) in the early 1980's. At a time where many roboticists were building relatively clumsy walking robots that moved very slowly (when they managed to stay up!), the Leg Laboratory produced a series of hopping machines that threw themselves through the air with considerable kinetic energy and considerable flair.

To this day, I'm still a bit surprised that impressive running robots (assuming you accept hopping as the first form of running) appeared before the impressive walking robots. I would have thought that running would be a more difficult control and engineering task than walking. But these hopping machines turned out to be an incredibly clever way to build something simple and very dynamic.

Shortly after Marc Raibert built his machines, Dan Koditschek and Martin Buehler started analyzing them with simpler models [2]. Years later, in collaboration with biologist Bob Full, they started to realize that the simple models used to describe the dynamics of Raibert's hoppers could also be used to describe experimental data obtained from running animals. In fact, they described an incredible range of experimental results, for animals ranging in size from a cockroach up to a horse[13]

(I think the best overall summary of this line of work is [14]). The most successful of the simple models was the so-called "spring-loaded inverted pendulum" (or "SLIP", for short).

### 4.3.1 The Spring-Loaded Inverted Pendulum (SLIP)

The model is a point mass,  $m$ , on top of a massless, springy leg with rest length of  $l_0$ , and spring constant  $k$ . The configuration of the system is given by the  $x, z$  position of the center of mass, and the length,  $l$ , and angle  $\theta$  of the leg. The dynamics are modeled piecewise - with one dynamics governing the flight phase, and another governing the stance phase.

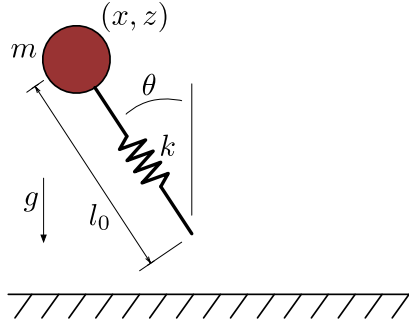


Figure 4.16 - The Spring-Loaded Inverted Pendulum (SLIP) model

In SLIP, we actually use different state variables for each phase of the dynamics. During the flight phase, we use the state variables:  $\mathbf{x} = [x, z, \dot{x}, \dot{z}]^T$ . The flight dynamics are simply

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{x} \\ \dot{z} \\ 0 \\ -g \end{bmatrix}.$$

Since the leg is massless, we assume that the leg can be controlled to any position instantaneously, so it's reasonable to take the leg angle as the control input  $\theta = u$ .

During the "stance phase", we write the dynamics in polar coordinates, with the foot anchored at the origin. Using the state variables:  $\mathbf{x} = [r, \theta, \dot{r}, \dot{\theta}]^T$ , the location of the mass is given by

$$\mathbf{x}_m = \begin{bmatrix} -r \sin \theta \\ r \cos \theta \end{bmatrix}.$$

The total energy is given by the kinetic energy (of the mass) and the potential energy from gravity and from the leg spring:

$$T = \frac{m}{2}(\dot{r}^2 + r^2\dot{\theta}^2), \quad U = mgr \cos \theta + \frac{k}{2}(l_0 - r)^2.$$

Plugging these into Lagrange yields the stance dynamics:

$$\begin{aligned} m\ddot{r} - mr\dot{\theta}^2 + mg \cos \theta - k(l_0 - r) &= 0, \\ mr^2\ddot{\theta} + 2mr\dot{r}\dot{\theta} - mgr \sin \theta &= 0. \end{aligned}$$

We assume that the stance phase is entirely ballistic; there are no control inputs during this phase.

Unlike the rimless wheel model, the idealization of a spring leg with a massless toe means that no energy is lost during the collision with the ground. The collision event causes a transition to a different state space, and to a different dynamics model,

but no instantaneous change in velocity. The transition from flight to stance happens when  $z - l_0 \cos \theta \leq 0$ . The transition from stance to flight happens when the stance leg reaches its rest length:  $r \geq l_0$ .

### Example 4.5 (Simulation of the SLIP model)

You can simulate the spring-loaded inverted pendulum using:

Take a moment to read the code that defines the `SpringLoadedInvertedPendulum` system. You can see that we carefully register the hybrid guards (witness functions) and reset maps, which tells the numerical integrator to use zero-crossing event detection to isolate precisely the time of the event and to apply the reset update at this event. It is quite elegant, and we've gone to some length to make sure that the numerical integration routines in `DRAKE` can support it.

We used the same witness functions and resets to simulate the walking models, but those were implemented in C++. This is a purely Python implementation, making it a bit easier to see those details.

### Analysis on the apex-to-apex map

The rimless wheel had only two state variables ( $\theta$ , and  $\dot{\theta}$ ), and since the surface of section drops one variable, we ended up with a one-dimensional Poincaré map. We have a lot more state variables here. Is there any hope for graphical analysis?

For this system, one can still make progress if we define the Poincaré section to be at the apex of the hop (when  $\dot{z} = 0$ ), and analyze the dynamics from apex to apex. We will ignore the absolute horizontal position,  $x$ , and the leg angle,  $\theta$ , as they are irrelevant to the flight dynamics. This still leaves us with two state variables at the apex: the horizontal velocity,  $\dot{x}$ , and the hopping height,  $z$ .

Importantly, at the apex, these two variables are linked -- when you know one, you know the other. This is because no energy is ever added or removed from the system. If you know the (constant) total energy in the system, and you know the height at the apex, then you know the horizontal velocity (up to a sign):

$$E_{\text{apex}}(z, \dot{x}) = \frac{1}{2} m \dot{x}^2 + mgz = E_0.$$

To make our analysis less dependent on our specific choice of parameters, we typically report any analysis using a dimensionless total energy,  $\tilde{E} = \frac{E}{mgl_0}$ , and a dimensionless spring constant,  $\tilde{k} = \frac{kl_0}{mg}$  [15].

Putting all of these together, we can numerically evaluate the apex-to-apex return map given a known total energy of the system as a one-dimensional Poincaré map.

### Example 4.6 (Numerical SLIP apex-to-apex map)

Numerically integrating the equations of motion from apex-to-apex, using zero-crossing event detection, can give us the apex-to-apex map.

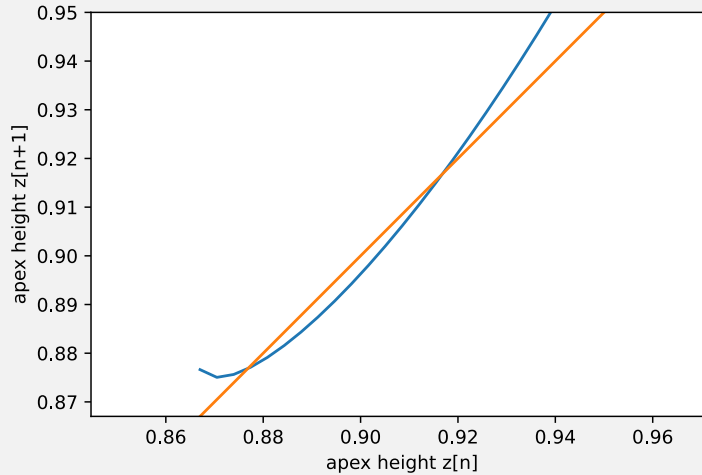


Figure 4.17 - Apex-to-apex return map of the SLIP model given  $\tilde{k} = 10.7$ ,  $\tilde{E} = 1.61$ , and a fixed touchdown angle of  $\theta = 30$  deg (the parameters used in [15]).

Using our graphical staircase analysis (for discrete-time systems), we can see that there are two fixed points of this map: one "stable" fixed point around 0.87 meters, and one unstable fixed point around 0.92 meters.

As always, you should run this code yourself to make sure you understand:

I'd also recommend running the simulation using initial conditions at various points on this plot. You should ask: why does the plot cut off so abruptly just below the stable fixed point? Well the apex-to-apex map becomes undefined if the apex is smaller than  $l_0 \cos \theta$  (the height at touchdown). Furthermore, when  $z$  is too large (relative to  $\tilde{E}$ ), then the system will not have enough horizontal energy to transition through the stance phase and continue forward motion: the center of mass will compress the spring and then fall backwards in the direction from which it came.

The system's conservation of total energy is quite convenient for our return map analysis, but it should give you pause. The rimless wheel actually *relied on the dissipation* due to collisions with the ground to acquire its stability. Is there any chance for a system that is energy conserving to be stable? The apex-to-apex return maps visualized above reveal "stable" fixed-points, but you must understand that the projection from  $(z, \dot{x})$  to just  $z$  is using the assumption that the energy is conserved. As a result, our analysis on the one-dimensional Poincaré map can only make a statement about *partial stability* (i.s.L., asymptotic, or exponential); our analysis does not say anything about stability against disturbances that change the total energy of the system.

Despite this limitation, it is quite interesting that such a simple system *converges* to particular hopping heights from various initial conditions. The observation that these "piecewise-Hamiltonian" systems (Hamiltonian because they conserve total energy) can exhibit this partial stability generated quite a bit of interest from the theoretical physics community. You can sense that enthusiasm when reading [14].

[15] also gives a more thorough analysis of this apex-to-apex map, including a closed-form approximation to the return map dynamics based on a linear (small angle) approximation of the stance dynamics.

## SLIP Control

Our SLIP analysis was simplified considerably by the assumption that the leg is massless, and can be moved instantaneously during the flight phase into the desired

touchdown position. This might sound to you like an offensive oversimplification, but in fact the strong trend in legged robots these days has been to build physical devices which make this sort of abstraction (approximately) valid. For quadrupeds especially, it's hard to fall over when you can move your foot instantaneously into the direction where your body is falling.



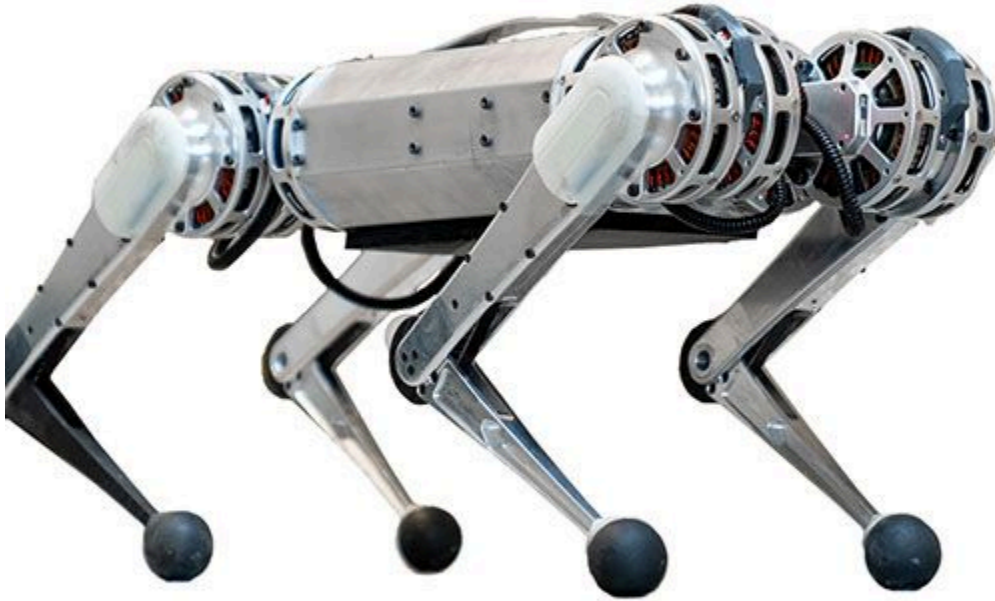


Figure 4.18 - (Left) Spot-mini from Boston Dynamic, (right) Mini-Cheetah from MIT. These are two of my favorite representatives from a fleet of legged robots that have extremely light legs (and approximately point feet), which simplifies control and can improve stability.

Assuming a massless leg also gives us a nice conceptual advantage -- it becomes natural to think about control design as making a control decision *once per step*. This fits nicely with our Poincaré analysis -- we can think of the apex-to-apex map of the SLIP model as being parameterized by the (scalar) control decision  $u[n]$  which sets the touchdown angle for the  $n$ th hop:

$$x_p[n+1] = P(x_p[n], u[n]).$$

This system can be stabilized like any other discrete-time system. For example, it would be perfectly reasonable to linearize the apex-to-apex map around the hopping fixed point, and use the (discrete-time) linear quadratic regulator to design a controller.

It turns out that for the SLIP model, there is a more interesting approach available. If we set the cost on control effort to zero (consistent with our massless leg assumption), then the optimal action to pick is the one that drives the system exactly to the desired fixed point in a single hop. This is the so-called *deadbeat* controller. Not all systems can be stabilized in this way, but [15] showed that the relationship between the touchdown angle and the hopping height in the apex-to-apex map was an invertible relationship (within the bounds from the total energy of the system): given a desired hopping height one can uniquely choose a touchdown angle that will achieve it at the next apex.



Take a moment to reflect on this. In continuous time, if I were to remove the cost on control input, then the optimal actions would be unbounded, or bang-bang if there are input limits. But in discrete-time systems, this is not necessarily the case -- one can often achieve perfect convergence in a finite number of steps with bounded control input -- this corresponds to a finite-time convergence in continuous time. For this simple running model, what is particularly elegant is that this convergence can even be achieved in just one step.

In fact, the story for deadbeat control of the SLIP model gets even better. [16] observed that the desired touchdown angle to achieve a deadbeat apex-to-apex max is a smooth function of the desired hopping height (or equivalently forward speed). Now imagine that we are trying to run/hop at a constant speed over rough terrain. For bonus points, let's do it blind -- with no sensing of the terrain height. It turns out that, given this smooth relationship between touchdown angle and running speed, one can sweep the leg through an *open-loop* leg angle trajectory (based only on the time since the apex), such that whenever the foot happens to collide with the ground, it will always be in exactly the deadbeat touchdown angle!

One would not expect to use the terms "blind", "running", "rough-terrain", and "deadbeat control" all at once to describe a control strategy. But there you have it. A very nice result! The authors went on to study whether this observation could help to explain the known phenomenon in biomechanics of "ground-speed matching" -- a human/animals' tendency to actively swing the leg around to meet the ground at the moment of touchdown.

## SLIP extensions

The original SLIP model, perhaps inspired by the hopping robots turned out to be quite effective in describing the vertical center of mass dynamics of a wide range of animals [14]. Remarkably, experiments showed that the dimensionless individual leg stiffness was very close to 10 for a huge range of animals, ranging from a 0.001 *kg* cockroach to a 135 *kg* horse [17]. But the SLIP models, and modest extensions to them, have also been used to understand lateral stability in cockroaches, resulting in some of my favorite experimental videos of all time [18].

### 4.3.2 Hopping robots from the MIT Leg Laboratory

## The Planar Monopod Hopper

A three-part control strategy: decoupling the control of hopping height, body

attitude, and forward velocity [19].

## Running on four legs as though they were one

bipeds, quadrupeds, and backflips...[20]

### 4.3.3 Towards human-like running

## 4.4 A SIMPLE MODEL THAT CAN WALK AND RUN

## 4.5 EXERCISES

### Exercise 4.1 (One-Dimensional Hopper)

In this exercise we implement a one-dimensional version of the controller proposed in [21]. The goal is to control the vertical motion of a hopper by generating a stable resonant oscillation that causes the system to hop off the ground at a given height. We accomplish this via a careful energy analysis of the hopping cycle: all the details are in . Besides understanding in detail the dynamics of the hopping system, in the notebook, you are asked to write two pieces of code:

- a. A function that, given the state of the hopper, returns its total mechanical energy.
- b. The hopping controller class. This is a **DRAKE** VectorSystem that, at each sampling time, reads the state of the hopper and returns the preload of the hopper spring necessary for the system to hop at the desired height. All the necessary information for the synthesis of this controller are given in the notebook.

## REFERENCES

1. Steven H. Strogatz, "Nonlinear Dynamics and Chaos: With Applications to Physics, Biology, Chemistry, and Engineering", Perseus Books , 1994.
2. Daniel E. Koditschek and Martin Buehler, "Analysis of a Simplified Hopping Robot", *International Journal of Robotics Research*, vol. 10, no. 6, pp. 587-605, Dec, 1991.
3. Steven H. Collins and Martijn Wisse and Andy Ruina, "A Three-Dimensional Passive-Dynamic Walking Robot with Two Legs and Knees", *International Journal of Robotics Research*, vol. 20, no. 7, pp. 607-615, July, 2001.
4. Simon Mochon and Thomas A. McMahon, "Ballistic walking: An improved model", *Mathematical Biosciences*, vol. 52, no. 3-4, pp. 241-260, December, 1980.
5. Tad McGeer, "Passive Dynamic Walking", *International Journal of Robotics Research*, vol. 9, no. 2, pp. 62-82, April, 1990.
6. Michael J. Coleman, "A Stability Study of a Three-Dimensional Passive-Dynamic Model of Human Gait", PhD thesis, Cornell University, 1998.
7. Ambarish Goswami and Benoit Thuilot and Bernard Espiau, "Compass-Like Biped Robot Part {I} : Stability and Bifurcation of Passive Gaits", Tech. Report, RR-2996, October, 1996.

8. A. Goswami, "Postural stability of biped robots and the foot rotation indicator ({FRI}) point", *International Journal of Robotics Research*, vol. 18, no. 6, 1999.
9. Mark W. Spong and Gagandeep Bhatia, "Further Results on Control of the Compass Gait Biped", *Proc. IEEE International Conference on Intelligent Robots and Systems (IROS)*, pp. 1933-1938, 2003.
10. Vanessa Hsu, "Passive dynamic walking with knees: A Point-foot model", , February, 2007. [ [link](#) ]
11. Simon Mochon and Thomas A. McMahon, "Ballistic Walking", *Journal of Biomechanics*, vol. 13, pp. 49-57, 1980.
12. Steven H. Collins and Andy Ruina and Russ Tedrake and Martijn Wisse, "Efficient bipedal robots based on passive-dynamic walkers", *Science*, vol. 307, pp. 1082-1085, February 18, 2005. [ [link](#) ]
13. R.J. Full and D.E. Koditschek, "Templates and anchors: neuromechanical hypotheses of legged locomotion on land", *Journal of Experimental Biology*, vol. 202, no. 23, pp. 3325-3332, 1999.
14. Philip Holmes and Robert J. Full and Dan Koditschek and John Guckenheimer, "The Dynamics of Legged Locomotion: Models, Analyses, and Challenges", *Society for Industrial and Applied Mathematics (SIAM) Review*, vol. 48, no. 2, pp. 207--304, 2006.
15. Hartmut Geyer, "Simple models of legged locomotion based on compliant limb behavior", PhD thesis, University of Jena, 2005.
16. Michael Ernst and Hartmut Geyer and Reinhard Blickhan, "Spring-legged locomotion on uneven ground: a control approach to keep the running speed constant", *International Conference on Climbing and Walking Robots*, pp. 639--644, 2009.
17. Daniel E Koditschek and Robert J Full and Martin Buehler, "Mechanical aspects of legged locomotion control", *Arthropod structure & development*, vol. 33, no. 3, pp. 251--272, 2004.
18. DL Jindrich and RJ Full, "Dynamic stabilization of rapid hexapedal locomotion", *J Exp Biol*, vol. 205, no. Pt 18, pp. 2803-23, Sep, 2002.
19. Marc H. Raibert, "Legged Robots That Balance", The MIT Press , 1986.
20. Raibert and M. H. and Chepponis and M. and Brown and H. B., "Running on four legs as though they were one", *IEEE Journal of Robotics and Automation*, vol. 2, no. 2, pp. 70--82, 1986.
21. Marc H. Raibert, "Hopping in legged systems: Modeling and simulation for the 2D one-legged case", *IEEE Trans. Systems, Man, and Cybernetics*, vol. 14, pp. 451-463, 1984.

# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 5

# Highly-articulated Legged Robots

The passive dynamic walkers and hopping robots described in the last chapter capture the fundamental dynamics of legged locomotion -- dynamics which are fundamentally nonlinear and punctuated by impulsive events due to making and breaking contact with the environment. But if you start reading the literature on humanoid robots, or many-legged robots like quadrupeds, then you will find a quite different set of ideas taking center stage: ideas like the "zero-moment point" and footstep planning. The goal of this chapter is to penetrate that world.

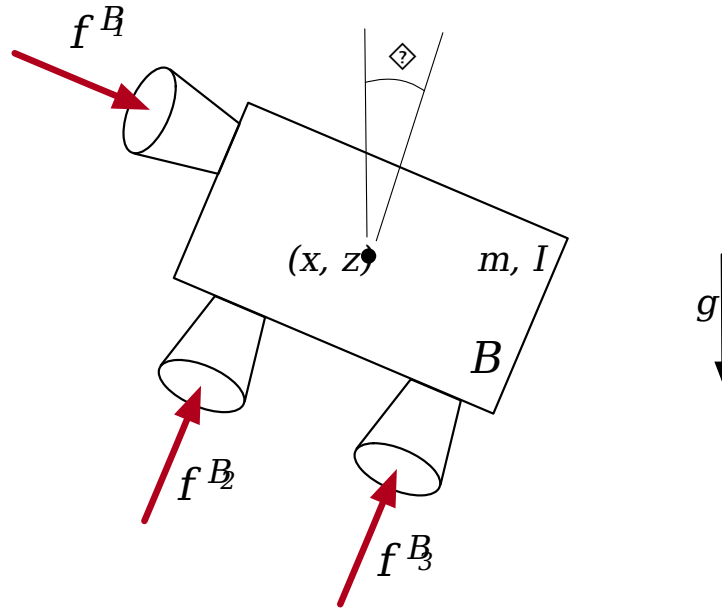
Perhaps one of the most surprising thing about the world of complex (highly-articulated) legged robots is the dominant use of simple models for estimation, planning, and control. These abstractions become valuable when you have enough degrees of freedom to move your feet independently (to an extent) from how you move your body.

Since our robots are getting more complicated, I'm going to need a bit stronger notation. In Drake we use [monogram notation](#) and define its spatial algebra [here for kinematics](#), [differential kinematics \(velocities\)](#) and [here for force](#).

### 5.1 A THOUGHT EXPERIMENT

Let's start the discussion with a model that might seem quite far from the world of legged robots, but I think it's a very useful way to think about the problem.

### 5.1.1 A spacecraft model



Imagine you have a flying vehicle modeled as a single rigid body in a gravity field with some number of force "thrusters" attached. We'll describe the pose of the body in the world frame,  ${}^W X^B$ , parameterized by its position  $(x, z)$  and orientation  $\theta$ , and take the center of mass to be at the origin of this body frame. The vector from the center of mass to thruster  $i$  in the world frame is given by  ${}^B p_W^{B_i}$ , yielding the equations of motion:

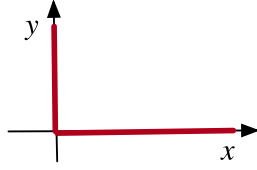
$$\begin{aligned} m\ddot{x} &= \sum_i f_{W_x}^{B_i}, \\ m\ddot{z} &= \sum_i f_{W_z}^{B_i} - mg, \\ I\ddot{\theta} &= \sum_i \left[ {}^B p_W^{B_i} \times f_W^{B_i} \right]_y, \end{aligned}$$

where I've used  $f_{W_x}^{B_i}$  to represent the  $x$ -component of the force applied to  $B$  at the location of thruster  $i$ , expressed in the world frame, and we take the  $y$ -axis component of the cross product on the last line.

Our goal is to move the spacecraft to a desired position/orientation and to keep it there. If we take the inputs to be  $f_W^{B_i}$ , then the dynamics are affine (linear plus a constant term). As such, we can stabilize a stabilizable fixed point using a change of coordinates plus LQR or even plan/track a desired trajectory using time-varying LQR. If I were to add additional linear constraints, for instance constraining  $f_{W_x}^{B_i} \leq f_{max}$ , then I can still use linear model-predictive control (MPC) to plan and stabilize a desired motion. By most accounts, this is a relatively easy control problem. (Note that it would be considerably harder if my control input or input constraints depend on the orientation,  $\theta$ , but we don't need that here yet).

Now imagine that you have just a small number of thrusters (let's say two) each with severe input constraints. To make things more interesting, let us say that you're allowed to move the thrusters, so  ${}^B v^{B_i} = \frac{d}{dt} {}^B p^{B_i}$  becomes an additional control input, but with some important limitations: you have to turn the thrusters off when they are moving (e.g.  $f_W^{B_i} v_W^{B_i} = 0$ ) and there is a limit to how quickly you can move the thrusters  $|{}^B v_W^{B_i}| \leq v_{max}$ . This problem is now a lot more difficult, due especially to the fact that constraints of the form  $xy = 0$  are non-convex (the red area in the

illustration below represents the feasible set for  $x \geq 0, y \geq 0$ ).



I find this a very useful thought exercise; somehow our controller needs to make an effectively discrete decision to turn off a thruster and move it. The framework of optimal control should support this - you are sacrificing a short-term loss of control authority for the long-term benefit of having the thruster positioned where you would like, but we haven't developed tools yet that deal well with this discrete plus continuous decision making. We'll need to address that here.

Unfortunately, although the problem is already difficult, we need a few more constraints to make it relevant for legged robots. Now let's say additionally, that the thrusters can only be turned on in certain locations, as cartooned here:

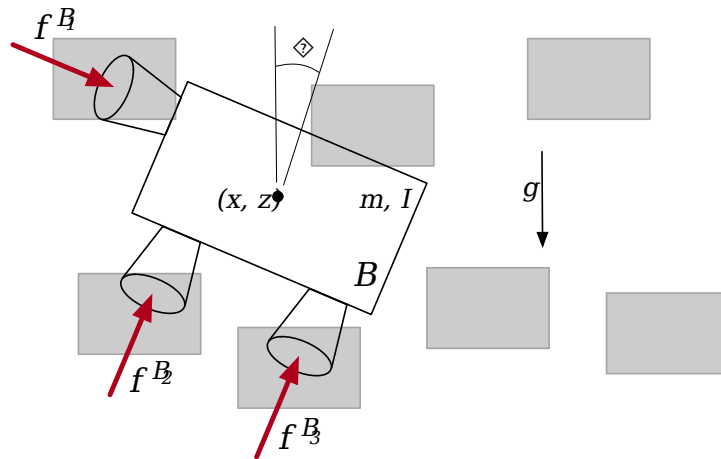


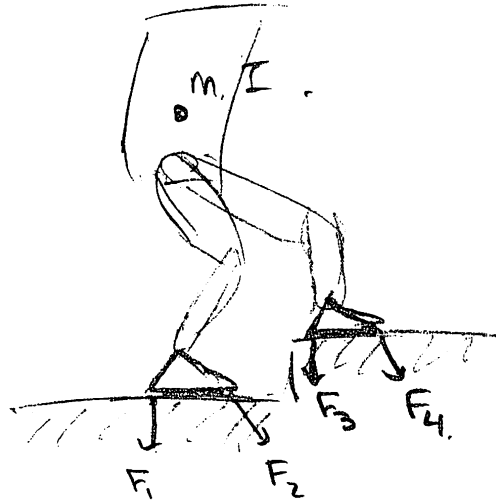
Figure 5.2 - What if we can only apply thrust when the thrusters are in a certain (world) position?

Even if the individual regions are convex, the union of these regions need not form a convex set. Furthermore, these locations are often expressed in the world frame, not the robot frame, e.g.

$$\mathbf{A}^w p_W^{R_i} \leq \mathbf{b}.$$

### 5.1.2 Robots with (massless) legs

In my view, the spacecraft with thrusters problem above is a central component of the walking problem. If we consider a walking robot as our single rigid body,  $B$ , with massless legs, then the feet are exactly like movable thrusters. As above, they are highly constrained - they can only produce force when they are touching the ground, and (typically) they can only produce forces in certain directions, for instance as described by a "friction cone" (you can push on the ground but not pull on the ground, and with Coulomb friction the forces tangential to the ground must be smaller than the forces normal to the ground, as described by a coefficient of friction, e.g.  $f_{C_z}^{B_i} \geq 0, |f_{C_x}^{B_i}| < \mu f_{C_z}^{B_i}$ ) where  $C$  is the [contact frame](#).



The constraints on where you can put your feet / thrusters will depend on the kinematics of your leg, and the speed at which you can move them will depend on the full dynamics of the leg -- these are difficult constraints to deal with. But the actual dynamics of the rigid body,  $B$ , are actually still affine, and still simple!

## 5.2 CENTROIDAL DYNAMICS

It turns out the specific equations that we used in our thought experiment aren't restricted to a single body and massless legs. If we simply repurpose the coordinates  $x, z$  to describe the location of the center of mass (CoM) of the entire robot (not just the center of mass of a single body,  $B$ ), and  $m$  to represent the entire mass of the robot, then the first two equations remain unchanged. The center of mass is a configuration dependent point, and does not have an orientation, but one important generalization of the orientation dynamics is given by the centroidal momentum matrix,  $A(\mathbf{q})$ , where  $A(\mathbf{q})\dot{\mathbf{q}}$  captures the linear and angular momentum of the robot around the center of mass [1]. Note that the center of mass dynamics are still affine -- even for a big complicated humanoid -- but the centroidal momentum dynamics are nonlinear.

### 5.2.1 Impact dynamics

In the previous chapter we devoted relatively a lot of attention to dynamics of impact, characterized for instance by a guard that resets dynamics in a hybrid dynamical system. In those models we used impulsive ground-reaction forces (these forces are instantaneously infinite, but doing finite work) in order to explain the discontinuous change in velocity required to avoid penetration with the ground. This story can be extended naturally to the dynamics of the center of mass.

For an articulated robot, though, there are a number of possible strategies for the legs which can affect the dynamics of the center of mass. For example, if the robot hits the ground with a stiff leg like the rimless wheel, then the angular momentum about the point of collision will be conserved, but any momentum going into the ground will be lost. However, if the robot has a spring leg and a massless toe like the SLIP model, then no energy need be lost.

One strategy that many highly-articulated legged robots use is to keep their *center of mass* at a constant velocity,

$$\dot{z} = c \quad \Rightarrow \quad \ddot{z} = 0,$$

and minimize their angular momentum about the center of mass (here  $\dot{\theta} = 0$ ). Using

this strategy, if the swinging foot lands roughly below the center of mass, then even with a stiff leg there is no energy dissipated in the collision - all of the momentum is conserved. This often (but does not always) justify ignoring the impacts in the center of mass dynamics of the system.

## 5.2.2 The special case of flat terrain

While not the only important case, it is extremely common for our robots to be walking over flat, or nearly flat terrain. In this situation, even if the robot is touching the ground in a number of places (e.g., two heels and two toes), the constraints on the center of mass dynamics can be summarized very efficiently.

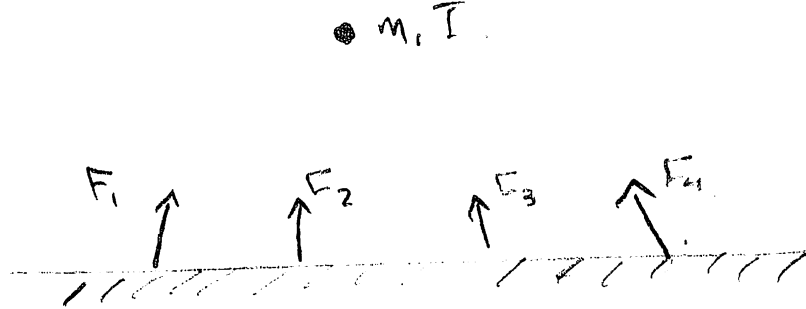


Figure 5.4 - External forces acting on a robot pushing on a flat ground

First, on flat terrain  $f_{W_z}^{B_i}$  represents the force that is normal to the surface at contact point  $i$ . If we assume that the robot can only push on the ground (and not pull), then this implies

$$\forall i, f_{W_z}^{B_i} \geq 0 \Rightarrow \sum_i f_{W_z}^{B_i} \geq 0 \Rightarrow \ddot{z} \geq -g.$$

In other words, if I cannot pull on the ground, then my center of mass cannot accelerate towards the ground faster than gravity.

Furthermore, if we use a Coulomb model of friction on the ground, with friction coefficient  $\mu$ , then

$$\forall i, |f_{W_x}^{B_i}| \leq \mu f_{W_z}^{B_i} \Rightarrow \sum_i |f_{W_x}^{B_i}| \leq \mu \sum_i f_{W_z}^{B_i} \Rightarrow |\ddot{x}| \leq \mu(\ddot{z} + g).$$

For instance, if I keep my center of mass height at a constant velocity, then  $\ddot{z} = 0$  and  $|\ddot{x}| \leq \mu g$ ; this is a nice reminder of just how important friction is if you want to be able to move around in the world.

Even better, let us define the "center of pressure" (CoP) as the point on the ground where

$$x_{cop} = \frac{\sum_i {}^W p_{W_z}^{B_i} f_{W_z}^{B_i}}{\sum_i f_{W_z}^{B_i}},$$

and since all  ${}^W p_{W_z}^{B_i}$  are equal on flat terrain,  $z_{cop}$  is just the height of the terrain. It turns out that the center of pressure is a "zero-moment point" (ZMP) -- a property that we will demonstrate below -- and moment-balance equation gives us a very important relationship between the location of the CoP and the dynamics of the CoM:

$$(m\ddot{z} + mg)(x_{cop} - x) = (z_{cop} - z)m\ddot{x} - I\ddot{\theta}. \quad (1)$$

If we use the strategy proposed above for ignoring collision dynamics,  $\ddot{z} = \ddot{\theta} = 0$ , and



the result is the famous "ZMP equations":

$$\ddot{x} = g \frac{(x - x_{cop})}{(z - z_{cop})}. \quad (2)$$

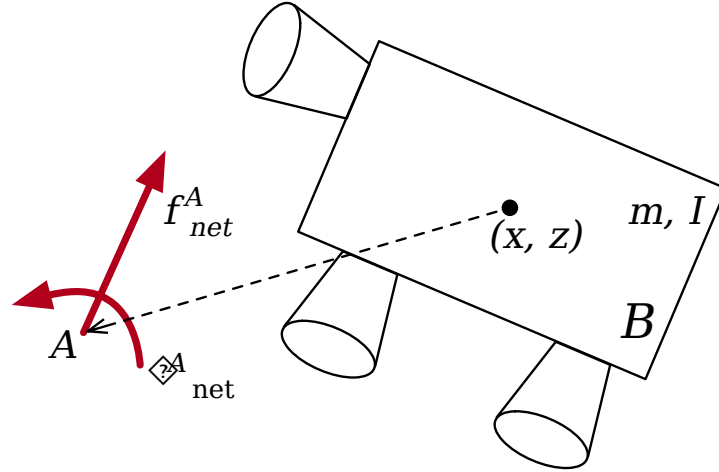
So the location of the center of pressure completely determines the acceleration of the center of mass, and vice versa! What's more, when  $z - z_{cop}$  is a constant (the center of mass remains at a constant height relative to the ground) or following a predetermined schedule, the remaining relationship is affine -- a property that we can exploit in a number of ways.

As an example, we can easily relate constraints on the CoP to constraints on  $\ddot{x}$ . It is easy to verify from the definition that the CoP must live inside the convex hull of the ground contact points. Therefore, if we use the  $\ddot{z} = \ddot{\theta} = 0$  strategy, then this directly implies strict bounds on the possible accelerations of the center of mass given the locations of the ground contacts.

### An aside: the zero-moment point derivation

The zero-moment point (ZMP) is discussed very frequently in the current literature on legged robots. It also has an unfortunate tendency to be surrounded by some confusion; a number of people have defined the ZMP in slightly different ways (see e.g. [2] for a summary). Therefore, it makes sense to provide a simple derivation here.

First let us recall that for rigid body systems I can always summarize the contributions from many external forces as a single wrench (force and torque) on the object -- this is simply because the  $F_i$  terms enter our equations of motion linearly. Specifically, the position  $p^A$  of an arbitrary point  $A$ :



I can re-write the equations of motion as

$$\begin{aligned} m\ddot{x} &= \sum_i f_{W_x}^{B_i} = f_{net, W_x}^A, \\ m\ddot{z} &= \sum_i f_{W_z}^{B_i} - mg = f_{net, W_z}^A - mg, \\ I\ddot{\theta} &= \sum_i \left[ {}^B p_W^{B_i} \times f_W^{B_i} \right]_y = [{}^B p_W^A \times f_{net, W}^A]_y + \tau_{net, W}^A, \end{aligned}$$

where  $f_{net}^A = \sum_i f^{B_i}$  and the value of  $\tau_{net}^A$  depends on the position,  $p^A$ . For some choices of  $p^A$ , we can make  $\tau_{net}^A = 0$ . We'll call these points ZMP, for which we have

$$\left[ {}^B p_W^{ZMP} \times f_{net, W}^{ZMP} \right]_y = {}^B p_{W_z}^{ZMP} f_{net, W_x}^{ZMP} - {}^B p_{W_x}^{ZMP} f_{net, W_z}^{ZMP} = \sum_i \left[ {}^B p_W^{B_i} \times f_W^{B_i} \right]_y, \quad (3)$$

we can see that whenever we have ground reaction forces ( $f_{net, W_z}^{ZMP} \neq 0$ ) there is an entire line of solutions,  ${}^B p_{W_x}^{ZMP} = a {}^B p_{W_z}^{ZMP} + b$ , including one that will intercept the terrain. For walking robots, it is this point on the ground from which the external wrench can be described by a single force vector (and no moment) that is the famous "zero-moment point" (ZMP).

We can rewrite the equations of motion in terms of the ZMP. Substituting the equations of motion into equation 3, to replace the individual terms of  $f_{net}^{ZMP}$ , we have

$${}^B p_{W_z}^{ZMP} m\ddot{x} - {}^B p_{W_x}^{ZMP} (m\ddot{z} + mg) = I\ddot{\theta}.$$

But this is precisely the equation 1 presented above.

Furthermore, since

$$\sum_i \left[ {}^B p_W^{B_i} \times f_W^{B_i} \right]_y = \sum_i \left( {}^B p_{W_z}^{B_i} f_{W_x}^{B_i} - {}^B p_{W_x}^{B_i} f_{W_z}^{B_i} \right),$$

and for *flat terrain* we have

$$\sum_i {}^B p_{W_z}^{B_i} f_{W_x}^{B_i} = {}^B p_{W_z}^{ZMP} \sum_i f_{W_x}^{B_i} = {}^B p_{W_z}^{ZMP} f_{net, W_x}^{ZMP},$$

then we can see that this ZMP is exactly the CoP:

$${}^B p_{W_x}^{ZMP} = \frac{\sum_i {}^B p_{W_z}^{B_i} f_{W_x}^{B_i}}{\sum_i f_{W_z}^{B_i}}.$$

In three dimensions, we solve for the point on the ground where  $\tau_{net, W_y}^{ZMP} = \tau_{net, W_x}^{ZMP} = 0$ , but allow  $\tau_{net, W_z}^{ZMP} \neq 0$  to extract the analogous equations in the  $y$ -axis:

$${}^B p_{W_z}^{ZMP} m\ddot{y} - {}^B p_{W_y}^{ZMP} (m\ddot{z} + mg) = I\ddot{\theta}.$$

## 5.3 ZMP-BASED PLANNING

Understanding the relationship between the center of pressure and the center of mass motion gives us a number of important tools for planning motions of our complex robots. I'll describe one of the most common pipeline for planning these motions, which decomposes the problem into:

1. planning the footstep (or more generally the contact) positions, then
2. planning the motion of the center of mass, and finally
3. calculating the individual joint motions to realize the desired center of mass and footstep trajectories

This pipeline started with the famous ZMP walkers (like Honda's ASIMO), with a focus on walking on flat terrain, but many of the concepts generalize naturally to 3D and more general robot motion.

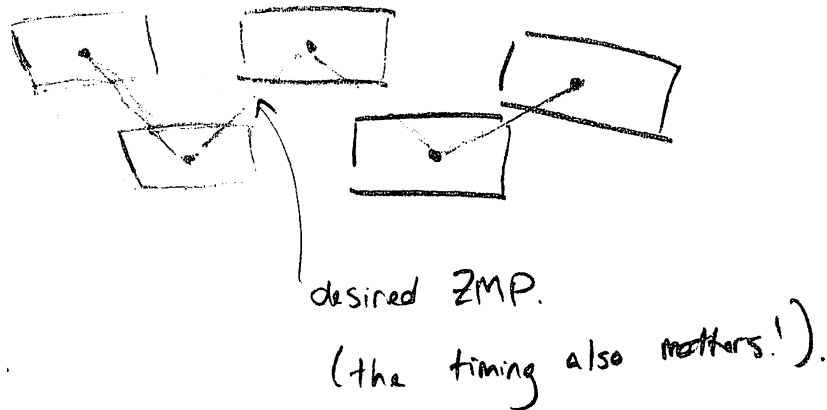
Most notable here is the decision to separate the complicated planning problem into a sequence of simpler planning problems. Of course, one pays some price for the decomposition -- footstep plans tend to be conservative and kinematic because they do not reason directly about the dynamics of the robot. And the centroidal planning cannot accurately account for joint-level constraints, such as joint-position limits or effort limits. But these approximations enable dramatically simpler planning solutions.

### 5.3.1 Heuristic footstep planning

Coming soon. For a description of our approach with Atlas, see [3, 4].

### 5.3.2 Planning trajectories for the center of mass

#### The ZMP "Stability" Metric



### 5.3.3 From a CoM plan to a whole-body plan

## 5.4 WHOLE-BODY CONTROL

Coming soon. See, for instance, [5, 4].

## 5.5 FOOTSTEP PLANNING AND PUSH RECOVERY

Coming soon. See, for instance, [3, 4]. For nice geometric insights on push recovery, see [6].

## 5.6 BEYOND ZMP PLANNING

Coming soon. See, for instance, [7, 4].

### Example 5.1 (LittleDog gait optimization)

## 5.7 EXERCISES

### Exercise 5.1 (Footstep Planning via Mixed-Integer Optimization)

In this exercise we implement a simplified version of the footstep planning method proposed in [3]. You will find all the details. Your goal is to code most of the components of the mixed-integer program at the core of the footstep planner:

- The constraint that limits the maximum step length.
- The constraint for which a foot cannot be in two stepping stones at the same time.
- The constraint that assigns each foot to a stepping stone, for each step of the robot.
- The objective function that minimizes the sum of the squares of the step

lengths.

### Exercise 5.2 (Understanding the zero moment point)

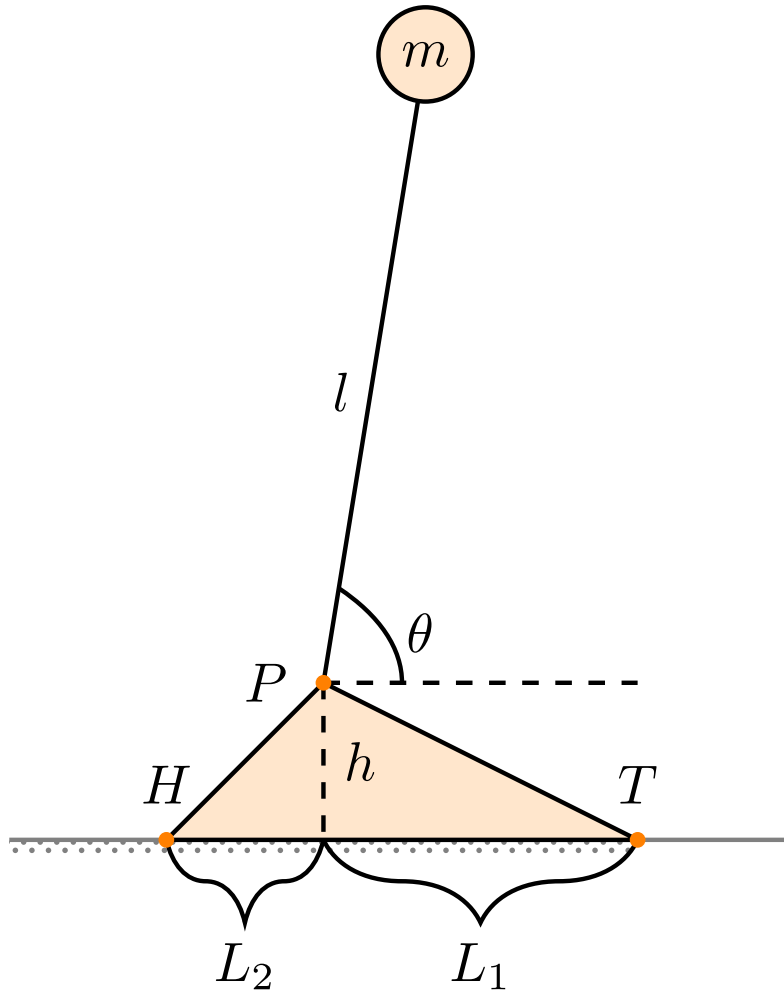


Figure 5.7 - External forces acting on a robot pushing on a flat ground  
Consider an abstracted legged robot. The foot and the leg are assumed to be massless. Suppose there is no actuator at the pin joint  $P$

- Is the system stable at  $\theta = \pi/2 + 10^{-3}$ ?
- Compute the zero moment point on the ground as a function of  $\theta$  and the constants  $m, h, g, l, L_1, L_2$  (if needed).
- For what value of  $\theta$  does the ZMP reach the toe  $T$ ?
  - For what value of  $\theta$  does the heel  $H$  of the robot begin to lift?
- Suppose now that there is an actuator at the ankle with a controller capable of perfectly cancelling the torque around  $P$  due to gravity so  $\ddot{\theta} = 0$ . What value of  $\theta$  can you achieve without falling down? Express your answer in terms of the constants  $m, h, l, L_1, L_2$  (if needed).
- Suppose you are designing a foot that maximizes the postures you can assume without falling down. Suppose we can apply a torque at the ankle without torque limits.
  - Would you want a taller or flatter foot?
  - Would you want a longer or shorter foot?
  - Where would you put the ankle with respect to the heel and

toe?

- f. Suppose you are designing a robot designed to only walk forward. Suppose we can apply a torque at the ankle without torque limits.
  - i. Explain the trade offs in foot height.
  - ii. Explain the trade offs in the design of  $L_1$ .
  - iii. Explain the trade offs in the design of  $L_2$ .

### Exercise 5.3 (Footstep Planning via Graph of Convex Sets (GCS))

In this exercise we implement a simplified version of the footstep planning problem formulated as a shortest path problem (SPP) in GCS (proposed in [8]). Compared to the MIQP formulation seen in [this exercise](#), the convex relaxation problem using GCS reduces to a linear program (LP) -- it is much more computationally efficient and consequently scales better. You will find all the details in . Your goal is to code most of the components that transcribe the planning task as a shortest path problem in GCS at the core of the footstep planner:

- a. Construct vertices of the GCS based on the stepping stones.
- b. Construct valid edges of the GCS between vertices.
- c. Add constraints associated with each edge that reflect the kinematic constraints of the robot.
- d. Add edge costs that reflect the optimization objective: minimizes the total number of steps.

## 5.8 REFERENCES

1. David E. Orin and Ambarish Goswami and Sung-Hee Lee, "Centroidal dynamics of a humanoid robot", *Autonomous Robots*, no. September 2012, pp. 1--16, jun, 2013.
2. A. Goswami, "Postural stability of biped robots and the foot rotation indicator ({FRI}) point", *International Journal of Robotics Research*, vol. 18, no. 6, 1999.
3. Robin Deits and Russ Tedrake, "Footstep Planning on Uneven Terrain with Mixed-Integer Convex Optimization", *Proceedings of the 2014 IEEE/RAS International Conference on Humanoid Robots (Humanoids 2014)*, 2014. [ [link](#) ]
4. Scott Kuindersma and Robin Deits and Maurice Fallon and Andr\{e}s Valenzuela and Hongkai Dai and Frank Permenter and Twan Koolen and Pat Marion and Russ Tedrake, "Optimization-based Locomotion Planning, Estimation, and Control Design for the {A}tlas Humanoid Robot", *Autonomous Robots*, vol. 40, no. 3, pp. 429--455, 2016. [ [link](#) ]
5. Scott Kuindersma and Frank Permenter and Russ Tedrake, "An Efficiently Solvable Quadratic Program for Stabilizing Dynamic Locomotion", *Proceedings of the International Conference on Robotics and Automation*, May, 2014. [ [link](#) ]
6. Twan Koolen and Tomas de Boer and John Rebula and Ambarish Goswami and Jerry Pratt, "Capturability-based analysis and control of legged locomotion, Part 1: Theory and application to three simple gait models", *The International Journal of Robotics Research*, vol. 31, no. 9, pp. 1094-1113, 2012.

7. Hongkai Dai and Andr es Valenzuela and Russ Tedrake, "Whole-body Motion Planning with Centroidal Dynamics and Full Kinematics", *IEEE-RAS International Conference on Humanoid Robots*, 2014. [ [link](#) ]
8. Tobia Marcucci and Jack Umenberger and Pablo A. Parrilo and Russ Tedrake, "Shortest Paths in Graphs of Convex Sets", *arXiv preprint*, 2023. [ [link](#) ]

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

---

[Accessibility](#)

  Russ Tedrake, 2024

# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 6

# Model Systems with Stochasticity

My goals for this chapter are to build intuition for the beautiful and rich behavior of nonlinear dynamical system that are subjected to random (noise/disturbance) inputs. So far we have focused primarily on systems described by

$$\dot{\mathbf{x}}(t) = f(\mathbf{x}(t), \mathbf{u}(t)) \quad \text{or} \quad \mathbf{x}[n+1] = f(\mathbf{x}[n], \mathbf{u}[n]).$$

In this chapter, I would like to broaden the scope to think about

$$\dot{\mathbf{x}}(t) = f(\mathbf{x}(t), \mathbf{u}(t), \mathbf{w}(t)) \quad \text{or} \quad \mathbf{x}[n+1] = f(\mathbf{x}[n], \mathbf{u}[n], \mathbf{w}[n]),$$

where this additional input  $\mathbf{w}$  is the (vector) output of some random process. In other words, we can begin thinking about stochastic systems by simply understanding the dynamics of our existing ODEs subjected to an additional random input.

This form is extremely general as written.  $\mathbf{w}(t)$  can represent time-varying random disturbances (e.g. gusts of wind), or even constant model errors/uncertainty. One thing that we are not adding, yet, is measurement uncertainty. There is a great deal of work on observability and state estimation that study the question of how you can infer the true state of the system given noise sensor readings. For this chapter we are assuming perfect measurements of the full state, and are focused instead on the way that "process noise" shapes the long-term dynamics of the system.

I will also stick primarily to discrete time dynamics for this chapter, simply because it is easier to think about the output of a discrete-time random process,  $\mathbf{w}[n]$ , than a  $\mathbf{w}(t)$ . But you should know that all of the ideas work in continuous time, too. Also, most of our examples will take the form of *additive noise*:

$$\mathbf{x}[n+1] = f(\mathbf{x}[n], \mathbf{u}[n]) + \mathbf{w}[n],$$

which is a particular useful and common specialization of our general form. And this form doesn't give up much -- the disturbance on step  $k$  can pass through the nonlinear function  $f$  on step  $k+1$  giving rich results -- but is often much easier to work with.

When implementing stochastic systems in Drake, we aim to be [rigorous about](#)

randomness, to facilitate advanced algorithms that are written against stochastic systems and to enable, e.g. random simulation roll outs but still enable deterministic replay (simulation is deterministic given the random seed).

## 6.1 THE MASTER EQUATION

Let's start by looking at some simple examples.

### Example 6.1 (A Bistable System + Noise)

Let's consider (a time-reversed version of) one of my favorite one-dimensional systems:

$$\dot{x} = x - x^3.$$

This deterministic system has stable fixed points at  $x^* = \{-1, 1\}$  and an unstable fixed point at  $x^* = 0$ .

A reasonable discrete-time approximation of these dynamics, now with additive noise, is

$$x[n+1] = x[n] + h(x[n] - x[n]^3 + w[n]).$$

When  $w[n] = 0$ , this system has the same fixed points and stability properties as the continuous time system. But let's examine the system when  $w[n]$  is instead the result of a *zero-mean Gaussian white noise process*, defined by:

$$\begin{aligned} \forall n, E[w[n]] &= 0, \\ E[w[i]w[j]] &= \begin{cases} \sigma^2, & \text{if } i = j, \\ 0, & \text{otherwise.} \end{cases} \end{aligned}$$

Here  $\sigma$  is the standard deviation of the Gaussian.

When you simulate this system for small values of  $\sigma$ , you will see trajectories move roughly towards one of the two fixed points (for the deterministic system), but every step is modified by the noise. In fact, even if the trajectory were to arrive exactly at what was once a fixed point, it is almost surely going to move again on the very next step. In fact, if we plot many runs of the simulation from different initial conditions all on the same plot, we will see something like the figure below.



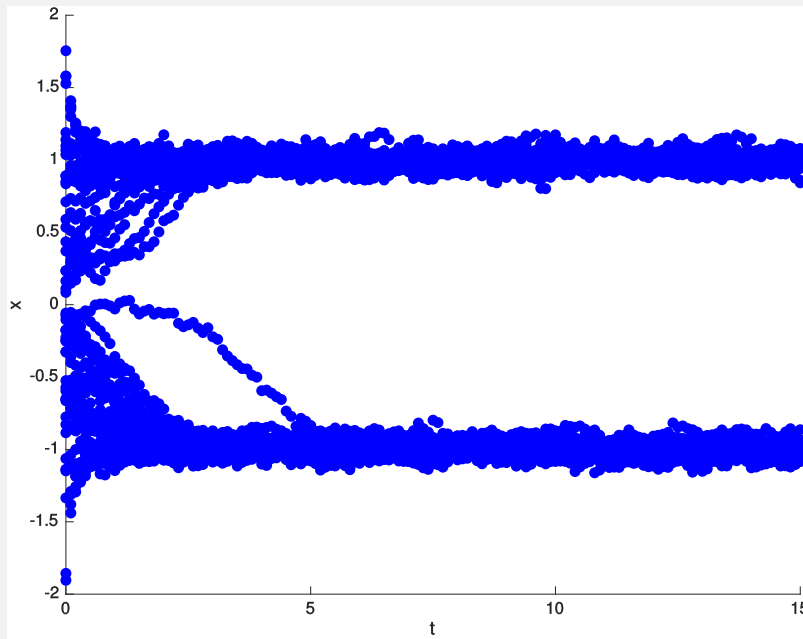


Figure 6.1 - Simulation of the bistable system with noise ( $h = 0.1, \sigma = 0.3$ ) from many initial conditions.

During any individual simulation, the state jumps around randomly for all time, and could even transition from the vicinity of one fixed point to the other fixed point. Another way to visualize this output is to animate a [histogram](#) of the particles over time.

Figure 6.2 - Histogram of the states of the bistable system with noise ( $\sigma = 1$ ) after simulating from random initial conditions until  $t = 20$ .

You can run this demo for yourself:

Let's take a moment to appreciate the implications of what we've just observed. Every time that we've analyzed a system to date, we've asked questions like "given  $x[0]$ , what is the long-term behavior of the system,  $\lim_{n \rightarrow \infty} x[n]$ ?", but now  $x[n]$  is a *random variable*. The trajectories of this system do not converge, and the system does not exhibit any form of stability that we've introduced so far.

All is not lost. If you watch the animation closely, you might notice the *distribution* of this random variable is actually very well behaved. This is the key idea for this chapter.

Let us use  $p_n(x)$  to denote the *probability density function* over the random variable  $x$  at time  $n$ . Note that this density function must satisfy

$$\int_{-\infty}^{\infty} p_n(x) dx = 1.$$

It is actually possible to write the *dynamics of the probability density* with the simple relation

$$p_{n+1}(x) = \int_{-\infty}^{\infty} p(x|x') p_n(x') dx',$$

where  $p(x|x')$  encodes the stochastic dynamics as a conditional distribution of the next state (here  $x$ ) as a function of the current state (here  $x'$ ). Dynamical systems that can be encoded in this way are known as *continuous-state Markov Processes*, and the governing equation above is often referred to as the "[master equation](#)" for the stochastic process. In fact this update is even linear(!) ; a fact that can

enable closed-form solutions to some impressive long-term statistics, like mean time to failure or first passage times[1]. Unfortunately, it is often difficult to perform the integral in closed form, so we must often resort to discretization or numerical analysis.

### Example 6.2 (The Logistic Map)

In fact, one does not actually need stochastic dynamics in order for the dynamics of the distribution to be the meaningful object of study; random initial conditions can be enough. One of the best examples comes from perhaps the simplest and most famous example of a chaotic system: the logistic map. This example is described beautifully in [2].

Consider the following difference equation:

$$x[n+1] = 4x[n](1 - x[n]),$$

which we will study over the (invariant) interval  $x \in [0, 1]$ .

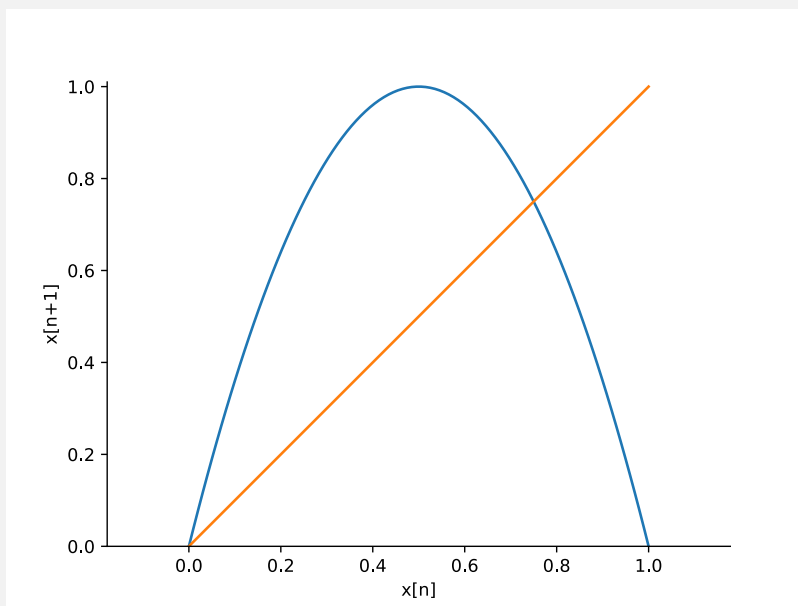


Figure 6.3 - The discrete-time dynamics of the "logistic map" (plotted alongside the line  $x[n+1] = x[n]$ ).

It takes only a moment of tracing your finger along the plot using the "staircase method" to see what makes this system so interesting -- rollouts from a single initial condition end up bouncing all over the interval  $(0, 1)$ , and neighboring initial conditions will end up taking arbitrarily different trajectories (this is the hallmark of a "chaotic" system).

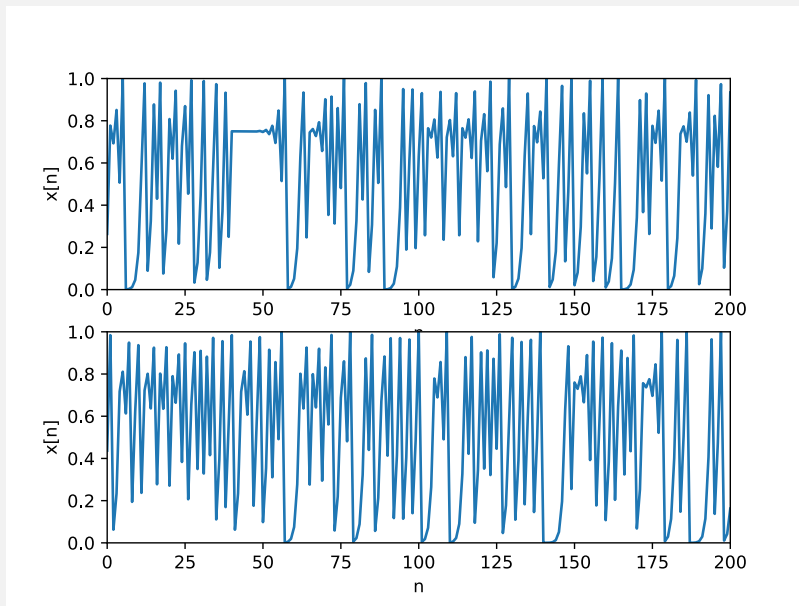


Figure 6.4 - Two simulations of the logistic map from different initial conditions. Remember, there is no stochasticity here -- the dynamics are entirely deterministic!

Here's what's completely fascinating -- even though the dynamics of any one initial condition for this system are extremely complex, if we study the dynamics of a distribution of states through the system, they are surprisingly simple and well-behaved. This system is one of the rare cases when we can write the master equation in closed form[2]:

$$p_{n+1}(x) = \frac{1}{4\sqrt{1-x}} \left[ p_n \left( \frac{1}{2} - \frac{1}{2}\sqrt{1-x} \right) + p_n \left( \frac{1}{2} + \frac{1}{2}\sqrt{1-x} \right) \right].$$

Moreover, this master equation has a steady-state solution:

$$p_*(x) = \frac{1}{\pi\sqrt{x(1-x)}}.$$

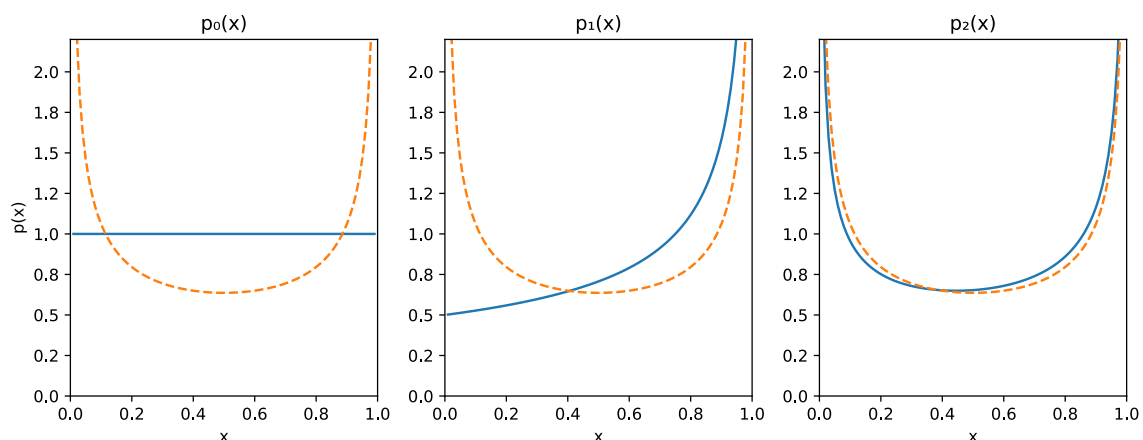


Figure 6.5 - Plotting the (closed-form) evolution of the master equation for the logistic map, initialized with  $p_0(x) = 1$  for  $x \in [0, 1]$ , reveals surprising simplicity, and rapid convergence to a stationary distribution (dashed orange line).

For this system (and many chaotic systems), the dynamics of a single initial condition are complicated, but the dynamics a *distribution* of initial conditions are

beautiful and simple!

Note: when the dynamical system under study has deterministic dynamics (but a distribution of initial conditions), the linear map given by the master equation is known as the *Perron-Frobenius operator*, and it gives rise to the Liouville equation that we will study later in the text.

The slightly more general form of the master equation, which works for multivariate distributions with state-domain  $\mathcal{X}$ , and systems with control inputs  $\mathbf{u}$ , is

$$p_{n+1}(\mathbf{x}) = \int_{\mathcal{X}} p(\mathbf{x}|\mathbf{x}', \mathbf{u}) p_n(\mathbf{x}') d\mathbf{x}'.$$

This is a (continuous-state) Markov Decision Process.

Continuous-time formulations are also possible -- these lead to the so-called *Fokker-Planck equation*.

## 6.2 STATIONARY DISTRIBUTIONS

In the example above, the histogram is our numerical approximation of the probability density. The logistic map example had the remarkable property that, although the individual trajectories of the system *do not* converge, the probability distribution actually *does* converge to what's known as a *stationary distribution* -- a fixed point of the master equation. Instead of thinking about the dynamics of the trajectories, we need to start thinking about the dynamics of the distribution.

The most important example of this analysis is for systems with linear dynamics and additive Gaussian noise; for this case we have closed-form solutions (in arbitrary dimensions). But to keep things simple, let's make sure we understand the scalar case first.

### Example 6.3 (A One-dimensional Linear Gaussian System)

Let's consider the one-dimensional linear system with additive noise:

$$x[n+1] = ax[n] + w[n].$$

When  $w[n] = 0$ , the system is stable (to the origin) for  $-1 < a < 1$ . Let's make sure that we can understand the dynamics of the master equation for the case when  $w[n]$  is Gaussian white noise with standard deviation  $\sigma$ .

First, recall that the probability density function of a Gaussian with mean  $\mu$  is given by

$$p_w(w) = \frac{1}{\sqrt{2\pi\sigma_w^2}} e^{-\frac{(w-\mu)^2}{2\sigma_w^2}}.$$

When conditioned on  $x[n]$ , the distribution given by the dynamics subjected to mean-zero Gaussian white noise is simply another Gaussian, with the mean given by  $ax[n]$ :

$$p(x[n+1]|x[n]) = \frac{1}{\sqrt{2\pi\sigma_w^2}} e^{-\frac{(x[n+1]-ax[n])^2}{2\sigma_w^2}},$$

yielding the master equation

$$p_{n+1}(x) = \frac{1}{\sqrt{2\pi\sigma_w^2}} \int_{-\infty}^{\infty} e^{-\frac{(x-ax')^2}{2\sigma_w^2}} p_n(x') dx'.$$

Now **here's the magic**. Let's push a distribution,  $p_n(x)$ , which is zero-mean, with standard deviation  $\sigma_n$ , through the master equation:

$$\begin{aligned} p_{n+1}(x) &= \frac{1}{\sqrt{2\pi\sigma_w^2}} \frac{1}{\sqrt{2\pi\sigma_n^2}} \int_{-\infty}^{\infty} e^{-\frac{(x-ax')^2}{2\sigma_w^2}} e^{-\frac{(x')^2}{2\sigma_n^2}} dx', \\ &= \frac{1}{\sqrt{2\pi(\sigma_w^2 + a^2\sigma_n^2)}} e^{-\frac{x^2}{2(\sigma_w^2 + a^2\sigma_n^2)}}. \end{aligned}$$

The result is another mean-zero Gaussian with  $\sigma_{n+1}^2 = \sigma_w^2 + a^2\sigma_n^2$ . This result generalizes to the multi-variate case, too, and might be familiar to you e.g. from the process update of the [Kalman filter](#).

Taking it a step further, we can see that a stationary distribution for this system is given by a mean-zero Gaussian with

$$\sigma_*^2 = \frac{\sigma_w^2}{1 - a^2}.$$

Note that this distribution is well defined when  $-1 < a < 1$  (only when the system is stable).

The stationary distribution of the linear Gaussian system reveals the fundamental and general balance between two terms in the governing equations of any stochastic dynamical system: the stability of the deterministic system is bringing trajectories together (smaller  $a$  means faster convergence of the deterministic system and results in a more narrow distribution), but the noise in the system is forcing trajectories apart (larger  $\sigma$  means larger noise and results in a wider distribution).

Given how rich the dynamics can be for deterministic nonlinear systems, you can probably imagine that the possible long-term dynamics of the probability are also extremely rich. If we simply flip the signs in the cubic polynomial dynamics we examined above, we'll get our next example:

### Example 6.4 (The Cubic Example + Noise)

Now let's consider the discrete-time approximation of

$$\dot{x} = -x + x^3,$$

again with additive noise:

$$x[n+1] = x[n] + h(-x[n] + x[n]^3 + w[n]).$$

With  $w[n] = 0$ , the system has only a single stable fixed point (at the origin), whose deterministic region of attraction is given by  $x \in (-1, 1)$ . If we again simulate the system from a set of random initial conditions and plot the histogram, we will see something like the figure below.

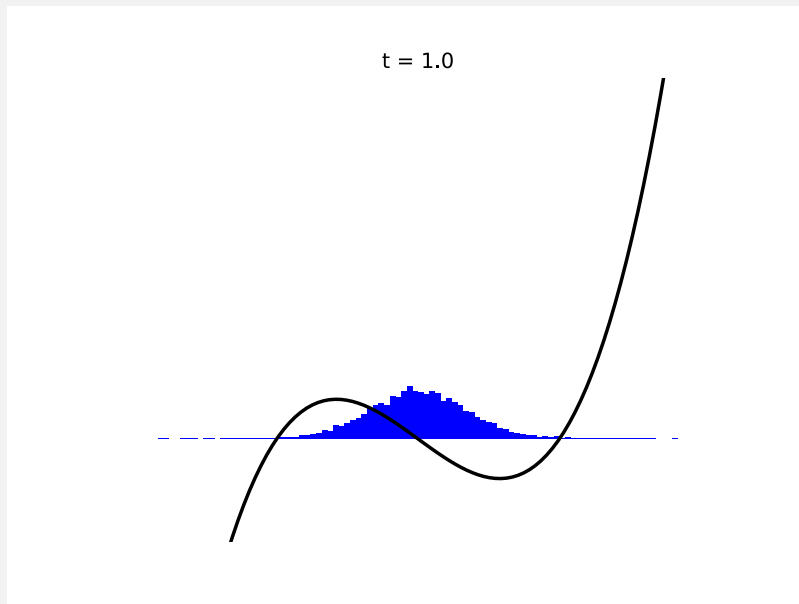


Figure 6.6 - Histogram of the states of the bistable system with noise ( $h = 0.1, \sigma = 1.5$ ) after simulating from random initial conditions until  $t = 1$ .  
 Figure 6.7 - Histogram of the states of the bistable system with noise ( $\sigma = 1.5$ ).

Be sure to watch the animation. Or better yet, run the simulation for yourself by changing the sign of the derivative in the bistability example and re-running:

You can run this demo for yourself:

What is the stationary distribution for this system? In fact, there isn't one. Although we initially collect probability density around the stable fixed point, you should notice a slow leak -- on every step there is some probability of transitioning past unstable fixed points and getting driven by the unstable dynamics off towards infinity. If we run the simulation long enough, there won't be any probability density left at  $x = 0$ .

### Example 6.5 (The Stochastic Van der Pol Oscillator)

One more example; this is a fun one. Let's think about one of the simplest examples that we had for a system that demonstrates limit cycle stability -- the Van der Pol oscillator -- but now we'll add Gaussian white noise,

$$\ddot{q} + (q^2 - 1)\dot{q} + q = w(t),$$

Here's the question: if we start with a small set of initial conditions centered around one point on the limit cycle, then what is the long-term behavior of this distribution?

Figure 6.8 - Randomly sampled initial conditions pictured with the stable limit cycle of the Van der Pol oscillator.

Since the long-term behavior of the deterministic system is periodic, it would be very logical to think that the state distribution for this stochastic system would fall into a stable periodic solution, too. But think about it a little more, and then watch the animation (or run the simulation yourself).

Figure 6.9 - Rollouts of the stochastic Van der Pol oscillator.

The explanation is simple: the periodic solution of the system is only *orbitally stable*; there is no stability along the limit cycle. So any disturbances that push the particles along the limit cycle will go unchecked. Eventually, the distribution will "mix" along the entire cycle. Perhaps surprisingly, this system that has a limit cycle stability when  $w = 0$  eventually reaches a stationary distribution (fixed point) in the master equation.

$t = 1000.0$

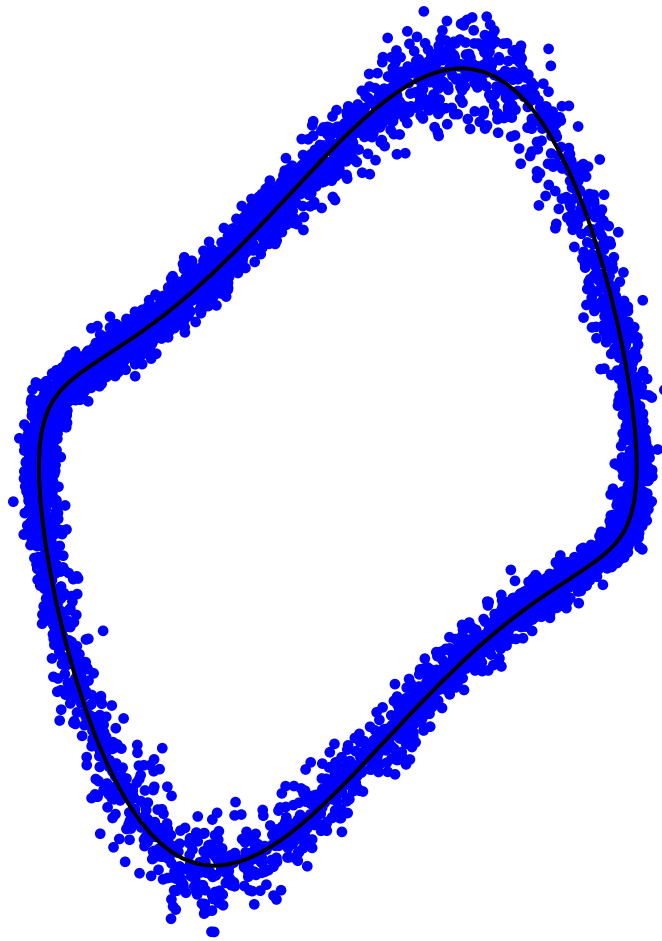


Figure 6.10 - Sampled approximation of the stationary distribution of the stochastic Van der Pol oscillator.

## 6.3 EXTENDED EXAMPLE: THE RIMLESS WHEEL ON ROUGH TERRAIN

My favorite example of a meaningful source of randomness on a model underactuated system is the rimless wheel rolling down stochastically "rough" terrain[1]. Generating interesting/relevant probabilistic models of terrain in general can be quite complex, but the rimless wheel makes it easy -- since the robot only contacts that ground at the point foot, we can model almost arbitrary rough terrain by simply taking the ramp angle,  $\gamma$ , to be a random variable. If we restrict our analysis to rolling only in one direction (e.g. only downhill), then we can even consider this ramp angle to be i.i.d.; after each footstep we will independently draw a new ramp angle  $\gamma[n]$  for the next step.

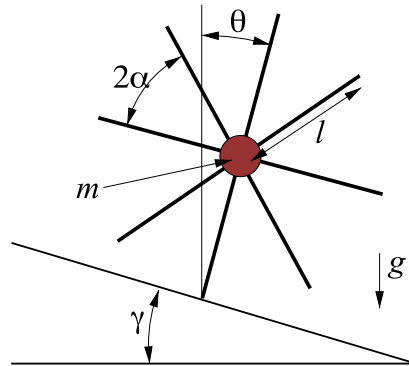


Figure 6.11 - The rimless wheel. The orientation of the stance leg,  $\theta$ , is measured clockwise from the vertical axis.

In our original analysis of the rimless wheel, we derived the "post-collision" return map -- mapping the angular velocity from the beginning of the stance phase to the (post-collision) angular velocity at the next stance phase. But now that the relative location of the ground is changing on every step, we instead write the "apex-to-apex" return map, which maps the angular velocity from one apex (the moment that the leg is vertical) to the next, which is given by:

$$\dot{\theta}[n+1] = \sqrt{\cos^2 \alpha \left( \dot{\theta}^2[n] + \frac{2g}{l}(1 - \cos(\alpha + \gamma[n])) \right) - \frac{2g}{l}(1 - \cos(\alpha - \gamma[n]))}.$$

More coming soon. Read the paper [1] and/or watch the [video](#).

## 6.4 NOISE MODELS FOR REAL ROBOTS/SYSTEMS.

Sensor models. Beam model from probabilistic robotics. RGB-D dropouts.

Perception subsystem. Output of a perception system is not Gaussian noise, it's missed detections/drop-outs...

Distributions over tasks/environments.

## REFERENCES

1. Katie Byl and Russ Tedrake, "Metastable Walking Machines", *International Journal of Robotics Research*, vol. 28, no. 8, pp. 1040-1064, August 1, 2009. [ [link](#) ]
2. Andrzej Lasota and Michael C Mackey, "Chaos, fractals, and noise: stochastic aspects of dynamics", Springer Science & Business Media , vol. 97, 2013.



[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

[Accessibility](#)

© Russ Tedrake, 2024

# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 7

# Dynamic Programming

In chapter 2, we spent some time thinking about the phase portrait of the simple pendulum, and concluded with a challenge: can we design a nonlinear controller to *reshape* the phase portrait, with a very modest amount of actuation, so that the upright fixed point becomes globally stable? With enough actuators and unbounded torque, feedback-cancellation solutions (e.g., invert gravity) can work well, but can also require an unnecessarily large amount of control effort. The energy-based swing-up control solutions presented for the Acrobot and cart-pole systems are considerably more appealing, but required some cleverness and might not scale to more complicated systems. Here we investigate another approach to the problem, using computational optimal control to synthesize a feedback controller directly.

### 7.1 FORMULATING CONTROL DESIGN AS AN OPTIMIZATION

In this chapter, we will introduce optimal control. *Reinforcement learning* is the machine learning name for optimal control; we'll discuss that machine learning perspective later in the notes. Optimal control is powerful for a number of reasons. First and foremost, it is very general - allowing us to specify the goal of control equally well for fully-actuated or underactuated, linear or nonlinear, deterministic or stochastic, and continuous or discrete systems. Second, it permits concise descriptions of potentially very complex desired behaviors, specifying the goal of control as a scalar objective (plus a list of constraints). Finally, and most importantly, optimal control is very amenable to numerical solutions. [\[1\]](#) is a fantastic reference on this material for those who want a somewhat rigorous treatment; [\[2\]](#) is an excellent (free) reference for those who want something more approachable.

The fundamental idea in optimal control is to formulate the goal of control as the *long-term* optimization of a scalar cost function. Let's introduce the basic concepts by considering a system that is even simpler than the simple pendulum.

## Example 7.1 (Optimal Control Formulations for the Double Integrator)

Consider the "double integrator" system

$$\ddot{q} = u, \quad |u| \leq 1.$$

If you would like a mechanical analog of the system (I always do), then you can think about this as a unit mass brick moving along the  $x$ -axis on a frictionless surface, with a control input which provides a horizontal force,  $u$ . The task is to design a control system,  $u = \pi(\mathbf{x}, t)$ ,  $\mathbf{x} = [q, \dot{q}]^T$  to regulate this brick to  $\mathbf{x} = [0, 0]^T$ .



Figure 7.1 - The double integrator as a unit-mass brick on a frictionless surface

In order to formulate this control design problem using optimal control, we must define a scalar objective which scores the long-term performance of running each candidate control policy,  $\pi(\mathbf{x}, t)$ , from each initial condition,  $(\mathbf{x}_0, t_0)$ , and a list of constraints that must be satisfied. For the task of driving the double integrator to the origin, one could imagine a number of optimal control formulations which would accomplish the task, e.g.:

- Minimum time:  $\min_{\pi} t_f$ , subject to  $\mathbf{x}(t_0) = \mathbf{x}_0$ ,  $\mathbf{x}(t_f) = \mathbf{0}$ .
- Quadratic cost:  $\min_{\pi} \int_{t_0}^{\infty} [\mathbf{x}^T(t) \mathbf{Q} \mathbf{x}(t)] dt$ ,  $\mathbf{Q} \succ 0$ .

The first is a constrained optimization formulation which optimizes time, and the second accrues a penalty at every instance according to the distance that the state is away from the origin (with weights defined by the matrix  $\mathbf{Q}$ ), and therefore encourages trajectories that go more directly towards the goal, possibly at the expense of requiring a longer time to reach the goal. In fact the quadratic cost will result in an exponential approach to the goal, whereas the minimum-time formulation will arrive at the goal in finite time. Note that both optimization problems only have well-defined solutions if it is possible for the system to actually reach the origin, otherwise the minimum-time problem cannot satisfy the terminal constraint, and the integral in the quadratic cost would not converge to a finite value as time approaches infinity (fortunately the double-integrator system is **controllable**, and therefore can be driven to the goal in finite time).

Note that the input limits,  $|u| \leq 1$  are also required to make this problem well-posed; otherwise both optimizations would result in the optimal policy using infinite control input to approach the goal infinitely fast. Besides input limits, another common approach to limiting the control effort is to add an additional quadratic cost on the input (or "effort"), e.g.  $\int [\mathbf{u}^T(t) \mathbf{R} \mathbf{u}(t)] dt$ ,  $\mathbf{R} \succ 0$ . This could be added to either formulation above. We will examine many of these formulations in some detail in the examples worked out at the end of this chapter.

Optimal control has a long history in robotics. It started out motivated by achieving peak performance in niche applications like programming minimum-time paths for a pick-and-place robot in a factory. But it is the **generality** of the approach (one can program almost any task using similar tools) that has made optimization-based and now machine-learning-based control dominate the way that state-of-the-art robot control systems are developed.

## Example 7.2 (The minimum time problem for the double integrator)

For more intuition, let's do an informal derivation of the solution to the minimum time problem for the double integrator with input constraints:

$$\begin{aligned} & \text{minimize}_{\pi} \quad t_f \\ & \text{subject to} \quad \mathbf{x}(t_0) = \mathbf{x}_0, \\ & \quad \quad \quad \mathbf{x}(t_f) = \mathbf{0}, \\ & \quad \quad \quad \ddot{q}(t) = u(t), \\ & \quad \quad \quad |u(t)| \leq 1. \end{aligned}$$

What behavior would you expect an optimal controller to exhibit?

Your intuition might tell you that the best thing that the brick can do, to reach the goal in minimum time with limited control input, is to accelerate maximally towards the goal until reaching a critical point, then hitting the brakes in order to come to a stop exactly at the goal. This is called a *bang-bang* control policy; these are often optimal for systems with bounded input, and it is in fact optimal for the double integrator although we will not prove it until we have developed more tools.

Let's work out the details of this bang-bang policy. First, we can figure out the states from which, when the brakes are fully applied, the system comes to rest precisely at the origin. Let's start with the case where  $q(0) < 0$ , and  $\dot{q}(0) > 0$ , and "hitting the brakes" implies that  $u = -1$ . Integrating the equations, we have

$$\begin{aligned} \ddot{q}(t) &= u = -1, \\ \dot{q}(t) &= \dot{q}(0) - t, \\ q(t) &= q(0) + \dot{q}(0)t - \frac{1}{2}t^2. \end{aligned}$$

Substituting  $t = \dot{q}(0) - \dot{q}$  into the solution reveals that the system trajectories are parabolic arcs:

$$q = -\frac{1}{2}\dot{q}^2 + c_-,$$

with  $c_- = q(0) + \frac{1}{2}\dot{q}^2(0)$ .

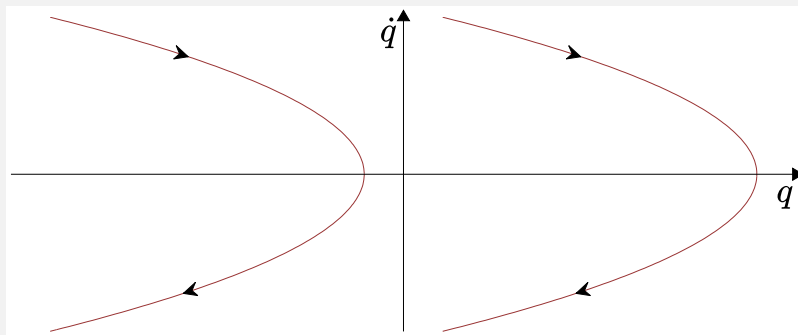


Figure 7.2 - Two solutions for the system with  $u = -1$

Similarly, the solutions for  $u = 1$  are  $q = \frac{1}{2}\dot{q}^2 + c_+$ , with  $c_+ = q(0) - \frac{1}{2}\dot{q}^2(0)$ .

Perhaps the most important of these trajectories are the ones that pass directly through the origin (e.g.,  $c_- = 0$ ). Following our initial logic, if the system is going slower than this  $\dot{q}$  for any  $q$ , then the optimal thing to do is to slam on the accelerator ( $u = -\text{sgn}(q)$ ). If it's going faster than the  $\dot{q}$  that we've solved for, then still the best thing to do is to brake; but inevitably the system will overshoot the

origin and have to come back. We can summarize this policy with:

$$u = \begin{cases} +1, & \text{if } (\dot{q} < 0 \text{ and } q \leq \frac{1}{2}\dot{q}^2) \text{ or } (\dot{q} \geq 0 \text{ and } q < -\frac{1}{2}\dot{q}^2), \\ 0, & \text{if } q = 0 \text{ and } \dot{q} = 0, \text{ or} \\ -1, & \text{otherwise.} \end{cases}$$

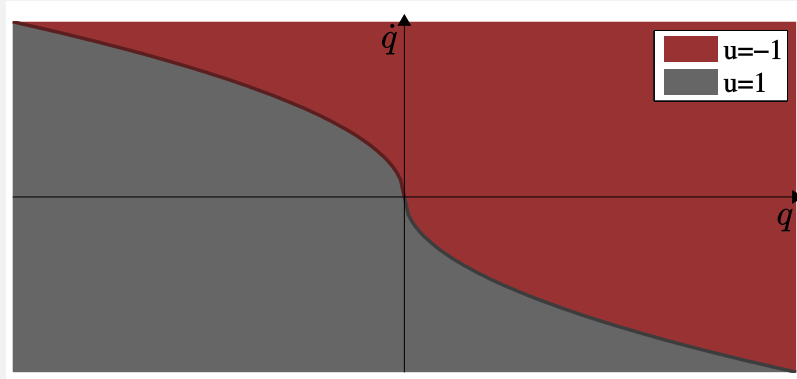


Figure 7.3 - Candidate optimal (bang-bang) policy for the minimum-time double integrator problem.

We can illustrate some of the optimal solution trajectories:

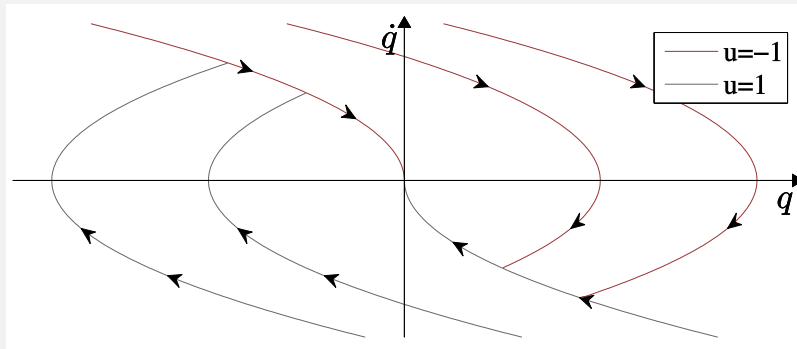
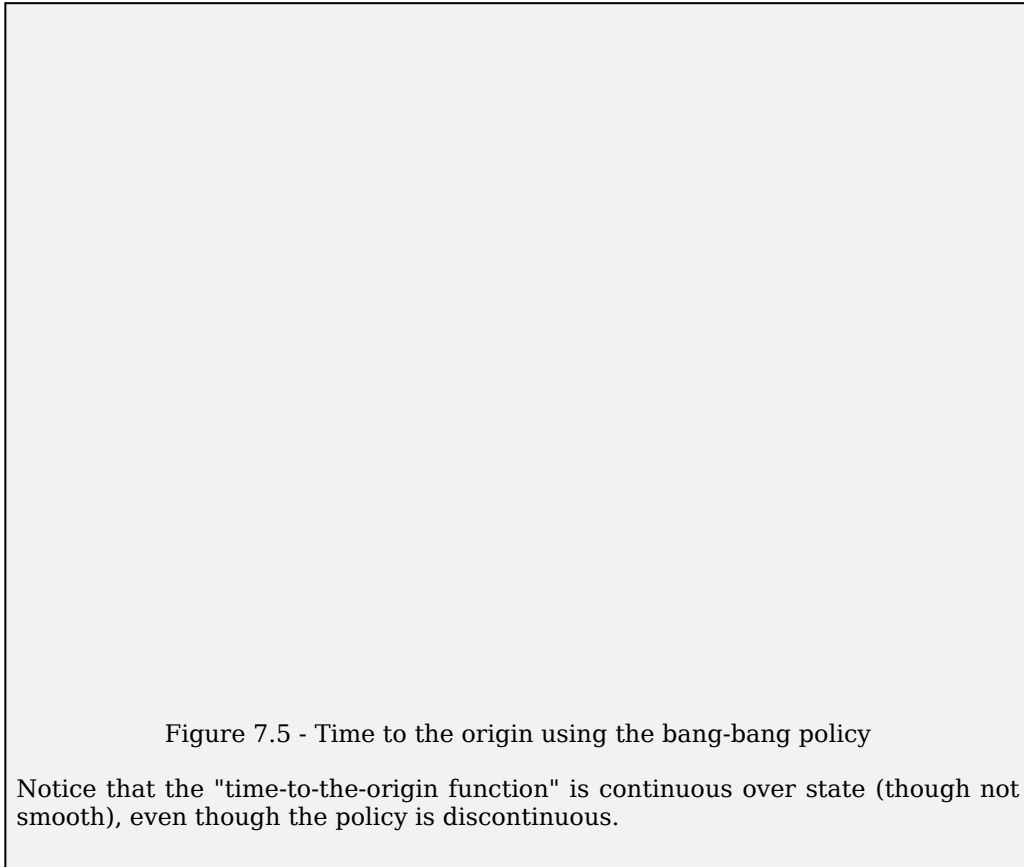


Figure 7.4 - Solution trajectories of system using the optimal policy

And for completeness, we can compute the optimal time to the goal by solving for the amount of time required to reach the switching surface plus the amount of time spent moving along the switching surface to the goal. With a little algebra, you will find that the time to the goal,  $J(\mathbf{x})$ , is given by

$$J(\mathbf{x}) = \begin{cases} 2\sqrt{\frac{1}{2}\dot{q}^2 - q - \dot{q}}, & \text{for } u = +1 \text{ regime,} \\ 0, & \text{for } u = 0, \\ \dot{q} + 2\sqrt{\frac{1}{2}\dot{q}^2 + q}, & \text{for } u = -1, \end{cases}$$

which is plotted here:



### 7.1.1 Additive cost

As we begin to develop theoretical and algorithmic tools for optimal control, we will see that some formulations are much easier to deal with than others. One important example is the dramatic simplification that can come from formulating objective functions using *additive cost*, because they often yield recursive solutions. In the additive cost formulation, the long-term objective for a trajectory can be written as

$$\int_0^T \ell(x(t), u(t)) dt,$$

where  $\ell()$  is the instantaneous cost (also referred to as the "running cost"), and  $T$  can be either a finite real number or  $\infty$ . We will call a problem specification with a finite  $T$  a "finite-horizon" problem, and  $T = \infty$  an "infinite-horizon" problem. Problems and solutions for infinite-horizon problems tend to be more elegant, but care is required to make sure that the integral converges for the optimal controller (typically by having an achievable goal that allows the robot to accrue zero cost, or by having costs that discount/decay to zero as time increases).

The "quadratic cost" formulation we considered in the double integrator example above is clearly written as an additive cost. At first glance, our "minimum-time" formulation doesn't appear to be of this form, but we actually can write it as an additive cost problem using an infinite horizon and the instantaneous cost

$$\ell(x, u) = \begin{cases} 0 & \text{if } x = 0, \\ 1 & \text{otherwise.} \end{cases}$$

We will examine a number of approaches to solving optimal control problems throughout the next few chapters. For the remainder of this chapter, we will focus on additive-cost problems and their solution via *dynamic programming*.

## 7.2 OPTIMAL CONTROL AS GRAPH SEARCH

For systems with continuous states and continuous actions, dynamic programming is a set of theoretical ideas surrounding additive-cost optimal control problems. For systems with a finite, discrete set of states and a finite, discrete set of actions, dynamic programming also represents a set of very efficient numerical *algorithms* which can compute optimal feedback controllers. Many of you will have seen it introduced before as a tool for graph search.

Imagine you have a directed graph with states (or nodes)  $\{s_1, s_2, \dots\} \in S$  and "actions" associated with edges labeled as  $\{a_1, a_2, \dots\} \in A$ , as in the following trivial example:

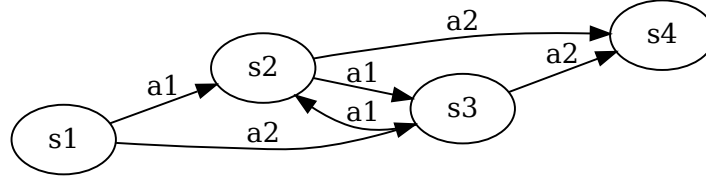


Figure 7.6 - A simple directed graph.

Let us also assume that each edge has an associated weight or cost, using  $\ell(s, a)$  to denote the cost of being in state  $s$  and taking action  $a$ . Furthermore we will denote the transition "dynamics" using

$$s[n+1] = f(s[n], a[n]).$$

For instance, in the graph above,  $f(s_1, a_1) = s_2$ .

There are many algorithms for finding (or approximating) the optimal path from a start to a goal on directed graphs. In dynamic programming, the key insight is that we can find the shortest path from every node by solving recursively for the *optimal cost-to-go* (the cost that will be accumulated when running the optimal controller), which we'll denote  $J^*(s)$ , from every node to the goal. One such algorithm starts by initializing an estimate  $\hat{J}^*(s) = 0$  for all  $s_i$ , then proceeds with an iterative algorithm which sets

$$\forall i \quad \hat{J}^*(s_i) \leftarrow \min_{a \in A} [\ell(s_i, a) + \hat{J}^*(f(s_i, a))]. \quad (1)$$

This notation assumes that the update  $f(s, a)$  and cost  $\ell(s, a)$  are defined for every state-action pair; we can add self transitions at each state for any missing actions which have zero cost for the goal state and sufficiently high cost anywhere else. In software,  $\hat{J}^*(s)$  can be represented as a vector with dimension equal to the number of discrete states, and equation (1) gives us an update for the entire vector. This algorithm, appropriately known as *value iteration*, is guaranteed to converge to the optimal cost-to-go up to a constant factor,  $\hat{J}^* \rightarrow J^* + c$  [3], and in practice does so rapidly. Typically the update is done *in batch* -- e.g. the estimate is updated for all  $i$  at once -- but the *asynchronous* version where states are updated one at a time is also known to converge, so long as every state is eventually updated infinitely often.

Value iteration is an amazingly simple algorithm, but it accomplishes something quite amazing: it efficiently computes the long-term cost of an optimal policy from *every* state by iteratively evaluating the one-step cost. If we know the optimal cost-to-go, then it's easy to extract the optimal policy,  $a = \pi^*(s)$ :

$$\pi^*(s_i) = \operatorname{argmin}_a [\ell(s_i, a) + J^*(f(s_i, a))]. \quad (2)$$

It's a simple algorithm, but playing with an example can help our intuition.

### Example 7.3 (Grid World)

Imagine a robot living in a grid (finite state) world [4]. The robot wants to get to the goal location, but has to negotiate cells with obstacles (denoted by the red pit in the animation below). The robot's actions are to move up, down, left, right, or do nothing.

[Click here for the animation.](#)

Figure 7.7 - The grid-world minimum-time problem. The goal state has a cost of zero, the obstacles have a cost of 10, and every other state has a cost of 1. The first frame shows the one-step cost, and the next frames show repeated applications of the value iteration update (1) -- until convergence.

As always, I encourage you to run the notebook and play with the code yourself:

### Example 7.4 (Dynamic Programming for the Double Integrator)

You can run value iteration for the double integrator (using barycentric interpolation to discretize the continuous state space, as we'll describe below) in **DRAKE** using:

Please do take some time to try different cost functions by editing the code yourself.

Let's take a minute to appreciate how amazing this is. Our solution to finding the optimal controller for the double integrator wasn't all that hard, but it required some mechanical intuition and solutions to differential equations. The resulting policy was non-trivial -- bang-bang control with a parabolic switching surface. The value iteration algorithm doesn't use any of this directly -- it's a simple algorithm for graph search. But remarkably, it can generate effectively the same policy with just a few moments of computation.

It's important to note that there *are* some differences between the computed policy and the optimal policy that we derived, due to discretization errors. We will ask you to explore these in the exercises.

The real value of this numerical solution, however, is unlike our analytical solution for the double integrator, we can apply this same algorithm to any number of dynamical systems virtually without modification. Let's apply it now to the simple pendulum, which was intractable analytically.

### Example 7.5 (Dynamic Programming for the Simple Pendulum)

You can run value iteration for the simple pendulum (again using barycentric interpolation) in **DRAKE** using:

Again, you can easily try different cost functions by editing the code yourself.

## 7.3 CONTINUOUS DYNAMIC PROGRAMMING

I find the graph search algorithm extremely satisfying as a first step, but also become quickly frustrated by the limitations of the discretization required to use it. In many cases, we can do better; coming up with algorithms which work more



natively on continuous dynamical systems. We'll explore those extensions in this section.

### 7.3.1 The Hamilton-Jacobi-Bellman Equation

It's important to understand that the value iteration equations, equations (1) and (2), are more than just an algorithm. They are also sufficient conditions for optimality: if we can produce a  $J^*$  and  $\pi^*$  which satisfy these equations, then  $\pi^*$  must be an optimal controller. There are an analogous set of conditions for the continuous systems. For a system  $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$  and an infinite-horizon additive cost  $\int_0^\infty \ell(\mathbf{x}, \mathbf{u})dt$ , we have:

$$0 = \min_{\mathbf{u}} \left[ \ell(\mathbf{x}, \mathbf{u}) + \frac{\partial J^*}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) \right], \quad (3)$$

$$\pi^*(\mathbf{x}) = \operatorname{argmin}_{\mathbf{u}} \left[ \ell(\mathbf{x}, \mathbf{u}) + \frac{\partial J^*}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) \right]. \quad (4)$$

Equation 3 is known as the *Hamilton-Jacobi-Bellman* (HJB) equation. [5] gives an informal derivation of these equations as the limit of a discrete-time approximation of the dynamics, and also gives the famous "sufficiency theorem". The treatment in [5] is for the finite-horizon case; I have modified it to one particular form of an infinite-horizon case here.

Technically, a Hamilton-Jacobi equation is a PDE whose *time derivative* depends on the first-order partial derivative over state, which we see in the finite-time derivation; Eq 3 is the steady-state solution of the Hamilton-Jacobi equation.

#### Theorem 7.1 - HJB Sufficiency Theorem (stated informally)

Consider a system  $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$  and an infinite-horizon additive cost  $\int_0^\infty \ell(\mathbf{x}, \mathbf{u})dt$ , with  $f$  and  $\ell$  continuous functions, and  $\ell$  a strictly-positive-definite function that obtains zero only at a unique state  $\mathbf{x}^*$ . Suppose  $J(\mathbf{x})$  is a solution to the HJB equation:  $J$  is continuously differentiable in  $\mathbf{x}$  and satisfies

$$0 = \min_{\mathbf{u} \in U} \left[ \ell(\mathbf{x}, \mathbf{u}) + \frac{\partial J}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) \right], \quad \text{for all } \mathbf{x}.$$

Further assume that  $J(\mathbf{x})$  is positive definite and that  $\pi(\mathbf{x})$  is the minimizer for all  $\mathbf{x}$ . Then, under some technical conditions on the existence and boundedness of solutions, we have that  $J(\mathbf{x}) - J(\mathbf{x}^*)$  is the optimal cost-to-go and  $\pi$  is an optimal policy.

Here is a more formal version from a personal communication with [Sasha Megretski](#).

Given an open subset  $\Omega \subset \mathbb{R}^n$ , with a selected element  $x^*$ , a subset  $U \subset \mathbb{R}^m$ , continuous functions  $f: \Omega \times U \rightarrow \mathbb{R}^n$ ,  $g: \Omega \times U \rightarrow [0, \infty)$ , continuously differentiable function  $V: \Omega \rightarrow [0, \infty)$ , and a function  $\mu: \Omega \rightarrow U$ , such that

- $g(x, u)$  is strictly positive definite, in the sense that  $\lim_{k \rightarrow \infty} x_k = x^*$  for every sequence of vectors  $x_k \in \Omega$ ,  $u_k \in U$  ( $k = 1, 2, \dots$ ) such that  $\lim_{k \rightarrow \infty} g(x_k, u_k) = 0$ ;
- the function  $g_V: \Omega \times U \rightarrow \mathbb{R}$ , defined by

$$g_V(x, u) = g(x, u) + \frac{dV(x)}{dx} f(x, u), \quad (5)$$

is non-negative, and achieves zero value whenever  $u = \mu(x)$ ;

- for every  $x_0 \in \Omega$ , the system of equations

$$\dot{x}(t) = f(x(t), u(t)), \quad x(0) = x_0 \quad \left( \text{i.e., } x(t) = x_0 + \int_0^t f(x(\tau), u(\tau))d\tau \right) \quad (6)$$

$$u(t) = \mu(x(t)), \quad (7)$$

has at least one piecewise continuous solution  $x : [0, \infty) \rightarrow \Omega$ ,  $u : [0, \infty) \rightarrow U$ .

Then, for every  $x_0 \in \Omega$ , the functional

$$J(x, u) = \int_0^\infty g(x(t), u(t)) dt,$$

defined on the set of all piecewise continuous pairs  $(x, u)$  satisfying (6), achieves its minimal value of  $V(x_0) - V(x^*)$  at every piecewise continuous solution of (6), (7).

**Proof.** First, observe that, for all piecewise continuous  $(x, u)$  satisfying (6),

I. due to (5),

$$\int_0^T g(x(t), u(t)) dt = V(x_0) - V(x(T)) + \int_0^T g_V(x(t), u(t)) dt; \quad (8)$$

II. whenever  $J(x, u) < \infty$ , there exists a sequence of positive numbers  $T_k$ ,  $k = 1, 2, \dots$ ,  $T_k \rightarrow +\infty$ , such that  $g(x(T_k), u(T_k)) \rightarrow 0$ , which, according to assumption (a), implies  $x(T_k) \rightarrow x^*$  as  $k \rightarrow \infty$ .

In particular, for a piecewise continuous solution  $(x, u)$  of (6), satisfying, in addition, equation (7), condition (I) means

$$\int_0^T g(x(t), u(t)) dt = V(x_0) - V(x(T)) \leq V(x_0).$$

Since the upper bound  $V(x_0)$  does not depend on  $T$ , and  $g(x(t), u(t)) \geq 0$  is non-negative, we conclude that

$$\int_0^\infty g(x(t), u(t)) dt < \infty.$$

Therefore observation (\*\*) applies, and, for the corresponding  $T_k$ ,

$$\begin{aligned} J(x, u) &= \int_0^\infty g(x(t), u(t)) dt = \lim_{k \rightarrow \infty} \int_0^{T_k} g(x(t), u(t)) dt \\ &= \lim_{k \rightarrow \infty} V(x_0) - V(x(T_k)) = V(x_0) - V(x^*). \end{aligned}$$

Moreover, since  $g$  is non-negative, the inequality  $V(x_0) \geq V(x^*)$  must hold for every  $x_0 \in \Omega$ .

To finish the proof, for an arbitrary piecewise continuous solution of (6), equation (8), together with non-negativity of  $g_V$ , implies

$$J(x, u) = \int_0^T g(x(t), u(t)) dt \geq V(x_0) - V(x(T)).$$

When  $J(x, u) < \infty$ , applying this to  $T = T_k$ , where  $T_k$  are described in observation (II), and taking the limit as  $k \rightarrow \infty$  (and  $x(T_k) \rightarrow x^*$ ) yields  $J(x, u) \geq V(x_0) - V(x^*)$ .  $\square$

It is possible to prove sufficiency under different assumptions, too. The particular assumptions here were chosen to ensure that  $J(\mathbf{x}(0)) < \infty$  implies that  $\mathbf{x}(t) \rightarrow \mathbf{x}^*$ . As Sasha says, "without something like this, all sorts of counter-examples emerge."

As a tool for verifying optimality, the HJB equations are actually surprisingly easy to work with: we can verify optimality for an infinite-horizon objective without doing any integration; we simply have to check a derivative condition on the optimal cost-to-go function  $J^*$ . Let's see this play out on the double integrator example.

### Example 7.6 (HJB for the Double Integrator)

Consider the problem of regulating the double integrator (this time without input limits) to the origin using a quadratic cost:

$$\ell(\mathbf{x}, \mathbf{u}) = q^2 + \dot{q}^2 + u^2.$$

I claim (without derivation) that the optimal controller for this objective is

$$\pi(\mathbf{x}) = -q - \sqrt{3}\dot{q}.$$

To convince you that this is indeed optimal, I have produced the following cost-to-go function:

$$J(\mathbf{x}) = \sqrt{3}q^2 + 2q\dot{q} + \sqrt{3}\dot{q}^2.$$

Taking

$$\frac{\partial J}{\partial q} = 2\sqrt{3}q + 2\dot{q}, \quad \frac{\partial J}{\partial \dot{q}} = 2q + 2\sqrt{3}\dot{q},$$

we can write

$$\ell(\mathbf{x}, \mathbf{u}) + \frac{\partial J}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) = q^2 + \dot{q}^2 + u^2 + (2\sqrt{3}q + 2\dot{q})\dot{q} + (2q + 2\sqrt{3}\dot{q})u$$

This is a convex quadratic function in  $u$ , so we can find the minimum with respect to  $u$  by finding where the gradient with respect to  $u$  evaluates to zero.

$$\frac{\partial}{\partial u} \left[ \ell(\mathbf{x}, \mathbf{u}) + \frac{\partial J}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) \right] = 2u + 2q + 2\sqrt{3}\dot{q}.$$

Setting this equal to 0 and solving for  $u$  yields:

$$u^* = -q - \sqrt{3}\dot{q},$$

thereby confirming that our policy  $\pi$  is in fact the minimizer. Substituting  $u^*$  back into the HJB reveals that the right side does in fact simplify to zero. I hope you are convinced!

Note that evaluating the HJB for the time-to-go of the minimum-time problem for the double integrator will also reveal that the HJB is satisfied wherever that gradient is well-defined. This is certainly mounting evidence in support of our bang-bang policy being optimal, but since  $\frac{\partial J}{\partial \mathbf{x}}$  is not defined everywhere, it does not actually satisfy the requirements of the sufficiency theorem as stated above. Since cusps are commonplace in optimal value functions, one could say that the assumption in the sufficiency theorem that  $\frac{\partial J}{\partial \mathbf{x}}$  is defined everywhere makes our current statement very weak.

### 7.3.2 Solving for the minimizing control

We still face a few barriers to actually using the HJB in an algorithm. The first barrier is the minimization over  $\mathbf{u}$ . When the action set was discrete, as in the graph search version, we could evaluate the one-step cost plus cost-to-go for every possible action, and then simply take the best. For continuous action spaces, in general we cannot rely on the strategy of evaluating a finite number of possible  $\mathbf{u}$ 's to find the



minimizer.

All is not lost. In the quadratic cost double integrator example above, we were able to solve explicitly for the minimizing  $\mathbf{u}$  in terms of the cost-to-go. It turns out that this strategy will actually work for a number of the problems we're interested in, even when the system (which we are given) or cost function (which we are free to pick, but which should be expressive) gets more complicated.

Recall that I've already tried to convince you that a majority of the systems of interest are *control affine*, e.g. I can write

$$f(\mathbf{x}, \mathbf{u}) = f_1(\mathbf{x}) + f_2(\mathbf{x})\mathbf{u}.$$

We can make another simplification by restricting ourselves to running cost functions of the form

$$\ell(\mathbf{x}, \mathbf{u}) = \ell_1(\mathbf{x}) + \mathbf{u}^T \mathbf{R} \mathbf{u}, \quad \mathbf{R} = \mathbf{R}^T \succ 0.$$

In my view, this is not very restrictive - many of the cost functions that I find myself choosing to write down can be expressed in this form. Given these assumptions, we can write the HJB as

$$0 = \min_{\mathbf{u}} \left[ \ell_1(\mathbf{x}) + \mathbf{u}^T \mathbf{R} \mathbf{u} + \frac{\partial J}{\partial \mathbf{x}} [f_1(\mathbf{x}) + f_2(\mathbf{x})\mathbf{u}] \right].$$

Since this is a positive quadratic function in  $\mathbf{u}$ , if the system does not have any constraints on  $\mathbf{u}$ , then we can solve in closed-form for the minimizing  $\mathbf{u}$  by taking the gradient of the right-hand side:

$$\frac{\partial}{\partial \mathbf{u}} = 2\mathbf{u}^T \mathbf{R} + \frac{\partial J}{\partial \mathbf{x}} f_2(\mathbf{x}) = 0,$$

and setting it equal to zero to obtain

$$\mathbf{u}^* = -\frac{1}{2} \mathbf{R}^{-1} f_2^T(\mathbf{x}) \frac{\partial J}{\partial \mathbf{x}}.$$

If there are linear constraints on the input, such as torque limits, then more generally this could be solved (at any particular  $\mathbf{x}$ ) as a quadratic program.

Exploiting the ability to solve for the optimal action in closed form is also nice for generating benchmark problems for numerical optimal control, as it can be used for "converse optimal control" [6].

### Example 7.7 (Converse optimal control for a cubic polynomial)

Consider a one-dimensional system of the form  $\dot{x} = f_1(x) + xu$ , and the running cost function  $\ell(x, u) = x^2 + u^2$ . The optimal policy is  $u^* = -\frac{1}{2}x \frac{\partial J^*}{\partial x}$ , leading to the HJB

$$0 = x^2 - \frac{1}{4}x^2 \left[ \frac{\partial J^*}{\partial x} \right]^2 + \frac{\partial J^*}{\partial x} f_1(x).$$

Choosing  $J^*(x) = x^2$ , we find that this is the cost-to-go when  $f_1(x) = -\frac{1}{2}x + \frac{1}{2}x^3$  and  $u^* = -x^2$ .

What happens in the case where our system is not control affine or if we really do need to specify an instantaneous cost function on  $\mathbf{u}$  that is not simply quadratic? If the goal is to produce an iterative algorithm, like value iteration, then one common approach is to make a (positive-definite) quadratic approximation in  $\mathbf{u}$  of the HJB, and updating that approximation on every iteration of the algorithm. This broad approach is often referred to as *differential dynamic programming* (c.f. [7]).

### 7.3.3 Numerical solutions for $J^*$

The other major barrier to using the HJB in a value iteration algorithm is that the estimated optimal cost-to-go function,  $\hat{J}^*$ , must somehow be represented with a finite set of numbers, but we don't yet know anything about the potential form it must take. In fact, knowing the time-to-goal solution for minimum-time problem with the double integrator, we see that this function might need to be non-smooth for even very simple dynamics and objectives.

One natural way to parameterize  $\hat{J}^*$  -- a scalar valued-function defined over the state space -- is to define the values on a [mesh](#). This approach then admits algorithms with close ties to the relatively very advanced numerical methods used to solve other partial differential equations (PDEs), such as the ones that appear in finite element modeling or fluid dynamics. One important difference, however, is that our PDE lives in the dimension of the state space, while many of the [mesh representations](#) from the other disciplines are optimized for two or three dimensional space. Also, our PDE may have discontinuities (or at least discontinuous gradients) at locations in the state space which are not known [apriori](#).

These days, the other natural choice is to represent  $J^*$  using a neural network, or some similarly richly parameterized [function approximator](#). This is common practice in [reinforcement learning](#), which we will discuss more later in the book.

### Value iteration with function approximation

If we approximate  $J^*$  with a finitely-parameterized function  $\hat{J}_\alpha^*$ , with parameter vector  $\alpha$ , then this immediately raises many important questions. If the true cost-to-go function does not live in the prescribed function class -- e.g., there does not exist an  $\alpha$  which satisfies the sufficiency conditions for all  $\mathbf{x}$  -- then we might instead write a cost function to match the optimality conditions as closely as possible. But our choice of cost function is important. Are errors in all states equally important? If we are balancing an Acrobot, presumably having an accurate cost-to-go in the vicinity of the upright condition is more important than some rarely visited state? Even if we can define a cost function that we like, then can we say anything about the convergence of our algorithm in this more general case?

Let us start by considering a least-squares approximation of the value iteration update.

Using the least squares solution in a value iteration update is sometimes referred to as [fitted value iteration](#), where  $\mathbf{x}_k$  are some number of samples taken from the continuous space and for discrete-time systems the iterative approximate solution to

$$\begin{aligned} J^*(\mathbf{x}_0) &= \min_{\mathbf{u}[\cdot]} \sum_{n=0}^{\infty} \ell(\mathbf{x}[n], \mathbf{u}[n]), \\ \text{s.t. } \mathbf{x}[n+1] &= f(\mathbf{x}[n], \mathbf{u}[n]), \mathbf{x}[0] = \mathbf{x}_0 \end{aligned}$$

becomes

$$J_k^d = \min_{\mathbf{u}} \left[ \ell(\mathbf{x}_k, \mathbf{u}) + \hat{J}_\alpha^*(f(\mathbf{x}_k, \mathbf{u})) \right], \quad (9)$$

$$\alpha \leftarrow \operatorname{argmin}_{\alpha} \sum_k \left( \hat{J}_\alpha^*(\mathbf{x}_k) - J_k^d \right)^2. \quad (10)$$

Since the desired values  $J_k^d$  are depend on the current guess of the cost-to-go, we will apply this value iteration algorithm iteratively until (hopefully) we achieve some numerical convergence.

Note that the update in (10) is not [quite](#) the same as doing least-squares optimization of

$$\sum_k \left( \hat{J}_\alpha^*(\mathbf{x}_k) - \min_{\mathbf{u}} \left[ \ell(\mathbf{x}_k, \mathbf{u}) + \hat{J}_\alpha^*(f(\mathbf{x}_k, \mathbf{u})) \right] \right)^2,$$

because in this equation  $\alpha$  has an effect on both occurrences of  $\hat{J}^*$ . In (10), we cut that dependence by taking  $J_k^d$  as fixed desired values; this version performs better in practice. In the deep reinforcement learning literature, we cut the dependence in a similar way by using a "target network" with the weights,  $\beta$ , that are updated more slowly than  $\alpha$ :

$$\sum_k \left( \hat{J}_\alpha^*(\mathbf{x}_k) - \min_{\mathbf{u}} \left[ \ell(\mathbf{x}_k, \mathbf{u}) + \hat{J}_\beta^*(f(\mathbf{x}_k, \mathbf{u})) \right] \right)^2.$$

For nonlinear function approximators, the update to  $\alpha$  in (10) is typically approximated by taking multiple steps of (stochastic) gradient descent.

## Linear function approximators

In general, the convergence and accuracy guarantees for value iteration with generic function approximators are quite weak. But we do have some results for the special case of *linear function approximators*. A linear function approximator takes the form:

$$\hat{J}_\alpha^*(\mathbf{x}) = \sum_i \alpha_i \psi_i(\mathbf{x}) = \psi^T(\mathbf{x})\alpha,$$

where  $\psi(\mathbf{x})$  is a vector of potentially nonlinear features. Common examples of features include polynomials, radial basis functions, or most interpolation schemes used on a mesh. The distinguishing feature of a linear function approximator is the ability to exactly solve for  $\alpha$  in order to represent a desired function optimally, in a least-squares sense. For linear function approximators, this is simply:

$$\alpha \Leftarrow \begin{bmatrix} \psi^T(\mathbf{x}_1) \\ \vdots \\ \psi^T(\mathbf{x}_K) \end{bmatrix}^+ \begin{bmatrix} J_1^d \\ \vdots \\ J_K^d \end{bmatrix},$$

where the  $^+$  notation refers to a Moore-Penrose pseudoinverse. One "theory of deep learning" argues that we can think of deep networks as linear function approximators (with randomly-initialized weights causing the last hidden layer to be a rich random basis vector, which is combined linearly into the outputs), which might justify how they can obtain zero training error quickly and then slowly adapt the features based on implicit regularization.

In finite state and action settings, fitted value iteration with linear function approximators is known to converge to the globally optimal  $\alpha^*$  [8]. [9] extends these results and treats the continuous state-action case. [10] gives the important generalization to actor-critic algorithms, which we will discuss soon. There can be some subtleties required for achieving convergence in practice. *In the next chapter*, we'll see why exponentially-discounted cost functions are often used help the value iterations converge even for infinite-horizon formulations.

## Value iteration on a mesh

Imagine that we use a mesh to approximate the cost-to-go function over that state space with  $K$  mesh points  $\mathbf{x}_k$ . We would like to perform the value iteration update:

$$\forall k, \hat{J}^*(\mathbf{x}_k) \Leftarrow \min_{\mathbf{u}} \left[ \ell(\mathbf{x}_k, \mathbf{u}) + \hat{J}^*(f(\mathbf{x}_k, \mathbf{u})) \right], \quad (11)$$

but must deal with the fact that  $f(\mathbf{x}_k, \mathbf{u})$  might not result in a next state that is directly at a mesh point. Most interpolation schemes for a mesh can be written as some weighted combination of the values at nearby mesh points, e.g.

$$\hat{J}^*(\mathbf{x}) = \sum_i \beta_i(\mathbf{x}) \hat{J}^*(\mathbf{x}_i), \quad \sum_i \beta_i = 1$$

with  $\beta_i$  the relative weight of the  $i$ th mesh point. In **DRAKE** we have implemented barycentric interpolation[11]. Taking  $\alpha_i = \hat{J}^*(\mathbf{x}_i)$ , the cost-to-go estimate at mesh point  $i$ , we can see that this is precisely an important special case of fitted value iteration with linear function approximation. Furthermore, assuming  $\beta_i(\mathbf{x}_i) = 1$ , (e.g., the only point contributing to the interpolation *at a mesh point* is the value at the mesh point itself), the update in Eq. (11) is precisely the least-squares update (and it achieves zero error). This is the representation used in the value iteration examples that you've already experimented with above.

## Neural fitted value iteration

Of course the most popular function approximators today are based on neural networks. In these much more general approximators, we cannot solve the least-squares update (10) exactly, but instead attempt to minimize the loss function: with a few steps of (stochastic) gradient descent on each iteration. In practice, we can use any of the standard optimizers (the examples below uses Adam[12] as the default).

What sort of neural architectures work best for value iteration in robotics? AlphaGo famously used convolutional networks[13]; here they could represent the state of the board as an image and the some notion of translational equivariance makes sense. But in "deep" reinforcement learning for robotics, when the input to the network is a state vector, then the standard choice is to use a simple multi-layered perceptron like the one I've used in the examples below. For rotational coordinates, instead of passing in  $\theta$  directly, we typically pass in both  $\sin \theta$  and  $\cos \theta$ .

When it comes to neural network software, I'll mostly use **PyTorch** throughout these notes. But for small problems (with small neural nets and modest batch sizes during training) like the ones we'll consider here, the cost of going back and forth to PyTorch and especially the GPU outweighs most of the benefits. I've also implemented a simple CPU-based **MultilayerPerceptron** in **DRAKE**, which can be used directly in the **Systems** framework, and will use it when it makes sense. My goal is to provide examples of both approaches so that you can pick and choose.

### Example 7.8 (Neural Fitted Value Iteration)

Let us try reproducing our double-integrator value iteration examples using neural networks:

### Example 7.9 (Continuous Neural Fitted Value Iteration)

This is nice work on continuous neural fitted value iteration in [14]. [15] has some of the most complete and convincing HJB solutions with neural networks that I have seen recently, and recommends a number of architectural and optimization training tweaks which could make a significant difference.

## Continuous-time systems

For solutions to systems with continuous-time dynamics, I have to uncover one of the details that I've so far been hiding to keep the notation simpler. Let us consider a problem with a finite-horizon:

$$\begin{aligned} \min_{\mathbf{u}[\cdot]} \quad & \sum_{n=0}^N \ell(\mathbf{x}[n], \mathbf{u}[n]), \\ \text{s.t.} \quad & \mathbf{x}[n+1] = f(\mathbf{x}[n], \mathbf{u}[n]), \mathbf{x}[0] = \mathbf{x}_0 \end{aligned}$$

When the horizon is finite, the optimal cost-to-go and the optimal policy, in general,

can depend on time (if the post office closes at 5pm, then your actions at 4:55pm might be very different than your actions at 4pm).

The way that we compute this is by solving the *time-varying cost-to-go function backwards* in time:

$$J^*(\mathbf{x}, N) = \min_{\mathbf{u}} \ell(\mathbf{x}, \mathbf{u})$$

$$J^*(\mathbf{x}, n-1) = \min_{\mathbf{u}} [\ell(\mathbf{x}, \mathbf{u}) + J^*(f(\mathbf{x}, \mathbf{u}), n)].$$

Even in the infinite-horizon setting, one can actually understand the convergence of the value iteration update as solving this time-varying cost-to-go backwards in time until it reaches a steady-state solution (the infinite-horizon solution). This is one way to see why value iteration only converges if the optimal cost-to-go is bounded.

Now let's consider the continuous-time version. Again, we have a time-varying cost-to-go,  $J^*(\mathbf{x}, t)$ . Now

$$\frac{dJ^*}{dt} = \frac{\partial J^*}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) + \frac{\partial J^*}{\partial t},$$

and our sufficiency condition is

$$0 = \min_{\mathbf{u}} \left[ \ell(\mathbf{x}, \mathbf{u}) + \frac{\partial J^*}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) + \frac{\partial J^*}{\partial t} \right].$$

But since  $\frac{\partial J^*}{\partial t}$  doesn't depend on  $\mathbf{u}$ , we can pull it out of the min and write the (true) HJB:

$$-\frac{\partial J^*}{\partial t} = \min_{\mathbf{u}} \left[ \ell(\mathbf{x}, \mathbf{u}) + \frac{\partial J^*}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) \right].$$

The standard numerical recipe [16] for solving this is to approximate  $\hat{J}^*(\mathbf{x}, t)$  on a mesh and then integrate the equations backwards in time (until convergence, if the goal is to find the infinite-horizon solution). If, for mesh point  $\mathbf{x}_i$  we have  $\alpha_i(t) = \hat{J}^*(\mathbf{x}_i, t)$ , then:

$$-\dot{\alpha}_i(t) = \min_{\mathbf{u}} \left[ \ell(\mathbf{x}_i, \mathbf{u}) + \frac{\partial J^*(\mathbf{x}_i, t)}{\partial \mathbf{x}} f(\mathbf{x}_i, \mathbf{u}) \right],$$

where the partial derivative is estimated with a suitable finite-difference approximation on the mesh and often some "viscosity" terms are added to the right-hand side to provide additional numerical robustness; see the Lax-Friedrichs scheme [16] (section 5.3.1) for an example.

Probably most visible and consistent campaign using numerical HJB solutions in applied control (at least in robotics) has come from [Claire Tomlin's group at Berkeley](#). Their work leverages [Ian Mitchell's Level Set Toolbox](#), which solves the Hamilton-Jacobi PDEs on a Cartesian mesh using the technique cartooned above, and even includes the minimum-time problem for the double integrator as a tutorial example[17].

## 7.4 EXTENSIONS

There are many many nice extensions to the basic formulations that we've presented so far. I'll try to list a few of the most important ones here. I've also had a number of students in this course explore very interesting extensions; for example [18] looked at imposing a low-rank structure on the (discrete-state) value function using ideas from matrix completion to obtain good estimates despite updating only a small fraction of the states.



### 7.4.1 Discounted and average cost formulations

Throughout this chapter, we have focused primarily on minimizing an infinite-horizon sum of running costs. Whenever you write an infinite sum (or integral), one should immediately question whether that sum converges to a finite value. For many control problems, it does -- for instance, if we set the cost to be zero at the goal and the controller can drive the system to the goal in finite time, or exponentially at a rate that is faster than cost is accrued. However, once we enter the realm of approximate optimal control, we can run into problems. For instance, if we form a discrete state and action approximation of the dynamics, it may not be possible for the discretized system to arrive *exactly* at the zero-cost goal.

A common way to protect oneself from this is to add an exponential discount factor to the integral cost. This is a natural choice since it preserves the recursive structure of the Bellman equation. In discrete time, it takes the form

$$\min_{\mathbf{u}[\cdot]} \sum_{n=0}^{\infty} \gamma^n \ell(\mathbf{x}[n], \mathbf{u}[n]),$$

which leads to the Bellman equation

$$J^*(\mathbf{x}) = \min_{\mathbf{u}} [\ell(\mathbf{x}, \mathbf{u}) + \gamma J^*(f(\mathbf{x}, \mathbf{u}))].$$

In continuous time we have[19]

$$\min_{\mathbf{u}} (\cdot) \int_0^{\infty} e^{-\frac{t}{\tau}} \ell(\mathbf{x}(t), \mathbf{u}(t)) dt,$$

which leads to the corresponding HJB:

$$\frac{1}{\tau} J^*(\mathbf{x}) = \min_{\mathbf{u}} \left[ \ell(\mathbf{x}, \mathbf{u}) + \frac{\partial J^*}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) \right].$$

A word of warning: this discount factor *does* change the optimal policy; and it is not without its pitfalls. We'll work out a specific case of the discounted cost for the LQR problem, and see that the discounted controller is exactly a controller which assumes that the plant is more stable than it actually is. It is possible to find an optimal controller for the discounted cost that results in an unstable controller.

A more robust alternative is to consider an average-cost formulations... (details coming soon!)

### 7.4.2 Stochastic control for finite MDPs

One of the most amazing features of the dynamic programming, additive cost approach to optimal control is the relative ease with which it extends to optimizing stochastic systems.

For discrete systems, we can generalize our dynamics on a graph by adding action-dependent transition probabilities to the edges. This new dynamical system is known as a Markov Decision Process (MDP), and we write the dynamics completely in terms of the transition probabilities

$$\Pr(s[n+1] = s' | s[n] = s, a[n] = a).$$

For discrete systems, this is simply a big lookup table. The cost that we incur for any execution of system is now a random variable, and so we formulate the goal of control as optimizing the expected cost, e.g.

$$J^*(s[0]) = \min_{a[\cdot]} E \left[ \sum_{n=0}^{\infty} \ell(s[n], a[n]) \right]. \quad (12)$$

Note that there are many other potential objectives, such as minimizing the worst-case error, but the expected cost is special because it preserves the dynamic programming recursion:

$$J^*(s) = \min_a E [\ell(s, a) + J^*(s')] = \min_a \left[ \ell(s, a) + \sum_{s'} \Pr(s'|s, a) J^*(s') \right].$$

Remarkably, if we use these optimality conditions to construct our value iteration algorithm

$$\hat{J}(s) \leftarrow \min_a \left[ \ell(s, a) + \sum_{s'} \Pr(s'|s, a) \hat{J}(s') \right],$$

then this algorithm has the same strong convergence guarantees of its counterpart for deterministic systems. And it is essentially no more expensive to compute!

We will continue to develop a much more general toolkit for stochastic and robust control later in the notes.

## Stochastic interpretation of deterministic, continuous-state value iteration

There is a particularly nice observation to be made here. Let's assume that we have discrete control inputs and discrete-time dynamics, but a continuous state space. Recall the fitted value iteration on a mesh algorithm described above. In fact, the resulting update is exactly the same as if we treated the system as a discrete state MDP with  $\beta_i$  representing the probability of transitioning to state  $\mathbf{x}_i$ ! This sheds some light on the impact of discretization on the solutions -- discretization error here will cause a sort of diffusion corresponding to the probability of spreading across neighboring nodes.

### 7.4.3 Linear Programming Dynamic Programming

For discrete MDPs, value iteration is a magical algorithm because it is simple, but known to converge to the global optimal,  $J^*$ . However, other important algorithms are known; one of the most important is a solution approach using linear programming. This formulation provides an alternative view, but may also be more generalizable and even more efficient for some instances of the problem.

Recall the optimality conditions from Eq. (1). If we describe the cost-to-go function as a vector,  $J_i = J(s_i)$ , then these optimality conditions can be rewritten in vector form as

$$\mathbf{J} = \min_a [\ell(a) + \mathbf{T}(a)\mathbf{J}], \quad (13)$$

where  $\ell_i(a) = \ell(s_i, a)$  is the cost vector, and  $T_{i,j}(a) = \Pr(s_j|s_i, a)$  is the transition probability matrix. Let us take  $\mathbf{J}$  as the vector of decision variables, and replace Eq. (13) with the constraints:

$$\forall a, \mathbf{J} \leq \ell(a) + \mathbf{T}(a)\mathbf{J}. \quad (14)$$

Clearly, for finite  $a$ , this is finite list of linear constraints, and for any vector  $\mathbf{J}$  satisfying these constraints we have  $\mathbf{J} \leq \mathbf{J}^*$  (elementwise). Now write the linear program:

$$\begin{aligned} & \text{maximize}_{\mathbf{J}} \quad \mathbf{c}^T \mathbf{J}, \\ & \text{subject to} \quad \forall a, \mathbf{J} \leq \ell(a) + \mathbf{T}(a)\mathbf{J}, \end{aligned}$$

where  $\mathbf{c}$  is any positive vector. The solution to this problem is  $\mathbf{J} = \mathbf{J}^*$ .

Perhaps even more interesting is that this approach can be generalized to linear

function approximators. Taking a vector form of my notation above, now we have a matrix of features with  $\Psi_{i,j} = \psi_j^T(\mathbf{x}_i)$  and we can write the LP

$$\text{maximize}_{\alpha} \quad \mathbf{c}^T \Psi \alpha, \quad (15)$$

$$\text{subject to} \quad \forall a, \Psi \alpha \leq \ell(a) + \mathbf{T}(a) \Psi \alpha. \quad (16)$$

This time the solution is not necessarily optimal, because  $\Psi \alpha^*$  only approximates  $\mathbf{J}^*$ , and the relative values of the elements of  $\mathbf{c}$  (called the "state-relevance weights") can determine the relative tightness of the approximation for different features [20].

## 7.4.4 Sums-of-Squares Dynamic Programming

There are some cases where the linear programming approach to dynamic programming can be extended to continuous states and actions, with strong guarantees. This approach follows naturally from our study of Lyapunov functions in an upcoming chapter, and builds on the computational tools we develop there. Feel free to [skip ahead](#).

## 7.5 EXERCISES

### Exercise 7.1 (Discounting and the Convergence of Value Iteration)

Let's consider the discrete time, state, and action version of value iteration:

$$\hat{J}^*(s_i) \leftarrow \min_{a \in A} [\ell(s_i, a) + \hat{J}^*(f(s_i, a))], \quad (17)$$

which finds the optimal policy for the infinite horizon cost function  $\sum_{n=0}^{\infty} \ell(s[n], a[n])$ .

- If  $J^*$  is the true optimal cost-to-go, show that any solution  $\hat{J}^* = J^* + c$ , where  $c$  is any constant scalar, is a fixed point of this value iteration update.
- Is the controller still optimal, even if the estimated cost-to-go function is off by a constant factor?

Surprisingly, adding a discount factor can help with this. Consider the infinite-horizon discounted cost:  $\sum_{n=0}^{\infty} \gamma^n \ell(s[n], a[n])$ , where  $0 < \gamma \leq 1$  is the discount factor. The corresponding value iteration update is

$$\hat{J}^*(s_i) \leftarrow \min_{a \in A} [\ell(s_i, a) + \gamma \hat{J}^*(f(s_i, a))]. \quad (18)$$

For simplicity, assume that there exists a state  $s_i$  that is a zero-cost fixed point under the optimal policy; e.g.  $\ell(s_i, \pi^*(s_i)) = 0$  and  $f(s_i, \pi^*(s_i)) = s_i$ .

- Is  $\hat{J}^* = J^* + c$  still a fixed point of the value iteration update when  $\gamma < 1$ ? Show your work.

## Exercise 7.2 (Choosing a Cost Function)

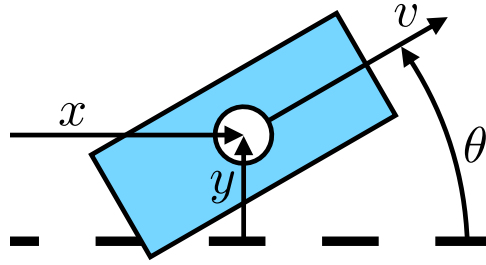


Figure 7.8 - Autonomous car moving at velocity  $v$  on a straight road.

The figure above shows an autonomous car moving at constant velocity  $v > 0$  on a straight road. Let  $x$  be the (longitudinal) position of the car along the road,  $y$  its (transversal) distance from the centerline, and  $\theta$  the angle between the centerline and the direction of motion. The only control action is the steering velocity  $u$ , which is constrained in the interval  $[u_{\min}, u_{\max}]$  (where  $u_{\min} < 0$  and  $u_{\max} > 0$ ). We describe the car dynamics with the simple kinematic model

$$\begin{aligned}\dot{x} &= v \cos \theta, \\ \dot{y} &= v \sin \theta, \\ \dot{\theta} &= u.\end{aligned}$$

Let  $\mathbf{x} = [x, y, \theta]^T$  be the state vector. To optimize the car trajectory we consider a quadratic objective function

$$J = \int_0^\infty [\mathbf{x}^T(t) \mathbf{Q} \mathbf{x}(t) + R u^2(t)] dt,$$

where  $\mathbf{Q}$  is a constant positive-semidefinite (hence symmetric) matrix and  $R$  is a constant nonnegative scalar (note that  $R = 0$  is allowed here).

- Suppose our only goal is to keep the distance  $y$  between the car and the centerline as small as possible, as fast as possible, without worrying about anything else. What would be your choice for  $\mathbf{Q}$  and  $R$ ?
- How would the behavior of the car change if you were to multiply the weights  $\mathbf{Q}$  and  $R$  from the previous point by an arbitrary positive coefficient  $\alpha$ ?
- The cost function from point (a) might easily lead to excessively sharp turns. Which entry of  $\mathbf{Q}$  or  $R$  would you increase to mitigate this issue?
- Country roads are more slippery on the sides than in the center. Is this class of objective functions (quadratic objectives) rich enough to include a penalty on sharp turns that increases with the distance of the car from the centerline?
- With this dynamics model and this objective function, would you ever choose a weight matrix  $\mathbf{Q}$  which is strictly positive definite (independent of the task you want the car to accomplish)? Why?

## Exercise 7.3 (Ill-Posed Optimal Control Problem)

In this exercise we will see how seemingly simple cost functions can give surprising results. Consider the single-integrator system  $\dot{x} = u$  with initial state  $x(0) = 0$ . We would like to find the control signal  $u(t)$  that minimizes the seemingly innocuous cost function

$$J = \int_0^T x^2(t) + (u^2(t) - 1)^2 dt,$$

with  $T$  finite. To this end, we consider a [square-wave](#) control parameterized by  $\tau > 0$ :

$$u_\tau(t) = \begin{cases} 1 & \text{if } t \in [0, \tau) \cup [3\tau, 5\tau) \cup [7\tau, 9\tau) \cup \dots \\ -1 & \text{if } t \in [\tau, 3\tau) \cup [5\tau, 7\tau) \cup [9\tau, 11\tau) \cup \dots \end{cases}.$$

- a. What are the states  $x$  and the inputs  $u$  for which the running cost

$$\ell(x, u) = x^2 + (u^2 - 1)^2$$

is equal to zero?

- b. Consider now two control signals  $u_{\tau_1}(t)$  and  $u_{\tau_2}(t)$ , with  $\tau_1 = \tau_2/2$ . Which one of the two incurs the lower cost  $J$ ? (Hint: start by sketching how the system state  $x(t)$  evolves under these two control signals.)
- c. What happens to the cost  $J$  when  $\tau$  gets smaller and smaller? What does the optimal control signal look like? Could you implement it with a finite-bandwidth controller?

## Exercise 7.4 (A Linear System with Quadratic Cost)

Consider the scalar control differential equation

$$\dot{x} = x + u,$$

and the infinite horizon cost function

$$J = \int_0^\infty [3x^2(t) + u^2(t)] dt.$$

As we will see in the [chapter on linear-quadratic regulation](#), the optimal cost-to-go for a problem of this kind assumes the form  $J^* = Sx^2$ . It is not hard to see that this, in turn, implies that the optimal controller has the form  $u^* = -Kx$ .

- a. Imagine that you plugged the expression  $J^* = Sx^2$  in the HJB equations for this problem, you solved them for  $S$ , and you got  $S \leq 0$ . Would this result ring any alarm bells? Explain your answer.
- b. Use the HJB sufficiency theorem to derive the optimal values of  $S$  and  $K$ .
- c. Working with digital controllers, we typically have to sample the dynamics of our systems, and approximate them with discrete-time models. Let us introduce a time step  $h > 0$  and discretize the dynamics as

$$\frac{x[n+1] - x[n]}{h} = x[n] + u[n],$$

and the objective as

$$h \sum_{n=0}^{\infty} (3x^2[n] + u^2[n]).$$

One of the following expressions is the correct cost-to-go  $J_h^*(x)$  for this discretized problem. Can you identify it without solving the discrete-time HJB equation? Explain your answer.

- i.  $J_h^*(x) = S_h x^4$  with  $S_h = 3 + h + \sqrt{6}h^2$ .
- ii.  $J_h^*(x) = S_h x^2$  with  $S_h = 1 + 2h + 2\sqrt{h^2 + h + 1}$ .
- iii.  $J_h^*(x) = S_h x^2$  with  $S_h = 3 + 2h^2 + \sqrt{h^2 + h + 2}$ .

## Exercise 7.5 (Value Iteration for Minimum-Time Problems)

In this exercise we analyze the performance of the value-iteration algorithm, considering its application to the [minimum time problem for the double integrator](#). , you will find everything you need for this analysis. Take the time to go through the notebook and understand the code in it, then answer the following questions.

- a. At the end of the notebook section "Performance of the Value-Iteration Policy", we plot the state trajectory of the double integrator in closed loop with the value-iteration policy, as well as the resulting control signal.
  - i. Does the state-space trajectory follow the theoretically-optimal quadratic arcs we have seen in [the example](#)?
  - ii. Is the control policy we get from value iteration a bang-bang policy? In other words, does the control signal take values in the set  $\{-1, 0, 1\}$  exclusively?
  - iii. Explain in a couple of sentences, what is the reason for this behavior.
- b. In the "Value Iteration Algorithm" section of the notebook, increase the number of knot points on the  $q$  and the  $\dot{q}$  axes to refine the state-space mesh used in the value iteration. Does this fix the issues you have seen in point (a)?
- c. In the final section of the notebook, implement the theoretically-optimal control policy from [the example](#), and use the plots to verify that the closed-loop system behaves as expected.

## Exercise 7.6 (Linear Program for Dynamic Programming)

In this exercise we solve the dynamic programming using a linear program, considering its application to the [grid world](#). , you will write a linear program for solving the optimal cost-to-go in the grid world.

## Exercise 7.7 (Continuous Fitted Value Iteration for Pendulum Swingup)

In this exercise we will implement value iteration for a continuous time system with continuous actions and states. We will achieve by training a neural network to approximate our value function. You will work exclusively in to implement the algorithms described in [the Continuous Dynamic Programming](#) section.

## Exercise 7.8 (Ordering of the Value Iteration Updates)

The batched Bellman update used in batched value iteration is as follows:

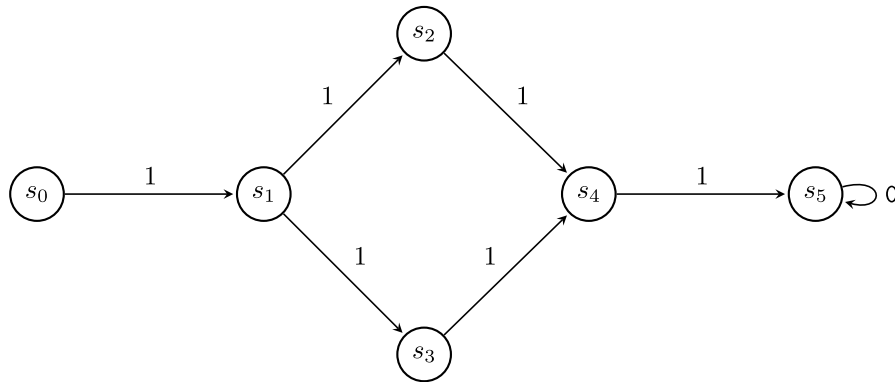
$$\forall i \in |S| \quad \hat{J}^*(s_i) \leftarrow \min_{a \in A} [l(s_i, a) + \hat{J}^*(f(s_i, a))]$$

The batched Bellman update consists of  $|S|$  individual Bellman updates on each iteration of the algorithm. Throughout this problem, we use the following goal reaching cost:

$$l(s_i, a) = \begin{cases} 0 & \text{if } s_i = s_5 \\ 1 & \text{else} \end{cases}$$

- a. Suppose we represent our dynamics with a graph  $G = (V, E)$  where

the vertices represent the states and edges represent the available actions/state transitions. Calculate the batched value iteration tables until convergence for the following directed acyclic graph (DAG), and calculate the total number of individual Bellman updates. Note: You do not need to include the updates required to check convergence.



- b. Suppose we are performing asynchronous value iteration with a hand-specified update order. Produce the update order for the above graph that minimizes the number of individual Bellman updates till convergence, and calculate the total number of individual updates. Note: You do not need to include the updates required to check convergence.
- c. Generalize the update order in part (b) to arbitrary DAGs with a single start node and single goal node by defining the set of states to be updated at iteration  $n + 1$  in terms of the set of states that were updated at iteration  $n$ . Note: Describe the update sequence iteratively in terms of sets of states instead of individual states (as defined by asynchronous VI) because this simplifies the description of the update sequence.
- d. Suppose we extend the class of dynamics graphs in part c) to allow cycles. Take an instance of this class of graphs and apply value iteration to get an optimal value function. Can an optimal policy associated with this value function contain a cycle? Explain your answer.
- e. (Optional, ungraded) Suppose we are uniformly sampling states under asynchronous value iteration (updating each  $i$  one at a time in a random order). What is the probability that asynchronous value iteration converges with less total Bellman updates than batched value iteration?

## REFERENCES

1. Dimitri P. Bertsekas, "Dynamic Programming and Optimal Control", Athena Scientific , 2000.
2. Richard S. Sutton and Andrew G. Barto, "Reinforcement Learning: An Introduction", MIT Press , 2018.
3. Dimitri P. Bertsekas and John N. Tsitsiklis, "Neuro-Dynamic Programming", Athena Scientific , October, 1996.
4. Richard S. Sutton and Andrew G. Barto, "Reinforcement Learning: An Introduction", MIT Press , 1998.
5. Dimitri P. Bertsekas, "Dynamic Programming \& Optimal Control", Athena Scientific , vol. I and II, May 1, 2005.
6. John Doyle and James A Primbs and Benjamin Shapiro and Vesna Nevistic, "Nonlinear games: examples and counterexamples", *Proceedings of 35th IEEE Conference on Decision and Control* , vol. 4, pp. 3915--3920, 1996.

7. David H. Jacobson and David Q. Mayne, "Differential Dynamic Programming", American Elsevier Publishing Company, Inc. , 1970.
8. John Tsitsiklis and Ben Van Roy, "An Analysis of Temporal-Difference Learning with Function Approximation", *IEEE Transactions on Automatic Control*, vol. 42, no. 5, pp. 674-690, May, 1997.
9. Rémi Munos and Csaba Szepesvári, "Finite-Time Bounds for Fitted Value Iteration", *J. Mach. Learn. Res.*, vol. 9, pp. 815--857, June, 2008.
10. V. R. Konda and J. N. Tsitsiklis, "Actor-Critic Algorithms", *SIAM Journal on Control and Optimization*, vol. 42, no. 4, pp. 1143-1166, 2003.
11. Remi Munos and Andrew Moore, "Barycentric Interpolators for Continuous Space and Time Reinforcement Learning", *Advances in Neural Information Processing Systems* , vol. 11, pp. 1024--1030, 1998.
12. Diederik P Kingma and Jimmy Ba, "Adam: A method for stochastic optimization", *arXiv preprint arXiv:1412.6980*, 2014.
13. David Silver and Aja Huang and Chris J Maddison and Arthur Guez and Laurent Sifre and George Van Den Driessche and Julian Schrittwieser and Ioannis Antonoglou and Veda Panneershelvam and Marc Lanctot and others, "Mastering the game of Go with deep neural networks and tree search", *nature*, vol. 529, no. 7587, pp. 484--489, 2016.
14. Michael Lutter and Shie Mannor and Jan Peters and Dieter Fox and Animesh Garg, "Value Iteration in Continuous Actions, States and Time", *International Conference on Machine Learning* , pp. 7224--7234, 2021.
15. Somil Bansal and Claire J Tomlin, "Deepreach: A deep learning approach to high-dimensional reachability", *2021 IEEE International Conference on Robotics and Automation (ICRA)* , pp. 1817--1824, 2021.
16. Stanley Osher and Ronald Fedkiw, "Level Set Methods and Dynamic Implicit Surfaces", Springer , 2003.
17. Ian M. Mitchell and Jeremy A. Templeton, "A Toolbox of Hamilton-Jacobi Solvers for Analysis of Nondeterministic Continuous and Hybrid Systems", *Proceedings of Hybrid Systems Computation and Control* , March, 2005.
18. Yuzhe Yang and Guo Zhang and Zhi Xu and Dina Katabi, "Harnessing Structures for Value-Based Planning and Reinforcement Learning", *International Conference on Learning Representations* , 2020.
19. Doya and K., "Reinforcement Learning in Continuous Time and Space", *Neural Computation*, vol. 12, no. 1, pp. 219-245, January, 2000.
20. Daniela Pucci de Farias, "The Linear Programming Approach to Approximate Dynamic Programming: Theory and Application", PhD thesis, Stanford University, June, 2002.



# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 8

# Linear Quadratic Regulators

While solving the dynamic programming problem for continuous systems is very hard in general, there are a few very important special cases where the solutions are very accessible. Most of these involve variants on the case of linear dynamics and convex (e.g. positive quadratic) cost. The simplest case, called the linear quadratic regulator (LQR), is formulated as stabilizing a time-invariant linear system to the origin.

The linear quadratic regulator is likely the most important and influential result in optimal control theory to date. In this chapter we will derive the basic algorithm and a variety of useful extensions.

### 8.1 BASIC DERIVATION

Consider a linear time-invariant system in state-space form,

$$\dot{\mathbf{x}} = \mathbf{Ax} + \mathbf{Bu},$$

with the infinite-horizon cost function given by

$$J = \int_0^\infty [\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u}] dt, \quad \mathbf{Q} = \mathbf{Q}^T \succeq \mathbf{0}, \mathbf{R} = \mathbf{R}^T \succ \mathbf{0}.$$

Our goal is to find the optimal cost-to-go function  $J^*(\mathbf{x})$  which satisfies the HJB:

$$\forall \mathbf{x}, \quad 0 = \min_{\mathbf{u}} \left[ \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u} + \frac{\partial J^*}{\partial \mathbf{x}} (\mathbf{Ax} + \mathbf{Bu}) \right].$$

There is one important step here -- it is well known that for this problem the optimal cost-to-go function is quadratic. This is easy to verify. Let us choose the form:

$$J^*(\mathbf{x}) = \mathbf{x}^T \mathbf{S} \mathbf{x}, \quad \mathbf{S} = \mathbf{S}^T \succeq \mathbf{0}.$$

The gradient of this function is

$$\frac{\partial J^*}{\partial \mathbf{x}} = 2\mathbf{x}^T \mathbf{S}.$$

Since we have guaranteed, by construction, that the terms inside the min are quadratic and convex (because  $\mathbf{R} \succ 0$ ), we can take the minimum explicitly by finding the solution where the gradient of those terms vanishes:

$$\frac{\partial}{\partial \mathbf{u}} = 2\mathbf{u}^T \mathbf{R} + 2\mathbf{x}^T \mathbf{S} \mathbf{B} = 0.$$

This yields the optimal policy

$$\mathbf{u}^* = \pi^*(\mathbf{x}) = -\mathbf{R}^{-1} \mathbf{B}^T \mathbf{S} \mathbf{x} = -\mathbf{K} \mathbf{x}.$$

Inserting this back into the HJB and simplifying yields

$$0 = \mathbf{x}^T [\mathbf{Q} - \mathbf{S} \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{S} + 2\mathbf{S} \mathbf{A}] \mathbf{x}.$$

All of the terms here are symmetric except for the  $2\mathbf{S} \mathbf{A}$ , but since  $\mathbf{x}^T \mathbf{S} \mathbf{A} \mathbf{x} = \mathbf{x}^T \mathbf{A}^T \mathbf{S} \mathbf{x}$ , we can write

$$0 = \mathbf{x}^T [\mathbf{Q} - \mathbf{S} \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{S} + \mathbf{S} \mathbf{A} + \mathbf{A}^T \mathbf{S}] \mathbf{x}.$$

and since this condition must hold for all  $\mathbf{x}$ , it is sufficient to consider the matrix equation

$$0 = \mathbf{S} \mathbf{A} + \mathbf{A}^T \mathbf{S} - \mathbf{S} \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{S} + \mathbf{Q}.$$

This extremely important equation is a version of the *algebraic Riccati equation*. Note that it is quadratic in  $\mathbf{S}$ , making its solution non-trivial, but it is well known that the equation has a single positive-definite solution if and only if the system is controllable. Moreover, there are good numerical methods for finding that solution, even in high-dimensional problems. Both the optimal policy and optimal cost-to-go function are available from **DRAKE** by calling `(K,S) = LinearQuadraticRegulator(A,B,Q,R)`.

If the appearance of the quadratic form of the cost-to-go seemed mysterious, consider that the solution to the linear system  $\dot{\mathbf{x}} = (\mathbf{A} - \mathbf{B} \mathbf{K}) \mathbf{x}$  takes the form  $\mathbf{x}(t) = e^{(\mathbf{A} - \mathbf{B} \mathbf{K})t} \mathbf{x}(0)$ , and try inserting this back into the integral cost function. You'll see that the cost takes the form  $J = \mathbf{x}^T(0) \mathbf{S} \mathbf{x}(0)$ .

It is worth examining the form of the optimal policy more closely. Since the value function represents cost-to-go, the optimal actions will move down this landscape as quickly as possible. Indeed,  $-\mathbf{S} \mathbf{x}$  is in the direction of steepest descent of the value function. However, not all directions are possible to achieve in state-space.  $-\mathbf{B}^T \mathbf{S} \mathbf{x}$  represents precisely the projection of the steepest descent onto the action space, and is the steepest descent achievable with the control inputs  $\mathbf{u}$ . Finally, the pre-scaling by the matrix  $\mathbf{R}^{-1}$  biases the direction of descent to account for relative cost weightings that we have placed on the different control inputs. Note that although this interpretation is straight-forward, the slope that we are descending (in the value function,  $\mathbf{S}$ ) is a function that takes into account the *long-term* dynamics and cost.

### Example 8.1 (LQR for the Double Integrator)

Now we can use LQR to reproduce our [HJB example](#) from the previous chapter:

```
import numpy as np
from pydrake.all import LinearQuadraticRegulator

# Define the double integrator's state space matrices.
A = np.array([[0, 1], [0, 0]])
B = np.array([[0], [1]])
Q = np.eye(2)
R = np.eye(1)
```

```
(K, S) = LinearQuadraticRegulator(A, B, Q, R)
print("K = " + str(K))
print("S = " + str(S))
```

As in the hand-derived example, our numerical solution returns

$$\mathbf{K} = [1, \sqrt{3}], \quad \mathbf{S} = \begin{bmatrix} \sqrt{3} & 1 \\ 1 & \sqrt{3} \end{bmatrix}.$$

### 8.1.1 Local stabilization of nonlinear systems

LQR is extremely relevant to us even though our primary interest is in nonlinear dynamics, because it can provide a local approximation of the optimal control solution for the nonlinear system. Given the nonlinear system  $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ , and a stabilizable operating point,  $(\mathbf{x}_0, \mathbf{u}_0)$ , with  $f(\mathbf{x}_0, \mathbf{u}_0) = 0$ . We can define a relative coordinate system

$$\bar{\mathbf{x}} = \mathbf{x} - \mathbf{x}_0, \quad \bar{\mathbf{u}} = \mathbf{u} - \mathbf{u}_0,$$

and observe that

$$\dot{\bar{\mathbf{x}}} = \dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}),$$

which we can approximate with a first-order Taylor expansion to

$$\dot{\bar{\mathbf{x}}} \approx f(\mathbf{x}_0, \mathbf{u}_0) + \frac{\partial f(\mathbf{x}_0, \mathbf{u}_0)}{\partial \mathbf{x}}(\mathbf{x} - \mathbf{x}_0) + \frac{\partial f(\mathbf{x}_0, \mathbf{u}_0)}{\partial \mathbf{u}}(\mathbf{u} - \mathbf{u}_0) = \mathbf{A}\bar{\mathbf{x}} + \mathbf{B}\bar{\mathbf{u}}.$$

Similarly, we can define a quadratic cost function in the error coordinates, or take a (positive-definite) second-order approximation of a nonlinear cost function about the operating point (linear and constant terms in the cost function can be easily incorporated into the derivation by parameterizing a full quadratic form for  $J^*$ , as seen in the Linear Quadratic Tracking derivation below).

The resulting controller takes the form  $\bar{\mathbf{u}}^* = -\mathbf{K}\bar{\mathbf{x}}$  or

$$\mathbf{u}^* = \mathbf{u}_0 - \mathbf{K}(\mathbf{x} - \mathbf{x}_0).$$

For convenience, `DRAKE` allows you to call `controller = LinearQuadraticRegulator(system, context, Q, R)` on most dynamical systems (including block diagrams built up of many subsystems); it will perform the linearization for you.

#### **Example 8.2 (LQR for Acrobats, Cart-Poles, and Quadrotors)**

LQR provides a very satisfying solution to the canonical "balancing" problem that we've [already described for a number of model systems](#). Here is the notebook with those examples, again:

I find it very compelling that the same derivation (and effectively identical code) can stabilize such a diversity of systems!

## 8.2 FINITE-HORIZON FORMULATIONS

Recall that the cost-to-go for finite-horizon problems is time-dependent, and

therefore the HJB sufficiency condition requires an additional term for  $\frac{\partial J^*}{\partial t}$ .

$$\forall \mathbf{x}, \forall t \in [t_0, t_f], \quad 0 = \min_{\mathbf{u}} \left[ \ell(\mathbf{x}, \mathbf{u}) + \frac{\partial J^*}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) + \frac{\partial J^*}{\partial t} \right].$$

### 8.2.1 Finite-horizon LQR

Consider systems governed by an LTI state-space equation of the form

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u},$$

and a finite-horizon cost function,  $J = h(\mathbf{x}(t_f)) + \int_0^{t_f} \ell(\mathbf{x}(t), \mathbf{u}(t)) dt$ , with

$$\begin{aligned} h(\mathbf{x}) &= \mathbf{x}^T \mathbf{Q}_f \mathbf{x}, & \mathbf{Q}_f &= \mathbf{Q}_f^T \succeq \mathbf{0} \\ \ell(\mathbf{x}, \mathbf{u}) &= \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u}, & \mathbf{Q} &= \mathbf{Q}^T \succeq \mathbf{0}, \mathbf{R} = \mathbf{R}^T \succ \mathbf{0} \end{aligned}$$

Writing the HJB, we have

$$0 = \min_{\mathbf{u}} \left[ \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u} + \frac{\partial J^*}{\partial \mathbf{x}} (\mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}) + \frac{\partial J^*}{\partial t} \right].$$

Due to the positive definite quadratic form on  $\mathbf{u}$ , we can find the minimum by setting the gradient to zero:

$$\begin{aligned} \frac{\partial}{\partial \mathbf{u}} &= 2\mathbf{u}^T \mathbf{R} + \frac{\partial J^*}{\partial \mathbf{x}} \mathbf{B} = 0 \\ \mathbf{u}^* &= \pi^*(\mathbf{x}, t) = -\frac{1}{2} \mathbf{R}^{-1} \mathbf{B}^T \frac{\partial J^*}{\partial \mathbf{x}} \end{aligned}$$

In order to proceed, we need to investigate a particular form for the cost-to-go function,  $J^*(\mathbf{x}, t)$ . Let's try a solution of the form:

$$J^*(\mathbf{x}, t) = \mathbf{x}^T \mathbf{S}(t) \mathbf{x}, \quad \mathbf{S}(t) = \mathbf{S}^T(t) \succ \mathbf{0}.$$

In this case we have

$$\frac{\partial J^*}{\partial \mathbf{x}} = 2\mathbf{x}^T \mathbf{S}(t), \quad \frac{\partial J^*}{\partial t} = \mathbf{x}^T \dot{\mathbf{S}}(t) \mathbf{x},$$

and therefore

$$\begin{aligned} \mathbf{u}^* &= \pi^*(\mathbf{x}, t) = -\mathbf{R}^{-1} \mathbf{B}^T \mathbf{S}(t) \mathbf{x} \\ 0 &= \mathbf{x}^T \left[ \mathbf{Q} - \mathbf{S}(t) \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{S}(t) + \mathbf{S}(t) \mathbf{A} + \mathbf{A}^T \mathbf{S}(t) + \dot{\mathbf{S}}(t) \right] \mathbf{x}. \end{aligned}$$

Therefore,  $\mathbf{S}(t)$  must satisfy the condition (known as the continuous-time *differential Riccati equation*):

$$-\dot{\mathbf{S}}(t) = \mathbf{S}(t) \mathbf{A} + \mathbf{A}^T \mathbf{S}(t) - \mathbf{S}(t) \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{S}(t) + \mathbf{Q},$$

and the terminal condition

$$\mathbf{S}(t_f) = \mathbf{Q}_f.$$

Since we were able to satisfy the HJB with the minimizing policy, we have met the sufficiency condition, and have found the optimal policy and optimal cost-to-go function.

Note that the infinite-horizon LQR solution described in the prequel is exactly the steady-state solution of this equation, defined by  $\dot{\mathbf{S}}(t) = 0$ . Indeed, for controllable systems this equation is stable (backwards in time), and as expected the finite-horizon solution converges on the infinite-horizon solution as the horizon time limits to infinity.

The solution  $\mathbf{S}(t)$  will be the result of numerical integration. On weakly controllable systems (like the [perching airplane](#)), I've found that even error-controlled integration can lead to crippling numerical artifacts such as the loss of symmetry or positive definiteness. For a more robust numerical solution, we can instead integrate a factorization of  $\mathbf{S}$  that ensures these properties are maintained. Taking  $\mathbf{S}(t) = \mathbf{P}(t)\mathbf{P}^T(t)$ , and again using the fact that  $\mathbf{x}^T\dot{\mathbf{P}}\mathbf{P}^T\mathbf{x} = \mathbf{x}^T\dot{\mathbf{P}}\mathbf{P}^T\mathbf{x}$ , we can write the "square-root form" of the Riccati differential equation:

$$-\dot{\mathbf{P}}(t) = \mathbf{A}^T\mathbf{P}(t) - \frac{1}{2}\mathbf{S}(t)\mathbf{B}\mathbf{R}^{-1}\mathbf{B}^T\mathbf{P}(t) + \frac{1}{2}\mathbf{Q}\mathbf{P}^{-T}(t), \quad \mathbf{P}(t_f) = \mathbf{Q}_f^{\frac{1}{2}}.$$

This form does require that  $\mathbf{P}(t)$  is invertible, which in turn requires that  $\mathbf{Q}_f \succ 0$ .

## 8.2.2 Time-varying LQR

The derivation above holds even if the dynamics are given by

$$\dot{\mathbf{x}} = \mathbf{A}(t)\mathbf{x} + \mathbf{B}(t)\mathbf{u}.$$

Similarly, the cost functions  $\mathbf{Q}$  and  $\mathbf{R}$  can also be time-varying. This is quite surprising, as the class of time-varying linear systems is a quite general class of systems. It requires essentially no assumptions on how the time-dependence enters, except perhaps that if  $\mathbf{A}$  or  $\mathbf{B}$  is discontinuous in time then one would have to use the proper techniques to accurately integrate the differential equation.

## 8.2.3 Local trajectory stabilization for nonlinear systems

One of the most powerful applications of time-varying LQR involves linearizing around a nominal trajectory of a nonlinear system and using LQR to provide a trajectory controller. This will tie in very nicely with the algorithms we develop in the [chapter on trajectory optimization](#).

Let us assume that we have a nominal trajectory,  $\mathbf{x}_0(t), \mathbf{u}_0(t)$  defined for  $t \in [t_1, t_2]$ . Similar to the time-invariant analysis, we begin by defining a local coordinate system relative to the trajectory:

$$\bar{\mathbf{x}}(t) = \mathbf{x}(t) - \mathbf{x}_0(t), \quad \bar{\mathbf{u}}(t) = \mathbf{u}(t) - \mathbf{u}_0(t).$$

Now we have

$$\dot{\bar{\mathbf{x}}} = \dot{\mathbf{x}} - \dot{\mathbf{x}}_0 = f(\mathbf{x}, \mathbf{u}) - f(\mathbf{x}_0, \mathbf{u}_0),$$

which we can again approximate with a first-order Taylor expansion to

$$\dot{\bar{\mathbf{x}}} \approx f(\mathbf{x}_0, \mathbf{u}_0) + \frac{\partial f(\mathbf{x}_0, \mathbf{u}_0)}{\partial \mathbf{x}}(\mathbf{x} - \mathbf{x}_0) + \frac{\partial f(\mathbf{x}_0, \mathbf{u}_0)}{\partial \mathbf{u}}(\mathbf{u} - \mathbf{u}_0) - f(\mathbf{x}_0, \mathbf{u}_0) = \mathbf{A}(t)\bar{\mathbf{x}} + \mathbf{B}(t)\bar{\mathbf{u}}.$$

This is very similar to using LQR to stabilize a fixed-point, but with some important differences. First, the linearization is time-varying. Second, our linearization is valid for any state along a feasible trajectory (not just fixed-points), because the coordinate system is moving along with the trajectory.

Similarly, we can define a quadratic cost function in the error coordinates, or take a (positive-definite) second-order approximation of a nonlinear cost function along the trajectory (linear and constant terms in the cost function can be easily incorporated into the derivation by parameterizing a full quadratic form for  $J^*$ , as seen in the Linear Quadratic Tracking derivation below).

The resulting controller takes the form  $\bar{\mathbf{u}}^* = -\mathbf{K}(t)\bar{\mathbf{x}}$  or

$$\mathbf{u}^* = \mathbf{u}_0(t) - \mathbf{K}(t)(\mathbf{x} - \mathbf{x}_0(t)).$$

**DRAKE** provides a [FiniteHorizonLinearQuadraticRegulator](#) method; if you pass it

a nonlinear systems it will perform the linearization in the proper coordinates for you using automatic differentiation.

Remember that stability is a statement about what happens as time goes to infinity. In order to talk about stabilizing a trajectory, the trajectory must be defined for all  $t \in [t_1, \infty)$ . This can be accomplished by considering a finite-time trajectory which terminates at a stabilizable fixed-point at a time  $t_2 \geq t_1$ , and remains at the fixed point for  $t \geq t_2$ . In this case, the finite-horizon Riccati equation is initialized with the infinite-horizon LQR solution:  $S(t_2) = S_\infty$ , and solved backwards in time from  $t_2$  to  $t_1$  for the remainder of the trajectory. And *now* we can say that we have *stabilized* the trajectory!

## 8.2.4 Linear Quadratic Optimal Tracking

For completeness, we consider a slightly more general form of the linear quadratic regulator. The standard LQR derivation attempts to drive the system to zero. Consider now the problem:

$$\begin{aligned}\dot{\mathbf{x}} &= \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \\ h(\mathbf{x}) &= (\mathbf{x} - \mathbf{x}_d(t_f))^T \mathbf{Q}_f (\mathbf{x} - \mathbf{x}_d(t_f)), \quad \mathbf{Q}_f = \mathbf{Q}_f^T \succeq 0 \\ \ell(\mathbf{x}, \mathbf{u}, t) &= (\mathbf{x} - \mathbf{x}_d(t))^T \mathbf{Q} (\mathbf{x} - \mathbf{x}_d(t)) + (\mathbf{u} - \mathbf{u}_d(t))^T \mathbf{R} (\mathbf{u} - \mathbf{u}_d(t)), \\ \mathbf{Q} &= \mathbf{Q}^T \succeq 0, \mathbf{R} = \mathbf{R}^T \succ 0\end{aligned}$$

Now, guess a solution

$$J^*(\mathbf{x}, t) = \mathbf{x}^T \mathbf{S}_{xx}(t) \mathbf{x} + 2\mathbf{x}^T \mathbf{s}_x(t) + s_0(t), \quad \mathbf{S}_{xx}(t) = \mathbf{S}_{xx}^T(t) \succ 0.$$

In this case, we have

$$\frac{\partial J^*}{\partial \mathbf{x}} = 2\mathbf{x}^T \mathbf{S}_{xx}(t) + 2\mathbf{s}_x^T(t), \quad \frac{\partial J^*}{\partial t} = \mathbf{x}^T \dot{\mathbf{S}}_{xx}(t) \mathbf{x} + 2\mathbf{x}^T \dot{\mathbf{s}}_x(t) + \dot{s}_0(t).$$

Using the HJB,

$$0 = \min_{\mathbf{u}} \left[ (\mathbf{x} - \mathbf{x}_d(t))^T \mathbf{Q} (\mathbf{x} - \mathbf{x}_d(t)) + (\mathbf{u} - \mathbf{u}_d(t))^T \mathbf{R} (\mathbf{u} - \mathbf{u}_d(t)) + \frac{\partial J^*}{\partial \mathbf{x}} (\mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}) + \frac{\partial J^*}{\partial t} \right],$$

we have

$$\begin{aligned}\frac{\partial}{\partial \mathbf{u}} &= 2(\mathbf{u} - \mathbf{u}_d(t))^T \mathbf{R} + (2\mathbf{x}^T \mathbf{S}_{xx}(t) + 2\mathbf{s}_x^T(t)) \mathbf{B} = 0, \\ \mathbf{u}^*(t) &= \mathbf{u}_d(t) - \mathbf{R}^{-1} \mathbf{B}^T [\mathbf{S}_{xx}(t) \mathbf{x} + \mathbf{s}_x(t)]\end{aligned}$$

The HJB can be satisfied by integrating backwards

$$\begin{aligned}-\dot{\mathbf{S}}_{xx}(t) &= \mathbf{Q} - \mathbf{S}_{xx}(t) \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{S}_{xx}(t) + \mathbf{S}_{xx}(t) \mathbf{A} + \mathbf{A}^T \mathbf{S}_{xx}(t) \\ -\dot{\mathbf{s}}_x(t) &= -\mathbf{Q} \mathbf{x}_d(t) + [\mathbf{A}^T - \mathbf{S}_{xx} \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T] \mathbf{s}_x(t) + \mathbf{S}_{xx}(t) \mathbf{B} \mathbf{u}_d(t) \\ -\dot{s}_0(t) &= \mathbf{x}_d(t)^T \mathbf{Q} \mathbf{x}_d(t) - \mathbf{s}_x^T(t) \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{s}_x(t) + 2\mathbf{s}_x(t)^T \mathbf{B} \mathbf{u}_d(t),\end{aligned}$$

from the final conditions

$$\begin{aligned}\mathbf{S}_{xx}(t_f) &= \mathbf{Q}_f \\ \mathbf{s}_x(t_f) &= -\mathbf{Q}_f \mathbf{x}_d(t_f) \\ s_0(t_f) &= \mathbf{x}_d(t_f)^T \mathbf{Q}_f \mathbf{x}_d(t_f).\end{aligned}$$

Notice that the solution for  $\mathbf{S}_{xx}$  is the same as the simpler LQR derivation, and is symmetric (as we assumed). Note also that  $s_0(t)$  has no effect on control (even indirectly), and so can often be ignored.

A quick observation about the quadratic form, which might be helpful in debugging. We know that  $J(\mathbf{x}, t)$  must be uniformly positive. This is true iff  $\mathbf{S}_{xx} \succ 0$  and  $s_0 > \mathbf{s}_x^T \mathbf{S}_{xx}^{-1} \mathbf{s}_x$ , which comes from evaluating the function at  $\mathbf{x}_{min}(t)$  defined by

$$\left[ \frac{\partial J^*(\mathbf{x}, t)}{\partial \mathbf{x}} \right]_{\mathbf{x}=\mathbf{x}_{min}(t)} = 0.$$

## 8.2.5 Linear Final Boundary Value Problems

The finite-horizon LQR formulation can be used to impose a strict final boundary value condition by setting an infinite  $\mathbf{Q}_f$ . However, integrating the Riccati equation backwards from an infinite initial condition isn't very practical. To get around this, let us consider solving for  $\mathbf{P}(t) = \mathbf{S}(t)^{-1}$ . Using the matrix relation  $\frac{d\mathbf{S}^{-1}}{dt} = -\mathbf{S}^{-1} \frac{d\mathbf{S}}{dt} \mathbf{S}^{-1}$ , we have:

$$-\dot{\mathbf{P}}(t) = -\mathbf{P}(t)\mathbf{Q}\mathbf{P}(t) + \mathbf{B}\mathbf{R}^{-1}\mathbf{B} - \mathbf{A}\mathbf{P}(t) - \mathbf{P}(t)\mathbf{A}^T,$$

with the final conditions

$$\mathbf{P}(t_f) = 0.$$

This Riccati equation can be integrated backwards in time for a solution.

It is very interesting, and powerful, to note that, if one chooses  $\mathbf{Q} = 0$ , therefore imposing no position cost on the trajectory before time  $T$ , then this inverse Riccati equation becomes a linear ODE which can be solved explicitly. These relationships are used in the derivation of the controllability **Grammian**, but here we use them to design a feedback controller.

## 8.3 VARIATIONS AND EXTENSIONS

### 8.3.1 Discrete-time Riccati Equations

Essentially all of the results above have a natural correlate for discrete-time systems. What's more, the discrete time versions tend to be simpler to think about in the model-predictive control (MPC) setting that we'll be discussing below and in the next chapters.

Consider the discrete time dynamics:

$$\mathbf{x}[n+1] = \mathbf{A}\mathbf{x}[n] + \mathbf{B}\mathbf{u}[n],$$

and we wish to minimize

$$\min \sum_{n=0}^{N-1} \mathbf{x}^T[n] \mathbf{Q} \mathbf{x}[n] + \mathbf{u}^T[n] \mathbf{R} \mathbf{u}[n], \quad \mathbf{Q} = \mathbf{Q}^T \succeq 0, \mathbf{R} = \mathbf{R}^T \succ 0.$$

The cost-to-go is given by

$$J(\mathbf{x}, n-1) = \min_{\mathbf{u}} \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u} + J(\mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}, n).$$

If we once again take

$$J(\mathbf{x}, n) = \mathbf{x}^T \mathbf{S}[n] \mathbf{x}, \quad \mathbf{S}[n] = \mathbf{S}^T[n] \succ 0,$$

then we have

$$\mathbf{u}^*[n] = -\mathbf{K}[n] \mathbf{x}[n] = -(\mathbf{R} + \mathbf{B}^T \mathbf{S}[n] \mathbf{B})^{-1} \mathbf{B}^T \mathbf{S}[n] \mathbf{A} \mathbf{x}[n],$$

yielding

$$\mathbf{S}[n-1] = \mathbf{Q} + \mathbf{A}^T \mathbf{S}[n] \mathbf{A} - (\mathbf{A}^T \mathbf{S}[n] \mathbf{B})(\mathbf{R} + \mathbf{B}^T \mathbf{S}[n] \mathbf{B})^{-1} (\mathbf{B}^T \mathbf{S}[n] \mathbf{A}), \quad \mathbf{S}[N] = 0,$$

which is the famous *Riccati difference equation*. The infinite-horizon LQR solution is given by the (positive-definite) fixed-point of this equation:

$$\mathbf{S} = \mathbf{Q} + \mathbf{A}^T \mathbf{S} \mathbf{A} - (\mathbf{A}^T \mathbf{S} \mathbf{B})(\mathbf{R} + \mathbf{B}^T \mathbf{S} \mathbf{B})^{-1}(\mathbf{B}^T \mathbf{S} \mathbf{A}).$$

Like in the continuous time case, this equation is so important that we have special numerical recipes for solving this discrete-time algebraic Riccati equation (DARE). **DRAKE** delegates to these numerical methods automatically when you evaluate the **LinearQuadraticRegulator** method on a system that has only discrete state and a single periodic time step.

### Example 8.3 (Discrete-time vs Continuous-time LQR)

You can explore the relationship between the discrete-time and continuous-time formulations in this notebook:

In reinforcement learning, it is popular to consider the infinite-horizon "discounted" cost:  $\min \sum_{n=0}^{\infty} \gamma^n (\mathbf{x}^T[n] \mathbf{Q} \mathbf{x}[n] + \mathbf{u}^T[n] \mathbf{R} \mathbf{u}[n])$ . The optimal controller is

$$\mathbf{u}^* = -\gamma(\mathbf{R} + \gamma \mathbf{B}^T \mathbf{S} \mathbf{B})^{-1} \mathbf{B}^T \mathbf{S} \mathbf{A} \mathbf{x}.$$

and the corresponding Riccati equation is

$$\mathbf{S} = \mathbf{Q} + \gamma \mathbf{A}^T \mathbf{S} \mathbf{A} - \gamma^2 (\mathbf{A}^T \mathbf{S} \mathbf{B})(\mathbf{R} + \gamma \mathbf{B}^T \mathbf{S} \mathbf{B})^{-1}(\mathbf{B}^T \mathbf{S} \mathbf{A}),$$

whose solution can be found using **DiscreteAlgebraicRiccatiEquation** ( $\sqrt{\gamma} \mathbf{A}, \mathbf{B}, \mathbf{Q}, \frac{1}{\gamma} \mathbf{R}$ ).

### Example 8.4 (LQR via Fitted Value Iteration)

In the dynamic programming chapter, we developed general purpose tools for approximating value functions with function approximators. I think it is particularly illustrative to see how these work out for LQR. Let's consider the discrete-time, infinite-horizon, discounted case.

For LQR, we know that the optimal value function will take a quadratic form,  $\mathbf{x}^T \mathbf{S} \mathbf{x}$ . Although it is quadratic in  $\mathbf{x}$ , this form is linear in the parameters,  $\mathbf{S}$ . We can leverage some of the special tools for fitted value iteration with a *linear function approximator*.

I've included two versions of the value iteration update in the notebook -- one that samples over both  $\mathbf{x}$  and  $\mathbf{u}$ , and one that samples only over  $\mathbf{x}$  and uses the LQR policy (given the current estimated cost-to-go) to determine  $\mathbf{u}$ . The difference is not subtle when  $\gamma \rightarrow 1$ . Take a look!

We must remember that there are multiple solutions to the Riccati equation -- the solution to the LQR problem is the (unique) positive definite solution. But there are non-positive definite solutions, too, which lead to unstable controllers -- these solutions do achieve zero Bellman residual. Every solution to the Riccati equation is a fixed point of the (fitted) value iteration update, but only the positive-definite solution is a stable fixed point of the algorithm.

To see this, write  $\hat{J}(\mathbf{x}) = \mathbf{x}^T (\mathbf{S}^* + \Delta) \mathbf{x}$ , where  $\mathbf{S}^*$  is any solution to the Riccati equation, and  $\Delta$  is a small deviation matrix. If we write the error dynamics through the fitted value iteration update, we can see that

$$\Delta_{i+1} = (\mathbf{A} - \mathbf{B} \mathbf{K}_i)^T \Delta_i (\mathbf{A} - \mathbf{B} \mathbf{K}_i),$$

where  $\mathbf{K}_i$  is the optimal controller given  $\mathbf{S}_i = \mathbf{S}^* + \Delta_i \approx \mathbf{K}^*$ . This error converges to zero if and only if  $(\mathbf{A} - \mathbf{B} \mathbf{K}^*)$  is stable, which is only if  $\mathbf{S}^* \succ 0$ .

Go ahead and try many different initial  $\mathbf{S}$  in the code to verify it for yourself.



### 8.3.2 LQR with input and state constraints

A natural extension for linear optimal control is the consideration of strict constraints on the inputs or state trajectory. Most common are linear inequality constraints, such as  $\forall n, |\mathbf{u}[n]| \leq 1$  or  $\forall n, \mathbf{x}[n] \geq -2$  (any linear constraints of the form  $\mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u} \leq \mathbf{e}$  can be solved with the same tools). Much is known about the solution to this problem in the discrete-time case, but it's computation is significantly harder than the unconstrained case. Almost always, we give up on solving for the best control policy in closed form, and instead solve for the optimal control trajectory  $\mathbf{u}[\cdot]$  from a particular initial condition  $\mathbf{x}[0]$  over some finite horizon. Fortunately, this problem is a convex optimization and we can often solve it quickly and reliably enough to solve it at every time step, effectively turning a motion planning algorithm into a feedback controller; this idea is famously known as model-predictive control (MPC). We will provide the details in the [trajectory optimization chapter](#).

### 8.3.3 LQR on a manifold

One important case that does have closed-form solutions is LQR with linear *equality* constraints. This is particularly relevant in the case of stabilizing robots with kinematic constraints such as a closed kinematic chain, which appears in four-bar linkages or even for the linearization of a biped robot with two feet on the ground.

Our formulation is as follows: in addition to the linear dynamics (in continuous time or discrete time), we have a linear equality constraint of the form  $\mathbf{F}\mathbf{x} = \mathbf{0}$ , with  $\mathbf{F} \in \mathbb{R}^{(n-d) \times n}$ . Form a matrix  $\mathbf{P} \in \mathbb{R}^{d \times n}$  that forms an orthonormal basis for the nullspace of  $\mathbf{F}$ , such that

$$\mathbf{P}\mathbf{P}^T = \mathbf{I}^{d \times d}, \quad \mathbf{P}\mathbf{F}^T = \mathbf{0}^{d \times (n-d)}.$$

Then for any  $\mathbf{x}$  we can write  $\mathbf{x} = \mathbf{P}^T\mathbf{y} + \mathbf{F}^T\mathbf{z}$ , for some  $\mathbf{y} \in \mathbb{R}^d$  and  $\mathbf{z} \in \mathbb{R}^{n-d}$ . However, as the result of the constraints we know that  $\mathbf{F}\mathbf{x} = \mathbf{z} = \mathbf{0}$ . Instead of stabilizing  $\mathbf{x}$  via LQR, we will stabilize  $\mathbf{y}$ , using the projected cost and dynamics given by

$$\mathbf{A}_y = \mathbf{P}\mathbf{A}_x\mathbf{P}^T, \quad \mathbf{B}_y = \mathbf{P}\mathbf{B}_x, \quad \mathbf{Q}_y = \mathbf{P}\mathbf{Q}_x\mathbf{P}^T$$

where  $\mathbf{A}_x, \mathbf{B}_x$ , and  $\mathbf{Q}_x$  are the matrices from the full coordinates model. I've chosen to write it this way because it is the same in both discrete and continuous time. The resulting cost-to-go is given by  $\mathbf{S}_x = \mathbf{P}^T\mathbf{S}_y\mathbf{P}$  and the controller is given by  $\mathbf{K}_x = \mathbf{K}_y\mathbf{P}$ , and the where  $\mathbf{K}_y, \mathbf{S}_y$  are the solutions to the reduced LQR problem in  $\mathbf{y}$ .

More generally, for a nonlinear system with nonlinear equality constraints, we would linearize both the system dynamics and the constraint, and then apply the derivation above. [1, IIIB] provides the time-varying LQR version, which follows naturally.

#### Example 8.5 (Balancing a Segway)

A nice example of this is the task of balancing a [Segway](#) in the  $x - z$  plane. In 3D, one could either consider the nonholonomic wheels of the [Segway](#), or turn it into a [Ballbot](#). For the purposes of simulation, one can simply construct the ground, the ball, and the bot with appropriate collision geometry, and drop it into [MultibodyPlant](#). The frictional contact between the ball and the ground generates the expected dynamics.

Designing the LQR controller is another story. We will use a [PlanarJoint](#) for the floating base, giving us 4 degrees of freedom:  $[x, z, \theta_{ball}, \theta_{bot-ball}]$ . If we linearize this model and call LQR, the LQR call will fail because *the system is not controllable in the full coordinates*. The first reason is easy to see: there is nothing that the actuators can do to change  $z$ , the vertical position of the ball. Perhaps the more subtle reason is that there is a (holonomic) constraint from the ball rolling without

slip on the ground:  $x = -r\theta_{ball}$ , where  $r$  is the radius of the ball.

I will use one small implementation "trick". In my code, rather than deal with finding the resting penetration of the ball, I decided to remove the vertical degree of freedom completely, and used a prismatic joint for  $x$  and a revolute joint for  $\theta_{ball}$  instead of the `PlanarJoint` for the floating base. I configured it so that the ball was sufficiently in penetration that the normal forces from the ground were substantial enough that the friction cone could support the horizontal frictional forces I needed to roll. This effectively implements the  $z$  constraint by construction (completely removing that degree of freedom).

To use LQR, we will write the constraints  $x = -r\theta_{ball}$  and the implied  $\dot{x} = -r\dot{\theta}_{ball}$  as

$$\mathbf{F}\mathbf{x} = \begin{bmatrix} 1 & r & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & r & 0 \end{bmatrix} \mathbf{x} = 0$$

The nullspace matrix  $\mathbf{P}$  can be obtained numerically, but here it was simple enough to get from inspection,

$$\mathbf{P} = \begin{bmatrix} \frac{r}{\sqrt{1+r^2}} & -\frac{1}{\sqrt{1+r^2}} & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{r}{\sqrt{1+r^2}} & -\frac{1}{\sqrt{1+r^2}} & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix}.$$

We can obtain the matrices  $\mathbf{A}_x \in \mathbb{R}^{6 \times 6}$ ,  $\mathbf{B}_x \in \mathbb{R}^{6 \times 1}$  by linearizing the `MultibodyPlant` dynamics. Then we will call LQR with the projected matrices. Note that the we will use the discrete-time dynamics mode of the `MultibodyPlant` (by passing in a non-zero `time_step` to the constructor), because discrete-time dynamics engine contains our most advanced and robust algorithms for simulating contact.

Importantly, although we design the LQR controller using the reduced model, we can run this controller directly on the original model.

### 8.3.4 LQR for linear systems in implicit form

Consider a linear system given by the state-space equations,

$$\mathbf{E}\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u},$$

where  $\mathbf{E}$  may or may not be invertible. We consider this model to be in "implicit form", and will discuss much more about the use of implicit forms, and their strong connections to mechanical systems, [in the next chapter](#). Under stabilizability and detectability conditions[2], the solution for the standard infinite-horizon LQR cost function is given by

$$J^*(\mathbf{x}) = \mathbf{x}^T \mathbf{E}^T \mathbf{S} \mathbf{E} \mathbf{x}, \quad \mathbf{u}^* = -\mathbf{R}^{-1} \mathbf{B}^T \mathbf{S} \mathbf{E} \mathbf{x},$$

where  $\mathbf{S}$  is the positive-definite solution to a generalized Riccati equation:

$$\mathbf{E}^T \mathbf{S} \mathbf{A} + \mathbf{A} \mathbf{S} \mathbf{E}^T - \mathbf{E}^T \mathbf{S} \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T \mathbf{S} \mathbf{E} + \mathbf{Q} = 0.$$

The discrete-time form follows similarly.

### 8.3.5 LQR as a convex optimization

One can also design the LQR gains using linear matrix inequalities (LMIs). I will defer the derivation til we cover the policy gradient view of LQR, because the LMI formulation is based on a change of variables from the basic policy evaluation

criterion. If you want to look ahead, you can find that formulation [here](#).

Solving the algebraic Riccati equation is still the preferred way of computing the LQR solution. But it is helpful to know that one could also compute it with convex optimization. In addition to deepening our understanding, this can be useful for generalizing the basic LQR solution (e.g. for [robust stabilization](#)) or to solve for the LQR gains jointly as part of a bigger optimization.

### 8.3.6 Finite-horizon LQR via least squares

We can also obtain the solution to the discrete-time finite-horizon (including the time-varying or tracking variants) LQR problem using optimization -- in this case it actually reduces to a simple least-squares problem. The presentation in this section can be viewed as a simple implementation of the [Youla parameterization](#) (sometimes called "Q-parameterization") from controls. Small variations on the formulation here play an important role in the minimax variants of LQR (which optimize a worst-case performance), which we will discuss in the robust control chapter (e.g. [3, 4]).

First, let us appreciate that the default parameterization is not convex. Given

$$\begin{aligned} \min \sum_{n=0}^{N-1} \mathbf{x}^T[n] \mathbf{Q} \mathbf{x}[n] + \mathbf{u}^T[n] \mathbf{R} \mathbf{u}[n], \quad \mathbf{Q} = \mathbf{Q}^T \succeq 0, \mathbf{R} = \mathbf{R}^T \succ 0 \\ \text{subject to } \mathbf{x}[n+1] = \mathbf{A} \mathbf{x}[n] + \mathbf{B} \mathbf{u}[n], \\ \mathbf{x}[0] = \mathbf{x}_0 \end{aligned}$$

if we wish to search over controllers of the form

$$\mathbf{u}[n] = \mathbf{K}_n \mathbf{x}[n],$$

then we have

$$\begin{aligned} \mathbf{x}[1] &= \mathbf{A} \mathbf{x}_0 + \mathbf{B} \mathbf{K}_0 \mathbf{x}_0, \\ \mathbf{x}[2] &= \mathbf{A}(\mathbf{A} + \mathbf{B} \mathbf{K}_0) \mathbf{x}_0 + \mathbf{B} \mathbf{K}_1 (\mathbf{A} + \mathbf{B} \mathbf{K}_0) \mathbf{x}_0 \\ \mathbf{x}[n] &= \left( \prod_{i=0}^{n-1} (\mathbf{A} + \mathbf{B} \mathbf{K}_i) \right) \mathbf{x}_0 \end{aligned}$$

As you can see, the  $\mathbf{x}[n]$  terms in the cost function include our decision variables multiplied together -- resulting in a non-convex objective. The trick is to re-parameterize the decision variables, and write the feedback in the form:

$$\mathbf{u}[n] = \tilde{\mathbf{K}}_n \mathbf{x}_0,$$

leading to

$$\begin{aligned} \mathbf{x}[1] &= \mathbf{A} \mathbf{x}_0 + \mathbf{B} \tilde{\mathbf{K}}_0 \mathbf{x}_0, \\ \mathbf{x}[2] &= \mathbf{A}(\mathbf{A} + \mathbf{B} \tilde{\mathbf{K}}_0) \mathbf{x}_0 + \mathbf{B} \tilde{\mathbf{K}}_1 \mathbf{x}_0 \\ \mathbf{x}[n] &= \left( \mathbf{A}^n + \sum_{i=0}^{n-1} \mathbf{A}^{n-i-1} \mathbf{B} \tilde{\mathbf{K}}_i \right) \mathbf{x}_0 \end{aligned}$$

Now all of the decision variables,  $\tilde{\mathbf{K}}_i$ , appear linearly in the solution to  $\mathbf{x}[n]$  and therefore (convex) quadratically in the objective.

We still have an objective function that depends on  $\mathbf{x}_0$ , but we would like to find the optimal  $\tilde{\mathbf{K}}_i$  for all  $\mathbf{x}_0$ . To achieve this let us evaluate the optimality conditions of this least squares problem, starting by taking the gradient of the objective with respect to  $\tilde{\mathbf{K}}_i$ , which is:

$$\mathbf{x}_0 \mathbf{x}_0^T \left( \tilde{\mathbf{K}}_i^T \left( \mathbf{R} + \sum_{m=i+1}^{N-1} \mathbf{B}^T (\mathbf{A}^{m-i-1})^T \mathbf{Q} \mathbf{A}^{m-i-1} \mathbf{B} \right) + \sum_{m=i+1}^{N-1} (\mathbf{A}^m)^T \mathbf{Q} \mathbf{A}^{m-i-1} \mathbf{B} \right).$$

We can satisfy this optimality condition for all  $\mathbf{x}_0$  by solving the *linear* matrix equation:

$$\tilde{\mathbf{K}}_i^T \left( \mathbf{R} + \sum_{m=i+1}^{N-1} \mathbf{B}^T (\mathbf{A}^{m-i-1})^T \mathbf{Q} \mathbf{A}^{m-i-1} \mathbf{B} \right) + \sum_{m=i+1}^{N-1} (\mathbf{A}^m)^T \mathbf{Q} \mathbf{A}^{m-i-1} \mathbf{B} = 0.$$

We can always solve for  $\tilde{\mathbf{K}}_i$  since it's multiplied by a (symmetric) positive definite matrix (it is the sum of a positive definite matrix and many positive semi-definite matrices), which is always invertible.

If you need to recover the original  $\mathbf{K}_i$  parameters, you can extract them recursively with

$$\begin{aligned} \tilde{\mathbf{K}}_0 &= \mathbf{K}_0, \\ \tilde{\mathbf{K}}_n &= \mathbf{K}_n \prod_{i=0}^{n-1} (\mathbf{A} + \mathbf{B} \mathbf{K}_i), \quad 0 < n \leq N-1. \end{aligned}$$

But often this is not actually necessary. In some applications it's enough to know the performance cost under LQR control, or to handle the response to disturbances explicitly with the disturbance-based feedback (which I've already promised for the robust control chapter). After all, the problem formulation that we've written here, which makes no mention of disturbances, assumes the model is perfect and the controls  $\tilde{\mathbf{K}}_n \mathbf{x}_0$  are just as suitable for deployment as  $\mathbf{K}_n \mathbf{x}[n]$ .

"System-Level Synthesis" (SLS) is the name for an important and slightly different approach, where one optimizes the *closed-loop response* directly[5]. Although SLS is a very general tool, for the particular formulation we are considering here it reduces to creating additional decision variables  $\Phi_i$ , such that

$$\mathbf{x}[n] = \Phi_n \mathbf{x}[0],$$

and writing the optimization above as

$$\begin{aligned} \min_{\tilde{\mathbf{K}}, \Phi} \quad & \sum_{n=0}^{N-1} \mathbf{x}^T[0] \left( \Phi_n^T \mathbf{Q} \Phi_n + \tilde{\mathbf{K}}_n^T \mathbf{R} \tilde{\mathbf{K}}_n \right) \mathbf{x}[0], \\ \text{subject to} \quad & \forall n, \quad \Phi_{n+1} = \mathbf{A} \Phi_n + \mathbf{B} \tilde{\mathbf{K}}_n. \end{aligned}$$

Once again, the algorithms presented here are not as efficient as solving the Riccati equation if we only want the solution to the simple case, but they become very powerful if we want to combine the LQR synthesis with other objectives/constraints. For instance, if we want to add some sparsity constraints (e.g. enforcing that some elements of  $\tilde{\mathbf{K}}_i$  are zero), then we could solve the quadratic optimization subject to linear equality constraints [6].

### 8.3.7 Minimum-time LQR

Coming soon. The basic result can be found in [7], and some robotics applications in, for instance, [8, 9]. For discrete-time systems, [10] provides a new computational approach.

### 8.3.8 Parameterized Riccati Equations

When we are designing LQR controllers based on a linearization (around some operating state or trajectory) of a nonlinear system, then it is interesting to ask: how does the LQR solution change as you change the nominal point (or trajectory)? If the nonlinear dynamics are smooth, does the solution also change smoothly?

The answer is yes -- the Riccati solution does change smoothly, and its local variation can be obtained with a little bit of work by taking Taylor approximation of the  $\mathbf{A}$  and

B matrices [11].

## 8.4 EXERCISES

### Exercise 8.1 (LQR on a Drake Diagram)

To help you get a little more familiar with Drake, the exercise in will step you through the process of building a Drake Diagram, and designing an LQR controller for it. Systems and Diagrams are the way we write modular code describing dynamical systems, and implementing an algorithm like LQR against the System abstraction becomes very powerful.

## 8.5 NOTES

### 8.5.1 Finite-horizon LQR derivation (general form)

For completeness, I've included here the derivation for continuous-time finite-horizon LQR with all of the bells and whistles.

Consider an time-varying affine (approximation of a) continuous-time dynamical system in state-space form:

$$\dot{\mathbf{x}} = \mathbf{A}(t)\mathbf{x} + \mathbf{B}(t)\mathbf{u} + \mathbf{c}(t),$$

and a running cost function in the general quadratic form:

$$\begin{aligned} \ell(t, \mathbf{x}, \mathbf{u}) &= \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}^T \mathbf{Q}(t) \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} + \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix}^T \mathbf{R}(t) \begin{bmatrix} \mathbf{u} \\ 1 \end{bmatrix} + 2\mathbf{x}^T \mathbf{N}(t) \mathbf{u}, \\ \forall t \in [t_0, t_f], \quad \mathbf{Q}(t) &= \begin{bmatrix} \mathbf{Q}_{xx}(t) & \mathbf{q}_x(t) \\ \mathbf{q}_x^T(t) & q_0(t) \end{bmatrix}, \mathbf{Q}_{xx}(t) \succeq 0, \quad \mathbf{R}(t) = \begin{bmatrix} \mathbf{R}_{uu}(t) & \mathbf{r}_u(t) \\ \mathbf{r}_u^T(t) & r_0(t) \end{bmatrix}, \mathbf{R}_{uu}(t) \succ 0. \end{aligned}$$

Observe that our LQR "optimal tracking" derivation fits in this form, as we can always write

$$(\mathbf{x} - \mathbf{x}_d(t))^T \mathbf{Q}_t (\mathbf{x} - \mathbf{x}_d(t)) + (\mathbf{u} - \mathbf{u}_d(t))^T \mathbf{R}_t (\mathbf{u} - \mathbf{u}_d(t)) + 2(\mathbf{x} - \mathbf{x}_d(t))^T \mathbf{N}_t (\mathbf{u} - \mathbf{u}_d(t)),$$

by taking

$$\begin{aligned} \mathbf{Q}_{xx} &= \mathbf{Q}_t, & \mathbf{q}_x &= -\mathbf{Q}_t \mathbf{x}_d - \mathbf{N}_t \mathbf{u}_d, & q_0 &= \mathbf{x}_d^T \mathbf{Q}_t \mathbf{x}_d + 2\mathbf{x}_d^T \mathbf{N}_t \mathbf{u}_d, \\ \mathbf{R}_{uu} &= \mathbf{R}_t, & \mathbf{r}_u &= -\mathbf{R}_t \mathbf{u}_d - \mathbf{N}_t^T \mathbf{x}_d, & r_0 &= \mathbf{u}_d^T \mathbf{R}_t \mathbf{u}_d, & \mathbf{N} &= \mathbf{N}_t. \end{aligned}$$

Of course, we can also add a quadratic final cost with  $\mathbf{Q}_f$ . Let's search for a positive quadratic, time-varying cost-to-go function of the form:

$$\begin{aligned} J(t, \mathbf{x}) &= \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}^T \mathbf{S}(t) \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}, \quad \mathbf{S}(t) = \begin{bmatrix} \mathbf{S}_{xx}(t) & \mathbf{s}_x(t) \\ \mathbf{s}_x^T(t) & s_0(t) \end{bmatrix}, \mathbf{S}_{xx}(t) \succ 0, \\ \frac{\partial J}{\partial \mathbf{x}} &= 2\mathbf{x}^T \mathbf{S}_{xx} + 2\mathbf{s}_x^T, & \frac{\partial J}{\partial t} &= \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}^T \dot{\mathbf{S}} \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}. \end{aligned}$$

Writing out the HJB:

$$\min_{\mathbf{u}} \left[ \ell(\mathbf{x}, \mathbf{u}) + \frac{\partial J}{\partial \mathbf{x}} [\mathbf{A}(t)\mathbf{x} + \mathbf{B}(t)\mathbf{u} + \mathbf{c}(t)] + \frac{\partial J}{\partial t} \right] = 0,$$

we can find the minimizing  $\mathbf{u}$  with

$$\frac{\partial}{\partial \mathbf{u}} = 2\mathbf{u}^T \mathbf{R}_{uu} + 2\mathbf{r}_u^T + 2\mathbf{x}^T \mathbf{N} + (2\mathbf{x}^T \mathbf{S}_{xx} + 2\mathbf{s}_x^T) \mathbf{B} = 0$$

$$\mathbf{u}^* = -\mathbf{R}_{uu}^{-1} \begin{bmatrix} \mathbf{N} + \mathbf{S}_{xx} \mathbf{B} \\ \mathbf{r}_u^T + \mathbf{s}_x^T \mathbf{B} \end{bmatrix}^T \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = -\mathbf{K}(t) \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} = -\mathbf{K}_x(t) \mathbf{x} - \mathbf{k}_0(t).$$

Inserting this back into the HJB gives us the updated Riccati differential equation. Since this must hold for all  $\mathbf{x}$ , we can collect the quadratic, linear, and offset terms and set them each individually equal to zero, yielding:

$$\begin{aligned} -\dot{\mathbf{S}}_{xx} &= \mathbf{Q}_{xx} - (\mathbf{N} + \mathbf{S}_{xx} \mathbf{B}) \mathbf{R}_{uu}^{-1} (\mathbf{N} + \mathbf{S}_{xx} \mathbf{B})^T + \mathbf{S}_{xx} \mathbf{A} + \mathbf{A}^T \mathbf{S}_{xx}, \\ -\dot{\mathbf{s}}_x &= \mathbf{q}_x - (\mathbf{N} + \mathbf{S}_{xx} \mathbf{B}) \mathbf{R}_{uu}^{-1} (\mathbf{r}_u + \mathbf{B}^T \mathbf{s}_x) + \mathbf{A}^T \mathbf{s}_x + \mathbf{S}_{xx} \mathbf{c}, \\ -\dot{s}_0 &= q_0 + r_0 - (\mathbf{r}_u + \mathbf{B}^T \mathbf{s}_x)^T \mathbf{R}_{uu}^{-1} (\mathbf{r}_u + \mathbf{B}^T \mathbf{s}_x) + 2\mathbf{s}_x^T \mathbf{c}, \end{aligned}$$

with the final conditions  $\mathbf{S}(t_f) = \mathbf{Q}_f$ .

In the discrete-time version, we have...

Phew! Numerical solutions to these equations can be obtained by calling the [FiniteHorizonLinearQuadraticRegulator](#) methods in [DRAKE](#). May you never have to type them in and unit test them yourself.

## REFERENCES

1. Michael Posa and Scott Kuindersma and Russ Tedrake, "Optimization and stabilization of trajectories for constrained dynamical systems", *Proceedings of the International Conference on Robotics and Automation (ICRA)*, pp. 1366-1373, May, 2016. [ [link](#) ]
2. Volker Mehrmann, "The autonomous linear quadratic control problem: theory and numerical solution", Springer, no. 163, 1991.
3. J. Lofberg, "Approximations of closed-loop minimax {MPC}", *42nd {IEEE} {International} {Conference} on {Decision} and {Control} ({IEEE} {Cat}. {No}.03CH37475)*, vol. 2, pp. 1438-1442 Vol.2, Dec, 2003.
4. Sadra Sadraddini and Russ Tedrake, "Robust Output Feedback Control with Guaranteed Constraint Satisfaction", *In the Proceedings of 23rd ACM International Conference on Hybrid Systems: Computation and Control*, pp. 12, April, 2020. [ [link](#) ]
5. James Anderson and John C. Doyle and Steven Low and Nikolai Matni, "System {Level} {Synthesis}", *arXiv:1904.01634 [cs, math]*, apr, 2019.
6. Yuh-Shyang Wang and Nikolai Matni and John C. Doyle, "Localized {LQR} {Optimal} {Control}", *arXiv:1409.6404 [cs, math]*, September, 2014.
7. E.I. Verriest and F.L. Lewis, "On the Linear Quadratic Minimum-Time Problem", *IEEE Transactions on Automatic Control*, vol. 36, no. 7, pp. 859-863, July, 1991.
8. Elena Glassman and Russ Tedrake, "A Quadratic Regulator-Based Heuristic for Rapidly Exploring State Space", *Under review*, 2010. [ [link](#) ]
9. Sikang Liu and Nikolay Atanasov and Kartik Mohta and Vijay Kumar, "Search-based motion planning for quadrotors using linear quadratic minimum time control", *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 2872-2879, Sep., 2017.
10. Tobia Marcucci and Jack Umenberger and Pablo A. Parrilo and Russ Tedrake, "Shortest Paths in Graphs of Convex Sets", *arXiv preprint*, 2023. [ [link](#) ]

11. Anirudha Majumdar and Mark Tobenkin and Russ Tedrake, "Algebraic Verification for Parameterized Motion Planning Libraries", *Proceedings of the 2012 American Control Conference (ACC)* , pp. 8, June, 2012. [ [link](#) ]

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

[Accessibility](#)

© Russ Tedrake, 2024

# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 9

# Lyapunov Analysis

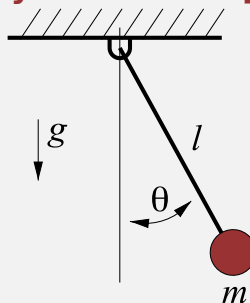
Optimal control provides a powerful framework for formulating control problems using the language of optimization. But solving optimal control problems for nonlinear systems is hard! In many cases, we don't really care about finding the *optimal* controller, but would be satisfied with any controller that is guaranteed to accomplish the specified task. In many cases, we still formulate these problems using computational tools from optimization, and in this chapter we'll learn about tools that can provide guaranteed control solutions for systems that are beyond the complexity for which we can find the optimal feedback.

There are many excellent books on Lyapunov analysis; for instance [1] is an excellent and very readable reference and [2] can provide a rigorous treatment. In this chapter I will summarize (without proof) some of the key theorems from Lyapunov analysis, but then will also introduce a number of numerical algorithms... many of which are new enough that they have not yet appeared in any mainstream textbooks.

### 9.1 LYAPUNOV FUNCTIONS

Let's start with our favorite simple example.

#### Example 9.1 (Stability of the Damped Pendulum)



Recall that the equations of motion of the damped simple pendulum are given by



$$ml^2\ddot{\theta} + mgl \sin \theta = -b\dot{\theta},$$

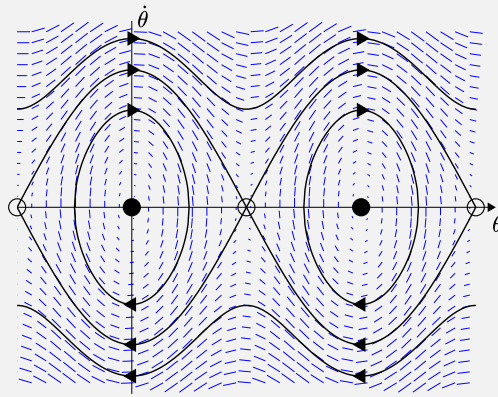
which I've written with the damping on the right-hand side to remind us that it is an external torque that we've modeled.

These equations represent a simple second-order differential equation; in chapter 2 we discussed at some length what was known about the solutions to this differential equation--in practice we do not have a closed-form solution for  $\theta(t)$  as a function of the initial conditions. Since we couldn't provide a solution analytically, in chapter 2 we resorted to a graphical analysis, and confirmed the intuition that there are fixed points in the system (at  $\theta = k\pi$  for every integer  $k$ ) and that the fixed points at  $\theta = 2\pi k$  are asymptotically stable with a large basin of attraction. The graphical analysis gave us this intuition, but can we actually prove this stability property? In a way that might also work for much more complicated systems?

One route forward was from looking at the total system energy (kinetic + potential), which we can write down:

$$E(\theta, \dot{\theta}) = \frac{1}{2}ml^2\dot{\theta}^2 - mgl \cos \theta.$$

Recall that the contours of this energy function are the orbits of the undamped pendulum.



A natural route to proving the stability of the downward fixed points is by arguing that energy (almost always) decreases for the damped pendulum ( $b > 0$ ) and so the system will eventually come to rest at the minimum energy,  $E = -mgl$ , which happens at  $\theta = 2\pi k$ . Let's make that argument slightly more precise.

Evaluating the time derivative of the energy reveals

$$\frac{d}{dt}E = \frac{\partial E}{\partial \theta}\dot{\theta} + \frac{\partial E}{\partial \dot{\theta}}\ddot{\theta} = \dot{\theta}(ml^2\ddot{\theta} + mgl \sin \theta) = -b\dot{\theta}^2 \leq 0.$$

So we know that energy will never increase, that energy is bounded, and that it obtains its minima at the downward (stable) fixed points. Moreover, we now know that energy is strictly decreasing whenever  $\dot{\theta} \neq 0$ . We're getting close.

In order to show that the system converges asymptotically to some fixed point, we need one more observation. For non-fixed-point states with  $\dot{\theta} = 0$ , we have  $\dot{E} = 0$ , but only for an instant. A moment later, we'll have some non-zero velocity once again, and again start dissipating energy. All together, these arguments are sufficient to demonstrate that  $\mathbf{x} \rightarrow$  a fixed point as  $t \rightarrow \infty$ . We'll make this argument more general and more precise below.

This is an important example. It demonstrated that we could use a relatively simple

function -- the total system energy -- to describe something about the long-term dynamics of the pendulum even though the actual trajectories of the system are (analytically) very complex. It also demonstrated one of the subtleties of using an energy-like function that is non-increasing (instead of strictly decreasing) to prove asymptotic stability.

Lyapunov functions generalize this notion of an energy function to more general systems, which might not be stable in the sense of some mechanical energy. If I can find any positive function, call it  $V(\mathbf{x})$ , that gets smaller over time as the system evolves, then I can potentially use  $V$  to make a statement about the long-term behavior of the system.  $V$  is called a *Lyapunov function*.

Recall that we defined three separate notions for stability of a fixed-point of a nonlinear system: stability i.s.L., asymptotic stability, and exponential stability. We can use Lyapunov functions to demonstrate each of these, in turn.

### Theorem 9.1 - Lyapunov's Direct Method (for local stability)

Given a system  $\dot{\mathbf{x}} = f(\mathbf{x})$ , with  $f$  continuous, and for some region  $\mathcal{D}$  around the origin (specifically an open subset of  $\mathbf{R}^n$  containing the origin), if I can produce a scalar, continuously-differentiable function  $V(\mathbf{x})$ , such that

$$V(\mathbf{x}) > 0, \forall \mathbf{x} \in \mathcal{D} \setminus \{0\} \quad V(0) = 0, \text{ and} \\ \dot{V}(\mathbf{x}) = \frac{\partial V}{\partial \mathbf{x}} f(\mathbf{x}) \leq 0, \forall \mathbf{x} \in \mathcal{D} \setminus \{0\} \quad \dot{V}(0) = 0,$$

then the origin ( $\mathbf{x} = 0$ ) is stable in the sense of Lyapunov (i.s.L.). [Note: the notation  $A \setminus B$  represents the set  $A$  with the elements of  $B$  removed.]

If, additionally, we have

$$\dot{V}(\mathbf{x}) = \frac{\partial V}{\partial \mathbf{x}} f(\mathbf{x}) < 0, \forall \mathbf{x} \in \mathcal{D} \setminus \{0\},$$

then the origin is (locally) asymptotically stable. And if we have

$$\dot{V}(\mathbf{x}) = \frac{\partial V}{\partial \mathbf{x}} f(\mathbf{x}) \leq -\alpha V(\mathbf{x}), \forall \mathbf{x} \in \mathcal{D} \setminus \{0\},$$

for some  $\alpha > 0$ , then the origin is (locally) exponentially stable.

Note that for the sequel we will use the notation  $V \succ 0$  to denote a *positive-definite function*, meaning that  $V(0) = 0$  and  $V(\mathbf{x}) > 0$ , for all  $\mathbf{x} \neq 0$ . We can use "positive definite in  $\mathcal{D}$ " if this applies  $\forall \mathbf{x} \in \mathcal{D}$ . Similarly, we'll use  $V \succeq 0$  for positive semi-definite,  $V \prec 0$  for negative-definite functions, etc.

The intuition here is exactly the same as for the energy argument we made in the pendulum example: since  $\dot{V}(x)$  is always zero or negative, the value of  $V(x)$  will only get smaller (or stay the same) as time progresses. Inside the subset  $\mathcal{D}$ , for every  $\epsilon$ -ball, I can choose a  $\delta$  such that  $|x(0)|^2 < \delta \Rightarrow |x(t)|^2 < \epsilon, \forall t$  by choosing  $\delta$  sufficiently small so that the sublevel set of  $V(x)$  for the largest value that  $V(x)$  takes in the  $\delta$  ball is completely contained in the  $\epsilon$  ball. Since the value of  $V$  can only get smaller (or stay constant) in time, this gives stability i.s.L.. If  $\dot{V}$  is strictly negative away from the origin, then it must eventually get to the origin (asymptotic stability). The exponential condition is implied by the fact that  $\forall t > 0, V(\mathbf{x}(t)) \leq V(\mathbf{x}(0))e^{-\alpha t}$ .

Notice that the system analyzed above,  $\dot{\mathbf{x}} = f(\mathbf{x})$ , did not have any control inputs. Lyapunov analysis is used to study either the passive dynamics of a system or the dynamics of a closed-loop system (e.g. the plant and controller connected in a feedback loop). We will see generalizations of the Lyapunov function ideas to input-

output systems later in the text.

### 9.1.1 Global Stability

The notion of a fixed point being stable i.s.L. is inherently a local notion of stability (defined with  $\epsilon$  and  $\delta$  balls around the origin), but the notions of asymptotic and exponential stability can be applied globally. The Lyapunov theorems work for this case, too, with only minor modification.

#### Theorem 9.2 - Lyapunov analysis for global stability

Given a system  $\dot{\mathbf{x}} = f(\mathbf{x})$ , with  $f$  continuous, if I can produce a scalar, continuously-differentiable function  $V(\mathbf{x})$ , such that

$$\begin{aligned} V(\mathbf{x}) &> 0, \\ \dot{V}(\mathbf{x}) = \frac{\partial V}{\partial \mathbf{x}} f(\mathbf{x}) &< 0, \text{ and} \\ V(\mathbf{x}) &\rightarrow \infty \text{ whenever } \|\mathbf{x}\| \rightarrow \infty, \end{aligned}$$

then the origin ( $\mathbf{x} = 0$ ) is globally asymptotically stable (G.A.S.).

If additionally we have that

$$\dot{V}(\mathbf{x}) \leq -\alpha V(\mathbf{x}),$$

for some  $\alpha > 0$ , then the origin is globally exponentially stable.

The new condition, on the behavior as  $\|\mathbf{x}\| \rightarrow \infty$  is known as "**radially unbounded**", and is required to make sure that trajectories cannot diverge to infinity even as  $V$  decreases; it is only required for global stability analysis. This radially unbounded condition is sufficient but is not necessary; other conditions can potentially be invoked to guarantee that the system avoids this pathology. More essential is that the resulting trajectory  $\mathbf{x}(t)$  is bounded for all  $t$ .

### 9.1.2 LaSalle's Invariance Principle

Perhaps you noticed the disconnect between the statement above and the argument that we made for the stability of the pendulum. In the pendulum example, using the mechanical energy resulted in a Lyapunov function time derivative that was only negative semi-definite, but we eventually argued that the fixed points were asymptotically stable. That took a little extra work, involving an argument about the fact that the fixed points were the only place that the system could stay with  $\dot{E} = 0$ ; every other state with  $\dot{E} = 0$  was only transient. We can formalize this idea for the more general Lyapunov function statements--it is known as LaSalle's Theorem.

#### Theorem 9.3 - LaSalle's Theorem

Given a system  $\dot{\mathbf{x}} = f(\mathbf{x})$  with  $f$  continuous. If we can produce a scalar function  $V(\mathbf{x})$  with continuous derivatives for which we have

$$V(\mathbf{x}) > 0, \quad \dot{V}(\mathbf{x}) \leq 0,$$

and  $V(\mathbf{x}) \rightarrow \infty$  as  $\|\mathbf{x}\| \rightarrow \infty$ , then  $\mathbf{x}$  will converge to the largest **invariant set** where  $\dot{V}(\mathbf{x}) = 0$ .

To be clear, an **invariant set**,  $\mathcal{G}$ , of the dynamical system is a set for which

$\mathbf{x}(0) \in \mathcal{G} \Rightarrow \forall t > 0, \mathbf{x}(t) \in \mathcal{G}$ . In other words, once you enter the set you never leave. The "largest invariant set" with  $\dot{V}(\mathbf{x}) = 0$  need not be connected; in fact for the pendulum example each fixed point is an invariant set with  $\dot{V} = 0$ , so the largest invariant set is the *union* of all the fixed points of the system. There are also variants of LaSalle's Theorem which work over a region. For example, the total energy of the pendulum does not actually satisfy the radially-unbounded condition (the energy is finite as  $\theta \rightarrow \infty$ ), but observing that  $\mathbf{x}(t)$  is bounded for all initial conditions is sufficient to invoke a variant of LaSalle's theorem [2, §4.2].

Finding a Lyapunov function which  $\dot{V} < 0$  is more difficult than finding one that has  $\dot{V} \preceq 0$ . LaSalle's theorem gives us the ability to make a statement about *asymptotic* stability even in this case. In the pendulum example, every state with  $\dot{\theta} = 0$  had  $\dot{E} = 0$ , but only the fixed points are in the largest invariant set.

### Example 9.2 (Swing-up for the Cart-Pole System)

Recall the [example of using partial-feedback linearization to generate a swing-up controller for the cart-pole system](#). We first examined the dynamics of the pole (pendulum) only, by writing it's energy:

$$E(\mathbf{x}) = \frac{1}{2}\dot{\theta}^2 - \cos \theta,$$

desired energy,  $E^d = 1$ , and the difference  $\tilde{E}(\mathbf{x}) = E(\mathbf{x}) - E^d$ . We were able to show that our proposed controller produced

$$\dot{\tilde{E}} = -k\dot{\theta}^2 \cos^2 \theta \tilde{E},$$

where  $k$  is a positive gain that we get to choose. And we said that was good!

Now we have the tools to understand that really we have a Lyapunov function

$$V(\mathbf{x}) = \frac{1}{2} \tilde{E}^2(\mathbf{x}),$$

and what we have shown is that  $\dot{V} \leq 0$ . By LaSalle, we can only argue that the closed-loop system will converge to the largest invariant set, which here is the entire homoclinic orbit:  $\tilde{E}(\mathbf{x}) = 0$ . We have to switch to the LQR controller in order to stabilize the upright.

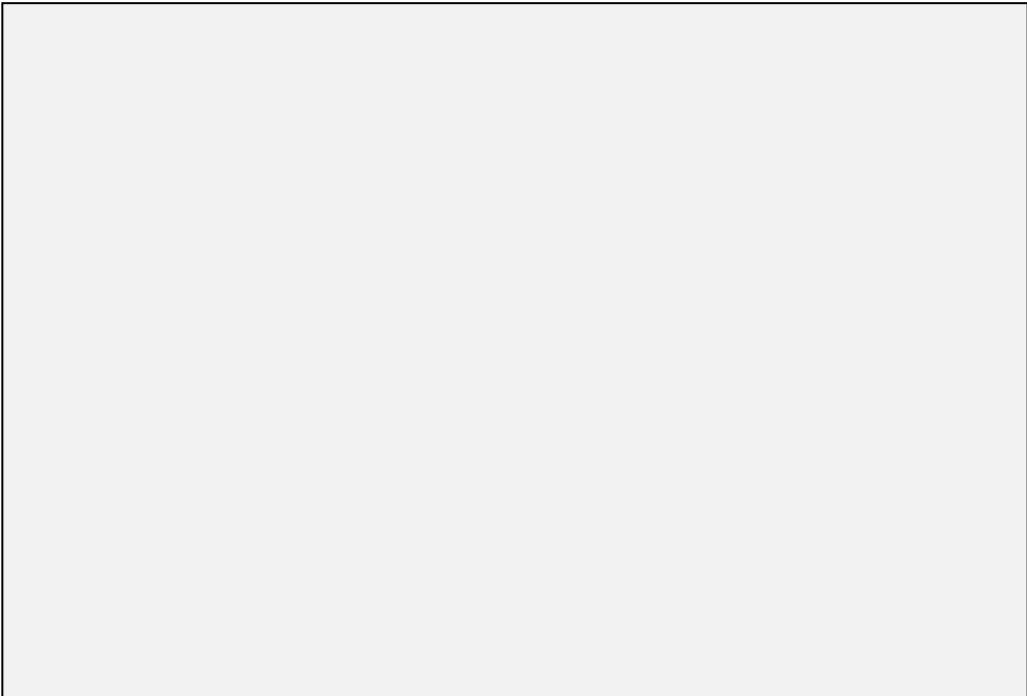


Figure 9.1 - Lyapunov function:  $V(\mathbf{x}) = \frac{1}{2} \tilde{E}^2(\mathbf{x})$ .

Figure 9.2 - Time-derivative of the Lyapunov function:  $\dot{V}(\mathbf{x})$ .

As you can see from the plots,  $\dot{V}(\mathbf{x})$  ends up being a quite non-trivial function! We'll develop the computational tools for verifying the Lyapunov/LaSalle conditions for systems of this complexity in the upcoming sections.

### 9.1.3 Relationship to the Hamilton-Jacobi-Bellman equations

At this point, you might be wondering if there is any relationship between Lyapunov functions and the cost-to-go functions that we discussed in the context of dynamic programming. After all, the cost-to-go functions also captured a great deal about the long-term dynamics of the system in a scalar function. We can see the connection if we re-examine the HJB equation

$$0 = \min_{\mathbf{u}} \left[ \ell(\mathbf{x}, \mathbf{u}) + \frac{\partial J^*}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) \right]$$

Let's imagine that we can solve for the optimizing  $\mathbf{u}^*(\mathbf{x})$ , then we are left with  $0 = \ell(\mathbf{x}, \mathbf{u}^*) + \frac{\partial J^*}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}^*)$  or simply

$$\dot{J}^*(\mathbf{x}) = -\ell(\mathbf{x}, \mathbf{u}^*) \quad \text{vs} \quad \dot{V}(\mathbf{x}) \leq 0.$$

In other words, in optimal control we must find a cost-to-go function which matches this gradient for every  $\mathbf{x}$ ; that's very difficult and involves solving a potentially high-dimensional partial differential equation. By contrast, Lyapunov analysis is asking for much less - any function which is going downhill (at any rate) for all states. This can be much easier, for theoretical work, but also for our numerical algorithms. Also note that if we do manage to find the optimal cost-to-go,  $J^*(\mathbf{x})$ , then it can also serve as a Lyapunov function so long as  $\ell(\mathbf{x}, \mathbf{u}^*(\mathbf{x})) \geq 0$ .

### 9.1.4 Lyapunov functions for estimating regions of attraction

There is another very important connection between Lyapunov functions and the concept of an invariant set: *any sublevel set of a Lyapunov function is also an invariant set*. This gives us the ability to use sublevel sets of a Lyapunov function as approximations of the region of attraction for nonlinear systems. For a locally attractive fixed point,  $\mathbf{x}^*$ , the *region of attraction* to  $\mathbf{x}^*$  is the largest set  $\mathcal{D} \subseteq \mathcal{X}$  for which  $\mathbf{x}(0) \in \mathcal{D} \Rightarrow \lim_{t \rightarrow \infty} \mathbf{x}(t) = \mathbf{x}^*$ . Regions of attraction are always open, connected, and invariant sets (with boundaries ).

#### Theorem 9.4 - Lyapunov invariant set and region of attraction theorem

Given a system  $\dot{\mathbf{x}} = f(\mathbf{x})$  with  $f$  continuous, if we can find a scalar function  $V(\mathbf{x}) > 0$  and a bounded sublevel set

$$\mathcal{G} : \{\mathbf{x} | V(\mathbf{x}) \leq \rho\}$$

on which

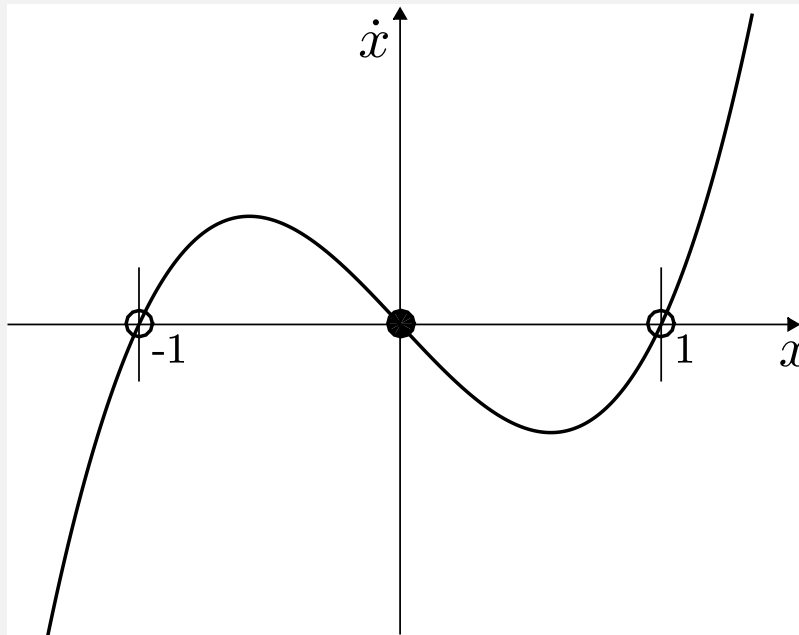
$$\forall \mathbf{x} \in \mathcal{G}, \dot{V}(\mathbf{x}) \leq 0,$$

then  $\mathcal{G}$  is an invariant set. By LaSalle,  $\mathbf{x}$  will converge to the largest invariant subset of  $\mathcal{G}$  on which  $\dot{V} = 0$ .

If  $\dot{V}(\mathbf{x}) < 0$  in  $\mathcal{G}$ , then the origin is locally asymptotically stable and furthermore the set  $\mathcal{G}$  is inside the region of attraction of this fixed point. Alternatively, if  $\dot{V}(\mathbf{x}) \leq 0$  in  $\mathcal{G}$  and  $\mathbf{x} = 0$  is the only invariant subset of  $\mathcal{G}$  where  $\dot{V} = 0$ , then the origin is asymptotically stable and the set  $\mathcal{G}$  is inside the region of attraction of this fixed point.

### Example 9.3 (Region of attraction for a one-dimensional system)

Consider the first-order, one-dimensional system  $\dot{x} = -x + x^3$ . We can quickly understand this system using our tools for graphical analysis.



In the vicinity of the origin,  $\dot{x}$  looks like  $-x$ , and as we move away it looks increasingly like  $x^3$ . There is a stable fixed point at the origin and unstable fixed points at  $\pm 1$ . In fact, we can deduce visually that the region of attraction to the stable fixed point at the origin is  $x \in (-1, 1)$ . Let's see if we can demonstrate this with a Lyapunov argument.

First, let us linearize the dynamics about the origin and use the Lyapunov equation for linear systems to find a candidate  $V(\mathbf{x})$ . Since the linearization is  $\dot{x} = -x$ , if we take  $\mathbf{Q} = 1$ , then we find  $\mathbf{P} = \frac{1}{2}$  is the positive definite solution to the algebraic Lyapunov equation (1). Proceeding with

$$V(\mathbf{x}) = \frac{1}{2}x^2,$$

we have

$$\dot{V} = x(-x + x^3) = -x^2 + x^4.$$

This function is zero at the origin, negative for  $|x| < 1$ , and positive for  $|x| > 1$ . Therefore we can conclude that the sublevel set  $V < \frac{1}{2}$  is invariant and the set  $x \in (-1, 1)$  is inside the region of attraction of the nonlinear system. In fact, this estimate is tight.

### 9.1.5 Robustness analysis using "common Lyapunov functions"

While we will defer most discussions on robustness analysis until [later in the notes](#), the idea of a *common Lyapunov function*, which we introduced briefly for linear systems in the [example above](#), can be readily extended to nonlinear systems and region of attraction analysis. Imagine that you have a model of a dynamical system

but that you are uncertain about some of the parameters. For example, you have a model of a quadrotor, and are fairly confident about the mass and lengths (both of which are easy to measure), but are not confident about the moment of inertia. One approach to robustness analysis is to define a bounded uncertainty, which could take the form of

$$\dot{\mathbf{x}} = f_{\alpha}(\mathbf{x}), \quad \alpha_{\min} \leq \alpha \leq \alpha_{\max},$$

with  $\alpha$  a vector of uncertain parameters in your model. Richer specifications of the uncertainty bounds are also possible, but this will serve for our examples.

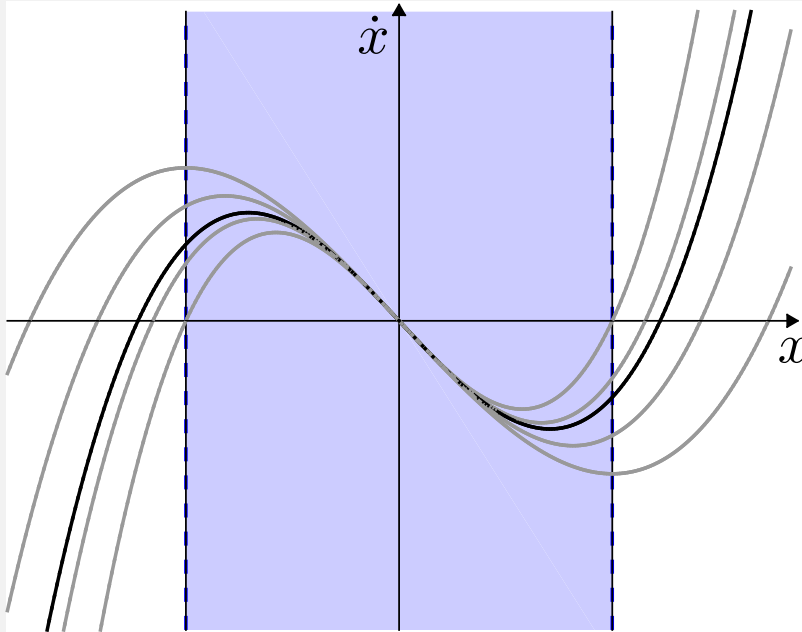
In standard Lyapunov analysis, we are searching for a function that goes downhill for all  $\mathbf{x}$  to make statements about the long-term dynamics of the system. In common Lyapunov analysis, we can make many similar statements about the long-term dynamics of an uncertain system if we can find a single Lyapunov function that goes downhill *for all possible values of  $\alpha$* . If we can find such a function, then we can use it to make statements with all of the variations we've discussed (local, regional, or global; in the sense of Lyapunov, asymptotic, or exponential).

### Example 9.4 (A one-dimensional system with gain uncertainty)

Let's consider the same one-dimensional example used above, but add an uncertain parameter into the dynamics. In particular, consider the system:

$$\dot{x} = -x + \alpha x^3, \quad \frac{3}{4} < \alpha < \frac{3}{2}.$$

Plotting the graph of the one-dimensional dynamics for a few values of  $\alpha$ , we can see that the fixed point at the origin is still stable, but the *robust region of attraction* to this fixed point (shaded in blue below) is smaller than the region of attraction for the system with  $\alpha = 1$ .



Taking the same Lyapunov candidate as above,  $V = \frac{1}{2}x^2$ , we have

$$\dot{V} = -x^2 + \alpha x^4.$$

This function is zero at the origin, and negative for all  $\alpha$  whenever  $x^2 > \alpha x^4$ , or



$$|x| < \frac{1}{\sqrt{\alpha_{max}}} = \sqrt{\frac{2}{3}}.$$

Therefore, we can conclude that  $|x| < \sqrt{\frac{2}{3}}$  is inside the robust region of attraction of the uncertain system.

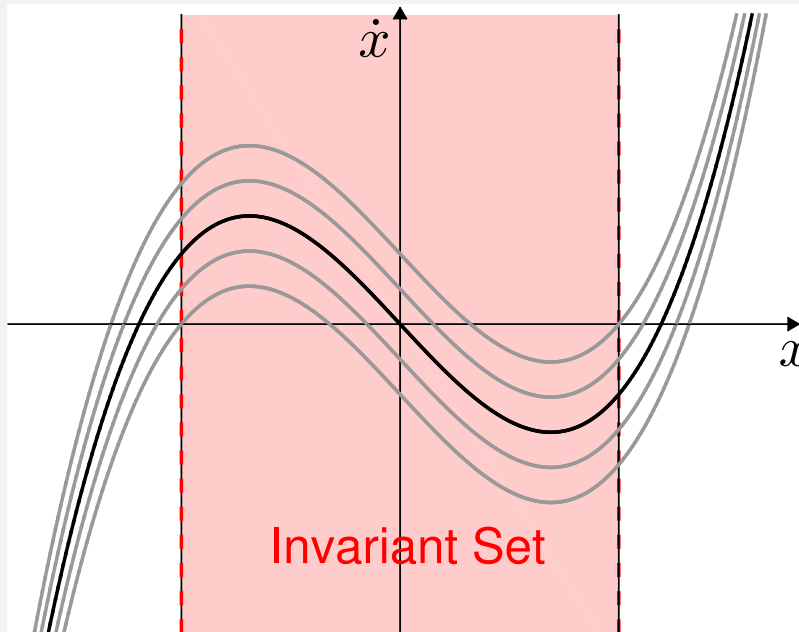
Not all forms of uncertainty are as simple to deal with as the gain uncertainty in that example. For many forms of uncertainty, we might not even know the location of the fixed points of the uncertain system. In this case, we can often still use common Lyapunov functions to give some guarantees about the system, such as guarantees of *robust set invariance*. For instance, if you have uncertain parameters on a quadrotor model, you might be ok with the quadrotor stabilizing to a pitch of 0.01 radians, but you would like to guarantee that it definitely does not flip over and crash into the ground.

### Example 9.5 (A one-dimensional system with additive uncertainty)

Now consider the system:

$$\dot{x} = -x + x^3 + \alpha, \quad -\frac{1}{4} < \alpha < \frac{1}{4}.$$

Plotting the graph of the one-dimensional dynamics for a few values of  $\alpha$ , this time we can see that the fixed point is not necessarily at the origin; the location of the fixed point moves depending on the value of  $\alpha$ . But we should be able to guarantee that the uncertain system will stay near the origin if it starts near the origin, using an invariant set argument.



Taking the same Lyapunov candidate as above,  $V = \frac{1}{2}x^2$ , we have

$$\dot{V} = -x^2 + x^4 + \alpha x.$$

This Lyapunov function allows us to easily verify, for instance, that  $V \leq \frac{1}{3}$  is a

*robust invariant set*, because whenever  $V = \frac{1}{3}$ , we have

$$\forall \alpha \in [\alpha_{min}, \alpha_{max}], \quad \dot{V}(x, \alpha) < 0.$$

Therefore  $V$  can never start at less than one-third and cross over to greater than one-third. To see this, see that

$$V = \frac{1}{3} \Rightarrow x = \pm \sqrt{\frac{2}{3}} \Rightarrow \dot{V} = -\frac{2}{9} \pm \alpha \sqrt{\frac{2}{3}} < 0, \forall \alpha \in \left[-\frac{1}{4}, \frac{1}{4}\right].$$

Note that not all sublevel sets of this invariant set are invariant. For instance  $V < \frac{1}{32}$  does not satisfy this condition, and by visual inspection we can see that it is in fact not robustly invariant.

### 9.1.6 Barrier functions

Lyapunov functions are used to certify stability or to establish invariance of a region. But the same conditions can be used to certify that the state of a dynamical system *will not visit* some region of state space. This can be useful for e.g. verifying that your autonomous car will not crash or, perhaps, that your walking robot will not fall down. When used this way, we call the associated functions,  $\mathcal{B}(\mathbf{x})$ , "barrier functions".

#### Theorem 9.5 - Barrier functions

Given a continuously-differentiable dynamical system described by  $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ , if I can find a function  $\mathcal{B}(\mathbf{x})$  such that  $\forall \mathbf{x}, \dot{\mathcal{B}}(\mathbf{x}) \leq 0$ , then the system will never visit states with  $\mathcal{B}(\mathbf{x}(t)) > \mathcal{B}(\mathbf{x}(0))$ .

In particular, if we ensure that  $\forall \mathbf{x}$  in "failure" regions of state space, we have  $\mathcal{B}(\mathbf{x}) > 0$ , then the level-set  $\mathcal{B}(\mathbf{x}) = 0$  can serve as a *barrier* certifying that trajectories starting with initial conditions with  $\mathcal{B}(\mathbf{x}) < 0$  will never enter the failure set.

Natural extensions exist for certifying these conditions over a region, considering piecewise-polynomial dynamics, worse-case robustness, etc.

## 9.2 LYAPUNOV ANALYSIS WITH CONVEX OPTIMIZATION

One of the primary limitations in Lyapunov analysis as I have presented it so far is that it is potentially very difficult to come up with suitable Lyapunov function candidates for interesting systems, especially for underactuated systems. ("Underactuated" is almost synonymous with "interesting" in my vocabulary.) Even if somebody were to give me a Lyapunov candidate for a general nonlinear system, the Lyapunov conditions can be difficult to check -- for instance, how would I check that  $\dot{V}$  is strictly negative for all  $\mathbf{x}$  except the origin if  $\dot{V}$  is some arbitrarily complicated nonlinear function over a vector  $\mathbf{x}$ ?

In this section, we'll look at some computational approaches to verifying the Lyapunov conditions, and even to searching for (the coefficients of) the Lyapunov functions themselves.

If you're imagining numerical algorithms to check the Lyapunov conditions for complicated Lyapunov functions and complicated dynamics, the first thought is probably that we can evaluate  $V$  and  $\dot{V}$  at a large number of sample points and

check whether  $V$  is positive and  $\dot{V}$  is negative. [This does work](#), and could potentially be combined with some smoothness or regularity assumptions to generalize beyond the sample points. But in many cases we will be able to do better -- providing optimization algorithms that will rigorously check these conditions *for all  $\mathbf{x}$*  without dense sampling; these will also give us additional leverage in formulating the search for Lyapunov functions.

### 9.2.1 Linear systems

The Lyapunov conditions we've explored above take a particularly nice form for linear systems.

#### **Theorem 9.6 - Lyapunov analysis for stable linear systems**

Imagine you have a linear system,  $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$ , and can find a Lyapunov function

$$V(\mathbf{x}) = \mathbf{x}^T \mathbf{P} \mathbf{x}, \quad \mathbf{P} = \mathbf{P}^T \succ 0,$$

which also satisfies

$$\dot{V}(\mathbf{x}) = \mathbf{x}^T \mathbf{P} \mathbf{A} \mathbf{x} + \mathbf{x}^T \mathbf{A}^T \mathbf{P} \mathbf{x} \prec 0.$$

Then the origin is globally exponentially stable.

Note that the radially-unbounded condition is satisfied by  $\mathbf{P} \succ 0$ , and that the derivative condition is equivalent to the matrix condition

$$\mathbf{P} \mathbf{A} + \mathbf{A}^T \mathbf{P} \prec 0.$$

For stable linear systems the existence of a quadratic Lyapunov function is actually a necessary (as well as sufficient) condition. Furthermore, if  $\mathbf{A}$  is stable, then a Lyapunov function can always be found by finding the positive-definite solution to the matrix Lyapunov equation

$$\mathbf{P} \mathbf{A} + \mathbf{A}^T \mathbf{P} = -\mathbf{Q}, \tag{1}$$

for any  $\mathbf{Q} = \mathbf{Q}^T \succ 0$ .

This is a very powerful result - for nonlinear systems it will be potentially difficult to find a Lyapunov function, but for linear systems it is straightforward. In fact, this result is often used to propose candidate Lyapunov functions for nonlinear systems, e.g., by linearizing the equations and solving a local linear Lyapunov function which should be valid in the vicinity of a fixed point.

Lyapunov analysis for linear systems has an extremely important connection to convex optimization. In particular, we could have also formulated the Lyapunov conditions for linear systems above using *semi-definite programming* (SDP). Semidefinite programming is a subset of convex optimization -- an extremely important class of problems for which we can produce efficient algorithms that are guaranteed find the global optima solution (up to a numerical tolerance and barring any numerical difficulties).

If you don't know much about convex optimization or want a quick refresher, please take a few minutes to read the optimization preliminaries in the appendix. The main requirement for this section is to appreciate that it is possible to formulate efficient optimization problems where the constraints include specifying that one or more matrices are positive semi-definite (PSD). These matrices must be formed from a linear combination of the decision variables. For a trivial example, the optimization

$$\min_a a, \quad \text{subject to} \begin{bmatrix} a & 0 \\ 0 & 1 \end{bmatrix} \succeq 0,$$

returns  $a = 0$  (up to numerical tolerances).

The value in this is immediate for linear systems. For example, we can formulate the search for a Lyapunov function for the linear system  $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$  by using the parameters  $\mathbf{p}$  to populate a symmetric matrix  $\mathbf{P}$  and then write the SDP:

$$\text{find}_{\mathbf{p}} \quad \text{subject to} \quad \mathbf{P} \succeq 0, \quad \mathbf{P}\mathbf{A} + \mathbf{A}^T\mathbf{P} \preceq 0. \quad (2)$$

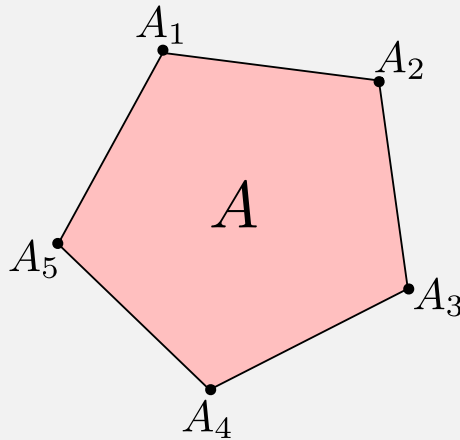
Note that you would probably never use that particular formulation, since there specialized algorithms for solving the simple Lyapunov equation which are more efficient and more numerically stable. But the SDP formulation does provide something new -- it will give us our first example of robustness analysis. In particular, we can now easily formulate the search for a "*common Lyapunov function*" for uncertain linear systems.

### Example 9.6 (Common Lyapunov analysis for linear systems)

Suppose you have a system governed by the equations  $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$ , where the matrix  $\mathbf{A}$  is unknown, but its uncertain elements can be bounded. There are a number of ways to write down this uncertainty set; let us choose to write this by describing  $\mathbf{A}$  as the convex combination of a number of known matrices,

$$\mathbf{A} = \sum_i \beta_i \mathbf{A}_i, \quad \sum_i \beta_i = 1, \quad \forall i, \beta_i > 0.$$

This is just one way to specify the uncertainty; geometrically it is describing a polygon of uncertain parameters (in the space of elements of  $\mathbf{A}$  with each  $\mathbf{A}_i$  as one of the vertices in the polygon.



Now we can formulate the search for a common Lyapunov function using

$$\text{find}_{\mathbf{p}} \quad \text{subject to} \quad \mathbf{P} \succeq 0, \quad \forall i, \mathbf{P}\mathbf{A}_i + \mathbf{A}_i^T\mathbf{P} \preceq 0.$$

The solver will then return a matrix  $\mathbf{P}$  which satisfies all of the constraints, or return saying "problem is infeasible". It can easily be verified that if  $\mathbf{P}$  satisfies the Lyapunov condition at all of the vertices, then it satisfies the condition for every  $\mathbf{A}$  in the set:

$$\mathbf{P}(\sum_i \beta_i \mathbf{A}_i) + (\sum_i \beta_i \mathbf{A}_i)^T \mathbf{P} = \sum_i \beta_i (\mathbf{P}\mathbf{A}_i + \mathbf{A}_i^T \mathbf{P}) \preceq 0,$$

since  $\forall i, \beta_i > 0$ . Note that, unlike the simple Lyapunov equation for a known linear system, this condition being satisfied is a sufficient but not a necessary condition -- it is possible that the set of uncertain matrices  $\mathbf{A}$  is robustly stable, but that this stability cannot be demonstrated with a common quadratic Lyapunov function.

You can try this example for yourself in [DRAKE](#).

As always, make sure that you open up the code and take a look.

Note that the argument above does not actually make use of the  $\sum_i \beta_i = 1$ ; that assumption can be safely removed. So really, our common Lyapunov function, when found, has certified the convex cone with the  $A_i$  matrices as the extreme rays. This is not surprising, since if we have any linear system/Lyapunov quadratic form pair  $A, P$ , then  $P$  also confirms that  $\beta A$  is stable for any  $\beta > 0$ . Multiplying a system matrix by a scalar will not change the sign of its eigenvalues.

There are many small variants of this result that are potentially of interest. For instance, a very similar set of conditions can certify "mean-square stability" for linear systems with multiplicative noise (see e.g. [3], § 9.1.1).

This example is very important because it establishes a connection between Lyapunov functions and (convex) optimization. But so far we've only demonstrated this connection for linear systems where the PSD matrices provide a magical recipe for establishing the positivity of the (quadratic) functions for all  $\mathbf{x}$ . Is there any hope of extending this type of analysis to more general nonlinear systems? Surprisingly, it turns out that there is.

## 9.2.2 Global analysis for polynomial systems

[Sums of squares \(SOS\) optimization](#) provides a natural generalization of the SDP formulation to nonlinear systems. In SDP, we were allowed to write constraints of the form,  $\mathbf{P} \succeq 0$ . In sums of squares optimization, we generalize this to writing constraints of the form " $p_\alpha(\mathbf{x})$  is SOS", where  $p(\mathbf{x})$  is a polynomial with coefficients  $\alpha$ . Let's examine this in the specific context of Lyapunov functions.

In linear systems, we searched over  $V(\mathbf{x}) = \mathbf{x}^T \mathbf{P} \mathbf{x}, \mathbf{P} \succeq 0$ . In the nonlinear generalization, we search over

$$V(\mathbf{x}) = \mathbf{m}^T(\mathbf{x}) \mathbf{P} \mathbf{m}(\mathbf{x}), \quad \mathbf{P} \succeq 0,$$

with  $\mathbf{m}(\mathbf{x})$  a set of nonlinear basis functions. This is a linear (in the parameters  $\mathbf{P}$ ) function approximator which is guaranteed to be strictly positive by virtue of being a sum of squares. While the positivity of this sums of squares is valid for any choice of  $\mathbf{m}(\mathbf{x})$ , the recipe is particularly powerful for polynomials, because for any polynomial,  $p(\mathbf{x})$ , we can confirm its positivity by creating a surrogate sums-of-squares representation, where we have strong recipes for automatically choosing good bases  $\mathbf{m}(\mathbf{x})$  given a polynomial  $p(\mathbf{x})$ , and we can constrain that  $p(\mathbf{x}) = \mathbf{m}^T(\mathbf{x}) \mathbf{P} \mathbf{m}(\mathbf{x})$  with [linear](#) constraints relating the coefficients of  $p(\mathbf{x})$  with the coefficient in  $\mathbf{P}$ .

This suggests that it may be possible to generalize the optimization approach using SDP to search for Lyapunov functions for linear systems to searching for Lyapunov functions for nonlinear systems:  $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ , where  $\mathbf{f}$  is a vector-valued [polynomial](#) function. Importantly, if  $V(\mathbf{x})$  is polynomial, and  $\mathbf{f}(\mathbf{x})$  is polynomial, then  $\dot{V}(\mathbf{x}) = \frac{\partial V}{\partial \mathbf{x}} \mathbf{f}(\mathbf{x})$  is also polynomial.

If we parameterize a [fixed-degree](#) Lyapunov candidate as a polynomial with unknown coefficients, e.g.,

$$V_\alpha(\mathbf{x}) = \alpha_0 + \alpha_1 x_1 + \alpha_2 x_2 + \alpha_3 x_1 x_2 + \alpha_4 x_1^2 + \dots,$$

then we can formulate the optimization:

$$\begin{aligned} \text{find}_{\alpha}, \quad \text{subject to} \quad & V_\alpha(\mathbf{x}) \text{ is SOS, } V_\alpha(0) = 0 \\ & -\dot{V}_\alpha(\mathbf{x}) = -\frac{\partial V_\alpha}{\partial \mathbf{x}} f(\mathbf{x}) \text{ is SOS.} \end{aligned}$$

Because this is a convex optimization, the solver will return a solution if one exists, and this solution would certify stability i.s.L. of the origin. Note that I don't actually need to include the constraint  $\dot{V}_\alpha(0) = 0$ ; if  $V_\alpha$  is a smooth function, is zero at zero, and is positive for all  $\mathbf{x}$ , then this already implies that  $\frac{\partial V_\alpha}{\partial \mathbf{x}} = 0$ .

If we wish to demonstrate (global) asymptotic stability, then we can write the constraint  $\dot{V}_\alpha(\mathbf{x}) \prec 0$  as

$$-\dot{V}_\alpha(\mathbf{x}) - \epsilon \mathbf{x}^T \mathbf{x} \text{ is SOS,}$$

where  $\epsilon$  is a small positive constant that need only be larger than the numerical tolerances of the solver. The conditions for exponential stability fit easily, too.

### Example 9.7 (Verifying a Lyapunov candidate via SOS)

This example is example 7.2 from [4]. Consider the nonlinear system:

$$\begin{aligned} \dot{x}_0 &= -x_0 - 2x_1^2 \\ \dot{x}_1 &= -x_1 - x_0 x_1 - 2x_1^3, \end{aligned}$$

and the *fixed* Lyapunov function  $V(x) = x_0^2 + 2x_1^2$ , test if  $\dot{V}(x)$  is negative definite.

The numerical solution can be written in a few lines of code, and is a convex optimization.

### Example 9.8 (Searching for a Lyapunov function via SOS)

Verifying a candidate Lyapunov function is all well and good, but the real excitement starts when we use optimization to *find* the Lyapunov function. In the following code, we parameterize  $V(x)$  as the polynomial containing all monomials up to degree 2, with the coefficients as decision variables:

$$V(x) = c_0 + c_1 x_0 + c_2 x_1 + c_3 x_0^2 + c_4 x_0 x_1 + c_5 x_1^2.$$

We will set the scaling (arbitrarily) to avoid numerical issues by setting  $V(0) = 0$ ,  $V([1, 0]) = 1$ . Then we write:

$$\begin{aligned} \text{find}_{\mathbf{c}} \quad \text{subject to} \quad & V \text{ is sos,} \\ & -\dot{V} \text{ is sos.} \end{aligned}$$

Up to numerical convergence tolerance, it discovers the same coefficients that we chose above (zeroing out the unnecessary terms).

### Example 9.9 (Cart-pole swingup (again))

In the [cart-pole swingup example](#), we took

$$V(\mathbf{x}) = \frac{1}{2} \tilde{E}^2(\mathbf{x}) = \frac{1}{2} \left( \frac{1}{2} \dot{\theta}^2 - \cos \theta - 1 \right)^2.$$

This is clearly a sum of squares. Furthermore, we showed that

$$\dot{V}(\mathbf{x}) = -\dot{\theta}^2 \cos^2 \theta \tilde{E}^2(\mathbf{x}),$$

which is also a sum of squares. So the proposal of using sums-of-squares optimization is not so different, actually, than the recipes that nonlinear control theorists have been using (on pen and paper) for years.

The SOS approach, as presented so far, could take only the nonlinear basis,  $\mathbf{m}(\mathbf{x})$  for the candidate Lyapunov function, the degree of that Lyapunov function, and the closed-loop nonlinear dynamics; it would recover the parameters of a Lyapunov function and a certificate demonstrating the  $V(\mathbf{x})$  and  $-\dot{V}(\mathbf{x})$  are SOS. In this case, I would need a basis vector that includes many more monomials:  $[1, \dot{\theta}, \cos \theta, \dots, \dot{\theta}^3, \dots]^T$ .

You might have noticed that I did something sneaky in here. The dynamics of the cart-pole are *not* simply polynomial in  $\mathbf{x}$ ; there are trigonometric terms in there. Setting up the equality constraints in an optimization requires more complicated term matching when trigonometric functions are involved, but the recipe still works. We will discuss this in more detail below.

It is important to remember that there are a handful of gaps which make the existence of this solution a sufficient condition (to guarantee that  $V(\mathbf{x})$  is a Lyapunov function for  $\mathbf{f}(\mathbf{x})$ ) instead of a necessary one. First, there is no guarantee that a stable polynomial system can be verified using a polynomial Lyapunov function (of any degree), and in fact there are known counter-examples [5]; here we are only searching over a fixed-degree polynomial. Second, even if a polynomial Lyapunov function does exist, there is a gap between the SOS polynomials and the positive polynomials [6, §3.1].

Despite these caveats, I have found this formulation to be surprisingly effective in practice. Intuitively, I think that this is because there is relatively a lot of flexibility in the Lyapunov conditions -- if you can find one function which is a Lyapunov function for the system, then there are also many "nearby" functions which will satisfy the same constraints. This is the fundamental benefit of [changing the equality in the HJB into an inequality in Lyapunov analysis](#).

### 9.2.3 Region of attraction estimation for polynomial systems

Now we have arrived at the tool that I believe can be a work-horse for many serious robotics applications. Most of our robots are not actually globally stable. That's not because they are robots -- if you push me hard enough, I will fall down, too! Asking for global stability is too much, but we can instead certify that our closed-loop systems behave well over a sufficiently large region of state space (and potentially for a range of possible parameters or disturbances). One of the simplest versions of this is estimating the region of attraction to a fixed point.

Sums-of-squares optimization effectively gives us an oracle which we can ask: is this polynomial positive for all  $\mathbf{x}$ ? To use this for regional analysis, we have to figure out how to modify our questions to the oracle so that the oracle will say "yes" or "no" when we ask if a function is positive over a certain region which is a subset of  $\mathbb{R}^n$ . That trick is called the S-procedure. It is closely related to the Lagrange multipliers from constrained optimization, and has deep connections to "Positivstellensatz" from algebraic geometry.

## The S-procedure

Consider a scalar polynomial,  $p(\mathbf{x})$ , and a **semi-algebraic set**  $g(\mathbf{x}) \leq 0$ . The term "semi-algebraic set" just means the set is defined by polynomial inequalities (here  $g(\mathbf{x}) \leq 0$ ). If I can find a polynomial "multiplier",  $\lambda(\mathbf{x})$ , such that

$$p(\mathbf{x}) + \lambda^T(\mathbf{x})g(\mathbf{x}) \text{ is SOS, and } \lambda(\mathbf{x}) \text{ is SOS,}$$

then this is sufficient to demonstrate that

$$p(\mathbf{x}) \geq 0 \quad \forall \mathbf{x} \in \{\mathbf{x} \mid g(\mathbf{x}) \leq 0\}.$$

To convince yourself, observe that when  $g(\mathbf{x}) \leq 0$ , it is only harder to be positive, but when  $g(\mathbf{x}) > 0$ , it is possible for the combined function to be SOS even if  $p(\mathbf{x})$  is negative. We will sometimes find it convenient to use the short-hand:

$$g(\mathbf{x}) \leq 0 \Rightarrow p(\mathbf{x}) \geq 0$$

to denote the implication certified by the S-procedure, e.g. whenever  $g(\mathbf{x}) \leq 0$ , we have  $p(\mathbf{x}) \geq 0$ .

We can also handle equality constraints with only a minor modification -- we no longer require the multiplier to be positive. If I can find a polynomial "multiplier",  $\lambda(\mathbf{x})$ , such that

$$p(\mathbf{x}) + \lambda^T(\mathbf{x})g(\mathbf{x}) \text{ is SOS}$$

then this is sufficient to demonstrate that

$$p(\mathbf{x}) \geq 0 \quad \forall \mathbf{x} \in \{\mathbf{x} \mid g(\mathbf{x}) = 0\}.$$

Note the difference: for  $g(\mathbf{x}) \leq 0$  we needed  $\lambda(\mathbf{x}) \geq 0$ , but for  $g(\mathbf{x}) = 0$ ,  $\lambda(\mathbf{x})$  can be arbitrary. The intuition is that  $\lambda(\mathbf{x})$  can add arbitrary positive terms to help me be SOS, but those terms contribute nothing precisely when  $g(\mathbf{x}) = 0$ .

## Basic region of attraction formulation

The S-procedure gives us the tool we need to evaluate positivity only over a region of state space, which is precisely what we need to certify the Lyapunov conditions for a region-of-attraction analysis. Let us start with a positive-definite polynomial Lyapunov candidate,  $V(\mathbf{x}) > 0$ , then we can write the Lyapunov conditions:

$$\dot{V}(\mathbf{x}) < 0 \quad \forall \mathbf{x} \in \{\mathbf{x} \mid V(\mathbf{x}) \leq \rho\},$$

using sums-of-squares and the S-procedure:

$$-\dot{V}(\mathbf{x}) + \lambda(\mathbf{x})(V(\mathbf{x}) - \rho) \text{ is SOS, and } \lambda(\mathbf{x}) \text{ is SOS,}$$

where  $\lambda(\mathbf{x})$  is a multiplier polynomial with free coefficients that are to be solved for in the optimization. Note that we've carefully chosen to denote the region of interest using a sub-level set of  $V(\mathbf{x})$ , so that we can be sure that region is also an invariant set (one of the requirements for the region of attraction analysis described above).

I think it's easiest to see the details in an example.

### **Example 9.10 (Region of attraction for the one-dimensional cubic system)**

Let's return to our example from above:

$$\dot{x} = -x + x^3$$



and try to use SOS optimization to demonstrate that the region of attraction of the fixed point at the origin is  $x \in (-1, 1)$ , using the Lyapunov candidate  $V(x) = x^2$ .

First, define the multiplier polynomial,

$$\lambda(x) = c_0 + c_1x + c_2x^2.$$

Then define the optimization

$$\begin{aligned} & \text{find}_{\mathbf{c}} \\ & \text{subject to} \quad -\dot{V}(x) + \lambda(x)(V(x) - 1) \text{ is SOS} \\ & \quad \lambda(x) \text{ is SOS} \end{aligned}$$

You can try these examples for yourself in [DRAKE](#).

In this example, we only verified that the one-sublevel set of the pre-specified Lyapunov candidate is negative (certifying the ROA that we already understood). More generally, we can replace the 1-sublevel set with any constant value  $\rho$ . It is tempting to try to make  $\rho$  a decision variable that we could optimize, but this would make the optimization problem non-convex because we would have terms like  $\rho c_0$ ,  $\rho c_1x$ , which are bilinear in the decision variables. We need the sums-of-squares constraints to be only linear in the decision variables.

Fortunately, because the problem is convex with  $\rho$  fixed (and therefore can be solved reliably), and  $\rho$  is just a scalar, we can perform a simple [line search](#) on  $\rho$  to find the largest value for which the convex optimization returns a feasible solution. This will be our estimate for the region of attraction.

There are a number of variations to this basic formulation; I will describe a few of them below. There are also important ideas like rescaling and degree-matching that can have a dramatic effect on the numerics of the problem, and potentially make them much better for the solvers. But you do not need to master them all in order to use the tools effectively.

## The equality-constrained formulation

Here is one important variation for finding the level set of a candidate region of attraction, which turns an inequality in the S-procedure into an equality. This formulation [is](#) jointly convex in  $\lambda(\mathbf{x})$  and  $\rho$ , so one can optimize them in a single convex optimization. It appeared informally in an example in [\[4\]](#), and was discussed a bit more in [\[7\]](#).

Under the assumption that the Hessian of  $\dot{V}(\mathbf{x})$  is negative-definite at the origin (which is easily checked), we can write

$$\begin{aligned} & \max_{\rho, \lambda(\mathbf{x})} \quad \rho \\ & \text{subject to} \quad (\mathbf{x}^T \mathbf{x})^d (V(\mathbf{x}) - \rho) + \lambda^T(\mathbf{x}) \dot{V}(\mathbf{x}) \text{ is SOS,} \end{aligned}$$

with  $d$  a fixed positive integer. As you can see,  $\rho$  no longer multiplies the coefficient of  $\lambda(\mathbf{x})$ . But why does this certify a region of attraction?

You can read the sums-of-squares constraint as certifying the implication that whenever  $\dot{V}(x) = 0$ , we have that either  $V(\mathbf{x}) \geq \rho$  OR  $\mathbf{x} = 0$ . Multiplying by some multiple of  $\mathbf{x}^T \mathbf{x}$  is just a clever way to handle the "or  $\mathbf{x} = 0$ " case, which is necessary since we expect  $\dot{V}(0) = 0$ . This implication is sufficient to prove that  $\dot{V}(\mathbf{x}) \leq 0$  whenever  $V(\mathbf{x}) \leq \rho$ , since  $V$  and  $\dot{V}$  are smooth polynomials; we examine this claim in full detail in [one of the exercises](#).

Using the S-procedure with equalities instead of inequalities has the immediate

advantage of removing the SOS constraint on  $\lambda(\mathbf{x})$ . But there is a more subtle, and likely more important advantage -- we have more mathematical tools for dealing with algebraic varieties (defined by equality constraints) than for the semi-algebraic sets (defined by inequality constraints). For an important example of how this can be used to dramatically simplify the SOS programs, see for instance [7] which uses sampling on the variety to provide an exact certification (for all  $\mathbf{x}$ ) on the quotient ring, completely removing the need for the Lagrange multiplier terms.

### Example 9.11 (Maximizing the ROA for the cubic polynomial system)

Let's return again to our example from above:

$$\dot{x} = -x + x^3$$

and try to use SOS optimization, this time to maximize the sub-level set of our Lyapunov candidate  $V(x) = x^2$ , in order to certify the largest ROA that we can using these tools.

Again, we define the multiplier polynomial,  $\lambda(x)$ , but this time we won't require it to be sums-of-squares (here it's a "free" polynomial). Then define the optimization

$$\begin{aligned} & \max_{\rho} \\ & \text{subject to } x^2(V(x) - \rho) - \lambda(x)\dot{V}(x) \text{ is SOS.} \end{aligned}$$

What degree should we choose for  $\lambda(x)$  here? Note that  $V(x)$  has degree 2, so  $x^2V(x)$  has degree 4, and  $\dot{V}(x)$  also has degree 4, so it's possible that even a zero-degree  $\lambda(x)$  (just a constant) can be sufficient here. Our numerical experiments confirm that this is indeed the case:

### Searching for $V(\mathbf{x})$

The machinery so far has used optimization to find the largest region of attraction that can be certified *given a candidate Lyapunov function*. This is not necessarily a bad assumption. For most stable fixed points, we can certify the local stability with a linear analysis, and this linear analysis gives us a candidate quadratic Lyapunov function that can be used for nonlinear analysis over a region. Practically speaking, when we solve an LQR problem, the cost-to-go from LQR is a good candidate Lyapunov function for the nonlinear system. If we are simply analyzing an existing system, then we can obtain this candidate by solving a Lyapunov equation (Eq 1).

Recall that there are cases when the stability of the linearization is not informative about the stability of the nonlinear system (if the linearization has only non-negative eigenvalues instead of strictly negative). Can we use our nonlinear tools to certify a region of attraction for these systems, even when the linearization doesn't yield a candidate Lyapunov function? Or perhaps we can certify a larger volume of state space by optimizing our original Lyapunov candidate to maximize the volume of the verified region for the nonlinear system, potentially even considering Lyapunov functions with degree greater than two (the linear analysis will only give quadratic candidates)? Yes. We can use sums-of-squares optimization in those cases, too!

To accomplish this, we'll combine the ideas from our global stability analysis (where we could search for  $V(\mathbf{x})$ ) with these new tools for regional analysis (where so far we had to fix  $V(\mathbf{x})$ ). Make  $V(\mathbf{x})$  a polynomial (of some fixed degree) with the coefficients as decision variables. First we will need to add constraints to ensure that

$$V(\mathbf{0}) = 0 \quad \text{and} \quad V(\mathbf{x}) - \epsilon \mathbf{x}^T \mathbf{x} \text{ is SOS,}$$

where  $\epsilon$  is some small positive constant. This  $\epsilon$  term simply ensures that  $V$  is strictly positive definite. Now let's consider our basic formulation:

$$-\dot{V}(\mathbf{x}) + \lambda(\mathbf{x})(V(\mathbf{x}) - 1) \text{ is SOS, and } \lambda(\mathbf{x}) \text{ is SOS.}$$

Notice that I've substituted  $\rho = 1$ ; now that we are searching for  $V$  the scale of the level-set can be set arbitrarily to 1. In fact, it's better to do so -- if we did not set  $V(0) = 0$  and  $V(\mathbf{x}) \leq 1$  as the sublevel set, then the optimization problem would be under constrained.

Unfortunately, the derivative constraint is now nonconvex (bilinear) in the decision variables, since we are searching for both the coefficients of  $\lambda$  and  $V$ , and they are multiplied together. The equality-constrained formulation has the same bilinearity, since the coefficients of  $V$  also appear in  $\dot{V}$ . Here we have to resort to some weaker form of optimization. In practice, we have had good practical success (see, for instance [8, 9]) using bilinear alternations:

1. Fix  $V(\mathbf{x})$  and solve for  $\rho$  and  $\lambda(\mathbf{x})$ :

$$\begin{aligned} & \max_{\rho, \lambda(\mathbf{x})} \quad \rho \\ & \text{subject to} \quad (\mathbf{x}^T \mathbf{x})^d (V(\mathbf{x}) - \rho) + \lambda^T(\mathbf{x}) \dot{V}(\mathbf{x}) \text{ is SOS,} \end{aligned}$$

with  $d$  a fixed, positive integer. Update  $V(\mathbf{x}) \leftarrow \frac{V(\mathbf{x})}{\rho}$ .

2. Fix  $\lambda(\mathbf{x})$  and solve for  $V(\mathbf{x}) = \mathbf{x}^T \mathbf{P} \mathbf{x}$ :

$$\begin{aligned} & \min_{\mathbf{P}} \quad \text{tr}(\mathbf{P}) \\ & \text{subject to} \quad (\mathbf{x}^T \mathbf{x})^d (V(\mathbf{x}) - 1) + \lambda^T(\mathbf{x}) \dot{V}(\mathbf{x}) \text{ is SOS,} \\ & \quad \quad \quad \mathbf{P} \succeq 0. \end{aligned}$$

with  $d$  the same fixed, positive integer.

These alternations require a feasible initial  $V(\mathbf{x})$ , which we typically get from solving the quadratic Lyapunov equation or LQR. We often prefer to do those optimizations in the equality-constrained formulation, because we have stronger tools for dealing with that equality constraint [7], but the basic formulation with line search can be used, too, and in some problems can be numerically more robust.

Where did the  $\text{tr}(\mathbf{P})$  objective come from? A typical choice for the objective of the alternations is to maximize the volume of the certified region -- for quadratic Lyapunov functions described in our standard quadratic form,  $\mathbf{x}^T \mathbf{P} \mathbf{x} \leq 1$ , this volume is proportional to  $\det(\mathbf{P}^{-\frac{1}{2}})$ . Minimizing this volume is a convex optimization[10, §8.4.1], but maximizing the volume is, in general, not. We tend to use a heuristic like minimizing  $\text{tr}(\hat{\mathbf{P}}^{-1} \mathbf{P})$  as a surrogate for the true objective [11, §6.3.2], where  $\hat{\mathbf{P}}$  is the  $\mathbf{P}$  from a previous alternation of the algorithm (or the identity matrix that I've written above), which is a linearization of the true objective[12]. For higher-degree Lyapunov functions, typically we (approximately) maximize the volume of a contained ellipse,

$$\mathbf{x}^T \mathbf{P} \mathbf{x} \leq 1 \Rightarrow V(\mathbf{x}) \leq 1,$$

as a surrogate for the true volume.

Barring numerical issues, this algorithm is guaranteed to have recursive feasibility and tends to converge in just a few alternations, but there is no guarantee that it finds the optimal solution.

### Example 9.12 (Searching for Lyapunov functions)

### Example 9.13 (Estimated regions of attraction need not be convex regions (in state space))

Although we are using convex optimization to find regions of attraction, this does not imply that the regions we are certifying need to be convex. To demonstrate that, let's make a system with a known, non-convex region of attraction. We'll do this by taking some interesting potential function,  $U(\mathbf{x})$  is SOS, and setting the dynamics to be

$$\dot{\mathbf{x}} = (U(\mathbf{x}) - 1) \frac{\partial U^T}{\partial \mathbf{x}},$$

which has  $U(x) \leq 1$  as the region of attraction.

Slightly more general is to write

$$\dot{\mathbf{x}} = (U(\mathbf{x}) - 1) \mathbf{R}(\theta) \frac{\partial U^T}{\partial \mathbf{x}},$$

where  $\mathbf{R}(\theta) = \begin{bmatrix} \cos \theta & \sin \theta \\ -\sin \theta & \cos \theta \end{bmatrix}$  is the 2D rotation matrix, and  $\theta < \pi$  is a constant parameter (not a decision variable nor indeterminate), which still has the same region of attraction.

Searching for Lyapunov functions of only degree 2 (quadratics) will, of course, limit our certified regions to convex regions. But if we increase the degree of the Lyapunov function to degree 4, then we can tighten our inner approximation, and indeed certify non-convex regions. While we can use the Lyapunov method for linear systems to initialize quadratic Lyapunov functions, the ability to search for the parameters of the Lyapunov functions really shines when the Lyapunov candidates are higher degree (such as the quartic we use here).

## Convex outer approximations

All of our region of attraction approximations so far have been "inner approximations" -- the certified region is guaranteed to be contained within the true region of attraction for the system. This is what we want if we are to give some guarantee of stability. As we increase the degree of our polynomial and multipliers, we expect that the estimated regions of attraction will converge on the true regions.

It turns out that if we instead consider outer approximations, which converge on the true ROA from the other side, we can write formulations that enable searching for the Lyapunov function directly as a convex optimization. As we will see below, this approach also allows one to write convex formulations for controller design. These methods have been explored in a very nice line of work by Henrion and Korda (e.g. [13]), using the method of moments from [14] also called "occupation measures". Their treatment emphasizes infinite-dimensional linear programs and hierarchies of LMI relaxations; these are just dual formulations to the SOS optimizations that we've been writing here. I find the SOS form more clear, so will stick to it here. Some of the occupation measure papers include the dual formulation of the infinite-dimension linear program which will look quite similar; my coauthors and I typically call out the sums-of-squares version in our papers on the topic (e.g. [15]).

To find an outer approximation, instead of solving for a Lyapunov function that certifies convergence to the origin, we instead search for a Lyapunov-like "**barrier certificate**",  $\mathcal{B}(\mathbf{x})$ . Like a Lyapunov function, we'd like  $\dot{\mathcal{B}}(\mathbf{x}) \leq 0$ ; this time we'll ask for this to be true everywhere (or at least in some set that is sufficiently larger than the ROA of interest). Then we will set  $\mathcal{B}(0) = 1 > 0$ . If we can find such a function, then

certainly any state for which  $\mathcal{B}(\mathbf{x}) < 0$  is outside of the region of attraction of the fixed point -- since  $\mathcal{B}$  cannot increase it can never reach the origin which has  $\mathcal{B} > 0$ . The *super*level set  $\{\mathbf{x} | \mathcal{B}(\mathbf{x}) \geq 0\}$  is an outer approximation of the true region of attraction:

$$-\dot{\mathcal{B}}(\mathbf{x}) \text{ is SOS} \quad \text{and} \quad \mathcal{B}(0) = 1.$$

In order to find the smallest such outer approximation (given a fixed degree polynomial), we choose an objective that tries to "push-down" on the value of  $\mathcal{B}(\mathbf{x})$ . We typically accomplish this by introducing another polynomial function  $W(\mathbf{x})$  with the requirements that

$$W(\mathbf{x}) \geq 0 \quad \text{and} \quad W(\mathbf{x}) \geq \mathcal{B}(\mathbf{x}) + 1,$$

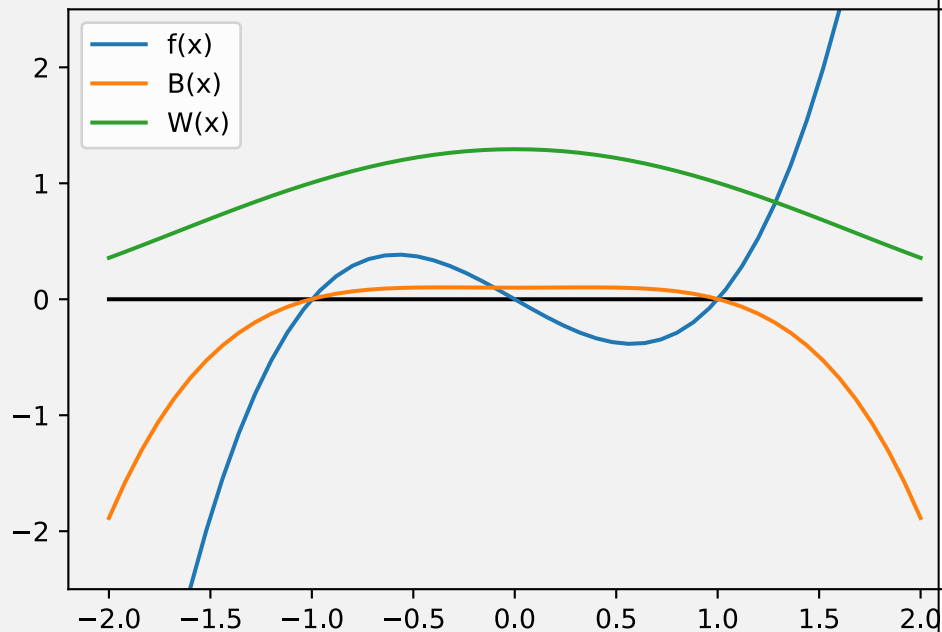
implemented as SOS constraints. Then we minimize the integral  $\int_{\mathbf{x}} W(\mathbf{x}) d\mathbf{x}$ , which can be readily computed over a compact set like a ball in  $\mathbb{R}^n$  [16] and is linear in the coefficients. More sophisticated alternatives also exist[17]. [18] writes the objective as minimizing the volume of the covering ellipse, which is convex[10, §8.4.1].

### Example 9.14 (Outer approximation for the cubic polynomial)

Let's revisit my favorite scalar dynamical system,  $\dot{x} = -x + x^3$ , which has a region of attraction for the fixed point at zero of  $\{x | |x| < 1\}$ . This time, we'll estimate the ROA using the outer approximation. We can accomplish this with the following program:

$$\begin{aligned} \min_{\mathcal{B}(x), W(x)} \quad & \int_{-2}^2 W(x) dx, \\ \text{subject to} \quad & -\dot{\mathcal{B}}(x) \quad \quad \quad \text{is SOS,} \\ & W(x) \quad \quad \quad \text{is SOS,} \\ & W(x) - \mathcal{B}(x) - 1.0 \quad \text{is SOS,} \\ & \mathcal{B}(0) = 0.1. \end{aligned}$$

To make the problem a little numerically better, you'll see in the code that I've asked for  $\dot{\mathcal{B}}(x)$  to be strictly negative definite, and I've chosen to only include even-degree monomials in  $\mathcal{B}(x)$  and  $W(x)$ . Plotting the solution reveals:



As you can see, the superlevel set,  $B(x) \geq 0$  is a tight outer-approximation of the true region of attraction. In the limit of increasing the degrees of the polynomials to infinity, we would expect that  $W(\mathbf{x})$  would converge to the indicator function that is one inside the region of attraction, and zero outside (we are quite far from that here, but nevertheless have a tight approximation from  $B(\mathbf{x}) \geq 0$ ).

Go ahead and try adjusting some of these decisions. For example, if you set  $B(0) = 1$ , then you can increase the degree of  $B(x)$  to get tighter and tighter approximations.

## Regions of attraction codes in Drake

In **DRAKE**, we have packaged most of the work in setting up and solving the sums-of-squares optimization for regions of attraction into a single method `RegionOfAttraction(system, context, options)`, which aims to implement the best methods described in this chapter.

### Example 9.15 (RegionOfAttraction for the cubic polynomial)

Using the `RegionOfAttraction` method, many of the notebooks above can be reduced to just a few lines:

```
x = Variable("x")
sys = SymbolicVectorSystem(state=[x], dynamics=[-x+x**3])
context = sys.CreateDefaultContext()
V = RegionOfAttraction(sys, context)
```

Remember that although we have tried to make it convenient to call these

functions, they are not a black box. I highly recommend opening up the code implementing the `RegionOfAttraction` method and understanding how it works (even if you don't love C++, the `MathematicalProgram` modeling language in Drake makes these formulations pretty readable). There are lots of different options / formulations, and numerous numerical recipes to improve the numerics of the optimization problem.

In order to develop our intuition for these tools, it is helpful to understand how "tight" the inner approximation can be for each choice of the fixed degree of the Lyapunov function / Lagrange multipliers. I've included below a few of my favorite examples of nonlinear systems which use some clever construction to have *known* regions of attraction.

### **Example 9.16 (Region of attraction for the time-reversed Van der Pol oscillator)**

We will study the Van der Pol oscillator in more detail when we study limit cycles. For now, it suffices to know that the dynamics are polynomial, and that the stable limit cycle of the system when simulated forward in time produces a known (via numerical integration) boundary for the region of attraction of the fixed point at the origin when we simulate the system *backwards* in time.

I've provided an example here for solving this problem with the `RegionOfAttraction` in Drake, but we will also work you through writing the sums-of-squares problem yourself in the [exercises](#).

### **Example 9.17 (Regions of attraction and LQR)**

So far we have only considered verifying regions of attraction for systems of the form  $\dot{\mathbf{x}} = f(\mathbf{x})$  (e.g. with no control input); the implication has been that one should design a feedback controller first and then analyze the stability of the closed-loop dynamics. But some control design techniques produce artifacts that can be useful for this verification. Often jointly designing a controller and certificate can be more tractable than trying to certify an arbitrary controller.

One nice example of this that we've already seen is the linear quadratic regulator. Even when it is applied to (the linearization of) nonlinear systems, it still produces a quadratic cost-to-go function  $\bar{\mathbf{x}}^T \mathbf{S} \bar{\mathbf{x}}$  which can be immediately used as a candidate Lyapunov function for the nonlinear system.

To demonstrate this...

## **9.2.4 Robustness analysis using the S-procedure**

## **9.2.5 Piecewise-polynomial systems**

### **Example 9.18 (Closed-loop dynamics with input saturations)**

## **9.2.6 Rigid-body dynamics are (rational) polynomial**

We've been talking a lot in this chapter about numerical methods for polynomial

systems. But even our simple pendulum has a  $\sin \theta$  in the dynamics. Have I been wasting your time? Must we just resort to polynomial approximations of the non-polynomial equations? It turns out that our polynomial tools can perform exact analysis of the manipulation equation for almost all<sup>†</sup> of our robots. We just have to do a little more work to reveal that structure.

<sup>†</sup>the most notable exception to this is if your robot has helical screw joints (see [19])

Let us first observe that rigid-body kinematics are polynomial (except the helical joint). This is fundamental -- the very nature of a "rigid body" assumption is that Euclidean distance is preserved between points on the body; if  $\mathbf{p}_1$  and  $\mathbf{p}_2$  are two points on a body, then the kinematics enforce that  $\|\mathbf{p}_1 - \mathbf{p}_2\|_2^2$  is constant -- these are polynomial constraints. Of course, we commonly write the kinematics in a minimal coordinates using  $\sin \theta$  and  $\cos \theta$ . But because of rigid body assumption, these terms only appear in the simplest forms, and we can simply make new variables  $s_i = \sin \theta_i$ ,  $c_i = \cos \theta_i$ , and add the constraint that  $s_i^2 + c_i^2 = 1$ . For a more thorough discussion see, for instance, [19] and [20]. Since the potential energy of a multi-body system is simply an accumulation of weight times the vertical position for all of the points on the body, the potential energy is polynomial.

If configurations (positions) of our robots can be described by polynomials, then velocities can as well: forward kinematics  $\mathbf{p}_i = f(\mathbf{q})$  implies that  $\dot{\mathbf{p}}_i = \frac{\partial f}{\partial \mathbf{q}} \dot{\mathbf{q}}$ , which is polynomial in  $s, c, \dot{\theta}$ . Since the kinetic energy of our robot is given by the accumulation of the kinetic energy of all the mass,  $T = \sum_i \frac{1}{2} m_i v_i^T v_i$ , the kinetic energy is polynomial, too (even when we write it with inertial matrices and angular velocities).

Finally, the [equations of motion](#) can be obtained by taking derivatives of the Lagrangian (kinetic minus potential). These derivatives are still polynomial!

### Example 9.19 (Global stability of the simple pendulum via SOS)

We opened this chapter using our intuition about energy to discuss stability on the simple pendulum. Now we'll replace that intuition with convex optimization (because it will also work for more difficult systems where our intuition fails).

Let's change coordinates from  $[\theta, \dot{\theta}]^T$  to  $\mathbf{x} = [s, c, \dot{\theta}]^T$ , where  $s \equiv \sin \theta$  and  $c \equiv \cos \theta$ . Then we can write the pendulum dynamics as

$$\dot{\mathbf{x}} = \begin{bmatrix} c\dot{\theta} \\ -s\dot{\theta} \\ -\frac{1}{ml^2}(b\dot{\theta} + mgl s) \end{bmatrix}.$$

Now let's parameterize a Lyapunov candidate  $V(s, c, \dot{\theta})$  as the polynomial with unknown coefficients which contains all monomials up to degree 2:

$$V = \alpha_0 + \alpha_1 s + \alpha_2 c + \dots + \alpha_9 s^2 + \alpha_{10} s c + \alpha_{11} s \dot{\theta}.$$

Now we'll formulate the feasibility problem:

$$\text{find}_{\alpha} \quad \text{subject to} \quad V \text{ is SOS}, \quad -\dot{V} \text{ is SOS}.$$

In fact, this is asking too much -- really  $\dot{V}$  only needs to be negative when  $s^2 + c^2 = 1$ . We can accomplish this with the S-procedure, and instead write

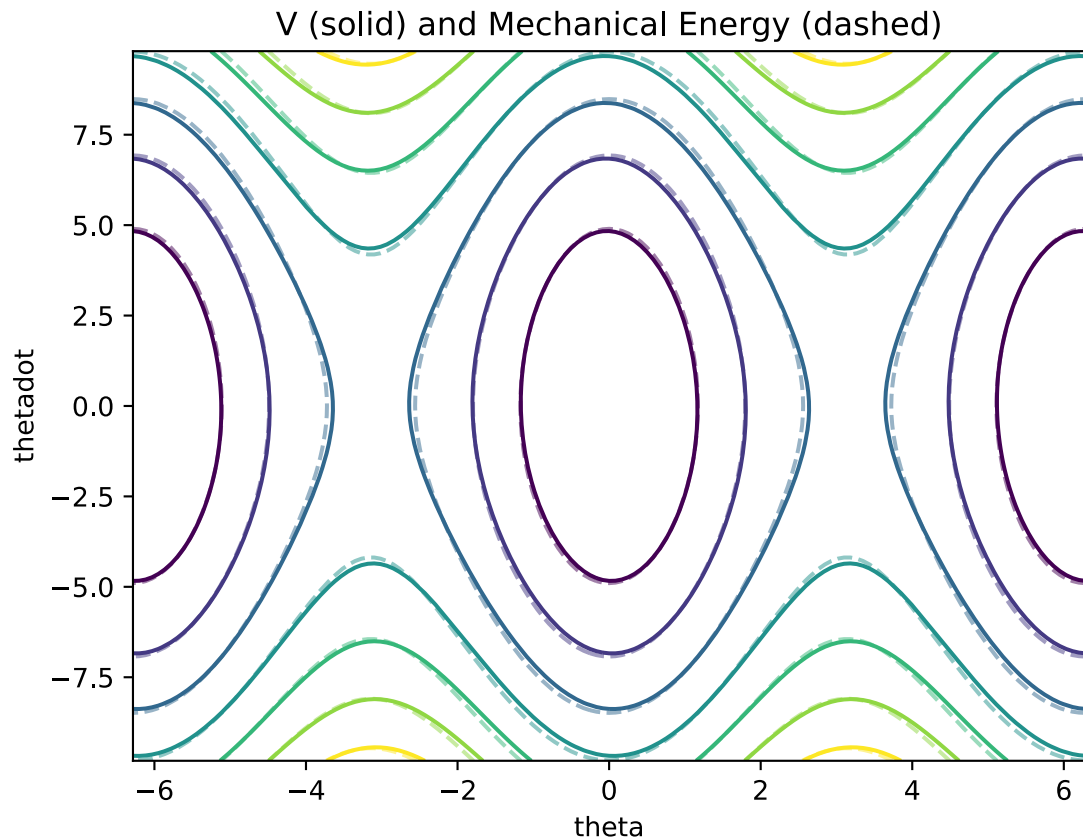
$$\text{find}_{\alpha, \lambda} \quad \text{subject to} \quad V \text{ is SOS}, \quad -\dot{V} - \lambda(\mathbf{x})(s^2 + c^2 - 1) \text{ is SOS}.$$

(Recall that  $\lambda(\mathbf{x})$  is another polynomial with free coefficients which the optimization can use to make terms arbitrarily more positive when  $s^2 + c^2 \neq 1$ .)



Finally, for style points, in the code example in [DRAKE](#) we ask for exponential stability:

As always, make sure that you open up the code and take a look. The result is a Lyapunov function that looks familiar (visualized as a contour plot here):



Aha! Not only does the optimization find us coefficients for the Lyapunov function which satisfy our Lyapunov conditions, but the result looks a lot like mechanical energy. In fact, the result is a little better than energy... there are some small extra terms added which prove exponential stability without having to invoke LaSalle's Theorem.

The one-degree-of-freedom pendulum did allow us to gloss over one important detail: while the manipulator equations  $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \dots$  are polynomial, in order to solve for  $\ddot{\mathbf{q}}$  we actually have to multiply both sides by  $\mathbf{M}^{-1}$ . This, unfortunately, is *not* a polynomial operation, so in fact the final dynamics of the multibody systems are *rational* polynomial. Not only that, but evaluating  $\mathbf{M}^{-1}$  symbolically is not advised -- the equations get very complicated very fast. But we can actually write the Lyapunov conditions using the dynamics in implicit form, e.g. by writing  $V(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})$  and asking it to satisfy the Lyapunov conditions everywhere that  $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \dots = \dots + \mathbf{B}\mathbf{u}$  is satisfied, using the S-procedure.

## Linear feedback and quadratic forms

There are a few things that *do* break this clean polynomial view of the world. Rotary springs, for instance, if modeled as  $\tau = k(\theta_0 - \theta)$  will mean that  $\theta$  appears alongside  $\sin \theta$  and  $\cos \theta$ , and the relationship between  $\theta$  and  $\sin \theta$  is sadly *not polynomial*. As we

have presented it so far, even linear feedback from LQR actually looks like a linear spring.

Looking more closely, though, there is a natural alternative. Consider the mechanical energy of the simple pendulum,  $E = \frac{1}{2}ml^2\dot{\theta}^2 + mgl(1 - \cos \theta)$ , where we've taken the potential energy so that  $E \geq 0$ . Observe that using half-angle formulas we can write this as a quadratic form:

$$E = \frac{1}{2} \begin{bmatrix} 2 \sin(\frac{\theta}{2}) \\ \dot{\theta} \end{bmatrix}^T \begin{bmatrix} mgl & 0 \\ 0 & ml^2 \end{bmatrix} \begin{bmatrix} 2 \sin(\frac{\theta}{2}) \\ \dot{\theta} \end{bmatrix}.$$

The term  $2 \sin(\frac{\theta}{2})$  has slope one at  $\theta = 0$ . This isn't unique to the pendulum,  $2 \sin^2(\frac{\theta}{2})$  is a natural choice for a Lyapunov candidate in the trigonometric coordinates that is periodic in  $2\pi$ . In the case of LQR applied to the linearization of a nonlinear mechanical system, one could replace  $-k\theta$  with  $-2k \sin(\frac{\theta}{2})$ ; after all their linearization is the same.

**Example 9.20 (Regions of attraction for LQR of trigonometric systems)**

### Alternatives for obtaining polynomial equations

In practice, you can also Taylor approximate any smooth nonlinear function using polynomials. This can be an effective strategy in practice, because you can limit the degrees of the polynomial, and because in many cases it is possible to provide conservative bounds on the errors due to the approximation.

One final technique that is worth knowing about is a change of coordinates, often referred to as the stereographic projection, that provides a coordinate system in which we can replace sin and cos with polynomials:

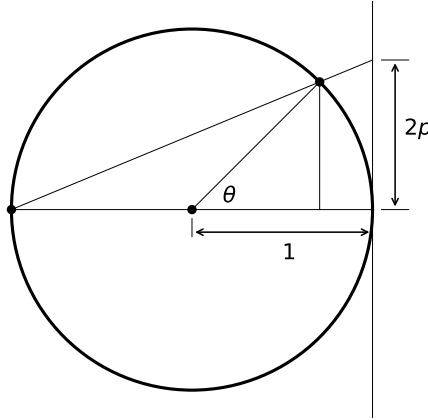


Figure 9.7 - The stereographic projection.

By projecting onto the line, and using similar triangles, we find that  $p = \frac{\sin \theta}{1 + \cos \theta}$ . Solving for  $\sin \theta$  and  $\cos \theta$  reveals

$$\cos \theta = \frac{1 - p^2}{1 + p^2}, \quad \sin \theta = \frac{2p}{1 + p^2}, \quad \text{and} \quad \frac{\partial p}{\partial \theta} = \frac{1 + p^2}{2},$$

where  $\frac{\partial p}{\partial \theta}$  can be used in the chain rule to derive the dynamics  $\dot{p}$ . Although the equations are rational, they share the denominator  $1 + p^2$  and can be treated efficiently in mass-matrix form. Compared to the simple substitution of  $s = \sin \theta$  and

$c = \cos \theta$ , this is a minimal representation (scalar to scalar, no  $s^2 + c^2 = 1$  required); unfortunately it does have a singularity at  $\theta = \pi$ , so likely cannot be used for global analysis.

## 9.2.7 Verifying dynamics in implicit form

Typically we write our differential equations in the form  $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$ . But let us consider for a moment the case where the dynamics are given in the form

$$\mathbf{g}(\mathbf{x}, \mathbf{u}, \dot{\mathbf{x}}) = 0.$$

This form is strictly more general because I can always write  $\mathbf{g}(\mathbf{x}, \mathbf{u}, \dot{\mathbf{x}}) = \mathbf{f}(\mathbf{x}, \mathbf{u}) - \dot{\mathbf{x}}$ , but importantly here I can also write the bottom rows of  $\mathbf{g}$  as  $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} - \tau_g - \mathbf{B}\mathbf{u}$ . This form can also represent [differential algebraic equations \(DAEs\)](#) which are more general than ODEs;  $\mathbf{g}$  could even include algebraic constraints like  $s_i^2 + c_i^2 - 1$ . Most importantly, for manipulators,  $\mathbf{g}$  can be polynomial, even if  $\mathbf{f}$  would have been rational. [DRAKE](#) provides access to continuous-time dynamics in implicit form via the [CalcImplicitTimeDerivativesResidual](#) method.

We typically write a linear system in implicit form as

$$\mathbf{E}\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}.$$

Systems in this form are particularly interesting when  $\mathbf{E}$  is singular, and are known as descriptor systems, semistate systems, generalized state-space systems or just singular systems (c.f. [21]). Just like our standard approach to linearization, we can potentially obtain the matrices  $\mathbf{E}, \mathbf{A}, \mathbf{B}$  from a first-order Taylor approximation of the nonlinear equations in  $\mathbf{g}(\mathbf{x}, \dot{\mathbf{x}}, \mathbf{u})$ . When it comes to Lyapunov analysis, linear systems are often analyzed using the so-called "generalized Lyapunov equation".

### Theorem 9.7 - Lyapunov analysis for implicit linear systems

Given a linear system in the form  $\mathbf{E}\dot{\mathbf{x}} = \mathbf{A}\mathbf{x}$ , the origin is asymptotically stable if there exists a matrix  $\mathbf{P}$  such that

$$\mathbf{P} = \mathbf{P}^T \succ 0, \quad \mathbf{A}^T \mathbf{P} \mathbf{E} + \mathbf{E}^T \mathbf{P} \mathbf{A} \prec 0.$$

Furthermore, if such a  $\mathbf{P}$  exists, then we can obtain one by solving the [generalized Lyapunov equation](#):

$$\mathbf{A}^T \mathbf{P} \mathbf{E} + \mathbf{E}^T \mathbf{P} \mathbf{A} = -\mathbf{Q},$$

for any  $\mathbf{Q} \succ 0$  [22, Ch 4].

This can be verified by taking

$$V(\mathbf{x}) = \mathbf{x}^T \mathbf{E}^T \mathbf{P} \mathbf{E} \mathbf{x},$$

as the Lyapunov function. A corresponding approach can be used (e.g. via sums-of-squares) for implicit nonlinear systems like our manipulator equations where  $\mathbf{g}(\mathbf{x}, \dot{\mathbf{x}}) = 0$  can be written in the form  $\mathbf{E}(\mathbf{x})\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x})$ .

Alternatively, we can check the Lyapunov conditions,  $\dot{V}(\mathbf{x}) \leq 0$ , directly on a system in its implicit form,  $\mathbf{g}(\mathbf{x}, \dot{\mathbf{x}}) = 0$ . Simply define a new function  $Q(\mathbf{x}, \mathbf{z}) = \frac{\partial V(\mathbf{x})}{\partial \mathbf{x}} \cdot \mathbf{z}$ . If we can show  $Q(\mathbf{x}, \mathbf{z}) \leq 0, \forall \mathbf{x}, \mathbf{z} \in \{\mathbf{x}, \mathbf{z} | \mathbf{g}(\mathbf{x}, \mathbf{z}) = 0\}$  using SOS, then we have verified that  $\dot{V}(\mathbf{x}) \leq 0$ , albeit at the non-trivial cost of adding indeterminates  $\mathbf{z}$  and an additional S-procedure.

To combine the trigonometric substitutions with the implicit form for a simple

second-order system (where the number of positions is equal to the number of velocities), we use  $\mathbf{z}$  to represent the implicit  $\dot{\mathbf{q}}$  and convert

$$\begin{aligned} V(\mathbf{q}, \dot{\mathbf{q}}) &\Rightarrow V(\mathbf{s}, \mathbf{c}, \dot{\mathbf{q}}) \\ \mathbf{g}(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{z}) &\Rightarrow \mathbf{g}(\mathbf{s}, \mathbf{c}, \dot{\mathbf{q}}, \mathbf{z}), \end{aligned}$$

and write the Lyapunov conditions using  $\dot{V} = \sum_i \frac{\partial V}{\partial s_i} c_i \dot{q}_i - \sum_i \frac{\partial V}{\partial c_i} s_i \dot{q}_i + \frac{\partial V}{\partial \dot{\mathbf{q}}} \mathbf{z}$ . The most common case for the number of positions is not equal to the number of velocities is when we deal with 3D rotations represented by unit quaternions; this case can be handled using the implicit form, too (we use the implicit form to relate  $\dot{q}$  to the velocities and verify the Lyapunov conditions only on the manifold defined by the quaternion unit norm constraint).

## 9.3 FINITE-TIME REACHABILITY

So far we have used Lyapunov analysis to analyze *stability*, which is fundamentally a description of the system's behavior as time goes to infinity. But Lyapunov analysis can also be used to analyze the finite-time behavior of nonlinear systems. We will see a number of applications of this finite-time analysis over the next few chapters. It can be applied even to unstable systems, or systems that are stable to a limit cycle instead of a fixed-point. It can also be applied to systems that are only defined over a finite interval of time, as might be the case if we are executing a planned trajectory with a bounded duration.

### 9.3.1 Time-varying dynamics and Lyapunov functions

Before focusing on the finite time, let us first realize that the basic (infinite-time) Lyapunov analysis can also be applied to time-varying systems:  $\dot{\mathbf{x}} = f(t, \mathbf{x})$ . We can analyze the stability (local, regional, or global) of this system with very little change. If we find

$$\begin{aligned} V(\mathbf{x}) &> 0, \\ \forall t, \forall \mathbf{x} \neq 0, \quad \dot{V}(t, \mathbf{x}) &= \frac{\partial V}{\partial \mathbf{x}} f(t, \mathbf{x}) < 0, \quad \dot{V}(t, 0) = 0, \end{aligned}$$

then all of our previous statement still hold. In our SOS formulations,  $t$  is simply one more indeterminate.

Similarly, even for a time-invariant system, it is also possible to define a time-varying Lyapunov function,  $V(t, \mathbf{x})$  and establish local, regional, or global stability using the almost the same conditions:

$$\begin{aligned} \forall t, \forall \mathbf{x} \neq 0, \quad V(t, \mathbf{x}) &> 0, \quad V(t, 0) = 0, \\ \forall t, \forall \mathbf{x} \neq 0, \quad \dot{V}(t, \mathbf{x}) &= \frac{\partial V}{\partial \mathbf{x}} f(\mathbf{x}) + \frac{\partial V}{\partial t} < 0, \quad \dot{V}(t, 0) = 0. \end{aligned}$$

These two ideas each stand on their own, but they very often go together, as time-varying dynamics are perhaps the best motivator for studying a time-varying Lyapunov function. Afterall, we do know that a stable time-invariant system must have a time-invariant Lyapunov function that demonstrates this stability (from the "converse Lyapunov function" theorems). But we do not know apriori how to represent this function; as an example, remember we know that there are stable polynomial systems that cannot be verified with a polynomial Lyapunov function. For global stability analysis, the time-varying Lyapunov analysis does not add modeling power: since the conditions must be satisfied for all  $t$ , we could have just set  $t$  to a constant and used the time-invariant function. But there may be cases where a time-varying analysis could afford a different analysis for the regional or local stability cases. We will see a good example of this when we study limit cycles.

### 9.3.2 Finite-time reachability

Finite-time reachability analysis is an important concept for control design and verification, where we seek to understand the behavior of a system over only a finite time interval,  $[t_1, t_2]$ . It is almost always a region-based analysis, and attempts to make a statement of the form:

$$\mathbf{x}(t_1) \in \mathcal{X}_1 \Rightarrow \mathbf{x}(t_2) \in \mathcal{X}_2,$$

where  $\mathcal{X}_1, \mathcal{X}_2 \subset \mathbb{R}^n$  are regions of state space. More generally, we might like to understand the time-varying reachable set  $\forall t \in [t_1, t_2], \mathcal{X}(t)$ .

*Reachability analysis* can be done forward in time: we choose  $\mathcal{X}_1$  and try to find the *smallest* region  $\mathcal{X}_2$  for which we can make the statement hold.  $\mathcal{X}(t)$  would be called the *forward-reachable set* (FRS), and can be very useful for certifying e.g. a motion plan. For instance, you might like to prove that your UAV does not crash into a tree in the next 5 seconds. In this case  $\mathcal{X}_1$  might be take to be a point representing the current state of the vehicle, or a region representing an outer-approximation of the current state if the state is uncertain. In this context, we would call  $\mathcal{X}_2$  a forward-reachable set. In this use case, we would typically choose any approximations in our Lyapunov analysis to certify that an estimate of the reachable region is also an outer-approximation:  $\mathcal{X}_2 \subseteq \hat{\mathcal{X}}_2$ .

Reachability analysis can also be done backward in time: we choose  $\mathcal{X}_2$  and try to *maximize* the region  $\mathcal{X}_1$  for which the statement can be shown to hold. Now  $\mathcal{X}(t)$  is called the *backward-reachable set* (BRS), and for robustness we typically try to certify that our estimates are an inner-approximation,  $\hat{\mathcal{X}}_1 \subseteq \mathcal{X}_1$ . The region-of-attraction analysis we studied above can be viewed as a special case of this, with  $\mathcal{X}_2$  taken to be the fixed-point,  $t_2 = 0$  and  $t_1 = -\infty$ . But finite-time BRS also have an important role to play, for instance when we are composing multiple controllers in order to achieve a more complicated task, which we will study soon.

### 9.3.3 Reachability via Lyapunov functions

Lyapunov functions can be used to certify finite-time reachability, even for continuous-time systems. The basic recipe is to certify the Lyapunov conditions over a (potentially time-varying) invariant set. Once again, we typically represent this as a (time-varying) level set of the Lyapunov function containing the origin,  $V(\mathbf{x}) \leq \rho(t)$ , where  $\rho(t)$  is now a positive scalar function of time. Since we already have time as a decision variable, we can easily accommodate time-varying dynamics and Lyapunov functions, as well, so:

$$\begin{aligned} \forall t \in [t_1, t_2], \quad \forall \mathbf{x}, \quad & V(t, \mathbf{x}) > 0, \quad V(t, 0) = 0 \\ \forall t \in [t_1, t_2], \quad \forall \mathbf{x} \in \{\mathbf{x} | V(t, \mathbf{x}) = \rho(t)\}, \quad & \dot{V}(t, \mathbf{x}) < \dot{\rho}(t). \end{aligned}$$

Note that finite-time reachability is about proving invariance of the set, not stability, and the  $\dot{V}$  condition need only be certified at the boundary of the level set. If  $V$  is decreasing at the boundary, then trajectories can never leave. One can certainly ask for more -- we may want to show that the system is converging towards  $V(t, \mathbf{x}) = 0$ , perhaps even at some rate -- but only invariance is required to certify reachability.

Again, for polynomial systems and dynamics, sums-of-squares optimization is a powerful tool for certifying these properties numerically, and optimizing the volume of the estimated regions. Having taken  $t$  as an indeterminate, we can use the S-procedure again to certify the conditions  $\forall t \in [t_1, t_2]$ .

Like in the case for region of attraction, we have many formulations. We can certify an existing Lyapunov candidate,  $V(t, \mathbf{x})$ , and just try to maximize/minimize  $\rho(t)$ . Or we can search for the parameters of  $V(t, \mathbf{x})$ , too. Again, we can initialize that search using the time-varying version of the Lyapunov equation, or the solutions to a time-varying LQR Riccati equation.

In practice, we often certify the Lyapunov conditions over  $\mathbf{x}$  at only a finite set of samples  $t_i \in [t_1, t_2]$ . I don't actually have anything against sampling in one dimension; there are no issues with scaling to higher dimensions, and one can make practical rigorous statement about bounding the sampling error. And in these systems, adding  $t$  into all of the equation otherwise can dramatically increase the degree of the polynomials required for the SOS certificates. All of this was written up nicely in [23], and robust variants of it were developed in [18, 9].

## 9.4 CONTROL DESIGN

Throughout this chapter, we've developed some very powerful tools for reasoning about stability of a closed-loop system (with the controller already specified). I hope you've been asking yourself -- can I use these tools to design better controllers? Of course the answer is "yes!". In this section, I'll discuss the control approaches that are the most direct consequences of the convex optimization approaches to Lyapunov functions. Another very natural idea is to use these tools in the content of a feedback motion planning algorithm, which is the subject of an [upcoming chapter](#).

### 9.4.1 Control design via alternations

For linear optimal control, we found controllers of the form,  $\mathbf{u} = -\mathbf{K}\mathbf{x}$ . The natural generalization of this to polynomial analysis will be to look for controllers of the form  $\mathbf{u} = \mathbf{K}(\mathbf{x})$ , where  $\mathbf{K}(\mathbf{x})$  is a polynomial vector. (For mechanical systems like the pendulum above, we might make  $\mathbf{K}(x)$  polynomial in  $s$  and  $c$ .)

#### Global stability

If we apply this control directly to the Lyapunov conditions for global analysis with control-affine dynamics, we quickly see the problem. We have

$$\dot{V}(\mathbf{x}) = \frac{\partial V}{\partial \mathbf{x}} [\mathbf{f}_1(\mathbf{x}) + \mathbf{f}_2(\mathbf{x})\mathbf{K}(\mathbf{x})],$$

and as a result if we are trying to search for the parameters of  $V$  and  $\mathbf{K}$  simultaneously, the decision variables multiply and the problem will be non-convex.

One very natural strategy is to use alternations. The idea is simple, we will fix  $\mathbf{K}$  and optimize  $V$ , then fix  $V$  and optimize  $\mathbf{K}$  and repeat until convergence. This approach has roots in the famous "DK iterations" for robust control (e.g. [24]).

For this approach, it is important to start with an initial feasible  $V$  or  $\mathbf{K}$ . For example, one can think of locally stabilizing a nonlinear system with LQR, and then searching for a linear control law (or even a higher-order polynomial control law) that achieves a larger basin of attraction. But note that once we move from global stability to region of attraction optimization, we now need to alternate between three sets of variables:  $V(\mathbf{x}), \mathbf{K}(\mathbf{x}), \lambda(\mathbf{x})$ , where  $\lambda(\mathbf{x})$  was the multiplier polynomial for the S-procedure. We took exactly this approach in [25]. In that paper, we showed that alternations could increase the certified region of attraction for the Acrobot.

The alternation approach takes advantage of a structured convex optimization in each step of the alternations, but it is still only a local approach to solving the non-convex joint optimization. It is subject to local minima. The primary advantage is that, barring numerical issues, we expect recursive feasibility (once we have a controller and Lyapunov function that achieves stability, even with a small region of attraction, we will not lose it) and monotonic improvement on the objective. It is also possible to attempt to optimize these objectives more directly with other non-convex optimization procedures (like stochastic gradient descent, or sequential quadratic programming [7]), but strict feasibility is harder to guarantee. Oftentimes the Lyapunov conditions are just evaluated at samples, or along sample trajectories. However, once we "learn" a Lyapunov candidate even by approximate methods, we may once again attempt to rigorously certify the Lyapunov conditions using just a

single convex optimization step the with controller fixed.

## Maximizing the region of attraction

An extension of this approach can be used to design controllers that are certified only over a region of attraction, but this requires some additional care. We can write the search for the maximal region of attraction as:

$$\begin{aligned} & \max_{\rho, \mathbf{K}(\mathbf{x}), V(\mathbf{x}), \lambda(\mathbf{x})} \quad \rho \\ & \text{subject to} \quad V(\mathbf{x}), \lambda(\mathbf{x}) \text{ are SOS,} \\ & \quad -\frac{\partial V}{\partial \mathbf{x}}(\mathbf{f}_1(\mathbf{x}) + \mathbf{f}_2(\mathbf{x})\mathbf{K}(\mathbf{x})) + \lambda(\mathbf{x})(V(\mathbf{x}) - \rho) \text{ is SOS,} \\ & \quad V(\mathbf{0}) = 0, \quad V(\mathbf{1}) = 1. \end{aligned}$$

The last line yields linear constraints which simply fixes the scaling of  $V(\mathbf{x})$ .

Once again, we find that this optimization is bilinear in the decision variables. But the conditions are linear in  $\lambda(\mathbf{x})$  and  $\mathbf{K}(\mathbf{x})$  for fixed  $V(\mathbf{x})$ , and are linear in  $V(\mathbf{x})$  for fixed  $\lambda(\mathbf{x})$  and  $\mathbf{K}(\mathbf{x})$ . In principle, we could use bilinear alternations, but [25] recommended a three-way alternation approach in order to include a step which explicitly optimizes  $\mathbf{K}(\mathbf{x})$  with an objective of maximizing  $\rho$ . Starting with an initial guess for  $V(\mathbf{x})$  we can repeat the following alternations:

1. Fix  $V(\mathbf{x})$  and  $\rho$ , and solve a feasibility problem for  $\mathbf{K}(\mathbf{x})$  and  $\lambda(\mathbf{x})$ :

$$\begin{aligned} & \text{find}_{\lambda(\mathbf{x}), \mathbf{K}(\mathbf{x})} \\ & \text{subject to} \quad -\frac{\partial V}{\partial \mathbf{x}}(\mathbf{f}_1(\mathbf{x}) + \mathbf{f}_2(\mathbf{x})\mathbf{K}(\mathbf{x})) + \lambda(\mathbf{x})(V(\mathbf{x}) - \rho) \text{ is SOS.} \end{aligned}$$

2. Fix  $V(\mathbf{x})$  and  $\lambda(\mathbf{x})$  and maximize  $\rho$  searching over  $\mathbf{K}(\mathbf{x})$ :

$$\begin{aligned} & \max_{\rho, \mathbf{K}(\mathbf{x})} \quad \rho \\ & \text{subject to} \quad \lambda(\mathbf{x}) \text{ is SOS} \\ & \quad -\frac{\partial V}{\partial \mathbf{x}}(\mathbf{f}_1(\mathbf{x}) + \mathbf{f}_2(\mathbf{x})\mathbf{K}(\mathbf{x})) + \lambda(\mathbf{x})(V(\mathbf{x}) - \rho) \text{ is SOS.} \end{aligned}$$

3. Fix  $\lambda(\mathbf{x})$  and  $\mathbf{K}(\mathbf{x})$  and maximize  $\rho$  searching over  $V(\mathbf{x})$ :

$$\begin{aligned} & \max_{\rho, V(\mathbf{x})} \quad \rho \\ & \text{subject to} \quad V(\mathbf{x}) \text{ is SOS} \\ & \quad -\frac{\partial V}{\partial \mathbf{x}}(\mathbf{f}_1(\mathbf{x}) + \mathbf{f}_2(\mathbf{x})\mathbf{K}(\mathbf{x})) + \lambda(\mathbf{x})(V(\mathbf{x}) - \rho) \text{ is SOS,} \\ & \quad V(\mathbf{0}) = 0, \quad V(\mathbf{1}) = 1. \end{aligned}$$

### 9.4.2 State feedback for linear systems

There is a clever reparameterization of the original problem that can make the co-design of Lyapunov functions and stabilizing controllers jointly convex in some cases.

Let's re-examine the Lyapunov conditions for linear systems from Eq. 2, but now add in a linear state feedback  $\mathbf{u} = \mathbf{K}\mathbf{x}$ , resulting in the closed-loop dynamics  $\dot{\mathbf{x}} = (\mathbf{A} + \mathbf{BK})\mathbf{x}$ . One can show that the set of all stabilizing  $\mathbf{K}$  can be characterized by the following two matrix inequalities [3]:

$$\mathbf{P} = \mathbf{P}^T \succ 0, \quad \mathbf{P}(\mathbf{A} + \mathbf{BK}) + (\mathbf{A} + \mathbf{BK})^T \mathbf{P} \prec 0.$$

Unfortunately, these inequalities are *bilinear* in the decision variables,  $\mathbf{P}$  and  $\mathbf{K}$ , and therefore non-convex. It turns out that we can turn these into *linear* matrix inequalities (LMIs) through a simple change of coordinates,  $\mathbf{Q} = \mathbf{P}^{-1}$ ,  $\mathbf{Y} = \mathbf{K}\mathbf{P}^{-1}$ ,



resulting in

$$\mathbf{Q} = \mathbf{Q}^T \succ 0, \quad \mathbf{A}\mathbf{Q} + \mathbf{Q}\mathbf{A}^T + \mathbf{B}\mathbf{Y} + \mathbf{Y}^T\mathbf{B}^T \prec 0.$$

Furthermore, given matrices  $\mathbf{A}$  and  $\mathbf{B}$ , there exists a matrix  $\mathbf{K}$  such that  $(\mathbf{A} + \mathbf{B}\mathbf{K})$  is stable if and only if there exist matrices  $\mathbf{Q}$  and  $\mathbf{Y}$  which satisfy this (strict) linear matrix inequality.

For global stability, there is a nice extension of this using sums of squares [26] to the case of

$$\dot{\mathbf{x}} = \mathbf{f}_1(\mathbf{x})\mathbf{m}(\mathbf{x}) + \mathbf{f}_2(\mathbf{x})\mathbf{u},$$

where  $\mathbf{m}(\mathbf{x})$  is a vector of monomials with  $\mathbf{m}(\mathbf{x}) = 0$  iff  $\mathbf{x} = 0$ , and the elements of  $\mathbf{f}_1(\mathbf{x})$  and  $\mathbf{f}_2$  are polynomial. The search is over polynomial feedback controllers of the form  $\mathbf{u} = \mathbf{K}(\mathbf{x})\mathbf{m}(\mathbf{x})$ .

### 9.4.3 Control-Lyapunov Functions

Another core idea connecting Lyapunov analysis with control design is the "[control-Lyapunov function](#)". Given a system  $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ , a control-Lyapunov function  $V$  is a positive-definite function for which

$$\forall \mathbf{x} \neq 0, \exists \mathbf{u} \quad \dot{V}(\mathbf{x}, \mathbf{u}) = \frac{\partial V}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) < 0 \quad \text{and} \quad \exists \mathbf{u} \quad \dot{V}(0, \mathbf{u}) = 0.$$

In words, for all  $\mathbf{x}$ , there exists a control input that would allow the  $V$  to decrease. Once again, we adorn this basic property with extra conditions (e.g. radially unbounded, or satisfied over a control-invariant set) in order to construct global or regional stability statements. What is important to understand is that we can design control-Lyapunov functions without explicitly parametrizing the controller; often the control action is not even determined until execution time by finding a particular  $\mathbf{u}$  that goes downhill.

Our sums-of-squares toolkit is well-suited for addressing questions with the  $\forall$  quantifier over indeterminates. Working with the  $\exists$  quantifier is much more difficult; we don't have an S-procedure-like solution for it. Interestingly, there is one approach that we've discussed above that effectively turns the  $\exists$  into a  $\forall$  -- that is the outer approximation approach to region of attraction analysis.

In the outer-approximation, we produce a barrier certificate to find the set of states where the controller cannot go (for any  $\mathbf{u}$ ). Our barrier certificate now has the form

$$\forall \mathbf{x}, \forall \mathbf{u}, \quad \dot{\mathcal{B}}(\mathbf{x}, \mathbf{u}) \leq 0.$$

Certainly,  $\mathcal{B}(\mathbf{x}) \geq 0$  is now an outer-approximation of the true "backward-reachable set" (BRS) of the fixed point [13, 27]. Again [15] has some nice examples of this written directly in sums-of-squares form.

Now here is where it gets a little frustrating. Certainly, sublevel sets of  $\mathcal{B}(\mathbf{x})$  are control-invariant (via the proper choice of  $\mathbf{u}$ ). But we do not (cannot) expect that the entire estimated region (the 0-superlevel set) can be rendered invariant. The estimated region is an *outer* approximation of the backward reachable set. [28] gave a recipe for extracting a polynomial control law from the BRS; an inner approximation of this control law can be obtained via the original SOS region of attraction tools. This is unfortunately suboptimal/conservative. Although we would like to certify the control-Lyapunov conditions directly in an inner approximation, the  $\exists$  quantifier remains as an obstacle.

### 9.4.4 Approximate dynamic programming with SOS

We have [already established](#) the most important connection between the HJB



conditions and the Lyapunov conditions:

$$\dot{J}^*(\mathbf{x}) = -\ell(\mathbf{x}, \mathbf{u}^*) \quad \text{vs} \quad \dot{V}(\mathbf{x}) \preceq 0.$$

The HJB involves solving a complex PDE; by changing this to an inequality, we relax the problem and make it amenable to convex optimization.

## Upper and lower bounds on cost-to-go

Asking for  $\dot{V}(\mathbf{x}) \preceq 0$  is sufficient for proving stability. But we can also use this idea to provide rigorous certificates as upper or lower bounds of the cost-to-go. Given a control-dynamical system  $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ , and a fixed controller  $\pi(\mathbf{x})$  we can find a function  $V(\mathbf{x})$ :

- $\forall \mathbf{x}, \dot{V}^\pi(\mathbf{x}) \leq -\ell(\mathbf{x}, \pi(\mathbf{x}))$  to provide an **upper bound**, or
- $\forall \mathbf{x}, \dot{V}^\pi(\mathbf{x}) \geq -\ell(\mathbf{x}, \pi(\mathbf{x}))$  to provide an **lower bound**.

To see this, take the integral of both sides along any *solution* trajectory,  $\mathbf{x}(t), \mathbf{u}(t)$ . For the upper-bound we get

$$\int_0^\infty \dot{V}^\pi(\mathbf{x}) dt = V^\pi(\mathbf{x}(\infty)) - V^\pi(\mathbf{x}(0)) \leq \int_0^\infty -\ell(\mathbf{x}(t), \pi(\mathbf{x}(t))) dt.$$

Assuming  $V^\pi(\mathbf{x}(\infty)) = 0$ , we have

$$V^\pi(\mathbf{x}(0)) \geq \int_0^\infty \ell(\mathbf{x}(t), \pi(\mathbf{x}(t))) dt.$$

The upper bound is the one that we would want to use in a certification procedure -- it provides a *guarantee* that the total cost achieved by the system started in  $\mathbf{x}$  is less than  $V(\mathbf{x})$ . But it turns out that the lower bound is much better for control design. This is because we can write

$$\forall \mathbf{x}, \min_{\mathbf{u}} [\ell(\mathbf{x}, \mathbf{u}) + \frac{\partial V}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u})] \geq 0 \quad \equiv \quad \forall \mathbf{x}, \forall \mathbf{u}, \ell(\mathbf{x}, \mathbf{u}) + \frac{\partial V}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) \geq 0.$$

Therefore, without having to specify apriori a controller, if we can find a function  $V(\mathbf{x})$  such that  $\forall \mathbf{x}, \forall \mathbf{u}, \dot{V}(\mathbf{x}, \mathbf{u}) \geq -\ell(\mathbf{x}, \mathbf{u})$ , then we have a lower-bound on the *optimal* cost-to-go.

You should take a minute to convince yourself that, unfortunately, the same trick does not work for the upper-bound. Again, we would need  $\exists$  as the quantifier on  $\mathbf{u}$  instead of  $\forall$ .

## Linear Programming Dynamic Programming

In the dynamic programming chapter, we introduced the *linear programming approach to dynamic programming*. This was exactly using the idea of pushing up on a lower bound on the cost-to-go. In the case of a discrete state, then this lower bound will be tight (and we obtain the true cost-to-go function); in the case of linear function approximators, then we hope for a tight lower bound.

## Sums-of-Squares Dynamic Programming

The linear programming approach works natively in discrete (or sampled) time/state/action settings. Sums-of-squares provides the a beautiful extension to continuous time/state/actions.

In the simple case, let us assume that  $f(\mathbf{x}, \mathbf{u})$  and  $\ell(\mathbf{x}, \mathbf{u})$  are polynomials (this assumption is discussed at length in the Lyapunov chapter). We can search for a polynomial  $\hat{J}_\alpha(\mathbf{x})$  which satisfies the HJB inequality

$$\forall \mathbf{x}, \forall \mathbf{u}, \quad 0 \leq \ell(\mathbf{x}, \mathbf{u}) + \frac{\partial \hat{J}_\alpha}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}),$$

using the following *convex* optimization:

$$\begin{aligned} & \max_{\alpha} \int_{\mathbf{X}} \hat{J}_\alpha(\mathbf{x}) d\mathbf{x} \\ & \text{subject to } \ell(\mathbf{x}, \mathbf{u}) + \frac{\partial \hat{J}_\alpha}{\partial \mathbf{x}} f(\mathbf{x}, \mathbf{u}) \text{ is SOS } (\forall \mathbf{x}, \forall \mathbf{u}) \\ & \hat{J}_\alpha(\mathbf{0}) = 0. \end{aligned}$$

Since  $\hat{J}_\alpha(\mathbf{x})$  is polynomial, the integral objective (over a bounded domain of interest) can be computed exactly, and is still linear in the coefficients  $\alpha$ . As in the linear programming approach, here the SOS constraint ensures that  $\hat{J}$  is a *lower* bound on the cost to go, and the objective "pushes up" on this lower bound. As we increase the degree of the polynomial representing  $\hat{J}$ , we expect better and better approximations of the true cost-to-go.

### Example 9.21 (Cubic polynomial optimal control)

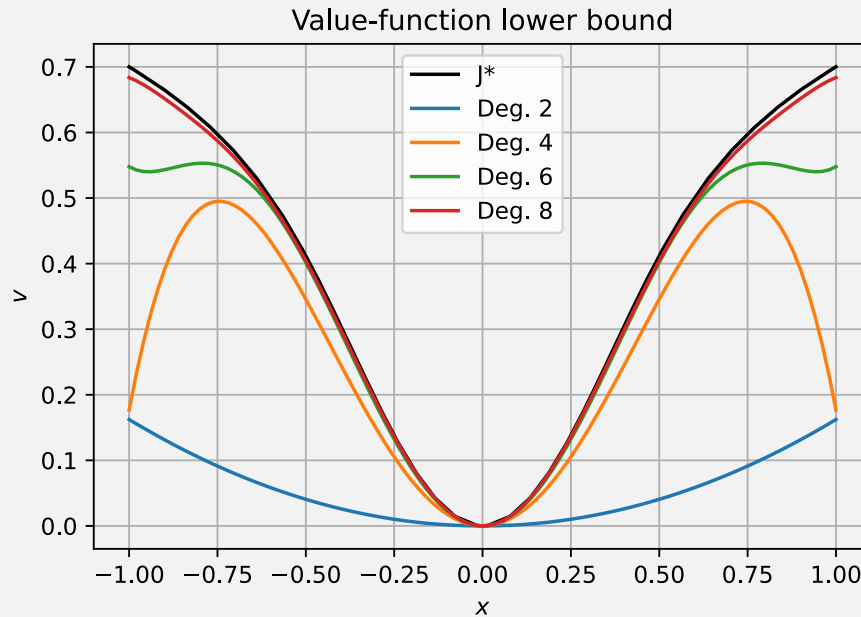
Consider a system with (scalar) dynamics:

$$\dot{x} = f(x, u) = x - 4x^3 + u, u \in [-1, 1]$$

and the running cost

$$\ell(\mathbf{x}, \mathbf{u}) = x^2 + u^2.$$

I've provided code that optimizes the cost-to-go over the interval  $\mathbf{x} \in [-1, 1]$ .



### Example 9.22 (Pendulum swing-up)

We can use this approach to design an approximation to the optimal controller for the (torque-limited) swing-up of the pendulum!

### **Example 9.23 (Cart-pole swing-up)**

This final example is adapted from [29]. Even the lower-bound controller can effectively swing-up the cart-pole system!

I've only provided simple implementations above; their effectiveness is limited by the poor numerics of using polynomials with exceedingly high degree ( $x^{20}$  is a very small number around  $x = 0$  and a very large number around  $x = 2\pi$ ). We should be able to improve the numerics dramatically by choosing more intelligent polynomial bases. I personally think this method has a lot of offer and it would be very interesting to see how far it could go with mature implementations.

## **9.5 ALTERNATIVE COMPUTATIONAL APPROACHES**

### **9.5.1 Sampling Quotient-Ring Sum-of-Squares**

[7]

### **9.5.2 "Satisfiability modulo theories" (SMT)**

Satisfiability modulo theories (SMT). dReal is available in DRAKE.

### **9.5.3 Mixed-integer programming (MIP) formulations**

### **9.5.4 Continuation methods**

## **9.6 NEURAL LYAPUNOV FUNCTIONS**

Details coming soon. For some recent work with a fairly extensive bibliography, see [30].

## **9.7 CONTRACTION METRICS**

## **9.8 OTHER VARIATIONS AND EXTENSIONS**

The analog of Lyapunov functions for stochastic verification are super-martingales [31]. We will discuss that more in the stochastic and robust control chapter.

## **9.9 EXERCISES**

### **Exercise 9.1 (Valid Lyapunov Function for Global Stability)**

For the system

$$\begin{aligned}\dot{x}_1 &= -\frac{6x_1}{(1+x_1^2)^2} + 2x_2, \\ \dot{x}_2 &= -\frac{2(x_1+x_2)}{(1+x_1^2)^2},\end{aligned}$$

you are given the positive definite function  $V(\mathbf{x}) = \frac{x_1^2}{1+x_1^2} + x_2^2$  (plotted [here](#)) and told that, for this system,  $\dot{V}(\mathbf{x})$  is negative definite over the entire space. Is  $V(\mathbf{x})$  a valid Lyapunov function to prove global asymptotic stability of the origin for the system described by the equations above? Motivate your answer.

## Exercise 9.2 (Invariant Sets and Regions of Attraction)

You are given a dynamical system  $\dot{\mathbf{x}} = f(\mathbf{x})$ , with  $f$  continuous, which has a fixed point at the origin. Let  $B_r$  be a ball of (finite) radius  $r > 0$  centered at the origin:  $B_r = \{\mathbf{x} : \|\mathbf{x}\| \leq r\}$ . Assume you found a continuously-differentiable scalar function  $V(\mathbf{x})$  such that:  $V(0) = 0$ ,  $V(\mathbf{x}) > 0$  for all  $\mathbf{x} \neq 0$  in  $B_r$ , and  $\dot{V}(\mathbf{x}) < 0$  for all  $\mathbf{x} \neq 0$  in  $B_r$ . Determine whether the following statements are true or false. Briefly justify your answer.

- $B_r$  is an invariant set for the given system, i.e.: if the initial state  $\mathbf{x}(0)$  lies in  $B_r$ , then  $\mathbf{x}(t)$  will belong to  $B_r$  for all  $t \geq 0$ .
- $B_r$  is a subset of the ROA of the fixed point  $\mathbf{x} = 0$ , i.e.: if  $\mathbf{x}(0)$  lies in  $B_r$ , then  $\lim_{t \rightarrow \infty} \mathbf{x}(t) = 0$ .

## Exercise 9.3 (Are Lyapunov Functions Unique?)

If  $V_1(\mathbf{x})$  and  $V_2(\mathbf{x})$  are valid Lyapunov functions that prove global asymptotic stability of the origin, does  $V_1(\mathbf{x})$  necessarily equal  $V_2(\mathbf{x})$ ? Justify your response.

## Exercise 9.4 (Proving Global Asymptotic Stability)

Consider the system given by

$$\begin{aligned}\dot{x}_1 &= x_2 - x_1^3, \\ \dot{x}_2 &= -x_1 - x_2^3.\end{aligned}$$

Show that the Lyapunov function  $V(\mathbf{x}) = x_1^2 + x_2^2$  proves global asymptotic stability of the origin for this system.

## Exercise 9.5 (Lyapunov Instability Theorem and Barrier functions)

Suppose that we are given a dynamical system  $\dot{\mathbf{x}} = f(\mathbf{x})$  with continuous  $f$ , and that there exists a scalar continuously-differentiable function  $V(\mathbf{x})$  such that:

$$\begin{aligned}\forall \mathbf{x}, V(\mathbf{x}) \leq 0 : \dot{V}(\mathbf{x}) \leq 0, \\ \exists \mathbf{x}_0 : V(\mathbf{x}_0) < V(0) = 0.\end{aligned}$$

- Show that the trajectory  $\mathbf{x}(t)$  with  $\dot{\mathbf{x}} = f(\mathbf{x})$  and  $\mathbf{x}(0) = \mathbf{x}_0$  does not converge to 0.

- b. With that in mind, can  $\dot{\mathbf{x}} = f(\mathbf{x})$  be globally asymptotically stable to 0?
- c. Can  $\dot{\mathbf{x}} = f(\mathbf{x})$  be stable ISL to 0?
- d. Consider the set  $B = \{\mathbf{x} | V(\mathbf{x}) \geq 0\}$ . Suppose that our system starts at  $\mathbf{x}(0)$  where  $V(\mathbf{x}(0)) < 0$ . Can there be a time  $t$ , such that  $\mathbf{x}(t) \in B$ ? Explain why  $V(\mathbf{x})$  serves as a Barrier function to set  $B$ .

## Exercise 9.6 (Converse Lyapunov Theorem for global exponential stability)

Suppose that we are given a dynamical system  $\dot{\mathbf{x}} = f(\mathbf{x})$  with continuous  $f$  that is globally exponentially stable to  $\mathbf{x}^*$  -- i.e., there exist positive constants  $C$  and  $\alpha$ , such that for any initial condition  $\mathbf{x}(0)$  and time  $t$ :  $\|\mathbf{x}(t) - \mathbf{x}^*\| < Ce^{-\alpha t} \|\mathbf{x}(0) - \mathbf{x}^*\|$ . Define  $V(\mathbf{z})$  according to:

$$V(\mathbf{z}) = \int_0^\infty \|\mathbf{x}(t) - \mathbf{x}^*\|^2 dt, \text{ where } \mathbf{x}(0) = \mathbf{z}, \dot{\mathbf{x}} = f(\mathbf{x}).$$

In this exercise, we will show that  $V(\mathbf{z})$  is a Lyapunov function that proves global exponential stability of  $\dot{\mathbf{x}} = f(\mathbf{x})$  as in Theorem 9.2, i.e., that  $V(\mathbf{z})$  is positive definite, radially unbounded, and that  $\exists \beta > 0$ , such that  $\forall \mathbf{z} : \dot{V}(\mathbf{z}) \leq -\beta V(\mathbf{z})$ .

- a. Show that  $(V(\mathbf{z}) - V(\mathbf{x}^*))$  is positive definite.
- b. Explain why  $(V(\mathbf{z}) - V(\mathbf{x}^*))$  is radially unbounded. An intuitive explanation is sufficient here.
- c. Compute  $\dot{V}(\mathbf{z})$ . Hint: notice that by definition, we have:

$$\dot{V}(\mathbf{z}) = \frac{d}{dt} V(\mathbf{x}(t)) \Big|_{t=0} = \lim_{t \rightarrow 0} \frac{V(\mathbf{x}(t)) - V(\mathbf{x}(0))}{t},$$

where  $\mathbf{x}(t)$  is trajectory with  $\mathbf{x}(0) = \mathbf{z}$ .

- d. Find  $\beta > 0$ , such that  $\forall \mathbf{z} : \dot{V}(\mathbf{z}) \leq -\beta V(\mathbf{z})$ . Hint: use the definition of global asymptotic stability.
- e. What is the intuition for the  $V(\mathbf{z})$  defined above? Where have we already seen it before?
- f. Suppose you have a dynamical system  $\dot{\mathbf{x}} = f(\mathbf{x})$ . Why can't you "simply" compute (analytically or numerically)  $V(\mathbf{z})$  as above to check for global asymptotic stability?

## Exercise 9.7 (Gradient Flow in Euclidean Space)

We can use Lyapunov analysis to analyze the behavior of optimization algorithms like gradient descent. Consider an objective function  $\ell : \mathbf{R}^n \rightarrow \mathbf{R}$ , and we want to minimize  $\ell(x)$ . The gradient flow is a continuous-time analog of gradient descent and is defined as  $\dot{x} = -\frac{\partial \ell}{\partial x}$ .

- a. Show that the objective function  $\ell(x) - \ell(x^*)$  is a Lyapunov function of the gradient flow where  $x^*$  is a unique minimizer.
- b. We can use Lyapunov to argue that an optimization problem will converge to a global optimum, even if it is non-convex. Suppose that the Lyapunov function  $\ell$ , has negative definite  $\dot{\ell}$ . Show that the objective function  $\ell$  has a unique minimizer at the origin.
- c. Consider the objective function in the figure below. Could this be a valid Lyapunov function for showing global asymptotic stability?

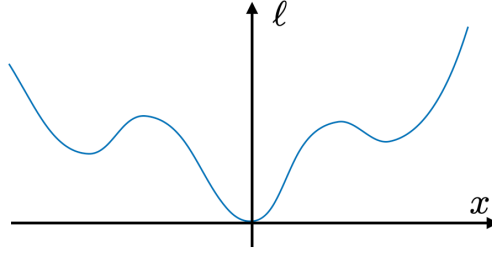


Figure 9.9 - Lyapunov function candidate for asymptotic stability.

- d. Consider the objective function  $\ell(x) = x_1^4 - 8x_1^3 + 18x_1^2 + x_2^2$  with  $x \in \mathbf{R}^2$ . Find the largest  $\rho$  such that  $\{x \mid \ell(x) < \rho\}$  is a valid region of attraction for the origin.

## Exercise 9.8 (Control-Lyapunov Function for a Wheeled Robot)

In this exercise, we examine the idea of a [control-Lyapunov function](#) to drive a wheeled robot, implementing the controller proposed in [32].

Similar to [this previous example](#), we use a kinematic model of the robot. We represent with  $z_1$  and  $z_2$  its Cartesian position and with  $z_3$  its orientation. The controls are the linear  $u_1$  and angular  $u_2$  velocities. The equations of motion read

$$\begin{aligned}\dot{z}_1 &= u_1 \cos z_3, \\ \dot{z}_2 &= u_1 \sin z_3, \\ \dot{z}_3 &= u_2.\end{aligned}$$

The goal is to design a feedback law  $\pi(\mathbf{z})$  that drives the robot to the origin  $\mathbf{z} = 0$  from any initial condition. As pointed out in [32], this problem becomes dramatically easier if we analyze it in polar coordinates. As depicted below, we let  $x_1$  be the radial and  $x_2$  the angular coordinate of the robot, and we define  $x_3 = x_2 - z_3$ . Analyzing the figure, basic kinematic considerations lead to

$$\begin{aligned}\dot{x}_1 &= u_1 \cos x_3, \\ \dot{x}_2 &= -\frac{u_1 \sin x_3}{x_1}, \\ \dot{x}_3 &= -\frac{u_1 \sin x_3}{x_1} - u_2.\end{aligned}$$

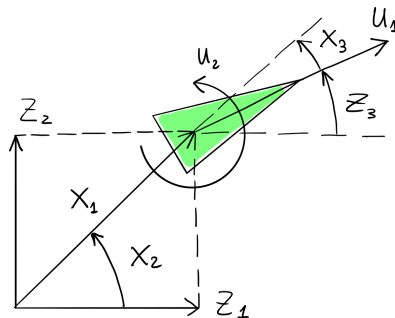


Figure 9.10 - Wheeled robot with Cartesian  $\mathbf{z}$  and polar  $\mathbf{x}$  coordinate system.

- a. For the candidate Lyapunov function  $V(\mathbf{x}) = V_1(x_1) + V_2(x_2, x_3)$ , with  $V_1(x_1) = \frac{1}{2}x_1^2$  and  $V_2(x_2, x_3) = \frac{1}{2}(x_2^2 + x_3^2)$ , compute the time derivatives  $\dot{V}_1(\mathbf{x}, u_1)$  and  $\dot{V}_2(\mathbf{x}, \mathbf{u})$ .

- b. Show that the choice

$$\begin{aligned} u_1 &= \pi_1(\mathbf{x}) = -x_1 \cos x_3, \\ u_2 &= \pi_2(\mathbf{x}) = x_3 + \frac{(x_2 + x_3) \cos x_3 \sin x_3}{x_3}, \end{aligned}$$

makes  $\dot{V}_1(\mathbf{x}, \pi_1(\mathbf{x})) \leq 0$  and  $\dot{V}_2(\mathbf{x}, \pi(\mathbf{x})) \leq 0$  for all  $\mathbf{x}$ . (Technically speaking,  $\pi_2(\mathbf{x})$  is not defined for  $x_3 = 0$ . In this case, we let  $\pi_2(\mathbf{x})$  assume its limiting value  $x_2 + 2x_3$ , ensuring continuity of the feedback law.)

- Explain why Lyapunov's direct method does not allow us to establish asymptotic stability of the closed-loop system.
- Substitute the control law  $\mathbf{u} = \pi(\mathbf{x})$  in the equations of motion, and derive the closed-loop dynamics  $\dot{\mathbf{x}} = f(\mathbf{x}, \pi(\mathbf{x}))$ . Use LaSalle's theorem to show (global) asymptotic stability of the closed-loop system.
- In we set up a simulation environment for you to try the controller we just derived. Type the control law from point (b) in the dedicated cell, and use the notebook plot to check your work.

## Exercise 9.9 (SOS Polynomials in Lyapunov Analysis)

- For each of the following polynomials, using `isSOS`, find whether they (1) admit a sum of squares formulation, and (2) are positive semi-definite functions:
  - $f_1(x) = x^2y^4 + x^4y^2 + 1 - 3x^2y^2$
  - $f_2(x) = x^4 + 3y^4 - 2xy^3$
  - $f_3(x, y, z; \lambda) = z^6 + x^2y^2(x^2 + y^2 - 3\lambda z^2)$ , where  $0 < \lambda \leq 1$ .
- If a fixed point of our dynamical system does not admit a SOS Lyapunov function, what can we conclude about its stability?
- If the optimal cost-to-go for a given dynamical system is not representable as sum of squares, can there still exist a SOS Lyapunov function? If no, explain why; otherwise, give an example of such a system.

## Exercise 9.10 (ROA Estimation for the Time-Reversed Van der Pol Oscillator)

In this exercise you will use SOS optimization to approximate the ROA of the time-reversed Van der Pol oscillator (a variation of the [classical Van der Pol oscillator](#) which evolves backwards in time). In `roa_est`, you are asked to test the following SOS formulations.

- The one from [the example above](#), augmented with a line search that maximizes the area of the ROA.
- A single-shot SOS program that can directly maximize the area of the ROA, without any line search.
- An improved version of the previous, where less SOS constraints are imposed in the optimization problem.

## Exercise 9.11 (Lyapunov Analysis for Robustness of Pendulum Swing-up Controllers)

In this problem, we examine the robustness of the swing-up controller designed for the pendulum in [this example](#) using Lyapunov analysis. We consider an undamped pendulum in part (a)-(d), and one with damping ratio  $b \neq 0$  in (e).

Assume (1) we can measure  $\theta$  and  $\dot{\theta}$  perfectly (2) the pendulum does not start and will not stay at the downright position at any time.

- a. Suppose we have perfect measurements of mass  $m$ , length  $l$ , and the gravitational constant  $g$ . Show that the fixed point is attractive using LaSalle's Theorem and the scalar function  $V(x)$ .
- b. The swing-up controller is robust with respect to mass measurement. Suppose the mass measurement is incorrect  $m_i \neq m$ . Show that the controller can still complete the swing-up task, i.e., the fixed point is still attractive.
- c. Consider the case where both mass and length measurements are incorrect,  $l_i \neq l$  and  $m_i \neq m$ . Is the fixed point still attractive? If not, identify conditions  $m_i, l_i, \dot{\theta}, \theta, \tilde{E}$  could satisfy such that  $\dot{V} > 0$ .
- d. Suppose in addition to uncertain mass and length, the gravitational constant is also uncertain,  $g_i \neq g$ . Suppose the pendulum's small-oscillation period  $T = 2\pi\sqrt{\frac{l}{g}}$  can be measured precisely, and the estimated values of length and gravitational constant satisfy  $T = 2\pi\sqrt{\frac{l_i}{g_i}}$ . Show that the controller can still complete the swing-up task, i.e., the fixed point is still attractive.
- e. Suppose we have accurate measurements of mass and length, but now the pendulum is damped, i.e.,  $b \neq 0$ , and we have incorrect estimates of the damping ratio  $b_i \neq b$ . Using a swing-up controller  $u = -k\dot{\theta}\tilde{E} + b_i\dot{\theta}$ , show that  $V(x) = \frac{1}{2}\tilde{E}^2$  cannot be used as a Lyapunov function to certify attractivity.
- f. When parametric uncertainties are present, the true values of pendulum parameters  $m, g, l$  are unknown. To implement the energy-shaping controller  $u = -k\dot{\theta}\tilde{E}$ , in which  $\tilde{E}$  depends on  $m, g, l$ , we substitute unknown parameters with their estimated values  $m_i, g_i, l_i$  online in order to compute an estimate for energy difference  $\tilde{E}_i$ . However, when we construct Lyapunov function  $V = \frac{1}{2}\tilde{E}^2$  to analyze robustness of our controller, the expressions of  $V$  and  $\dot{V}$  still contain unknown parameters  $m, g, l$ . In order to certify attractivity, is there a need to substitute these parameters with their estimates as well? Explain your reasonings.

## REFERENCES

1. Jean-Jacques E. Slotine and Weiping Li, "Applied Nonlinear Control", Prentice Hall , October, 1990.
2. Hassan K. Khalil, "Nonlinear Systems", Prentice Hall , December, 2001.
3. S. Boyd and L. El Ghaoui and E. Feron and V. Balakrishnan, "Linear Matrix Inequalities in System and Control Theory", SIAM , 1994.
4. Pablo A. Parrilo, "Structured Semidefinite Programs and Semialgebraic Geometry Methods in Robustness and Optimization", PhD thesis, California Institute of Technology, May 18, 2000.
5. Amir Ali Ahmadi and Miroslav Krstic and Pablo A. Parrilo, "A Globally Asymptotically Stable Polynomial Vector Field with no Polynomial Lyapunov Function", *Proceedings of the Conference on Decision and Control* , 2011.
6. Grigoriy Blekherman and Pablo A Parrilo and Rekha R Thomas, "Semidefinite optimization and convex algebraic geometry", {SIAM} , vol. 13, 2013.



7. Shen Shen and Russ Tedrake, "Sampling Quotient-Ring Sum-of-Squares Programs for Scalable Verification of Nonlinear Systems", *Proceedings of the 2020 59th IEEE Conference on Decision and Control (CDC)* , 2020. [ [link](#) ]
8. Russ Tedrake and Ian R. Manchester and Mark M. Tobenkin and John W. Roberts, "{LQR-Trees}: Feedback Motion Planning via Sums of Squares Verification", *International Journal of Robotics Research*, vol. 29, pp. 1038--1052, July, 2010. [ [link](#) ]
9. Anirudha Majumdar, "Funnel Libraries for Real-Time Robust Feedback Motion Planning", PhD thesis, Massachusetts Institute of Technology, Jun, 2016. [ [link](#) ]
10. Stephen Boyd and Lieven Vandenbergh, "Convex Optimization", Cambridge University Press , 2004.
11. Michael Posa, "Optimization for Control and Planning of Multi-contact Dynamic Motion", PhD thesis, Massachusetts Institute of Technology, Jun, 2017. [ [link](#) ]
12. Maryam Fazel and Haitham Hindi and Stephen P Boyd, "Log-det heuristic for matrix rank minimization with applications to Hankel and Euclidean distance matrices", *American Control Conference, 2003. Proceedings of the 2003* , vol. 3, pp. 2156--2162, 2003.
13. Didier Henrion and Milan Korda, "Convex computation of the region of attraction of polynomial control systems", *IEEE Transactions on Automatic Control*, vol. 59, no. 2, pp. 297-312, 2014.
14. Jean Bernard Lasserre, "Moments, Positive Polynomials and Their Applications", World Scientific , vol. 1, 2010.
15. Michael Posa and Twan Koolen and Russ Tedrake, "Balancing and Step Recovery Capturability via Sums-of-Squares Optimization", *Robotics: Science and Systems*, 2017. [ [link](#) ]
16. Gerald B Folland, "How to integrate a polynomial over a sphere", *The American Mathematical Monthly*, vol. 108, no. 5, pp. 446--448, 2001.
17. D. Henrion and J.B. Lasserre and C. Savorgnan, "Approximate volume and integration for basic semialgebraic sets", *SIAM Review*, vol. 51, no. 4, pp. 722--743, 2009.
18. Anirudha Majumdar, "Robust Online Motion Planning with Reachable Sets", , May, 2013. [ [link](#) ]
19. Charles W. Wampler and Andrew J. Sommese, "Numerical algebraic geometry and algebraic kinematics", *Acta Numerica*, vol. 20, pp. 469-567, 2011.
20. A.J. Sommese and C.W. Wampler, "The Numerical solution of systems of polynomials arising in engineering and science", World Scientific Pub Co Inc , 2005.
21. Volker Mehrmann, "The autonomous linear quadratic control problem: theory and numerical solution", Springer , no. 163, 1991.
22. Tatjana Stykel, "Analysis and numerical solution of generalized {Lyapunov} equations", *Institut für Mathematik, Technische Universität, Berlin*, 2002.
23. Mark M. Tobenkin and Ian R. Manchester and Russ Tedrake, "Invariant Funnels around Trajectories using Sum-of-Squares Programming",

*Proceedings of the 18th IFAC World Congress, extended version available online: arXiv:1010.3013 [math.DS], 2011. [ [link](#) ]*

24. R. Lind and G.J. Balas and A. Packard, "Evaluating {D-K} iteration for control design", *American Control Conference, 1994* , vol. 3, pp. 2792 - 2797 vol.3, 29 June-1 July, 1994.
25. Anirudha Majumdar and Amir Ali Ahmadi and Russ Tedrake, "Control Design along Trajectories with Sums of Squares Programming", *Proceedings of the 2013 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4054-4061, 2013. [ [link](#) ]
26. Stephen Prajna and Antonis Papachristodoulou and Fen Wu, "Nonlinear Control Synthesis by Sum of Squares Optimization: A {L}yapunov-based Approach", *Proceedings of the ASCC 2004* , 2004.
27. Milan Korda and Didier Henrion and Colin N Jones, "Controller design and region of attraction estimation for nonlinear dynamical systems", *The 19th World Congress of the International Federation of Automatic Control (IFAC)*, 2014.
28. Anirudha Majumdar and Ram Vasudevan and Mark M. Tobenkin and Russ Tedrake, "Convex Optimization of Nonlinear Feedback Controllers via Occupation Measures", *Proceedings of Robotics: Science and Systems (RSS)* , 2013. [ [link](#) ]
29. Lujie Yang and Hongkai Dai and Alexandre Amice and Russ Tedrake, "Suboptimal Controller Synthesis for Cart-Poles and Quadrotors via Sums-of-Squares", *In preparation* , 2023.
30. Hongkai Dai and Benoit Landry and Lujie Yang and Marco Pavone and Russ Tedrake, "Lyapunov-stable neural-network control", *Robotics: {S}cience and {S}ystems* , 2021. [ [link](#) ]
31. Jacob Steinhardt and Russ Tedrake, "Finite-time Regional Verification of Stochastic Nonlinear Systems", *International Journal of Robotics Research*, vol. 31, no. 7, pp. 901-923, June, 2012. [ [link](#) ]
32. Michele Aicardi and Giuseppe Casalino and Antonio Bicchi and Aldo Balestrino, "Closed loop steering of unicycle like vehicles via Lyapunov techniques", *IEEE Robotics & Automation Magazine*, vol. 2, no. 1, pp. 27-35, 1995.

# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 10

# Trajectory Optimization

I've argued that optimal control is a powerful framework for specifying complex behaviors with simple objective functions, letting the dynamics and constraints on the system shape the resulting feedback controller (and vice versa!). But the computational tools that we've provided so far have been limited in some important ways. The numerical approaches to dynamic programming which involve putting a mesh over the state space do not scale well to systems with state dimension more than four or five. Linearization around a nominal operating point (or trajectory) allowed us to solve for locally optimal control policies (e.g. using LQR) for even very high-dimensional systems, but the effectiveness of the resulting controllers is limited to the region of state space where the linearization is a good approximation of the nonlinear dynamics. The computational tools for Lyapunov analysis from the last chapter can provide, among other things, an effective way to compute estimates of those regions, and the sums-of-squares approaches to dynamic programming are still in their early days. We have not yet provided the mainstream computational tools for approximate optimal control that work for high-dimensional systems beyond the linearization around a goal. That is precisely the goal for this chapter.

In order to scale to high-dimensional systems, we are going to formulate a simpler version of the optimization problem. Rather than trying to solve for the optimal feedback controller for the entire state space, in this chapter we will instead attempt to find an optimal control solution that is valid from only a single initial condition. Instead of representing this as a feedback control function, we can represent this solution as a *trajectory*,  $\mathbf{x}(t), \mathbf{u}(t)$ , typically defined over a finite interval.

### 10.1 PROBLEM FORMULATION

Given a state-space model,  $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ , an initial condition,  $\mathbf{x}_0$ , and an input trajectory  $\mathbf{u}(t)$  defined over a finite interval,  $t \in [t_0, t_f]$ , we can simulate the dynamics forward to obtain  $\mathbf{x}(t)$  over the same interval. We will define the long-term (finite-horizon) cost of executing that trajectory using the standard additive-cost optimal control objective,

$$J_{\mathbf{u}(\cdot)}(\mathbf{x}_0) = \ell_f(\mathbf{x}(t_f)) + \int_{t_0}^{t_f} \ell(\mathbf{x}(t), \mathbf{u}(t)) dt.$$

$\ell_f$  is a *terminal cost*, evaluated at the end of the trajectory, which we did not have in the infinite-horizon formulations we've focused on so far. We will write the trajectory optimization problem as

$$\begin{aligned} \min_{\mathbf{u}(\cdot)} \quad & \ell_f(\mathbf{x}(t_f)) + \int_{t_0}^{t_f} \ell(\mathbf{x}(t), \mathbf{u}(t)) dt \\ \text{subject to} \quad & \dot{\mathbf{x}}(t) = f(\mathbf{x}(t), \mathbf{u}(t)), \quad \forall t \in [t_0, t_f] \\ & \mathbf{x}(t_0) = \mathbf{x}_0. \end{aligned}$$

Many trajectory optimization problems will also include additional constraints, such as collision avoidance (e.g., where the constraint is that the signed distance between the robot's geometry and the obstacles stays positive) or input limits (e.g.  $\mathbf{u}_{min} \leq \mathbf{u} \leq \mathbf{u}_{max}$ ). Constraints can be defined for all time or some subset of the trajectory.

As written, the optimization above is an optimization over continuous trajectories. In order to formulate this as a numerical optimization, we must parameterize it with a finite set of numbers. Perhaps not surprisingly, there are many different ways to write down this parameterization, with a variety of different properties in terms of speed, robustness, and accuracy of the results. We will outline just a few of the most popular below; I would recommend [1, 2] for additional details.

It is worth contrasting this parameterization problem with the one that we faced in our continuous-dynamic programming algorithms. For trajectory optimization, we need a finite-dimensional parameterization in only one dimension (time), whereas in the mesh-based value iteration algorithms we had to work in the dimension of the state space. Our mesh-based discretizations scaled badly with this state dimension, and led to numerical errors that were difficult to deal with. Discretizing over just one dimension (time), is fundamentally different. There is relatively much more known about discretizing solutions to differential equations over time, including work on error-controlled integration. And the number of parameters required for trajectory parameterizations scales linearly with the state dimension, instead of exponentially in mesh-based value iteration.

## 10.2 CONVEX FORMULATIONS FOR LINEAR SYSTEMS

Let us first consider the case of linear systems. In fact, if we start in discrete time with a state space model of the form  $\mathbf{x}[n+1] = \mathbf{A}\mathbf{x}[n] + \mathbf{B}\mathbf{u}[n]$ , we can even defer the question of how to best discretize the continuous-time problem. There are a few different ways that we might "transcribe" this optimization problem into a concrete mathematical program.

### 10.2.1 Direct Transcription

For instance, let us start by writing both  $\mathbf{u}[\cdot]$  and  $\mathbf{x}[\cdot]$  as decision variables. Then we can write:

$$\begin{aligned} \min_{\mathbf{x}[\cdot], \mathbf{u}[\cdot]} \quad & \ell_f(\mathbf{x}[N]) + \sum_{n=0}^{N-1} \ell(\mathbf{x}[n], \mathbf{u}[n]) \\ \text{subject to} \quad & \mathbf{x}[n+1] = \mathbf{A}\mathbf{x}[n] + \mathbf{B}\mathbf{u}[n], \quad \forall n \in [0, N-1] \\ & \mathbf{x}[0] = \mathbf{x}_0 \\ & + \text{additional constraints.} \end{aligned}$$

We call this modeling choice -- adding  $\mathbf{x}[\cdot]$  as decision variables and modeling the discrete dynamics as explicit constraints -- the "*direct transcription*". Importantly, for linear systems, the dynamics constraints are linear constraints in these decision variables. As a result, if we can restrict our additional constraints to linear inequality constraints and our objective function to being linear/quadratic in  $\mathbf{x}$  and  $\mathbf{u}$ , then the resulting trajectory optimization is a convex optimization (specifically a linear program or quadratic program depending on the objective). We can reliably solve

these problems to global optimality at quite large scale; these days it is common to solve these optimization online inside a high-rate feedback controller.

### Example 10.1 (Trajectory optimization for the Double Integrator)

We've looked at a few optimal control problems for the double integrator using value iteration. For one of them -- the quadratic objective with no constraints on  $\mathbf{u}$  -- we know now that we could have solved the problem "exactly" using LQR. But we have not yet given satisfying numerical solutions for the minimum-time problem, nor for the constrained LQR problem.

In the trajectory formulation, we can solve these problems exactly for the discrete-time double integrator, and with better accuracy for the continuous-time double integrator. Take a moment to appreciate that! The bang-bang policy and cost-to-go functions are fairly nontrivial functions of state; it's quite satisfying that we can evaluate them using convex optimization! The limitation, of course, is that we are only solving them for one initial condition at a time.

If you have not studied convex optimization before, you might be surprised by the modeling power of even of this framework. Consider, for instance, an objective of the form

$$\ell(\mathbf{x}, \mathbf{u}) = |\mathbf{x}| + |\mathbf{u}|.$$

This can be formulated as a linear program. To do it, add additional decision variables  $\mathbf{s}_x[\cdot]$  and  $\mathbf{s}_u[\cdot]$  -- these are commonly referred to as *slack variables* -- and write

$$\min_{\mathbf{x}, \mathbf{u}, \mathbf{s}_x, \mathbf{s}_u} \sum_n^{N-1} \mathbf{s}_x[n] + \mathbf{s}_u[n], \quad \text{s.t.} \quad \mathbf{s}_x[n] \geq \mathbf{x}[n], \quad \mathbf{s}_x[n] \geq -\mathbf{x}[n], \quad \dots$$

The field of convex optimization is replete with tricks like this. Knowing and recognizing them are skills of the (optimization) trade. But there are also many relevant constraints which cannot be recast into convex constraints (in the original coordinates) with any amount of skill. An important example is obstacle avoidance. Imagine a vehicle that must decide if it should go left or right around an obstacle. This represents a fundamentally non-convex constraint in  $\mathbf{x}$ ; we'll discuss the implications of using non-convex optimization for trajectory optimization below.

### 10.2.2 Direct Shooting

The savvy reader might have noticed that adding  $\mathbf{x}[\cdot]$  as decision variables was not strictly necessary. If we know  $\mathbf{x}[0]$  and we know  $\mathbf{u}[\cdot]$ , then we should be able to solve for  $\mathbf{x}[n]$  using forward simulation. For our discrete-time linear systems, this is particularly nice:

$$\begin{aligned} \mathbf{x}[1] &= \mathbf{A}\mathbf{x}[0] + \mathbf{B}\mathbf{u}[0] \\ \mathbf{x}[2] &= \mathbf{A}(\mathbf{A}\mathbf{x}[0] + \mathbf{B}\mathbf{u}[0]) + \mathbf{B}\mathbf{u}[1] \\ \mathbf{x}[n] &= \mathbf{A}^n \mathbf{x}[0] + \sum_{k=0}^{n-1} \mathbf{A}^{n-1-k} \mathbf{B} \mathbf{u}[k]. \end{aligned}$$

What's more, the solution is still linear in  $\mathbf{u}[\cdot]$ . This is amazing... we can get rid of a bunch of decision variables, and turn a constrained optimization problem into an unconstrained optimization problem (assuming we don't have any other constraints). This approach -- using  $\mathbf{u}[\cdot]$  but *not*  $\mathbf{x}[\cdot]$  as decision variables and using forward simulation to obtain  $\mathbf{x}[n]$  -- is called the *direct shooting* transcription. For linear systems with linear/quadratic objectives in  $\mathbf{x}$ , and  $\mathbf{u}$ , it is still a convex optimization, and has less decision variables and constraints than the direct transcription.

### 10.2.3 Computational Considerations

So is direct shooting uniformly better than the direct transcription approach? I think it is not. There are a few potential reason that one might prefer the direct transcription:

- Numerical conditioning. Shooting involves calculating  $\mathbf{A}^n$  for potentially large  $n$ , which can lead to a large range of coefficient values in the constraints. This problem is sometimes referred to as the "tail wagging the dog", or in very deep neural networks as "vanishing gradients" and/or "exploding gradients". In trajectory optimization, it is not simply an artifact of the shooting transcription, but is a real property of the problem we have formulated: the control input  $\mathbf{u}[0]$  really does have more opportunity to have a large impact on the total cost than control input  $\mathbf{u}[N-1]$ . The direct transcription approach combats the numerical issue by spreading this effect out over a large number of well-balanced constraints.
- Adding state constraints. Having  $\mathbf{x}[n]$  as explicit decision variables makes it very easy/natural to add additional state constraints; and the solver effectively reuses the computation of  $\mathbf{A}^n$  for each constraint. In shooting, one has to unroll those terms for each new constraint.
- Parallelization. For larger problems, evaluating the constraints can be a substantial cost. In direct transcription, one can evaluate the dynamics/constraints in parallel (because each iteration begins with  $\mathbf{x}[n]$  already given), whereas shooting is more fundamentally a serial operation.

For linear convex problems, the solvers are mature enough that these differences often don't amount to much. For nonlinear optimization problems, the differences can be substantial. If you look at trajectory optimization papers in mainstream robotics, you will see that both direct transcription and direct shooting approaches are used. (It used to be that you could guess which research lab wrote a paper simply by the transcription they use!)

It is also worth noting that the problems generated by the direct transcription have an important and exploitable "banded" sparsity pattern -- most of the constraints touch only a small number of variables. This is actually the same pattern that we exploit in the Riccati equations. Thanks to the importance of these methods in real applications, numerous specialized solvers have been written to explicitly exploit this sparsity (e.g. [3, 4]).

### 10.2.4 Continuous Time

If we wish to solve the continuous-time version of the problem, then we can discretize time and use the formulations above. The most important decision is the discretization / numerical integration scheme. For linear systems, if we assume that the control inputs are held constant for each time step (aka **zero-order hold**), then we can integrate the dynamics perfectly:

$$\mathbf{x}[n+1] = \mathbf{x}[n] + \int_{t_n}^{t_n+h} [\mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}[n]]dt = e^{\mathbf{A}h}\mathbf{x}[n] + \mathbf{A}^{-1}(e^{\mathbf{A}h} - \mathbf{I})\mathbf{B}\mathbf{u}[n],$$

is the simple case (when  $\mathbf{A}$  is invertible). But in general, we can use any finitely-parameterized representation of  $\mathbf{u}(t)$  and any numerical integration scheme. Many (but not all) integration schemes will still be convex in the decision variables.

## 10.3 NONCONVEX TRAJECTORY OPTIMIZATION

I strongly recommend that you study the convex trajectory optimization case; it can lead you to mental clarity and sense of purpose. But in practice trajectory optimization is often used to solve nonconvex problems. Our formulation can become nonconvex for a number of reasons. For example, if the dynamics are nonlinear, then

the dynamic constraints become nonconvex. You may also wish to have a nonconvex objective or additional nonconvex constraints (e.g. collision avoidance). Typically we formulate these problems using tools from [nonlinear programming](#).

### 10.3.1 Direct Transcription and Direct Shooting

The formulations that we wrote for direct transcription and direct shooting above are still valid when the dynamics are nonlinear, it's just that the resulting problem is nonconvex. For instance, the direct transcription for discrete-time systems becomes the more general:

$$\begin{aligned} \min_{\mathbf{x}[\cdot], \mathbf{u}[\cdot]} \quad & \ell_f(\mathbf{x}[N]) + \sum_{n=0}^{N-1} \ell(\mathbf{x}[n], \mathbf{u}[n]) \\ \text{subject to} \quad & \mathbf{x}[n+1] = \mathbf{f}(\mathbf{x}[n], \mathbf{u}[n]), \quad \forall n \in [0, N-1] \\ & \mathbf{x}[0] = \mathbf{x}_0 \\ & + \text{additional constraints.} \end{aligned}$$

Direct shooting still works, too, since on each iteration of the algorithm we can compute  $\mathbf{x}[n]$  given  $\mathbf{x}[0]$  and  $\mathbf{u}[\cdot]$  by forward simulation. But things get a bit more interesting when we consider continuous-time systems.

For nonlinear dynamics, we have many choices for how to approximate the discrete dynamics

$$\mathbf{x}[n+1] = \mathbf{x}[n] + \int_{t[n]}^{t[n+1]} \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t)) dt, \quad \mathbf{x}(t[n]) = \mathbf{x}[n].$$

For instance, in [DRAKE](#) we have an entire [suite of numerical integrators](#) that achieve different levels of simulation speed and/or accuracy, both of which can be highly dependent on the details of  $\mathbf{f}(\mathbf{x}, \mathbf{u})$ .

One very important idea in numerical integration of differential equations is the use of variable-step integration as a means for controlling integration error. [Runge-Kutta-Fehlberg](#), also known as "RK45", is one of the most famous variable-step integrators. We typically avoid using variable steps inside a constraint (it can lead to discontinuous gradients), but it is possible to accomplish something similar in trajectory optimization by allowing the sample times,  $t[\cdot]$ , themselves to be decision variables. This allows the optimizer to stretch or shrink the time intervals in order to solve the problem, and is particularly useful if you do not know apriori what the total duration of the trajectory should be. Adding some constraints to these time variables is essential in order to avoid trivial solutions (like collapsing to a trajectory of zero duration). One could potentially even add constraints to bound the integration error.

### 10.3.2 Direct Collocation

It is very satisfying to have a suite of numerical integration routines available for our direct transcription. But numerical integrators are designed to solve forward in time, and this represents a design constraint that we don't actually have in our direct transcription formulation. If our goal is to obtain an accurate solution to the differential equation with a small number of function evaluations / decision variables / constraints, then some new formulations are possible that take advantage of the constrained optimization formulation. These include the so-called [collocation methods](#).

In direct collocation (c.f., [\[5\]](#)), both the input trajectory and the state trajectory are represented explicitly as piecewise-polynomial functions. In particular, the sweet spot for this algorithm is taking  $\mathbf{u}(t)$  to be a first-order polynomial and  $\mathbf{x}(t)$  to be a cubic polynomial.

It turns out that in this sweet spot, the only decision variables we need in our



optimization are the sample values  $\mathbf{u}(t)$  and  $\mathbf{x}(t)$  at the so called "break" points of the spline. You might think that you would need the coefficients of the cubic spline parameters, but you do not. For the first-order interpolation on  $\mathbf{u}(t)$ , given  $\mathbf{u}(t_k)$  and  $\mathbf{u}(t_{k+1})$ , we can solve for every value  $\mathbf{u}(t)$  over the interval  $t \in [k, k+1]$ . But we also have everything that we need for the cubic spline: given  $\mathbf{x}(t_k)$  and  $\mathbf{u}(t_k)$ , we can compute  $\dot{\mathbf{x}}(t_k) = f(\mathbf{x}(t_k), \mathbf{u}(t_k))$ ; and the four values  $\mathbf{x}(t_k), \mathbf{x}(t_{k+1}), \dot{\mathbf{x}}(t_k), \dot{\mathbf{x}}(t_{k+1})$  completely define all of the parameters of the cubic spline over the interval  $t \in [t_k, t_{k+1}]$ . This is very convenient, because it is very natural to write constraints on  $\mathbf{u}$  and  $\mathbf{x}$  at the sample points (and would have been relatively harder to have to convert every constraint into constraints on the spline coefficients).

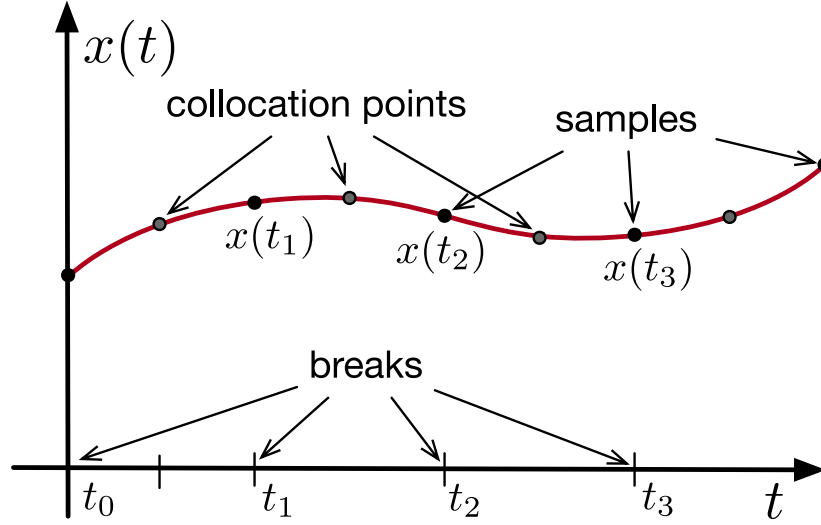


Figure 10.1 - Cubic spline parameters used in the direct collocation method.

Given the dynamics and the values of the decision variables ( $\mathbf{u}(t_k)$  and  $\mathbf{x}(t_k)$  at the break points), the spline is fully specified, but we need  $\dim(\mathbf{x})$  additional constraints to constrain that  $\mathbf{x}(t_{k+1})$  is (approximately)  $\mathbf{x}(t_k) + \int_{t_k}^{t_{k+1}} f(\mathbf{x}(t), \mathbf{u}(t)) dt$ . In direct collocation, we impose this with derivative constraints at the so-called *collocation points*. In particular, if we choose the collocation times to be the midpoints of the spline, then we have that

$$\begin{aligned} t_{c,k} &= \frac{1}{2}(t_k + t_{k+1}), & h_k &= t_{k+1} - t_k, \\ \mathbf{u}(t_{c,k}) &= \frac{1}{2}(\mathbf{u}(t_k) + \mathbf{u}(t_{k+1})), \\ \mathbf{x}(t_{c,k}) &= \frac{1}{2}(\mathbf{x}(t_k) + \mathbf{x}(t_{k+1})) + \frac{h}{8}(\dot{\mathbf{x}}(t_k) - \dot{\mathbf{x}}(t_{k+1})), \\ \dot{\mathbf{x}}(t_{c,k}) &= -\frac{3}{2h}(\mathbf{x}(t_k) - \mathbf{x}(t_{k+1})) - \frac{1}{4}(\dot{\mathbf{x}}(t_k) + \dot{\mathbf{x}}(t_{k+1})). \end{aligned}$$

These equations come directly from the equations that fit the cubic spline to the end points/derivatives then interpolate them at the midpoint. They give us precisely what we need to add the dynamics constraint to our optimization at the collocation times:

$$\begin{aligned} \min_{\mathbf{x}[\cdot], \mathbf{u}[\cdot]} \quad & \ell_f(\mathbf{x}[N]) + \sum_{n=0}^{N-1} h_n \ell(\mathbf{x}[n], \mathbf{u}[n]) \\ \text{subject to} \quad & \dot{\mathbf{x}}(t_{c,n}) = f(\mathbf{x}(t_{c,n}), \mathbf{u}(t_{c,n})), \quad \forall n \in [0, N-1] \\ & \mathbf{x}[0] = \mathbf{x}_0 \\ & + \text{additional constraints.} \end{aligned}$$

I hope this notation is clear -- I'm using  $\mathbf{x}[k] = \mathbf{x}(t_k)$  as the decision variables, and the collocation constraint at  $t_{c,k}$  depends on the decision variables:  $\mathbf{x}[k], \mathbf{x}[k+1], \mathbf{u}[k], \mathbf{u}[k+1]$ . The actual equations of motion get evaluated at both the break points,  $t_k$ , and the collocation times,  $t_{c,k}$ , because we need to evaluate the



dynamics at the break points in order to write  $\dot{\mathbf{x}}(t_{c,k})$ .

Once again, direct collocation effectively integrates the equations of motion by satisfying the constraints of the optimization -- this time producing an integration of the dynamics that is accurate to third order with effectively two evaluations of the plant dynamics per segment (since we use  $\dot{\mathbf{x}}(t_k)$  for two intervals). [5] claims, without proof, that as the break points are brought closer together, the trajectory will converge to a true solution of the differential equation. Once again it is very natural to add additional terms to the cost function or additional input/state constraints, and very easy to calculate the gradients of the objective and constraints. I personally find it very nice to explicitly account for the parametric encoding of the trajectory in the solution technique.

### Example 10.2 (Direct Collocation for the Pendulum, Acrobot, and Cart-Pole)

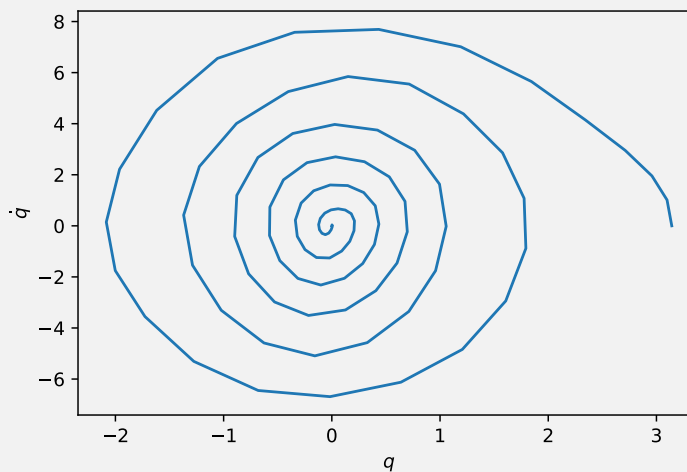


Figure 10.2 - A swing-up trajectory for the simple pendulum (with severe torque limits) optimized using direct collocation.

Direct collocation also easily solves the swing-up problem for the pendulum, Acrobot, and cart-pole system. Try it for yourself:

As always, make sure you take a look at the code!

### 10.3.3 Pseudo-spectral Methods

The direct collocation method of [5] was our first example of explicitly representing the solution of the optimal control problem as a parameterized trajectory, and adding constraints to the derivatives at a series of collocation times. In the algorithm above, the representation of choice was *piecewise-polynomials*, e.g. cubic splines, and the trajectory values at the break points were the decision variables. A closely related approach, often called "pseudo-spectral" optimal control, uses the same collocation idea, but represents the trajectories instead using a linear combination of *global, polynomial* basis functions. These methods use typically much higher-degree polynomials, but can leverage clever parameterizations to write sparse collocation objectives and to select the collocation times [6, 7]. Interestingly, the continuously-differentiable nature of the representation of these methods has led to comparatively more theorems and analysis than we have seen for other direct trajectory optimization methods [7] -- but despite some of the language used in these articles please remember they are still local optimization methods trying to solve a nonconvex optimization problem. While the direct collocation method above might be expected to converge to the true optimal solution by adding more segments to the piecewise polynomial (and having each segment represent a smaller interval

of time), here we expect convergence to happen as we increase the degree of the polynomials.

The pseudo-spectral methods are also sometimes known as "orthogonal collocation" because the  $N$  basis polynomials,  $\phi_j(t)$ , are chosen so that at the  $j$ th collocation time  $t_j$ , we have

$$\phi_i(t_j) = \begin{cases} 1 & i = j, \\ 0 & \text{otherwise.} \end{cases}$$

This can be accomplished by using the Lagrange polynomials,

$$\phi_i(t) = \prod_{j=0, j \neq i}^N \frac{t - t_j}{t_i - t_j}.$$

Note that for both numerical reasons and for analysis, time is traditionally rescaled from the interval  $[t_0, t_f]$  to  $[-1, 1]$ . Collocation times are chosen based on small variations of Gaussian quadrature, known as the "Gauss-Lobatto" which includes collocation times at  $t = -1$  and  $t = 1$ .

Interestingly, a number of papers have also considered infinite-horizon pseudo-spectral optimization by the nonlinear rescaling of the time interval  $t \in [0, \infty)$  to the half-open interval  $\tau \in [-1, 1)$  via  $\tau = \frac{t-1}{t+1}$  [6, 7]. In this case, one chooses the collocation times so that they include  $\tau = -1$  but do not include  $\tau = 1$ , using the so-called "Gauss-Radau" points [6].

## Dynamic constraints in implicit form

There is another, seemingly subtle but potentially important, opportunity that can be exploited in a few of these transcriptions, if our main interest is in optimizing systems with significant multibody dynamics. In some cases, we can actually write the dynamics constraints directly in their implicit form. We've [introduced this idea already](#) in the context of Lyapunov analysis. In many cases, it is nicer or more efficient to obtain the equations of motion in an implicit form,  $\mathbf{g}(\mathbf{x}, \mathbf{u}, \dot{\mathbf{x}}) = 0$ , and to avoid ever having to solve for the explicit form  $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$ . This can become even more important when we consider systems for which the explicit form doesn't have a unique solution -- we will see examples of this when we study trajectory optimization through contact because the Coulomb model for friction actually results in a differential inclusion instead of a differential equation.

The collocation methods, which operate on the dynamic constraints at collocation points directly in their continuous form, can use the implicit form directly. It is possible to write a time-stepping (discrete-time approximation) for direct transcription using implicit integrators -- again providing constraints in implicit form. The implicit form is harder to exploit in the shooting methods.

## 10.4 SOLUTION TECHNIQUES

The different transcriptions presented above represent different ways to map the (potentially continuous-time) optimal control problem into a finite set of decision variables, objectives, and constraints. But even once that choice is made, there are numerous approaches to solving this optimization problem. Any general approach to [nonlinear programming](#) can be applied here; in the python examples we've included so far, the problems are handed directly to the sequential-quadratic programming (SQP) solver SNOPT, or to the interior-point solver IPOPT.

There is also quite a bit of exploitable problem-specific structure in these trajectory optimization problems due to the sequential nature of the problem. As a result, there are some ideas that are fairly specific to the trajectory optimization formulation of optimal control, and customized solvers can often (and sometimes dramatically) outperform general purpose solvers.

This trajectory-optimization structure is easiest to discuss, and implement, in unconstrained formulations, so we will start there.

### 10.4.1 Efficiently computing gradients

Providing gradients of the objectives and constraints to the solver is not strictly required -- most solvers will obtain them from finite differences if they are not provided -- but I feel strongly that for smooth problems the solvers are faster and more robust when exact gradients are provided (systems that make and break contact might be a different story[8]). Providing the gradients for the direct transcription methods is very straight-forward -- we simply provide the gradients for each constraint individually. But in the direct shooting approach, where we have removed the  $\mathbf{x}$  decision variables from the program but still write objectives and constraints in terms of  $\mathbf{x}$ , it would become very inefficient to compute the gradients of each objective/constraint independently. We need to leverage the chain rule.

To be concise (and slightly more general), let us define  $\mathbf{x}[n+1] = f_d(\mathbf{x}[n], \mathbf{u}[n])$  as the discrete-time approximation of the continuous dynamics; for example, the forward Euler integration scheme used above would give  $f_d(\mathbf{x}[n], \mathbf{u}[n]) = \mathbf{x}[n] + f(\mathbf{x}[n], \mathbf{u}[n])dt$ . Then we have

$$\frac{\partial J}{\partial \mathbf{u}_k} = \frac{\partial \ell_f(\mathbf{x}[N])}{\partial \mathbf{u}_k} + \sum_{n=0}^{N-1} \left( \frac{\partial \ell(\mathbf{x}[n], \mathbf{u}[n])}{\partial \mathbf{x}[n]} \frac{\partial \mathbf{x}[n]}{\partial \mathbf{u}_k} + \frac{\partial \ell(\mathbf{x}[n], \mathbf{u}[n])}{\partial \mathbf{u}_k} \right),$$

where the gradient of the state with respect to the inputs can be computed during the "forward simulation",

$$\frac{\partial \mathbf{x}[n+1]}{\partial \mathbf{u}_k} = \frac{\partial f_d(\mathbf{x}[n], \mathbf{u}[n])}{\partial \mathbf{x}[n]} \frac{\partial \mathbf{x}[n]}{\partial \mathbf{u}_k} + \frac{\partial f_d(\mathbf{x}[n], \mathbf{u}[n])}{\partial \mathbf{u}_k}.$$

These simulation gradients can also be used in the chain rule to provide the gradients of any constraints. Note that there are a lot of terms to keep around here, on the order of (state dim)  $\times$  (control dim)  $\times$  (number of time steps). Ouch. Note also that many of these terms are zero; for instance with the Euler integration scheme above  $\frac{\partial \mathbf{u}[n]}{\partial \mathbf{u}_k} = 0$  if  $k \neq n$ . (If this looks like I'm mixing two notations here, recall that I'm using  $\mathbf{u}_k$  to represent the decision variable and  $\mathbf{u}[n]$  to represent the input used in the  $n$ th step of the simulation.)

### 10.4.2 The special case of direct shooting without state constraints

By solving for  $\mathbf{x}(\cdot)$  ourselves, we've removed a large number of constraints from the optimization. If no additional state constraints are present, and the only gradients we need to compute are the gradients of the objective, then a surprisingly efficient algorithm emerges. I'll give the steps here without derivation, but will derive it in the Pontryagin section below:

1. Simulate forward:

$$\mathbf{x}[n+1] = f_d(\mathbf{x}[n], \mathbf{u}_n),$$

from  $\mathbf{x}[0] = \mathbf{x}_0$ .

2. Calculate backwards:

$$\lambda[n-1] = \frac{\partial \ell(\mathbf{x}[n], \mathbf{u}[n])}{\partial \mathbf{x}[n]}^T + \frac{\partial f(\mathbf{x}[n], \mathbf{u}[n])}{\partial \mathbf{x}[n]}^T \lambda[n],$$

from  $\lambda[N-1] = \frac{\partial \ell_f(\mathbf{x}[N])}{\partial \mathbf{x}[N]}$ .

3. Extract the gradients:

$$\frac{\partial J}{\partial \mathbf{u}[n]} = \frac{\partial \ell(\mathbf{x}[n], \mathbf{u}[n])}{\partial \mathbf{u}[n]} + \lambda[n]^T \frac{\partial f(\mathbf{x}[n], \mathbf{u}[n])}{\partial \mathbf{u}[n]},$$

with  $\frac{\partial J}{\partial \mathbf{u}_k} = \sum_n \frac{\partial J}{\partial \mathbf{u}[n]} \frac{\partial \mathbf{u}[n]}{\partial \mathbf{u}_k}$ .

Here  $\lambda[n]$  is a vector the same size as  $\mathbf{x}[n]$  which has an interpretation as  $\lambda[n] = \frac{\partial J}{\partial \mathbf{x}[n+1]}^T$ . The equation governing  $\lambda$  is known as the *adjoint equation*, and it represents a dramatic efficiency improvement over calculating the huge number of simulation gradients described above. In case you are interested, yes the adjoint equation is exactly the *backpropagation algorithm* that is famous in the neural networks literature, or more generally a bespoke version of reverse-mode *automatic differentiation*. (The previous section would correspond to forward-mode automatic differentiation.)

### 10.4.3 Penalty methods and the Augmented Lagrangian

In fact, in recent years we have seen a surge in popularity in robotics for doing trajectory optimization using (often special-purpose) solvers for unconstrained trajectory optimization, where the constrained problems are transformed into unconstrained problem via penalty methods.

Manually adding penalty terms to the cost function with user-specified cost weighting can be nasty business. Often this generates a significant amount of effort in "cost function tuning"; setting the weights too large can cause the problems to become numerically ill-conditioned, but setting them too small can allow unwanted constraint violations. The *augmented Lagrangian* approach offers a principled solution to this problem, by automatically scheduling the weights in order to achieve "essentially exact" satisfaction of the constraints with finite penalties [9], [10, Ch. 17].

Methods based on the augmented Lagrangian are particularly popular for trajectory optimization these days [11, 12].

### 10.4.4 Zero-order optimization

Cross-entropy method (CEM) [13] and Model-predictive path-integral control (MPPI) [14].

In the last few years, we've seen trajectory optimization playing a major role in "model-based reinforcement learning", especially when the model is represented using a deep neural network (e.g. [15, 16]). Despite the fact that neural network toolboxes can provide efficient gradients, the gradient-free trajectory optimization methods described here normally the tool of choice. There are a few possible explanations for this. The trajectory optimization problem has local minima, and the zero-order optimizers do provide some aspect of "global optimization" (at least they tend to explore multiple possible minima before converging). Neural network models fit with limited data may even have more local minima than if one used the original physics model -- the standard regression loss could be small even if the neural representation of the function is locally bumpy/non-smooth. But I think the biggest reason for the popularity of the zero-order methods is that they are trivially parallelizable, and very nicely compatible with the existing GPU neural-network workflows. When rolling out 10,000 samples in parallel costs approximately the same as rolling out 1, then this certainly makes the zero-order methods more appealing.

### 10.4.5 Getting good solutions... in practice.

As you begin to play with these algorithms on your own problems, you might feel like you're on an emotional roller-coaster. You will have moments of incredible happiness

-- the solver may find very impressive solutions to highly nontrivial problems. But you will also have moments of frustration, where the solver returns an awful solution, or simply refuses to return a solution (saying "infeasible"). The frustrating thing is, you cannot distinguish between a problem that is actually infeasible, vs. the case where the solver was simply stuck in a local minima.

So the next phase of your journey is to start trying to "help" the solver along. There are two common approaches.

The first is tuning your cost function -- some people spend a lot of time adding new elements to the objective or adjusting the relative weight of the different components of the objective. This is a slippery slope, and I tend to try to avoid it (possibly to a fault; other groups tend to put out more compelling videos!).

The second approach is to give a better initial guess to your solver to put it in the vicinity of the "right" local minimal. I find this approach more satisfying, because for most problems I think there really is a "correct" formulation for the objective and constraints, and we should just aim to find the optimal solution. Once again, we do see a difference here between the direct shooting algorithms and the direct transcription / collocation algorithms. For shooting, we can only provide the solver with an initial guess for  $\mathbf{u}(\cdot)$ , whereas the other methods allow us to also specify an initial guess for  $\mathbf{x}(\cdot)$  directly. I find that this can help substantially, even with very simple initializations. In the direct collocation examples for the swing-up problem of the Acrobot and cart-pole, simply providing the initial guess for  $\mathbf{x}(\cdot)$  as a straight line trajectory between the start and the goal was enough to help the solver find a good solution; in fact it was necessary.

I'm old enough now, though, that I've tired of playing games with finicky solvers that require hand-holding. In recent work, my group has been working on more robust "global" solutions to the motion planning problem, based on an idea call Graphs of Convex Sets (GCS) [17]. The initial transcriptions of motion planning into GCS were focused on kinematic trajectory optimization (as needed in robotic manipulation) [18], but we are growing the capability towards dynamic (underactuated) trajectory optimization. [You can read more about here.](#)

## 10.5 LOCAL TRAJECTORY FEEDBACK DESIGN

Once we have obtained a locally optimal trajectory from trajectory optimization, we have found an open-loop trajectory that (at least locally) minimizes our optimal control cost. Up to numerical tolerances, this pair  $\mathbf{u}_0(t), \mathbf{x}_0(t)$  represents a feasible solution trajectory of the system. But we haven't done anything, yet, to ensure that this trajectory is locally stable.

In fact, there are a few notable approximations that we've already made to get to this point: the integration accuracy of our trajectory optimization tends to be much less than the accuracy used during forward simulation (we tend to take bigger time steps during optimization to avoid adding too many decision variables), and the default convergence tolerance from the optimization toolboxes tend to satisfy the dynamic constraints only to around  $10^{-6}$ . As a result, if you were to simulate the optimized control trajectory directly *even from the exact initial conditions* used in / obtained from trajectory optimization, you might find that the state trajectory diverges from your planned trajectory.

There are a number of things we can do about this. It is possible to evaluate the local stability of the trajectory during the trajectory optimization, and add a cost or constraint that rewards open-loop stability (e.g. [19, 20]). This can be very effective (though it does tend to be expensive). But open-loop stability is a quite restrictive notion. A potentially more generally useful approach is to design a feedback controller to regulate the system back to the planned trajectory.

## 10.5.1 Finite-horizon LQR

We have already developed one approach for [trajectory stabilization in the LQR chapter](#). This is one of my favorite approaches to trajectory feedback, because it provides a (numerically) closed-form solution for the controller,  $\mathbf{K}(t)$ , and even comes with a time-varying quadratic cost-to-go function,  $S(t)$ , that can be used for Lyapunov analysis.

The basic procedure is to create a time-varying linearization along the trajectory in the error coordinates:  $\bar{\mathbf{x}}(t) = \mathbf{x}(t) - \mathbf{x}_0(t)$ ,  $\bar{\mathbf{u}}(t) = \mathbf{u}(t) - \mathbf{u}_0(t)$ , and  $\dot{\bar{\mathbf{x}}}(t) = \mathbf{A}(t)\bar{\mathbf{x}}(t) + \mathbf{B}(t)\bar{\mathbf{u}}(t)$ . This linearization uses all of the same gradients of the dynamics that we have been using in our trajectory optimization algorithms. Once we have the time-varying linearization, then we can apply finite-horizon LQR (see the [LQR chapter](#) for the details).

A major virtue of this approach is that we can proceed immediately to verifying the performance of the closed-loop system under the LQR policy. Specifically, we can apply the finite-time reachability analysis to obtain "funnels" that certify a desired notion of invariance -- guaranteeing that trajectories which start near the planned trajectory will stay near the planned trajectory. Please see the [finite-time reachability analysis](#) section for those details. We will put all of these ideas together in the perching case-study below.

## 10.5.2 Model-Predictive Control

The maturity, robustness, and speed of solving trajectory optimization using convex optimization leads to a beautiful idea: if we can optimize trajectories quickly enough, then we can use our trajectory optimization as a feedback policy. The recipe is simple: (1) measure the current state, (2) optimize a trajectory from the current state, (3) execute the first action from the optimized trajectory, (4) let the dynamics evolve for one step and repeat. This recipe is known as *model-predictive control* (MPC).

Despite the very computational nature of this controller (there is no closed-form representation of this policy; it is represented only implicitly as the solution of the optimization), there is a bounty of theoretical and algorithmic results on MPC [21, 22]. The theoretical and practical aspects of Linear MPC are so well understood today that it is considered the de-facto generalization of LQR for controlling a linear system subject to (linear) constraints. And there are a few core ideas that practitioners should really understand.

### Receding-horizon MPC

One core idea is the concept of *receding-horizon* MPC. Since our trajectory optimization problems are formulated over a finite-horizon, we can think each optimization as reasoning about the next  $N$  time steps. If our true objective is to optimize the performance over a horizon longer than  $N$  (e.g. over the infinite horizon), then it is standard to continue solving for an  $N$  step horizon on each evaluation of the controller. In this sense, the total horizon under consideration continues to move forward in time (e.g. to recede).

### Recursive feasibility

Some care must be taken in receding-horizon formulations because on each new step we are introducing new costs and constraints into the problem (the ones that would have been associated with time  $N + 1$  on the previous solve) -- it would be very bad to march forward in time solving convex optimization problems only to suddenly encounter a situation where the solver returns "infeasible!". One can design MPC formulations that guarantee *recursive feasibility* -- e.g. guarantee that if a feasible solution is found at time  $n$ , then the solver will also find a feasible solution at time  $n + 1$ .

Perhaps the simplest mechanism for guaranteeing recursive feasibility in an optimization for stabilizing a fixed point,  $(\mathbf{x}^*, \mathbf{u}^*)$ , is to add a final-value constraint to the receding horizon,  $\mathbf{x}[N] = \mathbf{x}^*$ . This idea is simple but important. Considering the trajectories/constraints in absolute time, then on step  $k$  of the algorithm, we are optimizing for  $\mathbf{x}[k], \dots, \mathbf{x}[k+N]$ , and  $\mathbf{u}[k], \dots, \mathbf{u}[k+N-1]$ ; let us say that we have found a feasible solution for this problem. The danger in receding-horizon control is that when we shift to the next step  $(k+1)$  we introduce constraints on the system at  $\mathbf{x}[k+N+1]$  for the first time. But if our feasible solution in step  $k$  had  $\mathbf{x}[k+N] = \mathbf{x}^*$ , then we know that setting  $\mathbf{x}[k+N+1] = \mathbf{x}^*, \mathbf{u}[k+N] = \mathbf{u}^*$  is guaranteed to provide a feasible solution to the new optimization problem in step  $k+1$ . With feasibility guaranteed, the solver is free to search for a lower-cost solution (which may be available now because we've shifted the final-value constraint further into the future). It is also possible to formulate MPC problems that guarantee recursive feasibility even in the presence of modeling errors and disturbances (c.f. [23]).

## MPC and Lyapunov functions

Even though the MPC policy is only defined implicitly as the result of an optimization problem, it is still possible to make Lyapunov stability arguments. If, in the receding-horizon MPC setting, in addition to recursive feasibility (of the *constraints*) we have the property that the *cost* added at step  $k+N+1$  is less than or equal to the cost removed corresponding to step  $k$ . Then the optimal cost,  $J(\mathbf{x}, k)$ , of the trajectory optimization is a Lyapunov function:

$$\forall \mathbf{x}, k, \quad J^*(\mathbf{f}(\mathbf{x}, \mathbf{u}^*), k+1) \leq J^*(\mathbf{x}, k).$$

Once again, a particularly simple way to achieve this is to have a final-value constraint at the  $N$ th step of every optimization, which gets me to an equilibrium configuration with zero cost. But, again, there are ways to extend this Lyapunov argument even in the presence of modeling errors and disturbances (c.f. [23]).

Unlike the nice explicit time-varying quadratic Lyapunov functions that we get from finite-horizon LQR, this Lyapunov function in general does not admit a compact representation (this is the object of study in *explicit MPC*), and is more difficult to use with e.g. sums-of-squares optimization.

## 10.6 CASE STUDY: A GLIDER THAT CAN LAND ON A PERCH LIKE A BIRD

From 2008 til 2014, my group conducted a series of increasingly sophisticated investigations [24, 25, 26, 27, 28, 29] which asked the question: can a fixed-wing UAV land on a perch like a bird?

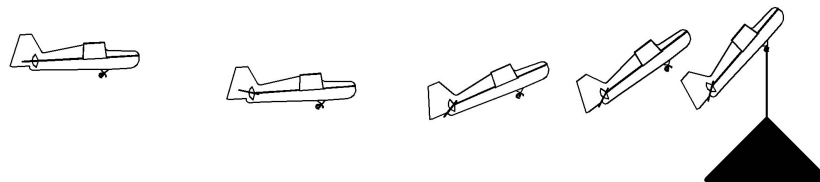


Figure 10.3 - Basic setup for the glider landing on a perch.

At the outset, this was a daunting task. When birds land on a perch, they pitch up and expose their wings to an "angle-of-attack" that far exceeds the typical flight envelope. Airplanes traditionally work hard to avoid this regime because it leads to aerodynamic "stall" -- a sudden loss of lift after the airflow separates from the wing. But this loss of lift is also accompanied by a significant increase in drag, and birds exploit this when they rapidly decelerate to land on a perch. Post-stall aerodynamics are a challenge for control because (1) the aerodynamics are time-varying (characterized by periodic vortex shedding) and nonlinear, (2) it is much harder to build accurate models of this flight regime, at least in a wind tunnel, and (3) stall implies a loss of attached flow on the wing and therefore on the control



surfaces, so a potentially large reduction in control authority.

We picked the project initially thinking that it would be a nice example for model-free control (like reinforcement learning -- since the models were unknown). In the end, however, it turned out to be the project that really taught me about the power of model-based trajectory optimization and linear optimal control. By conducting dynamic system identification experiments in a motion capture environment, we were able to fit both surprisingly simple models (based on flat-plate theory) to the dynamics[24], and also more accurate models using "neural-network-like" terms to capture the residuals between the model and the data [29]. This made model-based control viable, but the dynamics were still complex -- while trajectory optimization should work, I was quite dubious about the potential for regulating to those trajectories with only linear feedback.

I was wrong. Over and over again, I watched time-varying linear quadratic regulators take highly nontrivial corrective actions -- for instance, dipping down early in the trajectory to gain kinetic energy or tipping up to dump energy out of the system -- in order to land on the perch at the final time. Although the quality of the linear approximation of the dynamics did degrade the farther that we got from the nominal trajectory, the validity of the controller dropped off much less quickly (even as the vector field changed, the direction that the controller needed to push did not). This was also the thinking that got me initially so interested in understanding the regions of attraction of linear control on nonlinear systems.

In the end, the experiments were very successful. We started searching for the "simplest" aircraft that we could build that would capture the essential control dynamics, reduce complexity, and still accomplish the task. We ended up building a series of flat-plate foam gliders (no propeller) with only a single actuator to control the elevator. We added dihedral to the wings to help the aircraft stay in the longitudinal plane. The simplicity of these aircraft, plus the fact that they could be understood through the lens of quite simple models makes them one of my favorite canonical underactuated systems.

Figure 10.4 - The original perching experiments from [24] in a motion capture arena with a simple rope acting as the perch. The main videos were shot with high-speed cameras; an entire perching trajectory takes about .8 seconds.



### 10.6.1 The Flat-Plate Glider Model

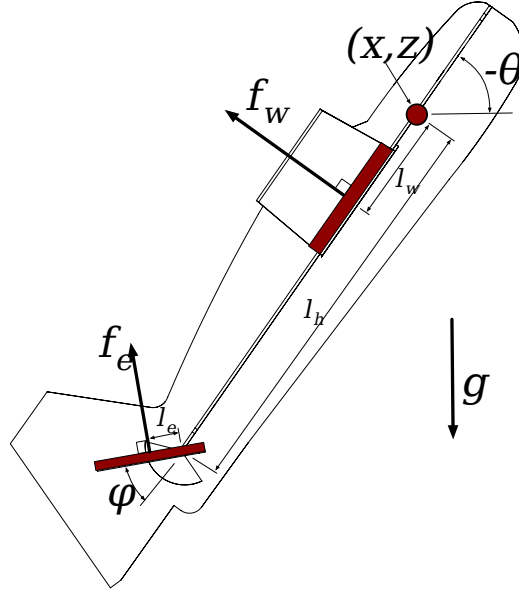


Figure 10.5 - The flat-plate glider model. Note that traditionally aircraft coordinates are chosen so that positive pitch brings the nose up; but this means that positive  $z$  is down (to have a right-handed system). For consistency, I've chosen to stick with the vehicle coordinate system that we use throughout the text -- positive  $z$  is up, but positive pitch is down.

In our experiments, we found the dynamics of our aircraft [24] were captured very well by the so-called "flat plate model" [30, Ch.4, Sec.6]. In flat plate theory lift and drag forces of a wing are summarized by a single force at the center-of-pressure of the wing acting normal to the wing, with magnitude:

$$f_n(S, \mathbf{n}, \mathbf{v}) = \rho S \sin \alpha |\mathbf{v}|^2 = -\rho S (\mathbf{n} \cdot \mathbf{v}) |\mathbf{v}|,$$

where  $\rho$  is the density of the air,  $S$  is the area of the wing,  $\alpha$  is the angle of attack of the surface,  $\mathbf{n}$  is the normal vector of the lifting surface, and  $\mathbf{v}$  is the velocity of the center of pressure relative to the air. This corresponds to having lift and drag coefficients

$$c_{\text{lift}} = 2 \sin \alpha \cos \alpha, \quad c_{\text{drag}} = 2 \sin^2 \alpha.$$

In our glider model, we summarize all of the aerodynamic forces by contributions of two lifting surfaces, the wing (including some contribution from the horizontal fuselage) denoted by subscript  $w$  and the elevator denoted by subscript  $e$ , with centers at  $\mathbf{p}_w = [x_w, z_w]^T$  and  $\mathbf{p}_e = [x_e, z_e]^T$  given by the kinematics:

$$\mathbf{p}_w = \mathbf{p} - l_w \begin{bmatrix} c_\theta \\ -s_\theta \end{bmatrix}, \quad \mathbf{p}_e = \mathbf{p} - l_h \begin{bmatrix} c_\theta \\ -s_\theta \end{bmatrix} - l_e \begin{bmatrix} c_{\theta+\phi} \\ -s_{\theta+\phi} \end{bmatrix},$$

where the origin of our vehicle coordinate system,  $\mathbf{p} = [x, z]^T$ , is chosen to be the center of mass. We assume still air, so  $\mathbf{v} = \dot{\mathbf{p}}$  for both the wing and the elevator. We assume that the elevator is massless, and the actuator controls velocity directly (note that this breaks our "control affine" structure, but is more realistic for the tiny hobby servos we were dealing with). This gives us a total of 7 state variables  $\mathbf{x} = [x, z, \theta, \phi, \dot{x}, \dot{z}, \dot{\theta}]^T$  and one control input  $\mathbf{u} = \dot{\phi}$ . The resulting equations of motion are:

$$\begin{aligned}
\mathbf{n}_w &= \begin{bmatrix} s_\theta \\ c_\theta \end{bmatrix}, & \mathbf{n}_e &= \begin{bmatrix} s_{\theta+\phi} \\ c_{\theta+\phi} \end{bmatrix}, \\
f_w &= f_n(S_w, \mathbf{n}_w, \dot{\mathbf{p}}_w), & f_e &= f_n(S_e, \mathbf{n}_e, \dot{\mathbf{p}}_e), \\
\ddot{x} &= \frac{1}{m}(f_w s_\theta + f_e s_{\theta+\phi}), \\
\ddot{z} &= \frac{1}{m}(f_w c_\theta + f_e c_{\theta+\phi}) - g, \\
\ddot{\theta} &= \frac{1}{I}(l_w f_w + (l_h c_\phi + l_e) f_e).
\end{aligned}$$

## 10.6.2 Trajectory optimization

## 10.6.3 Trajectory stabilization

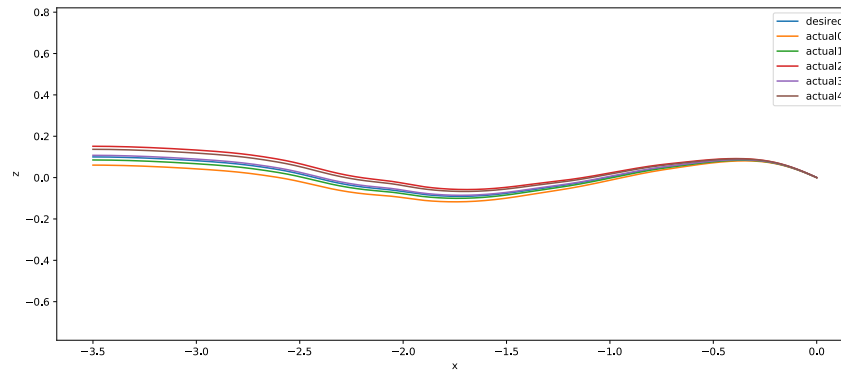


Figure 10.6 - Rollouts with the finite-horizon LQR regulator.

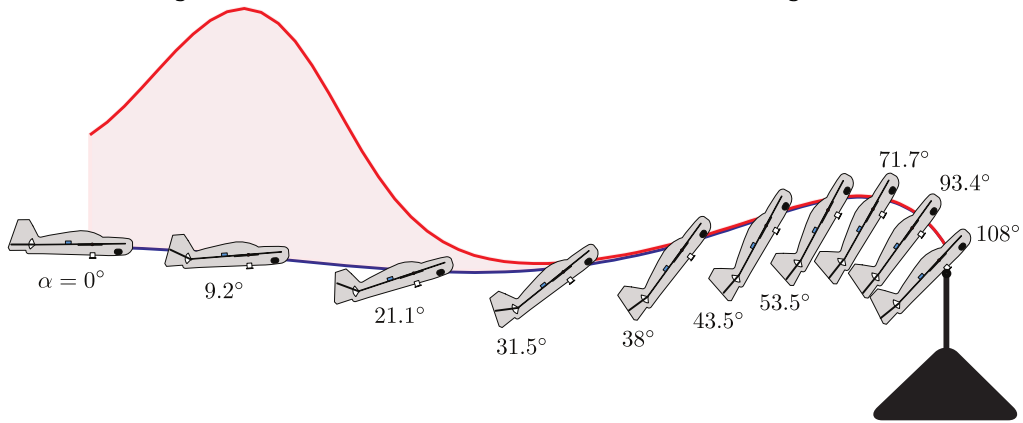
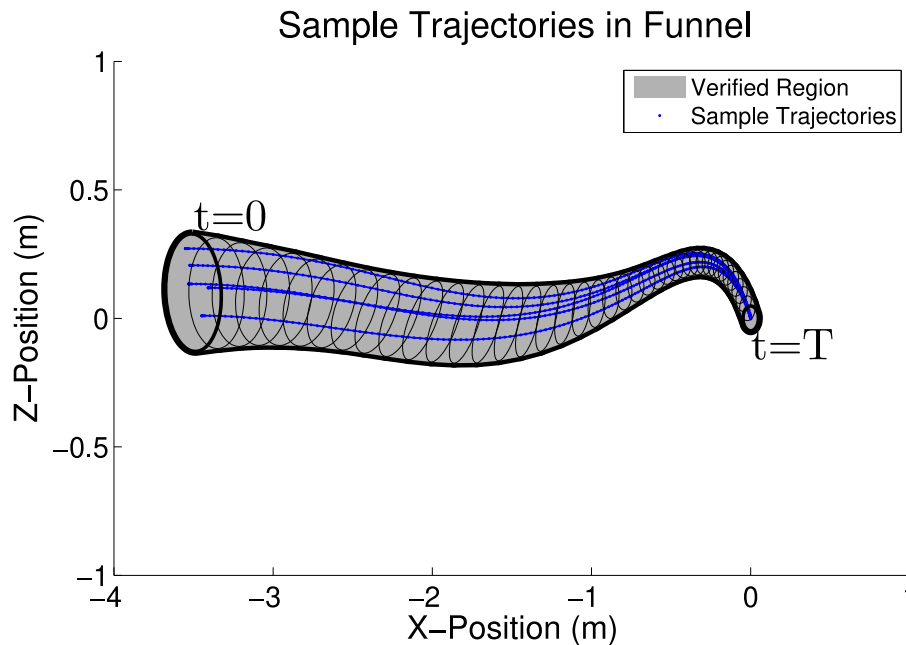


Figure 10.7 - Rendering of the inverse of the largest eigenvalue of  $\mathbf{S}(t)$  overlaid on the perching trajectory, demonstrating the regions of poor controllability as the velocity of the vehicle approaches zero [25].

### 10.6.4 Trajectory funnels



### 10.6.5 Beyond a single trajectory

The linear controller around a nominal trajectory was surprisingly effective, but it's not enough. We'll return to this example again when we talk about "feedback motion planning", in order to discuss how to find a controller that can work for many more initial conditions -- ideally all of the initial conditions of interest for which the aircraft is capable of getting to the goal.

## 10.7 PONTRYAGIN'S MINIMUM PRINCIPLE

The tools that we've been developing for numerical trajectory optimization are closely tied to theorems from (analytical) optimal control. Let us take one section to appreciate those connections.

What precisely does it mean for a trajectory,  $\mathbf{x}(\cdot), \mathbf{u}(\cdot)$ , to be locally optimal? It means that if I were to perturb that trajectory in any way (e.g. change  $\mathbf{u}_3$  by  $\epsilon$ ), then I would either incur higher cost in my objective function or violate a constraint. For an unconstrained optimization, a *necessary condition* for local optimality is that the gradient of the objective at the solution be exactly zero. Of course the gradient can also vanish at local maxima or saddle points, but it certainly must vanish at local minima. We can generalize this argument to constrained optimization using *Lagrange multipliers*.

### 10.7.1 Lagrange multiplier derivation of the adjoint equations

Let us use Lagrange multipliers to derive the necessary conditions for our constrained trajectory optimization problem in discrete time

$$\begin{aligned} \min_{\mathbf{x}[\cdot], \mathbf{u}[\cdot]} \ell_f(\mathbf{x}[N]) + \sum_{n=0}^{N-1} \ell(\mathbf{x}[n], \mathbf{u}[n]), \\ \text{subject to } \mathbf{x}[n+1] = f_d(\mathbf{x}[n], \mathbf{u}[n]). \end{aligned}$$

Formulate the Lagrangian,

$$L(\mathbf{x}[\cdot], \mathbf{u}[\cdot], \lambda[\cdot]) = \ell_f(\mathbf{x}[N]) + \sum_{n=0}^{N-1} \ell(\mathbf{x}[n], \mathbf{u}[n]) + \sum_{n=0}^{N-1} \lambda^T[n] (f_d(\mathbf{x}[n], \mathbf{u}[n]) - \mathbf{x}[n+1]),$$

and set the derivatives to zero to obtain the adjoint equation method described for the shooting algorithm above:

$$\begin{aligned} \forall n \in [0, N-1], \frac{\partial L}{\partial \lambda[n]} &= f_d(\mathbf{x}[n], \mathbf{u}[n]) - \mathbf{x}[n+1] = 0 \Rightarrow \mathbf{x}[n+1] = f(\mathbf{x}[n], \mathbf{u}[n]) \\ \forall n \in [0, N-1], \frac{\partial L}{\partial \mathbf{x}[n]} &= \frac{\partial \ell(\mathbf{x}[n], \mathbf{u}[n])}{\partial \mathbf{x}} + \lambda^T[n] \frac{\partial f_d(\mathbf{x}[n], \mathbf{u}[n])}{\partial \mathbf{x}} - \lambda^T[n-1] = 0 \\ &\Rightarrow \lambda[n-1] = \frac{\partial \ell(\mathbf{x}[n], \mathbf{u}[n])}{\partial \mathbf{x}}^T + \frac{\partial f_d(\mathbf{x}[n], \mathbf{u}[n])}{\partial \mathbf{x}}^T \lambda[n]. \\ \frac{\partial L}{\partial \mathbf{x}[N]} &= \frac{\partial \ell_f}{\partial \mathbf{x}}^T - \lambda^T[N-1] = 0 \Rightarrow \lambda[N-1] = \frac{\partial \ell_f}{\partial \mathbf{x}}^T \\ \forall n \in [0, N-1], \frac{\partial L}{\partial \mathbf{u}[n]} &= \frac{\partial \ell(\mathbf{x}[n], \mathbf{u}[n])}{\partial \mathbf{u}} + \lambda^T[n] \frac{\partial f_d(\mathbf{x}[n], \mathbf{u}[n])}{\partial \mathbf{u}} = 0. \end{aligned}$$

Therefore, if we are given an initial condition  $\mathbf{x}_0$  and an input trajectory  $\mathbf{u}[\cdot]$ , we can verify that it satisfies the necessary conditions for optimality by simulating the system forward in time to solve for  $\mathbf{x}[\cdot]$ , solving the adjoint equation backwards in time to solve for  $\lambda[\cdot]$ , and verifying that  $\frac{\partial L}{\partial \mathbf{u}[n]} = 0$  for all  $n$ . The fact that  $\frac{\partial L}{\partial \mathbf{u}} = \frac{\partial L}{\partial \mathbf{u}}$  when  $\frac{\partial L}{\partial \mathbf{x}} = 0$  and  $\frac{\partial L}{\partial \lambda} = 0$  follows from some basic results in the calculus of variations.

### 10.7.2 Necessary conditions for optimality in continuous time

You won't be surprised to hear that these necessary conditions have an analogue in continuous time. I'll state it here without derivation. Given the initial conditions,  $\mathbf{x}_0$ , a continuous dynamics,  $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ , and the instantaneous cost  $\ell(\mathbf{x}, \mathbf{u})$ , for a trajectory  $\mathbf{x}(\cdot), \mathbf{u}(\cdot)$  defined over  $t \in [t_0, t_f]$  to be optimal it must satisfy the conditions that

$$\begin{aligned} \forall t \in [t_0, t_f], \quad \dot{\mathbf{x}} &= f(\mathbf{x}, \mathbf{u}), & \mathbf{x}(0) &= \mathbf{x}_0 \\ \forall t \in [t_0, t_f], \quad -\dot{\lambda} &= \frac{\partial \ell}{\partial \mathbf{x}}^T + \frac{\partial f}{\partial \mathbf{x}}^T \lambda, & \lambda(T) &= \frac{\partial \ell_f}{\partial \mathbf{x}}^T \\ \forall t \in [t_0, t_f], \quad \frac{\partial \ell}{\partial \mathbf{u}} &+ \lambda^T \frac{\partial f}{\partial \mathbf{u}} &= 0. \end{aligned}$$

In fact the statement can be generalized even beyond this to the case where  $\mathbf{u}$  has constraints. The result is known as Pontryagin's minimum principle -- giving *necessary conditions* for a trajectory to be optimal.

#### Theorem 10.1 - Pontryagin's Minimum Principle

Adapted from [31]. Given the initial conditions,  $\mathbf{x}_0$ , a continuous dynamics,  $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ , and the instantaneous cost  $\ell(\mathbf{x}, \mathbf{u})$ , for a trajectory  $\mathbf{x}^*(\cdot), \mathbf{u}^*(\cdot)$  defined over  $t \in [t_0, t_f]$  to be optimal it must satisfy the conditions that

$$\begin{aligned}
\forall t \in [t_0, t_f], \quad \dot{\mathbf{x}}^* &= f(\mathbf{x}^*, \mathbf{u}^*), & \mathbf{x}^*(0) &= \mathbf{x}_0 \\
\forall t \in [t_0, t_f], \quad -\dot{\lambda}^* &= \frac{\partial \ell^T}{\partial \mathbf{x}} + \frac{\partial f^T}{\partial \mathbf{x}} \lambda^*, & \lambda^*(T) &= \frac{\partial \ell_f^T}{\partial \mathbf{x}} \\
\forall t \in [t_0, t_f], \quad \mathbf{u}^* &= \operatorname{argmin}_{\mathbf{u} \in \mathcal{U}} [\ell(\mathbf{x}^*, \mathbf{u}) + (\lambda^*)^T f(\mathbf{x}^*, \mathbf{u})].
\end{aligned}$$

Note that the terms which are minimized in the final line of the theorem are commonly referred to as the Hamiltonian of the optimal control problem,

$$H(\mathbf{x}, \mathbf{u}, \lambda, t) = \ell(\mathbf{x}, \mathbf{u}) + \lambda^T f(\mathbf{x}, \mathbf{u}).$$

It is distinct from, but inspired by, the Hamiltonian of classical mechanics. Remembering that  $\lambda$  has an interpretation as  $\frac{\partial J}{\partial \mathbf{x}}^T$ , you should also recognize it from the HJB.

## 10.8 VARIATIONS AND EXTENSIONS

### 10.8.1 Differential Flatness

There are some very important cases where nonconvex trajectory optimization can be turned back into convex trajectory optimization based on a clever change of variables. One of the key ideas in this space is the notion of "differential flatness", which is philosophically quite close to the idea of [partial feedback linearization](#) that we discussed for acrobats and cart-pole systems. But perhaps the most famous application of differential flatness, which we will explore here, is actually for quadrotors.

One of the most important lessons from partial feedback linearization, is the idea that if you have  $m$  actuators, then you basically get to control exactly  $m$  quantities of your system. Differential flatness exploits this same idea (choosing  $m$  outputs), but in the opposite direction. The idea is that, for some systems, if you give me a trajectory in those  $m$  coordinates, it may in fact dictate what all of the states/actuators must have been doing. The fact that you can only execute a subset of possible trajectories can, in this case, make trajectory planning much easier!

Let's start with an example...

#### Example 10.3 (Differential flatness for the Planar Quadrotor)

The planar quadrotor model [described earlier](#) has 3 degrees of freedom ( $x, y, \theta$ ) but only two actuators (one for each propeller). My claim is that, if you give me a trajectory for just the location of the center of mass:  $x(t), y(t), \forall t \in [t_0, t_f]$ , then I will be able to infer what  $\theta(t)$  must be over that same interval in order to be feasible. Furthermore, I can even infer  $\mathbf{u}(t)$ . There is one technical condition required: the trajectory you give me for  $x(t)$  and  $y(t)$  needs to be continuously differentiable (four times).

To see this, recall the equations of motion for this system were given by:

$$m\ddot{x} = -(u_1 + u_2) \sin \theta, \quad (1)$$

$$m\ddot{y} = (u_1 + u_2) \cos \theta - mg, \quad (2)$$

$$I\ddot{\theta} = r(u_1 - u_2) \quad (3)$$

Observe that from (1) and (2) we have

$$\frac{-m\ddot{x}}{m\ddot{y} + mg} = \frac{(u_1 + u_2) \sin \theta}{(u_1 + u_2) \cos \theta} = \tan \theta.$$

In words, given  $\ddot{x}(t)$  and  $\ddot{y}(t)$ , I can solve for  $\theta(t)$ . I can differentiate this relationship (in time) twice more to obtain  $\ddot{\theta}$ . Using (3) with (1) or (2) give us two linear equations for  $u_1$  and  $u_2$  which are easily solved.

Now you can see why we need the original trajectories to be smooth -- the solution to  $\mathbf{u}(t)$  depends on  $\ddot{\theta}(t)$  which depends on  $\frac{d^4 x(t)}{dt^4}$  and  $\frac{d^4 y(t)}{dt^4}$ ; we need those derivatives to exist along the entire trajectory.

What's more -- if we ignore input limits for a moment -- *any* sufficiently smooth trajectory of  $x(t), y(t)$  is feasible, so if I can simply find one that avoids the obstacles, then I have designed my state trajectory. As we will see, optimizing even high-degree piecewise-polynomials is actually an easy problem (it works out to be a quadratic program), assuming the constraints are convex. In practice, this means that once you have determined whether to go left or right around the obstacles, trajectory design is easy and fast.

I've coded up a simple example of that for you here:

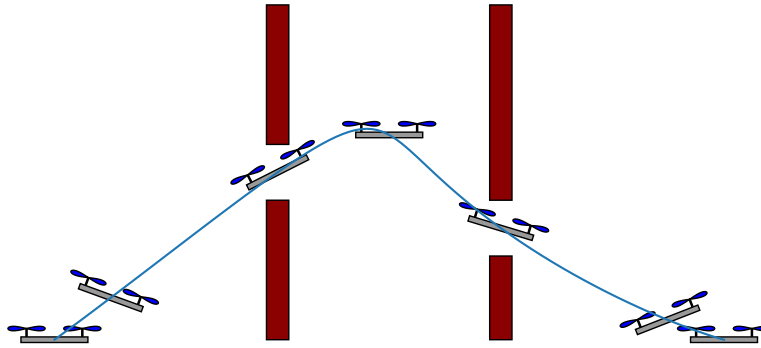


Figure 10.9 - Differential flatness for the planar quadrotor -- by solving a simple optimization to find a smooth trajectory for  $x(t)$  and  $y(t)$ , I can back out  $\theta(t)$  and even  $\mathbf{u}(t)$ .

The example above demonstrates that the planar quadrotor system is differentially flat in the outputs  $x(t)$ ,  $y(t)$ . More generally, if we have a dynamical system

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}),$$

and we design some output coordinates (essentially "task-space"):

$$\mathbf{z}(t) = h\left(\mathbf{x}, \mathbf{u}, \frac{d\mathbf{u}}{dt}, \dots, \frac{d^k \mathbf{u}}{dt^k}\right),$$

such that we can write the  $\mathbf{x}$  and  $\mathbf{u}$  purely as a function of the output and its time derivatives,

$$\begin{aligned}\mathbf{x}(t) &= \mathbf{x}\left(\mathbf{z}, \frac{d\mathbf{z}}{dt}, \dots, \frac{d^k \mathbf{z}}{dt^k}\right), \\ \mathbf{u}(t) &= \mathbf{u}\left(\mathbf{z}, \frac{d\mathbf{z}}{dt}, \dots, \frac{d^k \mathbf{z}}{dt^k}\right),\end{aligned}$$

then we say that the system  $f$  is differentially flat in the outputs  $\mathbf{z}$  [32]. And the

requirement for showing that a system is differentially flat in those outputs is to write the function which solves for  $\mathbf{x}(t)$  and  $\mathbf{u}(t)$  as a function of only  $\mathbf{z}(t)$  and its time derivatives.

I'm not aware of any numerical recipes for showing that a system is differentially flat nor for finding potential flat outputs, but I admit I haven't worked on it nor looked for those recipes. That would be interesting! I think of differential flatness as a property that one must find for your system -- typically via a little mechanical intuition and a lot of algebra. Once found, it can be very useful.

#### **Example 10.4 (Differential flatness for the 3D Quadrotor)**

Probably the most famous example of differential flatness is on the full 3D quadrotors. [33] showed that the 3D quadrotor is differentially flat in the outputs  $\{x, y, z, \theta_{yaw}\}$ . They used this, to dramatic effect, to perform all sorts of acrobatics. The resulting videos are awesome (and probably deserve a lot of credit for the widespread popularity of quadrotors in academia and industry over the next few years).

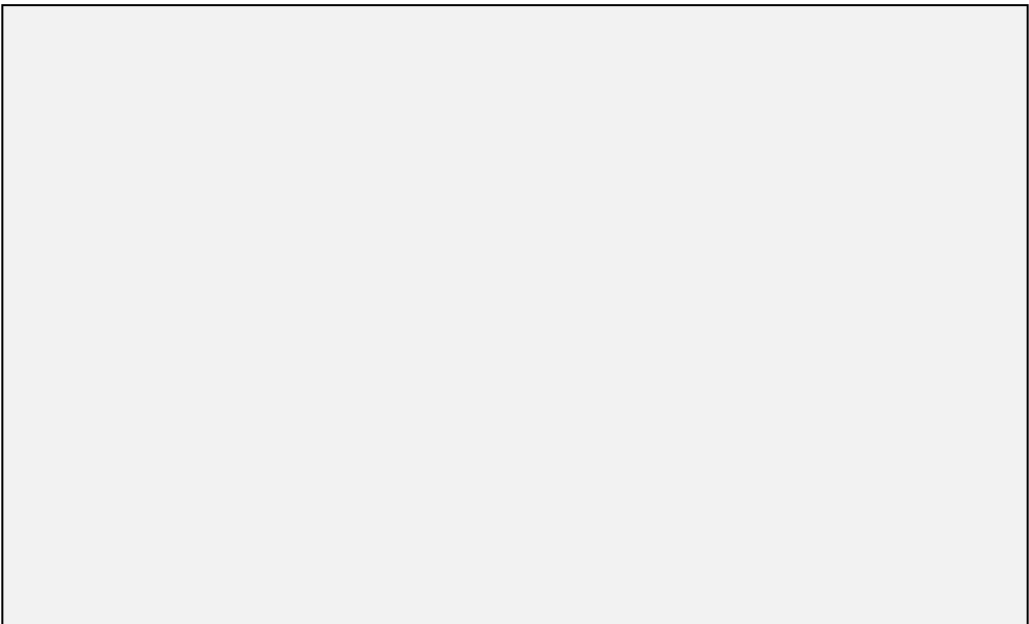


Figure 10.10 - Aggressive quadrotor trajectories using differential flatness from [33].

A few things to note about these examples, just so we also understand the limitations. First, the technique above is great for designing trajectories, but additional work is required to stabilizing those trajectories (we'll cover that topic in more detail later in the notes). Trajectory stabilization benefits greatly from good state estimation, and the examples above were all performed in a motion capture arena. Also, the "simple" version of the trajectory design that is fast enough to be used for flying through moving hoops is restricted to convex optimization formulations -- which means one has to hard-code apriori the decisions about whether to go left or right / up or down around each obstacle.

### 10.8.2 Iterative LQR and Differential Dynamic Programming

There is another approach to trajectory optimization (at least for initial-value problems) that has gotten quite popular lately. Iterative LQR (iLQR)[34] also known as Sequential Linear Quadratic optimal control) [35]. The idea is simple enough: given an initial guess at the input and state trajectory, make a linear approximation of the dynamics and a quadratic approximation of the cost function. Then compute and simulate the time-varying LQR controller to find a new input and state trajectory. Repeat until convergence.

The motivation behind iterative LQR is quite appealing -- it leverages the surprising structure of the Riccati equations to come up with a second-order update to the trajectory after a single backward pass of the Riccati equation followed by a forward simulation. Anecdotally, the convergence is fast and robust. Numerous applications...[36, 37].

One key limitation of iLQR is that it only supports unconstrained trajectory optimization -- at least via the Riccati solution. The natural extension for constrained optimization would be to replace the Riccati solution with an iterative linear model-predictive control (MPC) optimization; this would result in a quadratic program and is very close to what is happening in SQP. But a more common approach in the current robotics literature is to add any constraints into the objective function using penalty methods, especially using Augmented Lagrangian.



Differential Dynamic Programming (DDP) [38] is inspired by the idea of solving locally *quadratic* approximations of the HJB equations. The resulting algorithm is almost identical to iLQR; the only difference is that DDP uses a second-order approximation of the plant dynamics instead of a first-order (linear) one. The conventional wisdom is that taking these extra plant derivatives is probably not worth the extra cost of computing them; in most cases iLQR converges just as quickly as DDP (though the details will be problem dependent). Unfortunately, despite the distinction between these algorithms being unambiguous and uncontroversial, you will find many papers that incorrectly write DDP when they are actually using iLQR.

### 10.8.3 Mixed-integer convex optimization for non-convex constraints

#### Example 10.5 (Mixed-integer planning around obstacles)

[39], [40], [41, 42].

Recent work makes a much stronger connection between the optimization approaches to graph search with our continuous optimization [17]. In this work, the authors have gone to great lengths to produce *"strong" mixed-integer formulations with extremely tight convex relaxations* (solving a convex relaxation of the problem is often sufficient to solve the original problem). This approach provides excellent trajectory optimization solutions to minimum-time problems (with affine dynamics), and/or hybrid systems described as piecewise-affine systems (PWA) [17]. It also forms the basis for recent work on kinematic trajectory optimization around obstacles [18].

### 10.8.4 Explicit model-predictive control

We do actually understand what the optimal policy of the inequality-constrained LQR problem looks like, thanks to work on "explicit MPC" [43] -- the optimal policy is now piecewise-linear (though still continuous), with each piece described by a polytope, and the optimal cost-to-go is piecewise-quadratic on the same polytopes. Unfortunately, the number of pieces grows exponentially with the number of constraints and the horizon of the problem, making it impractical to compute for all but very small problems. There are, however, a number of promising approaches to approximate explicit MPC (c.f. [44]).

## 10.9 EXERCISES

### Exercise 10.1 (Direct Shooting vs Direct Transcription)

In this coding exercise we explore in detail the *computational advantages of direct transcription over direct shooting methods*. The exercise can be completed entirely in . To simplify the analysis, we will apply these two methods to a finite-horizon LQR problem. You are asked to complete four pieces of code:

- Use the given implementation of direct shooting to analyze the numerical conditioning of this approach.
- Complete the given implementation of the direct transcription method.
- Verify that the cost to go from direct trascription approaches the LQR cost to go as the time horizon grows.
- Analyze the numerical conditioning of direct transcription.
- Implement the dynamic programming recursion (a.k.a. "Riccati

recursion") to efficiently solve the linear algebra underlying the direct transcription method.

### **Exercise 10.2 (Orbital Transfer via Trajectory Optimization)**

For this exercise you will work exclusively in . You are asked to find, via nonlinear trajectory optimization, a path that efficiently transfers a rocket from the Earth to Mars, while avoiding a cloud of asteroids. The skeleton of the optimization problem is already there, but several important pieces are missing. More details are in the notebook, but you will need to:

- a. Enforce the maximum thrust constraints.
- b. Enforce the maximum velocity constraints.
- c. Ensure that the rocket does not collide with any of the asteroids.
- d. Add an objective function to minimize fuel consumption.

### **Exercise 10.3 (Iterative Linear Quadratic Regulator: Theory)**

Please read sections 1 and 2 of this paper [45], paying close attention to section 2 in particular. Then answer the following questions in your own words; 1-3 sentences per part should suffice.

- a. What optimization problem is iLQR solving? What is the output of the algorithm?
- b. Explain what the backward pass does and what purpose it serves.
- c. Explain what the forward pass does and what purpose it serves.
- d. Explain how the two interact with each other in an iterative manner. Do you expect the algorithm to converge? Why?
- e. Explain the difference between DDP and iLQR.
- f. Suppose the system dynamics aren't smooth (i.e., not continuously differentiable), but are still available in closed form. Can you still apply the algorithm? Would the backward pass still work? What about the forward pass?

### **Exercise 10.4 (Iterative Linear Quadratic Regulator: Implementation)**

The exercise is self-contained in . In this exercise you will derive and implement the iterative Linear Quadratic Regulator (iLQR). You will evaluate it's functionality by planning trajectories for an autonomous vehicle. You will need to:

- a. Define an integrator for continuous dynamics.
- b. Compute a trajectory given an initial state and a control trajectory.
- c. Sum up the total cost of that trajectory.
- d. Derive the coefficients of the quadratic Q-function.
- e. Optimize for the optimal control law.
- f. Derive the update of the value function backwards in time.
- g. Implement the forward pass of iLQR.
- h. Implement the backward pass of iLQR.

## Exercise 10.5 (Differential Flatness)

Consider a planar rigid body of mass  $m$  and moment of inertia  $I$  moving under the influence of gravity and forces  $F_1$  and  $F_2$  acting at point  $P$  of the body (shown in image). The state of the body at any time is defined by  $\mathbf{x} = (x, y, \theta)$  where  $x$  and  $y$  are the COM positions and  $\theta$  is the orientation of the body, respectively. The equations of motion are given by:

$$\begin{aligned} m\ddot{x} &= F_1 \cos \theta + F_2 \sin \theta \\ m\ddot{y} &= F_1 \sin \theta - F_2 \cos \theta - mg \\ I\ddot{\theta} &= rF_1 \end{aligned}$$

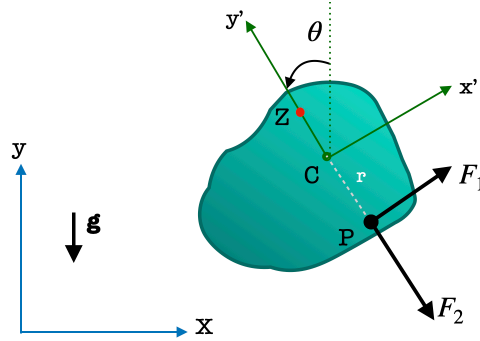


Figure 10.11 - Planar rigid body controlled by body fixed forces.

- a. Show that the system is differentially flat in the output  $Z$ , the center of oscillation, given by  $\mathbf{z} = (z_1, z_2)$

$$z_1 = x - \frac{I}{mr} \sin \theta \quad z_2 = y + \frac{I}{mr} \cos \theta$$

In other words, given a sufficiently differentiable trajectory of the center of oscillation,  $\mathbf{z}(t)$ , show that we can uniquely determine the body's trajectory  $\mathbf{x}(t)$ , upto an ambiguity of  $\pi$  in  $\theta$ .

*Hint:* Example 10.3 is a good reference. Eliminate the forces from one of the equations of motion. Use  $\ddot{z}_1$  and  $\ddot{z}_2$  to substitute for  $\ddot{x}$  and  $\ddot{y}$ .

- b. Suppose we can control the magnitude of forces  $F_1$  and  $F_2$  that act at point  $P$ , and want to find a trajectory such the body passes through certain waypoints. Assuming no constraints on  $F_1$  and  $F_2$ , what will be a good choice of decision variables to find a solution for the trajectory? Justify your answer.
- c. If the magnitude of forces  $F_1$  and  $F_2$  are constrained to be less than  $F_{max}$ , how will it affect the above trajectory optimization, in particular, will the formulation still be convex?

## REFERENCES

1. John T. Betts, "Survey of numerical methods for trajectory optimization", *Journal of Guidance, Control, and Dynamics*, vol. 21, no. 2, pp. 193-207, 1998.
2. John T. Betts, "Practical Methods for Optimal Control Using Nonlinear Programming", Society for Industrial and Applied Mathematics, 2001.
3. Yang Wang and Stephen Boyd, "Fast model predictive control using online optimization", *IEEE Transactions on control systems technology*, vol. 18, no. 2, pp. 267-278, 2009.

4. Frank Dellaert, "Factor graphs and GTSAM: A hands-on introduction", Tech. Report, GT-RIM-CP&R-2012-002, 2012.
5. C. R. Hargraves and S. W. Paris, "Direct Trajectory Optimization using Nonlinear Programming and Collocation", *J Guidance*, vol. 10, no. 4, pp. 338-342, July-August, 1987.
6. Divya Garg and Michael Patterson and Camila Francolin and Christopher Darby and Geoffrey Huntington and William Hager and Anil Rao, "Direct trajectory optimization and costate estimation of finite-horizon and infinite-horizon optimal control problems using a Radau pseudospectral method", *Computational Optimization and Applications*, vol. 49, pp. 335-358, 2011.
7. I. Michael Ross and Mark Karpenko, "A review of pseudospectral optimal control: {From} theory to flight", *Annual Reviews in Control*, vol. 36, no. 2, pp. 182--197, dec, 2012.
8. H. J. Terry Suh and Max Simchowitz and Kaiqing Zhang and Russ Tedrake, "Do Differentiable Simulators Give Better Policy Gradients?", *Proceedings of the 39th International Conference on Machine Learning*, vol. 162, pp. 20668--20696, July, 2022. [ [link](#) ]
9. Dimitri P Bertsekas, "Constrained optimization and Lagrange multiplier methods", Academic press, 1982.
10. Jorge Nocedal and Stephen J. Wright, "Numerical optimization", Springer, 2006.
11. TC Lin and JS Arora, "Differential dynamic programming technique for constrained optimal control", *Computational Mechanics*, vol. 9, no. 1, pp. 27--40, 1991.
12. Marc Toussaint, "A Novel Augmented Lagrangian Approach for Inequalities and Convergent Any-Time Non-Central Updates", , 2014.
13. Marin Kobilarov, "Cross-entropy motion planning", *The International Journal of Robotics Research*, vol. 31, no. 7, pp. 855--871, 2012.
14. Grady Williams and Andrew Aldrich and Evangelos A Theodorou, "Model predictive path integral control: From theory to parallel computation", *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 2, pp. 344--357, 2017.
15. Yunzhu Li and Jiajun Wu and Jun-Yan Zhu and Joshua B. Tenenbaum and Antonio Torralba and Russ Tedrake, "Propagation Networks for Model-Based Control Under Partial Observation", *Proceedings of the International Conference on Robotics and Automation (ICRA)*, May, 2019. [ [link](#) ]
16. Yunzhu Li and Jiajun Wu and Russ Tedrake and Joshua B. Tenenbaum and Antonio Torralba, "Learning Particle Dynamics for Manipulating Rigid Bodies, Deformable Objects, and Fluids", *Proceedings of the International Conference on Learning Representations (ICLR)*, 2019. [ [link](#) ]
17. Tobia Marcucci and Jack Umenberger and Pablo A. Parrilo and Russ Tedrake, "Shortest Paths in Graphs of Convex Sets", *arXiv preprint*, 2023. [ [link](#) ]
18. Tobia Marcucci and Mark Petersen and David von Wrangel and Russ Tedrake, "Motion Planning around Obstacles with Convex Optimization", , 2022. [ [link](#) ]
19. Katja D. Mombaur and Hans Georg Bock and Johannes P. Schloder and Richard W. Longman, "Open-loop stable solutions of periodic optimal control problems in robotics", *Z. Angew. Math. Mech. (ZAMM)*, vol. 85, no.

7, pp. 499–515, 2005.

20. Aaron M Johnson and Jennifer E King and Siddhartha Srinivasa, "Convergent planning", *IEEE Robotics and Automation Letters*, vol. 1, no. 2, pp. 1044–1051, 2016.
21. Carlos E Garcia and David M Prett and Manfred Morari, "Model predictive control: theory and practice—a survey", *Automatica*, vol. 25, no. 3, pp. 335–348, 1989.
22. Eduardo F Camacho and Carlos Bordons Alba, "Model predictive control", Springer Science & Business Media, 2013.
23. Alberto Bemporad and Manfred Morari, "Robust model predictive control: A survey", *Robustness in identification and control*, vol. 245, pp. 207–226, 1999.
24. Rick Cory and Russ Tedrake, "Experiments in Fixed-Wing {UAV} Perching", *Proceedings of the AIAA Guidance, Navigation, and Control Conference*, pp. 1–12, 2008. [ [link](#) ]
25. John W. Roberts and Rick Cory and Russ Tedrake, "On the Controllability of Fixed-Wing Perching", *Proceedings of the American Control Conference (ACC)*, 2009. [ [link](#) ]
26. Rick Cory, "Supermaneuverable Perching", PhD thesis, Massachusetts Institute of Technology, June, 2010. [ [link](#) ]
27. Joseph Moore, "Powerline Perching with a Fixed-wing {UAV}", , May, 2011. [ [link](#) ]
28. Joseph Moore and Russ Tedrake, "Control Synthesis and Verification for a Perching {UAV} using {LQR}-Trees", In *Proceedings of the IEEE Conference on Decision and Control*, pp. 8, December, 2012. [ [link](#) ]
29. Joseph Moore, "Robust Post-Stall Perching with a Fixed-Wing UAV", PhD thesis, Massachusetts Institute of Technology, September, 2014. [ [link](#) ]
30. Sigward F. Hoerner and Henry V. Borst, "Fluid-{Dynamic} {Lift}", Mrs. Liselotte A. Hoerner, 1985.
31. Dimitri P. Bertsekas, "Dynamic Programming and Optimal Control", Athena Scientific, 2000.
32. Richard M. Murray and Muruhan Rathinam and Willem Sluis, "Differential flatness of mechanical control systems: A catalog of prototype systems", *ASME international mechanical engineering congress and exposition*, 1995.
33. D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors", *Proceedings of the 2011 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 2520–2525, 2011.
34. Weiwei Li and Emanuel Todorov, "Iterative Linear Quadratic Regulator Design for Nonlinear Biological Movement Systems.", *International Conference on Informatics in Control, Automation and Robotics*, pp. 222–229, 2004.
35. Athanasios Sideris and James E. Bobrow, "A Fast Sequential Linear Quadratic Algorithm for Solving Unconstrained Nonlinear Optimal Control Problems", , February, 2005.
36. Farbod Farshidian and Edo Jelavic and Asutosh Satapathy and Markus Gfithaler and Jonas Buchli, "Real-time motion planning of legged robots:

A model predictive control approach", *2017 IEEE-RAS 17th International Conference on Humanoid Robotics (Humanoids)* , pp. 577--584, 2017.

37. Y. Tassa and T. Erez and E. Todorov, "Synthesis and stabilization of complex behaviors through online trajectory optimization", *Intelligent Robots and Systems (IROS), 2012 IEEE/RSJ International Conference on* , pp. 4906--4913, 2012.
38. David H. Jacobson and David Q. Mayne, "Differential Dynamic Programming", American Elsevier Publishing Company, Inc. , 1970.
39. A.~Richards and J.P.~How, "Aircraft trajectory planning with collision avoidance using mixed integer linear programming", *Proceedings of the 2002 American Control Conference* , vol. 3, pp. 1936--1941, 2002.
40. Daniel Mellinger and Alex Kushleyev and Vijay Kumar, "Mixed-integer quadratic program trajectory generation for heterogeneous quadrotor teams", *2012 IEEE international conference on robotics and automation* , pp. 477--483, 2012.
41. Robin Deits and Russ Tedrake, "Efficient Mixed-Integer Planning for {UAVs} in Cluttered Environments", *Proceedings of the {IEEE} International Conference on Robotics and Automation ({ICRA})* , 2015. [ [link](#) ]
42. Benoit Landry and Robin Deits and Peter R. Florence and Russ Tedrake, "Aggressive Quadrotor Flight through Cluttered Environments Using Mixed Integer Programming", *Proceedings of the International Conference on Robotics and Automation (ICRA)* , May, 2016. [ [link](#) ]
43. A. Alessio and A. Bemporad, "A survey on explicit model predictive control", *Int. Workshop on Assessment and Future Directions of Nonlinear Model Predictive Control* , 2009.
44. Tobia Marcucci and Robin Deits and Marco Gabiccini and Antonio Bicchi and Russ Tedrake, "Approximate Hybrid Model Predictive Control for Multi-Contact Push Recovery in Complex Environments", *Humanoid Robots (Humanoids), 2017 IEEE-RAS 17th International Conference on* , 2017. [ [link](#) ]
45. John N Nganga and Patrick M Wensing, "Accelerating second-order differential dynamic programming for rigid-body systems", *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 7659--7666, 2021.

# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube.](#)

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 11

# Policy Search

In our case study on [perching aircraft](#), we solved a challenging control problem, but our approach to control design was based on only linear optimal control (around an optimized trajectory). We've also discussed some approaches to nonlinear optimal control that could scale beyond small, discretized state spaces. These were based on estimating the cost-to-go function, including [value iteration using function approximation](#) and [approximate dynamic programming as a linear program or as a sums-of-squares program](#).

There are a lot of things to like about methods that estimate the cost-to-go function (aka value function). The cost-to-go function reduces the long-term planning problem into a one-step planning problem; it encodes all relevant information about the future (and nothing more). The HJB gives us optimality conditions for the cost-to-go that give us a strong algorithmic handle to work with.

In this chapter, we will explore another very natural idea: let us parameterize a controller with some decision variables, and then search over those decision variables directly in order to achieve a task and/or optimize a performance objective. One specific motivation for this approach is the (admittedly somewhat anecdotal) observation that often times simple policies can perform very well on complicated robots and with potentially complicated cost-to-go functions.

We'll refer to this broad class of methods as "policy search" or, when optimization methods are used, we'll sometimes use "policy optimization". This idea has not received quite as much attention from the controls community, probably because we know many relatively simple cases where it does not work well. But it has become very popular again lately due to the empirical success of "policy gradient" algorithms in reinforcement learning (RL). This chapter includes a discussion of the "model-based" version of these RL policy-gradient algorithms; we'll describe their "model-free" versions in [a future chapter](#).

### 11.1 PROBLEM FORMULATION

Consider a static full-state feedback policy,

$$\mathbf{u} = \pi_{\alpha}(\mathbf{x}),$$

where  $\pi$  is potentially a nonlinear function, and  $\alpha$  is the vector of parameters that describe the controller. The control might take time as an input, or might even have it's own internal state, but let's start with this simple form.

Using our prescription for optimal control using additive costs, we can evaluate the performance of this controller from any initial condition using, e.g.:

$$J_\alpha(\mathbf{x}) = \int_0^\infty \ell(\mathbf{x}(t), \mathbf{u}(t)) dt,$$

subject to  $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}), \quad \mathbf{u} = \pi_\alpha(\mathbf{x}), \quad \mathbf{x}(0) = \mathbf{x}.$

In order to provide a scalar cost for each set of policy parameters,  $\alpha$ , we need one more piece: a relative importance for the different initial conditions.

As we will further elaborate when we discuss [stochastic optimal control](#), a very natural choice -- one that preserves the recursive structure of the HJB -- is to optimize the expected value of the cost, given some distribution over initial conditions:

$$\min_{\alpha} E_{\mathbf{x} \sim \mathcal{X}_0} [J_\alpha(\mathbf{x})],$$

where  $\mathcal{X}_0$  is a probability distribution over initial conditions,  $\mathbf{x}(0)$ .

## 11.2 LINEAR QUADRATIC REGULATOR

To start thinking about the problem of searching directly in the policy parameters, it's very helpful to start with a problem we know and can understand well. In LQR problems for linear, time-invariant systems, we know that the optimal policy is a linear function:  $\mathbf{u} = -\mathbf{K}\mathbf{x}$ . So far, we have always obtained  $\mathbf{K}$  indirectly -- by solving a Riccati equation to find the cost-to-go and then backing out the optimizing policy. Here, let us study the case where we parameterize the elements of  $\mathbf{K}$  as decision variables, and attempt to optimize the expected cost-to-go directly.

### 11.2.1 Policy Evaluation

First, let's evaluate our objective for a given  $\mathbf{K}$ . This step is known as "*policy evaluation*". If we use a Gaussian with mean zero and covariance  $\mathbf{\Omega}$  as our distribution over initial conditions, then for LQR we have

$$E \left[ \int_0^\infty [\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u}] dt \right],$$

subject to  $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}, \quad \mathbf{u} = -\mathbf{K}\mathbf{x}, \quad \mathbf{x}(0) \sim \mathcal{N}(0, \mathbf{\Omega}).$

This problem is also known as the  $\mathcal{H}_2$  optimal control problem[1], since the expected-cost-to-go here is the  $\mathcal{H}_2$ -norm of the linear system from disturbance input (here only an impulse that generates our initial conditions) to performance output (which here would be e.g.  $\mathbf{z} = \begin{bmatrix} \mathbf{Q}^{\frac{1}{2}} \mathbf{x} \\ \mathbf{R}^{\frac{1}{2}} \mathbf{u} \end{bmatrix}$ ).

To evaluate a policy  $\mathbf{K}$ , let us first re-arrange the cost function slightly, using the properties of the matrix trace:

$$\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{x}^T \mathbf{K}^T \mathbf{R} \mathbf{K} \mathbf{x} = \mathbf{x}^T (\mathbf{Q} + \mathbf{K}^T \mathbf{R} \mathbf{K}) \mathbf{x} = \text{tr}((\mathbf{Q} + \mathbf{K}^T \mathbf{R} \mathbf{K}) \mathbf{x} \mathbf{x}^T),$$

and the linearity of the integral and expected value:

$$E \left[ \int_0^\infty \text{tr}((\mathbf{Q} + \mathbf{K}^T \mathbf{R} \mathbf{K}) \mathbf{x} \mathbf{x}^T) dt \right] = \text{tr} \left( (\mathbf{Q} + \mathbf{K}^T \mathbf{R} \mathbf{K}) E \left[ \int_0^\infty \mathbf{x} \mathbf{x}^T dt \right] \right),$$

For any given initial condition, the solution of the closed-loop dynamics is given by



the matrix exponential:

$$\mathbf{x}(t) = e^{(\mathbf{A}-\mathbf{BK})t}\mathbf{x}(0).$$

For the distribution of initial conditions, we have

$$E[\mathbf{x}(t)\mathbf{x}(t)^T] = e^{(\mathbf{A}-\mathbf{BK})t}E[\mathbf{x}(0)\mathbf{x}(0)^T]e^{(\mathbf{A}-\mathbf{BK})^T t} = e^{(\mathbf{A}-\mathbf{BK})t}\mathbf{\Omega}e^{(\mathbf{A}-\mathbf{BK})^T t},$$

which is just a (symmetric) matrix function of  $t$ . The integral of this function, call it  $\mathbf{X}$ , represents the expected 'energy' of the closed-loop response:

$$\mathbf{X} = E\left[\int_0^\infty \mathbf{x}\mathbf{x}^T dt\right].$$

Assuming  $\mathbf{K}$  is stabilizing,  $\mathbf{X}$  can be computed as the (unique) solution to the Lyapunov equation:

$$(\mathbf{A} - \mathbf{BK})\mathbf{X} + \mathbf{X}(\mathbf{A} - \mathbf{BK})^T + \mathbf{\Omega} = 0.$$

(You can see a closely related derivation [here](#)). Finally, the total policy evaluation is given by

$$E_{\mathbf{x} \sim \mathcal{N}(0, \mathbf{\Omega})}[J_{\mathbf{K}}(\mathbf{x})] = \text{tr}((\mathbf{Q} + \mathbf{K}^T \mathbf{R} \mathbf{K})\mathbf{X}). \quad (1)$$

### 11.2.2 A nonconvex objective in $\mathbf{K}$

Unfortunately, the Lyapunov equation represents a nonlinear constraint (on the pair  $\mathbf{K}, \mathbf{X}$ ). Indeed, it is well known that even the set of controllers that stabilizing a linear systems can be nonconvex in the parameters  $\mathbf{K}$  when there are 3 or more state variables[2].

#### Example 11.1 (The set of stabilizing $\mathbf{K}$ can be non-convex)

The following example was given in [2]. Consider a discrete-time linear system with  $\mathbf{A} = \mathbf{I}_{3 \times 3}, \mathbf{B} = \mathbf{I}_{3 \times 3}$ . The controllers given by

$$\mathbf{K}_1 = \begin{bmatrix} 1 & 0 & -10 \\ -1 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{and} \quad \mathbf{K}_2 = \begin{bmatrix} 1 & -10 & 0 \\ 0 & 1 & 0 \\ -1 & 0 & 1 \end{bmatrix},$$

are both stabilizing controllers for this system (all eigenvalues of  $\mathbf{A} - \mathbf{BK}$  are inside the unit circle of the complex plane). However the controller  $\hat{\mathbf{K}} = (\mathbf{K}_1 + \mathbf{K}_2)/2$  has two eigenvalues outside the unit circle.

Since the set of controllers that achieve finite total cost is non-convex, clearly the cost function we consider here is also non-convex.

As an aside, for this problem we do actually know a change of variables that make the problem convex. Let's introduce a new variable  $\mathbf{Y} = \mathbf{K}\mathbf{X}$ . Since  $\mathbf{X}$  is PSD, we can back out  $\mathbf{K} = \mathbf{Y}\mathbf{X}^{-1}$ . Now we can rewrite the optimization:

$$\begin{aligned} \min_{\mathbf{X}, \mathbf{Y}} \quad & \text{tr} \mathbf{Q}^{\frac{1}{2}} \mathbf{X} \mathbf{Q}^{\frac{1}{2}} + \text{tr} \mathbf{R}^{\frac{1}{2}} \mathbf{Y} \mathbf{X}^{-1} \mathbf{Y}^T \mathbf{R}^{\frac{1}{2}} \\ \text{subject to} \quad & \mathbf{A}\mathbf{X} - \mathbf{B}\mathbf{Y} + \mathbf{X}\mathbf{A}^T - \mathbf{Y}^T \mathbf{B}^T + \mathbf{\Omega} = 0, \\ & \mathbf{X} \succ 0. \end{aligned}$$

The second term in the objective appears to be nonconvex, but is actually convex. In order to write it as a SDP, we can replace it exactly with one more slack variable,  $\mathbf{Z}$ ,

and a Schur complement[3]:

$$\begin{aligned} \min_{\mathbf{X}, \mathbf{Y}, \mathbf{Z}} \quad & \text{tr } \mathbf{Q}^{\frac{1}{2}} \mathbf{X} \mathbf{Q}^{\frac{1}{2}} + \text{tr } \mathbf{Z} \\ \text{subject to} \quad & \mathbf{A} \mathbf{X} - \mathbf{B} \mathbf{Y} + \mathbf{X} \mathbf{A}^T - \mathbf{Y}^T \mathbf{B}^T + \mathbf{\Omega} = 0, \\ & \begin{bmatrix} \mathbf{Z} & \mathbf{R}^{\frac{1}{2}} \mathbf{Y} \\ \mathbf{Y}^T \mathbf{R}^{\frac{1}{2}} & \mathbf{X} \end{bmatrix} \succeq 0. \end{aligned}$$

Nevertheless, our original question is asking about searching directly in the original parameterization,  $\mathbf{K}$ . If the objective is nonconvex in those parameters, then how should we perform the search?

### 11.2.3 No local minima

Although convexity is sufficient to guarantee that an optimization landscape does not have any local minima, it is not actually necessary. [2] showed that for this LQR objective, all local optima are in fact global optima. This analysis was extended in [4] to give a simpler analysis and include convergence rates.

How does one show that an optimization landscape has no local minima (even though it may be non-convex)? One of the most popular tools is to demonstrate *gradient dominance* with the famous *Polyak-Lojasiewicz (PL) inequality* [5]. For an optimization problem

$$\min_{\mathbf{x} \in \mathbb{R}^d} f(\mathbf{x})$$

we first assume the function  $f$  is  $L$ -smooth (*Lipschitz* gradients):

$$\forall \mathbf{x}, \mathbf{x}', \quad \|\nabla f(\mathbf{x}') - \nabla f(\mathbf{x})\|_2 \leq L \|\mathbf{x}' - \mathbf{x}\|_2.$$

Then we say that the function satisfies the PL-inequality if the following holds for some  $\mu > 0$ :

$$\forall \mathbf{x}, \quad \frac{1}{2} \|\nabla f(\mathbf{x})\|_F^2 \geq \mu(f(\mathbf{x}) - f^*),$$

where  $f^*$  is a value obtained at the optima. In words, the gradient of the objective must grow faster than the gradient of a quadratic function. Note, however, that the distance here is measured in  $f(\mathbf{x})$ , not  $\mathbf{x}$ ; we do not require (nor imply) that the optimal solution is unique. It clearly implies that for any minima,  $\mathbf{x}'$  with  $\nabla f(\mathbf{x}') = 0$ , since the left-hand side is zero, we must have the right-hand side also be zero, so  $\mathbf{x}'$  is also a global optima:  $f(\mathbf{x}') = f^*$ .

#### Example 11.2 (A nonconvex function with no local minima)

Consider the function

$$f(x) = x^2 + 3 \sin^2(x).$$

We can establish that this function is not convex by observing that for  $a = \frac{\pi}{4}$ ,  $b = \frac{3\pi}{4}$ , we have

$$f\left(\frac{a+b}{2}\right) = \frac{\pi^2}{4} + 3 \approx 5.47 > \frac{f(a) + f(b)}{2} = \frac{5\pi^2}{16} + \frac{3}{2} \approx 4.58.$$

We can establish the PL conditions using the gradient

$$\nabla f(x) = 2x + 6 \sin(x) \cos(x).$$

We can establish that this function is  $L$ -smooth with  $L = 8$  by

$$\|\nabla f(b) - \nabla f(a)\|_2 = |2b - 2a + 6 \sin(b - a)| \leq 8|b - a|,$$

because  $\sin(x) \leq x$ . Finally, we have gradient-dominance from the PL-inequality:

$$\frac{1}{2}(2x + 6 \sin(x) \cos(x))^2 \geq \mu(x^2 + 3 \sin^2(x)),$$

with  $\mu = 0.175$ . (I confirmed this with a small [dReal program](#)).

[5] gives a convergence rate for convergence to an optima for [gradient descent](#) given the PL conditions. [4] showed that the gradients of the LQR cost we examine here with respect to  $\mathbf{K}$  satisfy the PL conditions on any sublevel set of the cost-to-go function.

### 11.2.4 True gradient descent

The results described above suggest that one can use gradient descent to obtain the optimal controller,  $\mathbf{K}^*$  for LQR. For the variations we've seen so far (where we know the model), I would absolutely recommend that solving the Riccati equations is a much better algorithm; it is faster and more robust, with no parameters like step-size to tune. But gradient descent becomes more interesting / viable when we think of it as a model for a less perfect algorithm, e.g. where the plant model is not given and the gradients are estimated from noisy samples.

It is a rare luxury, due here to our ability to integrate the linear plants/controllers, quadratic costs, and Gaussian initial conditions, that we could compute the value function exactly in (1). We can also compute the true gradient -- this is a pinnacle of exactness we should strive for in our methods but will rarely achieve again. The gradient is given by

$$\frac{\partial E[J_{\mathbf{K}}(\mathbf{x})]}{\partial \mathbf{K}} = 2(\mathbf{R}\mathbf{K} - \mathbf{B}^T \mathbf{P})\mathbf{X},$$

where  $\mathbf{P}$  satisfies another Lyapunov equation:

$$(\mathbf{A} - \mathbf{B}\mathbf{K})^T \mathbf{P} + \mathbf{P}(\mathbf{A} - \mathbf{B}\mathbf{K}) + \mathbf{Q} + \mathbf{K}^T \mathbf{R}\mathbf{K} = 0.$$

Note that the term *policy gradient* used in reinforcement learning typically refers to the slightly different class of algorithms I hinted at above. In those algorithms, we use the true gradients of the policy (only), but estimate the remainder of the terms in the gradient through sampling. These methods typically require many samples to estimate the gradients we compute here, and should only be weaker (less efficient) than the algorithms in this chapter. The papers investigating the convergence of gradient descent for LQR have also started exploring these cases. We will study these so-called *model-free" policy search* algorithms soon.

## 11.3 MORE CONVERGENCE RESULTS AND COUNTER-EXAMPLES

LQR /  $\mathcal{H}_2$  control is one of the good cases, where we know that for the objective parameterized directly in  $\mathbf{K}$ , all local optima are global optima. [6] extended this result for mixed  $\mathcal{H}_2/\mathcal{H}_\infty$  control. [7] gives a recent treatment of the tabular (finite) MDP case.

For LQR, we also know alternative parameterizations of the controller which make the objective actually convex, including the *LMI formulation* and the *Youla parameterization*. Their utility in a policy search setting was studied initially in [8].

Unfortunately, we do not expect these nice properties to hold in general. There are a number of nearby problems which are known to be nonconvex in the original parameters. The case of *static output feedback* is an important one. If we extend our plant model to include (potentially limited) observations:  $\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}$ ,  $\mathbf{y} = \mathbf{C}\mathbf{x}$ , then searching directly over controllers,  $\mathbf{u} = -\mathbf{K}\mathbf{y}$ , is known to be NP-hard[9]. This time, the set of stabilizing  $\mathbf{K}$  matrices may be not only nonconvex, but actually disconnected. We can see that with a simple example (given to me once during a conversation with Alex Megretski).

### Example 11.3 (Parameterizations of Static Output Feedback)

Consider the single-input, single-output LTI system

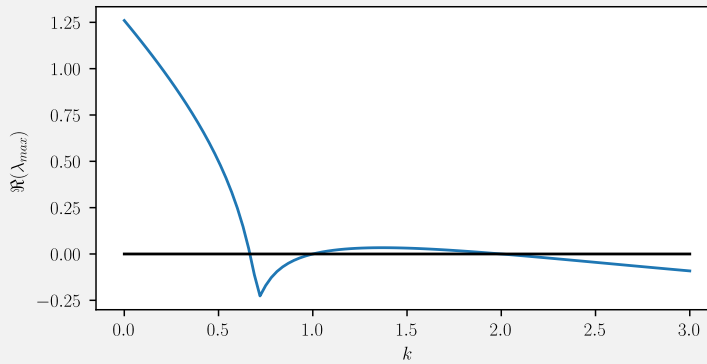
$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}u, \quad y = \mathbf{C}\mathbf{x},$$

with

$$\mathbf{A} = \begin{bmatrix} 0 & 0 & 2 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad \mathbf{C} = [1 \quad 1 \quad 3].$$

Here the linear static-output-feedback policy can be written as  $u = -ky$ , with a single scalar parameter  $k$ .

Here is a plot of the maximum eigenvalue (real-part) of the closed-loop system, as a function of  $k$ . The system is only stable when this maximum value is less than zero. You'll find the set of stabilizing  $k$ 's is a disconnected set.



There are many other known counter-examples. [10] gives a particularly simple one with a two state MDP. One of the big open questions is whether deep network parameterizations are an example of a good case.

## 11.4 TRAJECTORY-BASED POLICY SEARCH

Although we cannot expect gradient descent to converge to a global minima in general, it is still very reasonable to try using gradient descent to find policies for more complicated nonlinear control problems. In the general form, this means that the first step of optimizing

$$\min_{\alpha} E_{\mathbf{x} \sim \mathcal{X}_0} [J_{\alpha}(\mathbf{x})],$$

is estimating

$$\frac{\partial}{\partial \alpha} E_{\mathbf{x} \sim \mathcal{X}_0} [J_{\alpha}(\mathbf{x})].$$

In the LQR problem, we were able to compute these terms exactly; with the biggest simplification coming from the fact that [the response of a linear system to Gaussian initial conditions stays Gaussian](#). This is not true for more general nonlinear systems. So what are we to do?

The most common/general technique (despite it not being very efficient), is to approximate the expected cost-to-go using a sampling (Monte-Carlo) approximation using a large number,  $N$ , of samples:

$$E_{\mathbf{x} \sim \mathcal{X}_0} [J_{\alpha}(\mathbf{x})] \approx \frac{1}{N} \sum_{i=0}^{N-1} J_{\alpha}(\mathbf{x}_i), \quad \mathbf{x}_i \sim \mathcal{X}_0.$$

The gradients follow easily:

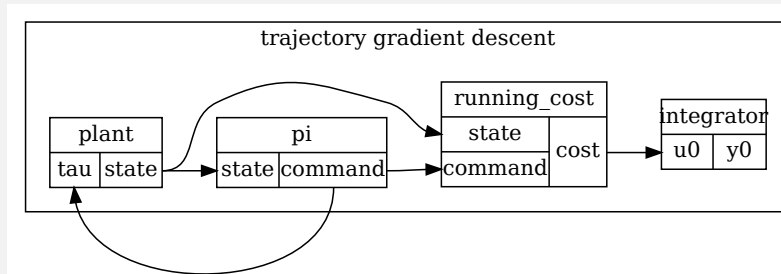
$$\frac{\partial}{\partial \alpha} E_{\mathbf{x} \sim \mathcal{X}_0}[J_\alpha(\mathbf{x})] \approx \frac{1}{N} \sum_{i=0}^{N-1} \frac{\partial J_\alpha(\mathbf{x}_i)}{\partial \alpha}, \quad \mathbf{x}_i \sim \mathcal{X}_0.$$

Our confidence in the accuracy of this estimate will improve as we increase  $N$ ; see e.g. Section 4.2 of [11] for details on the confidence intervals. The most optimistic among us will say that it's quite fine to have only a noisy estimate of the gradient -- this leads to stochastic gradient descent which can have some very desirable properties. But it does make the algorithm harder to analyze and debug.

Using the Monte-Carlo estimator, the total gradient update is just a sum over gradients with respect to particular initial conditions,  $\frac{\partial J_\alpha(\mathbf{x}_i)}{\partial \alpha}$ . But for finite-horizon objectives, these are precisely the gradients that we have already studied in the context of trajectory optimization. They can be computed efficiently using an adjoint method. The difference is that here we think of  $\alpha$  as the parameters of a feedback controller, whereas before we thought of them as the parameters of a trajectory, but this makes no difference to the chain rule.

### Example 11.4 (LQR with true gradients vs approximate gradients)

### Example 11.5 (Pendulum Swing-Up And Balance)



#### 11.4.1 Infinite-horizon objectives

#### 11.4.2 Search strategies for global optimization

To combat local minima... Evolutionary strategies, ...

### 11.5 POLICY ITERATION

In the chapter on Lyapunov analysis we also explored a handful techniques for control design. One of these even directly parameterized the controller (as a polynomial feedback), and used alternations to switch between policy evaluation and policy optimization. Another generated lower-bounds to the optimal cost-to-go.

Coming soon...

### REFERENCES

1. Ben M. Chen, "H2 {Optimal} {Control}", *Encyclopedia of {Systems} and {Control}*, pp. 515-520, 2015.
2. Maryam Fazel and Rong Ge and Sham M. Kakade and Mehran Mesbahi, "Global Convergence of Policy Gradient Methods for the Linear Quadratic Regulator", *International Conference on Machine Learning*, 2018.

3. Erik A Johnson and Baris Erkus, "Dissipativity and performance analysis of smart dampers via LMI synthesis", *Structural Control and Health Monitoring: The Official Journal of the International Association for Structural Control and Monitoring and of the European Association for the Control of Structures*, vol. 14, no. 3, pp. 471--496, 2007.
4. Hesameddin Mohammadi and Armin Zare and Mahdi Soltanolkotabi and Mihailo R Jovanović, "Global exponential convergence of gradient methods over the nonconvex landscape of the linear quadratic regulator", *2019 IEEE 58th Conference on Decision and Control (CDC)* , pp. 7474--7479, 2019.
5. Hamed Karimi and Julie Nutini and Mark Schmidt, "Linear convergence of gradient and proximal-gradient methods under the polyak-Łojasiewicz condition", *Joint European Conference on Machine Learning and Knowledge Discovery in Databases* , pp. 795--811, 2016.
6. Kaiqing Zhang and Bin Hu and Tamer Basar, "Policy Optimization for  $\mathcal{H}_2$  Linear Control with  $\mathcal{H}_\infty$  Robustness Guarantee: Implicit Regularization and Global Convergence", *Proceedings of the 2nd Conference on Learning for Dynamics and Control* , vol. 120, pp. 179--190, 10--11 Jun, 2020.
7. Alekh Agarwal and Sham M. Kakade and Jason D. Lee and Gaurav Mahajan, "On the Theory of Policy Gradient Methods: Optimality, Approximation, and Distribution Shift", , 2020.
8. John Roberts and Ian Manchester and Russ Tedrake, "Feedback Controller Parameterizations for Reinforcement Learning", *Proceedings of the 2011 IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning (ADPRL)* , 2011. [ [link](#) ]
9. Vincent Blondel and John N Tsitsiklis, "{NP}-hardness of some linear control design problems", *SIAM journal on control and optimization*, vol. 35, no. 6, pp. 2118--2127, 1997.
10. Jalaj Bhandari and Daniel Russo, "Global {Optimality} {Guarantees} {For} {Policy} {Gradient} {Methods}", *arXiv:1906.01786 [cs, stat]*, oct, 2020.
11. Reuven Y Rubinstein and Dirk P Kroese, "Simulation and the Monte Carlo method", John Wiley & Sons , vol. 10, 2016.

# UNDERACTUATED ROBOTICS

## *Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 12

# Motion Planning as Search

The term "motion planning" is a quite general term which almost certainly encompasses the dynamic programming, feedback design, and trajectory optimization algorithms that we have already discussed. However, there are a number of algorithms and ideas that we have not yet discussed which have grown from the idea of formulating motion planning as a search problem -- for instance searching for a path from a start to a goal in a graph which is too large solve completely with dynamic programming.

My goal for this chapter is to introduce these additional tools based on search into our toolkit. For robotics, they will play a particularly valuable role when the planning problem is geometrically complex (e.g. a robot moving around obstacles in 3D) or the optimization landscape is very nonconvex, because these are the problems where the trajectory optimization formulation we've studied before will potentially suffer badly from local minima. Many of these algorithms were developed initially for discrete or purely kinematic planning problems; a major theme of this chapter will be the adaptations that allow them to work for underactuated systems. There are also many deep mathematical connections between discrete search and *combinatorial* optimization; I hope to make a few of those connections for you here.

[1] is a very nice book on planning algorithms in general and on motion planning algorithms in particular. Compared to other planning problems, *motion planning* typically refers to problems where the planning domain is continuous (e.g. continuous state space, continuous action space), but many motion planning algorithms trace their origins back to ideas in discrete domains. The term *kinodynamic motion planning* is often used when the plant is subject to dynamic constraints like underactuation, in addition to kinematic constraints like obstacle avoidance.

### 12.1 ARTIFICIAL INTELLIGENCE AS SEARCH

Search algorithms have a long history in AI research -- even before the recent rise of the [large language models \(LLMs\)](#). For decades, many AI researchers felt that the route to creating intelligent machines was to collect large ontologies of knowledge, and then perform very efficient search. Some notable examples from this storied history include Samuel's checker players, theorem proving, Cyc, Deep Blue



playing chess, and IBM Watson. Now it seems that LLMs are providing evidence that humans have put enough "knowledge" on the web such that relatively simple learning paradigms like next-word prediction can lead to a surprising level of "intelligence". As of early 2023, the word on the street is that these models are not yet good at long-term reasoning -- making multi-step plans with more than a few steps [2]. But they appear to be able to use (software) tools, and *maybe* the tools that they need are basic planning/search algorithms.

Indeed, thanks to decades of research, planning algorithms in AI have also scaled to impressive heights, making efficient use of heuristics and factorizations to solve impressively large planning instances. Since 1998, the International Conference on Automated Planning and Scheduling (ICAPS) has been hosting [regular planning competitions](#) which have helped to solidify [problem specification formats](#) and to benchmark the state of the art.

These algorithms have focused on primarily on logical/discrete planning (although they do support "objects", and so can have a sort of open vocabulary). In the language of underactuated, this connects back to the discrete-state, discrete-action, discrete-time planning problems that we discussed to introduce [dynamic programming as graph search](#). Dynamic programming is a very efficient algorithm to solve for the cost-to-go from *every* state, but if we only need to find an (optimal) path from a single start state to the goal, we can potentially do better. In particular, in order to scale to very large planning instances, one of the essential ideas is the idea of "incremental" search, which can avoid every putting the entire (often exponentially large) graph into memory.

Is it possible to find an optimal path from the start to a goal without visiting every node? Yes! Indeed, one of the key insights that powers these incremental algorithms is the use of [admissible heuristics](#) to guide the search -- an approximate cost-to-go which is guaranteed to never over-estimate the cost-to-go. A great example of this would be searching for directions on a map -- the straight-line distance from the start to the goal ("as the crow flies") is guaranteed to never over-estimate the true cost to go (which has to stay on the roads). The most famous search algorithm to leverage these heuristics is [A\\*](#). In robotics, we often use online planning extensions, such as [D\\* and D\\*-Lite](#).

There are numerous other heuristics that power state-of-the-art large-scale logical search algorithms. Another important one is factorization. For a robotics example, consider a robot manipulating many possible objects -- it's reasonable to plan the manipulation of one object assuming it's independent of the other objects and then to revise that plan only when the optimal plan ends up putting two objects on intersecting paths. Many of these heuristics are summarized nicely in the Fast Downward paper[3]; Fast Downward has been at the forefront of the ICAPS planning competitions for many years.

Some of the most visible success stories in deep learning today still make use of planning. For example: DeepMind's [AlphaGo](#) and [AlphaZero](#) combine the planning algorithms developed over the years in discrete games, notably [Monte-Carlo Tree Search \(MCTS\)](#), with learned heuristics in the form of policies and value functions.

## 12.2 SAMPLING-BASED MOTION PLANNING

If you remember how we introduced dynamic programming initially as a graph search, you'll remember that there were some challenges in discretizing the state space. Let's assume that we have discretized the continuous space into some finite set of discrete nodes in our graph. Even if we are willing to discretize the action space for the robot (this might even be acceptable in practice), we had a problem where discrete actions from one node in the graph, integrated over some finite interval  $h$ , are extremely unlikely to land exactly on top of another node in the graph. To combat this, we had to start working on methods for interpolating the value function estimate between nodes.

Interpolation can work well if you are trying to solve for the cost-to-go function over

the entire state space, but it's less compatible with search methods which are trying to find just a single path through the space. If I start in node 1, and land between node 2 and node 3, then which node do I continue to expand from?

Fortunately, the incremental search algorithms from logical search already give us a way to think about this -- we can simply build a *search tree* as the search executes, instead of relying on a predefined mesh discretization. This tree will contain nodes rooted in the continuous space at exactly the points where system can reach.

Another potential problem with any fixed-mesh discretization of a continuous space, or even a fixed discretization of the action space, is that unless we have specific geometric / dynamic insights into our continuous system, it can be very difficult to provide a *complete* planning algorithm. A planning algorithm is complete if it always finds a path (from the start to the goal) when a path exists. Even if we can show that no path to the goal exists on the tree/graph, how can we be certain that there is no path for the continuous system? Perhaps a solution would have emerged if we had discretized the system differently, or more finely?

One approach to addressing this second challenge is to toss out the notion of a fixed discretizations, and replace them with random sampling (another approach would be to adaptively add resolution to the discretization as the algorithm runs). Random sampling, e.g. of the action space, can yield algorithms that are *probabilistically complete* for the continuous space -- if a solution to the problem exists, then a probabilistically complete algorithm will find that solution with probability 1 as the number of samples goes to infinity.

With these motivations in mind, we can build what is perhaps the simplest probabilistically complete algorithm for finding a path from a starting state to some goal region within a continuous state and action space:

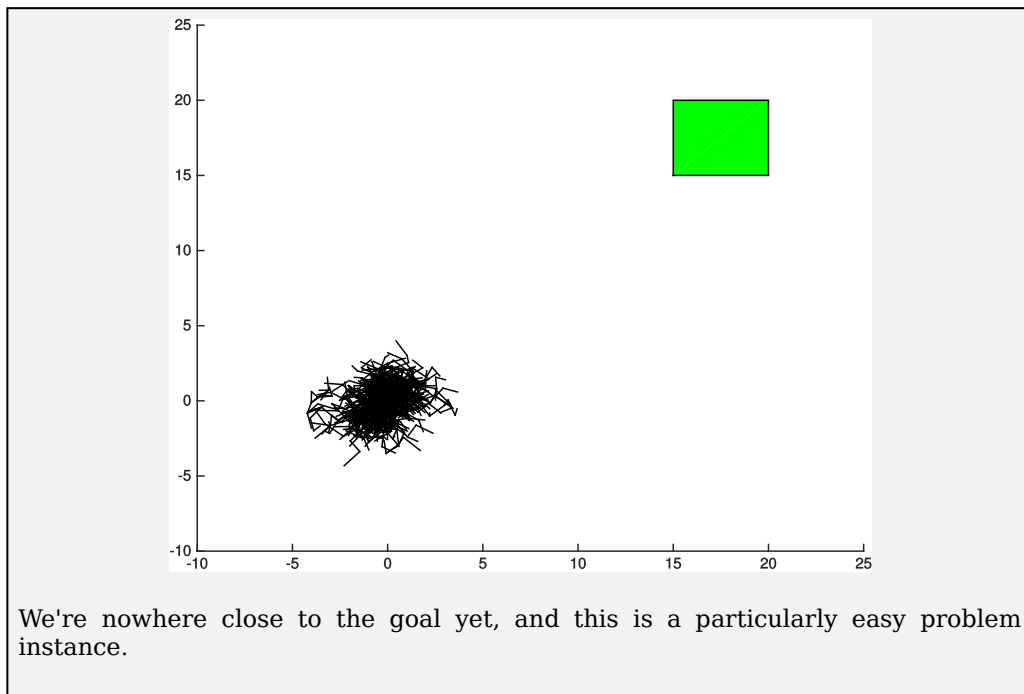
### Example 12.1 (Planning with a Random Tree)

Let us denote the data structure which contains the tree as  $\mathcal{T}$ . The algorithm is very simple:

- Initialize the tree with the start state:  $\mathcal{T} \leftarrow \mathbf{x}_0$ .
- On each iteration:
  - Select a random node,  $\mathbf{x}_{rand}$ , from the tree,  $\mathcal{T}$
  - Select a random action,  $\mathbf{u}_{rand}$ , from a distribution over feasible actions.
  - Compute the dynamics:  $\mathbf{x}_{new} = f(\mathbf{x}_{rand}, \mathbf{u}_{rand})$
  - If  $\mathbf{x}_{new} \in \mathcal{G}$ , then terminate. Solution found!
  - Otherwise add the new node to the tree,  $\mathcal{T} \leftarrow \mathbf{x}_{new}$ .

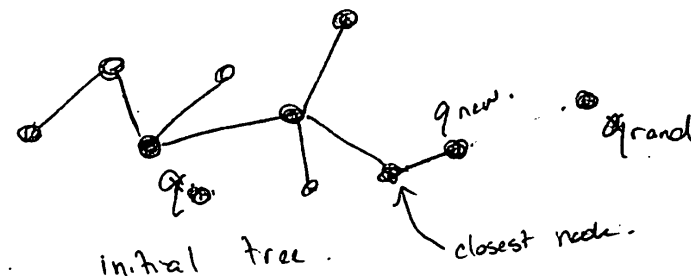
It can be shown that this algorithm is, in fact, probabilistically complete. However, without strong heuristics to guide the selection of the nodes scheduled for expansion, it can be extremely inefficient. For a simple example, consider the system  $\mathbf{x}[n] = \mathbf{u}[n]$  with  $\mathbf{x} \in \mathbb{R}^2$  and  $\mathbf{u}_i \in [-1, 1]$ . We'll start at the origin and put the goal region as  $\forall i, 15 \leq x_i \leq 20$ .

Although this "straw-man" algorithm is probabilistically complete, it is certainly not efficient. After expanding 1000 nodes, the tree is basically a mess of node points all right on top of each other:



The idea of generating a tree of feasible points has clear advantages, but it seems that we have lost the ability to mark a region of space as having been sufficiently explored. It seems that, to make randomized algorithms effective, we are going to at the very least need some form of heuristic for encouraging the nodes to spread out and explore the space.

### 12.2.1 Rapidly-Exploring Random Trees (RRTs)



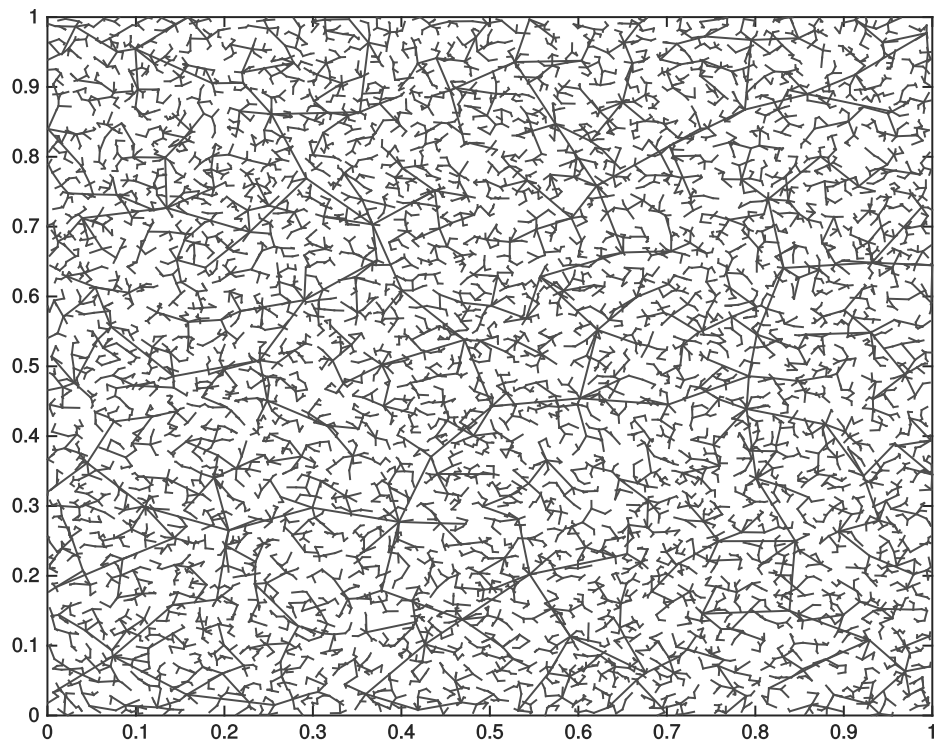


Figure 12.3 - ([Click here to watch the animation](#))

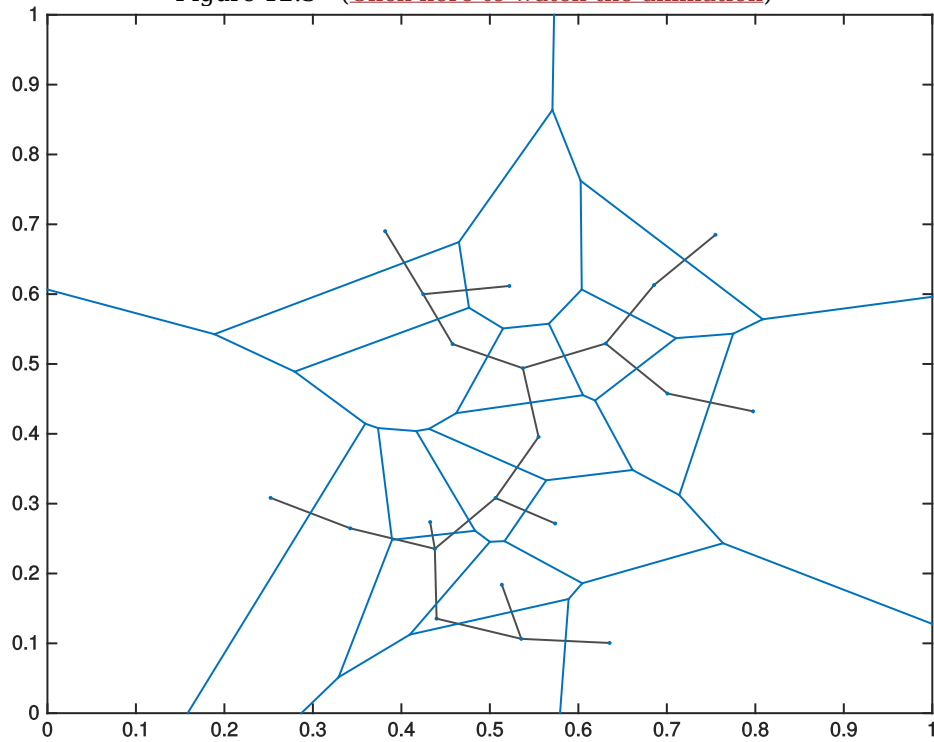


Figure 12.4 - ([Click here to watch the animation](#))

### 12.2.2 RRTs for robots with dynamics

### 12.2.3 Variations and extensions

Multi-query planning with PRMs, ...

RRT\*, RRT-sharp, RRTx, ...

Kinodynamic-RRT\*, LQR-RRT(\*)

Complexity bounds and dispersion limits

### 12.2.4 Discussion

Not sure yet whether randomness is fundamental here, or whether is a temporary "crutch" until we understand geometric and dynamic planning better.

## 12.3 DECOMPOSITION METHODS

Cell decomposition...

Approximate decompositions for complex environments (e.g. IRIS)

## 12.4 PLANNING AS COMBINATORIAL + CONTINUOUS OPTIMIZATION

Shortest path on a graph as a linear program.

Mixed-integer planning.

### 12.4.1 Motion Planning on Graphs of Convex Sets (GCS)

## 12.5 EXERCISES

### Exercise 12.1 (RRT Planning)

In this we will write code for the Rapidly-Exploring Random Tree (RRT). Building on this implementation we will also implement RRT\*, a variant of RRT that converges towards an optimal solution.

- a. Implement RRT
- b. Implement RRT\*

## REFERENCES

1. Steven M. LaValle, "Planning Algorithms", Cambridge University Press , 2006.
2. S{\e}bastien Bubeck and Varun Chandrasekaran and Ronen Eldan and Johannes Gehrke and Eric Horvitz and Ece Kamar and Peter Lee and Yin Tat Lee and Yuanzhi Li and Scott Lundberg and others, "Sparks of artificial general intelligence: Early experiments with gpt-4", *arXiv preprint arXiv:2303.12712*, 2023.
3. Malte Helmert, "The fast downward planning system", *Journal of Artificial Intelligence Research*, vol. 26, pp. 191--246, 2006.



# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

[Russ Tedrake](#)

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube.](#)

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 13

# Feedback Motion Planning

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

---

[Accessibility](#)

© Russ Tedrake, 2024

# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 14

# Robust and Stochastic Control

My goal of presenting a relatively consumable survey of a few of the main ideas is perhaps more important in this chapter than any other. It's been said that "robust control is encrypted" (as in you need to know the secret code to get in). The culture in the robust control community has been to leverage high-powered mathematics, sometimes at the cost of offering more simple explanations. This is unfortunate, I think, because robotics and machine learning would benefit from a richer connection to these tools, and are perhaps destined to reinvent many of them.

The classic reference for robust control is [1]. [2] has a treatment that does more of its development in the time-domain and via Riccati equations; I will do the same here.

### 14.1 STOCHASTIC MODELS

So far in the notes, we have concerned ourselves primarily with known, deterministic systems. In the [stochastic systems](#) chapter, we started our study of nonlinear dynamics of stochastic systems, which can be beautiful! In this chapter we will begin to consider computational tools for analysis and control of those systems. Stochasticity can come in many forms... we may not know the governing equations (e.g. the coefficient of friction in the joints), our robot may be [walking on unknown terrain, subject to unknown disturbances](#), or even be picking up unknown objects. There are a number of mathematical frameworks for considering this uncertainty; for our purposes this chapter will generalize our thinking to equations of the form:

$$\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u}, \mathbf{w}, t) \quad \text{or} \quad \mathbf{x}[n+1] = \mathbf{f}(\mathbf{x}[n], \mathbf{u}[n], \mathbf{w}[n], n),$$

where  $\mathbf{w}$  is a new *random* input signal to the equations capturing all of this potential variability. Although it is certainly possible to work in continuous time, and treat  $\mathbf{w}(t)$  as a continuous-time random signal (c.f. [Wiener process](#)), the notation and intuition is a bit simpler when we work with  $\mathbf{w}[n]$  as a discrete-time random signal. For this reason, we'll devote our attention in this chapter to the discrete-time systems.

In order to simulate equations of this form, or to design controllers against them, we



need to define the random process that generates  $\mathbf{w}[n]$ . It is typical to assume the values  $\mathbf{w}[n]$  are independent and identically distributed (i.i.d.), meaning that  $\mathbf{w}[i]$  and  $\mathbf{w}[j]$  are uncorrelated when  $i \neq j$ . As a result, we typically define our distribution via a probability density  $\mathbf{w}[n] \sim p_{\mathbf{w}}(\mathbf{w})$ . This is not as limiting as it may sound... if we wish to treat temporally-correlated noise (e.g. "colored noise") the format of our equations is rich enough to support this by adding additional state variables; we'll give an example below of a "whitening filter" for modeling wind gusts. The other source of randomness that we will now consider in the equations is randomness in the initial conditions; we will similarly define a probability density  $\mathbf{x}[0] \sim p_0(\mathbf{x})$ .

The notation  $\mathbf{w}[n] \sim p_{\mathbf{w}}(\mathbf{w})$  denotes that  $\mathbf{w}[n]$  is "drawn from" the distribution  $p_{\mathbf{w}}(\mathbf{x})$ .

This modeling framework is rich enough for us to convey the key ideas; but it is not quite sufficient for all of the systems I am interested in. In **DRAKE** we go to additional lengths to support more general cases of **stochastic systems**. This includes modeling system parameters that are drawn from random each time the model is initialized, but are fixed over the duration of a simulation; it is possible but inefficient to model these as additional state variables that have no dynamics. In other problems, even the *dimension of the state vector* may change in different realizations of the problem! Consider, for instance, the case of a robot manipulating random numbers of dishes in a sink. I do not know many control formulations that handle this type of randomness well, and I consider this a top priority to think more about! (We'll begin to address it in the **output feedback chapter**.)

## 14.2 COSTS AND CONSTRAINTS FOR STOCHASTIC SYSTEMS

Given a stochastic model, what sort of cost function should we write in order to capture the desired aspect of performance? There are a number of natural choices, which provide nice trade-offs between capturing the desired phenomena and computational tractability.

Remember that  $\mathbf{x}[n]$  is now a random variable, so we want to write our cost and constraints using the distribution described e.g.  $p_n(\mathbf{x})$ . Broadly speaking, one might specify a cost in a handful of ways:

- **Average cost**,  $E[\sum_n \ell(\mathbf{x}[n], \mathbf{u}[n])]$ , or some time-averaged/discounted variant. This is by far the most popular choice (especially in reinforcement learning), because it preserves the dynamic programming recursion for additive cost.
- **Worst-case cost**, e.g.  $\min_{\mathbf{u}[\cdot]} \max_{\mathbf{x}[0], \mathbf{w}[\cdot]} \sum_n \ell(\mathbf{x}[n], \mathbf{u}[n])$ . When we are able to upper-bound this cost and push down on that bound, then this is sometimes call *guaranteed cost control*.
- **Relative worst-case** (aka "gain bounds"). Even for linear systems, bounding the absolute performance (for all  $\mathbf{w}$ ) leads to vacuous bounds; we can do much better by evaluating our performance metric relative to the magnitude of the disturbance input.
- **Value at Risk (VaR)** and other related metrics from economics and operations research.  $\text{VaR}(\alpha)$  is the maximum cost if we ignore the tails of the distribution (worse outcomes whose combined probability is at most  $\alpha$ ). Note that VaR is easy to understand, but the related quantity "conditional value at risk" (CVaR) has superior mathematical properties, including connections to convex optimization [3].
- **Regret** formulations try to minimize the difference between the designed controller and e.g. an optimal controller in some class that has access to privileged information, such as knowing a priori the disturbances  $\mathbf{w}[n]$ . These formulations have become popular in the field of online optimization, and we are now starting to see "regret bounds" for control.

Similarly, we can think about how to generalize our deterministic constraints, e.g.  $g(\mathbf{x}) \leq 0$ , into a the stochastic framework:

- **Chance constraints**, which take the form e.g.  $\Pr[g(\mathbf{x}) > 0] \leq \alpha$ . For instance, we might like to guarantee that the probability that our airplane

will crash into a tree is less than  $\alpha$ . Even for Gaussian uncertainty and polytopic constraints, these quantities can be hard to evaluate exactly; we often end up making approximations[4] though there is some hope to provide rigorous bounds with Lyapunov-like arguments [5].

- **Worst-case constraints**, which are the limit of the chance constraints when we take  $\alpha \rightarrow 0$ , tend to be much more amenable to computation, and connect directly to [reachability analysis](#).

The term "robust control" is typically associated with the class of techniques that try to guarantee some *worst-case* performance or a worst-case bound (e.g. the gain bounds). The term "stochastic optimal control" or "stochastic control" is typically used as a catch-all for other methods that reason about stochasticity, without necessarily providing the strict robustness guarantees.

It's important to realize that proponents of robust control are not necessarily pessimistic by nature; there is a philosophical stance about how difficult it can be to model the true distribution of randomness that a control system will face in the world. Worst-case analysis typically requires only a definition of the *set* of possible values that the random variable can take, and not a detailed *distribution* over those possible values. It may be more practical to operationalize a concept like "this UAV will not crash so long as the peak wind gusts are below 35 mph" than requiring a detailed distribution over wind gust probabilities. But this simpler specification does come at some cost -- worst-case certificates are often pessimistic about the true performance of a system, and optimizing worst-case performance can lead to conservatism in the control design.

## 14.3 FINITE MARKOV DECISION PROCESSES

The Bellman equation.

Discounted cost. Infinite-horizon average cost.

We already had quick preview into stochastic optimal control in one of the cases where it is particularly easy: [finite Markov Decision Processes \(MDPs\)](#).

## 14.4 LINEAR OPTIMAL CONTROL

### 14.4.1 Stochastic LQR

Let's consider a stochastic extension of the (discrete-time) LQR problem, where the system is now subjected to additive Gaussian white noise:

$$\begin{aligned} \mathbf{x}[n+1] &= \mathbf{A}\mathbf{x}[n] + \mathbf{B}\mathbf{u}[n] + \mathbf{w}[n], \\ E[\mathbf{w}[i]] &= 0, \quad E[\mathbf{w}[i]\mathbf{w}^T[j]] = \delta_{ij}\mathbf{\Sigma}_w, \end{aligned}$$

where  $\delta_{ij}$  is one if  $i = j$  and zero otherwise, and  $\mathbf{\Sigma}_w$  is the covariance matrix of the disturbance, and we take the average cost:

$$\min E \left[ \sum_{n=0}^{\infty} \mathbf{x}^T[n]\mathbf{Q}\mathbf{x}[n] + \mathbf{u}^T[n]\mathbf{R}\mathbf{u}[n] \right], \quad \mathbf{Q} = \mathbf{Q}^T \succeq 0, \mathbf{R} = \mathbf{R}^T \succ 0.$$

Note that we saw one version of this already when we discussed [policy search for LQR](#).

In the standard LQR derivation, we started by assuming a quadratic form for the cost-to-go function. Is that a suitable starting place for this stochastic version? We've already [studied the dynamics of a linear system subjected to Gaussian noise](#), and learned that (at best) we should expect it to have a Gaussian stationary distribution. But that sounds problematic, no? If the system can not drive the system to zero and stay at zero in order to stop accruing cost, then won't the infinite-horizon cost be

infinite (regardless of the controller)?

Yes. The cost-to-go for this problem is infinite! But it turns out that it still has enough structure for us to work with. In particular, let's "guess" a cost-to-go function of the form:

$$J_n(\mathbf{x}) = \mathbf{x}^T \mathbf{S}_n \mathbf{x} + c_n,$$

where  $c_n$  is a (scalar) constant. Now we can write the Bellman equation and do some algebra:

$$\begin{aligned} J_n(\mathbf{x}) &= \min_{\mathbf{u}} E_{\mathbf{w}} [\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u} + [\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u} + \mathbf{w}]^T \mathbf{S}_{n+1} [\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u} + \mathbf{w}] + c_{n+1}] \\ &= \min_{\mathbf{u}} [\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u} + [\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}]^T \mathbf{S}_{n+1} [\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}] + c_{n+1} \\ &\quad + E_{\mathbf{w}} [[\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}]^T \mathbf{S}_{n+1} \mathbf{w} + \mathbf{w}^T \mathbf{S}_{n+1} [\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}]] + E_{\mathbf{w}} [\mathbf{w}^T \mathbf{w}]] \\ &= \min_{\mathbf{u}} [\mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u} + [\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}]^T \mathbf{S}_{n+1} [\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u}] + c_{n+1} + \text{tr}(\mathbf{\Sigma}_{\mathbf{w}})]. \end{aligned}$$

The second line follows by simply pulling all of the deterministic terms outside of the expected value. The third line follows by observing that  $\mathbf{x}$  and  $\mathbf{u}$  are uncorrelated with  $\mathbf{w}$ , and  $E[\mathbf{w}] = 0$ , so those cross terms are zero in expectation. Notice that, apart from the  $c_{n+1}$  and  $\mathbf{\Sigma}_{\mathbf{w}}$ , the remaining terms are exactly the same as the deterministic (discrete-time) LQR version.

Remarkably, we can therefore achieve our dynamic programming recursion by using  $\mathbf{S}_n$  as the solution of the discrete Riccati equation, and  $c_n = c_{n+1} + \text{tr}(\mathbf{\Sigma}_{\mathbf{w}})$ . As we take  $n \rightarrow \infty$ ,  $\mathbf{S}_n$  converges to the steady-state solution to the algebraic Riccati equation, and only  $c_n$  grows to infinity. As a result, even though the cost-to-go is infinite, the optimal control is still well defined: it is the same  $\mathbf{u}^* = -\mathbf{K} \mathbf{x}$  that we obtain from the deterministic LQR problem!

Take note of the fact that the true cost-to-go blows up to infinity. In reinforcement learning, for instance, it is common practice to avoid this blow-up by considering discounted-cost formulations,

$$\sum_{n=0}^{\infty} \gamma^n \ell(\mathbf{x}[n], \mathbf{u}[n]), \quad 0 < \gamma \leq 1,$$

or average-cost formulations,

$$\lim_{N \rightarrow \infty} \frac{1}{N} \sum_{n=0}^N \ell(\mathbf{x}[n], \mathbf{u}[n]).$$

These are satisfactory solutions to the problem, but please make sure to understand why they *must* be used.

## 14.4.2 Non-i.i.d. disturbances

The LQR derivation above assumed that the disturbances  $\mathbf{w}[n]$  were independent and identically distributed (i.i.d.). But many of the disturbances we would like to model are not i.i.d.. For instance, consider a UAV flying in the wind. The wind is correlated over time, sometimes building up to gusts but even those gusts are relatively long compared to any the sampling rate of a control system.

In fact, the standard models of wind are typically the output of a Gaussian i.i.d. random signal passed through a linear low-pass filter [6]. This suggests a very natural approach to using LQR to stabilize a UAV in the wind: design the controller for the joint wind + UAV system. The state of this system contains both the state of the UAV and the state of the wind, so the resulting controller will have the form

$$\mathbf{u} = -\mathbf{K} \begin{bmatrix} \mathbf{x}_{uav} \\ \mathbf{x}_{wind} \end{bmatrix}.$$

In practice, this means that the controller will need to rely on an online estimator for the wind state in addition to the standard (UAV) state estimator.

**Example 14.1 (A quadrotor flying in the wind.)**

### 14.4.3 $L_2$ gain

Which cost function should we use to do worst-case design and analysis for linear systems? Certainly we can put an absolute bound on the magnitude of  $\mathbf{w}$ , and perform reachability analysis (we will do it below). But knowing that the dependency on  $\mathbf{w}$  is linear allows us to do something more natural, which can lead to tighter bounds. In particular, we expect the magnitude of the deviation in  $\mathbf{x}$  compared to the undisturbed case to be proportional to the magnitude of the disturbance,  $\mathbf{w}$ . So the natural bound for a linear system is a *relative* bound on the magnitude of the response (from zero initial conditions) relative to the magnitude of the disturbance.

Typically, this is done with the a scalar " $L_2$  gain",  $\gamma$ , defined as:

$$\begin{aligned} \operatorname{argmin}_{\gamma} \quad \text{subject to} \quad & \sup_{\mathbf{w}(\cdot) \in \mathcal{F} \mid \|\mathbf{w}(t)\|^2 dt \leq \infty} \frac{\int_0^T \|\mathbf{x}(t)\|^2 dt}{\int_0^T \|\mathbf{w}(t)\|^2 dt} \leq \gamma^2, \quad \text{or} \\ \operatorname{argmin}_{\gamma} \quad \text{subject to} \quad & \sup_{\mathbf{w}[\cdot] \in \sum_n \|\mathbf{w}[n]\|^2 \leq \infty} \frac{\sum_0^N \|\mathbf{x}[n]\|^2}{\sum_0^N \|\mathbf{w}[n]\|^2} \leq \gamma^2. \end{aligned}$$

The name " $L_2$  gain" comes from the use of the  $\ell_2$  norm on the signals  $\mathbf{w}(t)$  and  $\mathbf{x}(t)$ , which is assumed only to be finite.

More often, these gains are written not in terms of  $\mathbf{x}[n]$  directly, but in terms of some "performance output",  $\mathbf{z}[n]$ . For instance, if we would like to bound the cost of a quadratic regulator objective as a function of the magnitude of the disturbance, we can minimize

$$\min_{\gamma} \quad \text{subject to} \quad \sup_{\mathbf{w}[n]} \frac{\sum_0^N \|\mathbf{z}[n]\|^2}{\sum_0^N \|\mathbf{w}[n]\|^2} \leq \gamma^2, \quad \mathbf{z}[n] = \begin{bmatrix} \sqrt{\mathbf{Q}}\mathbf{x}[n] \\ \sqrt{\mathbf{R}}\mathbf{u}[n] \end{bmatrix}.$$

This is a simple but important idea, and understanding it is the key to understanding the language around robust control. In particular the  $\mathcal{H}_2$  norm of a system (from input  $\mathbf{w}$  to output  $\mathbf{z}$ ) is the energy of the impulse response; when  $\mathbf{z}$  is chosen to represent the quadratic regulator cost as above, it corresponds to the expected LQR cost. The  $\mathcal{H}_{\infty}$  norm of a system (from  $\mathbf{w}$  to  $\mathbf{z}$ ) is the largest singular value of the transfer function; it corresponds to the  $L_2$  gain.

## Dissipation inequalities

One of the mechanisms for certifying an  $L_2$  gain for a system comes from a generalization of Lyapunov analysis to consider the contributions of system inputs via the so-called "dissipation inequalities". Dissipation inequalities are a general tool, and  $L_2$ -gain analysis is only one application of them; for a broader treatment see [7] or [8, Ch. 2].

Informally, the idea is to generalize the Lyapunov conditions,  $V(\mathbf{x}) > 0, \dot{V}(\mathbf{x}) \leq 0$ , into the more general form

$$V(\mathbf{x}) > 0, \quad \dot{V}(\mathbf{x}) \leq s(\mathbf{x}, \mathbf{w}),$$

where  $\mathbf{w}$  is the input of interest in this setting, and  $s()$  is a scalar quantity representing a "supply rate". Once a system has input, the value of  $V$  may go up or it may go down, but if we can bound the way that it goes up by a simple function of the input, then we may still be able to provide input-to-state or input-output bounds for the system. Integrating both sides of the derivative condition with respect to time

yields:

$$\forall \mathbf{x}(0), \quad V(\mathbf{x}(T)) \leq V(\mathbf{x}(0)) + \int_0^T s(\mathbf{x}(t), \mathbf{w}(t)) dt.$$

To obtain a bound on the  $L_2$  gain between input  $\mathbf{w}(t)$  and output  $\mathbf{z}(t)$ , the supply rate of interest is

$$s(\mathbf{x}, \mathbf{w}) = \gamma^2 \|\mathbf{w}\|^2 - \|\mathbf{z}\|^2,$$

which yields

$$\forall \mathbf{x}(0), \quad V(\mathbf{x}(T)) \leq V(\mathbf{x}(0)) + \int_0^T \gamma^2 \|\mathbf{w}(t)\|^2 dt - \int_0^T \|\mathbf{z}(t)\|^2 dt.$$

Now, since this must hold for all  $\mathbf{x}(0)$ , it holds for  $\mathbf{x}(0) = 0$ . Furthermore, we know  $V(\mathbf{x}(T))$  is non-negative, so we also have

$$0 \leq V(\mathbf{x}(T)) \leq \int_0^T \gamma^2 \|\mathbf{w}(t)\|^2 dt - \int_0^T \|\mathbf{z}(t)\|^2 dt.$$

Therefore, if we can find a  $V$  that satisfied the dissipation inequality for this storage function, we have certified the  $\gamma$  is an  $L_2$  gain for the system:

$$\frac{\int_0^T \|\mathbf{z}(t)\|^2 dt}{\int_0^T \|\mathbf{w}(t)\|^2 dt} \leq \gamma^2.$$

## Small-gain theorem

Coming soon...

## Model uncertainty as a special case.

### 14.4.4 Robust LQR as $\mathcal{H}_\infty$

Let's consider a robust variant of the LQR problem:

$$\begin{aligned} \min_{\mathbf{u}[\cdot]} \max_{\mathbf{w}[\cdot]} \sum_{n=0}^{\infty} \mathbf{x}^T[n] \mathbf{Q} \mathbf{x}[n] + \mathbf{u}^T[n] \mathbf{R} \mathbf{u}[n] - \gamma^2 \mathbf{w}^T[n] \mathbf{w}[n], \\ \mathbf{x}[n+1] = \mathbf{A} \mathbf{x}[n] + \mathbf{B} \mathbf{u}[n] + \mathbf{B}_w \mathbf{w}[n], \\ \mathbf{Q} = \mathbf{Q}^T \succeq 0, \mathbf{R} = \mathbf{R}^T \succ 0. \end{aligned}$$

The reason for this choice of cost function will become clear in the derivation, but the intuition is that we want to reward the controller for having a small response to large  $\mathbf{w}[\cdot]$ . Note that unlike the stochastic LQR formulation, here we do not specify the distribution over  $\mathbf{w}[n]$ , and in fact we don't even restrict it to a bounded set. All we know is that at each time step, an omniscient adversary is allowed to choose the  $\mathbf{w}[n]$  that tries to maximize this objective.

In fact, since we don't need to specify a continuous-time random process and the continuous-time derivation is both cleaner and by now, I think, more familiar, let's do this one in continuous time.

$$\begin{aligned} \min_{\mathbf{u}[\cdot]} \max_{\mathbf{w}[\cdot]} \int_{n=0}^{\infty} dt [\mathbf{x}^T(t) \mathbf{Q} \mathbf{x}(t) + \mathbf{u}^T(t) \mathbf{R} \mathbf{u}(t) - \gamma^2 \mathbf{w}^T(t) \mathbf{w}(t)], \\ \dot{\mathbf{x}}(t) = \mathbf{A} \mathbf{x}(t) + \mathbf{B} \mathbf{u}(t) + \mathbf{B}_w \mathbf{w}(t), \\ \mathbf{Q} = \mathbf{Q}^T \succeq 0, \mathbf{R} = \mathbf{R}^T \succ 0. \end{aligned}$$

We will once again guess a quadratic cost-to-go function:

$$J(\mathbf{x}) = \mathbf{x}^T \mathbf{S} \mathbf{x}, \quad \mathbf{S} = \mathbf{S}^T \succ 0.$$

The dynamic programming recursion still holds for this problem, resulting in the Bellman equation:

$$\forall \mathbf{x}, \quad 0 = \min_{\mathbf{u}} \max_{\mathbf{w}} \left[ \mathbf{x}^T \mathbf{Q} \mathbf{x} + \mathbf{u}^T \mathbf{R} \mathbf{u} - \gamma^2 \mathbf{w}^T \mathbf{w} + \frac{\partial J^*}{\partial \mathbf{x}} [\mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u} + \mathbf{B}_w \mathbf{w}] \right].$$

Since the inside is a concave quadratic form over  $\mathbf{w}$ , we can solve for the adversary by finding the maximum:

$$\begin{aligned} \frac{\partial}{\partial \mathbf{w}} &= -2\gamma^2 \mathbf{w}^T + 2\mathbf{x}^T \mathbf{S} \mathbf{B}_w, \\ \mathbf{w}^* &= \frac{1}{\gamma^2} \mathbf{B}_w^T \mathbf{S} \mathbf{x}. \end{aligned}$$

This is the disturbance input that can cause the largest cost; the optimal play for the adversary. Now we can solve the remainder as we did with the original LQR problem:

$$\begin{aligned} \frac{\partial}{\partial \mathbf{u}} &= 2\mathbf{u}^T \mathbf{R} + 2\mathbf{x}^T \mathbf{S} \mathbf{B}, \\ \mathbf{u}^* &= -\mathbf{R}^{-1} \mathbf{B}^T \mathbf{S} \mathbf{x} = -\mathbf{K} \mathbf{x}. \end{aligned}$$

Substituting this back into the Bellman equation gives:

$$0 = \mathbf{Q} + \mathbf{S} [\gamma^{-2} \mathbf{B}_w \mathbf{B}_w^T - \mathbf{B} \mathbf{R}^{-1} \mathbf{B}^T] \mathbf{S} + \mathbf{S}^T \mathbf{A} + \mathbf{A}^T \mathbf{S},$$

which is the original LQR Riccati equation with one additional term involving  $\gamma$ . And like the original LQR Riccati equation, we must ask whether it has a positive-definite solution for  $\mathbf{S}$ . It turns out that if the system is stabilizable and  $\gamma$  large enough, then it does have a PD solution. However, as we reduce  $\gamma$  towards zero, there will be some threshold  $\gamma$  beneath which this Riccati equation does not have a PD solution. Intuitively, if  $\gamma$  is too small, then the adversary is rewarded for injecting disturbances that are so large as to break the convergence.

Here is the fascinating thing: the  $\gamma$  in this robust LQR cost function can be interpreted as an  $L_2$  gain for the system. Recall that when we were making connections between the Bellman equation and Lyapunov equations, we observed that the Bellman equation can be written as  $\forall \mathbf{x}, \quad \dot{J}^*(\mathbf{x}) \leq -\ell(\mathbf{x}, \mathbf{u})$ ? Here this yields:

$$\forall \mathbf{x}, \quad \dot{J}^*(\mathbf{x}) \leq \gamma^2 \mathbf{w}^T(t) \mathbf{w}(t) - \mathbf{x}^T(t) \mathbf{Q} \mathbf{x}(t) - \mathbf{u}^T(t) \mathbf{R} \mathbf{u}(t).$$

This means that the cost-to-go is a valid dissipation inequality for the supply rate that provides an  $L_2$  gain for the performance output  $\mathbf{z} = \begin{bmatrix} \sqrt{\mathbf{Q}} \mathbf{x} \\ \sqrt{\mathbf{R}} \mathbf{u} \end{bmatrix}$ . Moreover, we can

find the minimal  $L_2$  gain by finding the minimal  $\gamma > 0$  for which the Riccati equation has a positive-definite solution. And given the properties of the Riccati equation, this can be done with a line search in  $\gamma$ .

Because the  $L_2$  gain is the  $\mathcal{H}_\infty$ -norm of the system, this recipe of searching for the smallest  $\gamma$  and taking the Riccati solution is an instance of  $\mathcal{H}_\infty$  control design.

### 14.4.5 Linear Exponential-Quadratic Gaussian (LEQG)

[9] observed that it is also straight-forward to minimize the objective:

$$J = E \left[ \prod_{n=0}^{\infty} e^{\mathbf{x}^T[n] \mathbf{Q} \mathbf{x}[n]} e^{\mathbf{u}^T[n] \mathbf{R} \mathbf{u}[n]} \right] = E \left[ e^{\sum_{n=0}^{\infty} \mathbf{x}^T[n] \mathbf{Q} \mathbf{x}[n] + \mathbf{u}^T[n] \mathbf{R} \mathbf{u}[n]} \right],$$

with  $\mathbf{Q} = \mathbf{Q}^T \geq 0, \mathbf{R} = \mathbf{R}^T > 0$ , by observing that the cost is monotonically related to  $\log J$ , and therefore has the same minima (this same trick forms the basis for

"geometric programming" [10]). This is known as the "Linear Exponential-Quadratic Gaussian" (LEG or LEQG), and for the deterministic version of the problem (no process nor measurement noise) the solution is identical to the LQR problem; it adds no new modeling power. But with noise, the LEQG optimal controllers are different from the LQG controllers; they depend explicitly on the covariance of the noise.

[11] provides an extensive treatment of this framework; nearly all of the analysis from LQR/LQG (including Riccati equations, Hamiltonian formulations, etc) have analogs for their versions with exponential cost, and he argues that LQG and H-infinity control can (should?) be understood as special cases of this approach.

#### 14.4.6 Adaptive control

The standard criticism of  $\mathcal{H}_2$  optimal control is that minimizing the expected value does not allow any guarantees on performance. The standard criticism of  $\mathcal{H}_\infty$  optimal control is that it concerns itself with the worst case, and may therefore be conservative, especially because distributions over possible disturbances chosen a priori may be unnecessarily conservative. One might hope that we could get some of this performance back if we are able to update our models of uncertainty online, adapting to the statistics of the disturbances we actually receive. This is one of the goals of adaptive control.

One of the fundamental problems in online adaptive control is the trade-off between exploration and exploitation. Some inputs might drive the system to build more accurate models of the dynamics / uncertainty quickly, which could lead to better performance. But how can we formalize this trade-off?

There has been some nice progress on this challenge in machine learning in the setting of (contextual) **multi-armed bandit** problems. For our purposes, you can think of bandits as a limiting case of an optimal control problem where there are no dynamics (the effects of one control action do not effect the results of the next control action). In this simpler setting, the online optimization community has developed exploration-exploitation strategies based on the notion of minimizing *regret* -- typically the accumulated difference in the performance achieved by my online algorithm vs the performance that would have been achieved if I had been privy to the data from my experiments before I started. This has led to methods that make use of concepts like upper-confidence bound (UCB) and more recently bounds using a least-squares confidence bound [12] to provide bounds on the regret.

In the last few years, we've see these results translated into the setting of linear optimal control...

#### 14.4.7 Structured uncertainty

#### 14.4.8 Linear parameter-varying (LPV) control

### 14.5 TRAJECTORY OPTIMIZATION

#### 14.5.1 Monte-carlo trajectory optimization

#### 14.5.2 Iterative $\mathcal{H}_2$ /iLQG

#### 14.5.3 Finite-time (reachability) analysis

Coming soon...

#### 14.5.4 Robust MPC

### 14.6 NONLINEAR ANALYSIS AND CONTROL

### 14.7 DOMAIN RANDOMIZATION

### 14.8 EXTENSIONS

#### 14.8.1 Alternative risk/robustness metrics

### REFERENCES

1. Kemin Zhou and John C. Doyle, "Essentials of Robust Control", Prentice Hall , 1997.
2. I. R. Petersen and V. A. Ugrinovskii and A. V. Savkin, "Robust Control Design using H-infinity Methods", Springer-Verlag , 2000.
3. Anirudha Majumdar and Marco Pavone, "How should a robot assess risk? towards an axiomatic theory of risk in robotics", *Robotics Research* , pp. 75--84, 2020.
4. Guy Scher and Sadra Sadraddini and Russ Tedrake and Hadas Kress-Gazit, "Elliptical Slice Sampling for Probabilistic Verification of Stochastic Systems with Signal Temporal Logic Specifications", *To appear in the Proceedings of Hybrid Systems: Computation and Control (HSCC)* , 2022. [ [link](#) ]
5. Jacob Steinhardt and Russ Tedrake, "Finite-time Regional Verification of Stochastic Nonlinear Systems", *International Journal of Robotics Research*, vol. 31, no. 7, pp. 901-923, June, 2012. [ [link](#) ]
6. Joseph Moore, "Robust Post-Stall Perching with a Fixed-Wing UAV", PhD thesis, Massachusetts Institute of Technology, September, 2014. [ [link](#) ]
7. Christian Ebenbauer and Tobias Raff and Frank Allgower, "Dissipation inequalities in systems theory: An introduction and recent results", *Invited Lectures of the International Congress on Industrial and Applied Mathematics 2007* , pp. 23-42, 2009.
8. Carsten Scherer and Siep Weiland, "Linear {Matrix} {Inequalities} in {Control}", Online Draft , pp. 293, 2015.
9. D. Jacobson, "Optimal stochastic linear systems with exponential performance criteria and their relation to deterministic differential games", *IEEE Transactions on Automatic Control*, vol. 18, no. 2, pp. 124--131, apr, 1973.
10. S. Boyd and S.-J. Kim and L. Vandenberghe and A. Hassibi, "A Tutorial on Geometric Programming", *Optimization and Engineering*, vol. 8, no. 1, pp. 67-127, 2007.
11. Peter Whittle, "Risk-sensitive optimal control", Wiley New York , vol. 20, 1990.
12. Dylan Foster and Alexander Rakhlin, "Beyond {UCB}: Optimal and Efficient Contextual Bandits with Regression Oracles", *Proceedings of the 37th International Conference on Machine Learning* , vol. 119, pp. 3199--3210,



13--18 Jul, 2020.

13. Sergey Levine and Chelsea Finn and Trevor Darrell and Pieter Abbeel, "End-to-end training of deep visuomotor policies", *The Journal of Machine Learning Research*, vol. 17, no. 1, pp. 1334--1373, 2016.
14. D.Q. Mayne and M.M. Seron and SV Rakovic, "Robust model predictive control of constrained linear systems with bounded disturbances", *Automatica*, vol. 41, no. 2, pp. 219--224, 2005.
15. Sadra Sadraddini and Russ Tedrake, "Linear Encodings for Polytope Containment Problems", *Proceedings of the 2019 IEEE 58th Conference on Decision and Control (CDC)* , pp. 8, 2019. [ [link](#) ]

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

---

[Accessibility](#)

© Russ Tedrake, 2024

# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 15

# Output Feedback (aka Pixels-to-Torques)

In this chapter we will finally start considering systems of the form:

$$\begin{aligned}\mathbf{x}[n+1] &= \mathbf{f}(\mathbf{x}[n], \mathbf{u}[n], \mathbf{w}[n]) \\ \mathbf{y}[n] &= \mathbf{g}(\mathbf{x}[n], \mathbf{u}[n], \mathbf{v}[n]),\end{aligned}$$

where most of these symbols have been [described before](#), but we have now added  $\mathbf{y}[n]$  as the output of the system, and  $\mathbf{v}[n]$  which is representing "measurement noise" and is typically the output of a random process. In other words, we'll finally start addressing the fact that we have to make decisions based on sensor measurements -- most of our discussions until now have tacitly assumed that we have access to the true state of the system for use in our feedback controllers (and that's already been a hard problem).

In some cases, we will see that the assumption of "full-state feedback" is not so bad -- we do have good tools for state estimation from raw sensor data. But even our best state estimation algorithms do add some dynamics to the system in order to filter out noisy measurements; if the time constants of these filters is near the time constant of our dynamics, then it becomes important that we include the dynamics of the estimator in our analysis of the closed-loop system.

In other cases, it's entirely too optimistic to design a controller assuming that we will have an estimate of the full state of the system. Some state variables might be completely unobservable, others might require specific "information-gathering" actions on the part of the controller.

For me, the problem of robot manipulation is one important application domain where more flexible approaches to output feedback become critically important. Imagine you are trying to design a controller for a robot that needs to button the buttons on your shirt. Our current tools would require us to first estimate the state of the shirt (how many degrees of freedom does my shirt have?); but certainly the full state of my shirt should not be required to button a single button. Or if you want to program a robot to make a salad -- what's the state of the salad? Do I really need to know the positions and velocities of every piece of lettuce in order to be successful? These questions are (finally) getting a lot of attention in the research community

these days, under the umbrella of "learning state representations". But what does it mean to be a good state representation? There are a number of simple lessons from output feedback in control that can shed light on this fundamental question.

## 15.1 BACKGROUND

### 15.1.1 The classical perspective

To some extent, this idea of calling out "output feedback" as an advanced topic is a relatively new problem. Before state-space and optimization-based approaches to control ushered in "modern control", we had "classical control". Classical control focused predominantly (though not exclusively) on linear time-invariant (LTI) systems, and made very heavy use of frequency-domain analysis (e.g. via the Fourier Transform/Laplace Transform). There are many excellent books on the subject; [1, 2] are nice examples of modern treatments that start with state-space representations but also treat the frequency-domain perspective. "Pole placement" and "loop shaping" are some of the tools of this trade.

What's important for us to acknowledge here is that in classical control, basically everything was built around the idea of output feedback. The fundamental concept is the transfer function of a system, which is a input-to-output map (in frequency domain) that can completely characterize an LTI system. Core concepts like pole placement and loop shaping were fundamentally addressing the challenge of output feedback that we are discussing here. Sometimes I feel that, despite all of the things we've gained with modern, optimization-based control, I worry that we've lost something in terms of considering rich characterizations of closed-loop performance (rise time, dwell time, overshoot, ...) and perhaps even in practical robustness of our systems to unmodeled errors.

### 15.1.2 From pixels to torques

Just like some of our oldest approaches to control were fundamentally solving an output feedback problem, some of our newest approaches to control are doing it, too. Deep learning has revolutionized computer vision, and "deep imitation learning" and "deep reinforcement learning" have been a recent source of many impressive demonstrations of control systems that can operate directly from pixels (e.g. consuming the output of a deep perception system), without explicitly representing nor estimating the full state of the system. Unfortunately, the success or failure of these methods are not yet well understood, and they often require a great deal of artisanal tuning and an embarrassing (sometimes prohibitive) amount of computation.

The synthesis of ideas between machine learning (both theoretical and applied) and control theory is one of the most exciting and productive frontiers for research today. I am highly optimistic that we will be able to uncover the underlying principles and help transition this budding field into a technology. I hope that summarizing some of the key lessons from control here can help.

## 15.2 STATIC OUTPUT FEEDBACK

One of the extremely important almost unstated lessons from dynamic programming with additive costs and the Bellman equation is that the optimal policy can *always* be represented as a function  $\mathbf{u}^* = \pi^*(\mathbf{x})$ . So far in these notes, we've assumed that the controller has direct access to the true state,  $\mathbf{x}$ . In this chapter, we are finally removing that assumption. Now the controller only has direct access to the potentially noisy observations  $\mathbf{y}$ .

So the natural first question to ask might be, what happens if we write our policies now as a function,  $\mathbf{u} = \pi(\mathbf{y})$ ? This is known as "static" output feedback, in contrast

to "dynamic" output feedback where the controller is not a static function, but is itself another input-output dynamical system. Unfortunately, in the general case it is *not* the case that optimal policies can be perfectly represented with static output feedback. But one can still try to solve an optimal control problem where we restrict our search to static policies; our goal will be to find the best controller in this class to minimize the cost.

### 15.2.1 A hardness result

We've already seen [an example](#) of a very simple linear control problem where the set of stabilizing feedback gains formed a disconnected set -- which is suggestive that it could be a difficult problem for optimization. For some other problems in control, we've been able to find a convex reparametrization.

Unfortunately, [3] showed that the question of whether a stabilizing static output feedback  $\mathbf{u} = -\mathbf{K}\mathbf{y}$  even exists for a given system of the form

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u}, \quad \mathbf{y} = \mathbf{C}\mathbf{x},$$

is NP-hard. Many of the strongest results from  $H_2$  and  $H_\infty$  design, for instance, are limited to dynamic controllers that can effectively reconstruct the entire state.

Just because this problem is NP-hard doesn't mean we can't find good controllers in practice. Some of the recent results from reinforcement learning have reminded us of this. We should not expect an efficient globally optimal algorithm that works for every problem instance; but we should absolutely keep working on the problem. Perhaps the class of problems that our robots will actually encounter in the real world is easier than this general case (the standard examples of bad cases in linear systems, e.g. with interleaved poles and zeros, do feel a bit contrived and unlikely to occur in practice).

### 15.2.2 Via policy search

Searching for the best controller within a parametric class of policies is generally referred to as [policy search](#). If we do policy search on a class of static output feedback policies, how well does it perform? Of course, the answer depends on the particular governing equations (for instance,  $\mathbf{y} = \mathbf{x}$  is a perfectly reasonable output, and in this case the policy can be optimal). But we also have very simple [counter-examples](#) which demonstrate that the set of even stabilizing static output feedback controllers can form a disconnected set.

## 15.3 OBSERVER-BASED FEEDBACK

Since we know so much about designing full-state feedback controllers, one of the most natural (and dominant) approaches to control is to first [design an observer](#) (aka "state estimator"), and then to use state feedback. Famously, this approach is actually optimal for the quadratic regular objective on linear-Gaussian systems (LQG) -- this is known as the "separation principle". But it is certainly not optimal in general!

### 15.3.1 Luenberger Observer

### 15.3.2 Linear Quadratic Regulator w/ Gaussian Noise (LQG)

### 15.3.3 Partially-observable Markov Decision Processes

### 15.3.4 Trajectory optimization with Iterative LQG

## 15.4 DISTURBANCE-BASED FEEDBACK

There is an interesting alternative to trying to observe/estimate the true state of the system, which can in some cases lead to convex formulations of the output-feedback objective. Rather than estimate the state (or belief state), one can try to estimate instead the *disturbances* which cause the state to deviate from the nominal trajectory, and design a feedback controller as a function of the disturbance. This is an old but important idea which was made famous first as the [Youla parameterization](#) (alternatively "Q-parameterization"). In the time domain this typically leads to controllers which are "unrolled in time" and depend on a potentially infinite history of disturbances; common practice is to approximate these with a finite-impulse response (FIR) truncation. One could imagine extracting a state-space realization of these FIR responses using the techniques from [linear system identification](#) [4].

We can understand the essence of this idea with a simple extension of the [LQR with least-squares derivation](#)... (it's a work in progress!)

Given the state space equations:

$$\mathbf{x}[n+1] = \mathbf{A}\mathbf{x}[n] + \mathbf{B}\mathbf{u}[n] + \mathbf{w}[n],$$

Consider parameterizing an output feedback policy of the form

$$\mathbf{u}[n] = \mathbf{K}_0[n]\mathbf{x}_0 + \sum_{i=1}^{n-1} \mathbf{K}_i[n]\mathbf{w}[n-i],$$

then the closed-loop state is convex in the control parameters,  $\mathbf{K}$ :

$$\mathbf{x}[n] = \left( \mathbf{A}^n + \sum_{i=0}^{n-1} \mathbf{A}^{n-i-1} \mathbf{B} \mathbf{K}_0[i] \right) \mathbf{x}_0 + \sum_{j=0}^{n-1} \sum_{i=0}^{n-1} \mathbf{A}^{n-i-1} \mathbf{B} \mathbf{K}_j[i] \mathbf{w}[i-j],$$

and therefore objectives that are convex in  $\mathbf{x}$  and  $\mathbf{u}$  (like LQR) are also convex in  $\mathbf{K}$ . Moreover, we can calculate  $\mathbf{w}[n]$  by the time that it is needed given our observations of  $\mathbf{x}[n+1]$ ,  $\mathbf{x}[n]$ , and knowledge of  $\mathbf{u}[n]$ .

We can extend this to output disturbance-based feedback:

$$\begin{aligned} \mathbf{x}[n+1] &= \mathbf{A}\mathbf{x}[n] + \mathbf{B}\mathbf{u}[n] + \mathbf{w}[n], \\ \mathbf{y}[n] &= \mathbf{C}\mathbf{x}[n] + \mathbf{v}[n], \end{aligned}$$

by parameterizing an output feedback policy of the form

$$\mathbf{u}[n] = \mathbf{K}_0[n]\mathbf{y}[0] + \sum_{i=1}^{n-1} \mathbf{K}_i[n]\mathbf{e}[n-i],$$

where  $\mathbf{e}[n] = \dots$  [5].

### 15.4.1 System-Level Synthesis

## 15.5 OPTIMIZING DYNAMIC POLICIES

### 15.5.1 Convex reparameterizations of $H_2$ , $H_\infty$ , and LQG

DGKF (solving two Riccati equations)[6]

Scherer's convex reparameterizations of LQG[7, §4.2].

### 15.5.2 Policy gradient for LQG

### 15.5.3 Sums-of-squares alternations

Coming soon. See, for instance, [8].

### 15.5.4 Teacher-student learning

## 15.6 FEEDBACK FROM PIXELS

In my opinion, one of the most important advances in control in the last decade has been the introduction of high-rate feedback from cameras. This advance was enabled by the revolution in computer vision that came with deep learning. Especially in the domain of robotic manipulation, the value of this feedback is undeniable. Unfortunately, these sensors break many of the synthesis tools that we've discussed in the notes -- not only are they very high dimensional, but the space of RGB images is horrible and non-smooth. As of this writing, conventional wisdom is that model-based control does not have a lot to offer to this problem -- to design control from cameras, we are often limited to either imitation learning or black-box reinforcement learning. (I personally think that we have thrown the baby out with the bathwater, and consider a highly important research area to close this gap.)

### 15.6.1 Visuomotor policies

Visuomotor policies.

Diffusion policies [9]. Transformer parameterizations [10].

## REFERENCES

1. Joao P. Hespanha, "Linear Systems Theory", Princeton Press , 2009.
2. Karl Johan Åström and Richard M Murray, "Feedback systems: an introduction for scientists and engineers", Princeton university press , 2010.
3. Vincent Blondel and John N Tsitsiklis, "{NP}-hardness of some linear control design problems", *SIAM journal on control and optimization*, vol. 35, no. 6, pp. 2118--2127, 1997.
4. James Anderson and Nikolai Matni, "Structured state space realizations for SLS distributed controllers", *2017 55th Annual Allerton Conference on Communication, Control, and Computing (Allerton)* , pp. 982-987, 2017.

5. Sadra Sadraddini and Russ Tedrake, "Robust Output Feedback Control with Guaranteed Constraint Satisfaction", *In the Proceedings of 23rd ACM International Conference on Hybrid Systems: Computation and Control*, pp. 12, April, 2020. [ [link](#) ]
6. John Doyle and Keith Glover and Pramod Khargonekar and Bruce Francis, "State-space solutions to standard  $H_2$  and  $H_\infty$  control problems", *1988 American Control Conference*, pp. 1691--1696, 1988.
7. Carsten Scherer and Siep Weiland, "Linear {Matrix} {Inequalities} in {Control}", Online Draft, pp. 293, 2015.
8. Glen Chou and Russ Tedrake, "Synthesizing Stable Reduced-Order Visuomotor Policies for Nonlinear Systems via Sums-of-Squares Optimization", *Under review*, 2023. [ [link](#) ]
9. Cheng Chi and Siyuan Feng and Yilun Du and Zhenjia Xu and Eric Cousineau and Benjamin Burchfiel and Shuran Song, "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion", *Proceedings of Robotics: Science and Systems*, 2023.
10. Tony Z Zhao and Vikash Kumar and Sergey Levine and Chelsea Finn, "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware", *arXiv preprint arXiv:2304.13705*, 2023.

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

[Accessibility](#)

© Russ Tedrake, 2024

# UNDERACTUATED ROBOTICS

## *Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 16

# Algorithms for Limit Cycles

The discussion of walking and running robots in Chapter 4 motivated the notion of limit cycle stability. Linear systems are not capable of producing stable limit cycle behavior, so this rich topic is unique to nonlinear systems design and analysis. Furthermore, the tools that are required to design, stabilize, and verify limit cycles will have applicability beyond simple periodic motions.

The first natural question we must ask is, given a system  $\dot{\mathbf{x}} = f(\mathbf{x})$ , or a control system  $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$ , how do we go about finding periodic solutions which may be passively stable, open-loop stable, or stabilizable via closed-loop feedback? It turns out that the trajectory optimization tools that we developed already are very well suited to this task.

### 16.1 TRAJECTORY OPTIMIZATION

I introduced the trajectory optimization tools as means for optimizing a control trajectory starting from a particular known initial condition. But the fundamental idea of optimizing over individual trajectories of the system is useful more broadly. Even for a passive system, we can formulate the search for a periodic solution as an optimization over trajectories that satisfy the dynamic constraints and *periodicity constraints*,  $\mathbf{x}[0] = \mathbf{x}[N]$ :

$$\begin{aligned} \text{find}_{\mathbf{x}[\cdot]} \quad \text{subject to} \quad & \mathbf{x}[n+1] = f(\mathbf{x}[n]), \quad \forall n \in [0, N-1] \\ & \mathbf{x}[0] = \mathbf{x}[N]. \end{aligned}$$

Certainly we can add control inputs back into the formulation, too, but let's start with this simple case. Take a moment to think about the feasible solutions to this problem formulation. Certainly a fixed point  $\mathbf{x}[n] = \mathbf{x}^*$  will satisfy the constraints; if we don't want these solutions to come out of the solver we might need to exclude them with constraints or add an objective that guides the solver towards the desired solutions. The other possible solutions are trajectories that are periodic in exactly  $N$  steps. That's pretty restrictive.

We can do better if we use the continuous-time formulations. For instance, in our introduction of [direct collocation](#), we wrote



$$\begin{aligned}
& \min_{\mathbf{x}[\cdot], \mathbf{u}[\cdot]} \quad \ell_f(\mathbf{x}[N]) + \sum_{n_0}^{N-1} h_n \ell(\mathbf{x}[n], \mathbf{u}[n]) \\
& \text{subject to} \quad \dot{\mathbf{x}}(t_{c,n}) = \mathbf{f}(\mathbf{x}(t_{c,n}), \mathbf{u}(t_{c,n})), \quad \forall n \in [0, N-1] \\
& \quad \mathbf{x}[0] = \mathbf{x}_0 \\
& \quad + \text{additional constraints.}
\end{aligned}$$

But we can also add  $h_n$  as decision variables in the optimization (reminder: I recommend setting a lower-bound  $h_n \geq h_{min} > 0$ ). This allows our  $N$ -step trajectory optimization to scale and shrink time in order to satisfy the periodicity constraint. The result is simple and powerful.

### Example 16.1 (Finding the limit cycle of the Van der Pol oscillator)

Recall the dynamics of the Van der Pol oscillator given by

$$\ddot{q} + \mu(q^2 - 1)\dot{q} + q = 0, \quad \mu > 0,$$

which exhibited a stable limit cycle.

Formulate the direct collocation optimization:

$$\begin{aligned}
& \text{find}_{\mathbf{x}[\cdot], h} \quad \text{subject to} \quad q[0] = 0, \quad \dot{q}[0] > 0, \\
& \quad \mathbf{x}[N] = \mathbf{x}[0], \text{ (periodicity constraint)} \\
& \quad \text{collocation dynamic constraints} \\
& \quad 0.01 \leq h \leq 0.5
\end{aligned}$$

Try it yourself:

As always, make sure that you take a look at the code. Poke around. Try changing some things.

One of the things that you should notice in the code is that I provide an initial guess for the solver. In most of the examples so far I've been able to avoid doing that--the solver takes small random numbers as a default initial guess and solves from there. But for this problem, I found that it was getting stuck in a local minima. Adding the initial guess that the solution moves around a circle in state space was enough.

## 16.2 LYAPUNOV ANALYSIS

Recall the important distinction between stability of a trajectory in time and stability of a limit cycle was that the limit cycle does not converge in phase -- trajectories near the cycle converge to the cycle, but trajectories on the cycle may not converge with each other. This type of stability, also known as *orbital stability* can be written as stability to the manifold described by a trajectory  $\mathbf{x}^*(t)$ ,

$$\min_{\tau} \|\mathbf{x}(t) - \mathbf{x}^*(\tau)\| \rightarrow 0.$$

In the case of limit cycles, this manifold is a periodic solution with  $\mathbf{x}^*(t + t_{period}) = \mathbf{x}^*(t)$ . Depending on exactly how that convergence happens, we can define orbital stability in the sense of Lyapunov, asymptotic orbital stability, exponential orbital stability, or even finite-time orbital stability.

In order to prove that a system is orbitally stable (locally, over a region, or globally), or to analyze the region of attraction of a limit cycle, we can use a Lyapunov function. When we analyzed our [simple models of walking and running](#), we carried out the analysis on the Poincare map. Indeed, if we can find a Lyapunov function that proves (i.s.L., asymptotic, or exponential) stability of the discrete Poincare

map, then this implies (i.s.L, asymptotic, or exponential) *orbital* stability of the cycle. But Lyapunov functions of this form are difficult to verify for a pretty fundamental reason: we rarely have an analytical expression for the Poincare map, since it is the result of integrating a nonlinear dynamics over the cycle.

Instead, we will focus our attention on constructing Lyapunov functions in the full state space, which use the continuous dynamics. In particular, we would like to consider Lyapunov functions which have the form cartooned below; they vanish to zero everywhere along the cycle, and are strictly positive everywhere away from the cycle.

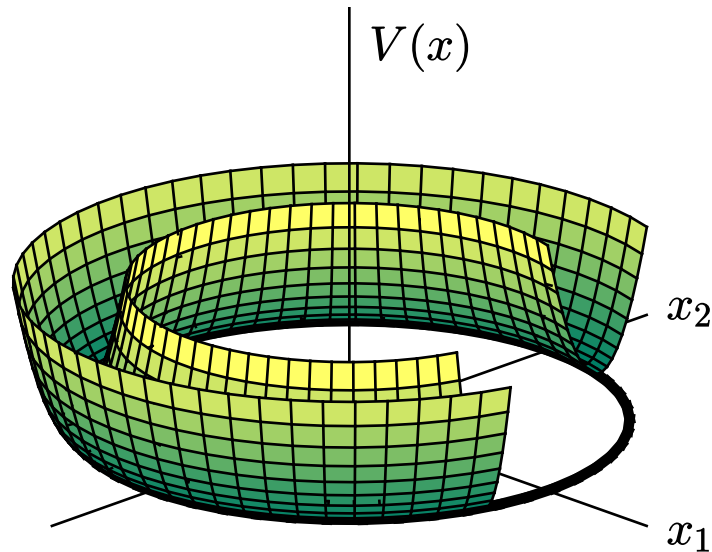


Figure 16.1 - Cartoon of a Lyapunov function which vanishes on a limit cycle, and is strictly positive everywhere else. (a small segment has been removed solely for the purposes of visualization).

### 16.2.1 Transverse coordinates

How can we parameterize this class of functions? For arbitrary cycles this could be very difficult in the original coordinates. For simple cycles like in the cartoon, one could imagine using polar coordinates. More generally, we will define a new coordinate system relative to the orbit, with coordinates

- $\tau$  - the phase along the orbit
- $\mathbf{x}_\perp(\tau)$  - the remaining coordinates, linearly independent from  $\tau$ .

Given a state  $\mathbf{x}$  in the original coordinates, we must define a smooth mapping  $\mathbf{x} \rightarrow (\tau, \mathbf{x}_\perp)$  to this new coordinate system. For example, for a simple ring oscillator we might have:

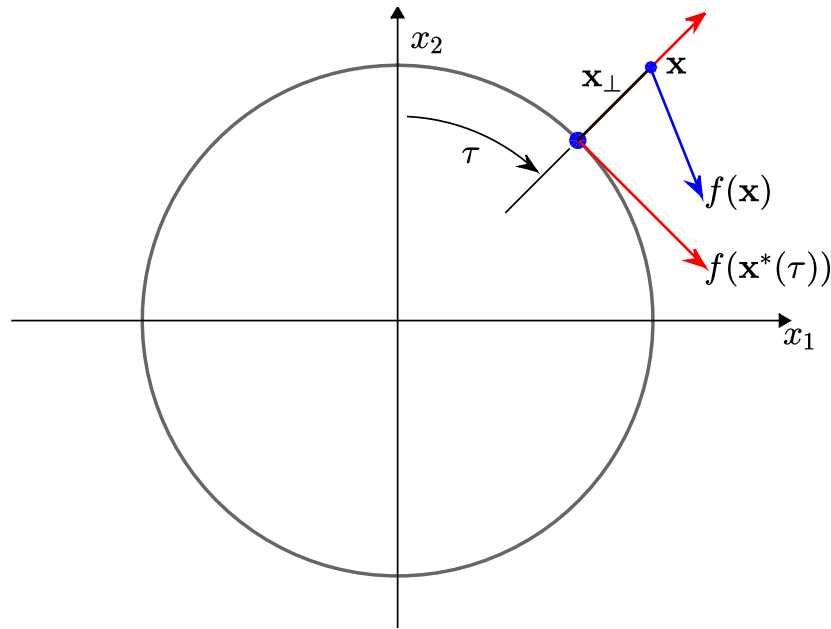


Figure 16.2 - A moving coordinate system along the limit cycle.

In general, for an  $n$ -dimensional state space,  $\tau$  will always be a scalar, and  $\mathbf{x}_\perp$  will be an  $(n - 1)$ -dimensional vector defining the remaining coordinates relative to  $\mathbf{x}^*(\tau)$ . In fact, although we use the notation  $\mathbf{x}_\perp$  the coordinate system need not be strictly orthogonal to the orbit, but must simply be *transversal* (not parallel). Having defined the smooth mapping  $\mathbf{x} \rightarrow (\tau, \mathbf{x}_\perp)$ , we can always rewrite the dynamics in this new coordinate system:

$$\begin{aligned}\dot{\tau} &= f_\tau(\mathbf{x}_\perp, \tau) \\ \dot{\mathbf{x}}_\perp &= f_\perp(\mathbf{x}_\perp, \tau).\end{aligned}$$

To keep our notation simpler for the remainder of these notes, we will assume that the origin of this new coordinate system is on the nominal trajectory ( $\min_\tau |\mathbf{x} - \mathbf{x}^*(\tau)| = 0 \Rightarrow \mathbf{x}_\perp = 0$ ). Similarly, by taking  $\tau$  to be the phase variable, we will leverage the fact that on the nominal trajectory, we have  $\dot{\tau} = f_\tau(0, \tau) = 1$ .

The value of this construction for Lyapunov analysis was proposed in [1] and has been extended nicely to control design in [2] and for region of attraction estimation in [3]. A quite general numerical strategy for defining the transversal coordinates is given in [4].

### Theorem 16.1 - A Lyapunov theorem for orbital stability

For a dynamical system  $\dot{\mathbf{x}} = f(\mathbf{x})$  with  $\mathbf{x} \in \mathbb{R}^n$ ,  $f$  continuous, a continuous periodic solution  $\mathbf{x}^*(\tau)$ , and a smooth mapping  $\mathbf{x} \rightarrow (\tau, \mathbf{x}_\perp)$  where  $\mathbf{x}_\perp$  vanishes on  $\mathbf{x}^*$ , then for some  $n - 1$  dimensional ball  $\mathcal{B}$  around the origin, if I can produce a  $V(\mathbf{x}_\perp, \tau)$  such that

$$\begin{aligned}\forall \tau, V(0, \tau) &= 0, \\ \forall \tau, \forall \mathbf{x}_\perp \in \mathcal{B}, \mathbf{x}_\perp \neq 0, V(\mathbf{x}_\perp, \tau) &> 0,\end{aligned}$$

with

$$\begin{aligned}\forall \tau, \dot{V}(0, \tau) &= 0, \\ \forall \tau, \forall \mathbf{x}_\perp \in \mathcal{B}, \mathbf{x}_\perp \neq 0, \dot{V}(\mathbf{x}_\perp, \tau) &< 0,\end{aligned}$$

then the solution  $\mathbf{x}^*(t)$  is *locally orbitally asymptotically stable*.

Orbital stability in the sense of Lyapunov and exponential orbital stability can also be verified (with  $\dot{V}_\perp \leq 0$  and  $\dot{V}_\perp \leq \alpha V_\perp$ , respectively).

### Example 16.2 (Simple ring oscillator)

Perhaps the simplest oscillator is the first-order system which converges to the unit circle. In cartesian coordinates, the dynamics are

$$\begin{aligned}\dot{x}_1 &= x_2 - \alpha x_1 \left( 1 - \frac{1}{\sqrt{x_1^2 + x_2^2}} \right) \\ \dot{x}_2 &= -x_1 - \alpha x_2 \left( 1 - \frac{1}{\sqrt{x_1^2 + x_2^2}} \right),\end{aligned}$$

where  $\alpha$  is a positive scalar gain.

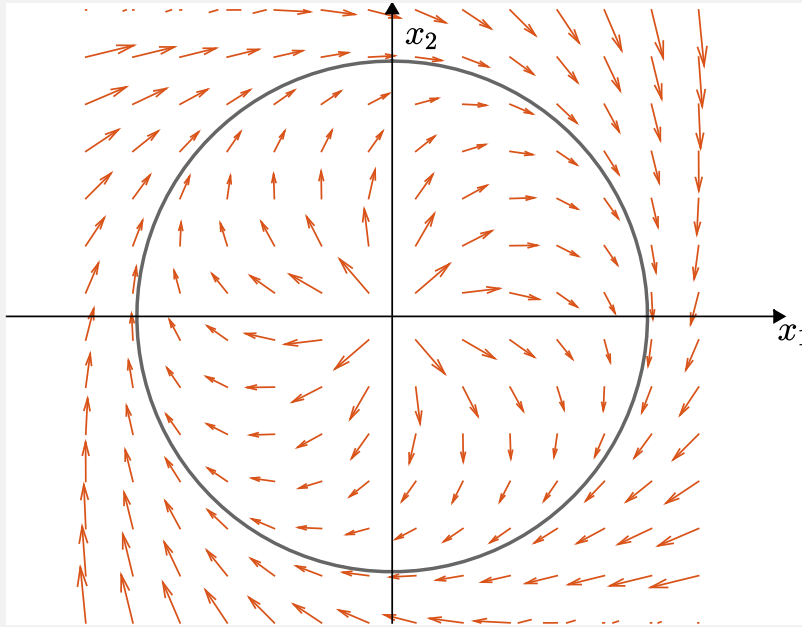


Figure 16.3 - Vector field of the ring oscillator

If we take the transverse coordinates to be the polar coordinates, shifted so that  $x_\perp$  is zero on the unit circle,

$$\begin{aligned}\tau &= \text{atan2}(-x_2, x_1) \\ x_\perp &= \sqrt{x_1^2 + x_2^2} - 1,\end{aligned}$$

which is valid when  $x_\perp > -1$ , then the simple transverse dynamics are revealed:

$$\begin{aligned}\dot{\tau} &= 1 \\ \dot{x}_\perp &= -\alpha x_\perp.\end{aligned}$$

Taking a Lyapunov candidate  $V(x_\perp, \tau) = x_\perp^2$ , we can verify that

$$\dot{V} = -2\alpha x_\perp^2 < 0, \quad \forall x_\perp > -1.$$

This demonstrates that the limit cycle is locally asymptotically stable, and furthermore that the invariant open-set  $V < 1$  is inside the region of attraction of that limit cycle. In fact, we know that all  $x_\perp > -1$  are in the region of attraction

that limit cycle, but this is not proven by the Lyapunov argument above.

Let's compare this approach again with the approach that we used in the chapter on walking robots, where we used a Poincaré map analysis to investigate limit cycle stability. In transverse coordinates approach, there is an additional burden to construct the coordinate system along the entire trajectory, instead of only at a single surface of section. In fact, the transverse coordinates approach is sometimes referred to as a "moving Poincaré section". Again, the reward for this extra work is that we can check a condition that only involves the instantaneous dynamics,  $f(\mathbf{x})$ , as opposed to having to integrate the dynamics over an entire cycle to generate the discrete Poincaré map,  $\mathbf{x}_p[n+1] = P(\mathbf{x}_p[n])$ . As we will see below, this approach will also be more compatible with designing continuous feedback controller that stabilize the limit cycle.

### 16.2.2 Transverse linearization

In the case of Lyapunov analysis around a fixed point, there was an important special case: for stable linear systems, we actually have a recipe for constructing a Lyapunov function. As a result, for nonlinear systems, we often found it convenient to begin our search by linearizing around the fixed point and using the Lyapunov candidate for the linear system as an initial guess. That same approach can be extended to limit cycle analysis.

In particular, let us make a linear approximation of the transverse dynamics:

$$\dot{\mathbf{x}}_{\perp} = f_{\perp}(\mathbf{x}_{\perp}, \tau) \approx \frac{\partial f_{\perp}}{\partial \mathbf{x}_{\perp}} \mathbf{x}_{\perp} = \mathbf{A}_{\perp}(\tau) \mathbf{x}_{\perp}.$$

Note that I've assumed that  $\mathbf{x}_{\perp}$  is zero on the nominal trajectory, so don't need the additional notation of working in the error coordinates here. Remember that  $\tau$  is the phase along the trajectory, but [1] showed that (exponential) stability of the *time-varying* linear system  $\dot{\mathbf{x}}_{\perp} = \mathbf{A}_{\perp}(t) \mathbf{x}_{\perp}$  implies local exponential orbital stability of the original system. In particular, if this transverse linear system is periodic and orbitally stable, then for each  $\mathbf{Q} = \mathbf{Q}^T \succ 0$ , there exists a unique positive definite solution to the periodic Riccati equation:

$$\begin{aligned} V(\mathbf{x}_{\perp}, \tau) &= \mathbf{x}_{\perp}^T \mathbf{P}(\tau) \mathbf{x}_{\perp}, & \mathbf{P}^T(\tau) &= \mathbf{P}(\tau), & \mathbf{P}(\tau + t_{\text{period}}) &= \mathbf{P}(\tau), \\ -\dot{\mathbf{P}}(\tau) &= \mathbf{P}(\tau) \mathbf{A}(\tau) + \mathbf{A}(\tau) \mathbf{P}^T(\tau) + \mathbf{Q}, & \mathbf{Q} &= \mathbf{Q}^T \succ 0. \end{aligned}$$

There is a surprisingly rich literature on the numerical methods for finding the periodic solutions to these Lyapunov (and corresponding Riccati) equations. In practice, for every problem I've ever tried, I've simply integrated the equations backwards in time until the integration converges to a periodic solution (this is not guaranteed to work, but almost always does).

This is very powerful. It gives us a general way to construct a Lyapunov candidate that certifies local stability of the cycle, and which can be used as a candidate for nonlinear Lyapunov analysis.

### 16.2.3 Region of attraction estimation using sums-of-squares

Coming soon. If you are interested, see [4, 3].

## 16.3 FEEDBACK DESIGN

Many of the tools we've developed for stabilization of fixed points or stabilization of trajectories can be adapted to this new setting. When we discussed [control for the spring-loaded inverted pendulum model](#), we considered discrete control decisions, made once per cycle, which could stabilize the discrete Poincare map,  $\mathbf{x}_p[n+1] = f_p(\mathbf{x}_p[n], \mathbf{u}[n])$ , with  $\mathbf{u}[n]$  the once-per-cycle decisions. While it's possible to get the local linear approximations of this map using our adjoint methods, remember that we rarely have an analytical expression for the nonlinear  $f_p$ .

How limiting is it to restrict ourselves to once-per-cycle decisions? Of course we can improve performance in general with continuous feedback. But there are also some cases where once-per-cycle decisions feel quite reasonable. Foot placement for legged robots is a good example -- once we've chosen a foot placement we might make minor corrections just before we place the foot, but rarely change that decision once the foot is on the ground; once per footstep seems like a reasonable approximation. Another good example is flapping flight: here the wing beat cycle times can be much faster than the dynamic timescales of the center of mass, and changing the parameters of the wing stroke one per flap seems pretty reasonable.

But the tools we've developed above also give us the machinery we need to consider continuous feedback throughout the trajectory. Let's look at a few important formulations.

### 16.3.1 For underactuation degree one.

It turns out many of our simple walking models -- particularly ones with a point foot that are derived in the minimal coordinates -- are only short one actuator (between the foot and the ground). One can represent even fairly complex robots this way; much of the theory that I'll elude to here was originally developed by Jessy Grizzle and his group in the context of the bipedal robot [RABBIT](#). Jessy's key observation was that limit cycle stability is effectively stability in  $n - 1$  degrees of freedom, and you can often achieve it easily with  $n - 1$  actuators -- he called this line of work "Hybrid Zero Dynamics" (HZD). We'll deal with the "hybrid" part of that in the next chapter, but here is a simple example to illustrate the "zero dynamics" concept. [Coming soon...]

Stabilizing the zero dynamics is synonymous with achieving orbital stability. Interestingly, it doesn't actually guarantee that you will go around the limit cycle. One "feature" of the HZD walkers is that you can actually stand in front of them and stop their forward progress with your hand -- the control system will continue to act, maintaining itself on the cycle, but progress along the cycle has stopped. In some of the robots you could even push them backwards and they would move through the same walking cycle in reverse. If one wants to certify that forward progress will be achieved, then the task is reduced to inspecting the one-dimensional dynamics of the phase variable along the cycle.

The notion of "zero dynamics" is certainly not restricted to systems with underactuation of degree one. In general, we can easily stabilize a manifold of dimension  $m$  with  $m$  actuators (we saw this in the section on [task-space partial feedback linearization](#)), and if being on that manifold is sufficient to achieve our task, or if we can certify that the resulting dynamics on the manifold are sufficient for our task, then life is good. But in the "underactuation degree one" case, the manifold under study is a trajectory/orbit, and the tools from this chapter are immediately applicable.

### 16.3.2 Transverse LQR

Another natural approach to stabilizing a cycle uses the transverse linearization. In particular, for a nominal (control) trajectory,  $[\mathbf{x}_0(t), \mathbf{u}_0(t)]$ , with  $\dot{\mathbf{x}}_0 = f(\mathbf{x}_0, \mathbf{u}_0)$ , we can

approximate the transverse dynamics:

$$\dot{\mathbf{x}}_{\perp} = \mathbf{f}_{\perp}(\mathbf{x}_{\perp}, \tau, \mathbf{u}) \approx \frac{\partial \mathbf{f}_{\perp}}{\partial \mathbf{x}_{\perp}} \mathbf{x}_{\perp} + \frac{\partial \mathbf{f}_{\perp}}{\partial \mathbf{u}} (\mathbf{u} - \mathbf{u}_0(\tau)) = \mathbf{A}_{\perp}(\tau) \mathbf{x}_{\perp} + \mathbf{B}_{\perp}(\tau) \bar{\mathbf{u}}.$$

[2] showed that the (periodic) time-varying LQR controller that stabilized this system achieves *orbital* stabilization of the original system.

Let's take a minute to appreciate the difference between this approach and time-varying LQR in the full coordinates. Of course, the behavior is quite different: time-varying LQR in the full coordinates will try to slow down or speed up to keep time with the nominal trajectory, where this controller makes no attempt to stabilize the phase variable. You might think you could design the original time-varying controller,  $\bar{\mathbf{u}} = \mathbf{K}(t)\bar{\mathbf{x}}$ , and just "project to the closest time" during execution (e.g. find the time  $t = \operatorname{argmin}_{\tau} \|\mathbf{x} - \mathbf{x}_0(\tau)\|$ ) and use the feedback corresponding to that time. But this projection is not guaranteed to be safe; you can find systems that are stabilizable by the transverse linearization are unstable in closed-loop using a feedback based on this projection.

### **Example 16.3 (Time-varying LQR simply projected in time can lead to instability.)**

Coming soon...

But there is an even more important reason to like the transverse LQR approach. If orbital stability is all you need, then this is a better formulation of the problem because you are asking for less. As we've just discussed with hybrid zero-dynamics, stabilizing  $m$  degrees of freedom with  $m$  actuators is potentially very different than stabilizing  $m + 1$  degrees of freedom. In the transverse formulation, we are asking LQR to minimize the cost only in the transverse coordinates (it is mathematically equivalent to designing a cost function in the full coordinates that is zero in the direction of the nominal trajectory). In practice, this can result in much smaller cost-to-go matrices,  $\mathbf{S}(t)$ , and smaller feedback gains,  $\mathbf{K}(t)$ . For underactuated systems, this difference can be dramatic.

### **16.3.3 Orbital stabilization for non-periodic trajectories**

The observation that orbital stabilization of a trajectory can ask much less of your underactuated control system (and lead to smaller LQR feedback gains), is quite powerful. It is not limited to stabilizing periodic motions. If your aim is to stabilize a non-periodic path through state-space, but do not actually care about the timing, then formulating your stabilization can be a very good choice. You can find examples where a system is stabilizable in the transverse coordinates, but not in the full coordinates.

### **Example 16.4 (Stabilizable in the transverse coordinates, but not the full coordinates.)**

Coming soon...

Take care, though, as the converse can also be true: it's possible that a system could be stabilizable in the full coordinates but *not* in some transverse coordinates; if you choose those coordinates badly.

### Example 16.5 (Stabilizable in the full coordinates, but not the transverse coordinates.)

Consider a trajectory with  $u(t) = 1$  (from any initial condition) for [the double integrator](#). We can certainly stabilize this trajectory in the full coordinates using LQR. Technically, one can choose transverse coordinates  $\tau = \dot{q}$ , and  $x_{\perp} = q$ . Clearly  $x_{\perp}$  is transverse to the nominal trajectory everywhere. But this transverse linearization is also clearly not stabilizable. It's a bad choice!

The general approach to designing the transverse coordinates [\[4\]](#) could address this concern by including a criteria for controllability when optimizing the transverse coordinates.

## REFERENCES

1. John Hauser and Chung Choo Chung, "Converse Lyapunov functions for exponentially stable periodic orbits", *Systems & Control Letters*, vol. 23, no. 1, pp. 27 -- 34, 1994.
2. Anton S. Shiriaev and Leonid B. Freidovich and Ian R. Manchester, "Can We Make a Robot Ballerina Perform a Pirouette? Orbital Stabilization of Periodic Motions of Underactuated Mechanical Systems", *Annual Reviews in Control*, vol. 32, no. 2, pp. 200--211, Dec, 2008.
3. Ian R. Manchester and Mark M. Tobenkin and Michael Levashov and Russ Tedrake, "Regions of Attraction for Hybrid Limit Cycles of Walking Robots", *Proceedings of the 18th IFAC World Congress, extended version available online: arXiv:1010.2247 [math.OC]*, 2011.
4. Ian R. Manchester, "Transverse Dynamics and Regions of Stability for Nonlinear Hybrid Limit Cycles", *Proceedings of the 18th IFAC World Congress, extended version available online: arXiv:1010.2241 [math.OC]*, Aug-Sep, 2011.



# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 17

# Planning and Control through Contact

So far we have developed a fairly strong toolbox for planning and control with "smooth" systems -- systems where the equations of motion are described by a function  $\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u})$  which is smooth everywhere. But our [discussion of the simple models of legged robots](#) illustrated that the dynamics of making and breaking contact with the world are more complex -- these are often modeled as hybrid dynamics with impact discontinuities at the collision event and constrained dynamics during contact (with either soft or hard constraints).

My goal for this chapter is to extend our computational tools into this richer class of models. Many of our core tools still work: trajectory optimization, Lyapunov analysis (e.g. with sums-of-squares), and LQR all have natural equivalents.

Let's start with a warm-up exercise: trajectory optimization for the rimless wheel. We already have basically everything that we need for this, and it will form a nice basis for generalizing our approach throughout the chapter.

### Example 17.1 (Trajectory optimization for the rimless wheel)

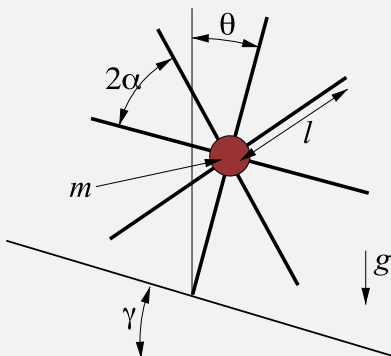


Figure 17.1 - The rimless wheel. The orientation of the stance leg,  $\theta$ , is measured

clockwise from the vertical axis.

The **rimless wheel** was our simplest example of a passive-dynamic walker; it has no control inputs but exhibits a passively stable rolling fixed point. We've also **already seen** that trajectory optimization can be used as a tool for finding limit cycles of a smooth passive system, e.g. by formulating a direct collocation problem:

$$\begin{aligned} \text{find}_{\mathbf{x}[\cdot], h} \quad \text{subject to} \quad & \text{collocation constraints}(\mathbf{x}[n], \mathbf{x}[n+1], h), \quad \forall n \in [0, N-1] \\ & \mathbf{x}[0] = \mathbf{x}[N], \\ & h_{\min} \leq h \leq h_{\max}, \end{aligned}$$

where  $h$  was the time step between the trajectory break points.

It turns out that applying this to the rimless wheel is quite straight forward. We still want to find a periodic trajectory, but now have to take into account the collision event. We can do this by modifying the periodicity condition. Let's force the initial state to be just after the collision, and the final state to be just before the collision, and make sure they are related to each other via the collision equation:

$$\begin{aligned} \text{find}_{\mathbf{x}[\cdot], h} \quad \text{subject to} \quad & \text{collocation constraints}(\mathbf{x}[n], \mathbf{x}[n+1], h), \quad \forall n \in [0, N-1] \\ & \theta[0] = \gamma - \alpha, \\ & \theta[N] = \gamma + \alpha, \\ & \dot{\theta}[0] = \dot{\theta}[N] \cos(2\alpha) \\ & h_{\min} \leq h \leq h_{\max}. \end{aligned}$$

Although it is likely not needed for this simple example (since the dynamics are sufficiently limited), for completeness one should also add constraints to ensure that none of the intermediate points are in contact,

$$\gamma - \alpha \leq \theta[n] < \gamma + \alpha, \quad \forall n \in [1, N-1].$$

The result is a simple and clean numerical algorithm for finding the rolling limit cycle solution of the rimless wheel. Please take it for a spin:

The specific case of the rimless wheel is quite clean. But before we apply it to the compass gait, the kneed compass gait, the spring-loaded inverted pendulum, etc, then we should stop and figure out a more general form.

## 17.1 (AUTONOMOUS) HYBRID SYSTEMS

Recall how we modeled the dynamics of the simple legged robots. First, we derived the equations of motion (independently) for each possible contact configuration -- for example, in the spring-loaded inverted pendulum (SLIP) model we had one set of equations governing the  $(x, y)$  position of the mass during the flight phase, and a completely separate set of equations written in polar coordinates,  $(r, \theta)$ , describing the stance phase. Then we did a little additional work to describe the transitions between these models -- e.g., in SLIP we transitioned from flight to stance when the foot first touches the ground. When simulating this model, it means that we have a discrete "event" which occurs at the moment of foot collision, and an immediate discontinuous change to the state of the robot (in this case we even change out the state variables).

The language of **hybrid systems** gives us a rich language for describing systems of this form, and a suite of tools for analyzing and controlling them. The term "hybrid systems" is a bit overloaded, here we use "hybrid" to mean both discrete- and continuous-time, and the particular systems we consider here are sometimes called **autonomous** hybrid systems because the internal dynamics can cause the discrete changes without any exogeneous input<sup>†</sup>. In the hybrid systems formulation, we describe a system by a set of **modes** each described by (ideally smooth) continuous dynamics, a set of **guards** which here are continuous functions whose zero-level set

<sup>†</sup>This is in contrast to, for instance, the model of a power-train where

describes the conditions which trigger an event, and a set of *resets* which describe the discrete update to the state that is triggered by the guard. Each guard is associated with a particular mode, and we can have multiple guards per mode. Every guard has at most one reset. You will occasionally hear guards referred to as "witness functions", since they play that role in simulation, and resets are sometimes referred to as "transition functions".

The imagery that I like to keep in my head for hybrid systems is illustrated below for a simple example of a robot's heel striking the ground. A solution trajectory of the hybrid system has a continuous trajectory inside each mode, punctuated by discrete updates when the trajectory hits the zero-level set of the guard (here the distance between the heel and the ground becomes zero), with the reset describing the discrete change in the state variables.

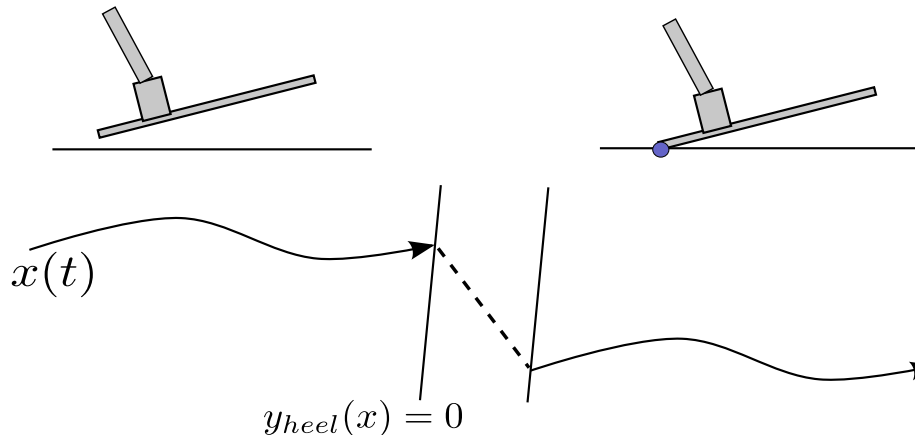


Figure 17.2 - Modeling contact as a hybrid system.

For this robot foot, we can decompose the dynamics into distinct modes: (1) foot in the air, (2) only heel on the ground, (3) heel and toe on the ground, (4) only toe on the ground (push-off). More generally, we will write the dynamics of mode  $i$  as  $\mathbf{f}_i$ , the guard which signals the transition mode  $i$  to mode  $j$  as  $\phi_{i,j}$  (where  $\phi_{i,j}(\mathbf{x}_i) > 0$  inside mode  $i$ ), and the reset map from  $i$  to  $j$  as  $\Delta_{i,j}$ , as illustrated in the following figure:

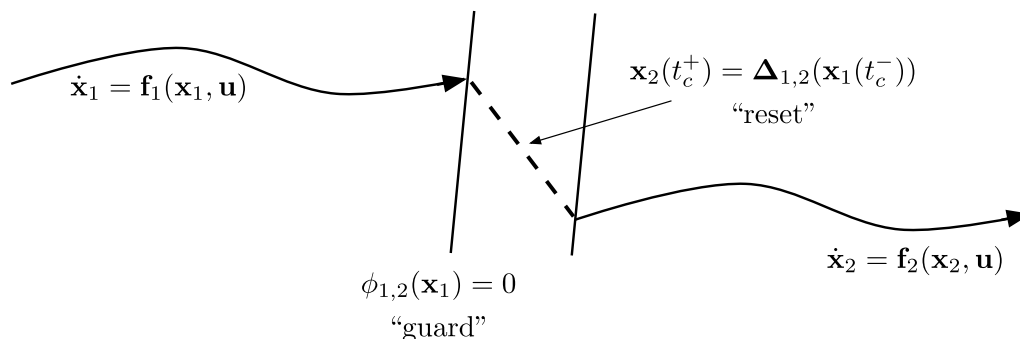


Figure 17.3 - The language of hybrid systems: modes, guards, and reset maps.

### 17.1.1 Hybrid trajectory optimization

Using the more general language of modes, guards, and resets, we can begin to formulate the "hybrid trajectory optimization" problem. In hybrid trajectory optimization, there is a major distinction between trajectory optimization where *the mode sequence is known a priori* and the optimization is just attempting to solve for the continuous-time trajectories, vs one in which we must also discover the mode sequence.

#### Given a fixed mode sequence

For the case when the mode sequence is fixed, then hybrid trajectory optimization can be as simple as stitching together multiple individual mathematical programs into a single mathematical program, with the boundary conditions constrained to enforce the guard/reset constraints. Using the shorthand  $\mathbf{x}_k$  for the state in the  $k$ th segment of the sequence, and  $m_k$  for the mode in segment  $k$ , we can write:

$$\begin{aligned} \text{find } \mathbf{x}_k[\cdot], h_k \quad & \text{subject to} \quad \mathbf{x}_0[0] = \mathbf{x}_0, \\ \forall k, \forall n_k \in [0, N_k - 1], \quad & \text{collocation constraints}_{m_k}(\mathbf{x}_k[n_k], \mathbf{x}_k[n_k + 1], h_k[n_k]), \\ & h_{\min} \leq h_k[n_k] \leq h_{\max}, \\ \forall k \in [0, K - 1] \quad & \phi_{m_k, m_{k+1}}(\mathbf{x}_k[N_k]) = 0, \\ & \mathbf{x}_{k+1}[0] = \Delta_{m_k, m_{k+1}}(\mathbf{x}_k[N_k]), \\ \forall k, n_k, m \in G \quad & \phi_{m_k, m}(\mathbf{x}_k[n_k]) > 0. \end{aligned}$$

The constraints in the last line ensure that no trajectories intersect an *unscheduled* guard, and  $G$  is taken to be the tuples of  $k, n_k, m$  which list all guards and times *except* the scheduled ones. It is then natural to add control inputs (as additional decision variables), and to add an objective and any more constraints.

### Example 17.2 (A basketball trick shot.)

As a simple example of this hybrid trajectory optimization, I thought it would be fun to see if we can formulate the search for initial conditions that optimizes a basketball "trick shot". A quick search turned up [this video](#) for inspiration.

Let's start simpler -- with just a "bounce pass". We can capture the dynamics of a bouncing ball (in the plane, ignoring spin) with some very simple dynamics:

$$\mathbf{q} = \begin{bmatrix} x \\ z \end{bmatrix}, \quad \ddot{\mathbf{q}} = \begin{bmatrix} 0 \\ -g \end{bmatrix}.$$

During any time interval without contact of duration  $h$ , we can actually integrate these dynamics perfectly:

$$\mathbf{x}(t+h) = \begin{bmatrix} x(t) + h\dot{x}(t) \\ z(t) + h\dot{z}(t) - \frac{1}{2}gh^2 \\ \dot{x}(t) \\ \dot{z}(t) - hg \end{bmatrix}.$$

With the bounce pass, we just consider collisions with the ground, so we have a guard,  $\phi(\mathbf{x}) = z$ , which triggers when  $z = 0$ , and a reset map which assumes an elastic collision with *coefficient of restitution*  $e$ :

$$\mathbf{x}^+ = \Delta(\mathbf{x}^-) = [x^- \quad z^- \quad \dot{x}^- \quad -e\dot{z}^-]^T.$$

We'll formulate the problem as this: given an initial ball position ( $x = 0, z = 1$ ), a final ball position 4m away ( $x = 4, z = 1$ ), find the initial velocity to achieve that goal in 5 seconds. Clearly, this implies that  $\dot{x}(0) = 4/5$ . The interesting question is -- what should we do with  $\dot{z}(0)$ ? There are multiple solutions -- which involve bouncing a different number of times. We can find them all with a simple hybrid trajectory optimization, using the observation that there are two possible solutions for each number of bounces -- one that starts with a positive  $\dot{z}(0)$  and one with a negative  $\dot{z}(0)$ .

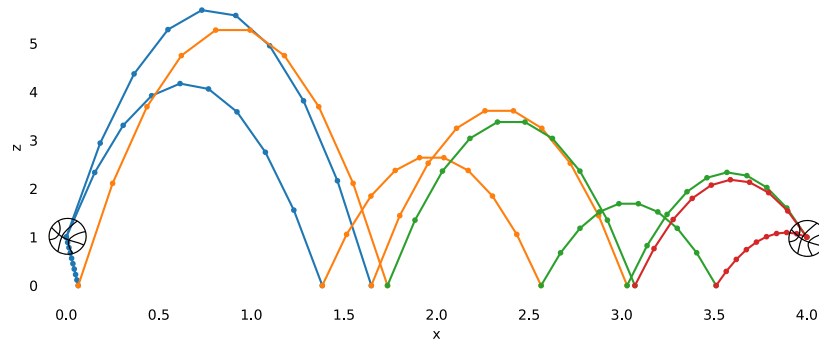


Figure 17.4 - Trajectory optimization to find solutions for a "bounce pass". I've plotted all solutions that were found for 2, 3, or 4 bounces... but I think it's best to stick to a single bounce if you're using this on the court.

Now let's try our trick shot. I'll move our goal to  $x_f = -1m, z_f = 3m$ , and introduce a vertical wall at  $x = 0$ , and move our initial conditions back to  $x_0 = -.25m$ . The collision dynamics, which now must take into account the spin of the ball, are [in the appendix](#). The first bounce is against the wall, the second is against the floor. I'll also constrain the final velocity to be down (have to approach the hoop from above). Try it out.

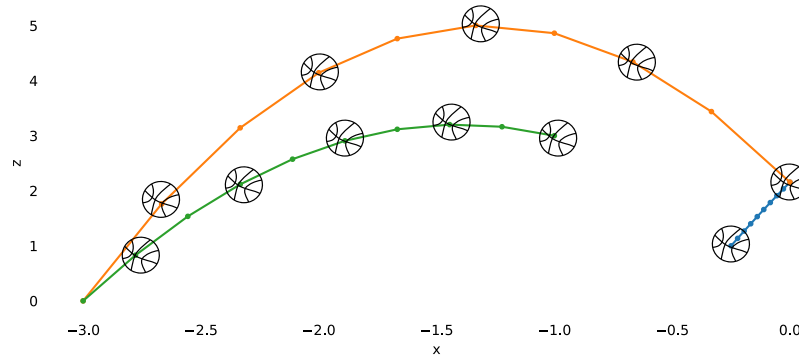


Figure 17.5 - Trajectory optimization for the "trick shot". Nothing but the bottom of the net! The crowd is going wild!

In this example, we could integrate the dynamics in each segment analytically. That is the exception, not the rule. But you can take the same steps with a little more code to use, e.g. direct transcription or collocation with multiple break points in each segment.

## Direct shooting

It is also possible to optimize the trajectory in a single shot by taking gradients through the guard/reset map. This method does not require an explicit mode sequence, but is prone to having local minima (since there is no gradient to "pull" the trajectory towards a guard that is not visited along the nominal trajectory).

In order to achieve this, we need to take the gradient of the cost with respect to the trajectory parameters. The methods we've developed previously for calculating gradients using the adjoint method (e.g. "backpropagation through time") in [discrete](#) and [continuous](#) time can be combined to handle the hybrid case. The derivation is [almost](#) as simple as using the continuous adjoint equation to solve (from the final time forward) for the continuous modes, and the discrete adjoint equation to handle the jump discontinuity. The only detail is that we also have to consider variations

relative to the contact time; this gradient is often referred to as the "Saltation matrix".

Consider the case of two modes and one transition in our canonical hybrid system picture above. Then one could write the total cost as

$$J_\alpha = \int_0^{t_c^-} \ell_1(\mathbf{x}_1, \mathbf{u}_\alpha) dt + \int_{t_c^+}^{t_f} \ell_2(\mathbf{x}_2, \mathbf{u}_\alpha) dt,$$

$$\dot{\mathbf{x}}_1 = \mathbf{f}_1(\mathbf{x}_1, \mathbf{u}_\alpha), \quad \dot{\mathbf{x}}_2 = \mathbf{f}_2(\mathbf{x}_2, \mathbf{u}_\alpha), \quad \mathbf{x}_2(t_c^+) = \mathbf{\Delta}_{1,2}(\mathbf{x}_1(t_c^-)).$$

We wish to take the gradient of  $J$  with respect to the trajectory parameters,  $\alpha$ . The gradient of the first and second integrals is exactly the familiar continuous-time case, but the second integral must be initialized with  $\frac{\partial \mathbf{x}_2(t_c^+)}{\partial \alpha_i}$ , which should be related to  $\frac{\partial \mathbf{x}_1(t_c^-)}{\partial \alpha_i}$  via the discrete transition. The important point is to capture the dependence of this transition not only on  $\mathbf{x}$  but also on  $t_c$ , which both depend on  $\alpha$ . Writing it out we have...

## Deriving hybrid models: minimal vs floating-base coordinates

There is some work to do in order to derive the equations of motion in this form. Do you remember how we did it for the [rimless wheel and compass gait](#) examples? In both cases we assumed that exactly one foot was attached to the ground and that it would not slip, this allowed us to write the Lagrangian as if there was a pin joint attaching the foot to the ground to obtain the equations of motion. For the SLIP model, we derived the flight phase and stance phase using separate Lagrangian equations each with different state representations. I would describe this as the [minimal coordinates](#) modeling approach -- it is elegant and has some important computational advantages that we will come to appreciate in the algorithms below. But it's a lot of work! For instance, if we also wanted to consider friction in the foot contact of the rimless wheel, we would have to derive yet another set of equations to describe the sliding mode (adding, for instance, a prismatic joint that moved the foot along the ramp), plus the guards which compute the contact force for a given state and the distance from the boundary of the friction cone, and on and on.

Fortunately, there is an alternative modeling approach for deriving the modes, guards, and resets for contact that is more general (though it makes the dynamics of each mode a bit more complex). We can instead model the robot in the [floating-base coordinates](#) -- we add a fictitious six degree-of-freedom "floating-base" joint connecting some part of the robot to the world (in planar models, we use just three degrees-of-freedom, e.g.  $(x, z, \theta)$ ). We can derive the equations of motion for the floating-base robot once, without considering contact, then add the additional constraints that come from being in contact as contact forces which get applied to the bodies. The resulting manipulator equations take the form

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u} + \sum_i \mathbf{J}_i^T(\mathbf{q})\lambda_i, \quad (1)$$

where  $\lambda_i$  are the constraint forces and  $\mathbf{J}_i$  are the constraint Jacobians. Conveniently, if the guard function in our contact equations is the signed distance from contact,  $\phi_i(\mathbf{q})$ , then this Jacobian is simply  $\mathbf{J}_i(\mathbf{q}) = \frac{\partial \phi_i}{\partial \mathbf{q}}$ . I've written the basic derivations for the common cases (position constraints, velocity constraints, impact equations, etc) in the [appendix](#). What is important to understand here is that this is an alternative formulation for the equations governing the modes, guards, and resets, but that is it no longer a minimal coordinate system -- the equations of motion are written in  $2N$  state variables but the system might actually be constrained to evolve only along a lower dimensional manifold (if we write the rimless wheel equations with three configuration variables for the floating base, it still only rotates around the toe when it is in stance and is inside the friction cone). This will have implications for our algorithms.

Note that there are some subtleties about using direct collocation methods with

systems parameterized in the floating-base coordinates with additional contact constraints -- the naive implementation of direct collocation may not have enough degrees of freedom in the spline to satisfy the dynamics constraints exactly at the knot points and the collocation points. [1] shows the natural extension of direct collocation to constrained systems; and describes more completely the advantages of parameterizing contact forces as decision variables.

For completeness, you might also hear the term *maximal coordinates*. This is the extreme case where every rigid body in the robot is parameterized with its own floating-base, and even the robot joints are implemented as constraints holding the links together. This has been used by some notable simulation engines (e.g. the *Open Dynamics Engine*), and may have advantages for some computational methods. But for our purposes, using the dynamics in floating-base coordinates is typically much more effective.

### 17.1.3 Discrete control (between events)

In our discussion of the *SLIP model* of running robots, we introduced a surprisingly simple but effective approach to achieving a "deadbeat" controller, with control decisions happening just once per cycle. There is a natural generalization of this idea to more general hybrid systems...

### 17.1.4 Hybrid LQR

Coming soon (see [2]).

### 17.1.5 Hybrid Lyapunov analysis

Coming soon (see [3, 4]).

## 17.2 CONTACT-IMPLICIT TRAJECTORY OPTIMIZATION

Coming soon. (for starters, see [5], or [6] for a recent survey).

## 17.3 EXERCISES

### Exercise 17.1 (Finding the compass gait limit cycle)

In this exercise we use trajectory optimization to identify a limit cycle for the compass gait. We use a rather general approach: the robot dynamics is described in floating-base coordinates and frictional contacts are accurately modeled. , you are asked to code many of the constraints this optimization problem requires:

- a. Enforce the contact between the stance foot and the ground at all the break points.
- b. Enforce the contact between the swing foot and the ground at the initial time.
- c. Prevent the penetration of the swing foot in the ground at all the break points. (In this analysis, we will neglect the scuffing between the swing foot and the ground which arises when the swing leg passes the stance leg.)
- d. Ensure that the contact force at the stance foot lies in the friction cone at all the break points.
- e. Ensure that the impulse generated by the collision of the swing foot with the ground lies in the friction cone.

## REFERENCES

1. Michael Posa and Scott Kuindersma and Russ Tedrake, "Optimization and stabilization of trajectories for constrained dynamical systems", *Proceedings of the International Conference on Robotics and Automation (ICRA)*, pp. 1366-1373, May, 2016. [ [link](#) ]
2. Ian R. Manchester and Uwe Mettin and Fumiya Iida and Russ Tedrake, "Stable Dynamic Walking Over Uneven Terrain", *The International Journal of Robotics Research (IJRR)*, vol. 30, no. 3, pp. 265-279, January 24, 2011. [ [link](#) ]
3. Ian R. Manchester, "Transverse Dynamics and Regions of Stability for Nonlinear Hybrid Limit Cycles", *Proceedings of the 18th IFAC World Congress, extended version available online: arXiv:1010.2241 [math.OC]*, Aug-Sep, 2011.
4. Ian R. Manchester and Mark M. Tobenkin and Michael Levashov and Russ Tedrake, "Regions of Attraction for Hybrid Limit Cycles of Walking Robots", *Proceedings of the 18th IFAC World Congress, extended version available online: arXiv:1010.2247 [math.OC]*, 2011.
5. Michael Posa and Cecilia Cantu and Russ Tedrake, "A Direct Method for Trajectory Optimization of Rigid Bodies Through Contact", *International Journal of Robotics Research*, vol. 33, no. 1, pp. 69-81, January, 2014. [ [link](#) ]
6. Patrick M Wensing and Micheal Posa and Yue Hu and Adriend Escande and Nicolas Mansard and Andrea Del Prete, "Optimization-based control for dynamic legged robots", *arXiv preprint arXiv:2211.11644*, 2022.



# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 18

# System Identification

My primary focus in these notes has been to build algorithms that design or analyze a control system *given a model* of the plant. In fact, we have in some places gone to great lengths to understand the structure in our models (the structure of the manipulator equations in particular) and tried to write algorithms which exploit that structure.

Our ambitions for our robots have grown over recent years to where it makes sense to question this assumption. If we want to program a robot to fold laundry, spread peanut butter on toast, or make a salad, then we should absolutely not assume that we are simply given a model (and the ability to estimate the state of that model). This has led many researchers to focus on the "model-free" approaches to optimal control that are popular in reinforcement learning. But I worry that the purely model-free approach is "throwing the baby out with the bathwater". We have fantastic tools for making long-term decisions given a model; the model-free approaches are correspondingly much much weaker.

So in this chapter I would like to cover the problem of learning a model. This is far from a new problem. The field of "system identification" is as old as controls itself, but new results from machine learning have added significantly to our algorithms and our analyses, especially in the high-dimensional and finite-sample regimes. But well before the recent machine learning boom, system identification as a field had a very strong foundation with thorough statistical understanding of the basic algorithms, at least in the asymptotic regime (the limit where the amount of data goes to infinity). Machine learning theory has brought a wealth of new results in the online optimization and finite-sample regimes. My goal for this chapter is to establish these foundations, and to provide some pointers so that you can learn more about this rich topic.

## 18.1 PROBLEM FORMULATION

### 18.1.1 Equation error vs simulation error



Our problem formulation inevitably begins with the data. In practice, if we have access to a physical system, instrumented using digital electronics, then we have a system in which we can apply input commands,  $\mathbf{u}_n$ , at some discrete rate, and measure the outputs,  $\mathbf{y}_n$  of the system at some discrete rate. We normally assume these rates are fixed, and often attempt to fit a state-space model of the form

$$\mathbf{x}[n+1] = f_\alpha(\mathbf{x}[n], \mathbf{u}[n]), \quad \mathbf{y}[n] = g_\alpha(\mathbf{x}[n], \mathbf{u}[n]), \quad (1)$$

where I have used  $\alpha$  again here to indicate a vector of parameters. In this setting, a natural formulation is to minimize a least-squares estimation objective:

$$\min_{\alpha, \mathbf{x}[0]} \sum_{n=0}^{N-1} \|\mathbf{y}[n] - \mathbf{y}_n\|_2^2, \quad \text{subject to (1)}.$$

I have written purely deterministic models to start, but in general we expect both the state evolution and the measurements to have randomness. Sometimes, as I have written, we fit a deterministic model to the data and rely on our least-squares objective to capture the errors; more generally we will look at fitting stochastic models to the data.

We often separate the identification procedure into two parts, in which we first estimate the state  $\hat{\mathbf{x}}_n$  given the input-output data  $\mathbf{u}_n, \mathbf{y}_n$ , and then focus on estimating the state-evolution dynamics in a second step. The dynamics estimation algorithms fall into two main categories:

- *Equation error* minimizes only the one-step prediction error:

$$\min_{\alpha} \sum_{n=0}^{N-2} \|f_\alpha(\hat{\mathbf{x}}_n, \mathbf{u}_n) - \hat{\mathbf{x}}_{n+1}\|_2^2.$$

- *Simulation error* captures the long-term prediction error:

$$\min_{\alpha} \sum_{n=1}^{N-1} \|\mathbf{x}[n] - \hat{\mathbf{x}}_n\|_2^2, \quad \text{subject to } \mathbf{x}[n+1] = f_\alpha(\mathbf{x}[n], \mathbf{u}_n), \mathbf{x}[0] = \hat{\mathbf{x}}_0,$$

The equation-error formulations often result in much more tractable optimization problems, but unfortunately we will see that optimizing the one-step error can still result in arbitrarily large simulation errors. Therefore, we generally consider the simulation error to be the true objective we hope to optimize, and the equation error only as a potentially useful surrogate.

Within this framework of model learning, we will face all of the standard questions from supervised learning about sample efficiency and generalization.

### 18.1.2 Online optimization

Theoretical machine learning has brought a number of fresh perspectives to this old problem of system identification. Perhaps most relevant is the perspective of online optimization [1, 2, 3]. In addition to changing the formulation to considering streaming online data (as opposed to offline, batch analysis), this field has

(re)popularized the notion of an old idea from decision theory: using *regret* as the loss function for identification.

In the regret formulation, we acknowledge that the class of parameterized models that we are searching over can likely not perfectly represent the input-output data. Rather than score ourselves directly on the prediction error, we can score ourselves relative to the predictions that would be made by best parameters in our model class. For equation-error formulations, the regret  $R$  at step  $N$  would take the form

$$R[N] = \sum_{n=0}^{N-2} \|f_{\alpha}(\hat{\mathbf{x}}_n, \mathbf{u}_n) - \hat{\mathbf{x}}_{n+1}\|_2^2 - \min_{\alpha^*} \sum_{n=0}^{N-2} \|f_{\alpha^*}(\hat{\mathbf{x}}_n, \mathbf{u}_n) - \hat{\mathbf{x}}_{n+1}\|_2^2.$$

The goal is to produce an online learning algorithm to select  $\alpha$  at each step in order to drive the regret to zero. This turns out to be an important distinction, as we will see below.

### 18.1.3 Learning models for control

So far we've described the problem purely as capturing the input-output behavior of the system. If our goal is to use this model for control, then that might not be quite the right objective for a number of reasons.

First of, predicting the observations might be *more* than we need for making optimal decisions; [4] tells that story nicely. Imagine you are trying to swing-up and balance a cart-pole system, where the only sensor is a camera. The background images are irrelevant; we know that only the state of the cart-pole should matter. If there was a movie playing on a screen in the background, you don't want to spend all of the representational power of your model predicting the next frame of the movie, at the cost of not predicting as well the next state of the cart-pole. These challenges have led to a nice active area of research on learning *task-relevant* models/representations; I'll devote a section to it below after we cover the basics.

There are other considerations, as well. To be useful for control, we'll need the state of our model to be observable (from online observations) and controllable. There may also be trade-offs in model complexity. If we learn a just linear model of the dynamics then we have very powerful control design tools available, but we might not capture the rich dynamics of the world. If we learn too complex of a model, then our online planning and control techniques might become a bottleneck. [5] makes that point nicely. At the time of this writing, I think it's broadly fair to say that deep learning models are able to describe incredibly rich dynamics, but that our control tools for making decisions with these models are still painfully weak.

## 18.2 PARAMETER IDENTIFICATION FOR MECHANICAL SYSTEMS

My primary focus throughout these notes is on (underactuated) mechanical systems, but when it comes to identification there is an important distinction to make. For some mechanical systems we know the structure of the model, including number of state variables and the topology of the kinematic tree. Legged robots like Spot or Atlas are good examples here -- the dynamics are certainly nontrivial, but the general form of the equations are known. In this case, the task of identification is really the task of estimating the parameters in a structured model. That is the subject of this section.

The examples of folding laundry or making a salad fall into a different category. In those examples, I might not even know a priori the number of state variables needed to provide a reasonable description of the behavior. That will force a more general examination of the the identification problem, which we will explore in the remaining sections.

Let's start with the problem of identifying a canonical underactuated mechanical system, like an Acrobot, Cart-Pole or Quadrotor, where we know the structure of the equations, but just need to fit the parameters. We will further assume that we have the ability to directly observe all of the state variables, albeit with noisy measurements (e.g. from joint sensors and/or inertial measurement units). The stronger [state estimation algorithms](#) that we will discuss soon assume a model, so we typically do not use them directly here.

Consider taking a minute to review the [example of deriving the manipulator equations for the double pendulum](#) before we continue.

## 18.2.1 Kinematic parameters and calibration

We can separate the parameters in the multibody equations again into kinematic parameters and dynamic parameters. The kinematic parameters, like link lengths, describe the coordinate transformation from one joint to another joint in the kinematic tree. It is certainly possible to write an optimization procedure to calibrate these parameters; you can find a fairly thorough discussion in e.g. Chapter 11 of [6]. But I guess I'm generally of the opinion that if you don't have accurate estimates of your link lengths, then you should probably invest in a tape measure before you invest in nonlinear optimization.

One notable exception to this is calibration with respect to joint offsets. This one can be a real nuisance in practice. Joint sensors can slip, and some robots even use relative rotary encoders, and rely on driving the joint to some known hard joint limit each time the robot is powered on in order to obtain the offset. I've worked on one advanced robot that had a quite elaborate and painful kinematic calibration procedure which involve fitting additional hardware over the joints to ensure they were in a known location and then running a script. Having a an expensive and/or unreliable calibration procedure can put a damper on any robotics project.

### Example 18.1 ([Acrobot balancing with calibration error](#))

Small kinematic calibration errors can lead to large steady-state errors when attempting to stabilize a system like the Acrobot. I've put together a simple notebook to show the effect here:

Our tools from robust / stochastic control are well-suited to identifying (and bounding / minimizing) these sensitivities, at least for the linearized model we use in LQR.

The general approach to estimating joint offsets from data is to write the equations with the joint offset as a parameter, e.g. for the [double pendulum](#) we would write the forward kinematics as:

$$\mathbf{p}_1 = l_1 \begin{bmatrix} \sin(\theta_1 + \bar{\theta}_1) \\ -\cos(\theta_1 + \bar{\theta}_1) \end{bmatrix}, \quad \mathbf{p}_2 = \mathbf{p}_1 + l_2 \begin{bmatrix} \sin(\theta_1 + \bar{\theta}_1 + \theta_2 + \bar{\theta}_2) \\ -\cos(\theta_1 + \bar{\theta}_1 + \theta_2 + \bar{\theta}_2) \end{bmatrix}.$$

We try to obtain independent measurements of the end-effector position (e.g. from motion capture, from perhaps some robot-mounted cameras, or from some mechanical calibration rig) with their corresponding joint measurements, to obtain data points of the form  $\langle \mathbf{p}_2, \theta_1, \theta_2 \rangle$ . Then we can solve a small nonlinear optimization problem to estimate the joint offsets to minimize a least-squares residual.

If independent measurements of the kinematics are not available, it is possible to estimate the offsets along with the dynamic parameters, using the trigonometric identities, e.g.  $s_{\theta+\bar{\theta}} = s_{\theta}c_{\bar{\theta}} + c_{\theta}s_{\bar{\theta}}$ , and then including the  $s_{\bar{\theta}}, c_{\bar{\theta}}$  terms (separately) in the "lumped parameters" we discuss below.

## 18.2.2 Estimating inertial parameters (and friction)

Now let's think about estimating the inertial parameters of a multibody system. We've been writing the manipulation equations in the form:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u} + \text{friction, etc.} \quad (2)$$

Each of the terms in this equation can depend on the parameters  $\alpha$  that we're trying to estimate. But the parameters enter the multibody equations in a particular structured way: *the inverse dynamics equations are affine in the inertial parameters* [7]. Notably, the forward dynamics (solving for  $\ddot{\mathbf{q}}$ ) involves a  $\mathbf{M}^{-1}(\mathbf{q})$ ; these dynamics are *not* affine in the parameters. Generic system identification approaches which treat the multibody dynamics as  $\mathbf{x}[n+1] = f(\mathbf{x}[n], \mathbf{u}[n])$  will not be leveraging this convexity directly.

To proceed with the inverse dynamics formulation, let us first choose a parameterization for the inertias. [8] shows that the natural parameterization of the inertial terms for a particular rigid body,  $B$ , are in terms of the *density-weighted moment matrices*:

$$\begin{aligned} m_B &= \iiint_{V_B} \rho(\mathbf{x}) dV \in \mathbb{R} \\ \mathbf{h}_B &= \iiint_{V_B} \mathbf{x} \rho(\mathbf{x}) dV \in \mathbb{R}^3 \\ \boldsymbol{\Sigma}_B &= \iiint_{V_B} \mathbf{x} \mathbf{x}^T \rho(\mathbf{x}) dV \in S_3, \end{aligned}$$

where  $V_B$  is the volume of the geometry in body  $B$ , the integral notation is for the 3D volume integral, and  $S_3$  is the group of 3x3 (real) symmetric matrices.  $m_B$  is the total mass of the body,  $\mathbf{h}_B$  is simply the mass times the position of the center of mass, and  $\boldsymbol{\Sigma}_B$  is closely related to the more standard form of the inertia matrix. With the integral using  $\mathbf{x}$  represented in the body frame,  $B$ , we can extract  ${}^B\mathbf{p}^{B_{com}} = \frac{1}{m} \mathbf{h}_B$  and  $\mathbf{I}^{B/Bo} = \text{tr}(\boldsymbol{\Sigma}_B) \mathbb{I}_{3 \times 3} - \boldsymbol{\Sigma}_B$ . Since  $\boldsymbol{\Sigma}_B$  is symmetric, we can summarize all of the inertial parameters for body  $B$  in 10 real numbers:

$$\alpha_B = [m_B, \mathbf{h}_B^T, \text{vec}_{\text{sym}}(\boldsymbol{\Sigma}_B)] \in \mathbb{R}^{10},$$

where  $\text{vec}_{\text{sym}}$  takes the 6 upper triangular elements of the 3x3 symmetric matrices into a vector. Naturally, if we aim to estimate the parameters of many bodies simultaneously, we can compose these into a single parameter vector  $\alpha$ .

Remarkably, the inverse dynamics are affine in these parameters. This means that the manipulator equations above can be refactored into the form

$$\mathbf{W}(\mathbf{q}_n, \dot{\mathbf{q}}_n, \ddot{\mathbf{q}}_n, \mathbf{u}_n) \alpha_B + \mathbf{w}_0(\mathbf{q}_n, \dot{\mathbf{q}}_n, \ddot{\mathbf{q}}_n, \mathbf{u}_n) = 0. \quad (3)$$

We sometimes refer to the  $\mathbf{W}$  which is assembled by vertically stacking these equations evaluated at all time samples,  $n$ , as the "data matrix". Given data trajectories of positions,  $\mathbf{q}_n$ , and torque measurements,  $\mathbf{u}_n$ , we can estimate  $\dot{\mathbf{q}}_n$  and  $\ddot{\mathbf{q}}_n$  with (acausal) filtering. Then we can estimate the parameters using least-squares estimation. As such, we can leverage all of the strong results and variants from linear estimation. For instance, we can add terms to regularize the estimate (e.g. to stay close to an initial guess), and we can write efficient recursive estimators for optimal online estimation of the parameters using recursive least-squares. My favorite recursive least-squares algorithm uses incremental QR factorization[9].

Not all values of  $\alpha_B$  describe physically-valid inertias. For instance, we know that  $m_B$  has to be positive. There are constraints on the inertia matrix itself. [8] showed that these constraints are exactly transcribed as positive-definiteness of the "pseudo-inertia":

$$\mathbf{J}(\alpha_B) := \begin{bmatrix} \Sigma_B & \mathbf{h}_B \\ \mathbf{h}_B^T & m_B \end{bmatrix} \succ 0.$$

If we know the geometry of the body,  $B$ , that we are estimating, then we can optionally write additional convex constraints on these parameters. For instance, we know that the center of mass must lie inside the convex hull of the geometry:

$${}^B\mathbf{p}^{B_{com}} \in \text{conv}(V_B) \Rightarrow \mathbf{h}_B \in m_B \text{conv}(V_B).$$

This non-negative scaling of a convex set is still a convex set (known as the "perspective" of the set). [8] also gives a convex constraint on the second-moment matrix based on e.g. the minimum-volume Loewner-John ellipsoid containing the known geometry.

Do these additional constraints matter? In the limit of infinite data, then one might hope that the least-squares estimator would obtain the correct values without the constraints. But there are two foils to this ideal. First, we often wish to operate in a relatively low-data regime. Second, it is important to understand that *not all of the inertial parameters will be identifiable*. For instance, if you imagine a robotic arm bolted to a table where the first link is a rotation about the  $z$  axis. Then no amount of data obtained from this robot will given any information about the moments of inertia of the first link about the  $x$  and  $y$  axes. Naively including these parameters in the estimation problem could not only lead to poor estimates of the unidentifiable parameters, but can even render the physical-validity constraints on the inertial parameters vacuous.

There are two general approaches to dealing with the unidentifiable parameters. First, we can exclude them from the optimization. There are a number of algorithms for symbolically extracting the identifiable parameters, but it is more common (and likely just as effective) to simply evaluate the null-space of a sufficiently rich data matrix,  $W$ , to find the unidentifiable parameters by using, e.g.,  $\mathbf{R}_1\alpha_l$  from the QR factorization:

$$\mathbf{W} = [\mathbf{Q}_1 \quad \mathbf{Q}_2] \begin{bmatrix} \mathbf{R}_1 \\ 0 \end{bmatrix}, \Rightarrow \mathbf{W}\alpha_B = \mathbf{Q}_1(\mathbf{R}_1\alpha_B) = \mathbf{Q}_1\bar{\alpha}_B,$$

where  $\bar{\alpha}_B$  are the identifiable parameters. To implement the physical-validity constraints, we would need to use estimates of the unidentifiable parameters. Alternatively, one can formulate the optimization directly on the full parameters and add a regularization term which (ideally) operates in the nullspace of the primary objective to drive the unidentifiable parameters towards a prior estimate. [10] gives a thorough treatment of this approach and provides a convex formulation of a geometrically consistent regularizer, which they call the *entropic divergence*.

The least-squares inverse-dynamics formulation also allows one to simultaneously fit a number of other relevant dynamic parameters which enter linearly into the equations. These include reflected inertial (as a diagonal matrix times  $\ddot{\mathbf{q}}$ ), joint friction (as a constant torque multiplied by the sign of  $\dot{\mathbf{q}}$ ), and damping (as a linear multiple of  $\dot{\mathbf{q}}$ ).

Allow me to mention one subtle point about numerical scaling. All of the equations in the least-squares inverse-dynamics formulation (3) have units of torque. But the torques used on the base link of a robotic arm might be numerically much larger than the torques corresponding to the more distal joints. [10] addresses this by solving the unconstrained least-squares problem once, then taking the error covariance of this solution into the objective function for the constrained least-squares optimization. This serves to make the least-squares objective coordinate independent.

### Example 18.2 (Inertial identification for a Kuka iiwa)

Solving the least-squares optimizations described here subject to convex (semidefinite) constraints requires solving small semidefinite programs. There has

been some nice work on parameterizations which lead to unconstrained, smooth, but nonconvex, optimization problems which can be solved with e.g. gradient descent [11]. In some cases, one can still guarantee that all local minima are global minima. But my impression from talking to the authors is that numerics of the semidefinite programming formulation can be notable better, and that it should be preferred if SDP solvers are available.

### 18.2.3 Simultaneous kinematic and inertial identification via lumped parameters.

Although it is highly recommended to separate kinematic and inertial estimation, some amount of joint estimation is possible when needed. We know that the inverse dynamics are affine in the inertial parameters. But it turns out they can also be written as affine in the "lumped parameters": nonlinear combinations of inertial and kinematic which do not depend on the data. I feel this is best explained with a few simple examples.

#### Example 18.3 (Lumped parameters for the simple pendulum)

The now familiar equations of motion for the simple pendulum are

$$ml^2\ddot{\theta} + b\dot{\theta} + mgl \sin \theta = \tau.$$

For parameter estimation, we will factor this into

$$\begin{bmatrix} \ddot{\theta} & \dot{\theta} & \sin \theta \end{bmatrix} \begin{bmatrix} ml^2 \\ b \\ mgl \end{bmatrix} - \tau = 0.$$

The terms  $ml^2$ ,  $b$ , and  $mgl$  together form the "lumped parameters".

It is worth taking a moment to reflect on this factorization. First of all, it does represent a somewhat special statement about the multibody equations: the nonlinearities enter only in a particular way. For instance, if I had terms in the equations of the form,  $\sin(m\theta)$ , then I would *not* be able to produce an affine decomposition separating  $m$  from  $\theta$ . Fortunately, that doesn't happen in our mechanical systems [6]. Again, this structure is particular to the *inverse dynamics*, as we have written here. If you were to write the forward dynamics, multiplying by  $\mathbf{M}^{-1}$  in order to solve for  $\ddot{\mathbf{q}}$ , then once again you would destroy this affine structure.

#### Example 18.4 (Multibody parameters in DRAKE)

Very few robotics simulators have any way for you to access the parameters of the dynamics. In Drake, we explicitly declare all of the parameters of a multibody system in a separate data structure to make them available, and we can leverage Drake's symbolic engine to extract and manipulate the equations with respect to those variables.

As a simple example, I've loaded the cart-pole system model from URDF, created a symbolic version of the `MultibodyPlant`, and populated the `Context` with symbolic variables for the quantities of interest. Then I can evaluate the (inverse) dynamics in order to obtain my equations.

The output looks like:

Symbolic dynamics:



```
(0.10000000000000001 * v(0) - u(0) + (pow(v(1), 2) * mp * l * sin(q(1))) + (vd(0) * mc) + (0.10000000000000001 * v(1) - (vd(0) * mp * l * cos(q(1))) + (vd(1) * mp * pow(l, 2)) + 9.8
```

Go ahead and compare these with the [cart-pole equations](#) that we derived by hand.

Drake offers a method [DecomposeLumpedParameters](#) that will take this expression and factor it into the affine expression above. For this cart-pole example, it extracts the lumped parameters  $[m_c + m_p, m_p l, m_p l^2]$ .

The existence of the lumped-parameter decomposition reveals that the [equation error](#) for lumped-parameter estimation, with the error taken in the torque coordinates, can again be solved using least squares. Importantly, because we have reduced this to a least-squares problem, we can also understand when it will *not* work. Once again, not all of the lumped-parameters are identifiable, so we will typically need a second step to operate on the identifiable lumped parameters or to add a regularization term.

### Example 18.5 (Parameter estimation for the Cart-Pole)

Having extracted the lumped parameters from the URDF file above, we can now take this to fruition. I've kept the example simple: I've simulated the cart-pole for a few seconds running just simple sine wave trajectories at the input, then constructed the data matrix and performed the least squares fit.

The output looks like this:

```
Estimated Lumped Parameters:
(mc + mp). URDF: 11.0, Estimated: 10.905425349337081
(mp * l). URDF: 0.5, Estimated: 0.5945797067753813
(mp * pow(l, 2)). URDF: 0.25, Estimated: 0.302915745122919
```

Note that we could have easily made the fit more accurate with more data (or more carefully selected data).

Should we be happy with only estimating the (identifiable) lumped parameters? Isn't it the true original parameters that we are after? The linear algebra decomposition of the data matrix (assuming we apply it to a sufficiently rich set of data), is actually revealing something fundamental for us about our system dynamics. Rather than feel disappointed that we cannot accurately estimate some of the parameters, we should embrace that *we don't need to estimate those parameters* for any of the dynamic reasoning we will do about our equations (simulation, verification, control design, etc). The identifiable lumped parameters are precisely the subset of the lumped parameters that we need.

## 18.2.4 Identification using energy instead of inverse dynamics.

In addition to leveraging tools from linear algebra, there are a number of other refinements to the basic recipe that leverage our understanding of mechanics. One important example is the "energy formulations" of parameter estimation [12].

We have already observed that evaluating the equation error in torque space (inverse dynamics) is likely better than evaluating it in state space space (forward dynamics), because we can factorized the inverse dynamics. But this property is not exclusive to inverse dynamics. The total energy of the system (kinetic + potential) is also affine in the lumped parameters. We can use the relation



$$\dot{E}(\mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}}) = \dot{\mathbf{q}}^T (\mathbf{B}\mathbf{u} + \text{friction}, \dots).$$

Why might we prefer to work in energy coordinates rather than torque? The differences are apparent in the detailed numerics. In the torque formulation, we find ourselves using  $\ddot{\mathbf{q}}$  directly. Conventional wisdom holds that joint sensors can reliably report joint positions and velocities, but that joint accelerations, often obtained by numerically differentiating *twice*, tend to be much more noisy. In some cases, it might be numerically better to apply finite differences to the total energy of the system rather than to the individual joints<sup>†</sup>. [13] provides a study of various filtering formulations which leads to a recommendation in [12] that the energy formulation tends to be numerically superior. This distinction is probably more important in online parameter estimation, where the filters must be causal, as opposed to offline parameter estimation where acausal filters can be used.

<sup>†</sup> Sometimes these methods are written as the numerical integration of the power input, rather than the differentiation of the total energy, but the numerics should be different.

### 18.2.5 Residual physics models with linear function approximators

The term "residual physics" has become quite popular recently (e.g. [14]) as people are looking for ways to combine the modeling power of deep neural networks with our best tools from mechanics. But actually this idea is quite old, and there is a natural class of residual models that fit very nicely into our least-squares lumped-parameter estimation. Specifically, we can consider models of the form:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \tau_g(\mathbf{q}) + \alpha_r \Phi(\mathbf{q}, \dot{\mathbf{q}}) + \mathbf{B}\mathbf{u} + \text{friction}, \quad \text{etc.}, \quad (4)$$

with  $\alpha_r$  the additional parameters of the residual and  $\Phi$  a set of (fixed) nonlinear basis functions. The hope is that these residual models can capture any "slop terms" in the dynamics that are predictable, but which we did not include in our original parametric model. Nonlinear friction and aerodynamic drag are commonly cited examples.

Common choices for these basis functions,  $\Phi(\mathbf{q}, \dot{\mathbf{q}})$ , for use with the manipulator equations include radial basis functions[15] or wavelets [16]. Although allowing the basis functions to depend on  $\ddot{\mathbf{q}}$  or  $\mathbf{u}$  would be fine for the parameter estimation, we tend to restrict them to  $\mathbf{q}$  and  $\dot{\mathbf{q}}$  to maintain some of the other nice properties of the manipulator equations (e.g. control-affine).

Due to the maturity of least-squares estimation, it is also possible to use least-squares to effectively determine a subset of basis functions that effectively describe the dynamics of your system. For example, in [17], we applied least-squares to a wide range of physically-inspired basis functions in order to make a better ODE approximation of the post-stall aerodynamics during perching, and ultimately discarded all but the small number of basis functions that best described the data. Nowadays, we could apply algorithms like **LASSO** for least-squares regression with an  $\ell_1$ -regularization, or [18] uses an alternative based on sequential thresholded least-squares.

### 18.2.6 Experiment design as a trajectory optimization

One key assumption for any claims about our parameter estimation algorithms recovering the true identifiable lumped parameters is that the data set was sufficiently rich; that the trajectories were "parametrically exciting". Basically we need to assume that the trajectories produced motion so that the data matrix,  $\mathbf{W}$  contains information about all of the identifiable lumped parameters. Thanks to our linear parameterization, we can evaluate this via numerical linear algebra on  $\mathbf{W}$ .

Moreover, if we have an opportunity to change the trajectories that the robot executes when collecting the data for parameter estimation, then we can design trajectories which try to maximally excite the parameters, and produce a numerically-well conditioned least squares problem. One natural choice is to minimize the **condition number** of the data matrix,  $\mathbf{W}$ . The condition number of a

matrix is the ratio of the largest to smallest singular values  $\frac{\sigma_{\max}(\mathbf{W})}{\sigma_{\min}(\mathbf{W})}$ . The condition number is always greater than one, by definition, and the lower the value the better the condition. The condition number of the data matrix is a nonlinear function of the data taken over the entire trajectory, but it can still be optimized in a nonlinear trajectory optimization ([6], § 12.3.4).

### 18.2.7 Online estimation and adaptive control

The field of adaptive control is a huge and rich literature; many books have been written on the topic (e.g. [19]). Allow me to make a few quick references to that literature here.

I have mostly discussed parameter estimation so far as an offline procedure, but one important approach to adaptive control is to perform online estimation of the parameters as the controller executes. Since our estimation objective can be linear in the (lumped) parameters, this often amounts to the recursive least-squares estimation. To properly analyze a system like this, we can think of the system as having an augmented state space,  $\bar{\mathbf{x}} = \begin{bmatrix} \mathbf{x} \\ \alpha \end{bmatrix}$ , and study the closed-loop dynamics of

the state and parameter evolution jointly. Some of the strongest results in adaptive control for robotic manipulators are confined to fully-actuated manipulators, but for instance [20] gives a nice example of analyzing an adaptive controller for underactuated systems using many of the tools that we have been developing in these notes.

As I said, adaptive control is a rich subject. One of the biggest lessons from that field, however, is that one may not need to achieve convergence to the true (lumped) parameters in order to achieve a task. Many of the classic results in adaptive control make guarantees about task execution but explicitly do *not* require/guarantee convergence in the parameters.

### 18.2.8 Identification with contact

Can we apply these same techniques to e.g. walking robots that are making and breaking contact with the environment?

There is certainly a version of this problem that works immediately: if we know the contact Jacobians and have measurements of the contact forces, then we can add these terms directly into the manipulator equations and continued with the least-squares estimation of the lumped parameters, even including frictional parameters.

One can also study cases where the contact forces are not measured directly. For instance, [21] studies the extreme case of identifiability of the inertial parameters of a passive object with and without explicit contact force measurements.

## 18.3 IDENTIFYING (TIME-DOMAIN) LINEAR DYNAMICAL SYSTEMS

If multibody parameter estimation forms the first relevant pillar of established work in system identification, then identification of linear systems forms the second. Linear models have been a primary focus of system identification research since the field was established, and have witnessed a resurgence in popularity during just the last few years as new results from machine learning have contributed new bounds and convergence results, especially in the finite-sample regime (e.g. [22, 23, 24, 25]).

A significant portion of the linear system identification literature (e.g. [26]) is focused on identifying linear models in the frequency domain. Indeed, transfer-

function realizations provide important insights and avoid some of the foibles that we'll need to address with state-space realizations. However, I will focus my attention in this chapter on time-domain descriptions of linear dynamical systems; some of the lessons here are easier to generalize to nonlinear dynamics (plus we unfortunately haven't built the foundations for the frequency domain techniques yet in these notes).

### 18.3.1 From state observations

Let's start our treatment with the easy case: fitting a linear model from direct (potentially noisy) measurements of the state. Fitting a discrete-time model,  $\mathbf{x}[n+1] = \mathbf{A}\mathbf{x}[n] + \mathbf{B}\mathbf{u}[n] + \mathbf{w}[n]$ , to sampled data  $(\mathbf{u}_n, \mathbf{x}_n = \mathbf{x}[n] + \mathbf{v}[n])$  using the *equation-error* objective is just another *linear* least-squares problem. Typically, we form some data matrices and write our least-squares problem as:

$$\mathbf{X}' \approx [\mathbf{A} \quad \mathbf{B}] \begin{bmatrix} \mathbf{X} \\ \mathbf{U} \end{bmatrix}, \quad \text{where}$$

$$\mathbf{X} = \begin{bmatrix} | & | & & | \\ \mathbf{x}_0 & \mathbf{x}_1 & \cdots & \mathbf{x}_{N-2} \\ | & | & & | \end{bmatrix}, \quad \mathbf{X}' = \begin{bmatrix} | & | & & | \\ \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_{N-1} \\ | & | & & | \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} | & | & & | \\ \mathbf{u}_0 & \mathbf{u}_1 & \cdots & \mathbf{u}_{N-2} \\ | & | & & | \end{bmatrix}.$$

By the virtues of linear least squares, this estimator is unbiased with respect to the uncorrelated process and/or measurement noise,  $\mathbf{w}[n]$  and  $\mathbf{v}[n]$ .

#### Example 18.6 (Cart-pole balancing with an identified model)

I've provide a notebook demonstrating what a practical application of linear identification might look like for the cart-pole system, in a series of steps. First, I've designed an LQR balancing controller, but using the *wrong* parameters for the cart-pole (I changed the masses, etc). This LQR controller is enough to keep the cart-pole from falling down, but it doesn't converge nicely to the upright. I wanted to ask the question, can I use data generated from this experiment to identify a better linear model around the fixed-point, and improve my LQR controller?

Interestingly, the simple answer is "no". If you only collect the input and state data from running this controller, you will see that the least-squares problem that we formulate above is rank-deficient. The estimated  $\mathbf{A}$  and  $\mathbf{B}$  matrices, denoted  $\hat{\mathbf{A}}$  and  $\hat{\mathbf{B}}$  describe the data, but do not reveal the true linearization. And if you design a controller based on these estimates, you will be disappointed!

Fortunately, we could have seen this problem by checking the rank of the least-squares solution. Generating more examples won't fix this problem. Instead, to generate a richer dataset, I've added a small additional signal to the input:  $u(t) = \pi_{lqr}(\mathbf{x}(t)) + 0.1 \sin(t)$ . That makes all of the difference.

I hope you try the code. The basic algorithm for estimation is disarmingly simple, but there are a lot of details to get right to actually make it work.

### Model-based Iterative Learning Control (ILC)

#### Example 18.7 (The Hydrodynamic Cart-Pole)

One of my favorite examples of model-based ILC was a series of experiments in which we explored the dynamics of a "hydrodynamic cart-pole" system. Think of it as a cross between the classic cart-pole system and a fighter jet (perhaps a little closer to the cartpole)!

Here we've replaced the pole with an airfoil (hydrofoil), turned the entire system on its side, and dropped it into a water tunnel. Rather than swing-up and balance the pole against gravity, the task is to balance the foil in its unstable configuration against the hydrodynamic forces. These forces are the result of unsteady fluid-body interactions; unlike the classic cart-pole system, this time we do not have an tractable parameterized ODE model for the system. It's a perfect problem for system identification and ILC.

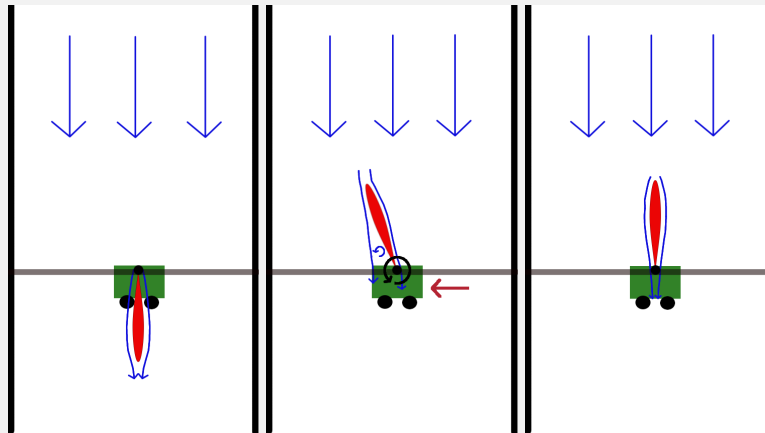


Figure 18.2 - A cartoon of the hydrodynamic cart-pole system. The cart is actuated horizontally, the foil pivots around a passive joint, and the fluid is flowing in the direction of the arrows. (The entire system is horizontal, so there is no effect from gravity.) The aerodynamic center of the airfoil is somewhere in the middle of the foil; because the pin joint is at the tail, the passive system will "weather vane" to the stable "downward" equilibrium (left). Balancing corresponds to stabilizing the unstable "upward" equilibrium (right). The fluid-body dynamics during the transition (center) are unsteady and very nonlinear.

In a series of experiments, first we attempted to stabilize the system using an LQR controller derived with an approximate model (using [flat-plate theory](#)). This controller didn't perform well, but was just good enough to collect relevant data in the vicinity of the unstable fixed point. Then we fit a linear model, recomputed the LQR controller using the model, and got notably better performance.

To investigate more aggressive maneuvers we considered making a rapid step change in the desired position of the cart (like a fighter jet rapidly changing altitude). Using only the time-invariant LQR balancing controller with a shifted set point, we naturally observed a very slow step-response. Using trajectory optimization on the time-invariant linear model used in balancing, we could do much better. But we achieved considerably better performance by iteratively fitting a time-varying linear model in the neighborhood of this trajectory and performing model-based ILC.

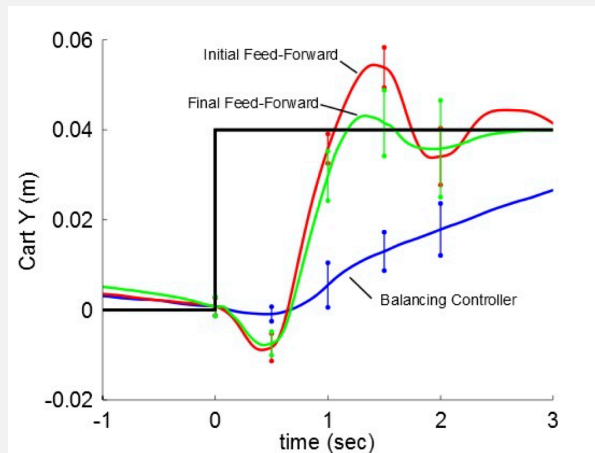


Figure 18.3 - Comparison of the step response using three different controllers: the balancing LQR controller (blue), LQR with a feed-forward term obtained from trajectory optimization with the LTI model (red), and a controller obtained via ILC with a time-varying linear model and iLQR (green).

These experiments were quite beautiful and very visual. They went on from here to consider the effect of stabilizing against incoming vortex disturbances using real-time perception of the oncoming fluid. If you're at all interested, I would encourage you to check out John Robert's thesis[27] and/or even the [video of his thesis defense](#).

## Compression using the dominant eigenmodes

For high-dimensional state or input vectors, one can use singular value decomposition (SVD) to solve this least-squares problem using only the dominant eigenvalues (and corresponding eigenvectors) of  $\mathbf{A}$  [28, Section 7.2] using the so-called "Dynamic Mode Decomposition" (DMD). There have been many empirical success stories of using a small number of dominant modes to produce an excellent fit to the data (this is especially relevant in problems like fluid dynamics where the state vector  $\mathbf{x}$  might be an entire image corresponding to a fluid flow). In DMD, we would write the linear dynamics in the coordinates of these eigenmodes (which can always be projected back to the full coordinates).

## Linear dynamics in a nonlinear basis

A potentially useful generalization that we can consider here is identifying dynamics that evolve linearly in a coordinate system defined by some (potentially high-dimensional) basis vectors  $\phi(\mathbf{x})$ . In particular, we might consider dynamics of the form  $\dot{\phi} = \frac{\partial \phi}{\partial \mathbf{x}} \dot{\mathbf{x}} = \mathbf{A}\phi(x)$ . Much ado has been made of this particular form, due to the connection to Koopman operator theory that we will discuss briefly below. For our purposes, we also need a control input, so might consider a form like  $\dot{\phi} = \mathbf{A}\phi(x) + \mathbf{B}u$ .

Once again, thanks to the maturity of least-squares, with this approach it is possible to include list many possible basis functions, then use sparse least-squares and/or the modal decomposition to effectively find the important subset.

Note that multibody parameter estimation described above is not this, although it is closely related. The least-squares lumped-parameter estimation for the manipulator equations uncovered dynamics that were still *nonlinear* in the state variables.

## 18.3.2 From input-output data (the state-realization problem)

In the more general form, we would like to estimate a model of the form

$$\begin{aligned}\mathbf{x}[n+1] &= \mathbf{A}\mathbf{x}[n] + \mathbf{B}\mathbf{u}[n] + \mathbf{w}[n] \\ \mathbf{y}[n] &= \mathbf{C}\mathbf{x}[n] + \mathbf{D}\mathbf{u}[n] + \mathbf{v}[n].\end{aligned}$$

Once again, we will apply least-squares estimation, but combine this with the famous "Ho-Kalman" algorithm (also known as the "Eigen System Realization" (ERA) algorithm) [29]. My favorite presentation of this algorithm is [24].

First, observe that

$$\begin{aligned}\mathbf{y}[0] &= \mathbf{C}\mathbf{x}[0] + \mathbf{D}\mathbf{u}[0] + \mathbf{v}[0], \\ \mathbf{y}[1] &= \mathbf{C}(\mathbf{A}\mathbf{x}[0] + \mathbf{B}\mathbf{u}[0] + \mathbf{w}[0]) + \mathbf{D}\mathbf{u}[1] + \mathbf{v}[1], \\ \mathbf{y}[n] &= \mathbf{C}\mathbf{A}^n\mathbf{x}[0] + \mathbf{D}\mathbf{u}[n] + \mathbf{v}[n] + \sum_{k=0}^{n-1} \mathbf{C}\mathbf{A}^{n-k-1}(\mathbf{B}\mathbf{u}[k] + \mathbf{w}[k]).\end{aligned}$$

For the purposes of identification, let's write  $\mathbf{y}[n]$  as a function of the most recent  $N+1$  inputs (for  $k \geq N$ ):

$$\begin{aligned}\mathbf{y}[n] &= [\mathbf{C}\mathbf{A}^{N-1}\mathbf{B} \quad \mathbf{C}\mathbf{A}^{N-2}\mathbf{B} \quad \dots \quad \mathbf{C}\mathbf{B} \quad \mathbf{D}] \begin{bmatrix} \mathbf{u}[n-N] \\ \mathbf{u}[n-N+1] \\ \vdots \\ \mathbf{u}[n] \end{bmatrix} + \delta[n] \\ &= \mathbf{G}[n]\bar{\mathbf{u}}[n] + \delta[n]\end{aligned}$$

where  $\delta[n]$  captures the remaining terms from initial conditions, noise and control inputs before the (sliding) window.  $\mathbf{G}[n] = [\mathbf{C}\mathbf{A}^{N-1}\mathbf{B} \quad \mathbf{C}\mathbf{A}^{N-2}\mathbf{B} \quad \dots \quad \mathbf{C}\mathbf{B} \quad \mathbf{D}]$ , and  $\bar{\mathbf{u}}[n]$  represents the concatenated  $\mathbf{u}[n]$ 's from time  $n-N$  up to  $n$ . Importantly we have that  $\delta[n]$  is uncorrelated with  $\bar{\mathbf{u}}[n]$ :  $\forall k > n$  we have  $E[\mathbf{u}_k\delta_n] = 0$ . This is sometimes known as Neyman orthogonality, and it implies that we can estimate  $\mathbf{G}$  using simple least-squares  $\hat{\mathbf{G}} = \text{argmin}_{\mathbf{G}} \sum_{n \geq N} \|\mathbf{y}_n - \mathbf{G}\bar{\mathbf{u}}_n\|^2$ . [24] gives bounds on the norm of the estimation error as a function of the number of samples and the variance of the noise.

How should we pick the window size  $N$ ? By Neyman orthogonality, we know that our estimates will be unbiased for any choice of  $N \geq 0$ . But if we choose  $N$  too small, then the term  $\delta[n]$  will be large, leading to a potentially large variance in our estimate. For stable systems, the  $\delta$  terms will get smaller as we increase  $N$ . In practice, we choose  $N$  based on the characteristic settling time in the data (roughly until the impulse response becomes sufficiently small).

If you've studied linear systems,  $\mathbf{G}$  will look familiar; it is precisely this (multi-input, multi-output) matrix impulse response, also known as the "Markov parameters". In fact, estimating  $\hat{\mathbf{G}}$  may even be sufficient for control design, as it is closely-related to the parameterization used in disturbance-based feedback for partially-observable systems [30, 31]. But the Ho-Kalman algorithm can be used to extract good estimates  $\hat{\mathbf{A}}, \hat{\mathbf{B}}, \hat{\mathbf{C}}, \hat{\mathbf{D}}$  with state-dimension  $\dim(\mathbf{x}) = n_x$  from  $\hat{\mathbf{G}}$ , assuming that the true system is observable and controllable with order at least  $n_x$  and the data matrices we form below are sufficiently rich [24].

It is important to realize that many system matrices,  $\mathbf{A}, \mathbf{B}, \mathbf{C}$ , can describe the same input-output data. In particular, for any invertible matrix (aka [similarity transform](#)),  $\mathbf{T}$ , the system matrices  $\mathbf{A}, \mathbf{B}, \mathbf{C}$  and  $\mathbf{A}', \mathbf{B}', \mathbf{C}'$  with,

$$\mathbf{A}' = \mathbf{T}^{-1}\mathbf{A}\mathbf{T}, \quad \mathbf{B}' = \mathbf{T}^{-1}\mathbf{B}, \quad \mathbf{C}' = \mathbf{C}\mathbf{T},$$

describe the same input-output behavior. The Ho-Kalman algorithm returns a

*balanced realization*. A balanced realization is one in which we have effectively applied a similarity transform,  $\mathbf{T}$ , which makes the controllability and observability Grammians equal and diagonal [28, Ch. 9] and which orders the states in terms of diminishing effect on the input/output behavior. This ordering is relevant for determining the system order and for model reduction.

Note that  $\hat{\mathbf{D}}$  is the last block in  $\hat{\mathbf{G}}$  so is extracted trivially. The Ho-Kalman algorithm tells us how to extract  $\hat{\mathbf{A}}, \hat{\mathbf{B}}, \hat{\mathbf{C}}$ , with another application of the SVD on suitably constructed data matrices (see e.g. [24, §5.1], [32, §10.5], or [28, §9.3]).

### Example 18.8 (Ho-Kalman identification of the cart-pole from keypoints)

Let's repeat the cart-pole example. But this time, instead of getting observations of the joint position and velocities directly, we will consider the problem of identifying the dynamics from a camera rendering the scene, but let us proceed thoughtfully.

The output of a standard camera is an RGB image, consisting of 3 x width x height real values. We could certainly columnate these into a vector and use this as the outputs/observations,  $y(t)$ . But I don't recommend it. This "pixel-space" is not a nice space. For example, you could easily find a pixel that has the color of the cart in one frame, but after an incremental change in the position of the cart, now (discontinuously) takes on the color of the background. Deep learning has given us fantastic new tools for transforming raw RGB images into better "feature spaces", that will be robust enough to deploy on real systems but can make our modeling efforts much more tractable. My group has made heavy use of "keypoint networks" [33] and self-supervised "dense correspondences" [34, 35, 36] to convert RGB outputs into a more consumable form.

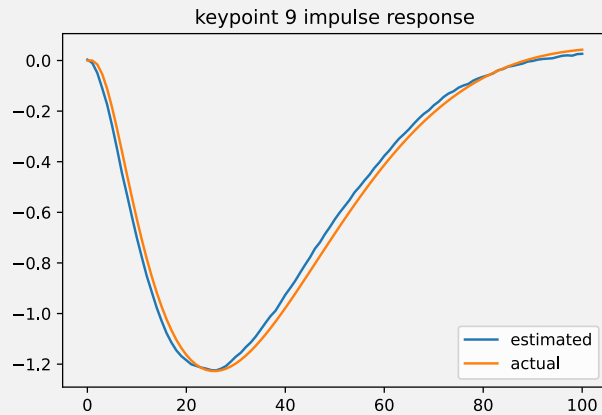
The existence of these tools makes it reasonable to assume that we have observations representing the 2D positions of some number of "key points" that are rigidly fixed to the cart and to the pole. The location of any keypoint  $K$  on a pole, for instance, is given by the [cart-pole kinematics](#):

$${}^w\mathbf{p}^K = \begin{bmatrix} x \\ 0 \end{bmatrix} + \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix} {}^P\mathbf{p}^K,$$

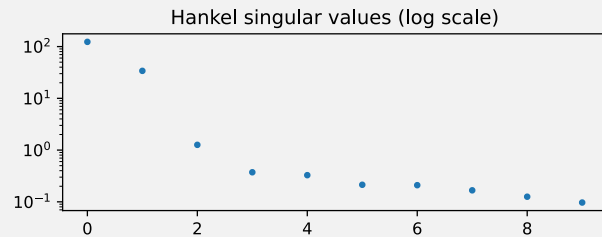
where  $x$  is the position of the cart, and  $\theta$  is the angle of the pole. I've used

Monogram notation to denote that  ${}^P\mathbf{p}^K$  is the Cartesian position of a point  $A$  in the pole's frame,  $P$ , and  ${}^W\mathbf{p}^K$  is that same position in the camera/world frame,  $W$ . While it is probably unreasonable to linearize the RGB outputs as a function of the cart-pole positions,  $x$  and  $\theta$ , it *is* reasonable to approximate the keypoint positions by linearizing this equation for small angles.

You'll see in the example that we recover the impulse response quite nicely.



The big question is: does Ho-Kalman discover the 4-dimensional state vector that we know and like for the cart-pole? Does it find something different? We would typically choose the order by looking at the magnitude of the singular values of the Hankel data matrix. In this example, in the case of no noise, you see clearly that 2 states describe most of the behavior, and we have diminishing returns after 4 or 5:



As an exercise, try adding process and/or measurement noise to the system that generated the data, and see how they effect this result.

### 18.3.3 Adding stability constraints

Details coming soon. See, for instance [37].

### 18.3.4 Autoregressive models

Another important class of linear models predict the output directly from a history of recent inputs and outputs:

$$\mathbf{y}[n+1] = \mathbf{A}_0\mathbf{y}[n] + \mathbf{A}_1\mathbf{y}[n-1] + \dots + \mathbf{A}_k\mathbf{y}[n-k] + \mathbf{B}_0\mathbf{u}[n] + \mathbf{B}_1\mathbf{u}[n-1] + \dots + \mathbf{B}_k\mathbf{u}[n-k]$$

These are the so-called "AutoRegressive models with eXogenous input (ARX)" models. The coefficients of ARX models can be fit directly from input-output data



using linear least squares.

While it is certainly possible to think of these models as state-space models, where we collect the finite history  $\mathbf{y}$  and  $\mathbf{u}$  (except not  $\mathbf{u}[n]$ ) into our state vector. But this is not necessarily a very efficient representation; the Ho-Kalman algorithm might be able to find a much smaller state representation. This is particularly important for partially-observable systems that might require a very long history,  $k$ .

### Example 18.9 (An ARX model of cart-pole keypoint dynamics)

## 18.3.5 Statistical analysis of learning linear models

## 18.4 IDENTIFICATION OF FINITE (PO)MDPs

There is one more case that we can understand well: when states, actions, and observations (and time) are all discrete. Recall that in the very beginnings of our discussion about optimal control and [dynamic programming](#), we used graph search and discretization of the state space to give a particularly nice version of the value iteration algorithm. In general, we can write the *stochastic* dynamics using conditional probabilities:

$$\Pr(s[n+1]|s[n], a[n]), \quad \Pr(o[n]|s[n], a[n]),$$

where I've (again) used  $s$  for discrete states,  $a$  for discrete actions, and  $o$  for discrete observations. With everything discrete, these conditional probabilities are represented naturally with matrices/tensors. We'll use the tensors  $T_{ijk} \equiv \Pr(s[n+1] = s_i | s[n] = s_j, a[n] = a_k)$  to denote the transition matrix, and  $O_{ijk} \equiv \Pr(o[n] = o_i | s[n] = s_j, a[n] = a_k)$  for the observation matrix.

### 18.4.1 From state observations

Following analogously to our discussion on linear dynamical systems, the first case to understand is when we have direct access to state and action trajectories:  $s[\cdot], a[\cdot]$ . This corresponds to identifying a Markov chain or Markov decision process (MDP). The transition matrices can be extracted directly from the statistics of the transitions:

$$T_{ijk} = \frac{\text{number of transitions to } s_i \text{ from } s_j, a_k}{\text{total number of transitions from } s_j, a_k}.$$

Not surprisingly, for this estimate to converge to the true transition probabilities asymptotically, we need the identification (exploration) dynamics to be *ergodic* (for every state/action pair to be visited infinitely often).

For large MDPs, analogous to the modal decomposition that we described for linear dynamical systems, we can also consider factored representations of the transition matrix. [Coming soon...]

### 18.4.2 Identifying Hidden Markov Models (HMMs)

## 18.5 NEURAL NETWORK MODELS

If multibody parameter estimation, linear system identification, and finite state systems are the first pillars of system identification, then (at least these days) deep learning is perhaps the last major pillar. It's no coincidence that I've put this section

just after the section on linear dynamical systems. Neural networks are powerful tools, but they can be a bit harder to think about carefully. Thankfully, we'll see that many of the basic lessons and insights from linear dynamical systems still apply here. In particular, we can follow the same progression: learning the dynamics function directly from state observations, learning state-space models from input-output data (with recurrent networks), and learning input-output (autoregressive) models with feedforward networks using a history of outputs.

Deep learning tools allow us to quite reliably tune the neural networks to fit our training data. The major challenges are with respect to data efficiency/generalization, and with respect to actually designing planners / controllers based on these models that have such complex descriptions.

### 18.5.1 Generating training data

One extremely interesting question that arises when fitting rich nonlinear models like neural networks is the question of how to generate the training data. You might have the general sense that we would like data that provides some amount of "coverage" of the state space; this is particularly challenging for underactuated systems since we have dynamic constraints restricting our trajectories in state space.

For multibody parameter estimation [we discussed](#) using the condition number of the data matrix as an explicit objective for experiment design. This was a luxury of using *linear* least-squares optimization for the regression and it takes advantage of the specialized knowledge we have from the manipulator equations about the model structure. Unfortunately, we don't have an analogous concept in the more general case of nonlinear least squares with general function approximators. This approach also works for identifying linear dynamical systems. The challenge is here perhaps considered less severe since for linear models in the noise-free case even data generated from the impulse response is sufficient for identification.

This topic has received considerable attention lately in the context of *model-based reinforcement learning* (e.g. [38]). Broadly speaking, in the ideal case we would like the training data we use for system identification to match the distribution of data that we will encounter when executing the optimal policy. But one cannot know this distribution until we've designed our controller, which (in our current discussion) requires us to have a model. It is a classic "chicken and the egg" problem. In most cases, it speaks to the importance to interleaving system identification and control design instead of the simpler notion of performing identification once and then using the model forevermore.

### 18.5.2 From state observations

### 18.5.3 State-space models from input-output data (recurrent networks)

### 18.5.4 Input-output (autoregressive) models

You might have heard about a neural network model called the Transformer. There are a massive number of online resources where you can [learn about Transformers](#). These started in the natural language processing (NLP) community, where they rapidly replaced recurrent neural network models which were, at the time, struggling to handle very long sentences (trajectories). Transformers get around this problem by switching back to an autoregressive-style model, but they compress the very long history of previous observations into a compact representation using attention mechanisms. This is what now enables large language models, like chat-GPT, to e.g. write an entire essay, "remembering" names/concepts that it used early in the essay correctly very late in the essay.

### 18.5.5 Particle-based models

### 18.5.6 Object-centric models

### 18.5.7 Modeling stochasticity

### 18.5.8 Control design for neural network models

## 18.6 ALTERNATIVES FOR NONLINEAR SYSTEM IDENTIFICATION

## 18.7 IDENTIFICATION OF HYBRID SYSTEMS

## 18.8 TASK-RELEVANT MODELS

As discussed briefly in the introduction, the standard objective for system identification is to match the input-output behavior of a dynamical system. Certainly if we can find a model that predicts all outputs perfectly, then this model should be sufficient for making optimal decisions. But when we cannot make perfect predictions, due to limited data and/or limited model representation power, then the relative cost placed on predicting different outputs becomes important. This feels particularly important when the observations come from a camera -- in most cases predicting every pixel in the scene accurately is not the right objective for a robot trying to use the camera to accomplish a particular task.

Fortunately, the problem of learning task-relevant models or "learning state representations" has become a darling of the machine learning community. There are a lot of smart people thinking about the problem and the field has made considerable progress in the last years. In the immense volume of papers, I think there are a few really core ideas.

What makes a good state representation for control? Two defining properties are: 1) it should be observable, and 2) it should be sufficient for making optimal decisions. In other words, if  $\mathbf{z}[n]$  is the state of the dynamic controller (which we can think of as an observer of the learned state representation), then we would like that the optimal policy given this state to take the same action as an optimal policy which uses the entire available history:

$$\pi^*(\mathbf{z}[n]) \approx \pi^*(\mathbf{y}[n], \mathbf{y}[n-1], \mathbf{y}[n-2], \dots, \mathbf{u}[n-1], \mathbf{u}[n-2], \dots).$$

Certainly the *belief state*, which is by definition a sufficient statistic for predicting future observations, has this property. But much more minimal representations may also be sufficient. So a third, slightly more tenuous property of a good state representation is that we would like it to be tractable for learning, representation, and decision making.

Being sufficient for making optimal control decisions is a difficult objective because it couples the problem of representation learning / system identification with the problem of control design. How can we know what the optimal action is without solving the optimal control problem? This is one reason that imitation learning [35, 39] is a potentially valuable undertaking -- by providing demonstrations of an "optimal" policy, it reduces representation learning back to classical system identification and allows us to focus on the question of state representation without simultaneously solving control design. Other works like [40] attempt use a planner on the learned representation even during learning, and take a loss function requesting that the actions taken from a policy using the current state matches the actions taken from a planner using the representation to optimize multiple steps into

the future.

A closely related idea would ask for the representation to be sufficient to predict the optimal value function (rather than the optimal action). Certainly if we have a model and a value function that is consistent with that model, then this would be sufficient for making optimal decisions. But it should be noted that there are cases where predicting the optimal cost-to-go is actually harder than predicting the optimal action. Imagine an agent in a grid world that can traverse one of two corridors and has learned that the left corridor always has more reward than the right. Predicting the precise amount of reward that will be obtained when moving down the left corridor is not needed to choose the optimal action. Predicting optimal values also does not break the "chicken-and-the-egg" problem (how can you know the optimal values until you have an optimal policy?), but may have more subtle merits in terms of representation or learning efficiency.

One proposal which *does* fundamentally break the chicken-and-the-egg problem is given by [41] and called "approximate information states" (AIS). The key idea in this work is that if a state representation,  $\mathbf{z}[n]$  is 1) sufficient to predict the one-step reward, and 2) sufficient to predict  $\mathbf{z}[n+1]$ , then it is sufficient for making optimal decisions. This motivates their notion of an *approximate* information state, as a representation that approximately accomplishes these two goals. They provide one particular bound for the suboptimality of the resulting policy, based on worst case errors in the one-step reward and next state predictions. In my group, we've been studying this approach, trying to understand the implications for the two cases we can deeply understand: finite (tabular) Markov decision processes [42] and linear-Gaussian systems [43]. In particular, [42] gives an algorithm for solving the AIS conditions to optimality for a class of POMDPs; it provides evidence that representations which are optimal by the AIS metric are not necessarily optimal for reproducing the optimal actions, and that the AIS bounds on performance can be very loose.

Approximate information states are themselves closely related to prior work on "state aggregation" in MDPs via the so-called "bisimulation metrics". The basic idea is that if the future outcomes from two states are indistinguishable, then these states can be aggregated into a single state, thus reducing the state space. See [44] for the original work on exact bisimulation, [45] for approximate bisimulation, [46] for an extension to POMDPs, and [4] for a more modern version using neural networks.

Interestingly, the famous work by DeepMind on MuZero [47] learns a task-relevant state representation using a combination of all of these ideas: the loss function for the state combines a loss for predicting the action, the value function, and the one-step reward. But I have seen a few projects that struggled to reproduce its success in robotics domains.

This area is moving fast; I will try to update this section as results appear, and will give some worked out examples soon.

## 18.9 EXERCISES

### Exercise 18.1 (Linear System Identification with different objective functions)

Consider a discrete-time linear system of the form

$$\mathbf{x}[n+1] = A\mathbf{x}[n] + B\mathbf{u}[n]$$

where  $\mathbf{x}[n] \in \mathbf{R}^p$  and  $\mathbf{u}[n] \in \mathbf{R}^q$  are state and control at step  $n$ . The system matrix  $A \in \mathbf{R}^{p \times p}$  and  $B \in \mathbf{R}^{p \times q}$  are unknown parameters of the model, and your task is to identify the parameters given a simulated trajectory. Using the trajectory simulation provided in , implement the solution for the following problems.

- a. Identify the model parameters by solving

$$\min_{A,B} \sum_{n=0}^{N-1} \|x[n+1] - Ax[n] - Bu[n]\|_2^2.$$

- b. Identify the model parameters by solving

$$\min_{A,B} \sum_{n=0}^{N-1} \|x[n+1] - Ax[n] - Bu[n]\|_\infty.$$

- c. Identify the model parameters by solving

$$\min_{A,B} \sum_{n=0}^{N-1} \|x[n+1] - Ax[n] - Bu[n]\|_1.$$

## Exercise 18.2 (System Identification for the Perching Glider)

In this exercise we will use physically-inspired basis functions to fit the nonlinear dynamics of a perching glider. , you will need to implement least-squares fitting and find the best set of basis functions that describe the dynamics of the glider. Take the time to go through the notebook and understand the code in it, and then answer the following questions. The written question will also be listed in the notebook for your convenience.

- Work through the coding sections in the notebook.
- All of the basis configurations we tested used at most 3 basis functions to compute a single acceleration. If we increase the number of basis functions used to compute a single acceleration to 4, the least-squares residual goes down. Why would we limit ourselves to 3 basis functions if by using more we can generate a better fit?

## REFERENCES

- Alexander Rakhlin and Karthik Sridharan, "Statistical Learning and Sequential Prediction", Online Draft , September, 2014.
- Elad Hazan and others, "Introduction to online convex optimization", *Foundations and Trends in Optimization*, vol. 2, no. 3-4, pp. 157--325, 2016.
- A. Rakhlin, "IDS.160 - Mathematical Statistics: A Non-Asymptotic Approach", Online lecture notes , Spring, 2022.
- Amy Zhang and Rowan McAllister and Roberto Calandra and Yarin Gal and Sergey Levine, "Learning invariant representations for reinforcement learning without reconstruction", *arXiv preprint arXiv:2006.10742*, 2020.
- Manuel Watter and Jost Springenberg and Joschka Boedecker and Martin Riedmiller, "Embed to Control: A Locally Linear Latent Dynamics Model for Control from Raw Images", *Advances in Neural Information Processing Systems* , vol. 28, 2015.
- W Khalil and E Dombre, "Modeling, Identification and Control of Robots", Elsevier , 2004.
- Christopher G Atkeson and Chae H An and John M Hollerbach, "Estimation

of inertial parameters of manipulator loads and links", *The International Journal of Robotics Research*, vol. 5, no. 3, pp. 101--119, 1986.

8. Patrick M Wensing and Sangbae Kim and Jean-Jacques E Slotine, "Linear matrix inequalities for physically consistent inertial parameter identification: A statistical perspective on the mass distribution", *IEEE Robotics and Automation Letters*, vol. 3, no. 1, pp. 60--67, 2017.
9. M. Kaess and A. Ranganathan and F. Dellaert, "iSAM: Incremental Smoothing and Mapping", *IEEE Transactions on Robotics*, vol. 24, no. 6, pp. 1365-1378, 2008.
10. Taeyoon Lee and Patrick M Wensing and Frank C Park, "Geometric robot dynamic identification: A convex programming approach", *IEEE Transactions on Robotics*, vol. 36, no. 2, pp. 348--365, 2019.
11. Caleb Rucker and Patrick M Wensing, "Smooth parameterization of rigid-body inertia", *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 2771--2778, 2022.
12. Maxime Gautier, "Dynamic identification of robots with power model", *Proceedings of the {IEEE} International Conference on Robotics and Automation*, vol. 3, pp. 1922--1927 vol.3, 1997.
13. Maxime Gautier, "A comparison of filtered models for dynamic identification of robots", *Proceedings of the 35th {IEEE} Conference on Decision and Control*, vol. 1, pp. 875--880, 1996.
14. Andy Zeng and Shuran Song and Johnny Lee and Alberto Rodriguez and Thomas Funkhouser, "Tossingbot: Learning to throw arbitrary objects with residual physics", *IEEE Transactions on Robotics*, vol. 36, no. 4, pp. 1307--1319, 2020.
15. R. M. Sanner and J. E. Slotine, "Gaussian Networks for Direct Adaptive Control", *1991 American Control Conference*, pp. 2153-2159, 1991.
16. Robert M Sanner and Jean-Jacques E Slotine, "Structurally dynamic wavelet networks for adaptive control of robotic systems", *International Journal of Control*, vol. 70, no. 3, pp. 405--421, 1998.
17. Warren Hoburg and Russ Tedrake, "System Identification of Post Stall Aerodynamics for {UAV} Perching", *Proceedings of the AIAA Infotech@Aerospace Conference*, April, 2009. [ [link](#) ]
18. Steven L Brunton and Joshua L Proctor and J Nathan Kutz, "Discovering governing equations from data by sparse identification of nonlinear dynamical systems", *Proceedings of the national academy of sciences*, vol. 113, no. 15, pp. 3932--3937, 2016.
19. Karl J. Åström and Björn Wittenmark, "Adaptive {Control}: {Second} {Edition}", Courier Corporation, apr, 2013.
20. Joseph Moore and Russ Tedrake, "Adaptive Control Design for Underactuated Systems Using Sums-of-Squares Optimization", *Proceedings of the 2014 American Control Conference (ACC)*, June, 2014. [ [link](#) ]
21. Nima Fazeli and Roman Kolbert and Russ Tedrake and Alberto Rodriguez, "Parameter and contact force estimation of planar rigid-bodies undergoing frictional contact", *International Journal of Robotics Research*, vol. 36, no. 13-14, pp. 1437-1454, 2017. [ [link](#) ]
22. Moritz Hardt and Tengyu Ma and Benjamin Recht, "Gradient descent learns linear dynamical systems", *arXiv preprint arXiv:1609.05191*, 2016.

23. Elad Hazan and Holden Lee and Karan Singh and Cyril Zhang and Yi Zhang, "Spectral filtering for general linear dynamical systems", *arXiv preprint arXiv:1802.03981*, 2018.
24. Samet Oymak and Necmiye Ozay, "Non-asymptotic identification of lti systems from a single trajectory", *2019 American control conference (ACC)* , pp. 5655--5661, 2019.
25. Max Simchowitz and Ross Boczar and Benjamin Recht, "Learning linear dynamical systems with semi-parametric least squares", *Conference on Learning Theory* , pp. 2714--2802, 2019.
26. L. Ljung, "System Identification: Theory for the User", Prentice Hall , 1999.
27. John W. Roberts, "Control of Underactuated Fluid-Body Systems with Real-Time Particle Image Velocimetry", PhD thesis, Massachusetts Institute of Technology, June, 2012. [ [link](#) ]
28. Steven L Brunton and J Nathan Kutz, "Data-driven science and engineering: Machine learning, dynamical systems, and control", Cambridge University Press , 2019.
29. BL Ho and Rudolf E K{\a}lm{\a}n, "Effective construction of linear state-variable models from input/output functions", *at-Automatisierungstechnik*, vol. 14, no. 1-12, pp. 545--548, 1966.
30. Sadra Sadraddini and Russ Tedrake, "Robust Output Feedback Control with Guaranteed Constraint Satisfaction", *In the Proceedings of 23rd ACM International Conference on Hybrid Systems: Computation and Control* , pp. 12, April, 2020. [ [link](#) ]
31. Max Simchowitz and Karan Singh and Elad Hazan, "Improper learning for non-stochastic control", *Conference on Learning Theory* , pp. 3320--3436, 2020.
32. Jer-Nan Juang and Minh Q. Phan, "Identification and {Control} of {Mechanical} {Systems}", Cambridge University Press , aug, 2001.
33. Lucas Manuelli\* and Wei Gao\* and Peter Florence and Russ Tedrake, "{kPAM: KeyPoint Affordances for Category-Level Robotic Manipulation}", *arXiv e-prints*, pp. arXiv:1903.06684, Mar, 2019. [ [link](#) ]
34. Peter R. Florence\* and Lucas Manuelli\* and Russ Tedrake, "Dense Object Nets: Learning Dense Visual Object Descriptors By and For Robotic Manipulation", *Conference on Robot Learning (CoRL)* , October, 2018. [ [link](#) ]
35. Peter Florence and Lucas Manuelli and Russ Tedrake, "Self-Supervised Correspondence in Visuomotor Policy Learning", *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 492-499, April, 2020. [ [link](#) ]
36. Lucas Manuelli and Yunzhu Li and Pete Florence and Russ Tedrake, "Keypoints into the Future: Self-Supervised Correspondence in Model-Based Reinforcement Learning", *Conference on Robot Learning (CoRL)* , 2020. [ [link](#) ]
37. Jack Umenberger and Johan W{\aa}gberg and Ian R Manchester and Thomas B Sch{\o}n, "Maximum likelihood identification of stable linear dynamical systems", *Automatica*, vol. 96, pp. 280--292, 2018.
38. Alekh Agarwal and Nan Jiang and Sham M. Kakade and Wen Sun, "Reinforcement Learning: Theory and Algorithms", Online Draft , 2020.
39. Pete Florence and Corey Lynch and Andy Zeng and Oscar A Ramirez and



Ayzaan Wahid and Laura Downs and Adrian Wong and Johnny Lee and Igor Mordatch and Jonathan Tompson, "Implicit behavioral cloning", *Conference on Robot Learning*, pp. 158--168, 2022.

40. Aravind Srinivas and Allan Jabri and Pieter Abbeel and Sergey Levine and Chelsea Finn, "Universal planning networks: Learning generalizable representations for visuomotor control", *International Conference on Machine Learning*, pp. 4732--4741, 2018.
41. Jayakumar Subramanian and Aditya Mahajan, "Approximate information state for partially observed systems", *2019 IEEE 58th Conference on Decision and Control (CDC)*, pp. 1629--1636, 2019.
42. Lujie Yang and Kaiqing Zhang and Alexandre Amice and Yunzhu Li and Russ Tedrake, "Discrete Approximate Information States in Partially Observable Environments", *Proceedings of the 2022 American Control Conference (ACC)*, 2022. [ [link](#) ]
43. Yi Tian and Kaiqing Zhang and Russ Tedrake and Suvrit Sra, "Can Direct Latent Model Learning Solve Linear Quadratic Gaussian Control?", *arXiv preprint arXiv:2212.14511*, 2022.
44. Robert Givan and Thomas Dean and Matthew Greig, "Equivalence notions and model minimization in Markov decision processes", *Artificial Intelligence*, vol. 147, no. 1-2, pp. 163--223, 2003.
45. Norm Ferns and Prakash Panangaden and Doina Precup, "Metrics for Finite Markov Decision Processes.", *UAI*, vol. 4, pp. 162--169, 2004.
46. Pablo Samuel Castro and Prakash Panangaden and Doina Precup, "Equivalence Relations in Fully and Partially Observable Markov Decision Processes.", *IJCAI*, vol. 9, pp. 1653--1658, 2009.
47. Julian Schrittwieser and Ioannis Antonoglou and Thomas Hubert and Karen Simonyan and Laurent Sifre and Simon Schmitt and Arthur Guez and Edward Lockhart and Demis Hassabis and Thore Graepel and others, "Mastering {A}tari, {G}o, chess and shogi by planning with a learned model", *Nature*, vol. 588, no. 7839, pp. 604--609, 2020.



# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

[Russ Tedrake](#)

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 19

# State Estimation

### [19.1 OBSERVERS AND THE KALMAN FILTER](#)

### [19.2 RECURSIVE BAYESIAN FILTERS](#)

### [19.3 SMOOTHING](#)

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

[Accessibility](#)

© Russ Tedrake, 2024

# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## CHAPTER 20

# Model-Free Policy Search

Reinforcement learning (RL) is a collection of algorithms for solving the same optimal control problem that we've focused on through the text, but the real gems from the RL literature are the algorithms for (almost) *black-box optimization* of stochastic optimal control problems. The idea of an algorithm that only has a "black-box" interface to the optimization problem means that it can obtain (potentially noisy) samples of the optimal cost via trial and error, but does not have access to the underlying model, and does not have direct access to complete gradient information.

This is a hard problem! In general we cannot expect RL algorithms to optimize as quickly as more structured optimization, and we can typically only guarantee convergence to a local optima at best. But the framework is extremely general, so it can be applied to problems that are inaccessible to any of the other algorithms we've examined so far. My favorite examples for reinforcement learning are control in complex fluid dynamics (e.g. [1]). These systems are often very difficult to model, or the model is so high-dimensional and complicated so that it's prohibitive for control design. In these problems, it might actually be faster to optimize via trial-and-error in physical experiments.

In this chapter we will examine one particular style of reinforcement learning that attempts to find good controllers by explicitly parameterizing a family of policies (e.g. via a parameter vector  $\alpha$ ), then searching directly for the parameters that optimize the long-term cost. For a stochastic optimal control problem with our favorite additive form of the objective, this might look like:

$$\min_{\alpha} E \left[ \sum_{n=0}^N \ell(\mathbf{x}[n], \mathbf{u}[n]) \right] \quad (1)$$

where the random variables are drawn from probability densities

$$\begin{aligned} \mathbf{x}[0] &\sim p_0(\mathbf{x}), \\ \mathbf{x}[n] &\sim p(\mathbf{x}[n]|\mathbf{x}[n-1], \mathbf{u}[n-1]), \\ \mathbf{u}[n] &\sim p_{\alpha}(\mathbf{u}[n]|\mathbf{x}[n]). \end{aligned}$$

The last equation is a probabilistic representation of the control policy -- on each time-step actions  $\mathbf{u}$  are drawn from a distribution that is conditioned on the current state,  $\mathbf{x}$ .

Of course, the controls community has investigated ideas like this, too, for instance under the umbrellas of *extremum-seeking control* and *iterative learning control*. I'll try to make connections whenever possible.

## 20.1 POLICY GRADIENT METHODS

One of the standard approaches to policy search in RL is to estimate the gradient of the expected long-term cost with respect to the policy parameters by evaluating some number of sample trajectories, then performing (stochastic) gradient descent. Many of these so-called "policy gradient" algorithms leverage a derivation called the *likelihood ratio method* that was perhaps first described in [2] then popularized in the REINFORCE algorithm [3]. It is based on what looks like a trick with logarithms to estimate the gradient; I feel like this trick is often presented with an air of mystery about it. Let's try to make sure we understand it.

### 20.1.1 The Likelihood Ratio Method (aka REINFORCE)

Let's start with a simpler optimization over a stochastic function:

$$\min_{\alpha} E[g(\mathbf{x})] \text{ with } \mathbf{x} \sim p_{\alpha}(\mathbf{x})$$

I hope the notation is clear?  $\mathbf{x}$  is a random vector, drawn from the distribution  $p_{\alpha}(\mathbf{x})$ , with the subscript indicating that the distribution depends on the parameter vector  $\alpha$ . What is the gradient of this function? The REINFORCE derivation is

$$\begin{aligned} \frac{\partial}{\partial \alpha} E[g(\mathbf{x})] &= \frac{\partial}{\partial \alpha} \int d\mathbf{x} g(\mathbf{x}) p_{\alpha}(\mathbf{x}) \\ &= \int d\mathbf{x} g(\mathbf{x}) \frac{\partial}{\partial \alpha} p_{\alpha}(\mathbf{x}) \\ &= \int d\mathbf{x} g(\mathbf{x}) p_{\alpha}(\mathbf{x}) \frac{\partial}{\partial \alpha} \log p_{\alpha}(\mathbf{x}) \\ &= E \left[ g(\mathbf{x}) \frac{\partial}{\partial \alpha} \log p_{\alpha}(\mathbf{x}) \right]. \end{aligned}$$

To achieve this, we used the derivative of the logarithm:

$$\begin{aligned} y &= \log u \\ \frac{\partial y}{\partial u} &= \frac{1}{u} \frac{\partial u}{\partial x} \end{aligned}$$

to write

$$\frac{\partial}{\partial \alpha} p_{\alpha}(\mathbf{x}) = p_{\alpha}(\mathbf{x}) \frac{\partial}{\partial \alpha} \log p_{\alpha}(\mathbf{x}).$$

This suggests a simple Monte Carlo algorithm for estimating the policy gradient: draw  $N$  random samples  $\mathbf{x}_i$  then estimate the gradient as

$$\frac{\partial}{\partial \alpha} E[g(\mathbf{x})] \approx \frac{1}{N} \sum_i g(\mathbf{x}_i) \frac{\partial}{\partial \alpha} \log p_{\alpha}(\mathbf{x}_i).$$

This trick is even more potent in the optimal control case. For a finite-horizon problem we have

$$\begin{aligned} \frac{\partial}{\partial \alpha} E \left[ \sum_{n=0}^N \ell(\mathbf{x}[n], \mathbf{u}[n]) \right] &= \int d\mathbf{x}[\cdot] d\mathbf{u}[\cdot] \left[ \sum_{n=0}^N \ell(\mathbf{x}[n], \mathbf{u}[n]) \right] \frac{\partial}{\partial \alpha} p_{\alpha}(\mathbf{x}[\cdot], \mathbf{u}[\cdot]) \\ &= E \left[ \left( \sum_{n=0}^N \ell(\mathbf{x}[n], \mathbf{u}[n]) \right) \frac{\partial}{\partial \alpha} \log p_{\alpha}(\mathbf{x}[\cdot], \mathbf{u}[\cdot]) \right], \end{aligned}$$

where  $\mathbf{x}[\cdot]$  is shorthand for the entire trajectory  $\mathbf{x}[0], \dots, \mathbf{x}[N]$ , and

$$p_{\alpha}(\mathbf{x}[\cdot], \mathbf{u}[\cdot]) = p_0(\mathbf{x}[0]) \left( \prod_{k=1}^N p(\mathbf{x}[k] | \mathbf{x}[k-1], \mathbf{u}[k-1]) \right) \left( \prod_{k=0}^N p_{\alpha}(\mathbf{u}[k] | \mathbf{x}[k]) \right).$$

Taking the log we have

$$\log p_{\alpha}(\mathbf{x}[\cdot], \mathbf{u}[\cdot]) = \log p_0(\mathbf{x}[0]) + \sum_{k=1}^N \log p(\mathbf{x}[k] | \mathbf{x}[k-1], \mathbf{u}[k-1]) + \sum_{k=0}^N \log p_{\alpha}(\mathbf{u}[k] | \mathbf{x}[k]).$$

Only the last terms depend on  $\alpha$ , which yields

$$\begin{aligned} \frac{\partial}{\partial \alpha} E \left[ \sum_{n=0}^N \ell(\mathbf{x}[n], \mathbf{u}[n]) \right] &= E \left[ \left( \sum_{n=0}^N \ell(\mathbf{x}[n], \mathbf{u}[n]) \right) \left( \sum_{k=0}^N \frac{\partial}{\partial \alpha} \log p_{\alpha}(\mathbf{u}[k] | \mathbf{x}[k]) \right) \right] \\ &= E \left[ \sum_{n=0}^N \left( \ell(\mathbf{x}[n], \mathbf{u}[n]) \sum_{k=0}^n \frac{\partial}{\partial \alpha} \log p_{\alpha}(\mathbf{u}[k] | \mathbf{x}[k]) \right) \right], \end{aligned}$$

where the last step uses the fact that  $E \left[ \ell(\mathbf{x}[n], \mathbf{u}[n]) \frac{\partial}{\partial \alpha} \log p_{\alpha}(\mathbf{u}[k] | \mathbf{x}[k]) \right] = 0$  for all  $k > n$ .

This update should surprise you. It says that I can find the gradient of the long-term cost by taking only the gradient of the policy... but not the gradient of the plant, nor the cost! The intuition is that one can obtain the gradient by evaluating the policy along a number of (random) trajectory roll-outs of the closed-loop system, evaluating the cost on each, then increasing the probability in the policy of taking the actions correlated with lower long-term costs.

This derivation is often presented as **the** policy gradient derivation. The identity is certainly correct, but I'd prefer that you think of this as just one way to obtain the policy gradient. It's a particularly clever one in that it uses exactly the information that we have available in reinforcement learning -- we have access to the instantaneous costs,  $\ell(\mathbf{x}[n], \mathbf{u}[n])$ , and to the policy -- so are allowed to take gradients of the policy -- but does not require any understanding whatsoever of the plant model. Although it's clever, it is not particularly efficient -- the Monte Carlo approximations of the expected value have a high variance, so many samples are required for an accurate estimate. Other derivations are possible, some even simpler, and others that **do** make use of the plant gradients if you have access to them, and these will perform differently in the finite-sample approximations.

For the remainder of this section, I'd like to dig in and try to understand the nature of the stochastic update a little better, to help you understand what I mean.

## 20.1.2 Sample efficiency

Let's take a step back and think more generally about how one can use gradient descent in black-box (unconstrained) optimization. Imagine you have a simple optimization problem:

$$\min_{\alpha} g(\alpha),$$

and you can directly evaluate  $g()$ , but not  $\frac{\partial g}{\partial \alpha}$ . How can you perform gradient descent?

A standard technique for estimating the gradients, in lieu of analytical gradient information, is the method of finite differences[4]. The finite differences approach to estimating the gradient involves making the same small perturbation,  $\epsilon$  to the input parameters in every dimension independently, and using:

$$\frac{\partial g}{\partial \alpha_i} \approx \frac{g(\alpha + \epsilon_i) - g(\alpha)}{\epsilon},$$

where  $\epsilon_i$  is the column vector with  $\epsilon$  in the  $i$ th row, and zeros everywhere else. Finite difference methods can be computationally very expensive, requiring  $n + 1$  evaluations of the function for every gradient step, where  $n$  is the length of the input vector.

What if each function evaluation is expensive? Perhaps it means picking up a physical robot and running it for 10 seconds for each evaluation of  $g()$ . Suddenly there is a premium on trying to optimize the cost function with the fewest number of evaluations on  $g()$ . This is the name of the game in reinforcement learning -- it's often called RL's *sample complexity*. Can we perform gradient descent using less evaluations?

### 20.1.3 Stochastic Gradient Descent

This leads us to the question of doing approximate gradient descent, or "stochastic" gradient descent. Thinking of the cost landscape as a Lyapunov function<sup>†</sup>, then any update that moves downhill on each step will eventually reach the optimal. More generally, any update that moves downhill on average will eventually reach a minimum... and sometimes "stochastic" gradient descent updates that occasionally go uphill but go downhill on average can even have desirable properties like bouncing out of small local minima. The figure below gives some graphical intuition; for a formal treatment of stochastic gradient methods, see e.g. [5].

<sup>†</sup> the Lyapunov function  $V(\alpha) = g(\alpha) - g(\alpha^*)$  is commonly used in convergence/stability analysis of optimization algorithms.

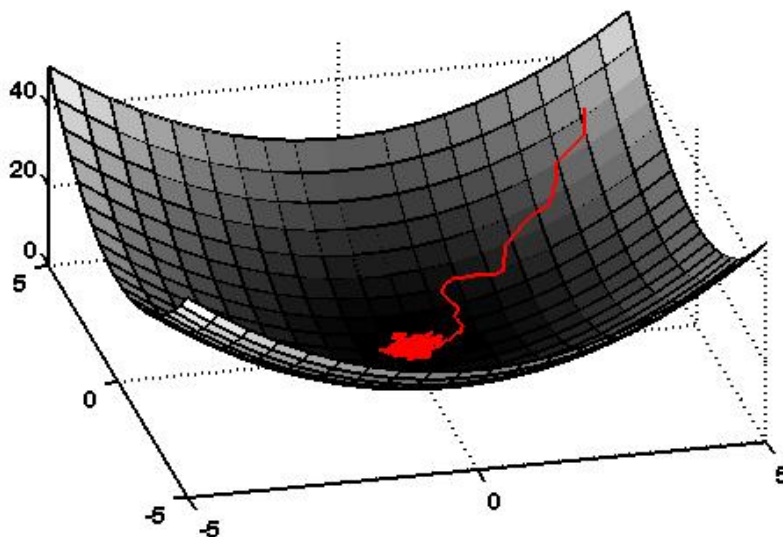


Figure 20.1 - Stochastic gradient descent of a quadratic cost function .

### 20.1.4 The Weight Perturbation Algorithm

Instead of sampling each dimension independently, consider making a single small random change,  $\beta$ , to the parameter vector,  $\alpha$ . Now consider the update of the form:

$$\Delta \alpha = -\eta[g(\alpha + \beta) - g(\alpha)]\beta.$$

The intuition here is that if  $g(\alpha + \beta) < g(\alpha)$ , then  $\beta$  was a good change and we'll move in the direction of  $\beta$ . If the cost increased, then we'll move in the opposite direction. Assuming the function is smooth and  $\beta$  is small, then we will always move downhill on the Lyapunov function.

Even more interesting, on average, the update is actually in the direction of the true gradient. To see this, again we can assume the function is locally smooth and  $\beta$  is small to write the Taylor expansion:

$$g(\alpha + \beta) \approx g(\alpha) + \frac{\partial g}{\partial \alpha} \beta.$$

Now we have

$$\begin{aligned} \Delta \alpha &\approx -\eta \left[ \frac{\partial g}{\partial \alpha} \beta \right] \beta = -\eta \beta \beta^T \frac{\partial g}{\partial \alpha}^T \\ E[\Delta \alpha] &\approx -\eta E[\beta \beta^T] \frac{\partial g}{\partial \alpha}^T \end{aligned}$$

If we select each element of  $\beta$  independently from a distribution with zero mean and variance  $\sigma_\beta^2$ , or  $E[\beta_i] = 0, E[\beta_i \beta_j] = \sigma_\beta^2 \delta_{ij}$ , then we have

$$E[\Delta \alpha] \approx -\eta \sigma_\beta^2 \frac{\partial g}{\partial \alpha}^T.$$

Note that the distribution  $p_\alpha(\mathbf{x})$  need not be Gaussian, but it is the variance of the distribution which determines the scaling on the gradient.

## 20.1.5 Weight Perturbation with an Estimated Baseline

The weight perturbation update above requires us to evaluate the function  $g$  twice for every update of the parameters. This is low compared to the method of finite differences, but let us see if we can do better. What if, instead of evaluating the function twice per update  $[g(\alpha + \beta)$  and  $g(\alpha)]$ , we replace the evaluation of  $g(\alpha)$  with an estimator,  $b = \hat{g}(\alpha)$ , obtained from previous trials? In other words, consider an update of the form:

$$\Delta \alpha = -\frac{\eta}{\sigma_\beta^2} [g(\alpha + \beta) - b] \beta \quad (2)$$

The estimator can take any of a number of forms, but perhaps the simplest is based on the update (after the  $n$ th trial):

$$b[n+1] = \gamma g[n] + (1 - \gamma) b[n], \quad b[0] = 0, 0 \leq \gamma \leq 1,$$

where  $\gamma$  parameterizes the moving average. Let's compute the expected value of the update, using the same Taylor expansion that we used above:

$$\begin{aligned} E[\Delta \alpha] &= -\frac{\eta}{\sigma_\beta^2} E \left[ [g(\alpha) + \frac{\partial g}{\partial \alpha} \beta - b] \beta \right] \\ &= -\frac{\eta}{\sigma_\beta^2} E [[g(\alpha) - b] \beta] - \frac{\eta}{\sigma_\beta^2} E [\beta \beta^T] \frac{\partial g}{\partial \alpha}^T \\ &= -\eta \frac{\partial g}{\partial \alpha}^T \end{aligned}$$

In other words, the baseline does not effect our basic result - that the expected update is in the direction of the gradient. Note that this computation works for any baseline estimator which is uncorrelated with  $\beta$ , as it should be if the estimator is a function of the performance in previous trials.

Although using an estimated baseline doesn't effect the average update, it can have a dramatic effect on the performance of the algorithm. As we will see in a later section, if the evaluation of  $g$  is stochastic, then the update with a baseline estimator can actually outperform an update using straight function evaluations.

Let's take the extreme case of  $b = 0$ . This seems like a very bad idea... on each step

we will move in the direction of every random perturbation, but we will move more or less in that direction based on the evaluated cost. In average, we will still move in the direction of the true gradient, but only because we will eventually move more downhill than we move uphill. It feels very naive.

### 20.1.6 REINFORCE w/ additive Gaussian noise

Now let's consider the simple form of the REINFORCE update:

$$\frac{\partial}{\partial \alpha} E[g(\mathbf{x})] = E \left[ g(\mathbf{x}) \frac{\partial}{\partial \alpha} \log p_{\alpha}(\mathbf{x}) \right].$$

It turns out that weight perturbation is a REINFORCE algorithm. To see this, take  $\mathbf{x} = \alpha + \beta$ ,  $\beta \in N(0, \sigma^2)$ , to have

$$p_{\alpha}(\mathbf{x}) = \frac{1}{(2\pi\sigma^2)^N} e^{-\frac{(\mathbf{x}-\alpha)^T(\mathbf{x}-\alpha)}{2\sigma^2}}$$

$$\log p_{\alpha}(\mathbf{x}) = \frac{-(\mathbf{x}-\alpha)^T(\mathbf{x}-\alpha)}{2\sigma^2} + \text{terms that don't depend on } \alpha$$

$$\frac{\partial}{\partial \alpha} \log p_{\alpha}(\mathbf{x}) = \frac{1}{\sigma^2} (\alpha - \mathbf{x})^T = \frac{1}{\sigma^2} \beta^T.$$

If we use only a single trial for each Monte-Carlo evaluation, then the REINFORCE update is

$$\Delta \alpha = -\frac{\eta}{\sigma^2} g(\alpha + \beta) \beta,$$

which is precisely the weight perturbation update (the crazy version with  $b = 0$  discussed above). Although it will move in the direction of the gradient on average, it can be highly inefficient. In practice, people use many more than one sample to estimate the policy gradient.

### 20.1.7 Summary

The policy gradient "trick" from REINFORCE using log probabilities provides one means of estimating the true policy gradient. It is not the only way to obtain the policy gradient... in fact the trivial weight-perturbation update obtains the same gradient for the case of a policy with a mean linear in  $\alpha$  and a fixed diagonal covariance. Its cleverness lies in the fact that it makes use of the information we do have (instantaneous cost values and the gradients of the policy), and that it provides an unbiased estimate of the gradient (note that taking the gradients of a model that is only slightly wrong might not have this virtue). But its inefficiency comes from having a potentially very high variance. Variance reduction for policy gradient, often through baseline estimation, continues to be an active area of research.

## 20.2 SAMPLE PERFORMANCE VIA THE SIGNAL-TO-NOISE RATIO.

### 20.2.1 Performance of Weight Perturbation

The simplicity of the REINFORCE / weight perturbation updates makes it tempting to apply them to problems of arbitrary complexity. But a major concern for the algorithm is its performance - although we have shown that the update is in the direction of the true gradient *on average*, it may still require a prohibitive number of computations to obtain a local minima.

In this section, we will investigate the performance of the weight perturbation algorithm by investigating its *signal-to-noise* ratio (SNR). This idea was explored in

[1] -- my goal is just to give you a taste here. The SNR is the ratio of the power in the signal (here desired update in the direction of the true gradient) and the power in the noise (the remaining component of the update, so that  $\Delta\alpha = -\eta \frac{\partial g}{\partial \alpha}^T + \text{noise}$ )<sup>†</sup>

$$\text{SNR} = \frac{-\eta \frac{\partial g}{\partial \alpha}^T{}^2}{E \left[ \Delta\alpha + \eta \frac{\partial g}{\partial \alpha}^T{}^2 \right]}.$$

In the special case of the unbiased update, the equation reduces to:

$$\text{SNR} = \frac{E[\Delta\alpha]^T E[\Delta\alpha]}{E[(\Delta\alpha)^T(\Delta\alpha)] - E[\Delta\alpha]^T E[\Delta\alpha]}.$$

For the weight perturbation update we have:

$$\begin{aligned} E[\Delta\alpha]^T E[\Delta\alpha] &= \eta^2 \frac{\partial g}{\partial \alpha} \frac{\partial g}{\partial \alpha}^T = \eta^2 \sum_{i=1}^N \left( \frac{\partial g}{\partial \alpha_i} \right)^2, \\ E[(\Delta\alpha)^T(\Delta\alpha)] &= \frac{\eta^2}{\sigma_\beta^4} E \left[ [g(\alpha + \beta) - g(\alpha)]^2 \beta^T \beta \right] \\ &\approx \frac{\eta^2}{\sigma_\beta^4} E \left[ \left[ \frac{\partial g}{\partial \alpha} \beta \right]^2 \beta^T \beta \right] \\ &= \frac{\eta^2}{\sigma_\beta^4} E \left[ \left( \sum_i \frac{\partial g}{\partial \alpha_i} \beta_i \right) \left( \sum_j \frac{\partial g}{\partial \alpha_j} \beta_j \right) \left( \sum_k \beta_k^2 \right) \right] \\ &= \frac{\eta^2}{\sigma_\beta^4} E \left[ \sum_i \frac{\partial g}{\partial \alpha_i} \beta_i \sum_j \frac{\partial g}{\partial \alpha_j} \beta_j \sum_k \beta_k^2 \right] \\ &= \frac{\eta^2}{\sigma_\beta^4} \sum_{i,j,k} \frac{\partial g}{\partial \alpha_i} \frac{\partial g}{\partial \alpha_j} E[\beta_i \beta_j \beta_k^2] \\ E[\beta_i \beta_j \beta_k^2] &= \begin{cases} 0 & i \neq j \\ \sigma_\beta^4 & i = j \neq k \\ \mu_4(\beta) & i = j = k \end{cases} \\ &= \eta^2 (N-1) \sum_i \left( \frac{\partial g}{\partial \alpha_i} \right)^2 + \frac{\eta^2 \mu_4(z)}{\sigma_\beta^4} \sum_i \left( \frac{\partial g}{\partial \alpha_i} \right)^2 \end{aligned}$$

where  $\mu_n(z)$  is the  $n$ th central moment of  $z$ :

$$\mu_n(z) = E[(z - E[z])^n].$$

Putting it all together, we have:

$$\text{SNR} = \frac{1}{N - 2 + \frac{\mu_4(\beta_i)}{\sigma_\beta^4}}.$$

### Example 20.1 (Signal-to-noise ratio for additive Gaussian noise)

For  $\beta_i$  drawn from a Gaussian distribution, we have  $\mu_1 = 0, \mu_2 = \sigma_\beta^2, \mu_3 = 0, \mu_4 = 3\sigma_\beta^4$ , simplifying the above expression to:

$$\text{SNR} = \frac{1}{N + 1}.$$

<sup>†</sup> SNR could alternatively be defined as the ratio of the power of the component of the update in the direction of the signal and the component orthogonal to the signal (does not penalize the magnitudes of the updates):

$$\frac{E[a^2]}{E[|\Delta\alpha - a\mathbf{v}|^2]},$$

where  $\mathbf{v} = \frac{\frac{\partial g}{\partial \alpha}^T}{\left\| \frac{\partial g}{\partial \alpha} \right\|}$ ,  $a = \Delta\alpha \cdot \mathbf{v}$ . This form is probably a better definition, but is more difficult to work with.



### Example 20.2 (Signal-to-noise ratio for additive uniform noise)

For  $\beta_i$  drawn from a uniform distribution over  $[-a, a]$ , we have  $\mu_1 = 0, \mu_2 = \frac{a^2}{3} = \sigma_\beta^2, \mu_3 = 0, \mu_4 = \frac{a^4}{5} = \frac{9}{5}\sigma_\beta^4$ , simplifying the above to:

$$\text{SNR} = \frac{1}{N - \frac{1}{5}}.$$

Performance calculations using the SNR can be used to design the parameters of the algorithm in practice. For example, based on these results it is apparent that noise added through a uniform distribution yields better gradient estimates than the Gaussian noise case for very small  $N$ , but that these differences are negligible for large  $N$ .

Similar calculations can yield insight into picking the size of the additive noise (scaled by  $\sigma_\beta$ ). The calculations in this section seem to imply that larger  $\sigma_\beta$  can only reduce the variance, overcoming errors or noise in the baseline estimator,  $\tilde{b}$ ; this is a short-coming of our first-order Taylor expansion. If the cost function is not linear in the parameters, then an examination of higher-order terms reveals that large  $\sigma_\beta$  can increase the SNR. The derivation with a second-order Taylor expansion is left for an exercise.

## REFERENCES

1. John W. Roberts, "Motor Learning on a Heaving Plate via Improved-{SNR} Algorithms", , February, 2009. [ [link](#) ]
2. Peter W. Glynn, "Likelihood ratio gradient estimation for stochastic systems", *Communications of the ACM*, vol. 33, no. 10, pp. 75--84, oct, 1990.
3. R.J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning", *Machine Learning*, vol. 8, pp. 229-256, 1992.
4. William H. Press and Saul A. Teukolsky and William T. Vetterling and Brian P. Flannery, "Numerical Recipes in C: The Art of Scientific Computing", Cambridge University Press , 1992.
5. Dimitri P Bertsekas, "Nonlinear programming", Athena scientific Belmont , 1999.

# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## APPENDIX A Drake

**DRAKE** is the main software toolbox that we use for this text, which in fact originated in a large part due to the MIT underactuated course. The **DRAKE** website is the main source for information and documentation. The goal of this chapter is to provide any additional information that can help you get up and running with the examples and exercises provided in these notes.

### A.1 PYDRAKE

**DRAKE** is primarily a C++ library, with rigorous coding standards and a maturity level intended to support even professional applications in industry. In order to provide a gentler introduction, and to facilitate rapid prototyping, I've written these notes exclusively in python, using Drake's python bindings (pydrake). These bindings are less mature than the C++ backend; your feedback (and even contributions) are very welcome. It is still improving rapidly.

In particular, while the C++ API documentation is excellent, the autogenerated python docs are still a work in progress. I currently recommend using the [C++ documentation](#) to find what you need, then checking the [Python documentation](#) only if you need to understand how the class or method is spelled in pydrake.

There are also a number of [tutorials](#) in **DRAKE** that can help you get started.

### A.2 ONLINE JUPYTER NOTEBOOKS

I will provide nearly all examples and exercises in the form of a [Jupyter Notebook](#) so that we can leverage the fantastic and relatively recent availability of (free) cloud resources.

#### A.2.1 Running on Deepnote

We'll use Deepnote as the primary platform for the course. After following any of the links from the chapters, you should:

1. Log in (the free account will be sufficient for this class)
2. "Duplicate" the document. Icon is in the top right next to Login.
3. Run all of the cells (can use the "Run notebook" icon just above this cell)
4. Many of the notebooks use [MeshCat](#) for interactive visualizations. Click on the url printed just below "StartMeshcat" (often the second code cell of the notebook) to see the MeshCat window.

### A.2.2 Running on Google Colab

As of now, Drake no longer supports Google Colab, which is stuck on Ubuntu 18.04 and Python 3.7. We will try to support it again if/when they upgrade.

### A.2.3 Enabling licensed solvers

You can enable more powerful solvers for `MathematicalProgram` if you have a license (most are free for academics). Please see this [tutorial](#) for instructions.

## A.3 RUNNING ON YOUR OWN MACHINE

As you get more advanced, you will likely want to run (and extend) these examples on your own machine. On [platforms/configurations that Drake supports](#) (the latest two releases of Mac and Ubuntu using the system default python versions). The simplest installation is via `pip`; I would typically recommend using a virtual environment:

```
python3 -m venv venv
source venv/bin/activate
pip3 install underactuated --extra-index-url https://drake-packages.csail.mit.edu/whl/night
```

 Copy code

Then you can download notebooks directly from Deepnote and run them locally (or develop your own notebooks).

The [DRAKE](#) website also has a number of alternative [installation options](#), including precompiled binaries and Docker instances.

If you want to download all of the notebooks at once, you can `git clone` the [course notes repository](#). You'll likely want to start from the repository's root directory. Then launch your notebook with:

```
jupyter notebook
```

 Copy code

The examples for each chapter that has them will be in a `.ipynb` file right alongside the chapter's html file, and the notebook exercises are all located in the [exercises](#) subdirectory.

## A.4 GETTING HELP

If you have trouble with [DRAKE](#), please follow the advice [here](#). If you have trouble with the underactuated repo, you can check for known issues (and potentially file a new one) [here](#).



# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## APPENDIX B

# Multi-Body Dynamics

### B.1 DERIVING THE EQUATIONS OF MOTION

The equations of motion for a standard robot can be derived using the method of Lagrange. Using  $T$  as the total kinetic energy of the system, and  $U$  as the total potential energy of the system,  $L = T - U$ , and  $\tau_i$  as the element of the generalized force vector corresponding to  $q_i$ , the Lagrangian dynamic equations are:

$$\frac{d}{dt} \frac{\partial L}{\partial \dot{q}_i} - \frac{\partial L}{\partial q_i} = \tau_i. \quad (1)$$

You can think of them as a generalization of Newton's equations. For a particle, we have  $T = \frac{1}{2} m \dot{x}^2$ , so  $\frac{d}{dt} \frac{\partial L}{\partial \dot{x}} = m \ddot{x}$ , and  $\frac{\partial L}{\partial x} = -\frac{\partial U}{\partial x} = f$  amounting to  $f = ma$ . But the Lagrangian derivation works in generalized coordinate systems and for constrained motion.

#### Example B.1 (Simple Double Pendulum)

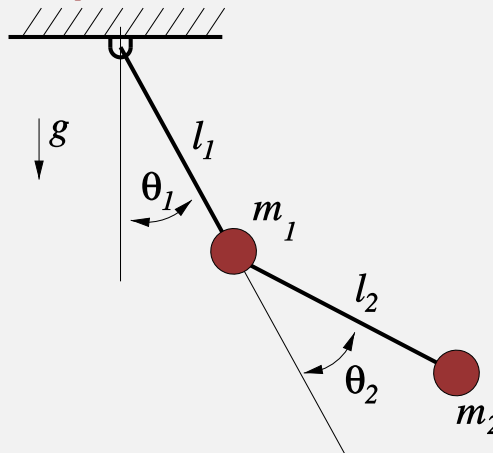


Figure B.1 - Simple double pendulum

Consider the simple double pendulum with torque actuation at both joints and all of the mass concentrated in two points (for simplicity). Using  $\mathbf{q} = [\theta_1, \theta_2]^T$ , and  $\mathbf{p}_1, \mathbf{p}_2$  to denote the locations of  $m_1, m_2$ , respectively, the kinematics of this system are:

$$\begin{aligned}\mathbf{p}_1 &= l_1 \begin{bmatrix} s_1 \\ -c_1 \end{bmatrix}, \quad \mathbf{p}_2 = \mathbf{p}_1 + l_2 \begin{bmatrix} s_{1+2} \\ -c_{1+2} \end{bmatrix} \\ \dot{\mathbf{p}}_1 &= l_1 \dot{q}_1 \begin{bmatrix} c_1 \\ s_1 \end{bmatrix}, \quad \dot{\mathbf{p}}_2 = \dot{\mathbf{p}}_1 + l_2 (\dot{q}_1 + \dot{q}_2) \begin{bmatrix} c_{1+2} \\ s_{1+2} \end{bmatrix}\end{aligned}$$

Note that  $s_1$  is shorthand for  $\sin(q_1)$ ,  $c_{1+2}$  is shorthand for  $\cos(q_1 + q_2)$ , etc. From this we can write the kinetic and potential energy:

$$\begin{aligned}T &= \frac{1}{2} m_1 \dot{\mathbf{p}}_1^T \dot{\mathbf{p}}_1 + \frac{1}{2} m_2 \dot{\mathbf{p}}_2^T \dot{\mathbf{p}}_2 \\ &= \frac{1}{2} (m_1 + m_2) l_1^2 \dot{q}_1^2 + \frac{1}{2} m_2 l_2^2 (\dot{q}_1 + \dot{q}_2)^2 + m_2 l_1 l_2 \dot{q}_1 (\dot{q}_1 + \dot{q}_2) c_2 \\ U &= m_1 g y_1 + m_2 g y_2 = -(m_1 + m_2) g l_1 c_1 - m_2 g l_2 c_{1+2}\end{aligned}$$

Taking the partial derivatives

$$\begin{aligned}\frac{\partial T}{\partial q_1} &= 0, \\ \frac{\partial U}{\partial q_1} &= (m_1 + m_2) g l_1 s_1 + m_2 g l_2 s_{1+2}, \\ \frac{\partial T}{\partial q_2} &= -m_2 l_1 l_2 \dot{q}_1 (\dot{q}_1 + \dot{q}_2) s_2, \\ \frac{\partial U}{\partial q_2} &= m_2 g l_2 s_{1+2}, \\ \frac{\partial T}{\partial \dot{q}_1} &= (m_1 + m_2) l_1^2 \dot{q}_1 + m_2 l_2^2 (\dot{q}_1 + \dot{q}_2) + m_2 l_1 l_2 (2\dot{q}_1 + \dot{q}_2) c_2, \\ \frac{\partial T}{\partial \dot{q}_2} &= m_2 l_2^2 (\dot{q}_1 + \dot{q}_2) + m_2 l_1 l_2 \dot{q}_1 c_2, \\ \frac{d}{dt} \frac{\partial T}{\partial \dot{q}_1} &= (m_1 + m_2) l_1^2 \ddot{q}_1 + m_2 l_2^2 (\ddot{q}_1 + \ddot{q}_2) + m_2 l_1 l_2 (2\ddot{q}_1 + \ddot{q}_2) c_2 - m_2 l_1 l_2 (2\dot{q}_1 + \dot{q}_2) \dot{q}_2 s_2, \\ \frac{d}{dt} \frac{\partial T}{\partial \dot{q}_2} &= m_2 l_2^2 (\ddot{q}_1 + \ddot{q}_2) + m_2 l_1 l_2 \ddot{q}_1 c_2 - m_2 l_1 l_2 \dot{q}_1 \dot{q}_2 s_2,\end{aligned}$$

and plugging them into the Lagrangian, reveals the equations of motion:

$$\begin{aligned}(m_1 + m_2) l_1^2 \ddot{q}_1 + m_2 l_2^2 (\ddot{q}_1 + \ddot{q}_2) + m_2 l_1 l_2 (2\ddot{q}_1 + \ddot{q}_2) c_2 \\ - m_2 l_1 l_2 (2\dot{q}_1 + \dot{q}_2) \dot{q}_2 s_2 + (m_1 + m_2) l_1 g s_1 + m_2 g l_2 s_{1+2} = \tau_1 \\ m_2 l_2^2 (\ddot{q}_1 + \ddot{q}_2) + m_2 l_1 l_2 \ddot{q}_1 c_2 + m_2 l_1 l_2 \dot{q}_1^2 s_2 + m_2 g l_2 s_{1+2} = \tau_2\end{aligned}$$

As we saw in chapter 1, numerically integrating (and animating) these equations in **DRAKE** produces the expected result.

If you are not comfortable with these equations, then any good book chapter on rigid body mechanics can bring you up to speed. [1] is an excellent practical guide to robot kinematics/dynamics. But [2] is my favorite mechanics book, by far; I highly recommend it if you want something more. It's also ok to continue on for now thinking of Lagrangian mechanics simply as a recipe than you can apply in a great many situations to generate the equations of motion.

I've also included a few more details about the connections between these equations and the variational principles of mechanics in the notes [below](#), to the extent they may prove useful in our quest to write better optimizations for mechanical systems.

## B.2 THE MANIPULATOR EQUATIONS

If you crank through the Lagrangian dynamics for a few simple robotic manipulators, you will begin to see a pattern emerge - the resulting equations of motion all have a characteristic form. For example, the kinetic energy of your robot can always be written in the form:

$$T = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}}, \quad (2)$$

where  $\mathbf{M}$  is the state-dependent inertia matrix (aka mass matrix). This observation affords some insight into general manipulator dynamics - for example we know that  $\mathbf{M}$  is always positive definite, and symmetric[3, p.107] and has a beautiful sparsity pattern[4] that we should take advantage of in our algorithms.

Continuing our abstractions, we find that the equations of motion of a general robotic manipulator (without kinematic loops) take the form

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u}, \quad (3)$$

where  $\mathbf{q}$  is the joint position vector,  $\mathbf{M}$  is the inertia matrix,  $\mathbf{C}$  captures Coriolis forces, and  $\tau_g$  is the gravity vector. The matrix  $\mathbf{B}$  maps control inputs  $\mathbf{u}$  into generalized forces. Note that we pair  $\mathbf{M}\ddot{\mathbf{q}} + \mathbf{C}\dot{\mathbf{q}}$  together on the left side because these terms together represent the force of inertia; the contribution from kinetic energy to the Lagrangian:

$$\frac{d}{dt} \frac{\partial T}{\partial \dot{\mathbf{q}}} - \frac{\partial T}{\partial \mathbf{q}} = \mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}.$$

Whenever I write Eq. (3), I see  $ma = f$ .

### Example B.2 (Manipulator Equation form of the Simple Double Pendulum)

The equations of motion from Example 1 can be written compactly as:

$$\begin{aligned} \mathbf{M}(\mathbf{q}) &= \begin{bmatrix} (m_1 + m_2)l_1^2 + m_2l_2^2 + 2m_2l_1l_2c_2 & m_2l_2^2 + m_2l_1l_2c_2 \\ m_2l_2^2 + m_2l_1l_2c_2 & m_2l_2^2 \end{bmatrix} \\ \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) &= \begin{bmatrix} 0 & -m_2l_1l_2(2\dot{q}_1 + \dot{q}_2)s_2 \\ \frac{1}{2}m_2l_1l_2(2\dot{q}_1 + \dot{q}_2)s_2 & -\frac{1}{2}m_2l_1l_2\dot{q}_1s_2 \end{bmatrix} \\ \tau_g(\mathbf{q}) &= -g \begin{bmatrix} (m_1 + m_2)l_1s_1 + m_2l_2s_{1+2} \\ m_2l_2s_{1+2} \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \end{aligned}$$

Note that this choice of the  $\mathbf{C}$  matrix was not unique.

The manipulator equations are very general, but they do define some important characteristics. For example,  $\ddot{\mathbf{q}}$  is (state-dependent) linearly related to the control input,  $\mathbf{u}$ . This observation justifies the control-affine form of the dynamics assumed throughout the notes. There are many other important properties that might prove useful. For instance, we know that the  $i$  element of  $\mathbf{C}\dot{\mathbf{q}}$  is given by [3, §5.2.3]:

$$[\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}]_i = \dot{\mathbf{q}}^T \bar{\mathbf{C}}_i \dot{\mathbf{q}}, \quad \bar{\mathbf{C}}_{i,j,k}(\mathbf{q}) = \frac{1}{2} \left[ \frac{\partial M_{ij}}{\partial q_k} + \frac{\partial M_{ik}}{\partial q_j} - \frac{\partial M_{jk}}{\partial q_i} \right],$$

(e.g. it also quadratic in  $\dot{\mathbf{q}}$ ). For the appropriate choice of  $\mathbf{C}$ , we have that  $\dot{\mathbf{M}}(\mathbf{q}) - 2\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$  is skew-symmetric[5, §9.1].

Note that we have chosen to use the notation of second-order systems (with  $\dot{\mathbf{q}}$  and  $\ddot{\mathbf{q}}$  appearing in the equations) throughout this book. Although I believe it provides more clarity, there is an important limitation to this notation: it is impossible to describe 3D rotations in "minimal coordinates" using this notation without

introducing kinematic singularities (like the famous "gimbal lock"). For instance, a common singularity-free choice for representing a 3D rotation is the unit quaternion, described by 4 real values (plus a norm constraint). However we can still represent the rotational velocity without singularities using just 3 real values. This means that the length of our velocity vector is no longer the same as the length of our position vector. For this reason, you will see that most of the software in **DRAKE** uses the more general notation with  $\mathbf{v}$  to represent velocity,  $\mathbf{q}$  to represent positions, and the manipulator equations are written as

$$\mathbf{M}(\mathbf{q})\dot{\mathbf{v}} + \mathbf{C}(\mathbf{q}, \mathbf{v})\mathbf{v} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u}, \quad (4)$$

which is not necessarily a second-order system.  $\mathbf{v}$  and  $\dot{\mathbf{q}}$  are related linearly by  $\dot{\mathbf{q}} = \mathbf{N}(\mathbf{q})\mathbf{v}$ . Although  $\mathbf{N}(\mathbf{q})$  is not necessarily square, its left Moore-Penrose pseudo-inverse  $\mathbf{N}^+(\mathbf{q})$  can be used to invert that relationship without residual error, provided that  $\dot{\mathbf{q}}$  is in the range space of  $\mathbf{N}(\mathbf{q})$  (that is, if it could have been produced as  $\dot{\mathbf{q}} = \mathbf{N}(\mathbf{q})\mathbf{v}$  for some  $\mathbf{v}$ ). See [6] for a nice discussion of this topic.

## B.2.1 Recursive Dynamics Algorithms

The equations of motions for our machines get complicated quickly. Fortunately, for robots with a tree-link kinematic structure, there are very efficient and natural recursive algorithms for generating these equations of motion. For a detailed reference on these methods, see [7]; some people prefer reading about the Articulated Body Method in [8]. The implementation in **DRAKE** uses a related formulation from [9].

## B.2.2 Bilateral Position Constraints

If our robot has closed-kinematic chains, for instance those that arise from a four-bar linkage, then we need a little more. The Lagrangian machinery above assumes "minimal coordinates"; if our state vector  $\mathbf{q}$  contains all of the links in the kinematic chain, then we do not have a minimal parameterization -- each kinematic loop adds (at least) one constraint so should remove (at least) one degree of freedom. Although some constraints can be solved away, the more general solution is to use the Lagrangian to derive the dynamics of the unconstrained system (a kinematic tree without the closed-loop constraints), then add additional generalized forces that ensure that the constraint is always satisfied.

Consider the constraint equation

$$\mathbf{h}(\mathbf{q}) = 0. \quad (5)$$

For example, in the case of the kinematic closed-chain, this can be the kinematic constraint that the location of one end of the chain is equal to the location of the other end of the chain. In the following derivation, we'll be using the time derivatives of this constraint:

$$\dot{\mathbf{h}} = \mathbf{H}\mathbf{v} = 0, \quad (6)$$

$$\ddot{\mathbf{h}} = \mathbf{H}\dot{\mathbf{v}} + \dot{\mathbf{H}}\mathbf{v} = 0, \quad (7)$$

where  $\mathbf{H}(\mathbf{q}) = \frac{\partial \mathbf{h}}{\partial \mathbf{q}}\mathbf{N}(\mathbf{q})$  is the "Jacobian with respect to  $\mathbf{v}$ ". Note that  $\dot{\mathbf{H}}\mathbf{v}$  is best calculated directly (rather than trying to solve for  $\dot{\mathbf{H}}$  as an object by itself).

The equations of motion can be written

$$\mathbf{M}(\mathbf{q})\dot{\mathbf{v}} + \mathbf{C}(\mathbf{q}, \mathbf{v})\mathbf{v} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u} + \mathbf{H}^T(\mathbf{q})\lambda, \quad (8)$$

where  $\lambda$  is the constraint force. Let's use the shorthand

$$\mathbf{M}(\mathbf{q})\dot{\mathbf{v}} = \tau(\mathbf{q}, \dot{\mathbf{q}}, \mathbf{u}) + \mathbf{H}^T(\mathbf{q})\lambda. \quad (9)$$



In an optimization-based formulation (for instance, to implement this constraint in a trajectory optimization), we can simply make  $\lambda$  a decision variable, and use equations (7) and (9) as constraints.

In other cases, it can be desirable to solve explicitly for  $\lambda$ . Inserting the dynamics (9) into (7) yields

$$\lambda = -(\mathbf{H}\mathbf{M}^{-1}\mathbf{H}^T)^+(\mathbf{H}\mathbf{M}^{-1}\boldsymbol{\tau} + \dot{\mathbf{H}}\mathbf{v}). \quad (10)$$

In many cases, this matrix will be full rank (for instance, in the case of multiple independent four-bar linkages) and the traditional inverse could have been used. When the matrix drops rank (multiple solutions), then the pseudo-inverse will select the solution with the smallest constraint forces in the least-squares sense.

To combat numerical "constraint drift", one might like to add a restoring force in the event that the constraint is not satisfied to numerical precision. To accomplish this, rather than solving for  $\ddot{\mathbf{h}} = 0$  as above, we can instead solve for

$$\ddot{\mathbf{h}} = \mathbf{H}\dot{\mathbf{v}} + \dot{\mathbf{H}}\mathbf{v} = -2\alpha\dot{\mathbf{h}} - \alpha^2\mathbf{h}, \quad (11)$$

where  $\alpha > 0$  is a stiffness parameter. This is known as Baumgarte's stabilization technique, implemented here with a single parameter to provide a critically-damped response. Carrying this through yields

$$\lambda = -(\mathbf{H}\mathbf{M}^{-1}\mathbf{H}^T)^+(\mathbf{H}\mathbf{M}^{-1}\boldsymbol{\tau} + \dot{\mathbf{H}}\mathbf{v} + 2\alpha\mathbf{H}\mathbf{v} + \alpha^2\mathbf{h}). \quad (12)$$

There is an important optimization-based derivation/interpretation of these equations, which we will build on below, using [Gauss's Principle of Least Constraint](#). This principle says that we can derive the constrained dynamics as :

$$\begin{aligned} \min_{\dot{\mathbf{v}}} \quad & \frac{1}{2}(\dot{\mathbf{v}} - \dot{\mathbf{v}}_{uc})^T \mathbf{M}(\dot{\mathbf{v}} - \dot{\mathbf{v}}_{uc}), \\ \text{subject to} \quad & \ddot{\mathbf{h}}(\mathbf{q}, \mathbf{v}, \dot{\mathbf{v}}) = 0 = \mathbf{H}\dot{\mathbf{v}} + \dot{\mathbf{H}}\mathbf{v}, \end{aligned} \quad (13)$$

where  $\dot{\mathbf{v}}_{uc} = \mathbf{M}^{-1}\boldsymbol{\tau}$  is the "unconstrained acceleration" [10]. Equation (8) comes right out of the optimality conditions with  $\lambda$  acting precisely as the Lagrange multiplier. This is a [convex quadratic program](#) with equality constraints, which has a closed-form solution that is precisely Equation (10) above. It is also illustrative to look at the dual formulation of this optimization, which can be written as

$$\min_{\lambda} \frac{1}{2}\lambda^T \mathbf{H}\mathbf{M}^{-1}\mathbf{H}^T \lambda - \lambda^T (\mathbf{H}\mathbf{M}^{-1}\boldsymbol{\tau} + \dot{\mathbf{H}}\mathbf{v}).$$

Observe that the linear term contains the acceleration of  $\mathbf{h}$  if the dynamics evolved without the constraint forces applied; let's call that  $\ddot{\mathbf{h}}_{uc}$ :

$$\min_{\lambda} \frac{1}{2}\lambda^T \mathbf{H}\mathbf{M}^{-1}\mathbf{H}^T \lambda - \lambda^T \ddot{\mathbf{h}}_{uc}.$$

The primal formulation has accelerations as the decision variables, and the dual formulation has constraint forces as the decision variables. For now this is merely a cute observation; we'll build on it more when we get to discussing contact forces.

### B.2.3 Bilateral Velocity Constraints

Consider the constraint equation

$$\mathbf{h}_v(\mathbf{q}, \mathbf{v}) = 0, \quad (14)$$

where  $\frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} \neq 0$ . These are less common, but arise when, for instance, a joint is driven through a prescribed motion. Again, we will use the derivatives,

$$\dot{\mathbf{h}}_v = \frac{\partial \mathbf{h}_v}{\partial \mathbf{q}} \mathbf{N}(\mathbf{q}) \mathbf{v} + \frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} \dot{\mathbf{v}} = 0, \quad (15)$$

and, the manipulator equations given by

$$\mathbf{M} \dot{\mathbf{v}} = \boldsymbol{\tau} + \frac{\partial \mathbf{h}_v}{\partial \mathbf{v}}^T \lambda. \quad (16)$$

These constraints can be added directly to an optimization with  $\lambda$  as a decision variable. Alternatively, we can solve for  $\lambda$  explicitly yielding

$$\lambda = - \left( \frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} \mathbf{M}^{-1} \frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} \right)^+ \left[ \frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} \mathbf{M}^{-1} \boldsymbol{\tau} + \frac{\partial \mathbf{h}_v}{\partial \mathbf{q}} \mathbf{N}(\mathbf{q}) \mathbf{v} \right]. \quad (17)$$

Again, to combat constraint drift, we can ask instead for  $\dot{\mathbf{h}}_v = -\alpha \mathbf{h}_v$ , which yields

$$\lambda = - \left( \frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} \mathbf{M}^{-1} \frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} \right)^+ \left[ \frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} \mathbf{M}^{-1} \boldsymbol{\tau} + \frac{\partial \mathbf{h}_v}{\partial \mathbf{q}} \mathbf{N}(\mathbf{q}) \mathbf{v} + \alpha \mathbf{h}_v \right]. \quad (18)$$

### Example B.3 (Joint locking)

Consider the simple, but important, case of "locking" a joint (making a particular joint act as if it is welded in the current configuration). One could write this as a bilateral velocity constraint

$$\mathbf{h}_v(\mathbf{q}, \mathbf{v}) = v_i = 0.$$

Since  $\frac{\partial \mathbf{h}_v}{\partial \mathbf{v}} = \mathbf{e}_i$ , (the vector with 1 in the  $i$ th element and zero everywhere else, equation (15) reduces to  $\dot{v}_i = 0$ , and  $\frac{\partial \mathbf{h}_v}{\partial \mathbf{v}}^T \lambda$  impacts the equation for  $\dot{v}_i$  and \emph{only} the equation for  $\dot{v}_i$ . The resulting equations can be obtained by simply removing the terms associated with  $v_i$  from the manipulator dynamics (and setting  $\dot{v}_i = 0$ ).

## B.2.4 Hybrid models via constraint forces

For hybrid models of contact, it can be useful to work with a description of the dynamics in the floating-base coordinates and solve for the contact forces which enforce the constraints in each mode. We'll define a mode by a list of pairs of contact geometries that are in contact, and even define whether each contact is in sticking contact (with zero relative tangential velocity) or sliding contact. We can ask our geometry engine to return the contact points on the bodies,  ${}^A \mathbf{p}^{C_a}$  and  ${}^B \mathbf{p}^{C_b}$ , where  $C_a$  is the contact point (or closest point to contact) in body  $A$  and  $C_b$  is the contact point (or closest point to contact) in body  $B$ . More generally, we will refer to  $C_a$  and  $C_b$  as contact frames with their origins at these points, their  $z$ -axis pointing out along the contact normal, and their  $x - y$ -axes chosen as an orthonormal basis orthogonal to the normal.

For each sticking contact, we have a bilateral position constraint of the form

$$\mathbf{h}_{A,B}(\mathbf{q}) = {}^W \mathbf{p}_{C_a}^{C_a} - {}^W \mathbf{p}_{C_b}^{C_b} = 0.$$

We can assemble all of these constraints into a single bilateral constraint vector  $\mathbf{h}(\mathbf{q}) = 0$ , and use almost precisely the recipe above. Note that applying Baumgarte stabilization here would only stabilize in the normal direction, by definition the vector between the two points, which is consistent with having (only) a zero-velocity constraint in the tangential direction. For sliding contacts, we have just the scalar constraint,

$$\mathbf{h}_{A,B}(\mathbf{q}) = {}^W \mathbf{p}_{C_{a;z}}^{C_a} - {}^W \mathbf{p}_{C_{b;z}}^{C_b} = 0,$$

which says that the distance is zero in the normal direction, plus a constraint the imposes maximum dissipation

$$\lambda_{C_{a;x,y}}^{C_a} = -\mu \lambda_{C_{a;z}}^{C_a} \frac{W \mathbf{v}_{C_{a;x,y}}^{C_a}}{|W \mathbf{v}_{C_{a;x,y}}^{C_a}|},$$

that can be accommodated when we solve for  $\lambda$ .

## B.3 THE DYNAMICS OF CONTACT

The dynamics of multibody systems that make and break contact are closely related to the dynamics of constrained systems, but tend to be much more complex. In the simplest form, you can think of non-penetration as an *inequality* constraint: the signed distance between collision bodies must be non-negative. But, as we have seen in the chapters on walking, the transitions when these constraints become active correspond to collisions, and for systems with momentum they require some care. We'll also see that frictional contact adds its own challenges.

There are three main approaches used to modeling contact with "rigid" body systems: 1) rigid contact approximated with stiff compliant contact, 2) hybrid models with collision event detection, impulsive reset maps, and continuous (constrained) dynamics between collision events, and 3) rigid contact approximated with time-averaged forces (impulses) in a time-stepping scheme. Each modeling approach has advantages/disadvantages for different applications.

Before we begin, there is a bit of notation that we will use throughout. Let  $\phi(\mathbf{q})$  indicate the relative (signed) distance between two rigid bodies. For rigid contact, we would like to enforce the unilateral constraint:

$$\phi(\mathbf{q}) \geq 0. \quad (19)$$

We'll use  $\mathbf{n} = \frac{\partial \phi}{\partial \mathbf{q}}$  to denote the "contact normal", as well as  $\mathbf{t}_1$  and  $\mathbf{t}_2$  as a basis for the tangents at the contact (orthonormal vectors in the Cartesian space, projected into joint space), all represented as row vectors in joint coordinates.

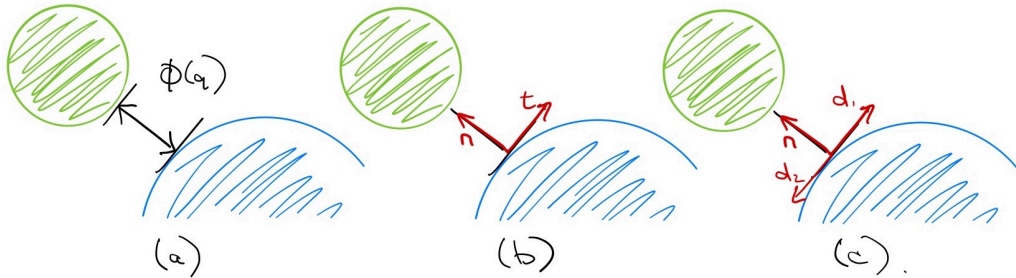


Figure B.2 - Contact coordinates in 2D. (a) The signed distance between contact bodies,  $\phi(\mathbf{q})$ . (b) The normal ( $\mathbf{n}$ ) and tangent ( $\mathbf{t}$ ) contact vectors -- note that these can be drawn in 2D when the  $\mathbf{q}$  is the  $x, y$  positions of one of the bodies, but more generally the vectors live in the configuration space. (c) Sometimes it will be helpful for us to express the tangential coordinates using  $d_1$  and  $d_2$ ; this will make more sense in the 3D case.

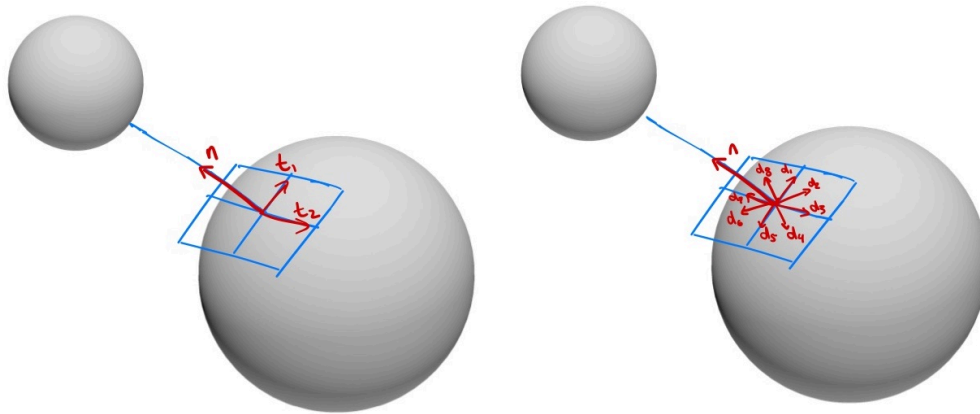


Figure B.3 - Contact coordinates in 3D.

We will also find it useful to assemble the contact normal and tangent vectors into a single matrix,  $\mathbf{J}$ , that we'll call the *contact Jacobian*:

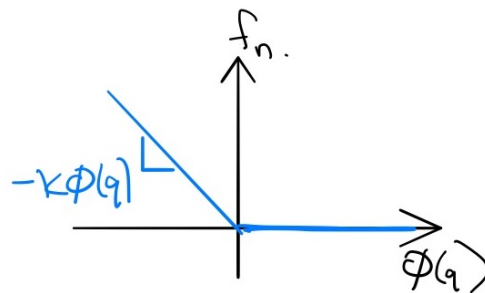
$$\mathbf{J}(\mathbf{q}) = \begin{bmatrix} \mathbf{n} \\ \mathbf{t}_1 \\ \mathbf{t}_2 \end{bmatrix}.$$

As written,  $\mathbf{J}\mathbf{v}$  gives the relative velocity of the closest points in the contact coordinates; it can be also be extended with three more rows to output a full spatial velocity (e.g. when modeling torsional friction). The generalized force produced by the contact is given by  $\mathbf{J}^T\lambda$ , where  $\lambda = [f_n, f_{t1}, f_{t2}]^T$ :

$$\mathbf{M}(\mathbf{q})\dot{\mathbf{v}} + \mathbf{C}(\mathbf{q}, \mathbf{v})\mathbf{v} = \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u} + \mathbf{J}^T(\mathbf{q})\lambda. \quad (20)$$

### B.3.1 Compliant Contact Models

Most compliant contact models are conceptually straight-forward: we will implement contact forces using a stiff spring (and damper[11]) that produces forces to resist penetration (and grossly model the dissipation of collision and/or friction). For instance, for the normal force,  $f_n$ , we can use a simple (piecewise) linear spring law:



Coulomb friction is described by two parameters --  $\mu_{static}$  and  $\mu_{dynamic}$  -- which are the coefficients of static and dynamic friction. When the contact tangential velocity (given by  $\mathbf{tv}$ ) is zero, then friction will produce whatever force is necessary to resist motion, up to the threshold  $\mu_{static}f_n$ . But once this threshold is exceeding, we have slip (non-zero contact tangential velocity); during slip friction will produce a constant drag force  $|f_t| = \mu_{dynamic}f_n$  in the direction opposing the motion. This behavior is not a simple function of the current state, but we can approximate it with a continuous function as pictured below.

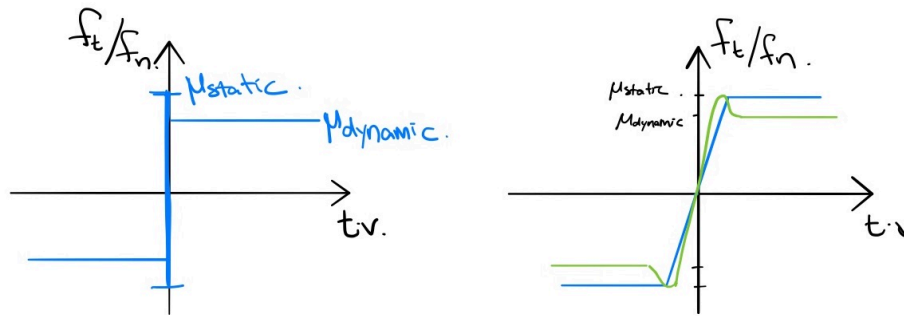


Figure B.5 - (left) The Coloumb friction model. (right) A continuous piecewise-linear approximation of friction (blue) and the [Stribeck approximation of Coloumb friction](#) (green); the  $x$ -axis is the contact tangential velocity, and the  $y$ -axis is the friction coefficient.

With these two laws, we can recover the contact forces as relatively simple functions of the current state. However, the devil is in the details. There are a few features of this approach that can make it numerically unstable. If you've ever been working in a robotics simulator and watched your robot take a step only to explode out of the ground and go flying through the air, you know what I mean.

In order to tightly approximate the (nearly) rigid contact that most robots make with the world, the stiffness of the contact "springs" must be quite high. For instance, I might want my 180kg humanoid robot model to penetrate into the ground no more than 1mm during steady-state standing. The challenge with adding stiff springs to our model is that this results in [stiff differential equations](#) (it is not a coincidence that the word *stiff* is the common term for this in both springs and ODEs). As a result, the best implementations of compliant contact for simulation make use of special-purpose numerical integration recipes (e.g. [12]) and compliant contact models are often difficult to use in e.g. trajectory/feedback optimization.

Note that there is one other type of friction that we talk about in robotics. In joints we also, additionally, model joint damping, often as a linear force proportional to the velocity which resists motion. Phenomenologically, this is due to a viscous (fluid) drag force produced by lubricants in the joint. The static and dynamic friction described above are sometimes collectively referred to as "dry" friction.

But there is another serious numerical challenge with the basic implementation of this model. Computing the signed-distance function,  $\phi(\mathbf{q})$ , when it is non-negative is straightforward, but robustly computing the "penetration depth" once two bodies are in collision is not. Consider the case of use  $\phi(\mathbf{q})$  to represent the maximum penetration depth of, say, two boxes that are contacting each other. With compliant contact, we must have some penetration to produce any contact force. But the direction of the normal vector,  $\mathbf{n} = \frac{\partial \phi}{\partial \mathbf{q}}$ , can easily change discontinuously as the point of maximal penetration moves, as I've tried to illustrate here:

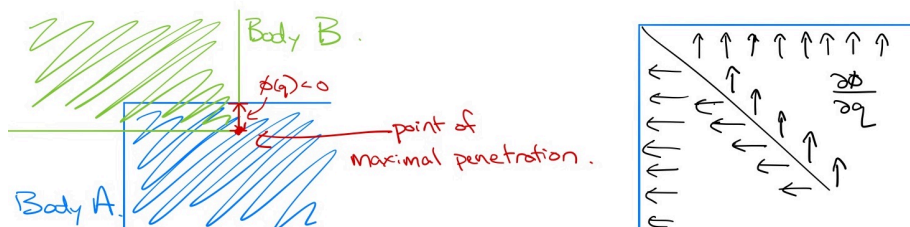


Figure B.6 - (Left) Two boxes in penetration, with the signed distance defined by the maximum penetration depth. (Right) The contact normal for various points of maximal penetration.

If you really think about this example, it actually reveals even more of the foibles of trying to even define the contact points and normals consistently. It seems reasonable to define the point on body B as the point of maximal penetration into

body A, but notice that as I've drawn it, that isn't actually unique! The corresponding point on body A should probably be the point on the surface with the minimal distance from the max penetration point (other tempting choices would likely cause the distance to be discontinuous at the onset of penetration). But this whole strategy is asymmetric; why shouldn't I have picked the vector going with maximal penetration into body B? My point is simply that these cases exist, and that even our best software implementations get pretty unsatisfying when you look into the details. And in practice, it's a lot to expect the collision engine to give consistent and smooth contact points out in every situation.

Some of the advantages of this approach include (1) the fact that it is easy to implement, at least for simple geometries, (2) by virtue of being a continuous-time model it can be simulated with error-controlled integrators, and (3) the tightness of the approximation of "rigid" contact can be controlled through relatively intuitive parameters. However, the numerical challenges of dealing with penetration are very real, and they motivate our other two approaches that attempt to more strictly enforce the non-penetration constraints.

### B.3.2 Rigid Contact with Event Detection

#### Impulsive Collisions

The collision event is described by the zero-crossings (from positive to negative) of  $\phi$ . Let us start by assuming frictionless collisions, allowing us to write

$$\mathbf{M}\dot{\mathbf{v}} = \tau + \lambda \mathbf{n}^T, \quad (21)$$

where  $\mathbf{n}$  is the "contact normal" we defined above and  $\lambda \geq 0$  is now a (scalar) impulsive force that is well-defined when integrated over the time of the collision (denoted  $t_c^-$  to  $t_c^+$ ). Integrate both sides of the equation over that (instantaneous) interval:

$$\int_{t_c^-}^{t_c^+} dt [\mathbf{M}\dot{\mathbf{v}}] = \int_{t_c^-}^{t_c^+} dt [\tau + \lambda \mathbf{n}^T]$$

Since  $\mathbf{M}$ ,  $\tau$ , and  $\mathbf{n}$  are constants over this interval, we are left with

$$\mathbf{M}\mathbf{v}^+ - \mathbf{M}\mathbf{v}^- = \mathbf{n}^T \int_{t_c^-}^{t_c^+} \lambda dt,$$

where  $\mathbf{v}^+$  is short-hand for  $\mathbf{v}(t_c^+)$ . Multiplying both sides by  $\mathbf{n}\mathbf{M}^{-1}$ , we have

$$\mathbf{n}\mathbf{v}^+ - \mathbf{n}\mathbf{v}^- = \mathbf{n}\mathbf{M}^{-1}\mathbf{n}^T \int_{t_c^-}^{t_c^+} \lambda dt.$$

After the collision, we have  $\dot{\phi}^+ = -e\dot{\phi}^-$ , with  $0 \leq e \leq 1$  denoting the [coefficient of restitution](#), yielding:

$$\mathbf{n}\mathbf{v}^+ - \mathbf{n}\mathbf{v}^- = -(1+e)\mathbf{n}\mathbf{v}^-,$$

and therefore

$$\int_{t_c^-}^{t_c^+} \lambda dt = -(1+e)[\mathbf{n}\mathbf{M}^{-1}\mathbf{n}^T]^\# \mathbf{n}\mathbf{v}^-.$$

I've used the notation  $A^\#$  here to denote the pseudoinverse of  $A$  (I would normally write  $A^+$ , but have changed it for this section to avoid confusion). Substituting this back in above results in

$$\mathbf{v}^+ = [\mathbf{I} - (1+e)\mathbf{M}^{-1}\mathbf{n}^T[\mathbf{n}\mathbf{M}^{-1}\mathbf{n}^T]^\#\mathbf{n}] \mathbf{v}^-. \quad (22)$$

We can add friction at the contact. Rigid bodies will almost always experience contact at only a point, so we typically ignore torsional friction [7], and model only tangential friction by extending  $\mathbf{n}$  to a matrix

$$\mathbf{J} = \begin{bmatrix} \mathbf{n} \\ \mathbf{t}_1 \\ \mathbf{t}_2 \end{bmatrix},$$

to capture the gradient of the contact location tangent to the contact surface. Then  $\lambda$  becomes a Cartesian vector representing the contact impulse, and (for infinite friction) the post-impact velocity condition becomes  $\mathbf{J}\mathbf{v}^+ = \text{diag}(-e, 0, 0)\mathbf{J}\mathbf{v}^-$ , resulting in the equations:

$$\mathbf{v}^+ = \left[ \mathbf{I} - \mathbf{M}^{-1}\mathbf{J}^T \text{diag}(1 + e, 1, 1) [\mathbf{J}\mathbf{M}^{-1}\mathbf{J}^T]^\# \mathbf{J} \right] \mathbf{v}^-. \quad (23)$$

Often in optimization, we can directly use the implicit form

$$\begin{aligned} \mathbf{M}(\mathbf{v}^+ - \mathbf{v}^-) &= \mathbf{J}^T \boldsymbol{\Lambda} \\ \mathbf{J}\mathbf{v}^+ &= \text{diag}(-e, 0, 0)\mathbf{J}\mathbf{v}^- \end{aligned}$$

If  $\boldsymbol{\Lambda}$  is restricted to a friction cone, as in Coulomb friction, in general we have to solve an optimization to solve for  $\mathbf{v}^+$  subject to the friction cone constraints on  $\boldsymbol{\Lambda}$ .

### Example B.4 (A spinning ball bouncing on the ground.)

Imagine a ball (a hollow-sphere) in the plane with mass  $m$ , radius  $r$ . The configuration is given by  $\mathbf{q} = [x, z, \theta]^T$ . The equations of motion are

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} = \begin{bmatrix} m & 0 & 0 \\ 0 & m & 0 \\ 0 & 0 & \frac{2}{3}mr^2 \end{bmatrix} \ddot{\mathbf{q}} = \begin{bmatrix} 0 \\ -g \\ 0 \end{bmatrix} + \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 0 & r \end{bmatrix} \begin{bmatrix} \lambda_z \\ \lambda_x \end{bmatrix} = \tau_g + \mathbf{J}^T \lambda,$$

where  $\lambda$  are the contact forces (which are zero except during the impulsive collision). Given a coefficient of restitution  $e$  and a collision with a horizontal ground, the post-impact velocities are:

$$\dot{\mathbf{q}}^+ = \begin{bmatrix} \frac{3}{5} & 0 & -\frac{2}{5}r \\ 0 & -e & 0 \\ -\frac{3}{5r} & 0 & \frac{2}{5} \end{bmatrix} \dot{\mathbf{q}}^-.$$

## Putting it all together

We can put the bilateral constraint equations and the impulsive collision equations together to implement a hybrid model for unilateral constraints of the form. Let us generalize

$$\phi(\mathbf{q}) \geq 0, \quad (24)$$

to be the vector of all pairwise (signed) distance functions between rigid bodies; this vector describes all possible contacts. At every moment, some subset of the contacts are active, and can be treated as a bilateral constraint ( $\phi_i = 0$ ). The guards of the hybrid model are when an inactive constraint becomes active ( $\phi_i > 0 \rightarrow \phi_i = 0$ ), or when an active constraint becomes inactive ( $\lambda_i > 0 \rightarrow \lambda_i = 0$  and  $\dot{\phi}_i > 0$ ). Note that collision events will (almost always) only result in new active constraints when  $e = 0$ , e.g. we have pure inelastic collisions, because elastic collisions will rarely permit sustained contact.

If the contact involves Coulomb friction, then the transitions between sticking and sliding can be modeled as additional hybrid guards.

### B.3.3 Time-stepping Approximations for Rigid Contact

So far we have seen two different approaches to enforcing the inequality constraints of contact,  $\phi(\mathbf{q}) \geq 0$  and the friction cone. First we introduced compliant contact which effectively enforces non-penetration with a stiff spring. Then we discussed event detection as a means to switch between different models which treat the active constraints as equality constraints. But, perhaps surprisingly, one of the most popular and scalable approaches is something different: it involves formulating a mathematical program that can solve the inequality constraints directly on each time step of the simulation. These algorithms may be more expensive to compute on each time step, but they allow for stable simulations with potentially much larger and more consistent time steps.

#### Complementarity formulations

What class of mathematical program do we need to simulate contact? The discrete nature of contact suggests that we might need some form of combinatorial optimization. Indeed, the most common transcription is to a Linear Complementarity Problem (LCP) [13], as introduced by [14] and [15]. An LCP is typically written as

$$\begin{aligned} \text{find}_{\mathbf{w}, \mathbf{z}} \quad & \text{subject to} \quad \mathbf{w} = \mathbf{q} + \mathbf{M}\mathbf{z}, \\ & \mathbf{w} \geq 0, \mathbf{z} \geq 0, \mathbf{w}^T \mathbf{z} = 0. \end{aligned}$$

They are directly related to the optimality conditions of a (potentially non-convex) quadratic program and [15] showed that the LCP's generated by our rigid-body contact problems can be solved by Lemke's algorithm. As short-hand we will write these complementarity constraints as

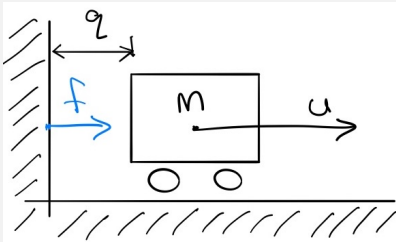
$$\text{find}_{\mathbf{z}} \quad \text{subject to} \quad 0 \leq (\mathbf{q} + \mathbf{M}\mathbf{z}) \perp \mathbf{z} \geq 0.$$

Rather than dive into the full transcription, which has many terms and can be relatively difficult to parse, let me start with two very simple examples.

#### Example B.5 (Time-stepping LCP: Normal force)

Consider our favorite simple mass being actuated by a horizontal force (with the double integrator dynamics), but this time we will add a wall that will prevent the cart position from taking negative values: our non-penetration constraint is  $q \geq 0$ . Physically, this constraint is implemented by a normal (horizontal) force,  $f$ , yielding the equations of motion:

$$m\ddot{q} = u + f.$$



$f$  is defined as the force required to enforce the non-penetration constraint; certainly the following are true:  $f \geq 0$  and  $q \cdot f = 0$ .  $q \cdot f = 0$  is the "complementarity constraint", and you can read it here as "either  $q$  is zero or force is zero" (or both); it is our "no force at a distance" constraint, and it is clearly non-convex. It turns out that satisfying these constraints, plus  $q \geq 0$ , is sufficient to fully define  $f$ .

To define the LCP, we first discretize time, by approximating



$$q[n+1] = q[n] + hv[n+1],$$

$$v[n+1] = v[n] + \frac{h}{m}(u[n] + f[n]).$$

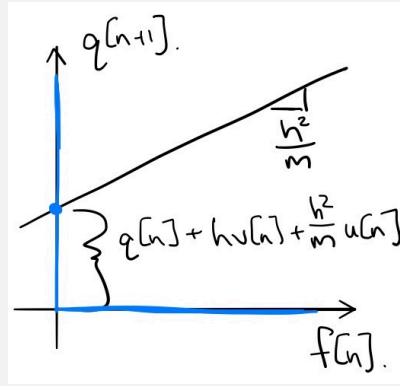
This is almost the standard Euler approximation, but note the use of  $v[n+1]$  in the right-hand side of the first equation -- this is actually a [semi-implicit Euler approximation](#), and this choice is essential in the derivation.

Given  $h, q[n], v[n]$ , and  $u[n]$ , we can solve for  $f[n]$  and  $q[n+1]$  simultaneously, by solving the following LCP:

$$q[n+1] = \left[ q[n] + hv[n] + \frac{h^2}{m}u[n] \right] + \frac{h^2}{m}f[n]$$

$$q[n+1] \geq 0, \quad f[n] \geq 0, \quad q[n+1] \cdot f[n] = 0.$$

Take a moment and convince yourself that this fits into the LCP prescription given above.



Note: Please don't confuse this visualization with the more common visualization of the solution space for an LCP (in two or more dimensions) in terms of "complementary cones"[\[13\]](#).

Perhaps it's also helpful to plot the solution,  $q[n+1], f[n]$  as a function of  $q[n], v[n]$ . I've done it here for  $m = 1, h = 0.1, u[n] = 0$ :

In the (time-stepping, "impulse-velocity") LCP formulation, we write the contact problem in it's combinatorial form. In the simple example above, the complementarity constraints force any solution to lie on *either* the positive x-axis ( $f[n] \geq 0$ ) *or* the positive y-axis ( $q[n+1] \geq 0$ ). The equality constraint is simply a line that will intersect with this constraint manifold at the solution. But in this frictionless case, it is important to realize that these are simply the optimality conditions of a convex optimization problem: the discrete-time version of Gauss's principle that we used above. Using  $\mathbf{v}'$  as shorthand for  $\mathbf{v}[n+1]$ , and replacing  $\mathbf{\ddot{q}} = \frac{\mathbf{v}' - \mathbf{v}}{h}$  in Eq (13) and scaling the objective by  $h^2$  we have:

$$\min_{\mathbf{v}'} \quad \frac{1}{2}(\mathbf{v}' - \mathbf{v} - h\mathbf{M}^{-1}\boldsymbol{\tau})^T \mathbf{M} (\mathbf{v}' - \mathbf{v} - h\mathbf{M}^{-1}\boldsymbol{\tau})$$

$$\text{subject to} \quad \frac{1}{h}\phi(\mathbf{q}') = \frac{1}{h}\phi(\mathbf{q} + h\mathbf{v}') \approx \frac{1}{h}\phi(\mathbf{q}) + \mathbf{n}\mathbf{v}' \geq 0.$$

The objective is even cleaner/more intuitive if we denote the next step velocity that would have occurred before the contact impulse is applied as  $\mathbf{v}^- = \mathbf{v} + h\mathbf{M}^{-1}\boldsymbol{\tau}$ :

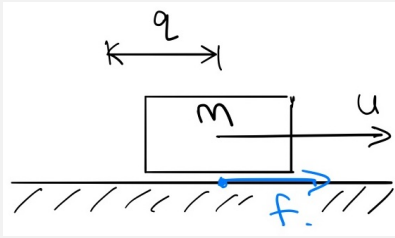
$$\begin{aligned} \min_{\mathbf{v}'} \quad & \frac{1}{2} (\mathbf{v}' - \mathbf{v}^-)^T \mathbf{M} (\mathbf{v}' - \mathbf{v}^-) \\ \text{subject to} \quad & \frac{1}{h} \phi(\mathbf{q}') \geq 0. \end{aligned}$$

The LCP for the frictionless contact dynamics is precisely the optimality conditions of this convex (because  $\mathbf{M} \succeq 0$ ) quadratic program, and once again the contact impulse,  $\mathbf{f} \geq 0$ , plays the role of the Lagrange multiplier (with units  $N \cdot s$ ).

So why do we talk so much about LCPs instead of QPs? Well LCPs can also represent a wider class of problems, which is what we arrive at with the standard transcription of Coulomb friction. In the limit of infinite friction, then we could add an additional constraint that the tangential velocity at each contact point was equal to zero (but these equation may not always have a feasible solution). Once we admit limits on the magnitude of the friction forces, the non-convexity of the disjunctive form rear's its ugly head.

### Example B.6 (Time-stepping LCP: Coulomb Friction)

We can use LCP to find a feasible solution with Coulomb friction, but it requires some gymnastics with slack variables to make it work. For this case in particular, I believe a very simple example is best. Let's take our brick and remove the wall and the wheels (so we now have friction with the ground).



The dynamics are the same as our previous example,

$$m\ddot{q} = u + f,$$

but this time I'm using  $f$  for the friction force which is inside the friction cone if  $\dot{q} = 0$  and on the boundary of the friction cone if  $\dot{q} \neq 0$ ; this is known as the principle of maximum dissipation[14]. Here the magnitude of the normal force is always  $mg$ , so we have  $|f| \leq \mu mg$ , where  $\mu$  is the coefficient of friction. And we will use the same semi-implicit Euler approximation to cast this into discrete time.

Now, to write the friction constraints as an LCP, we must introduce some slack variables. First, we'll break the force into a positive component and a negative component:  $f[n] = f^+ - f^-$ . And we will introduce one more variable  $v_{abs}$  which will be non-zero if the velocity is non-zero (in either direction). Now we can write the LCP:

$$\begin{aligned} \text{find}_{v_{abs}, f^+, f^-} \quad & \text{subject to} \\ 0 \leq v_{abs} + v[n+1] \quad & \perp \quad f^+ \geq 0, \\ 0 \leq v_{abs} - v[n+1] \quad & \perp \quad f^- \geq 0, \\ 0 \leq \mu mg - f^+ - f^- \quad & \perp \quad v_{abs} \geq 0, \end{aligned}$$

where each instance of  $v[n+1]$  we write out with

$$v[n+1] = v[n] + \frac{h}{m}(u + f^+ - f^-).$$

It's enough to make your head spin! But if you work it out, all of the constraints we want are there. For example, for  $v[n+1] > 0$ , then we must have  $f^+ = 0$ ,

$v_{abs} = v[n+1]$ , and  $f^- = \mu mg$ . When  $v[n+1] = 0$ , we can have  $v_{abs} = 0$ ,  $f^+ - f^- \leq \mu mg$ , and those forces must add up to make  $v[n+1] = 0$ .

We can put it all together and write an LCP with both normal forces and friction forces, related by Coulomb friction (using a polyhedral approximation of the friction cone in 3d)[14]. Although the solution to any LCP can also be represented as the solution to a QP, the QP for this problem is non-convex.

## Anitescu's convex formulation

In [16], Anitescu described the natural convex formulation of this problem by dropping the maximum dissipation inequalities and allowing any force inside the friction cone. Consider the following optimization:

$$\begin{aligned} \min_{\mathbf{v}'} \quad & \frac{1}{2}(\mathbf{v}' - \mathbf{v}^-)^T \mathbf{M} (\mathbf{v}' - \mathbf{v}^-) \\ \text{subject to} \quad & \frac{1}{h}\phi(\mathbf{q}') + \mu \mathbf{d}_i \mathbf{v}' \geq 0, \quad \forall i \in 1, \dots, m \end{aligned}$$

where  $\mathbf{d}_i, \forall i \in 1, \dots, m$  are a set of tangent row vectors in joint coordinates parameterizing the polyhedral approximation of the friction cone (as in [14]). Importantly, for each  $\mathbf{d}_i$  we must also have  $-\mathbf{d}_i$  in the set. By writing the Lagrangian,

$$L(\mathbf{v}', \beta) = \frac{1}{2}(\mathbf{v}' - \mathbf{v}^-)^T \mathbf{M} (\mathbf{v}' - \mathbf{v}^-) - \sum_i \beta_i \left( \frac{1}{h}\phi(\mathbf{q}) + (\mathbf{n} + \mu \mathbf{d}_i) \mathbf{v}' \right),$$

and checking the stationarity condition:

$$\frac{\partial L}{\partial \mathbf{v}'}^T = \mathbf{M}(\mathbf{v}' - \mathbf{v}) - h\tau - \sum_i \beta_i (\mathbf{n} + \mu \mathbf{d}_i)^T = 0,$$

we can see that the Lagrange multiplier,  $\beta_i \geq 0$ , associated with the  $i$ th constraint is the magnitude of the impulse (with units  $N \cdot s$ ) in the direction  $\mathbf{n} - \mu \mathbf{d}_i$ , where  $\mathbf{n} = \frac{\partial \phi}{\partial \mathbf{q}}$  is the contact normal; the sum of the forces is an affine combination of these extreme rays of the polyhedral friction cone. As written, the optimization is a QP.

But be careful! Although the primal solution was convex, and the dual problems are always convex, the objective here can be positive semi-definite. This isn't a mistake -- [16] describes a few simple examples where the solution to  $\mathbf{v}'$  is unique, but the impulses that produce it are not (think of a table with four legs).

When the tangential velocity is zero, this constraint is tight; for sliding contact the relaxation effectively causes the contact to "hydroplane" up out of contact, because  $\phi(\mathbf{q}') \geq h\mu \mathbf{d}_i \mathbf{v}'$ . It seems like a quite reasonable approximation to me, especially for small  $h$ !

Let's continue on and write the dual program. To make the notation a little better, let us stack the Jacobian vectors into  $\mathbf{J}_\beta$ , such that the  $i$ th row is  $\mathbf{n} + \mu \mathbf{d}_i$ , so that we have

$$\frac{\partial L}{\partial \mathbf{v}'}^T = \mathbf{M}(\mathbf{v}' - \mathbf{v}) - h\tau - \mathbf{J}_\beta^T \beta = 0.$$

Substituting this back into the Lagrangian give us the dual program:

$$\min_{\beta \geq 0} \frac{1}{2} \beta^T \mathbf{J}_\beta \mathbf{M}^{-1} \mathbf{J}_\beta^T \beta + \frac{1}{h} \phi(\mathbf{q}) \sum_i \beta_i + \beta^T \mathbf{J}_\beta \mathbf{v}^-.$$

One final note: I've written just one contact point here to simplify the notation, but of course we repeat the constraint(s) for each contact point; [16] studies the typical

case of only including potential contact pairs with  $\phi(\mathbf{q}) \leq \epsilon$ , for small  $\epsilon \geq 0$ .

## Todorov's regularization

According to [17], the popular [MuJoCo simulator](#) uses a slight variation of this convex relaxation, with the optimization:

$$\begin{aligned} \min_{\lambda} \quad & \frac{1}{2} \lambda^T \mathbf{J} \mathbf{M}^{-1} \mathbf{J}^T \lambda + \frac{1}{h} \lambda^T (\mathbf{J} \mathbf{v}^- - \dot{\mathbf{x}}^d), \\ \text{subject to} \quad & \lambda \in \mathcal{FC}(\mathbf{q}), \end{aligned}$$

where  $\dot{\mathbf{x}}^d$  is a "desired contact velocity", and  $\mathcal{FC}(\mathbf{q})$  describes the friction cone. The friction cone constraint looks new but is not; observe that  $\mathbf{J}^T \lambda = \mathbf{J}_\beta^T \beta$ , where  $\beta \geq 0$  is a clever parameterization of the friction cone in the polyhedral case. The bigger difference is in the linear term: [17] proposed  $\dot{\mathbf{x}}^d = \mathbf{J} \mathbf{v} - h \mathcal{B} \mathbf{J} \mathbf{v} - h \mathcal{K}[\phi(\mathbf{q}), 0, 0]^T$ , with  $\mathcal{B}$  and  $\mathcal{K}$  stabilization gains that are chosen to yield a critically damped response.

How should we interpret this? If you expand the linear terms, you will see that this is almost exactly the dual form we get from the position equality constraints formulation, including the Baumgarte stabilization. It's interesting to think about the implications of this formulation -- like the Anitescu relaxation, it is possible to get some "force at a distance" because we have not in any way expressed that  $\lambda = 0$  when  $\phi(\mathbf{q}) > 0$ . In Anitescu, it happened in a velocity-dependent boundary layer; here it could happen if you are "coming in hot" -- moving towards contact with enough relative velocity that the stabilization terms want to slow you down.

In dual form, it is natural to consider the full conic description of the friction cone:

$$\mathcal{FC}(\mathbf{q}) = \left\{ \lambda = [f_n, f_{t1}, f_{t2}]^T \mid f_n \geq 0, \sqrt{f_{t1}^2 + f_{t2}^2} \leq \mu f_n \right\}.$$

The resulting dual optimization is has a quadratic objective and second-order cone constraints (SOCP).

There are a number of other important relaxations that happen in MuJoCo. To address the positive indefiniteness in the objective, [17] relaxes the dual objective by adding a small term to the diagonal. This guarantees convexity, but the convex optimization can still be poorly conditioned. The stabilization terms in the objective are another form of relaxation, and [17] also implements adding additional stabilization to the inertial matrix, called the "implicit damping inertia". These relaxations can dramatically improve simulation speed, both by making each convex optimization faster to solve, and by allowing the simulator to take larger integration time steps. MuJoCo boasts a special-purpose solver that can simulate complex scenes at seriously impressive speeds -- often orders of magnitude faster than real time. But these relaxations can also be abused -- it's unfortunately quite common to see physically absurd MuJoCo simulations, especially when researchers who don't care much about the physics start changing simulation parameters to optimize their learning curves!

## The Semi-Analytic Primal (SAP) solver

The contact solver that we use in Drake further extends and improves upon this line of work... [18].

## Beyond Point Contact

Coming soon... Box-on-plane example. Multi-contact.

Also a single point force cannot capture effects like torsional friction, and performs badly in some very simple cases (imaging a box sitting on a table). As a result, many of the most effective algorithms for contact restrict themselves to very limited/simplified geometry. For instance, one can place "point contacts" (zero-radius spheres) in the four corners of a robot's foot; in this case adding forces at exactly

those four points as they penetrate some other body/mesh can give more consistent contact force locations/directions. A surprising number of simulators, especially for legged robots, do this.

In practice, most collision detection algorithms will return a list of "collision point pairs" given the list of bodies (with one point pair per pair of bodies in collision, or more if they are using the aforementioned "multi-contact" heuristics), and our contact force model simply attaches springs between these pairs. Given a point-pair,  $p_A$  and  $p_B$ , both in world coordinates, ...

Hydroelastic model in drake[19].

## B.4 VARIATIONAL MECHANICS

The field of analytic mechanics (or variational mechanics), has given us a number of different and potentially powerful interpretations of these equations. While our principle of virtual work was very much rooted in mechanical work, these variational principles have proven to be much more general. In the context of these notes, the variational principles also provide some deep connections between mechanics and optimization. So let's consider some of the equations above, but through the more general lens of variational mechanics.

### B.4.1 Virtual work

Most of us are comfortable with the idea that a body is in equilibrium if the sum of all forces acting on that body (including gravity) is zero:

$$\sum_i \mathbf{f}_i = 0.$$

It turns out that there are some limitations in working with forces directly: forces require us to (arbitrarily) define a coordinate system, and to achieve force balance we might need to consider *all* of the forces in the system, including internal forces like the forces at the joints that hold our robots together.

So it turns out to be better to think not about forces directly, but to think instead about the *work*,  $w$ , done by those forces. Work is force times distance. It's a scalar quantity, and it is invariant to the choice of coordinate system. Even for a system that is in equilibrium, we can reason about the *virtual work* that would occur if we were to make an infinitesimal change to the configuration of the system. Even better, since we know that the internal forces do no work, we can ignore them completely and only consider the virtual work that occurs due to a variation in the configuration variables. A multibody system in equilibrium must have zero virtual work:

$$\delta w = \sum_i \tau_i^T \delta \mathbf{q} = 0.$$

Notice that these forces are the generalized forces (they have the same dimension as  $\mathbf{q}$ ). We can map between a Cartesian force and the generalized force using a kinematic Jacobian:  $\tau_i = \mathbf{J}_i^T(\mathbf{q}) \mathbf{f}_i$ . In particular, forces due to gravity, or any forces described by a "potential energy" function, can be written as

$$\tau_g = - \frac{\partial U_g(\mathbf{q})}{\partial \mathbf{q}}^T.$$

If all of the generalized forces can be derived from such a potential function, then we can go even further. For a system to be in equilibrium, we must have that variations in the potential energy function must be zero,  $\delta U(\mathbf{q}) = 0$ . This follows simply from  $\delta U = \frac{\partial U}{\partial \mathbf{q}} \delta \mathbf{q}$  and the principle of virtual work. Moreover, for the system to be in a *stable* equilibrium,  $\mathbf{q}$  must not only be a stationary point of the potential, but a *minimum* of the potential. This is our first important connection in this section

between mechanics and optimization.

## B.4.2 D'Alembert's principle and the force of inertia

What about bodies in motion? d'Alembert's principle states that we can reduce dynamics problems to statics problems by simply including a "force of inertia", which is the familiar mass times acceleration for simple particles. In order to have  $\delta w = \sum_i \tau_i^T \delta \mathbf{q} = 0$ , for all  $\delta \mathbf{q}$ , we must have  $\sum_i \tau_i = 0$ , so for multibody systems we know that the force of inertia must be:

$$\tau_{inertia} = -\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} - \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}.$$

But where did that term come from? Is it the result of nature "optimizing" some scalar function?

A natural candidate would be the kinetic energy:  $T(\mathbf{q}, \dot{\mathbf{q}}) = \frac{1}{2} \dot{\mathbf{q}}^T \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}}$ . What would it mean to take a variation of the kinetic energy,  $\delta T$ ? So far we've only had variations with respect to  $\delta \mathbf{q}$ , but the kinetic energy depends on both  $\mathbf{q}$  and  $\dot{\mathbf{q}}$ . We cannot vary these variables independently, but how are they related?

The solution is to take a variation not at a single point, but along a trajectory, where the relationship between  $\delta \mathbf{q}$  and  $\delta \dot{\mathbf{q}}$  is well defined. If we consider

$$A(\mathbf{q}(t)) = \int_{t_0}^{t_1} T(\mathbf{q}, \dot{\mathbf{q}}) dt,$$

then a variation of  $A$  (using the [calculus of variations](#)) w.r.t. any trajectory  $\mathbf{q}(t)$ , will take the form:

$$\delta A = A(\mathbf{q}(t) + \delta \mathbf{q}(t)) - A(\mathbf{q}(t)) = \int_{t_0}^{t_1} T(\mathbf{q} + \delta \mathbf{q}(t), \dot{\mathbf{q}} + \delta \dot{\mathbf{q}}(t)) dt - \int_{t_0}^{t_1} T(\mathbf{q}(t), \dot{\mathbf{q}}(t)) dt,$$

where  $\delta \mathbf{q}(t)$  is defined to be an arbitrary (small) differentiable function in the space of  $\mathbf{q}$  that is zero at  $t_0$  and  $t_1$ .

The derivation takes a few steps, which can be found in most mechanics texts. [Click here to see my version.](#)

We can expand the term in the first integral:

$$T(\mathbf{q}(t) + \delta \mathbf{q}(t), \dot{\mathbf{q}}(t) + \delta \dot{\mathbf{q}}(t)) = T(\mathbf{q}(t), \dot{\mathbf{q}}(t)) + \frac{\partial T}{\partial \mathbf{q}(t)} \delta \mathbf{q}(t) + \frac{\partial T}{\partial \dot{\mathbf{q}}(t)} \delta \dot{\mathbf{q}}(t).$$

Substituting this back in and simplifying leaves:

$$\delta A = \int_{t_0}^{t_1} \left[ \frac{\partial T}{\partial \mathbf{q}(t)} \delta \mathbf{q}(t) + \frac{\partial T}{\partial \dot{\mathbf{q}}(t)} \delta \dot{\mathbf{q}}(t) \right] dt.$$

Now use integration by parts on the second term:

$$\int_{t_0}^{t_1} \frac{\partial T}{\partial \dot{\mathbf{q}}(t)} \delta \dot{\mathbf{q}}(t) dt = \left[ \frac{\partial T}{\partial \dot{\mathbf{q}}(t)} \delta \mathbf{q}(t) \right]_{t_0}^{t_1} - \int_{t_0}^{t_1} \frac{d}{dt} \frac{\partial T}{\partial \dot{\mathbf{q}}(t)} \delta \mathbf{q}(t) dt,$$

and observe that the first term in the right-hand side is zero since  $\delta \mathbf{q}(t_0) = \delta \mathbf{q}(t_1) = 0$ . This leaves

$$\delta A = \int_{t_0}^{t_1} \left[ \frac{\partial T}{\partial \mathbf{q}} - \frac{d}{dt} \frac{\partial T}{\partial \dot{\mathbf{q}}} \right] \delta \mathbf{q}(t) dt.$$

If our goal is to analyze the stationary points of this variation, then this quantity must integrate to zero for any  $\mathbf{q}(t)$ , so we must have

$$\frac{\partial T}{\partial \mathbf{q}} - \frac{d}{dt} \frac{\partial T}{\partial \dot{\mathbf{q}}} = 0.$$

The result is that we find that the natural generalization of  $-\frac{\partial U(\mathbf{q})}{\partial \mathbf{q}}^T$  for stationary points of variations of quantities that also depend on velocity is

$$\tau_{inertia} = -\frac{\partial T}{\partial \mathbf{q}}^T + \frac{d}{dt} \frac{\partial T}{\partial \dot{\mathbf{q}}}^T.$$

This is exactly (the negation of) our familiar  $\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}$ .

### B.4.3 Principle of Stationary Action

The integral quantity  $A$  that we used in the last section is commonly called the "action" of the system. More generally, the action is defined as

$$A = \int_{t_0}^{t_1} L(\mathbf{q}, \dot{\mathbf{q}}) dt,$$

where  $L(\mathbf{q}, \dot{\mathbf{q}})$  is an appropriate "Lagrangian" for the system. The Lagrangian has units of energy, but it is not uniquely defined. Any function which generates the correct equations of motion, in agreement with physical laws, can be taken as a Lagrangian. For our mechanical systems, the familiar form of the Lagrangian is what we presented above:  $L = T - U$  (kinetic energy minus potential energy). Since the terms involving  $U(\mathbf{q})$  do not depend directly on  $\dot{\mathbf{q}}$ , they only contribute the  $\frac{\partial U}{\partial \mathbf{q}}^T$  terms that we expect, and taking  $\delta A = 0$  for this Lagrangian is a way to generate the entire equations of motion in one go. For our purposes, where we care about conservative forces but also control inputs (as generalized forces  $\mathbf{B}\mathbf{u}$ ), the principle of virtual work tells us that we will have  $\delta A + \int_{t_0}^{t_1} \mathbf{u}^T \mathbf{B}^T \delta \mathbf{q}(t) dt = 0$ .

The viewpoint of variational mechanics is that it is this scalar quantity  $L(\mathbf{q}, \dot{\mathbf{q}})$  that should be the central object of study (as opposed to coordinate dependent forces), and this has proven to generalize beautifully. [20] says

The principle of least action -- really the principle of stationary action -- is the most compact form of the classical laws of physics. This simple rule (it can be written in a single line) summarizes everything! Not only the principles of classical mechanics, but electromagnetism, general relativity, quantum mechanics, everything known about chemistry -- right down to the ultimate known constituents of matter, elementary particles.

### B.4.4 Hamiltonian Mechanics

In some cases, it might be preferable to use the [Hamiltonian formulation of the dynamics](#), with state variables  $\mathbf{q}$  and  $\mathbf{p}$  (instead of  $\dot{\mathbf{q}}$ ), where  $\mathbf{p} = \frac{\partial L}{\partial \dot{\mathbf{q}}}^T = \mathbf{M}\dot{\mathbf{q}}$  results in the dynamics:

$$\mathbf{M}(\mathbf{q})\dot{\mathbf{q}} = \mathbf{p}$$

$$\dot{\mathbf{p}} = \mathbf{c}_H(\mathbf{q}, \mathbf{p}) + \tau_g(\mathbf{q}) + \mathbf{B}\mathbf{u}, \quad \text{where} \quad \mathbf{c}_H(\mathbf{q}, \mathbf{p}) = \frac{1}{2} \left( \frac{\partial [\dot{\mathbf{q}}^T \mathbf{M}(\mathbf{q}) \dot{\mathbf{q}}]}{\partial \mathbf{q}} \right)^T.$$

Recall that the Hamiltonian is  $H(\mathbf{p}, \mathbf{q}) = \mathbf{p}^T \dot{\mathbf{q}} - L(\mathbf{q}, \dot{\mathbf{q}})$ ; these equations of motion are the familiar  $\dot{\mathbf{q}} = \frac{\partial H}{\partial \mathbf{p}}^T$ ,  $\dot{\mathbf{p}} = -\frac{\partial H}{\partial \mathbf{q}}^T$ .

## B.5 EXERCISES

### Exercise B.1 (Time-stepping LCP dynamics for block collision)

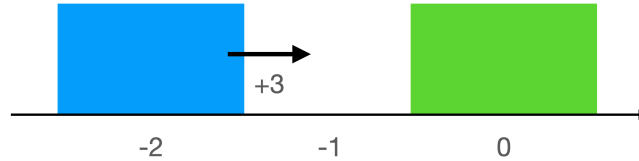


Figure B.11 - Initial conditions for block collision scenario.

Suppose you have a block A of width 1, block B of width 1, interacting on a friction-less surface. The state of a single block in our system is  $\mathbf{x} = \begin{bmatrix} q_x \\ \dot{q}_x \end{bmatrix}$ , which represents the horizontal position (where the position point is located at the geometric center of the block) and horizontal velocity of the associated block. We have as our initial condition  $\mathbf{x}_0^{(A)} = \begin{bmatrix} -2 \\ 3 \end{bmatrix}$  and  $\mathbf{x}_0^{(B)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$ . In other words, at time step 0 block A is moving towards a stationary block B located at the origin.

- Derive the positions, velocities, and contact forces at time steps 1, 2, and 3 for the two block collision scenario described at the start of this problem, under implicit Euler integration. Use time step  $h = 1.0$  and block masses  $m_1 = m_2 = 1$ .  
*Hint:* See Example B.4 in the Appendix setting up the LCP optimization problem for a single cart and wall under semi-implicit integration. In implicit Euler integration, both the position and velocity are integrated with backward finite differences ( $x[n+1] = x[n] + hf[n+1]$ ), whereas in the semi-implicit integration, only the position is integrated with backward finite differences.  
*Hint:* To solve your LCP by hand, you should use the fact that time-stepping LCP produces perfectly inelastic collisions. What does the perfectly inelastic nature of collision imply about the signed distance during collision?
- Since the block B is stationary, we might expect  $q_{1,x}^{(A)} = -1$ . Why is this not the case?
- What would you expect the velocities to be post-collision for perfectly inelastic collision and under conservation of momentum in continuous-time physics? How do these velocities compare with your time-stepped velocities?

## REFERENCES

- John Craig, "Introduction to Robotics: Mechanics and Control", Addison Wesley, January, 1989.
- Cornelius Lanczos, "The variational principles of mechanics", University of Toronto Press, no. 4, 1970.
- H. Asada and J.E. Slotine, "Robot Analysis and Control", pp. 93-131, 1986.
- Roy Featherstone, "Efficient Factorization of the Joint Space Inertia Matrix



for Branched Kinematic Trees", *International Journal of Robotics Research*, vol. 24, no. 6, pp. 487-500, 2005.

5. Jean-Jacques E. Slotine and Weiping Li, "Applied Nonlinear Control", Prentice Hall , October, 1990.
6. Vincent Duindam, "Port-Based Modeling and Control for Efficient Bipedal Walking Robots", PhD thesis, University of Twente, March, 2006.
7. Roy Featherstone, "Rigid Body Dynamics Algorithms", Springer , 2007.
8. Brian Mirtich, "Impulse-based Dynamic Simulation of Rigid Body Systems", PhD thesis, University of California at Berkeley, 1996.
9. Abhinandan Jain, "Robot and Multibody Dynamics: Analysis and Algorithms", Springer US , 2011.
10. Firdaus E Udwadia and Robert E Kalaba, "A new perspective on constrained motion", *Proceedings of the Royal Society of London. Series A: Mathematical and Physical Sciences*, vol. 439, no. 1906, pp. 407--410, 1992.
11. K. H. Hunt and F. R. E. Crossley, "Coefficient of restitution interpreted as damping in vibroimpact", *Journal of Applied Mechanics*, vol. 42 Series E, no. 2, pp. 440-445, 1975.
12. Alejandro M Castro and Ante Qu and Naveen Kuppuswamy and Alex Alspach and Michael Sherman, "A Transition-Aware Method for the Simulation of Compliant Contact with Regularized Friction", *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1859--1866, 2020.
13. Katta G Murty and Feng-Tien Yu, "Linear complementarity, linear and nonlinear programming", *Citeseer* , vol. 3, 1988.
14. D.E. Stewart and J.C. Trinkle, "AN IMPLICIT TIME-STEPPING SCHEME FOR RIGID BODY DYNAMICS WITH INELASTIC COLLISIONS AND COULOMB FRICTION", *International Journal for Numerical Methods in Engineering*, vol. 39, no. 15, pp. 2673--2691, 1996.
15. M. Anitescu and F.A. Potra, "Formulating dynamic multi-rigid-body contact problems with friction as solvable linear complementarity problems", *Nonlinear Dynamics*, vol. 14, no. 3, pp. 231--247, 1997.
16. Mihai Anitescu, "Optimization-based simulation of nonsmooth rigid multibody dynamics", *Mathematical Programming*, vol. 105, no. 1, pp. 113--143, jan, 2006.
17. Emanuel Todorov, "Convex and analytically-invertible dynamics with contacts and constraints: Theory and implementation in {MuJoCo}", *2014 IEEE International Conference on Robotics and Automation (ICRA)* , pp. 6054--6061, 2014.
18. Alejandro Castro and Frank Permenter and Xuchen Han, "An Unconstrained Convex Formulation of Compliant Contact", *arXiv preprint arXiv:2110.10107*, 2021.
19. Ryan Elandt and Evan Drumwright and Michael Sherman and Andy Ruina, "A pressure field model for fast, robust approximation of net contact force and moment between nominally rigid objects", , pp. 8238-8245, 11, 2019.
20. Leonard Susskind and George Hrabovsky, "The theoretical minimum: what you need to know to start doing physics", Basic Books , 2014.



# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## APPENDIX C

# Optimization and Mathematical Programming

The view of dynamics and controls taken in these notes builds heavily on tools from optimization -- and our success in practice depends heavily on the effective application of numerical optimization. There are many excellent books on optimization, for example [1] is an excellent reference on smooth optimization and [2] is an excellent reference on convex optimization (which we use extensively). I will provide more references for the specific optimization formulations below.

The intention of this chapter is, therefore, mainly to provide a launchpad and to address some topics that are particularly relevant in robotics but might be more hidden in the existing general treatments. You can get far very quickly as a consumer of these tools with just a high-level understanding. But, as with many things, sometimes the details matter and it becomes important to understand what is going on under the hood. We can often formulate an optimization problem in multiple ways that might be mathematically equivalent, but perform very differently in practice.

### C.1 OPTIMIZATION SOFTWARE

Some of the algorithms from optimization are quite simple to implement yourself; stochastic gradient descent is perhaps the classic example. Even more of them are conceptually fairly simple, but for some of the algorithms, the implementation details matter a great deal -- the difference between an expert implementation and a novice implementation of the numerical recipe, in terms of speed and/or robustness, can be dramatic. These packages often use a wealth of techniques for numerically conditioning the problems, for discarding trivially valid constraints, and for warm-starting optimization between solves. Not all solvers support all types of objectives and constraints, and even if we have two commercial solvers that both claim to solve, e.g. quadratic programs, then they might perform differently on the particular structure/conditioning of your problem. In some cases, we also have nicely designed open-source alternatives to these commercial solvers (often written by academics who are experts in optimization) -- sometimes they compete well with the commercial solvers or even outperform them in particular problem types.

As a result, there are also a handful of software packages that attempt to provide a layer of abstraction between the formulation of a mathematical program and the instantiation of that problem in each of the particular solvers. Famous examples of this include [CVX](#) and [YALMIP](#) for MATLAB, and [JuMP](#) for Julia. [DRAKE](#)'s `MathematicalProgram` classes provide a middle layer like this for C++ and Python; its creation was motivated initially and specifically by the need to support the optimization formulations we use in this text.

We have a number of tutorials on mathematical programming in [DRAKE](#), starting with a general introduction [here](#). Drake [supports a number of custom, open-source, and commercial solvers](#) (and even some of the commercial solvers are free to academic users).

## C.2 GENERAL CONCEPTS

The general formulation is ...

It's important to realize that even though this formulation is incredibly general, it does have its limits. As just one example, when we write optimizations to plan trajectories of a robot, in this formulation we typically have to choose a-priori as particular number of decision variables that will encode the solution. Although we can of course write algorithms that change the number of variables and call the optimizer again, somehow I feel that this "general" formulation fails to capture, for instance, the type of mathematical programming that occurs in sample-based optimization planning -- where the resulting solutions can be described by any finite number of parameters, and the computation flexibly transitions amongst them.

### C.2.1 Convex vs nonconvex optimization

Local optima. Convex functions, convex constraints.

If an optimization problem is nonconvex, it does not necessarily mean that the optimization is hard. There are many cases in deep learning where we can reliably solve seemingly very high-dimensional and nonconvex optimization problems. Our understanding of these phenomenon is evolving rapidly, and I suspect anything I write here will shortly become outdated. But the current thinking in supervised learning mostly revolves around the idea that over-parameterization is key -- that many of these success stories are happening in a regime where we actually have more decision variables than we have data, and that the search space is actually dense with solutions that can fit the data perfectly (the so-called "interpolating solutions"), that not all global minima are equally robust, and that the optimization algorithms are performing some form of implicit regularization in order to pick a "good" interpolating solution.

### C.2.2 Constrained optimization with Lagrange multipliers

Given the equality-constrained optimization problem

$$\text{minimize}_{\mathbf{z}} \ell(\mathbf{z}) \quad \text{subject to} \quad \phi(\mathbf{z}) = 0,$$

where  $\phi$  is a vector. Define a vector  $\lambda$  of Lagrange multipliers, the same size as  $\phi$ , and the scalar Lagrangian function,

$$L(\mathbf{z}, \lambda) = \ell(\mathbf{z}) + \lambda^T \phi(\mathbf{z}).$$

A necessary condition for  $\mathbf{z}^*$  to be an optimal value of the constrained optimization is that the gradients of  $L$  vanish with respect to both  $\mathbf{z}$  and  $\lambda$ :

$$\frac{\partial L}{\partial \mathbf{z}} = 0, \quad \frac{\partial L}{\partial \lambda} = 0.$$

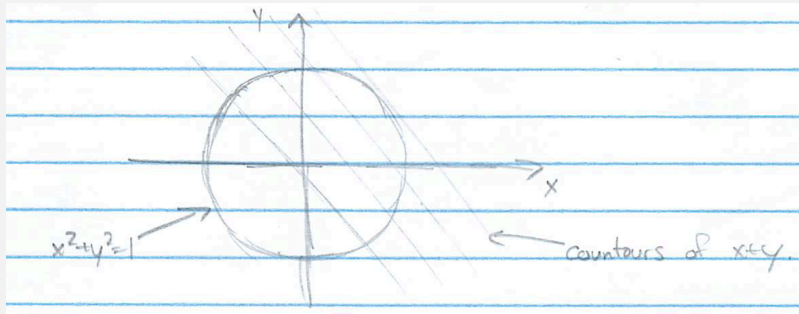
Note that  $\frac{\partial L}{\partial \lambda} = \phi(\mathbf{z})$ , so requiring this to be zero is equivalent to requiring the constraints to be satisfied.

### Example C.1 (Optimization on the unit circle)

Consider the following optimization:

$$\min_{x,y} x + y, \quad \text{subject to } x^2 + y^2 = 1.$$

The level sets of  $x + y$  are straight lines with slope  $-1$ , and the constraint requires that the solution lives on the unit circle.



Simply by inspection, we can determine that the optimal solution should be  $x = y = -\frac{\sqrt{2}}{2}$ . Let's make sure we can obtain the same result using Lagrange multipliers.

Formulating

$$L = x + y + \lambda(x^2 + y^2 - 1),$$

we can take the derivatives and solve

$$\begin{aligned} \frac{\partial L}{\partial x} = 1 + 2\lambda x = 0 &\Rightarrow \lambda = -\frac{1}{2x}, \\ \frac{\partial L}{\partial y} = 1 + 2\lambda y = 1 - \frac{y}{x} = 0 &\Rightarrow y = x, \\ \frac{\partial L}{\partial \lambda} = x^2 + y^2 - 1 = 2x^2 - 1 = 0 &\Rightarrow x = \pm \frac{1}{\sqrt{2}}. \end{aligned}$$

Given the two solutions which satisfy the necessary conditions, the negative solution is clearly the minimizer of the objective.

## C.3 CONVEX OPTIMIZATION

### C.3.1 Linear Programs/Quadratic Programs/Second-Order Cones

Example: Balancing force control on Atlas

### C.3.2 Semidefinite Programming and Linear Matrix Inequalities

Semidefinite programming relaxation of general

## quadratic optimization

One of the most powerful use cases for semidefinite programming is in writing convex relaxations of more general non-convex optimization problems. One particularly useful example of this is the general quadratic optimization problem, written as

$$\begin{aligned} \min_{\mathbf{x}} \quad & \mathbf{x}^T \mathbf{Q} \mathbf{x} \\ \text{subject to} \quad & \mathbf{x}^T \mathbf{A}_i \mathbf{x} \geq 0, \\ & \mathbf{B} \mathbf{x} \geq \mathbf{0}, \\ & \mathbf{x} = \begin{bmatrix} 1 \\ \mathbf{y} \end{bmatrix}. \end{aligned}$$

In the following, we won't make any assumptions about  $\mathbf{Q}$  or  $\mathbf{A}_i$  being PSD. For instance, quadratic equality constraints of the form  $\mathbf{x}^T \mathbf{A} \mathbf{x} = 0$  (which are generically nonconvex) can be encoded with two constraints:  $\mathbf{x}^T \mathbf{A} \mathbf{x} \geq 0$  and  $-\mathbf{x}^T \mathbf{A} \mathbf{x} \geq 0$ . The SDP relaxation [3, 4] is given by

$$\begin{aligned} \min_{\mathbf{X}} \quad & \text{tr}(\mathbf{Q} \mathbf{X}) \\ \text{subject to} \quad & \mathbf{e}_1^T \mathbf{X} \mathbf{e}_1 = 1 \\ & \text{tr}(\mathbf{A}_i \mathbf{X}) \geq 0, \\ & \mathbf{B} \mathbf{X} \mathbf{e}_1 \geq \mathbf{0}, \\ & \mathbf{B} \mathbf{X} \mathbf{B}^T \geq \mathbf{0}, \\ & \mathbf{X} \succeq \mathbf{0}, \end{aligned}$$

where  $\mathbf{e}_1$  is the vector with 1 on the first element and the other elements all zero. Let's try to understand the power of this relaxation through a few simple examples.

### Example C.2 (Semidefinite programming relaxation of non-convex quadratic constraints)

Consider the problem:

$$\begin{aligned} \min_x \quad & \|x - a\|^2 \\ \text{subject to} \quad & \|x - b\|^2 \geq 1 \end{aligned}$$

This problem is non-convex, however we can write a convex relaxation of the problem using the semidefinite-programming relaxation:

$$\begin{aligned} \min_{x,y} \quad & y - 2ax + a^2 \\ \text{subject to} \quad & y - 2bx + b^2 \geq 1 \\ & y \geq x^2 \end{aligned}$$

where we write  $y \geq x^2$  as the semidefinite constraint  $\begin{bmatrix} y & x \\ x & 1 \end{bmatrix} \succeq \mathbf{0}$ .<sup>†</sup>

<sup>†</sup> Note that this particular SDP constraint is so simple that it could have alternatively been written as a second-order cone constraint.

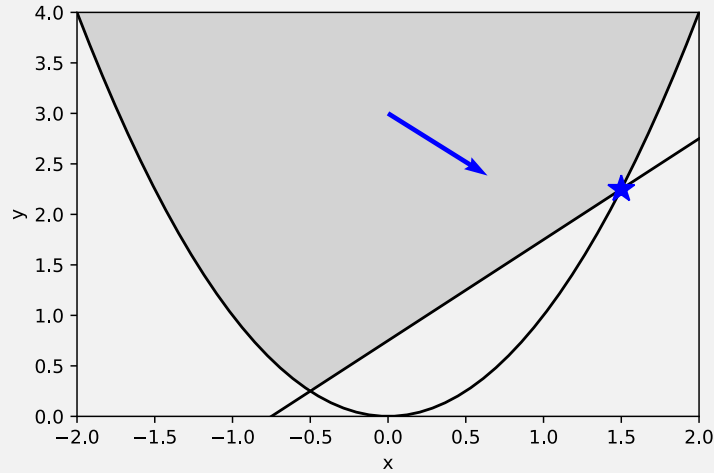


Figure C.2 - Optimization landscape for  $a = .8, b = .5$ .

I've plotted the feasible region and the objective with an arrow. As you can see, the feasible region is the epigraph of  $y = x^2$  intersected with the linear constraint. Now here is the key point: for linear objectives, the optimal solution will (almost) never be active on the linear constraint unless it's at a vertex of the feasible set (this is related to the classical picture from linear programming). In this example specifically, the constraint  $y = x^2$  will be inactive only if  $a = b$ ; for every other objective, this relaxation will give  $y = x^2$ , and will give the optimal solution to the original problem. Note that we could have equally well written a quadratic equality constraint.

### Example C.3 (Semidefinite programming relaxation of the unit circle)

Another useful example is

$$\begin{aligned} \min_{\mathbf{x}} \quad & \mathbf{x}^T \mathbf{A} \mathbf{x} + \mathbf{b}^T \mathbf{x}, \\ \text{subject to} \quad & \mathbf{x}^T \mathbf{x} = 1. \end{aligned}$$

Again this optimization is non-convex, but we can write the semidefinite-programming relaxation as

$$\begin{aligned} \min_{\mathbf{Y}, \mathbf{x}} \quad & \text{tr}(\mathbf{A} \mathbf{Y}) + \mathbf{b}^T \mathbf{x}, \\ \text{subject to} \quad & \text{tr}(\mathbf{Y}) = 1, \\ & \mathbf{Y} \succeq \mathbf{x} \mathbf{x}^T, \end{aligned}$$

where the PSD constraint is again written using the Schur complement. For the case where  $\mathbf{x} \in \mathbb{R}^2$ , the PSD constraint gives us that  $Y_{11} \geq x_1^2$  and  $Y_{22} \geq x_2^2$ ; combined with the trace constraint this gives us

$$Y_{22} = 1 - Y_{11} \geq x_2^2 \Rightarrow 1 - x_2^2 \geq Y_{11} \geq x_1^2 \Rightarrow 1 \geq x_1^2 + x_2^2.$$

Once again the picture looks like the affine section of a cone (now in higher dimensions). Given the linear objective, we expect the optimal solution to be unbounded, or active on one or more of the constraints. Whenever the optimal solution is unique, the optimization will obtain the rank one solution for the PSD constraint (the matrix  $\mathbf{Y}$  is rank one, and the Schur complement matrix is also rank one), which implies that  $\mathbf{Y} = \mathbf{x} \mathbf{x}^T$  and we have  $\mathbf{x}^T \mathbf{x} = 1$ . When the convex

relaxation gives the optimal solution to the original problem, we say that the convex relaxation is "tight".

Let's compare this to the simpler relaxation,

$$\begin{aligned} \min_{\mathbf{x}} \quad & \mathbf{x}^T \mathbf{A} \mathbf{x} + \mathbf{b}^T \mathbf{x}, \\ \text{subject to} \quad & \mathbf{x}^T \mathbf{x} \leq 1, \end{aligned}$$

which can be written as a second-order cone program (SOCP). When the objective is linear ( $\mathbf{A} = 0, \mathbf{b} \neq 0$ ), even this simpler relaxation will be tight. But when the objective is quadratic, if the minimum of the quadratic falls inside the unit disk, then the relaxation will not be tight (it will be "loose") -- the optimal solution will satisfy the constraints  $\mathbf{x}^T \mathbf{x} \leq 1$ , but will not have  $\mathbf{x}^T \mathbf{x} = 1$ . In the PSD relaxation, the convex relaxation will be tight, because the objective is now linear (in the lifted coordinates), and will push towards the boundary so long as  $\mathbf{b} \neq 0$ .

I've given a numerical example you can play with in the notebook: What's amazing is that this SDP relaxation even gives the optimal solution when the objective is non-convex. I've given an example in the notebook with a concave objective (e.g.  $\mathbf{A} \preceq 0$ ).

### C.3.3 Sums-of-squares optimization

It turns out that in the same way that we can use SDP to search over the positive quadratic equations, we can generalize this to search over the positive polynomial equations. To be clear, for quadratic equations we have

$$\mathbf{P} \succeq 0 \quad \Rightarrow \quad \mathbf{x}^T \mathbf{P} \mathbf{x} \geq 0, \quad \forall \mathbf{x}.$$

It turns out that we can generalize this to

$$\mathbf{P} \succeq 0 \quad \Rightarrow \quad \mathbf{m}^T(\mathbf{x}) \mathbf{P} \mathbf{m}(\mathbf{x}) \geq 0, \quad \forall \mathbf{x},$$

where  $\mathbf{m}(\mathbf{x})$  is a vector of polynomial equations, typically chosen as a vector of *monomials* (polynomials with only one term). The set of positive polynomials parameterized in this way is exactly the set of polynomials that can be written as a *sum of squares*[5]. While it is known that not all positive polynomials can be written in this form, much is known about the gap. For our purposes this gap is very small (papers have been written about trying to find polynomials which are uniformly positive but not sums of squares); we should remember that it exists but it won't hinder too many of our use cases.

Even better, there is quite a bit that is known about how to choose the terms in  $\mathbf{m}(\mathbf{x})$ . For example, if you give me the polynomial

$$p(x) = 2 - 4x + 5x^2$$

and ask if it is positive for all real  $x$ , I can convince you that it is by producing the sums-of-squares factorization

$$p(x) = 1 + (1 - 2x)^2 + x^2,$$

or the factorization

$$p(x) = \begin{bmatrix} 1 \\ x \end{bmatrix}^T \begin{bmatrix} 2 & -2 \\ -2 & 5 \end{bmatrix} \begin{bmatrix} 1 \\ x \end{bmatrix},$$

since the matrix in this quadratic form is PSD. In general, I can produce a monomial vector containing all monomials with degree up to the the half of the degree of  $p$  (we can actually do much better, but I will not cover those details). In practice, what



this means for us is that people have authored optimization front-ends which take a high-level specification with constraints on the positivity of polynomials and they automatically generate the SDP problem for you, without having to think about what terms should appear in  $\mathbf{m}(\mathbf{x})$  (c.f. [6]). This class of optimization problems is called Sums-of-Squares (SOS) optimization.

As it happens, the particular choice of  $\mathbf{m}(\mathbf{x})$  can have a huge impact on the numerics of the resulting semidefinite program (and on your ability to solve it with commercial solvers). **DRAKE** implements some particularly novel/advanced algorithms to make this work well [7].

We will write optimizations using sums-of-squares constraints as

$$p(\mathbf{x}) \text{ is SOS}$$

as shorthand for the constraint that  $p(\mathbf{x}) \geq 0$  for all  $\mathbf{x}$ , as demonstrated by finding a sums-of-squares decomposition.

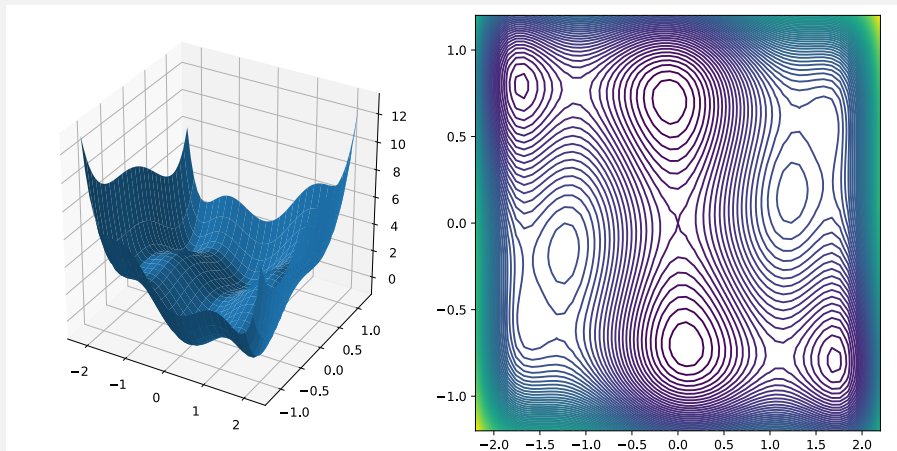
This is surprisingly powerful. It allows us to use convex optimization to solve what appear to be very non-convex optimization problems.

### Example C.4 (Global minimization via SOS)

Consider the following famous non-linear function of two variables (called the "six-hump-camel"):

$$p(x) = 4x^2 + xy - 4y^2 - 2.1x^4 + 4y^4 + \frac{1}{3}x^6.$$

This function has six local minima, two of them being global minima [8].



By formulating a simple sums-of-squares optimization, we can actually find the minimum value of this function (technically, it is only a lower bound, but in this case and many cases, it is surprisingly tight) by writing:

$$\begin{aligned} \max_{\lambda} \quad & \lambda \\ \text{s.t.} \quad & p(x) - \lambda \text{ is sos.} \end{aligned}$$

Go ahead and play with the code (most of the lines are only for plotting; the actual optimization problem is nice and simple to formulate).

Note that this finds the minimum value, but does not immediately produce the  $\mathbf{x}$  value which minimizes it. To extract the minimizer, one needs to examine the dual program (see, for instance [8]). In many applications, such as **Lyapunov analysis**, obtaining the minimizer is not even necessary.

## Sums of squares on a Semi-Algebraic Set

The S-procedure.

## Sums of squares optimization on an Algebraic Variety

The S-procedure

Using the quotient ring

Quotient rings via sampling

## DSOS and SDSOS

### C.3.4 Solution techniques

Interior point (Gurobi, Mosek, Sedumi, ...), First order methods

## C.4 NONLINEAR PROGRAMMING

The generic formulation of a nonlinear optimization problem is

$$\min_z c(z) \quad \text{subject to} \quad \phi(z) \leq 0,$$

where  $z$  is a vector of *decision variables*,  $c$  is a scalar *objective function* and  $\phi$  is a vector of *constraints*. Note that, although we write  $\phi \leq 0$ , this formulation captures positivity constraints on the decision variables (simply multiply the constraint by  $-1$ ) and equality constraints (simply list both  $\phi \leq 0$  and  $-\phi \leq 0$ ) as well.

The picture that you should have in your head is a nonlinear, potentially non-convex objective function defined over (multi-dimensional)  $z$ , with a subset of possible  $z$  values satisfying the constraints.

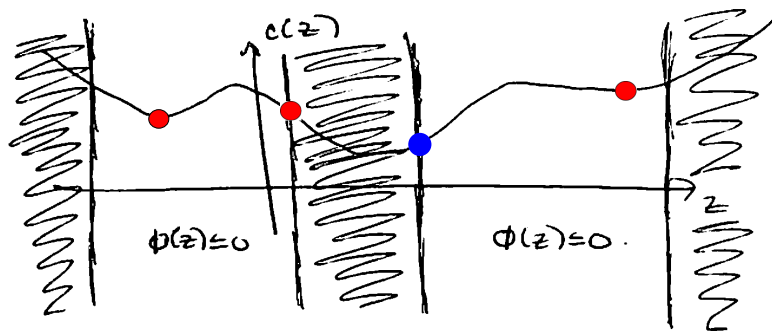


Figure C.3 - One-dimensional cartoon of a nonlinear optimization problem. The red dots represent local minima. The blue dot represents the optimal solution.

Note that minima can be the result of the objective function having zero derivative *or* due to a sloped objective up against a constraint.

Numerical methods for solving these optimization problems require an initial guess,  $\hat{z}$ , and proceed by trying to move down the objective function to a minima. Common approaches include *gradient descent*, in which the gradient of the objective function is computed or estimated, or second-order methods such as *sequential quadratic programming (SQP)* which attempts to make a local quadratic approximation of the objective function and local linear approximations of the constraints and solves a quadratic program on each iteration to jump directly to the minimum of the local approximation.

While not strictly required, these algorithms can often benefit a great deal from having the gradients of the objective and constraints computed explicitly; the alternative is to obtain them from numerical differentiation. Beyond pure speed considerations, I strongly prefer to compute the gradients explicitly because it can help avoid numerical accuracy issues that can creep in with finite difference methods. The desire to calculate these gradients will be a major theme in the discussion below, and we have gone to great lengths to provide explicit gradients of our provided functions and automatic differentiation of user-provided functions in [DRAKE](#).

When I started out, I was of the opinion that there is nothing difficult about implementing gradient descent or even a second-order method, and I wrote all of the solvers myself. I now realize that I was wrong. The commercial solvers available for nonlinear programming are substantially higher performance than anything I wrote myself, with a number of tricks, subtleties, and parameter choices that can make a huge difference in practice. Some of these solvers can exploit sparsity in the problem (e.g., if the constraints operate in a sparse way on the decision variables). Nowadays, we make heaviest use of SNOPT [9], which now comes bundled with the binary distributions of [DRAKE](#), but also [support a large suite of numerical solvers](#). Note that while I do advocate using these tools, you do not need to use them as a black box. In many cases you can improve the optimization performance by understanding and selecting non-default configuration parameters.

### **C.4.1 Second-order methods (SQP / Interior-Point)**

### **C.4.2 First-order methods (SGD / ADMM)**

#### **Penalty methods**

Augmented Lagrangian

#### **Projected Gradient Descent**

### **C.4.3 Zero-order methods (CMA)**

### **C.4.4 Example: Inverse Kinematics**

## **C.5 COMBINATORIAL OPTIMIZATION**

### **C.5.1 Search, SAT, First order logic, SMT solvers, LP interpretation**

### **C.5.2 Mixed-integer convex optimization**

An advanced, but very readable book on MIP [10]. Nice survey paper on MILP [11].

## **C.6 "BLACK-BOX" OPTIMIZATION**

Derivative-free methods. Some allow noisy evaluations.

## **REFERENCES**

1. Jorge Nocedal and Stephen J. Wright, "Numerical optimization", Springer ,

2006.

2. Stephen Boyd and Lieven Vandenberghe, "Convex Optimization", Cambridge University Press , 2004.
3. Pablo Parrilo, "Lecture notes from MIT 6.7230 - Algebraic techniques and semidefinite optimization", , 2023.
4. Jaehyun Park and Stephen Boyd, "General heuristics for nonconvex quadratically constrained quadratic programming", *arXiv preprint arXiv:1703.07870*, 2017.
5. Pablo A. Parrilo, "Structured Semidefinite Programs and Semialgebraic Geometry Methods in Robustness and Optimization", PhD thesis, California Institute of Technology, May 18, 2000.
6. Stephen Prajna and Antonis Papachristodoulou and Peter Seiler and Pablo A. Parrilo, "SOSTOOLS: Sum of Squares Optimization Toolbox for MATLAB User's guide", , June 1, 2004.
7. Frank Noble Permenter, "Reduction methods in semidefinite and conic optimization", PhD thesis, Massachusetts Institute of Technology, 2017. [ [link](#) ]
8. Didier Henrion and Jean-Bernard Lasserre and Johan Lofberg, "GloptiPoly 3: moments, optimization and semidefinite programming", *Optimization Methods & Software*, vol. 24, no. 4-5, pp. 761--779, 2009.
9. Philip E. Gill and Walter Murray and Michael A. Saunders, "User's Guide for SNOPT Version 7: Software for Large -Scale Nonlinear Programming", , February 12, 2006.
10. Michele Conforti and Gérard Cornuéjols and Giacomo Zambelli and others, "Integer programming", Springer , vol. 271, 2014.
11. Juan Pablo Vielma, "Mixed integer linear programming formulation techniques", *Siam Review*, vol. 57, no. 1, pp. 3--57, 2015.

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

[Accessibility](#)

© Russ Tedrake, 2024

# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

## APPENDIX D

# An Optimization Playbook

This chapter is intended to provide a collection of tips and formulations that can make seemingly non-smooth constraints smooth, seemingly non-convex constraints convex, etc. (It's very much a work in progress...)

### D.1 MATRICES

### D.2 ELLIPSOIDS

Imagine an ellipsoid centered at the origin:  $E = \{x | x^T S x \leq 1\}$  with  $S = S^T \succ 0$ . The volume of this ellipse is proportional to  $(\det S^{-1})^{\frac{1}{2}}$ . We know that  $\log \det(S)$  is a concave function in the elements of  $S$  [1].

maximizing volume.

minimizing volume.

volume of a semi-algebraic set. (via a contained ellipsoid)

### **D.3 POLYTOPES**

### **D.4 (MIXED-)INTEGER PROGRAMMING**

### **D.5 BILINEAR MATRIX INEQUALITIES (BMIs)**

### **D.6 GEOMETRY (SE(3), PENETRATION, AND CONTACT)**

### **REFERENCES**

1. Stephen Boyd and Lieven Vandenberghe, "Convex Optimization", Cambridge University Press , 2004.

[Previous Chapter](#)

[Table of contents](#)

[Next Chapter](#)

[Accessibility](#)

© Russ Tedrake, 2024

# UNDERACTUATED ROBOTICS

*Algorithms for Walking, Running, Swimming, Flying, and Manipulation*

Russ Tedrake

© Russ Tedrake, 2024

Last modified 2024-3-24.

[How to cite these notes, use annotations, and give feedback.](#)

**Note:** These are working notes used for [a course being taught at MIT](#). They will be updated throughout the Spring 2024 semester. [Lecture videos are available on YouTube](#).

[Previous Chapter](#)

[Table of contents](#)

## APPENDIX E Miscellaneous

### E.1 HOW TO CITE THESE NOTES

Thank you for citing these notes in your work. Please use the following citation:

Russ Tedrake. *Underactuated Robotics: Algorithms for Walking, Running, Swimming, Flying, and Manipulation (Course Notes for MIT 6.832)*. Downloaded on [date] from <https://underactuated.csail.mit.edu/>

```
@book{underactuated,  
  title      = "Underactuated Robotics",  
  subtitle   = "Algorithms for Walking, Running, Swimming, Flying, and Manipulation",  
  howpublished = "Course Notes for MIT 6.832",  
  author     = "Tedrake, Russ",  
  year       = 2023,  
  url        = "https://underactuated.csail.mit.edu",  
}
```

### E.2 ANNOTATION TOOL ETIQUETTE

My primary goal for the annotation tool is to host a completely open dialogue on the intellectual content of the text. However, it has turned out to serve an additional purpose: it's a convenient way to point out my miscellaneous typos and grammatical blips. The only problem is that if you highlight a typo, and I fix it 10 minutes later, your highlight will persist forevermore. Ultimately this pollutes the annotation content.

There are two possible solutions:

- you can make a public editorial comment, but must promise to delete that comment once it has been addressed.
- You can [join my "editorial" group](#) and post your editorial comments [using this group "scope"](#).

Ideally, once I mark a comment as "done", I would appreciate it if you can delete that

comment.

I highly value both the discussions and the corrections. Please keep them coming, and thank you!

## **E.3 SOME GREAT FINAL PROJECTS**

Each semester students put together a final project using the tools from this class. Many of them are fantastic! Here is a small sample.

### **Spring 2023:**

- [Trajectory optimization for swing-up of multi-link cart-pole](#) by Siro Corsi
- [SalmonBot](#) by Erick Fuentes
- [Cooperative Shared-Load Carrying by Quadrotors](#) by Seiji Shaw and Tommy Cohn
- [Flying bird robot](#) by Quang Kieu
- [QP inverse dynamics for Spot with an arm](#) by Namir Jawdat
- [Staff Spinning Using Trajectory Optimization](#) by Shruti Garg
- [GCS as a policy](#) by Savva Morozov

### **Spring 2022:**

- [Bowling Strikes with Trajectory Optimization](#) by Benjamin Qi
- [Quantum Control via Constrained Trajectory Optimization](#) by Shantanu Jha and Shoumik Chowdhury
- [Trajectory Optimization of Simple Skateboarding Tricks through Contact](#) by Michael Burgess
- [Contact-Aware Controller Design via Alternating Optimization](#) by Richard Li and Timur Garipov
- [Certifying Stability for Magnetically Actuated Satellites](#) by Alex Meredith
- [Quadrotor Trajectory Optimization with Stochastic Wind Disturbance](#) by Mason Darveaux
- [Contraction Metrics-based Stability Analysis via SOS Programming](#) by Sunbochen Tang
- [Brachiation Trajectory Optimization](#) by Kaleb Blake and John Flynn

### **Spring 2021:**

- [Linearized MPC for a kinematic bicycle model](#) by Mike Schoder. [[code](#)]
- [Trajectory Optimization of Interplanetary Travel using Gravity Assist Maneuver](#)
- [Differential Flatness Trajectory Stabilization of Quadrotors](#) by Charles Vorbach
- [Trajectory Planning of a Quadrotor in Random Environments](#) by Susan Ni, Christian Schillinger, and Max Thomsen

### **Spring 2020:**

- [Collision Free Mixed Integer Planning for Quadrotors Using Convex Safe Regions](#) by Bernhard Paus Græsdal
- [Pancake flipping via trajectory optimization](#) by Charles Dawson
- [Dynamic Soaring](#) by Lukas Lao Beyer
- [Acrobatic Humanoid](#) by Matt Chignoli and AJ Miller
- [Trajectory Optimization for the Furuta Pendulum](#) by Samuel Cherna and Philip Murzynowski

Even before we started posting project presentations on YouTube, some great projects from class have turned into full publications. Here are a few examples:

- [Feedback Control of the Pusher-Slider System: A Story of Hybrid and Underactuated Contact Dynamics](#)
- [Harnessing Structures for Value-Based Planning and Reinforcement](#)



#### [Learning](#)

- The famous Mini Cheetah backflip in [Mini Cheetah: A Platform for Pushing the Limits of Dynamic Quadruped Control](#) started as an [underactuated project](#).

## **E.4 PLEASE GIVE ME FEEDBACK!**

I'm very interested in your feedback. The annotation tool is one mechanism, but you can also comment directly on the YouTube lectures, or even add issues to the [github repo](#) that hosts these course notes.

I've also created [this simple survey](#) to collect your general comments/feedback.

[Previous Chapter](#)

[Table of contents](#)

---

[Accessibility](#)

© Russ Tedrake, 2024